



PHIÊN BẢN
TOÀN CẦU

Stuart
Russell
Peter
Norvig

Artificial Intelligence A Modern Approach

Fourth Edition



Trí tuệ nhân tạo

Cách tiếp cận hiện đại

Phiên bản thứ tưPhiên bản toàn cầu



PEARSON SERIES IN TRÍ TUỆ
NHÂN TẠO Stuart Russell và Peter
Norvig, Biên tập viên

FORSYTH & PONCE
GRAHAM
JURAFSKY & MARTIN
NEAPOLITAN
RUSSELL & NORVIG

Thị giác máy tính: Phương pháp tiếp cận hiện đại, ấn bản thứ 2.
ANSI Common Lisp
Xử lý Giọng nói và Ngôn ngữ, ấn bản thứ
Đọc mạng Bayes
Trí tuệ nhân tạo: Một cách tiếp cận hiện đại, ấn bản thứ
4.

Trí tuệ nhân tạo

Cách tiếp cận hiện đại

Phiên bản thứ tưPhiên bản toàn
cầu

Stuart J. Russell và Peter Norvig

Các tác giả đóng góp:

Minh Ngụy Chang

Jacob DevlinAnca

DraganDavid

ForsythIan

GoodfellowJitendr

a M. MalikVikash

Mansinghka

Judea PearlMichael Wooldridge



Tin dụng hình ảnh bìa: Alan Turing: Hình ảnh Lịch sử Khoa học / Ảnh chứng khoán Alamy; Tượng Aristotle: Panos Karas / Shutterstock; Ada Lovelace - Pictorial Press Ltd / Alamy Stock Photo; Xe tự lái: Andrey Suslov / Shutterstock; Robot Atlas: Boston Dynamics, Inc.; Berkeley Campanile và Cầu GoldenGate: Ben Chu / Shutterstock; Các nút bóng ma nền: Eugene Sergeev / Alamy Stock Photo; Bàn cờ với hình cờ vua: Titania / Shutterstock; Mars Rover: Stocktrek Images, Inc./Alamy StockPhoto; Kasparov: Hình ảnh KATHY WILLENS / AP

Pearson Education
Limited KAO Two KAO
Park Hockham
Way Harlow CM17
9SR United Kingdom

và các công ty liên kết trên toàn thế giới

Ghé thăm chúng tôi trên World Wide Web tại: www.pearsonglobaleditions.com

Vui lòng liên hệ với <https://support.pearson.com/getsupport/s/contactsupport> nếu có bất kỳ thắc mắc nào về nội dung này

© Công ty TNHH Giáo dục Pearson 2022

Quyền của Stuart Russell và Peter Norvig được xác định là tác giả của tác phẩm này đã được họ khẳng định phù hợp với Đạo luật Bản quyền, Kiểu dáng và Bảng sáng chế năm 1988.

Chuyển thể được ủy quyền từ ấn bản Hoa Kỳ, có tựa đề Trí tuệ nhân tạo: Phương pháp tiếp cận hiện đại, Ấn bản thứ 4, ISBN 978-0-13-461099-3 của Stuart J. Russell và Peter Norvig, được xuất bản bởi Pearson Education © 2021.

Đã đăng ký Bản quyền. Không một phần nào của ấn phẩm này có thể được sao chép, lưu trữ trong hệ thống truy xuất hoặc truyền tải dưới bất kỳ hình thức nào hoặc bằng bất kỳ phương tiện nào, điện tử, cơ khí, sao chụp, ghi âm hoặc cách khác, mà không có sự cho phép trước bằng văn bản của nhà xuất bản hoặc giấy phép cho phép sao chép hạn chế ở Vương quốc Anh do Cơ quan Cấp phép Bản quyền Công ty TNHH phát hành, Saffron House, 6–10 Kirby Street, Luân Đôn EC1N 8TS. Để biết thông tin về quyền, biểu mẫu yêu cầu và các liên hệ thích hợp trong bộ phận Quyền & Quyền Toàn cầu của Pearson Education, vui lòng truy cập www.pearsoned.com/permissions/.

Tất cả các nhãn hiệu được sử dụng ở đây là tài sản của chủ sở hữu tương ứng. Việc sử dụng bất kỳ nhãn hiệu nào trong văn bản này không trao cho tác giả hoặc nhà xuất bản bất kỳ quyền sở hữu nhãn hiệu nào đối với các nhãn hiệu đó, cũng như việc sử dụng các nhãn hiệu đó không ngụ ý bất kỳ liên kết hoặc xác nhận cuốn sách này bởi các chủ sở hữu đó.

PEARSON, ALWAYS LEARNING và MYLAB là các nhãn hiệu độc quyền tại Hoa Kỳ và / hoặc các quốc gia khác thuộc sở hữu của Pearson Education, Inc. hoặc các chi nhánh của nó.

Trừ khi có quy định khác ở đây, bất kỳ nhãn hiệu nào của bên thứ ba có thể xuất hiện trong tác phẩm này là tài sản của chủ sở hữu tương ứng và bất kỳ tham chiếu nào đến nhãn hiệu, logo hoặc trang phục thương mại khác của bên thứ ba chỉ dành cho mục đích minh họa hoặc mô tả. Các tài liệu tham khảo như vậy không nhằm ngụ ý bất kỳ tài trợ, xác nhận, ủy quyền hoặc quảng bá sản phẩm của Pearson bởi chủ sở hữu của các nhãn hiệu đó hoặc bất kỳ mối quan hệ nào giữa chủ sở hữu và Pearson Education, Inc. hoặc các chi nhánh, tác giả, người được cấp phép hoặc nhà phân phối của nó.

ISBN 10: 1-292-40113-3 ISBN 13:
978-1-292-40113-3 Sách điện tử
ISBN 13: 978-1-292-40117-1

Dữ liệu lập danh mục trong xuất bản của Thư viện Anh Hò sơ danh mục cho cuốn sách này có sẵn từ Thư viện Anh

Sắp chữ bởi SPi GlobaleBook được định dạng bởi B2R Technologies Pvt. Ltd.

Đối với Loy, Gordon, Lucy, George và Isaac - SJR

Đối với Kris, Isabella và Juliet - PN

Trang này cố tình để trống

Lời tựa

Trí tuệ nhân tạo (AI) là một lĩnh vực lớn, và đây là một cuốn sách lớn. Chúng tôi đã cố gắng khám phá toàn bộ phạm vi của lĩnh vực này, bao gồm logic, xác suất và toán học liên tục; nhận thức, lý luận, học hỏi và hành động; công bằng, tin cậy, lợi ích xã hội và an toàn; và các ứng dụng bao gồm từ các thiết bị vi điện tử đến robot thám hiểm hành tinh đến các dịch vụ trực tuyến với hàng tỷ người dùng.

Phụ đề của cuốn sách này là "Một cách tiếp cận hiện đại". Điều đó có nghĩa là chúng tôi đã chọn kể câu chuyện từ góc độ hiện tại. Chúng tôi tổng hợp những gì hiện nay đã biết thành một khuôn khổ chung, đúc lại các tác phẩm ban đầu bằng cách sử dụng các ý tưởng và thuật ngữ phổ biến ngày nay. Chúng tôi xin lỗi những người có các trường con ít được nhận ra hơn.

Mới cho phiên bản này

Ấn bản này phản ánh những thay đổi trong AI kể từ phiên bản cuối cùng vào năm 2010:

- Chúng tôi tập trung nhiều hơn vào học máy hơn là kỹ thuật tri thức thủ công, do sự sẵn có của dữ liệu, tài nguyên máy tính và các thuật toán mới ngày càng tăng.
- Các hệ thống học sâu, lập trình xác suất và đa tác nhân nhận được phạm vi mở rộng, mỗi hệ thống có chương riêng.
- Phạm vi hiểu ngôn ngữ tự nhiên, robot và thị giác máy tính đã được sửa đổi để phản ánh tác động của học sâu.
- Chương robot hiện bao gồm robot tương tác với con người và ứng dụng học tăng cường cho robot.

Trước đây, chúng tôi đã định nghĩa mục tiêu của AI là tạo ra các hệ thống cố gắng tối đa hóa tiện ích mong đợi, trong đó thông tin tiện ích cụ thể - mục tiêu - được cung cấp bởi con người thiết kế hệ thống. Bây giờ chúng ta không còn cho rằng mục tiêu đã được cố định và được hệ thống AI biết đến; Thay vào đó, hệ thống có thể không chắc chắn về mục tiêu thực sự của con người mà nó hoạt động thay mặt. Nó phải học những gì cần tối đa hóa và phải hoạt động phù hợp ngay cả khi không chắc chắn về mục tiêu.

- Chúng tôi tăng cường phạm vi về tác động của AI đối với xã hội, bao gồm các vấn đề quan trọng về đạo đức, công bằng, tin cậy và an toàn.
- Chúng tôi đã chuyển các bài tập từ cuối mỗi chương sang một trang web trực tuyến. Điều này cho phép chúng tôi liên tục bổ sung, cập nhật và cải thiện các bài tập, để đáp ứng nhu cầu của người hướng dẫn và phản ánh những tiến bộ trong lĩnh vực này và trong các công cụ phần mềm liên quan đến AI.
- Nhìn chung, khoảng 25% tài liệu trong cuốn sách là hoàn toàn mới. 75% còn lại phần lớn đã được viết lại để trình bày một bức tranh thống nhất hơn về lĩnh vực này. 22% trích dẫn trong ấn bản này là các tác phẩm được xuất bản sau năm 2010.

Tổng quan về cuốn sách

Chủ đề thống nhất chính là ý tưởng về một tác nhân thông minh. Chúng tôi định nghĩa AI là nghiên cứu của các tác nhân nhận được nhận thức từ môi trường và thực hiện các hành động. Mỗi tác nhân như vậy thực hiện một chức năng ánh xạ các chuỗi nhận thức với các hành động và chúng tôi đề cập đến các cách khác nhau để biểu diễn các chức năng này, chẳng hạn như tác nhân phản ứng, lập kế hoạch thời gian thực, lý thuyết quyết định

hệ thống và hệ thống học sâu. Chúng tôi nhấn mạnh việc học vừa như một phương pháp xây dựng cho các hệ thống có thẩm quyền vừa là một cách để mở rộng phạm vi tiếp cận của nhà thiết kế vào các môi trường chưa biết. Chúng tôi coi robot và tầm nhìn không phải là những vấn đề được xác định độc lập, mà xảy ra để phục vụ cho việc đạt được mục tiêu. Chúng tôi nhấn mạnh tầm quan trọng của môi trường nhiệm vụ trong việc xác định thiết kế tác nhân phù hợp.

Mục đích chính của chúng tôi là truyền đạt những ý tưởng đã xuất hiện trong bảy mươi năm qua của nghiên cứu AI và hai thiên niên kỷ qua của công việc liên quan. Chúng tôi đã cố gắng tránh quá hình thức trong việc trình bày những ý tưởng này, trong khi vẫn giữ được độ chính xác. Chúng tôi đã bao gồm các công thức toán học và thuật toán mà giả để làm cho các ý tưởng chính trở nên cụ thể; Các khái niệm và ký hiệu toán học được mô tả trong Phụ lục A và mã giả của chúng tôi được mô tả trong Phụ lục B.

Cuốn sách này chủ yếu nhằm mục đích sử dụng trong một khóa học đại học hoặc chuỗi khóa học. Cuốn sách có 29 chương, mỗi chương yêu cầu các bài giảng khoảng một tuần, vì vậy việc đọc toàn bộ cuốn sách đòi hỏi một chuỗi hai học kỳ. Một khóa học một học kỳ có thể sử dụng các chương được chọn để phù hợp với sở thích của người hướng dẫn và sinh viên. Cuốn sách cũng có thể được sử dụng trong một khóa học cấp độ sau đại học (có thể với việc bổ sung một số nguồn chính được đề xuất trong ghi chú thư mục), hoặc để tự học hoặc làm tài liệu tham khảo.

Thời hạn

Xuyên suốt cuốn sách, những điểm quan trọng được đánh dấu bằng biểu tượng hình tam giác ở lề. I Bất cứ nơi nào một thuật ngữ mới được định nghĩa, nó cũng được ghi chú ở lề. Các ứng dụng quan trọng tiếp theo của thuật ngữ được in đậm, nhưng không ở lề. Chúng tôi đã bao gồm một chỉ mục toàn diện và thư mục mở rộng.

Điều kiện tiên quyết duy nhất là làm quen với các khái niệm cơ bản của khoa học máy tính (thuật toán, cấu trúc dữ liệu, độ phức tạp) ở cấp độ năm hai. Giải tích sinh viên năm nhất và đại số tuyến tính rất hữu ích cho một số chủ đề.

Tài nguyên trực tuyến

Các tài nguyên trực tuyến có sẵn thông qua pearsonglobaleditions.com. Ở đó bạn sẽ tìm thấy:

- Bài tập, dự án lập trình và dự án nghiên cứu. Những điều này không còn ở cuối mỗi chương; họ chỉ trực tuyến. Trong cuốn sách, chúng tôi đề cập đến một bài tập trực tuyến với tên như "Bài tập 6.NARY". Hướng dẫn trên trang web cho phép bạn tìm các bài tập theo tên hoặc theo chủ đề.
- Triển khai các thuật toán trong cuốn sách bằng Python, Java và các ngôn ngữ lập trình khác.
- Tài liệu bổ sung và liên kết cho sinh viên và người hướng dẫn.
- Hướng dẫn về cách báo cáo lỗi trong sách trong trường hợp có thể có một số lỗi tồn tại.

Bìa sách

Bìa mô tả vị trí cuối cùng từ ván quyết định 6 của trận cờ vua năm 1997, trong đó chương trình Deep Blue đánh bại Garry Kasparov (chơi Đen), đây là lần đầu tiên một máy tính đánh bại một nhà vô địch thế giới trong một trận đấu cờ vua. Kasparov được hiển thị ở trên cùng. Bên phải của anh ấy là một vị trí quan trọng từ ván thứ hai của trận đấu cờ vây lịch sử giữa cựu vô địch thế giới Lee Sedol và chương trình ALPHAGO của DeepMind. Nước đi 37 bởi ALPHAGO đã vi phạm chính thống cờ vây hàng thế kỷ và ngay lập tức được các chuyên gia con người nhìn thấy

như một sai lầm đáng xấu hổ, nhưng hóa ra nó là một nước đi chiến thắng. Trên cùng bên trái là một robot Atlashumanoid do Boston Dynamics chế tạo. Một mô tả về một chiếc xe tự lái cảm nhận được môi trường của nó xuất hiện giữa Ada Lovelace, lập trình viên máy tính đầu tiên trên thế giới, và Alan Turing, người có công trình cơ bản xác định trí tuệ nhân tạo. Ở dưới cùng của bản cover là một robot thám hiểm Sao Hỏa và một bức tượng của Aristotle, người đi tiên phong trong việc nghiên cứu logic; thuật toán lập kế hoạch của anh ấy từ De Motu Animalium xuất hiện phía sau tên của các tác giả. Đằng sau bản cover là một mô hình lập trình xác suất được Tổ chức Hiệp ước Cẩm Thử nghiệm Hạt nhân Toàn diện của Liên Hợp Quốc sử dụng để phát hiện các vụ nổ hạt nhân từ các tín hiệu địa chấn.

Lời cảm ơn

Cần có một ngôi làng toàn cầu để tạo ra một cuốn sách. Hơn 600 người đã đọc các phần của cuốn sách và đưa ra các đề xuất để cải thiện. Danh sách đầy đủ là ở pearsonglobaleditions.com; Chúng tôi biết ơn tất cả họ. Chúng tôi có không gian ở đây để đề cập đến một vài người đóng góp đặc biệt quan trọng. Đầu tiên các nhà văn đóng góp:

- Judea Pearl (Mục 13.5, Mạng lưới nhân quả);
- Michael Wooldridge (Chương 17, Ra quyết định đa tác nhân);
- Vikash Mansinghka (Phần 18.4, Chương trình như mô hình xác suất);
- Ian Goodfellow (Chương 22, Học sâu);
- Jacob Devlin và Mei-Wing Chang (Chương 25, Học sâu cho Xử lý ngôn ngữ tự nhiên);
- Anca Dragan (Chương 26, Người máy);
- Jitendra Malik và David Forsyth (Chương 27, Thị giác Máy tính). Sau đó, một số vai trò chính:
 - Cynthia Yeung và Malika Cantor (quản lý dự án);
 - Julie Sussman và Tom Galloway (biên tập và viết đề xuất);
 - Omari Stephens (hình minh họa);
 - Tracy Johnson (biên tập viên);
 - Erin Ault và Rose Kernan (chuyển đổi bìa và màu);
 - Nalin Chhibber, Sam Goto, Raymond de Lacaze, Ravi Mohan, Ciaran O'Reilly, Amit Patel, Dragomir Radiv và Samagra Sharma (cố vấn và phát triển mã trực tuyến);
- Sinh viên Google Summer of Code (phát triển mã trực tuyến). Stuart muốn cảm ơn vợ mình, Loy Sheflott, vì sự kiên nhẫn vô tận và sự khôn ngoan vô hạn của cô. Anh hy vọng rằng Gordon, Lucy, George và Isaac sẽ sớm đọc cuốn sách này sau khi họ đã tha thứ cho anh vì đã làm việc quá lâu với nó. THẨM (Nhóm Sinh viên Bất thường của Russell) đã hữu ích một cách bất thường, như mọi khi. Peter muốn cảm ơn cha mẹ anh ấy (Torsten và Gerda) vì đã giúp anh ấy bắt đầu, và vợ anh ấy (Kris), các con (Bella và Juliet), đồng nghiệp, sếp và bạn bè đã khuyến khích và khoan dung anh ấy trong suốt thời gian dài viết và viết lại.

Giới thiệu về các tác giả

Stuart Russell sinh năm 1962 tại Portsmouth, Anh. Ông nhận bằng cử nhân hạng nhất về vật lý tại Đại học Oxford năm 1982 và bằng tiến sĩ khoa học máy tính tại Stanford năm 1986. Sau đó, ông gia nhập giảng viên của Đại học California tại Berkeley, nơi ông là giáo sư và cựu chủ tịch khoa học máy tính, giám đốc Trung tâm AI tương thích với con người và giữ Chủ tịch Smith-Zadeh về Kỹ thuật. Năm 1990, ông nhận được Giải thưởng Điều tra viên Trẻ của Tổng thống của Quỹ Khoa học Quốc gia, và năm 1995, ông đồng đoạt Giải thưởng Máy tính và Tư tưởng. Ông là thành viên của Hiệp hội Trí tuệ nhân tạo American, Hiệp hội Máy tính và Hiệp hội Tiến bộ Khoa học Hoa Kỳ, Thành viên Danh dự của Wadham College, Oxford và Thành viên Andrew Carnegie. Ông đã giữ Chaire Blaise Pascal ở Paris từ năm 2012 đến năm 2014. Ông đã xuất bản hơn 300 bài báo về nhiều chủ đề trong trí tuệ nhân tạo. Các cuốn sách khác của ông bao gồm *The Use of Knowledge in Analogy and Induction*, *Do the Right Thing: Studies in Limited Rationality* (với Eric Wefald), và *Human Compatible: Artificial Intelligence and the Problem of Control*.

Peter Norvig hiện là Giám đốc Nghiên cứu tại Google, Inc., và trước đây là giám đốc chịu trách nhiệm về các thuật toán tìm kiếm Web cốt lõi. Anh ấy đã đồng giảng dạy một lớp học AI trực tuyến đã đăng ký 160.000 sinh viên, giúp khởi động vòng học trực tuyến mở lớn hiện tại. Ông là người đứng đầu Bộ phận Khoa học Tính toán tại Trung tâm Nghiên cứu Ames của NASA, giám sát nghiên cứu và phát triển trí tuệ nhân tạo và robot. Ông nhận bằng Cử nhân toán ứng dụng tại Đại học Brown và bằng Tiến sĩ về khoa học máy tính từ Berkeley. Ông đã từng là giáo sư tại Đại học Nam California và là giảng viên tại Berkeley và Stanford. Ông là thành viên của Hiệp hội Trí tuệ nhân tạo Hoa Kỳ, Hiệp hội Máy tính Máy tính, Viện Hàn lâm Khoa học và Nghệ thuật Hoa Kỳ và Viện Hàn lâm Khoa học California. Các cuốn sách khác của ông là *Paradigms of AI Programming: Case Studies in Common Lisp*, *Verbmobil: A Translation System for Face-to-Face Dialog*, và *Intelligent Help Systems for UNIX*.

Hai tác giả đã chia sẻ giải thưởng Nhà giáo dục Xuất sắc AAAI / EAAI khai mạc vào năm 2016.

Nội dung

I Trí tuệ nhân tạo

1 Giới thiệu	19
1.1 AI là gì?	19
1.2 Nền tảng của trí tuệ nhân tạo	23
1.3 Lịch sử của trí tuệ nhân tạo	35
1.4 Tình trạng của nghệ thuật	45
1.5 Rủi ro và lợi ích của AI	49
Tóm tắt.....	52
Ghi chú thư mục và lịch sử	53
2 Đại lý thông minh	54
2.1 Tác nhân và môi trường	54
2.2 Hành vi tốt: Khái niệm về tính hợp lý	57
2.3 Bản chất của môi trường	60
2.4 Cơ cấu của các đại lý	65
Tóm tắt.....	78
Ghi chú thư mục và lịch sử	78

II Giải quyết vấn đề

3 Giải quyết vấn đề bằng cách tìm kiếm	81
3.1 Tác nhân giải quyết vấn đề	81
3.2 Các vấn đề ví dụ	84
3.3 Thuật toán tìm kiếm	89
3.4 Chiến lược tìm kiếm thiếu thông tin	94
3.5 Chiến lược tìm kiếm được thông tin (phỏng đoán)	102
3.6 Chức năng phỏng đoán	115
Tóm tắt.....	122
Ghi chú thư mục và lịch sử	124
4 Tìm kiếm trong môi trường phức tạp	128
4.1 Các vấn đề tìm kiếm và tối ưu hóa cục bộ	128
4.2 Tìm kiếm địa phương trong không gian liên tục	137
4.3 Tìm kiếm bằng các hành động không xác định	140
4.4 Tìm kiếm trong môi trường có thể quan sát được một phần	144
4.5 Các đại lý tìm kiếm trực tuyến và môi trường không xác định	152
Tóm tắt.....	159
Ghi chú thư mục và lịch sử	160
5 Vấn đề thỏa mãn ràng buộc	164
5.1 Xác định các vấn đề thỏa mãn ràng buộc	164
5.2 Truyền bá ràng buộc: Suy luận trong CSP	169

5.3	Tìm kiếm lùi cho CSP	175
5.4	Tìm kiếm CSP tại địa phương	181
5.5	Cấu trúc của các vấn đề.	183
	Tóm tắt.....	187
	Ghi chú thư mục và lịch sử	188
6	Trò chơi và tìm kiếm đối thủ	192
6.1	Lý thuyết trò chơi	192
6.2	Quyết định tối ưu trong trò chơi	194
6.3	Tìm kiếm cây Alpha – Beta heuristic	202
6.4	Tìm kiếm cây Monte Carlo	207
6.5	Trò chơi ngẫu nhiên	210
6.6	Trò chơi có thể quan sát được một phần	214
6.7	Hạn chế của thuật toán tìm kiếm trò chơi	219
	Tóm tắt.....	220
	Ghi chú thư mục và lịch sử	221
III Kiến thức, lý luận và lập kế hoạch		
7	Tác nhân logic	226
7.1	Tác nhân dựa trên tri thức	227
7.2	Thế giới Wumpus	228
7.3	Lôgic.....	232
7.4	Logic mệnh đề: Một logic rất đơn giản	235
7.5	Định lý mệnh đề Chứng minh	240
7.6	Kiểm tra mô hình mệnh đề hiệu quả	250
7.7	Các tác nhân dựa trên logic mệnh đề	255
	Tóm tắt.....	264
	Ghi chú thư mục và lịch sử	265
8	Logic bậc nhất	269
8.1	Đại diện được xem xét lại.	269
8.2	Cú pháp và ngữ nghĩa của logic bậc nhất	274
8.3	Sử dụng logic bậc nhất	283
8.4	Kỹ thuật tri thức trong logic bậc nhất	289
	Tóm tắt.....	295
	Ghi chú thư mục và lịch sử	296
9	Suy luận trong logic bậc nhất	298
9.1	Suy luận mệnh đề so với bậc nhất	298
9.2	Thống nhất và suy luận bậc nhất	300
9.3	Chuỗi chuyển tiếp	304
9.4	Xích lùi	311
9.5	Nghị quyết.....	316
	Tóm tắt.....	327
	Ghi chú thư mục và lịch sử	328

10 Đại diện kiến thức	332
10.1 Kỹ thuật bản thể	332
10.2 Danh mục và đối tượng. .	335
10.3 Sự kiện.....	340
10.4 Các đối tượng tinh thần và logic phương thức . . .	344
10.5 Hệ thống lý luận cho các loại	347
10.6 Lý luận với thông tin mặc định	351
Tóm tắt.....	355
Ghi chú thư mục và lịch sử	356
11 Lập kế hoạch tự động	362
11.1 Định nghĩa về quy hoạch cổ điển	362
11.2 Các thuật toán cho lập kế hoạch cổ điển	366
11.3 Phỏng đoán lập kế hoạch	371
11.4 Lập kế hoạch phân cấp	374
11.5 Lập kế hoạch và hành động trong các lĩnh vực không xác định	383
11.6 Thời gian, lịch trình và nguồn lực	392
11.7 Phân tích các phương pháp tiếp cận lập kế hoạch.	396
Tóm tắt.....	397
Ghi chú thư mục và lịch sử	398
IV Kiến thức và lý luận không chắc chắn	
12 Định lượng sự không chắc chắn	403
12.1 Hành động trong điều kiện không chắc chắn	403
12.2 Ký hiệu xác suất cơ bản	406
12.3 Suy luận sử dụng phân phối chung đầy đủ	413
12.4 Độc lập.....	415
12.5 Quy tắc Bayes và cách sử dụng nó	417
12.6 Mô hình Bayes ngây thơ	420
12.7 Thế giới Wumpus được xem xét lại	422
Tóm tắt.....	425
Ghi chú thư mục và lịch sử	426
13 Lý luận xác suất	430
13.1 Đại diện cho kiến thức trong một lĩnh vực không chắc chắn.	430
13.2 Ngữ nghĩa của mạng Bayes	432
13.3 Suy luận chính xác trong mạng Bayes	445
13.4 Suy luận gần đúng cho mạng Bayes	453
13.5 Mạng lưới nhân quả	467
Tóm tắt.....	471
Ghi chú thư mục và lịch sử	472
14 Lý luận xác suất theo thời gian	479
14.1 Thời gian và sự không chắc chắn	479
14.2 Suy luận trong các mô hình thời gian	483

14.3	Mô hình Markov ẩn	491
14.4	Bộ lọc Kalman	497
14.5	Mạng Bayes động	503
	Tóm tắt.....	514
	Ghi chú thư mục và lịch sử	515
15	Đưa ra quyết định đơn giản	518
15.1	Kết hợp niềm tin và mong muốn trong sự không chắc chắn	518
15.2	Cơ sở của lý thuyết tiện ích	519
15.3	Chức năng tiện ích	522
15.4	Chức năng tiện ích đa thuộc tính	530
15.5	Mạng lưới quyết định	534
15.6	Giá trị của thông tin	537
15.7	Sở thích không xác định	543
	Tóm tắt.....	547
	Ghi chú thư mục và lịch sử	547
16	Đưa ra quyết định phức tạp	552
16.1	Các vấn đề quyết định tuần tự	552
16.2	Thuật toán cho MDP	562
16.3	Vấn đề cướp	571
16.4	Các MDP có thể quan sát được một phần	578
16.5	Thuật toán để giải quyết POMDP	580
	Tóm tắt.....	585
	Ghi chú thư mục và lịch sử	586
17	Ra quyết định đa tác nhân	589
17.1	Thuộc tính của môi trường đa tác nhân	589
17.2	Lý thuyết trò chơi không hợp tác	595
17.3	Lý thuyết trò chơi hợp tác	616
17.4	Đưa ra quyết định tập thể.	622
	Tóm tắt.....	635
	Ghi chú thư mục và lịch sử	636
18	Lập trình xác suất	641
18.1	Mô hình xác suất quan hệ	642
18.2	Mô hình xác suất vũ trụ mở.	648
18.3	Theo dõi một thế giới phức tạp	655
18.4	Các chương trình như mô hình xác suất	660
	Tóm tắt.....	664
	Ghi chú thư mục và lịch sử	665
V Máy học		
19	Học hỏi từ các ví dụ	669
19.1	Các hình thức học tập.	669

19.2	Học tập có giám sát	671
19.3	Cây quyết định học tập	675
19.4	Lựa chọn và tối ưu hóa mô hình	683
19.5	Lý thuyết học tập	690
19.6	Hồi quy tuyến tính và phân loại	694
19.7	Mô hình phi tham số	704
19.8	Học tập tổng hợp	714
19.9	Phát triển hệ thống máy học	722
	Tóm tắt.....	732
	Ghi chú thư mục và lịch sử	733
20	Kiến thức trong học tập	739
20.1	Một công thức hợp lý của học tập.	739
20.2	Kiến thức trong học tập.	747
20.3	Học tập dựa trên giải thích	750
20.4	Học tập sử dụng thông tin liên quan	754
20.5	Lập trình logic quy nạp	758
	Tóm tắt.....	767
	Ghi chú thư mục và lịch sử	768
21	Học các mô hình xác suất	772
21.1	Học thống kê.	772
21.2	Học tập với dữ liệu đầy đủ	775
21.3	Học với các biến ẩn: Thuật toán EM	788
	Tóm tắt.....	797
	Ghi chú thư mục và lịch sử	798
22	Học sâu	801
22.1	Mạng chuyển tiếp đơn giản	802
22.2	Đồ thị tính toán cho học sâu	807
22.3	Mạng tích chập	811
22.4	Thuật toán học tập	816
22.5	Tổng quát.....	819
22.6	Mạng nơ-ron tuần hoàn	823
22.7	Học tập không giám sát và học chuyển giao	826
22.8	Ứng dụng.....	833
	Tóm tắt.....	835
	Ghi chú thư mục và lịch sử	836
23	Học tăng cường	840
23.1	Học hỏi từ Phần thưởng	840
23.2	Học tăng cường thụ động	842
23.3	Học tập tăng cường tích cực	848
23.4	Tổng quát hóa trong Học tăng cường	854
23.5	Tìm kiếm chính sách.	861
23.6	Học nghề và Học tăng cường nghịch đảo	863

23.7 Ứng dụng của Học tăng cường	866
Tóm tắt.....	869
Ghi chú thư mục và lịch sử	870

VI Giao tiếp, nhận thức và hành động

24 Xử lý ngôn ngữ tự nhiên	874
24.1 Mô hình ngôn ngữ	874
24.2 Ngữ pháp.....	884
24.3 Phân tích.....	886
24.4 Ngữ pháp tăng cường	892
24.5 Sự phức tạp của ngôn ngữ tự nhiên thực sự	896
24.6 Nhiệm vụ ngôn ngữ tự nhiên	900
Tóm tắt.....	901
Ghi chú thư mục và lịch sử	902
25 Học sâu để xử lý ngôn ngữ tự nhiên	907
25.1 Nhúng từ	907
25.2 Mạng nơ-ron tuần hoàn cho NLP	911
25.3 Mô hình trình tự	915
25.4 Kiến trúc máy biến áp	919
25.5 Đào tạo trước và học chuyển tiếp	922
25.6 Hiện đại của nghệ thuật	926
Tóm tắt.....	929
Ghi chú thư mục và lịch sử	929
26 Người máy	932
26.1 Robot.....	932
26.2 Phần cứng robot	933
26.3 Robot đang giải quyết loại vấn đề gì?	937
26.4 Nhận thức robot	938
26.5 Lập kế hoạch và kiểm soát.	945
26.6 Lập kế hoạch cho các phong trào không chắc chắn	963
26.7 Học tăng cường trong robot	965
26.8 Con người và Robot	968
26.9 Khung robot thay thế	975
26.10 Lĩnh vực ứng dụng	978
Tóm tắt.....	981
Ghi chú thư mục và lịch sử	982
27 Thị giác máy tính	988
27.1 Giới thiệu.....	988
27.2 Hình ảnh hình ảnh	989
27.3 Các tính năng hình ảnh đơn giản	995
27.4 Phân loại hình ảnh.	1002
27.5 Phát hiện đối tượng	1006

27.6	Thế giới 3D. . .	1008
27.7	Sử dụng thị giác máy tính	1013
	Tóm tắt . . .	1026
	Ghi chú thư mục và lịch sử	1027

VII Kết luận

28	Triết lý, Đạo đức và An toàn của AI	1032
28.1	Giới hạn của AI. .	1032
28.2	Máy móc có thể thực sự suy nghĩ?	1035
28.3	Đạo đức của AI . .	1037
	Tóm tắt. . .	1056
	Ghi chú thư mục và lịch sử	1057
29	Tương lai của AI	1063
29.1	Các thành phần AI. .	1063
29.2	Kiến trúc AI. . .	1069
	Nền tảng toán học	1074
A.1	Phân tích độ phức tạp và ký hiệu $O()$	1074
A.2	Vector, Ma trận và Đại số tuyến tính	1076
A.3	Phân phối xác suất . . .	1078
	Ghi chú thư mục và lịch sử	1080
B	Lưu ý về ngôn ngữ và thuật toán	1081
B.1	Định nghĩa ngôn ngữ với dạng Backus–Naur (BNF)	1081
B.2	Mô tả các thuật toán bằng mã giả	1082
B.3	Tài liệu bổ sung trực tuyến	1083
	Thư mục	1084
	Chỉ số	1119

Trang này cố tình để trống

CHƯƠNG 1

GIỚI THIỆU

Trong đó chúng tôi cố gắng giải thích lý do tại sao chúng tôi coi trí tuệ nhân tạo là một chủ đề đáng nghiên cứu nhất và trong đó chúng tôi cố gắng quyết định chính xác nó là gì, đây là một điều tốt để quyết định trước khi bắt đầu.

Chúng ta tự gọi mình là Homo sapiens - người khôn ngoan - bởi vì trí thông minh của chúng ta rất quan trọng cho chúng tôi. Trí thông minh
Trong hàng ngàn năm, chúng ta đã cố gắng hiểu cách chúng ta suy nghĩ và hành động – nghĩa là, làm thế nào bộ não của chúng ta, chỉ một số ít vật chất, có thể nhận thức, hiểu, dự đoán và thao túng một thế giới lớn hơn và phức tạp hơn nhiều so với chính nó. Lĩnh vực trí tuệ nhân tạo, hay AI, Trí tuệ nhân tạo
không chỉ quan tâm đến sự hiểu biết mà còn xây dựng các thực thể thông minh - những cỗ máy có thể tính toán cách hành động hiệu quả và an toàn trong nhiều tình huống mới mẻ.

Các cuộc khảo sát thường xuyên xếp hạng AI là một trong những lĩnh vực thú vị nhất và phát triển nhanh nhất, và nó đã tạo ra doanh thu hơn một nghìn tỷ đô la mỗi năm. Chuyên gia AI Kai-Fu Lee dự đoán rằng tác động của nó sẽ "hơn bất cứ điều gì trong lịch sử nhân loại". Hơn nữa, biên giới trí tuệ của AI rất rộng mở. Trong khi một sinh viên của một ngành khoa học cũ hơn như vật lý có thể cảm thấy rằng những ý tưởng tốt nhất đã được phát hiện bởi Galileo, Newton, Curie, Ein-stein và những người còn lại, AI vẫn có nhiều cơ hội cho những người chủ mưu toàn thời gian.

AI hiện bao gồm rất nhiều lĩnh vực phụ, từ tổng quát (học tập, suy luận, nhận thức, v.v.) đến cụ thể, chẳng hạn như chơi cờ, chứng minh định lý toán học, làm thơ, lái xe ô tô hoặc chẩn đoán bệnh. AI có liên quan đến bất kỳ nhiệm vụ trí tuệ nào; nó thực sự là một lĩnh vực phổ quát.

1.1 AI là gì?

Chúng tôi đã tuyên bố rằng AI rất thú vị, nhưng chúng tôi chưa nói nó là gì. Trong lịch sử, những người tìm kiếm lại đã theo đuổi một số phiên bản khác nhau của AI. Một số người đã định nghĩa trí thông minh về sự trung thành với hiệu suất của con người, trong khi những người khác thích một định nghĩa trừu tượng, chính thức về trí thông minh được gọi là hợp lý - nói một cách lỏng lẻo, làm "điều đúng đắn". Chủ đề

Tính hợp lý

Bản thân nó cũng khác nhau: một số coi trí thông minh là một thuộc tính của quá trình suy nghĩ và lý luận bên trong, trong khi những người khác tập trung vào hành vi thông minh, một đặc điểm bên ngoài.

Từ hai khía cạnh này - con người so với lý trí và suy nghĩ so với hành vi - có bốn sự kết hợp có thể xảy ra, và đã có những người ủng hộ và chương trình nghiên cứu cho tất cả mọi người

1 Trong mắt công chúng, đôi khi có sự nhầm lẫn giữa các thuật ngữ "trí tuệ nhân tạo" và "máy học". Học máy là một lĩnh vực phụ của AI nghiên cứu khả năng cải thiện hiệu suất dựa trên kinh nghiệm. Một số hệ thống AI sử dụng các phương pháp học máy để đạt được năng lực, nhưng một số thì không. 2 Chúng tôi không cho rằng con người là "phi lý" theo nghĩa từ điển là "bị tước đoạt sự minh mẫn tinh thần bình thường". Chúng ta chỉ thừa nhận rằng các quyết định của con người không phải lúc nào cũng hoàn hảo về mặt toán học.

Bốn. Các phương pháp được sử dụng nhất thiết phải khác nhau: việc theo đuổi trí thông minh giống con người phải là một phần khoa học thực nghiệm liên quan đến tâm lý học, liên quan đến các quan sát và giả thuyết về hành vi và quá trình suy nghĩ thực tế của con người; Mặt khác, một cách tiếp cận duy lý liên quan đến sự kết hợp giữa toán học và kỹ thuật, và kết nối với thống kê, lý thuyết kiểm soát và kinh tế. Các nhóm khác nhau đã vừa chệch bại vừa giúp đỡ lẫn nhau. Chúng ta hãy xem xét bốn cách tiếp cận chi tiết hơn.

1.1.1 Hành động con người: Phương pháp thử nghiệm Turing

Kiểm tra Turing

Thử nghiệm Turing, được đề xuất bởi Alan Turing (1950), được thiết kế như một thí nghiệm tư duy sẽ né tránh sự mơ hồ triết học của câu hỏi "Một cỗ máy có thể suy nghĩ không?" Một người tính toán vượt qua bài kiểm tra nếu một người thẩm vấn con người, sau khi đặt ra một số câu hỏi bằng văn bản, không thể biết liệu câu trả lời bằng văn bản đến từ một người hay từ máy tính. Chương 28 thảo luận chi tiết về bài kiểm tra và liệu một máy tính có thực sự thông minh nếu nó vượt qua hay không. Bây giờ, chúng tôi lưu ý rằng lập trình máy tính để vượt qua một bài kiểm tra được áp dụng nghiêm ngặt cung cấp rất nhiều công việc. Máy tính sẽ cần các khả năng sau:

Xử lý ngôn ngữ tự nhiên
Kiến thức đại diện

- xử lý ngôn ngữ tự nhiên để giao tiếp thành công bằng ngôn ngữ con người
- kiến thức đại diện để lưu trữ những gì nó biết hoặc nghe;

Lý luận tự động

- lý luận tự động để trả lời các câu hỏi và rút ra kết luận mới;

Máy học

- Học máy để thích ứng với hoàn cảnh mới và phát hiện và ngoại suy các mẫu.

Kiểm tra Turing tổng số

Turing coi mô phỏng vật lý của một người là không cần thiết để thể hiện trí thông minh. Tuy nhiên, các nhà nghiên cứu khác đã đề xuất một thí nghiệm Turing tổng thể, đòi hỏi sự tương tác với đồ vật và con người trong thế giới thực. Để vượt qua bài kiểm tra Turing tổng thể, một robot sẽ cần

Thị giác máy tính

- thị giác máy tính và nhận dạng giọng nói để nhận thức thế giới;

Robotics

- Robot để điều khiển các vật thể và di chuyển xung quanh

Sáu lĩnh vực này tạo nên hầu hết AI. Tuy nhiên, các nhà nghiên cứu AI đã dành rất ít nỗ lực để vượt qua bài kiểm tra Turing, tin rằng điều quan trọng hơn là nghiên cứu các nguyên tắc cơ bản của trí thông minh. Cuộc tìm kiếm "chuyến bay nhân tạo" đã thành công khi các kỹ sư và nhà phát minh ngừng bắt chước chim và bắt đầu sử dụng đường hầm gió và tìm hiểu về khí động học. Các văn bản kỹ thuật hàng không không định nghĩa mục tiêu của lĩnh vực của họ là tạo ra "những cỗ máy bay giống hệt như chim bồ câu đến mức chúng có thể đánh lừa ngay cả những con chim bồ câu khác".

1.1.2 Tư duy con người: Phương pháp tiếp cận mô hình nhận thức

Để nói rằng một chương trình suy nghĩ như một con người, chúng ta phải biết con người suy nghĩ như thế nào. Chúng ta có thể tìm hiểu về suy nghĩ của con người theo ba cách:

Mẫu
Thí nghiệm tâm lý

- nội tâm - cố gắng nắm bắt những suy nghĩ của chính chúng ta khi chúng trôi qua;
- thí nghiệm tâm lý - quan sát một người đang hoạt động;

Hình ảnh não

- Hình ảnh não — quan sát não hoạt động.

Một khi chúng ta có một lý thuyết đủ chính xác về tâm trí, nó trở nên có thể diễn tả lý thuyết như một chương trình máy tính. Nếu hành vi đầu vào-đầu ra của chương trình phù hợp với hành vi tương ứng của con người, đó là bằng chứng cho thấy một số cơ chế của chương trình cũng có thể hoạt động ở người.

Ví dụ, Allen Newell và Herbert Simon, những người đã phát triển GPS, "General Problem Solver" (Newell và Simon, 1961), không hài lòng với việc chỉ đơn thuần để chương trình của họ giải quyết

vấn đề một cách chính xác. Họ quan tâm nhiều hơn đến việc so sánh trình tự và thời gian của các bước lý luận của nó với các bước của các đối tượng con người giải quyết các vấn đề tương tự. Lĩnh vực liên phân chia của khoa học nhận thức tập hợp các mô hình máy tính từ AI và thực nghiệm

Khoa học nhận thức

các kỹ thuật từ tâm lý học để xây dựng các lý thuyết chính xác và có thể kiểm tra được về tâm trí con người.

Khoa học nhận thức là một lĩnh vực hấp dẫn, xứng đáng với một số sách giáo khoa và ít nhất một bách khoa toàn thư (Wilson và Keil, 1999). Thành thạo chúng tôi sẽ nhận xét về những điểm tương đồng hoặc khác biệt giữa các kỹ thuật AI và nhận thức của con người. Tuy nhiên, khoa học nhận thức thực sự nhất thiết phải dựa trên điều tra thực nghiệm về con người hoặc động vật thực tế. Chúng tôi sẽ để lại điều đó cho những cuốn sách khác, vì chúng tôi cho rằng người đọc chỉ có một máy tính để thử nghiệm.

Trong những ngày đầu của AI, thường có sự nhầm lẫn giữa các cách tiếp cận. Một tác giả sẽ lặp lại rằng một thuật toán hoạt động tốt trong một nhiệm vụ và do đó nó là một mô hình tốt về hiệu suất của con người, hoặc ngược lại. Các tác giả hiện đại tách biệt hai loại tuyên bố; sự khác biệt này đã cho phép cả AI và khoa học nhận thức phát triển nhanh hơn. Hai lĩnh vực này bổn phân cho nhau, đáng chú ý nhất là trong thị giác máy tính, kết hợp bằng chứng sinh lý thần kinh vào các mô hình tính toán. Gần đây, sự kết hợp của các phương pháp hình ảnh thần kinh kết hợp với các kỹ thuật học máy để phân tích dữ liệu như vậy đã dẫn đến sự khởi đầu của khả năng "đọc suy nghĩ" - nghĩa là xác định nội dung ngữ nghĩa của suy nghĩ bên trong của một người. Khả năng này có thể làm sáng tỏ thêm về cách thức hoạt động của nhận thức con người.

1.1.3 Suy nghĩ hợp lý: Cách tiếp cận "quy luật tư tưởng"

Nhà triết học Hy Lạp Aristotle là một trong những người đầu tiên cố gắng hệ thống hóa "suy nghĩ đúng đắn" - nghĩa là các quá trình lý luận không thể chối cãi. Tam đoạn luận của ông đã cung cấp các mẫu cho cấu trúc lập luận-

Tam đoạn luận

luôn mang lại kết luận chính xác khi được đưa ra các tiền đề chính xác. Ví dụ kinh điển bắt đầu với Socrates là một người đàn ông và tất cả mọi người đều là người chết và kết luận rằng Socrates là người chết. (Ví dụ này có lẽ là do Sextus Empiricus chứ không phải Aristotle.) Những quy luật tư tưởng này được cho là chi phối hoạt động của tâm trí; Nghiên cứu của họ đã khởi xướng lĩnh vực được gọi là logic.

Sau nhà logic học trong thế kỷ 19 đã phát triển một ký hiệu chính xác cho các tuyên bố về các đối tượng trong thế giới và mối quan hệ giữa chúng. (Tương phản điều này với ký hiệu số học thông thường, chỉ cung cấp cho các câu về số.) Đến năm 1965, về nguyên tắc, các chương trình có thể giải quyết bất kỳ vấn đề nào có thể giải quyết được mô tả trong ký hiệu logic. Cái gọi là truyền thống logic học

Nhà logic học

Trong trí tuệ nhân tạo hy vọng sẽ xây dựng dựa trên các chương trình như vậy để tạo ra các hệ thống thông minh.

Logic như được hiểu theo quy ước đòi hỏi kiến thức về thế giới là chắc chắn – một điều kiện, trong thực tế, hiếm khi đạt được. Chúng ta chỉ đơn giản là không biết các quy tắc, chẳng hạn, chính trị hoặc chiến tranh giống như cách chúng ta biết các quy tắc của cờ vua hoặc số học. Lý thuyết xác suất lấp đầy khoảng trống này, cho phép suy luận nghiêm ngặt với thông tin không chắc chắn. Trong

Xác suất

nguyên tắc, nó cho phép xây dựng một mô hình toàn diện về tư duy hợp lý, dẫn từ thông tin nhận thức thô đến sự hiểu biết về cách thế giới hoạt động đến các dự đoán về tương lai. Những gì nó không làm, là tạo ra hành vi thông minh. Để làm được điều đó, chúng ta cần một lý thuyết về hành động hợp lý. Tư tưởng hợp lý, bản thân nó, là không đủ.

1.1.4 Hành động hợp lý: Cách tiếp cận tác nhân hợp lý

Tác nhân chỉ là một cái gì đó hành động (tác nhân xuất phát từ tiếng Latinh agere, to do). Tất nhiên

Người đại lý

Tất cả các chương trình máy tính đều làm điều gì đó, nhưng các tác nhân máy tính được kỳ vọng sẽ làm nhiều hơn thế: hoạt động tự động, nhận thức môi trường của chúng, tồn tại trong một khoảng thời gian dài, thích nghi

Tác nhân hợp lý thay đổi, tạo ra và theo đuổi mục tiêu. Một tác nhân hợp lý là một tác nhân hành động để đạt được kết quả tốt nhất hoặc, khi có sự không chắc chắn, kết quả mong đợi tốt nhất. Trong cách tiếp cận "quy luật tư tưởng" đối với AI, trọng tâm là suy luận chính xác. Tạo ra những suy luận đúng đắn đôi khi là một phần của việc trở thành một tác nhân hợp lý, bởi vì một cách để hành động một cách hợp lý là suy luận rằng một hành động nhất định là tốt nhất và sau đó hành động theo kết luận đó. Mặt khác, có những cách hành động hợp lý không thể nói là liên quan đến suy luận. Ví dụ, giặt từ bếp nóng là một hành động phản xạ thường thành công hơn hành động chậm hơn được thực hiện sau khi cân nhắc cẩn thận. Tất cả các kỹ năng cần thiết cho bài kiểm tra Turing cũng cho phép một tác nhân hành động hợp lý. Trình bày kiến thức và lý luận cho phép các tác nhân đưa ra quyết định đúng đắn. Chúng ta cần có khả năng tạo ra những câu để hiểu bằng ngôn ngữ tự nhiên để vượt qua trong một xã hội phức tạp. Chúng ta cần học hỏi không chỉ để uyên bác, mà còn vì nó cải thiện khả năng tạo ra hành vi hiệu quả của chúng ta, đặc biệt là trong những hoàn cảnh mới.

Cách tiếp cận tác nhân hợp lý đối với AI có hai lợi thế so với các cách tiếp cận khác. Thứ nhất, nó tổng quát hơn cách tiếp cận "quy luật tư tưởng" vì suy luận chính xác chỉ là một trong một số cơ chế khả thi để đạt được tính hợp lý. Thứ hai, nó dễ dàng hơn với sự phát triển khoa học. Tiêu chuẩn của tính hợp lý được xác định rõ về mặt toán học và hoàn toàn tổng quát. Chúng ta thường có thể làm việc trở lại từ đặc điểm kỹ thuật này để rút ra các thiết kế tác nhân có thể chứng minh được nó - một điều mà phần lớn là không thể nếu mục tiêu là bắt chước hành vi hoặc quá trình suy nghĩ của con người.

Vì những lý do này, cách tiếp cận tác nhân hợp lý đối với AI đã chiếm ưu thế trong hầu hết lịch sử của lĩnh vực này. Trong những thập kỷ đầu, các tác nhân hợp lý được xây dựng trên nền tảng logic và hình thành các kế hoạch xác định để đạt được các mục tiêu cụ thể. Sau đó, các phương pháp dựa trên lý thuyết xác suất và học máy cho phép tạo ra các tác nhân có thể đưa ra quyết định không chắc chắn để đạt được kết quả mong đợi tốt nhất. Tóm lại, AI đã tập trung vào nghiên cứu I và xây dựng các tác nhân làm điều đúng đắn. Điều gì được coi là điều đúng đắn được xác định làm điều đúng đắn

Mô hình tiêu chuẩn theo mục tiêu mà chúng tôi cung cấp cho đại lý. Mô hình chung này phổ biến đến mức chúng ta có thể gọi nó là mô hình tiêu chuẩn. Nó chiếm ưu thế không chỉ trong AI, mà còn trong lý thuyết điều khiển, trong đó bộ điều khiển giảm thiểu chức năng chi phí; trong nghiên cứu hoạt động, trong đó một chính sách tối đa hóa một khoản tiền thưởng; trong thống kê, trong đó quy tắc quyết định giảm thiểu hàm thua lỗ; và trong kinh tế học, nơi người ra quyết định tối đa hóa tiện ích hoặc một số biện pháp phúc lợi xã hội. Chúng ta cần thực hiện một sự tinh chỉnh quan trọng đối với mô hình tiêu chuẩn để giải thích thực tế rằng tính hợp lý hoàn hảo - luôn thực hiện hành động tối ưu chính xác - là không khả thi trong môi trường phức tạp. Nhu cầu tính toán quá cao. Chương 6 và 16 đề cập đến vấn đề lý trí hạn chế - hành động thích hợp khi không có đủ thời gian để làm tất cả những tính toán mà người ta có thể thích. Tuy nhiên, tính hợp lý hoàn hảo thường vẫn là một điểm khởi đầu tốt cho phân tích lý thuyết.

1.1.5 Máy có lý

Mô hình tiêu chuẩn đã là một hướng dẫn hữu ích cho nghiên cứu AI kể từ khi thành lập, nhưng nó có lẽ không phải là mô hình phù hợp về lâu dài. Lý do là mô hình tiêu chuẩn giả định rằng chúng tôi sẽ cung cấp một mục tiêu được chỉ định đầy đủ cho ~~Đó~~ với một nhiệm vụ được xác định nhân tạo như cờ vua hoặc tính toán đường dẫn ngắn nhất, nhiệm vụ đi kèm với một mục tiêu được tích hợp sẵn — vì vậy mô hình tiêu chuẩn có thể áp dụng. Tuy nhiên, khi chúng ta bước vào thế giới thực, ngày càng trở nên khó khăn hơn để xác định mục tiêu hoàn toàn và

Chính xác. Ví dụ, khi thiết kế một chiếc xe tự lái, người ta có thể nghĩ rằng mục tiêu là đến đích an toàn. Nhưng lái xe dọc theo bất kỳ con đường nào cũng có nguy cơ bị thương do những người lái xe sai lầm khác, hỏng hóc thiết bị, v.v.; Vì vậy, một mục tiêu nghiêm ngặt về an toàn đòi hỏi phải ở trong nhà để xe. Có một sự đánh đổi giữa việc tiến bộ đến đích và có nguy cơ chấn thương. Sự đánh đổi này nên được thực hiện như thế nào? Hơn nữa, chúng ta có thể cho phép chiếc xe thực hiện các hành động gây khó chịu cho những người lái xe khác ở mức độ nào? Xe nên điều chỉnh tăng tốc, đánh lái và phanh bao nhiêu để tránh làm rung lắc hành khách? Những loại câu hỏi này rất khó để trả lời tiên nghiệm. Chúng đặc biệt có vấn đề trong lĩnh vực tương tác chung giữa con người và robot, trong đó xe tự lái là một ví dụ.

Vấn đề đạt được sự đồng thuận giữa sở thích thực sự của chúng ta và mục tiêu mà chúng ta đưa vào máy được gọi là vấn đề cân chỉnh giá trị: các giá trị hoặc mục tiêu được đưa vào Vấn đề cân chỉnh giá trị

Máy phải phù hợp với máy của con người. Nếu chúng ta đang phát triển một hệ thống AI trong phòng thí nghiệm hoặc trong trình mô phỏng — như đã từng xảy ra trong hầu hết lịch sử của lĩnh vực này — thì có một cách khắc phục dễ dàng cho một mục tiêu được chỉ định không chính xác: đặt lại hệ thống, sửa mục tiêu và thử lại. Khi lĩnh vực này tiến tới các hệ thống thông minh ngày càng có khả năng được triển khai trong thế giới thực, cách tiếp cận này không còn khả thi. Một hệ thống được triển khai với mục tiêu không chính xác sẽ gây ra những hậu quả tiêu cực. Hơn nữa, hệ thống càng thông minh thì hậu quả càng tiêu cực.

Quay trở lại ví dụ dường như không có vấn đề gì về cờ vua, hãy xem xét điều gì sẽ xảy ra nếu cỗ máy đủ thông minh để lý luận và hành động vượt ra ngoài giới hạn của bàn cờ. Trong trường hợp đó, nó có thể cố gắng tăng cơ hội chiến thắng bằng những mảnh khoe như thổi miên hoặc tổng tiền đối thủ của mình hoặc hối lộ khán giả để tạo ra những tiếng ồn sột soạt trong thời gian suy nghĩ của đối thủ.³ Nó cũng có thể cố gắng chiếm đoạt sức mạnh tinh thần bổ sung cho chính mình. Những hành vi này không phải là "không thông minh" hay "điên rồ"; chúng là một hệ quả hợp lý mà Jof định nghĩa chiến thắng là mục tiêu duy nhất của cỗ máy.

Không thể dự đoán tất cả các cách mà một cỗ máy theo đuổi một mục tiêu cố định có thể hoạt động sai. Do đó, có lý do chính đáng để nghĩ rằng mô hình tiêu chuẩn là không đủ. Chúng tôi không muốn những cỗ máy thông minh theo nghĩa theo đuổi mục tiêu của chúng; chúng tôi muốn họ theo đuổi mục tiêu của chúng tôi. Nếu chúng ta không thể chuyển những mục tiêu đó một cách hoàn hảo cho máy, thì chúng ta cần một công thức mới - một công thức trong đó máy móc đang theo đuổi các mục tiêu của chúng ta, nhưng nhất thiết không chắc chắn về chúng là gì. Khi một cỗ máy biết rằng nó không biết mục tiêu hoàn chỉnh, nó có động lực để hành động thận trọng, xin phép, tìm hiểu thêm về sở thích của chúng ta thông qua quan sát và tuân theo sự kiểm soát của con người. Cuối cùng, chúng ta muốn các tác nhân Có lợi có lợi cho con người. Chúng ta sẽ quay trở lại chủ đề này trong Phần 1.5.

1.2 Nền tảng của trí tuệ nhân tạo

Trong phần này, chúng tôi cung cấp lịch sử ngắn gọn về các ngành đã đóng góp ý tưởng, quan điểm và kỹ thuật cho AI. Giống như bất kỳ lịch sử nào, lịch sử này tập trung vào một số ít người, sự kiện và ý tưởng và bỏ qua những người khác cũng quan trọng. Chúng tôi sắp xếp lịch sử xung quanh một loạt các câu hỏi. Chúng tôi chắc chắn không muốn tạo ấn tượng rằng những câu hỏi này là những câu hỏi duy nhất mà các ngành học giải quyết hoặc tất cả các ngành học đã và đang hướng tới AI như là kết quả cuối cùng của chúng.

³ Trong một trong những cuốn sách đầu tiên về cờ vua, Ruy Lopez (1561) đã viết, "Luôn đặt bàn cờ sao cho mặt trời chiếu vào mắt đối thủ của bạn."

1.2.1 Triết học

- Các quy tắc chính thức có thể được sử dụng để đưa ra kết luận hợp lệ không?
- Làm thế nào để tâm thức phát sinh từ một bộ não vật lý?
- Kiến thức đến từ đâu?
- Kiến thức dẫn đến hành động như thế nào?

Aristotle (384–322 trước Công nguyên) là người đầu tiên xây dựng một bộ định luật chính xác chi phối phần lý trí của tâm trí. Ông đã phát triển một hệ thống không chính thức của tam đoạn luận để lý luận đúng đắn, về nguyên tắc cho phép người ta đưa ra kết luận một cách máy móc, đưa ra các tiền đề ban đầu.

Ramon Llull (khoảng 1232–1315) đã nghĩ ra một hệ thống lý luận được xuất bản với tên gọi *Ars Magna* hoặc *Nghệ thuật vĩ đại* (1305). Llull đã cố gắng triển khai hệ thống của mình bằng cách sử dụng một thiết bị cơ khí thực tế: một bộ bánh xe giấy có thể được xoay thành các hoán vị khác nhau.

Khoảng năm 1500, Leonardo da Vinci (1452–1519) đã thiết kế nhưng không chế tạo một máy tính cơ khí; Các công trình tài tạo gần đây đã cho thấy thiết kế có chức năng. Cổ máy tính toán đầu tiên được biết đến được chế tạo vào khoảng năm 1623 bởi nhà khoa học người Đức Wilhelm Schickard (1592–1635). Blaise Pascal (1623–1662) đã xây dựng *Pascaline* vào năm 1642 và viết rằng nó "tạo ra các hiệu ứng đường như gần với suy nghĩ hơn tất cả các hành động của động vật." Gottfried Wilhelm Leibniz (1646–1716) đã chế tạo một thiết bị cơ khí nhằm thực hiện các hoạt động dựa trên các khái niệm hơn là các con số, nhưng phạm vi của nó khá hạn chế. Trong cuốn sách *Leviathan* năm 1651 của mình, Thomas Hobbes (1588–1679) đã đề xuất ý tưởng về một cỗ máy tư duy, một "động vật nhân tạo", theo lời của ông, lập luận "Vi trí tìm là gì mà là một mùa xuân; và dây thần kinh, nhưng rất nhiều dây; và các khớp, nhưng rất nhiều bánh xe." Ông cũng gọi ý rằng lý luận giống như tính toán số: "Vi lý trí"... không có gì khác ngoài 'tính toán', đó là cộng và trừ."

Nói rằng tâm thức hoạt động, ít nhất là một phần, theo các quy tắc logic hoặc *numerical*, và xây dựng các hệ thống vật lý mô phỏng một số quy tắc đó. Đó là một cách khác để nói rằng bản thân tâm trí là một hệ thống vật lý như vậy. René Descartes (1596–1650) đã đưa ra cuộc thảo luận rõ ràng đầu tiên về sự khác biệt giữa tâm trí và vật chất. Ông lưu ý rằng một quan niệm vật lý thuần túy về tâm trí dường như để lại rất ít chỗ cho ý chí tự do. Nếu tâm trí bị chi phối bởi các quy luật vật lý, thì nó không có ý chí tự do hơn một tảng đá "quyết định" rơi xuống. Descartes là một người đề xướng thuyết nhị nguyên. Ông cho rằng có một phần của tâm trí con người (hoặc

linh hồn hoặc tinh thần) nằm ngoài tự nhiên, được miễn trừ khỏi các quy luật vật lý. Mặt khác, động vật không sở hữu phẩm chất kép này; chúng có thể được coi là máy móc.

Một giải pháp thay thế cho thuyết nhị nguyên là chủ nghĩa duy vật, cho rằng hoạt động của bộ não phù hợp với các định luật vật lý cấu thành tâm trí. Ý chí tự do chỉ đơn giản là cách mà nhận thức về các lựa chọn có sẵn xuất hiện đối với thực thể lựa chọn. Các thuật ngữ chủ nghĩa vật lý và chủ nghĩa tự nhiên cũng được sử dụng để mô tả quan điểm này trái ngược với siêu nhiên.

Với một tâm trí vật lý thao túng kiến thức, vấn đề tiếp theo là thiết lập nguồn gốc của kiến thức.

Phong trào chủ nghĩa kinh nghiệm, bắt đầu với phong trào của Francis Bacon (1561–1626) *Novum Organum*,⁴ được đặc trưng bởi một câu châm ngôn của John Locke (1632–1704): "Không có gì trong sự hiểu biết, mà không phải là đầu tiên trong các giác quan."

David Hume (1711–1776) *A Treatise of Human Nature* (Hume, 1739) đã đề xuất cái mà ngày nay được gọi là nguyên tắc quy nạp: rằng các quy tắc chung có được bằng cách tiếp xúc với liên kết lặp đi lặp lại giữa các yếu tố của chúng.

⁴ *Novum Organum* là một bản cập nhật của *Organon* của Aristotle, hoặc công cụ tư tưởng.

Dualism

Chủ nghĩa kinh nghiệm

Cảm ứng

Dựa trên công trình của Ludwig Wittgenstein (1889–1951) và Bertrand Russell (1872–1970), Vòng tròn Vienna nổi tiếng (Sigmund, 2017), một nhóm các nhà triết học và nhà toán học gặp nhau ở Vienna vào những năm 1920 và 1930, đã phát triển học thuyết về chủ nghĩa thực chứng logic.

Chủ nghĩa thực chứng
logic

Học thuyết này cho rằng tất cả kiến thức có thể được đặc trưng bởi các lý thuyết logic được kết nối với các câu quan sát tương ứng với đầu vào cảm giác; do đó chủ nghĩa thực chứng logic

Câu quan sát

kết hợp chủ nghĩa duy lý và chủ nghĩa kinh nghiệm.

Lý thuyết xác nhận của Rudolf Carnap (1891–1970) và Carl Hempel (1905–1997) Lý thuyết xác nhận cố gắng phân tích việc tiếp thu kiến thức từ kinh nghiệm bằng cách định lượng mức độ tin tưởng nên được gán cho các câu logic dựa trên mối liên hệ của chúng với các quan sát xác nhận hoặc không xác nhận chúng. Cuốn sách của Carnap Cấu trúc logic của thế giới (1928) có lẽ là lý thuyết đầu tiên về tâm trí như một quá trình tính toán.

Yếu tố cuối cùng trong bức tranh triết học của tâm trí là sự kết nối giữa tri thức và hành động. Câu hỏi này rất quan trọng đối với AI vì trí thông minh đòi hỏi hành động cũng như lý luận. Hơn nữa, chỉ bằng cách hiểu cách các hành động được biện minh, chúng ta mới có thể hiểu làm thế nào để xây dựng một tác nhân có hành động là chính đáng (hoặc hợp lý).

Aristotle lập luận (trong *De Motu Animalium*) rằng các hành động được biện minh bởi một mối liên hệ logic giữa các mục tiêu và kiến thức về kết quả của hành động:

Nhưng làm thế nào mà suy nghĩ đôi khi đi kèm với hành động và đôi khi không, đôi khi bằng chuyển động, và đôi khi không? Có vẻ như điều tương tự xảy ra như trong trường hợp lý luận và suy luận về các đối tượng không thay đổi. Nhưng trong trường hợp đó, kết thúc là một mệnh đề suy đoán. . . trong khi ở đây kết luận kết quả từ hai tiền đề là một hành động. . . . Tôi cần che chắn; áo choàng là một tấm che phủ. Tôi cần một chiếc áo choàng. Những gì tôi cần, tôi phải làm; Tôi cần một chiếc áo choàng. Tôi phải làm một chiếc áo choàng. Và kết luận, "Tôi phải làm một chiếc áo choàng", là một hành động.

Trong Đạo đức học Nicomachean (Quyển III. 3, 1112b), Aristotle giải thích thêm về chủ đề này, đề xuất một thuật toán:

Chúng ta cần nhắc không phải về mục đích, nhưng về phương tiện. Vì một bác sĩ không cần nhắc liệu anh ta có chữa lành hay không, cũng như một nhà hùng biện có thuyết phục hay không, . . . Họ giả định mục đích và quan tâm đến việc đạt được nó như thế nào và bằng phương tiện nào, và nếu nó có vẻ dễ dàng và tốt nhất được tạo ra bởi nó; trong khi nếu nó đạt được chỉ bằng một phương tiện, họ xem xét nó sẽ đạt được như thế nào bằng cách này và bằng cách nào điều này sẽ đạt được, cho đến khi họ đi đến nguyên nhân đầu tiên, . . . và cái gì cuối cùng trong thứ tự phân tích dường như là thứ nhất trong thứ tự trở thành. Và nếu chúng ta đến một điều không thể, chúng ta từ bỏ việc tìm kiếm, ví dụ, nếu chúng ta cần tiền và điều này không thể có được; nhưng nếu một điều có thể xảy ra, chúng tôi cố gắng làm điều đó.

Thuật toán của Aristotle được thực hiện 2300 năm sau đó bởi Newell và Simon trong chương trình General Problem Solver của họ. Bây giờ chúng ta sẽ gọi nó là một hệ thống lập kế hoạch hồi quy tham lam (xem Chương 11). Các phương pháp dựa trên lập kế hoạch logic để đạt được các mục tiêu xác định đã thống trị trong vài thập kỷ đầu tiên của nghiên cứu lý thuyết về AI.

Suy nghĩ thuần túy về hành động đạt được mục tiêu thường hữu ích nhưng đôi khi không thể thực hiện được. Ví dụ, nếu có một số cách khác nhau để đạt được mục tiêu, cần phải có cách nào đó để lựa chọn trong số đó. Quan trọng hơn, có thể không thể đạt được mục tiêu một cách chắc chắn, nhưng vẫn phải thực hiện một số hành động. Vậy thì người ta nên quyết định như thế nào? Antoine Ar-nauld (1662), phân tích khái niệm về các quyết định hợp lý trong cơ học, đã đề xuất một công thức định lượng để tối đa hóa giá trị tiền tệ kỳ vọng của kết quả. Sau đó, Daniel Bernoulli (1738) đã giới thiệu khái niệm tổng quát hơn về tiện ích để nắm bắt giá trị bên trong, chủ quan

Tiện ích

của một kết quả. Khái niệm hiện đại về việc ra quyết định hợp lý trong điều kiện không chắc chắn liên quan đến việc tối đa hóa tiện ích mong đợi, như đã giải thích trong Chương 15.

Chủ nghĩa thực dụng Trong các vấn đề đạo đức và chính sách công, người ra quyết định phải xem xét lợi ích của nhiều cá nhân. Jeremy Bentham (1823) và John Stuart Mill (1863) đã thúc đẩy ý tưởng về chủ nghĩa thực dụng: rằng việc ra quyết định hợp lý dựa trên tối đa hóa tiện ích nên được áp dụng

cho tất cả các lĩnh vực hoạt động của con người, bao gồm cả các quyết định chính sách công được đưa ra thay mặt cho nhiều cá nhân. Chủ nghĩa vị lợi là một loại chủ nghĩa hậu quả cụ thể: ý tưởng rằng những gì đúng và sai được xác định bởi kết quả mong đợi của một hành động.

Đạo đức Deontological Ngược lại, Immanuel Kant, vào năm 1785, đã đề xuất một lý thuyết về đạo đức dựa trên quy tắc hoặc deontological, trong đó "làm điều đúng" được xác định không phải bởi kết quả mà bởi xã hội phổ quát

luật điều chỉnh các hành động được phép, chẳng hạn như "không nói dối" hoặc "không giết người". Do đó, một người theo chủ nghĩa thực dụng có thể nói dối trắng nếu điều tốt được mong đợi lớn hơn điều xấu, nhưng một người Kant sẽ không bị ràng buộc, bởi vì nói dối vốn dĩ là sai. Mill thừa nhận giá trị của các quy tắc, nhưng hiểu chúng là các thủ tục quyết định hiệu quả được biên soạn từ lý luận nguyên tắc đầu tiên về hậu quả. Nhiều hệ thống AI hiện đại áp dụng chính xác cách tiếp cận này.

1.2.2 Toán học

- Các quy tắc chính thức để đưa ra kết luận hợp lệ là gì?
- Những gì có thể được tính toán?
- Làm thế nào để chúng ta lý luận với thông tin không chắc chắn? Các nhà triết học đã đưa ra một số ý tưởng cơ bản của AI, nhưng bước nhảy vọt sang một khoa học chính thức đòi hỏi phải toán học hóa logic và xác suất và giới thiệu một nhánh toán học mới: tính toán. Ý tưởng về logic hình thức có thể bắt nguồn từ các triết gia của Hy Lạp cổ đại, Ấn Độ,

Logic hình thức

và Trung Quốc, nhưng sự phát triển toán học của nó thực sự bắt đầu với công trình của George Boole (1815–1864), người đã tìm ra các chi tiết của logic mệnh đề, hoặc Boolean (Boole, 1847). Năm 1879, Gottlob Frege (1848–1925) đã mở rộng logic của Boole để bao gồm các đối tượng và mối quan hệ, tạo ra logic bậc nhất được sử dụng ngày nay.5 Ngoài vai trò trung tâm của nó trong nghiên cứu AI ban đầu, logic bậc nhất đã thúc đẩy công việc của Gödel và Turing làm nền tảng cho chính tính toán, như chúng tôi giải thích bên dưới.

Xác suất

Lý thuyết xác suất có thể được coi là khái quát hóa logic cho các tình huống không chắc chắn thông tin — một sự cân nhắc có tầm quan trọng lớn đối với AI. Gerolamo Cardano (1501–1576) lần đầu tiên đưa ra ý tưởng về xác suất, mô tả nó dưới dạng kết quả có thể xảy ra của các sự kiện cờ bạc. Năm 1654, Blaise Pascal (1623–1662), trong một bức thư gửi cho Pierre Fermat (1601–1665), đã chỉ ra cách dự đoán tương lai của một trò chơi cờ bạc chưa hoàn thành và chỉ định mức thanh toán trung bình cho những người đánh bạc. Xác suất nhanh chóng trở thành một phần vô giá của khoa học định lượng, giúp đối phó với các phép đo không chắc chắn và các lý thuyết không đầy đủ. Jacob Bernoulli (1654–1705, chú của Daniel), Pierre Laplace (1749–1827), và những người khác đã đưa ra lý thuyết và giới thiệu các phương pháp thống kê mới. Thomas Bayes (1702–1761) đề xuất một quy tắc cập nhật xác suất dưới ánh sáng của bằng chứng mới; Quy tắc của Bayes là một công cụ quan trọng cho các hệ thống AI.

Thống kê

Việc chính thức hóa xác suất, kết hợp với sự sẵn có của dữ liệu, dẫn đến sự nổi bật của thống kê như một lĩnh vực. Một trong những cách sử dụng đầu tiên là phân tích của John Grount về Lon-

5 Ký hiệu được đề xuất của Frege cho logic bậc nhất - một sự kết hợp bí ẩn giữa các đặc điểm văn bản và hình học - không bao giờ trở nên phổ biến.

Dữ liệu điều tra dân số năm 1662. Ronald Fisher được coi là nhà thống kê hiện đại đầu tiên (Fisher, 1922). Ông đã tập hợp các ý tưởng về xác suất, thiết kế thí nghiệm, phân tích dữ liệu và máy tính — vào năm 1919, ông nhấn mạnh rằng ông không thể thực hiện công việc của mình nếu không có một máy tính cơ học có tên là MILLIONAIRE (máy tính đầu tiên có thể thực hiện phép nhân), mặc dù chi phí của máy tính cao hơn mức lương hàng năm của ông (Ross, 2012).

Lịch sử tính toán cũng lâu đời như lịch sử của các con số, nhưng thuật toán không tầm thường đầu tiên được cho là thuật toán của Euclid để tính toán các ước chung lớn nhất. Các thuật toán từ xuất phát từ Muhammad ibn Musa al-Khwarizmi, một nhà toán học thế kỷ thứ 9, người có các tác phẩm cũng giới thiệu chữ số và đại số Ả Rập đến châu Âu. Boole và những người khác đã thảo luận về các thuật toán để suy luận logic, và vào cuối thế kỷ 19, những nỗ lực đã được tiến hành để chính thức hóa lý luận toán học tổng quát như suy luận logic.

Kurt Gödel (1906–1978) đã chỉ ra rằng tồn tại một thủ tục hiệu quả để chứng minh bất kỳ tuyên bố đúng nào trong logic bậc nhất của Frege và Russell, nhưng logic bậc nhất đó không thể bao gồm nguyên lý quy nạp toán học cần thiết để mô tả các số tự nhiên. Năm 1931, Gödel đã chỉ ra rằng các giới hạn về khả trừ vẫn tồn tại. Định lý không đầy đủ của ông cho thấy

Định lý không đầy đủ

rằng trong bất kỳ lý thuyết hình thức nào mạnh như số học Peano (lý thuyết cơ bản của các số tự nhiên), nhất thiết phải có những phát biểu đúng không có bằng chứng trong lý thuyết.

Kết quả cơ bản này cũng có thể được hiểu là cho thấy rằng một số hàm trên số nguyên không thể được biểu diễn bằng một thuật toán — nghĩa là chúng không thể được tính toán.

Điều này thúc đẩy Alan Turing (1912–1954) cố gắng mô tả chính xác hàm nào có thể tính toán được - có khả năng được tính toán bằng một thủ tục hiệu quả. Luận điểm Church-Turing Khả năng tính toán

đề xuất xác định khái niệm chung về khả năng tính toán với các hàm được tính toán bởi một máy Turing (Turing, 1936). Turing cũng chỉ ra rằng có một số hàm mà không Turingmachine nào có thể tính toán. Ví dụ, không có máy nào có thể nói chung liệu một chương trình nhất định sẽ trả về câu trả lời trên một đầu vào nhất định hay chạy mãi mãi.

Mặc dù khả năng tính toán rất quan trọng đối với sự hiểu biết về tính toán, nhưng khái niệm về khả năng tính toán thậm chí còn có tác động lớn hơn đến AI. Nói một cách đại khái, một vấn đề được gọi là Khả năng điều khiển

khó giải quyết nếu thời gian cần thiết để giải quyết các trường hợp của vấn đề tăng theo cấp số nhân theo kích thước của các trường hợp. Sự khác biệt giữa sự phức tạp tăng trưởng đa thức và hàm mũ lần đầu tiên được nhấn mạnh vào giữa những năm 1960 (Cobham, 1964; Edmonds, 1965). Điều này rất quan trọng vì tăng trưởng theo cấp số nhân có nghĩa là ngay cả những trường hợp lớn vừa phải cũng không thể được giải quyết trong bất kỳ thời gian hợp lý nào.

Lý thuyết về sự hoàn chỉnh của NP, được tiên phong bởi Cook (1971) và Karp (1972), cung cấp một

NP-hoàn chỉnh

cơ sở để phân tích khả năng xử lý của các vấn đề: bất kỳ lớp vấn đề nào mà lớp bài toán NP-complete có thể được giảm xuống đều có khả năng khó giải quyết. (Mặc dù chưa được chứng minh rằng các vấn đề NP-complete nhất thiết phải giải quyết, nhưng hầu hết các nhà lý thuyết đều tin điều đó.) Những kết quả này trái ngược với sự lạc quan mà báo chí đại chúng chào đón những chiếc máy tính đầu tiên - "Siêu bộ não điện tử" "Nhanh hơn Einstein!" Mặc dù tốc độ ngày càng tăng của máy tính, việc sử dụng cẩn thận các tài nguyên và sự không hoàn hảo cần thiết sẽ đặc trưng cho các hệ thống thông minh. Nói một cách thô thiển, thế giới là một ví dụ về vấn đề cực kỳ lớn!

1.2.3 Kinh tế

- Chúng ta nên đưa ra quyết định phù hợp với sở thích của mình như thế nào?
- Chúng ta nên làm điều này như thế nào khi người khác có thể không đồng ý?
- Chúng ta nên làm điều này như thế nào khi phần thưởng có thể còn xa trong tương lai?

Khoa học kinh tế bắt nguồn từ năm 1776, khi Adam Smith (1723–1790) xuất bản *An Inquiry into the Nature and Causes of the Wealth of Nations*. Smith đề xuất phân tích các nền kinh tế bao gồm nhiều tác nhân riêng lẻ quan tâm đến lợi ích riêng của họ. Tuy nhiên, Smith không ủng hộ lòng tham tài chính như một quan điểm đạo đức: cuốn sách *The Theory of Moral Sentiments* trước đó (1759) của ông bắt đầu bằng cách chỉ ra rằng mối quan tâm đến hạnh phúc của người khác là một thành phần thiết yếu của lợi ích của mỗi cá nhân.

Hầu hết mọi người nghĩ về kinh tế học là về tiền bạc, và thực sự phân tích toán học đầu tiên về các quyết định trong điều kiện không chắc chắn, công thức giá trị kỳ vọng tối đa của *Arnauld* (1662), đề cập đến giá trị tiền tệ của đặt cược. *Daniel Bernoulli* (1738) nhận thấy rằng *for-mula* này dường như không hoạt động tốt với số tiền lớn hơn, chẳng hạn như đầu tư vào các cuộc thám hiểm buôn bán hàng hải. Thay vào đó, ông đề xuất một nguyên tắc dựa trên việc tối đa hóa tiện ích mong đợi, và giải thích các lựa chọn đầu tư của con người bằng cách đề xuất rằng tiện ích cận biên của một lượng tiền bổ sung giảm khi người ta có được nhiều tiền hơn.

Léon Walras (phát âm là "*Valrasse*") (1834–1910) đã cung cấp cho lý thuyết tiện ích một nền tảng tổng quát hơn về sở thích giữa các cạnh bạc trên bất kỳ kết quả nào (không chỉ kết quả tiền tệ). Lý thuyết này đã được cải thiện bởi *Ramsey* (1931) và sau đó bởi *John von Neumann* và *Oskar Morgenstern* trong cuốn sách của họ *Lý thuyết về trò chơi và hành vi kinh tế* (1944). Kinh tế học không còn là nghiên cứu về tiền tệ nữa; thay vào đó là nghiên cứu về mong muốn và sở thích.

Lý thuyết quyết định

Lý thuyết quyết định, kết hợp lý thuyết xác suất với lý thuyết tiện ích, cung cấp một khuôn khổ mal và hoàn chỉnh cho các quyết định cá nhân (kinh tế hoặc cách khác) được thực hiện dưới sự không chắc chắn - nghĩa là, trong trường hợp các mô tả xác suất nắm bắt một cách thích hợp môi trường của người tạo ra giải quyết. Điều này phù hợp với các nền kinh tế "lớn", nơi mỗi tác nhân không cần chú ý đến hành động của các tác nhân khác với tư cách cá nhân. Đối với các nền kinh tế "nhỏ", tình huống giống như một trò chơi: hành động của một người chơi có thể ảnh hưởng đáng kể đến tiện ích của người khác (tích cực hoặc tiêu cực). Sự phát triển của *Von Neumann* và *Morgenstern* về lý thuyết trò chơi (xem thêm *Luce* và *Raiffa*, 1957) bao gồm kết quả đáng ngạc nhiên rằng, đối với một số trò chơi, một tác nhân hợp lý nên áp dụng các chính sách (hoặc ít nhất dường như) ngẫu nhiên. Không giống như lý thuyết quyết định, lý thuyết trò chơi không đưa ra một đơn thuốc rõ ràng để lựa chọn hành động. Trong AI, các quyết định liên quan đến nhiều tác nhân được nghiên cứu dưới tiêu đề hệ thống đa tác nhân (Chương 17).

Nghiên cứu hoạt động

Các nhà kinh tế, với một số ngoại lệ, đã không giải quyết câu hỏi thứ ba được liệt kê ở trên: làm thế nào để đưa ra quyết định hợp lý khi phân thưởng từ các hành động không phải là ngay lập tức mà thay vào đó là kết quả của một số hành động được thực hiện theo trình tự. Chủ đề này đã được theo đuổi trong lĩnh vực nghiên cứu hoạt động,

nổi lên trong Thế chiến II từ những nỗ lực ở Anh để tối ưu hóa việc lắp đặt radar, và sau đó tìm thấy vô số ứng dụng dân sự. Công trình của *Richard Bellman* (1957) đã chính thức hóa một lớp các bài toán quyết định tuần tự được gọi là quá trình quyết định Markov, mà chúng ta nghiên cứu trong Chương 16 và, dưới tiêu đề học tăng cường, trong Chương 23.

Công việc trong lĩnh vực kinh tế học và nghiên cứu hoạt động đã đóng góp nhiều vào khái niệm của chúng ta về các tác nhân duy lý, nhưng trong nhiều năm, nghiên cứu AI đã phát triển theo những con đường hoàn toàn riêng biệt. Một lý do là sự phức tạp rõ ràng của việc đưa ra các quyết định hợp lý. Nhà nghiên cứu AI tiên phong *Herbert Simon* (1916–2001) đã giành giải Nobel kinh tế năm 1978 cho công trình ban đầu của ông cho thấy các mô hình dựa trên sự thỏa mãn - đưa ra các quyết định "đủ tốt".

Thỏa mãn

hơn là tính toán một quyết định tối ưu - đưa ra một mô tả tốt hơn về hành vi thực tế của con người (*Simon*, 1947). Kể từ những năm 1990, đã có sự quan tâm trở lại đối với các kỹ thuật lý thuyết quyết định cho AI.

1.2.4 Khoa học thần kinh

• Bộ não xử lý thông tin như thế nào?

Khoa học thần kinh là nghiên cứu về hệ thần kinh, đặc biệt là não. Mặc dù chính xác Neuroscience Cách mà bộ não cho phép suy nghĩ là một trong những bí ẩn lớn của khoa học, thực tế là nó cho phép suy nghĩ đã được đánh giá cao trong hàng ngàn năm vì bằng chứng cho thấy những cú đánh mạnh vào đầu có thể dẫn đến mất khả năng tinh thần. Từ lâu người ta cũng biết rằng bộ não con người bằng cách nào đó khác nhau; vào khoảng năm 335 trước Công nguyên, Aristotle đã viết, "Trong tất cả các loài động vật, con người có bộ não lớn nhất tương ứng với kích thước của nó." 6 Tuy nhiên, phải đến giữa thế kỷ 18, bộ não mới được công nhận rộng rãi là nơi ở của ý thức. Trước đó, các vị trí ứng cử viên bao gồm tim và lá lách.

Cuộc điều tra của Paul Broca (1824–1880) về chứng mất ngôn ngữ (khiếm khuyết ngôn ngữ) ở những người bị tổn thương não vào năm 1861 đã bắt đầu nghiên cứu về tổ chức chức năng của não bằng cách xác định một vùng lo-calized ở bán cầu não trái - bây giờ được gọi là vùng Broca - chịu trách nhiệm cho việc tạo ra lời nói. 7 Vào thời điểm đó, người ta biết rằng não phần lớn bao gồm các tế bào thần kinh, hoặc neu-ron, nhưng phải đến năm 1873, Camillo Golgi (1843–1926) mới phát triển một kỹ thuật nhuộm

Neuron

cho phép quan sát các tế bào thần kinh riêng lẻ (xem Hình 1.1). Kỹ thuật này đã được sử dụng bởi Santiago Ramon y Cajal (1852–1934) trong các nghiên cứu tiên phong của ông về tổ chức tế bào thần kinh. 8 Hiện nay người ta chấp nhận rộng rãi rằng các chức năng nhận thức là kết quả của hoạt động điện hóa của các cấu trúc này. Nghĩa là, một tập hợp các tế bào đơn giản có thể dẫn đến suy nghĩ, hành động và Jconsciousness. Theo những lời ngắn gọn của John Searle (1992), bộ não gây ra tâm trí.

Bây giờ chúng ta có một số dữ liệu về ánh xạ giữa các khu vực của não và các bộ phận của cơ thể mà chúng kiểm soát hoặc từ đó chúng nhận được đầu vào cảm giác. Các bản đồ như vậy có thể thay đổi hoàn toàn trong vài tuần và một số loài động vật dường như có nhiều bản đồ. Hơn nữa, chúng tôi không hiểu đầy đủ làm thế nào các khu vực khác có thể tiếp quản chức năng khi một khu vực bị hư hại. Hầu như không có lý thuyết nào về cách một bộ nhớ cá nhân được lưu trữ hoặc về cách các chức năng nhận thức cấp cao hơn hoạt động.

Việc đo lường hoạt động của não nguyên vẹn bắt đầu vào năm 1929 với phát minh của Hans Bergerof điện não đồ (EEG). Sự phát triển của hình ảnh cộng hưởng từ chức năng (fMRI) (Ogawa và cộng sự, 1990; Cabeza và Nyberg, 2001) đang cung cấp cho các nhà thần kinh học những hình ảnh chi tiết chưa từng có về hoạt động của não, cho phép các phép đo tương ứng với các quá trình nhận thức đang diễn ra. Chúng được tăng cường bởi những tiến bộ trong ghi lại điện đơn bào về hoạt động của tế bào thần kinh và bằng các phương pháp quang di truyền học (Crick, 1999;

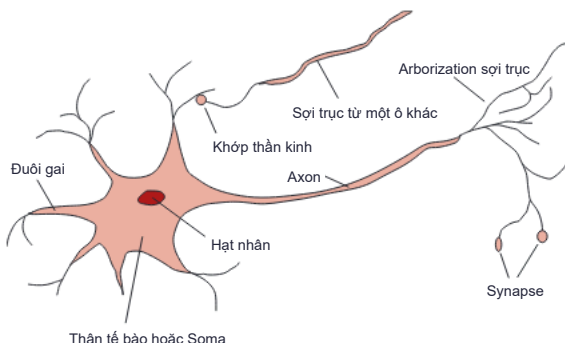
Quang di truyền học

Zemelman và cộng sự, 2002; Han và Boyden, 2007), cho phép đo lường và kiểm soát các tế bào thần kinh riêng lẻ được sửa đổi để nhạy cảm với ánh sáng.

Giao diện não-máy

Sự phát triển của giao diện não-máy (Lebedev và Nicolelis, 2006) cho cả hai Cảm biến và điều khiển vận động không chỉ hứa hẹn khôi phục chức năng cho những người khuyết tật mà còn làm sáng tỏ nhiều khía cạnh của hệ thống thần kinh. Một phát hiện đáng chú ý từ công trình này là bộ não có thể tự điều chỉnh để giao tiếp thành công với một thiết bị bên ngoài, điều trị nó trên thực tế giống như một cơ quan cảm giác hoặc chi khác.

6 Kể từ đó, người ta đã phát hiện ra rằng chuột chù cây và một số loài chim vượt quá tỷ lệ não / cơ thể con người. 7 Nhiều người trích dẫn Alexander Hood (1824) như một nguồn có thể có trước đó. 8 Golgi kiên trì với niềm tin của mình rằng các chức năng của não được thực hiện chủ yếu trong một môi trường liên tiếp trong đó các tế bào thần kinh được nhúng vào, trong khi Cajal đề xuất "học thuyết tế bào thần kinh". Hai người chia sẻ giải Nobel năm 1906 nhưng đã có những bài phát biểu nhận giải đối kháng lẫn nhau.



Hình 1.1 Các bộ phận của tế bào thần kinh hoặc tế bào thần kinh. Mỗi tế bào thần kinh bao gồm một thân tế bào, hoặc soma, chứa một nhân tế bào. Phân nhánh ra khỏi cơ thể tế bào là một số sợi được gọi là đuôi gai và một sợi dài duy nhất được gọi là sợi trục. Sợi trục kéo dài một khoảng cách dài, dài hơn nhiều so với tỷ lệ trong sơ đồ này chỉ ra. Thông thường, một sợi trục dài 1 cm (100 lần đường kính của thân tế bào), nhưng có thể lên tới 1 mét. Một tế bào thần kinh kết nối với 10 đến 100.000 tế bào thần kinh khác tại các điểm nối được gọi là khớp thần kinh. Các tín hiệu được truyền từ tế bào thần kinh này sang tế bào thần kinh khác bằng một phản ứng điện hóa phức tạp. Các tín hiệu kiểm soát hoạt động của não trong ngắn hạn và cũng cho phép những thay đổi lâu dài trong kết nối của các tế bào thần kinh. Những cơ chế này được cho là tạo cơ sở cho việc học trong não. Hầu hết quá trình xử lý thông tin diễn ra trong vỏ não, lớp ngoài của não. Đơn vị cơ quan-zational cơ bản dường như là một cột mô có đường kính khoảng 0,5 mm, chứa khoảng 20.000 tế bào thần kinh và mở rộng toàn bộ chiều sâu của vỏ não (khoảng 4 mm ở người).

Bộ não và máy tính kỹ thuật số có các đặc tính hơi khác nhau. Hình 1.2 cho thấy máy tính có thời gian chu kỳ nhanh hơn một triệu lần so với bộ não. Bộ não bù đắp cho điều đó bằng nhiều dung lượng lưu trữ và kết nối hơn cả một máy tính cá nhân cao cấp, mặc dù các siêu máy tính lớn nhất phù hợp với bộ não trên một số chỉ số. Các nhà tương lai học tạo ra phần lớn những con số này, chỉ ra một điểm kỳ dị đang đến gần mà tại đó máy tính đạt được một mức độ cao

mức độ hiệu suất của con người (Vinge, 1993; Kurzweil, 2005; Doctorow và Stross, 2012), và sau đó nhanh chóng cải thiện bản thân hơn nữa. Nhưng so sánh các con số thô không đặc biệt cung cấp thông tin. Ngay cả với một máy tính có dung lượng gần như không giới hạn, chúng ta vẫn đòi hỏi những đột phá khái niệm hơn nữa trong sự hiểu biết của chúng ta về trí thông minh (xem Chương 29). Nói một cách thô thiển, nếu không có lý thuyết đúng, máy móc nhanh hơn chỉ đưa ra câu trả lời sai cho bạn nhanh hơn.

1.2.5 Tâm lý học

• Con người và động vật suy nghĩ và hành động như thế nào? Nguồn gốc của tâm lý học khoa học thường bắt nguồn từ công trình của nhà vật lý học người Đức Hermann von Helmholtz (1821–1894) và học trò của ông là Wilhelm Wundt (1832–1920). Helmholtz đã áp dụng phương pháp khoa học để nghiên cứu thị giác của con người, và Sổ tay Quang học Sinh lý học của ông đã được mô tả là "chuyên luận quan trọng nhất về vật lý và sinh lý học của thị giác con người" (Nalwa, 1993, tr.15). Năm 1879, Wundt mở phòng thí nghiệm đầu tiên về tâm lý học thực nghiệm, tại Đại học Leipzig. Wundt nhấn mạnh cần thận

	Siêu máy tính	Máy tính cá nhân	Bộ não con người
Đơn vị tính toán	106 GPU + CPU	8 lõi CPU	106 cột
1015 bóng bán dẫn	1010 bóng bán dẫn	1011 tế bào thần kinh	1011 đơn vị lưu
trữ 1016 byte RAM	1010 byte RAM	1011 tế bào thần kinh	
	Đĩa 1017 byte	Đĩa 1012 byte	1014 khớp thần kinh
Thời gian chu kỳ	10–9 giây	10–9 giây	10–3 giây
Hoạt động/giây	1018	1010	1017

Hình 1.2 So sánh sơ bộ của một siêu máy tính hàng đầu, Summit (Feldman, 2017); một máy tính cá nhân điển hình của năm 2019; và bộ não con người. Sức mạnh não bộ của con người đã không thay đổi nhiều trong hàng ngàn năm, trong khi các siêu máy tính đã được cải thiện từ megaFLOP vào những năm 1960 lên gigaFLOP vào những năm 1980, teraFLOP vào những năm 1990, petaFLOP vào năm 2008 và exaFLOP vào năm 2018 (1 exaFLOP = 1018 phép toán dấu phẩy động mỗi giây).

các thí nghiệm có kiểm soát, trong đó công nhân của ông sẽ thực hiện một nhiệm vụ nhận thức hoặc liên kết trong khi xem xét nội tâm về quá trình suy nghĩ của họ. Các biện pháp kiểm soát cẩn thận đã đi một chặng đường dài để biến tâm lý học thành một khoa học, nhưng bản chất chủ quan của dữ liệu khiến các nhà thí nghiệm khó có thể bác bỏ lý thuyết của chính họ.

Mặt khác, các nhà sinh vật học nghiên cứu hành vi động vật thiếu dữ liệu nội tâm và phát triển một phương pháp luận khách quan, như được mô tả bởi HS Jennings (1906) trong tác phẩm có ảnh hưởng của ông Hành vi của các sinh vật thấp hơn. Áp dụng quan điểm này cho con người, phong trào chủ nghĩa hành vi, do John Watson (1878–1958) lãnh đạo, bác bỏ bất kỳ lý thuyết nào liên quan đến tinh thần

Chủ nghĩa hành vi

quá trình với lý do nội tâm không thể cung cấp bằng chứng đáng tin cậy. Các nhà hành vi chủ nghĩa khẳng khái chỉ nghiên cứu các biện pháp khách quan về nhận thức (hoặc kích thích) được đưa ra cho động vật và các hành động (hoặc phản ứng) kết quả của nó. Chủ nghĩa hành vi đã khám phá ra rất nhiều về chuột và chim bồ câu nhưng ít thành công hơn trong việc hiểu con người.

Tâm lý học nhận thức, xem não bộ như một thiết bị xử lý thông tin, có thể ít nhất là bắt nguồn từ các tác phẩm của William James (1842–1910). Helmholtz cũng nhấn mạnh rằng nhận thức liên quan đến một hình thức suy luận logic vô thức. Quan điểm nhận thức phần lớn bị lu mờ bởi chủ nghĩa hành vi ở Hoa Kỳ, nhưng tại Đơn vị Tâm lý học được áp dụng của Cambridge, do Frederic Bartlett (1886–1969) chỉ đạo, mô hình nhận thức đã có thể phát triển. Bản chất của giải thích, của sinh viên và người kế nhiệm của Bartlett là Kenneth Craik (1943), đã thiết lập lại tính hợp pháp của các thuật ngữ "tinh thần" như niềm tin và mục tiêu, lập luận rằng chúng cũng khoa học như sử dụng áp suất và nhiệt độ để nói về khí, mặc dù khí được tạo thành từ các phân tử không có cả hai.

Tâm lý học nhận thức

Craik chỉ định ba bước chính của một tác nhân dựa trên tri thức: (1) kích thích phải được chuyển thành một đại diện bên trong, (2) sự đại diện được thao túng bởi các quá trình nhận thức để rút ra các đại diện bên trong mới, và (3) những điều này lần lượt được chuyển lại thành hành động. Anh ấy giải thích rõ ràng tại sao đây là một thiết kế tốt cho một đại lý:

Nếu cơ thể mang một "mô hình quy mô nhỏ" về thực tại bên ngoài và các hành động khả dĩ của chính nó trong đầu, nó có thể thử các lựa chọn thay thế khác nhau, kết luận đầu là tốt nhất trong số chúng, phản ứng với các tình huống trong tương lai trước khi chúng phát sinh, sử dụng kiến thức về các sự kiện trong quá khứ để đối phó với hiện tại và tương lai, và bằng mọi cách để phản ứng đầy đủ hơn nhiều. an toàn hơn và có thẩm quyền hơn đối với các trường hợp khẩn cấp phải đối mặt. (Craik, 1943)

Sau cái chết của Craik trong một vụ tai nạn xe đạp vào năm 1945, công việc của ông được tiếp tục bởi Donald Broadbent, người có cuốn sách Nhận thức và Truyền thông (1958) là một trong những tác phẩm đầu tiên mô hình hóa các hiện tượng tâm lý như xử lý thông tin. Trong khi đó, ở Hoa Kỳ, sự phát triển của mô hình máy tính đã dẫn đến việc tạo ra lĩnh vực khoa học nhận thức. Lĩnh vực này có thể nói là đã bắt đầu tại một hội thảo vào tháng 9 năm 1956 tại MIT - chỉ hai tháng sau hội nghị mà chính AI đã "ra đời".

Tại hội thảo, George Miller trình bày The Magic Number Seven, Noam Chomsky trình bày Three Models of Language, và Allen Newell và Herbert Simon trình bày The Logic Theory Machine. Ba bài báo có ảnh hưởng này cho thấy cách các mô hình máy tính có thể được sử dụng để giải quyết tâm lý trí nhớ, ngôn ngữ và tư duy logic, tương ứng. Bây giờ quan điểm phổ biến (mặc dù không phổ biến) giữa các nhà tâm lý học rằng "lý thuyết nhận thức nên giống như một chương trình máy tính" (Anderson, 1980); nghĩa là, nó nên mô tả hoạt động của một chức năng nhận thức về mặt xử lý thông tin.

Đối với mục đích của bài đánh giá này, chúng tôi sẽ tính đến lĩnh vực tương tác giữa người và máy tính (HCI) theo tâm lý học. Doug Engelbart, một trong những người tiên phong của HCI, đã ủng hộ ý tưởng tăng cường trí thông minh — IA hơn là AI. Ông tin rằng máy tính nên tăng cường

Tăng cường trí thông minh

khả năng của con người thay vì tự động hóa các nhiệm vụ của con người. Năm 1968, "mẹ của tất cả" của Engelbart lần đầu tiên khoe chuột máy tính, một hệ thống cửa sổ, siêu văn bản và hội nghị truyền hình - tất cả trong nỗ lực chứng minh những gì những người lao động trí thức có thể đạt được với một số tăng cường trí thông minh.

Ngày nay, chúng ta có nhiều khả năng coi IA và AI là hai mặt của cùng một đồng xu, với cái trước nhấn mạnh sự kiểm soát của con người và cái sau nhấn mạnh hành vi thông minh của máy móc. Cả hai đều cần thiết để máy móc hữu ích cho con người.

1.2.6 Kỹ thuật máy tính

- Làm thế nào chúng ta có thể xây dựng một máy tính hiệu quả? Máy tính điện tử kỹ thuật số hiện đại được phát minh độc lập và gần như đồng thời bởi các nhà khoa học ở ba quốc gia bị chiến đấu trong Thế chiến II. Máy tính toán hoạt động đầu tiên là Heath Robinson cơ điện, được chế tạo vào năm 1943 bởi nhóm của Alan Turing cho một mục đích duy nhất: giải mã các thông điệp của Đức. Năm 1943, cùng một nhóm đã phát triển Colossus, một cỗ máy đa năng mạnh mẽ dựa trên ống chân không.¹⁰ Máy tính lập trình hoạt động đầu tiên là Z-3, phát minh của Konrad Zuse ở Đức

1941. Zuse cũng phát minh ra các số dấu phẩy động và ngôn ngữ lập trình cấp cao đầu tiên, Plankalkül. Máy tính điện tử đầu tiên, ABC, được lắp ráp bởi John Atanasoff và sinh viên của ông là Clifford Berry từ năm 1940 đến năm 1942 tại Đại học Bang Iowa. Nghiên cứu của Atanasoff nhận được rất ít sự hỗ trợ hoặc công nhận; đó là ENIAC, được phát triển như một phần của dự án quân sự se-cret tại Đại học Pennsylvania bởi một nhóm bao gồm John Mauchly và J. Presper Eckert, người đã chứng tỏ là tiền thân có ảnh hưởng nhất của máy tính hiện đại. Kể từ thời điểm đó, mỗi thế hệ phần cứng máy tính đã mang lại sự gia tăng về tốc độ và dung lượng và giảm giá - một xu hướng được ghi lại trong định luật Moore. Hiệu suất tăng gấp đôi

Định luật Moore

cứ sau 18 tháng hoặc lâu hơn cho đến khoảng năm 2005, khi các nhà sản xuất có vấn đề tiêu tán điện năng

⁹ Một cỗ máy phức tạp được đặt theo tên của một họa sĩ biếm họa người Anh, người đã mô tả những thứ kỳ lạ và phức tạp vô lý cho các công việc hàng ngày như nướng bơ.¹⁰ Trong thời kỳ hậu chiến, Turing muốn sử dụng những máy tính này để nghiên cứu AI - ví dụ, ông đã tạo ra phác thảo của chương trình cờ vua đầu tiên (Turing et al., 1953) - nhưng chính phủ Anh đã ngăn chặn nghiên cứu này.

để bắt đầu nhân số lỗi CPU thay vì tốc độ xung nhịp Kỳ vọng hiện tại là sự gia tăng chức năng trong tương lai sẽ đến từ sự song song lớn - một sự hội tụ kỳ lạ với các đặc tính của não. Chúng ta cũng thấy các thiết kế phần cứng mới dựa trên ý tưởng rằng khi đối phó với một thế giới không chắc chắn, chúng ta không cần độ chính xác 64 bit trong các con số của chúng ta; Chỉ cần 16 bit (như ở định dạng bfloat16) hoặc thậm chí 8 bit là đủ và sẽ cho phép xử lý nhanh hơn.

Chúng ta chỉ mới bắt đầu thấy phần cứng được điều chỉnh cho các ứng dụng AI, chẳng hạn như đơn vị xử lý đồ họa (GPU), đơn vị xử lý tensor (TPU) và công cụ quy mô wafer (WSE). Từ những năm 1960 đến khoảng năm 2012, lượng sức mạnh tính toán được sử dụng để đào tạo các ứng dụng học máy hàng đầu tuân theo định luật Moore. Bắt đầu từ năm 2012, mọi thứ đã thay đổi: từ năm 2012 đến năm 2018 đã tăng gấp 300.000 lần, tăng gấp đôi sau mỗi 100 ngày (Amodei và Hernandez, 2018). Một mô hình học máy mất cả ngày để đào tạo vào năm 2014 chỉ mất hai phút vào năm 2018 (Ying và cộng sự, 2018). Mặc dù nó chưa thực tế, điện toán lượng tử hứa hẹn về gia tốc lớn hơn nhiều đối với một số quan trọng

Điện toán lượng tử

các lớp con của thuật toán AI.

Tất nhiên, đã có các thiết bị tính toán trước máy tính điện tử. Các máy tự động sớm nhất, có niên đại từ thế kỷ 17, đã được thảo luận ở trang 24. Máy có thể lập trình đầu tiên là một khung cửi, được phát minh vào năm 1805 bởi Joseph Marie Jacquard (1752–1834), sử dụng các thẻ đục lỗ để lưu trữ các hướng dẫn cho mẫu được dệt.

Vào giữa thế kỷ 19, Charles Babbage (1792–1871) đã thiết kế hai máy tính toán, cả hai đều không được hoàn thành. Difference Engine được thiết kế để tính toán các bảng toán học cho các dự án kỹ thuật và khoa học. Cuối cùng nó đã được xây dựng và cho thấy hoạt động vào năm 1991 (Swade, 2000). Công cụ phân tích của Babbage tham vọng hơn nhiều: nó bao gồm bộ nhớ địa chỉ, các chương trình được lưu trữ dựa trên thẻ đục lỗ của Jacquard và các bước nhảy liên tục. Đây là cỗ máy đầu tiên có khả năng tính toán phổ quát.

Đồng nghiệp của Babbage, Ada Lovelace, con gái của nhà thơ Lord Byron, hiểu tiềm năng của nó, mô tả nó như "một suy nghĩ hoặc... một cỗ máy lý luận", một cỗ máy có khả năng suy luận về "tất cả các chủ thể trong vũ trụ" (Lovelace, 1843). Cô cũng dự đoán các chu kỳ cường điệu của AI, viết, "Nên đề phòng khả năng của những ý tưởng phóng đại có thể phát sinh do sức mạnh của Công cụ phân tích." Thật không may, máy móc của Babbage và Sideas của Lovelace phần lớn đã bị lãng quên.

AI cũng nợ phần mềm của khoa học máy tính, vốn đã cung cấp hệ điều hành, ngôn ngữ lập trình và các công cụ cần thiết để viết các chương trình hiện đại (và các bài báo về chúng). Nhưng đây là một lĩnh vực mà khoản nợ đã được trả nợ: công việc trong AI đã tạo ra nhiều ý tưởng đã quay trở lại khoa học máy tính chính thống, bao gồm chia sẻ thời gian, trình thông dịch tương tác, máy tính cá nhân với cửa sổ và chuột, môi trường phát triển nhanh, kiểu dữ liệu danh sách liên kết, quản lý lưu trữ tự động và các khái niệm chính của lập trình biểu tượng, chức năng, khai báo và hướng đối tượng.

1.2.7 Lý thuyết điều khiển và điều khiển học

- Làm thế nào các hiện vật có thể hoạt động dưới sự kiểm soát của chính chúng? Ktesibios của Alexandria (khoảng 250 trước Công nguyên) đã chế tạo cỗ máy tự điều khiển đầu tiên: đồng hồ nước với bộ điều khiển duy trì tốc độ dòng chảy không đổi. Phát minh này đã thay đổi định nghĩa về những gì một hiện vật có thể làm. Trước đây, chỉ có sinh vật sống mới có thể thay đổi hành vi của chúng để đáp ứng với những thay đổi của môi trường. Các ví dụ khác về kiểm soát phản hồi tự điều chỉnh

hệ thống bao gồm bộ điều khiển động cơ hơi nước, được tạo ra bởi James Watt (1736–1819), và bộ điều nhiệt, được phát minh bởi Cornelis Drebbel (1572–1633), người cũng đã phát minh ra tàu ngầm. James Clerk Maxwell (1868) khởi xướng lý thuyết toán học về hệ thống điều khiển.

Lý thuyết điều khiển

Một nhân vật trung tâm trong sự phát triển của lý thuyết điều khiển sau chiến tranh là Norbert Wiener

(1894–1964). Wiener là một nhà toán học xuất sắc, người đã làm việc với Bertrand Russell, trong số những người khác, trước khi phát triển mối quan tâm đến các hệ thống điều khiển sinh học và cơ học và mối liên hệ của chúng với nhận thức. Giống như Craik (người cũng sử dụng hệ thống điều khiển làm mô hình tâm lý), Wiener và các đồng nghiệp của ông là Arturo Rosenblueth và Julian Bigelow đã thách thức chính thống hành vi (Rosenblueth et al., 1943). Họ xem hành vi có mục đích phát sinh từ một cơ chế điều tiết cố gắng giảm thiểu "lỗi" - sự khác biệt giữa trạng thái hiện tại và trạng thái mục tiêu. Vào cuối những năm 1940, Wiener, cùng với Warren McCulloch, Walter Pitts và John von Neumann, đã tổ chức một loạt các hội nghị có ảnh hưởng nhằm khám phá các mô hình toán học và tính toán mới của nhận thức. Cuốn sách của Wiener *Cybernetics* (1948)

Cybernetics

trở thành một cuốn sách bán chạy nhất và đánh thức công chúng về khả năng của những cỗ máy thông minh nhân tạo.

Trong khi đó, ở Anh, W. Ross Ashby đi tiên phong trong những ý tưởng tương tự (Ashby, 1940). Ashby, Alan Turing, Grey Walter và những người khác đã thành lập Câu lạc bộ Tỷ lệ cho "những người có Wiener's ideas trước khi cuốn sách của Wiener xuất hiện". Thiết kế cho một bộ não của Ashby (1948, 1952) trình bày chi tiết ý tưởng của ông rằng trí thông minh có thể được tạo ra bằng cách sử dụng các thiết bị cân bằng nội môi có chứa

Cân bằng nội môi

các vòng phản hồi thích hợp để đạt được hành vi thích ứng ổn định.

Chức năng chi phí

Lý thuyết điều khiển hiện đại, đặc biệt là nhánh được gọi là điều khiển tối ưu ngẫu nhiên, có mục tiêu thiết kế các hệ thống giảm thiểu chức năng chi phí theo thời gian. Điều này gần giống với mô hình tiêu chuẩn của AI: thiết kế các hệ thống hoạt động tối ưu. Vậy tại sao AI và lý thuyết điều khiển lại là hai lĩnh vực khác nhau, mặc dù có mối liên hệ chặt chẽ giữa những người sáng lập chúng? Câu trả lời nằm ở sự kết hợp chặt chẽ giữa các kỹ thuật toán học quen thuộc với những người tham gia và các tập hợp các vấn đề tương ứng được bao gồm trong mỗi thế giới quan. Giải tích và đại số ma trận, công cụ của lý thuyết điều khiển, cho phép các hệ thống có thể mô tả được bằng các tập hợp cố định của các biến liên tục, trong khi AI được thành lập một phần như một cách để thoát khỏi những hạn chế nhận thức này. Các công cụ suy luận và tính toán logic cho phép các nhà nghiên cứu AI xem xét các vấn đề như ngôn ngữ, tầm nhìn và lập kế hoạch biểu tượng hoàn toàn nằm ngoài phạm vi của nhà lý thuyết điều khiển.

1.2.8 Ngôn ngữ học

• Ngôn ngữ liên quan đến suy nghĩ như thế nào? Năm 1957, BF Skinner xuất bản *Hành vi bằng lời nói*. Đây là một bản tóm tắt toàn diện, chi tiết về cách tiếp cận hành vi học ngôn ngữ, được viết bởi chuyên gia hàng đầu trong lĩnh vực này. Nhưng kỳ lạ thay, một bài đánh giá về cuốn sách đã trở nên nổi tiếng như chính cuốn sách, và gần như giết chết sự quan tâm đến chủ nghĩa hành vi. Tác giả của bài đánh giá là nhà ngôn ngữ học Noam Chomsky, người vừa xuất bản một cuốn sách về lý thuyết của riêng mình, *Cấu trúc cú pháp*. Chomsky chỉ ra rằng lý thuyết hành vi không đề cập đến khái niệm sáng tạo trong ngôn ngữ - nó không giải thích làm thế nào trẻ em có thể hiểu và tạo ra những câu mà chúng chưa bao giờ nghe trước đây. Lý thuyết của Chomsky - dựa trên các mô hình cú pháp quay trở lại nhà ngôn ngữ học Ấn Độ Panini (khoảng 350 trước Công nguyên) - có thể giải thích điều này, và không giống như các lý thuyết trước đây, nó đủ chính thức để về nguyên tắc nó có thể được lập trình. Ngôn ngữ học hiện đại và AI, sau đó, được "sinh ra" vào cùng một thời điểm, và lớn lên, giao nhau trong một lĩnh vực lại được gọi là ngôn ngữ học tính toán hoặc ngôn ngữ tự nhiên

Xử lý. Vấn đề hiểu ngôn ngữ hóa ra phức tạp hơn đáng kể so với năm 1957. Hiểu ngôn ngữ đòi hỏi sự hiểu biết về chủ đề và ngữ cảnh, không chỉ hiểu cấu trúc của câu. Điều này có vẻ hiển nhiên, nhưng nó không được đánh giá cao rộng rãi cho đến những năm 1960. Phần lớn công trình ban đầu trong đại diện tri thức (nghiên cứu về cách đưa kiến thức vào một hình thức mà máy tính có thể lý luận) gắn liền với ngôn ngữ và được thông báo bởi nghiên cứu về ngôn ngữ học, được kết nối với nhiều thập kỷ công việc về phân tích triết học của ngôn ngữ.

1.3 Lịch sử của trí tuệ nhân tạo

Một cách nhanh chóng để tóm tắt các cột mốc quan trọng trong lịch sử AI là liệt kê những người chiến thắng Giải thưởng Turing: Marvin Minsky (1969) và John McCarthy (1971) để xác định nền tảng của lĩnh vực dựa trên đại diện và lý luận; Allen Newell và Herbert Simon (1975) cho các mô hình biểu tượng về giải quyết vấn đề và nhận thức của con người; Ed Feigenbaum và Raj Reddy (1994) phát triển các hệ thống chuyên gia mã hóa kiến thức của con người để giải quyết các vấn đề trong thế giới thực; Judea Pearl (2011) vì đã phát triển các kỹ thuật suy luận xác suất đối phó với sự không chắc chắn theo cách có nguyên tắc; và cuối cùng là Yoshua Bengio, Geoffrey Hinton và Yann LeCun (2019) vì đã biến "học sâu" (mạng nơ-ron nhiều lớp) trở thành một phần quan trọng của điện toán hiện đại. Phần còn lại của phần này đi vào chi tiết hơn về từng giai đoạn của lịch sử AI.

1.3.1 Sự ra đời của trí tuệ nhân tạo (1943–1956)

Công trình đầu tiên hiện được công nhận rộng rãi là AI được thực hiện bởi Warren McCulloch và Walter Pitts (1943). Lấy cảm hứng từ công trình mô hình toán học của cổ vấn Nicolas Rashevsky của Pitts (1936, 1938), họ đã rút ra ba nguồn: kiến thức về sinh lý học cơ bản và chức năng của tế bào thần kinh trong não; một phân tích chính thức về logic mệnh đề do Russell and Whitehead; và lý thuyết tính toán của Turing. Họ đề xuất một mô hình tế bào thần kinh nhân tạo trong đó mỗi tế bào thần kinh được đặc trưng là "bật" hoặc "tắt", với việc chuyển sang "bật" xảy ra để đáp ứng với sự kích thích của đủ số lượng tế bào thần kinh lân cận. Trạng thái của một tế bào thần kinh được quan niệm là "tương đương trên thực tế với một mệnh đề đề xuất kích thích thích hợp". Ví dụ, họ chỉ ra rằng bất kỳ hàm có thể tính toán nào cũng có thể được tính toán bởi một số mạng lưới các tế bào thần kinh được kết nối và tất cả các liên kết logic (AND, OR, NOT, v.v.) có thể được thực hiện bởi các cấu trúc mạng đơn giản. McCulloch và Pitts cũng gợi ý rằng các mạng được xác định phù hợp có thể học hỏi. Donald Hebb (1949) đã chứng minh một quy tắc cập nhật đơn giản để sửa đổi cường độ kết nối giữa các tế bào thần kinh. Quy tắc của ông, bây giờ được gọi là học tập Hebbian, vẫn là một mô hình có ảnh hưởng cho đến ngày nay.

Học Hebbian

Hai sinh viên đại học tại Harvard, Marvin Minsky (1927–2016) và Dean Edmonds, đã chế tạo máy tính mạng nơ-ron đầu tiên vào năm 1950. SNARC, như nó được gọi, đã sử dụng 3000 ống chân không và một cơ chế phi công tự động dư thừa từ một máy bay ném bom B-24 để mô phỏng một mạng lưới gồm 40 tế bào thần kinh. Sau đó, tại Princeton, Minsky nghiên cứu tính toán phổ quát trong mạng thần kinh. Ủy ban tiến sĩ của ông đã hoài nghi về việc liệu loại công việc này có nên được coi là toán học hay không, nhưng von Neumann được cho là đã nói, "Nếu không phải bây giờ, nó sẽ là một ngày nào đó."

Có một số ví dụ khác về công việc ban đầu có thể được mô tả là AI, bao gồm hai chương trình chơi cờ được phát triển độc lập vào năm 1952 bởi Christopher Strachey tại Đại học Manchester và Arthur Samuel tại IBM. Tuy nhiên, tầm nhìn của Alan Turing là có ảnh hưởng nhất. Ông đã có các bài giảng về chủ đề này ngay từ năm 1947 tại Hiệp hội Toán học London và nêu rõ một chương trình nghị sự thuyết phục trong bài báo năm 1950 của mình "Com-

đặt Máy móc và Trí tuệ." Trong đó, ông giới thiệu bài kiểm tra Turing, học máy, thuật toán di truyền và học tăng cường. Ông đã giải quyết nhiều phản đối được nêu ra đối với khả năng của AI, như được mô tả trong Chương 28. Ông cũng gợi ý rằng sẽ dễ dàng hơn để tạo ra AI ở cấp độ con người bằng cách phát triển các thuật toán học tập và sau đó dạy máy hơn là bằng cách lập trình trí thông minh của nó bằng tay. Trong các bài giảng tiếp theo, ông cảnh báo rằng đạt được mục tiêu này có thể không phải là điều tốt nhất cho loài người.

Năm 1955, John McCarthy của Đại học Dartmouth đã thuyết phục Minsky, Claude Shannon và Nathaniel Rochester giúp ông tập hợp các nhà nghiên cứu Hoa Kỳ quan tâm đến máy tự động, mạng lưới thần kinh và nghiên cứu về trí thông minh. Họ đã tổ chức một hội thảo kéo dài hai tháng tại Dartmouth vào mùa hè năm 1956. Tổng cộng có 10 người tham dự, bao gồm Allen Newell và Herbert Simon từ Carnegie Tech, 11 Trenchard More từ Princeton, Arthur Samuel từ IBM, Ray Solomonoff và Oliver Selfridge từ MIT. Đề xuất nêu rõ:12

Chúng tôi đề xuất rằng một nghiên cứu 2 tháng, 10 người về trí tuệ nhân tạo được thực hiện trong mùa hè năm 1956 tại Đại học Dartmouth ở Hanover, New Hampshire. Nghiên cứu được tiến hành trên cơ sở phỏng đoán rằng mọi khía cạnh của học tập hoặc bất kỳ đặc điểm nào khác của trí thông minh về nguyên tắc có thể được mô tả chính xác đến mức một cỗ máy có thể được tạo ra để mô phỏng nó. Một nỗ lực sẽ được thực hiện để tìm ra cách làm cho máy móc sử dụng ngôn ngữ, hình thành các khái niệm trừu tượng và khái niệm, giải quyết các loại vấn đề hiện dành cho con người và cải thiện bản thân. Chúng tôi nghĩ rằng một tiến bộ đáng kể có thể được thực hiện trong một hoặc nhiều vấn đề này nếu một nhóm các nhà khoa học được lựa chọn cẩn thận làm việc cùng nhau trong một mùa hè. Bất chấp dự đoán lạc quan này, hội thảo Dartmouth đã không dẫn đến bất kỳ đột phá nào. Newell và Simon đã trình bày có lẽ là công trình trưởng thành nhất, một hệ thống chứng minh định lý toán học được gọi là Nhà lý thuyết logic (LT). Simon tuyên bố, "Chúng tôi đã phát minh ra một chương trình máy tính có khả năng suy nghĩ phi số, và do đó giải quyết vấn đề tâm trí và cơ thể đáng kính." Ngay sau hội thảo, chương trình đã có thể chứng minh hầu hết các định lý trong Chương 2 của Principia Mathematica của Russell và Whitehead. Russell được cho là đã mất cảm hứng khi được thông báo rằng LT đã đưa ra một chứng minh cho một định lý ngắn hơn định lý trong Principia. Các biên tập viên của Tạp chí Logic Biểu tượng ít ấn tượng hơn; họ đã bác bỏ một bài báo do Newell, Simon và Logic Theorist đồng tác giả.

1.3.2 Nhiệt huyết ban đầu, kỳ vọng lớn (1952–1969)

Giới trí thức của những năm 1950, nói chung, thích tin rằng "một cỗ máy không bao giờ có thể làm được X". (Xem Chương 28 để biết một danh sách dài các X do Turing thu thập.) Các nhà nghiên cứu AI đã phản ứng một cách tự nhiên bằng cách chứng minh hết X này đến X khác. Họ đặc biệt tập trung vào các nhiệm vụ được coi là dấu hiệu của trí thông minh ở con người, bao gồm trò chơi, câu đố, toán học và bài kiểm tra IQ. John McCarthy gọi thời kỳ này là kỷ nguyên "Nhìn kia, mẹ, không có tay!".

11 Bây giờ là Đại học Carnegie Mellon (CMU).12 Đây là cách sử dụng chính thức đầu tiên của thuật ngữ trí tuệ nhân tạo của McCarthy. Có lẽ "tính hợp lý tính toán" sẽ chính xác hơn và ít đe dọa hơn, nhưng "AI" đã bị mắc kẹt. Tại lễ kỷ niệm 50 năm hội nghị Dartmouth, McCarthy tuyên bố rằng ông đã chống lại các thuật ngữ "máy tính" hoặc "tính toán" để tôn trọng Norbert Wiener, người đang quảng bá các thiết bị điều khiển học tương tự hơn là máy tính kỹ thuật số.13 Newell và Simon cũng phát minh ra một ngôn ngữ xử lý danh sách, IPL, để viết LT. Họ không có trình biên dịch và dịch nó thành mã máy bằng tay. Để tránh lỗi, họ làm việc song song, gọi các số nhị phân cho nhau khi họ viết từng lệnh để đảm bảo rằng họ đồng ý.

Newell và Simon tiếp nối thành công của họ với LT với General Problem Solver, hoặc GPS. Không giống như LT, chương trình này được thiết kế ngay từ đầu để bắt chước các giao thức giải quyết vấn đề của con người. Trong lớp câu đố hạn chế mà nó có thể xử lý, hóa ra thứ tự mà chương trình xem xét các mục tiêu phụ và các hành động có thể xảy ra tương tự như thứ tự mà con người tiếp cận cùng một vấn đề. Do đó, GPS có lẽ là chương trình đầu tiên thể hiện cách tiếp cận "suy nghĩ của con người". Sự thành công của GPS và các chương trình tiếp theo như các mô hình nhận thức đã khiến Newell và Simon (1976) xây dựng giả thuyết hệ thống ký hiệu vật lý nổi tiếng, trong đó nói rằng "một hệ thống biểu tượng vật lý có cần thiết và suf-

Hệ thống biểu tượng vật lý

phương tiện thông minh cho hành động thông minh nói chung." Ý của họ là bất kỳ hệ thống nào (con người hoặc máy móc) thể hiện trí thông minh đều phải hoạt động bằng cách thao túng các cấu trúc dữ liệu bao gồm các ký hiệu. Sau này chúng ta sẽ thấy rằng giả thuyết này đã bị thách thức từ nhiều hướng.

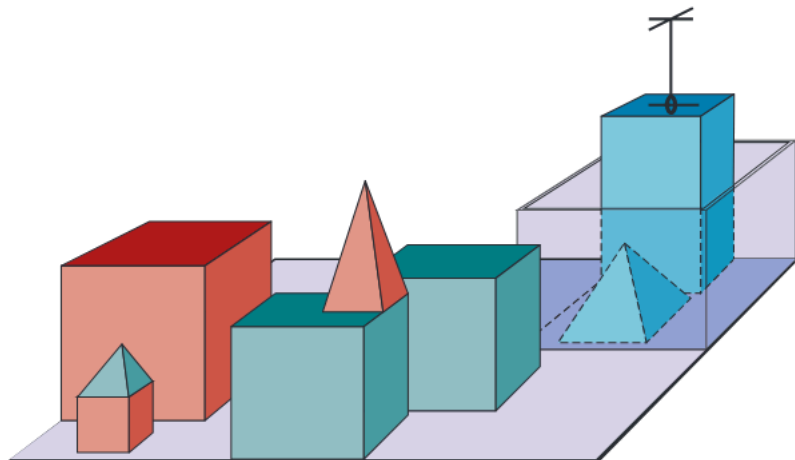
Tại IBM, Nathaniel Rochester và các đồng nghiệp của ông đã tạo ra một số chương trình AI đầu tiên. Herbert Gelernter (1959) đã xây dựng Trình chứng minh Định lý Hình học, có thể chứng minh các định lý mà nhiều sinh viên toán học sẽ thấy khá khó khăn. Công trình này là một tiền đề của những người chứng minh định lý toán học hiện đại.

Trong tất cả các công việc thăm dò được thực hiện trong thời kỳ này, có lẽ có ảnh hưởng nhất về lâu dài là của Arthur Samuel về cờ đam (bàn nháp). Sử dụng các phương pháp mà bây giờ chúng ta gọi là học tăng cường (xem Chương 23), các chương trình của Samuel đã học cách chơi ở cấp độ mạnh mẽ. Do đó, ông bác bỏ ý tưởng rằng máy tính chỉ có thể làm những gì chúng được yêu cầu: chương trình của ông nhanh chóng học cách chơi một trò chơi tốt hơn người tạo ra nó. Chương trình đã được chiếu trên truyền hình vào năm 1956, tạo ấn tượng mạnh mẽ. Giống như Turing, Samuel gặp khó khăn trong việc tìm kiếm thời gian máy tính. Làm việc vào ban đêm, ông sử dụng những máy móc vẫn còn trên sàn thử nghiệm tại nhà máy sản xuất của IBM. Chương trình của Samuel là tiền thân của các hệ thống sau này như TD-GAMMON (Tesauro, 1992), là một trong những người chơi cờ thỏ cáo giỏi nhất thế giới và ALPHAGO (Silver et al., 2016), đã gây sốc cho thế giới khi đánh bại nhà vô địch thế giới con người tại cờ vây (xem Chương 6).

Năm 1958, John McCarthy đã có hai đóng góp quan trọng cho AI. Trong MIT AI Lab MemoNo. 1, ông đã định nghĩa ngôn ngữ cấp cao Lisp, sẽ trở thành chuyên gia AI thống trị. Nói nôm na

ngôn ngữ ngữ pháp trong 30 năm tiếp theo. Trong một bài báo có tựa đề Chương trình với lẽ thường, ông đã đưa ra một đề xuất khái niệm cho các hệ thống AI dựa trên kiến thức và lý luận. Bài báo mô tả Advice Taker, một chương trình giả định sẽ thể hiện kiến thức chung về thế giới và có thể sử dụng nó để đưa ra các kế hoạch hành động. Khái niệm này được minh họa bằng các tiên đề logic đơn giản đủ để tạo ra một kế hoạch lái xe đến sân bay. Chương trình cũng được thiết kế để chấp nhận các tiên đề mới trong quá trình hoạt động bình thường, do đó cho phép nó đạt được năng lực trong các lĩnh vực mới mà không cần được lập trình lại. Người nhận lời khuyên đã đưa ra các nguyên tắc trung tâm của đại diện tri thức và lý luận: rằng rất hữu ích khi có một đại diện chính thức, rõ ràng về thế giới và hoạt động của nó và có thể tạo ra sự biểu diễn đó bằng các quá trình suy diễn. Bài báo đã ảnh hưởng đến quá trình của AI và vẫn còn phù hợp cho đến ngày

Năm 1958 cũng đánh dấu năm Marvin Minsky chuyển đến MIT. Tuy nhiên, sự hợp tác ban đầu của ông với McCarthy không kéo dài. McCarthy nhấn mạnh sự đại diện và lý luận trong logic for-mal, trong khi Minsky quan tâm nhiều hơn đến việc làm cho các chương trình hoạt động và cuối cùng phát triển một quan điểm phản logic. Năm 1963, McCarthy bắt đầu phòng thí nghiệm AI tại Stanford. Kế hoạch sử dụng logic của ông để xây dựng Advice Taker cuối cùng đã được thúc đẩy bởi J. A. Robinson vào năm 1965 về phương pháp resolution (một thuật toán chứng minh định lý hoàn chỉnh cho bậc nhất



Hình 1.3 Một cảnh từ thế giới khối. SHRDLU (Winograd, 1972) vừa hoàn thành mệnh lệnh "Tìm một khối cao hơn khối bạn đang cầm và đặt nó vào hộp."

lôgic; xem Chương 9). Công việc tại Stanford nhấn mạnh các phương pháp mục đích chung để lý luận logic. Các ứng dụng của logic bao gồm hệ thống lập kế hoạch và trả lời câu hỏi của Cordell Green (Green, 1969b) và dự án robot Shakey tại Viện Nghiên cứu Stanford (SRI). Dự án thứ hai, được thảo luận thêm trong Chương 26, là dự án đầu tiên chứng minh sự tích hợp hoàn toàn của lý luận logic và hoạt động thể chất.

Tại MIT, Minsky giám sát một loạt sinh viên chọn những vấn đề hạn chế đường như đòi hỏi trí thông minh để giải quyết. Những miền giới hạn này được gọi là vi thế giới.

Chương trình SAINT của James Slagle (1963) đã có thể giải quyết các vấn đề tích hợp giải tích dạng đóng điển hình của các khóa học đại học năm nhất. Chương trình ANALOGY của Tom Evans (1968) đã giải quyết các vấn đề tương tự hình học xuất hiện trong các bài kiểm tra IQ. Chương trình STUDENT (1967) của Daniel Bobrow đã giải quyết các vấn đề câu chuyện đại số, chẳng hạn như sau:

Nếu số lượng khách hàng mà Tom nhận được gấp đôi bình phương của 20 phần trăm số lượng quảng cáo mà anh ấy chạy và số lượng quảng cáo anh ấy chạy là 45, thì số lượng khách hàng mà Tom nhận được là bao nhiêu?

Vi thế giới nổi tiếng nhất là thế giới khối, bao gồm một tập hợp các khối rắn đặt trên mặt bàn (hoặc thường xuyên hơn là mô phỏng mặt bàn), như trong Hình 1.3. Một nhiệm vụ điển hình trong thế giới này là sắp xếp lại các khối theo một cách nhất định, sử dụng bàn tay robot có thể nhặt từng khối một. Thế giới khối là nơi có dự án tầm nhìn của David Huffman (1971), công trình tầm nhìn và lan truyền ràng buộc của David Waltz (1975), lý thuyết học tập của Patrick Winston (1970), chương trình hiểu ngôn ngữ tự nhiên của Terry Winograd (1972) và người lập kế hoạch của Scott Fahlman (1974). Công việc ban đầu xây dựng trên mạng nơ-ron của McCulloch và Pitts cũng phát triển mạnh mẽ. Nghiên cứu của Shmuel Winograd và Jack Cowan (1963) cho thấy một số lượng lớn các yếu tố

Thế giới vi mô

Thế giới khối

có thể đại diện chung cho một khái niệm riêng lẻ, với sự gia tăng tương ứng về tính mạnh mẽ và song song. Phương pháp học tập của Hebb đã được Bernie Widrow nâng cao (Widrow and Hoff, 1960; Widrow, 1962), người đã gọi mạng lưới của mình là adalines, và bởi Frank Rosenblatt (1962) với các bộ phận của mình. Định lý hội tụ perceptron (Block et al., 1962) nói rằng thuật toán học có thể điều chỉnh cường độ kết nối của một perceptron để khớp với bất kỳ dữ liệu đầu vào nào, miễn là có sự trùng khớp như vậy.

1.3.3 Một liều lượng thực tế (1966–1973)

Ngay từ đầu, các nhà nghiên cứu AI đã không ngại đưa ra dự đoán về những thành công sắp tới của họ. Tuyên bố sau đây của Herbert Simon vào năm 1957 thường được trích dẫn:

Mục đích của tôi không phải là làm bạn ngạc nhiên hay sốc - nhưng cách đơn giản nhất tôi có thể tóm tắt là nói rằng bây giờ trên thế giới có những cỗ máy suy nghĩ, học hỏi và sáng tạo. Hơn nữa, khả năng của họ để làm những điều này sẽ tăng lên nhanh chóng cho đến khi – trong một tương lai có thể nhìn thấy – phạm vi các vấn đề mà họ có thể xử lý sẽ đồng mở rộng với phạm vi mà tâm trí con người đã được áp dụng.

Thuật ngữ "tương lai hữu hình" rất mơ hồ, nhưng Simon cũng đưa ra những dự đoán cụ thể hơn: rằng trong vòng 10 năm một máy tính sẽ là nhà vô địch cờ vua và một định lý toán học quan trọng sẽ được chứng minh bởi máy móc. Những dự đoán này đã trở thành sự thật (hoặc xấp xỉ đúng) trong vòng 40 năm thay vì 10 năm. Sự tự tin thái quá của Simon là do hiệu suất đầy hứa hẹn của các hệ thống AI ban đầu trên các ví dụ đơn giản. Tuy nhiên, trong hầu hết các trường hợp, các hệ thống ban đầu này đều thất bại trong các vấn đề khó khăn hơn.

Có hai lý do chính cho thất bại này. Đầu tiên là nhiều hệ thống AI ban đầu chủ yếu dựa trên "nội tâm có thông tin" về cách con người thực hiện một nhiệm vụ, thay vì dựa trên phân tích cần thận về nhiệm vụ, ý nghĩa của việc trở thành một giải pháp và những gì một thuật toán cần làm để tạo ra các giải pháp như vậy một cách đáng tin cậy.

Lý do thứ hai cho thất bại là thiếu đánh giá cao tính khó giải quyết của nhiều vấn đề mà AI đang cố gắng giải quyết. Hầu hết các hệ thống giải quyết vấn đề ban đầu hoạt động bằng cách thử các kết hợp các bước khác nhau cho đến khi tìm ra giải pháp. Strat-egy này hoạt động ban đầu vì các vi thể giới chứa rất ít đối tượng và do đó rất ít hành động có thể xảy ra và trình tự nghiệm rất ngắn. Trước khi lý thuyết về tính toán phức tạp được phát triển, người ta cho rằng "mở rộng" cho các vấn đề lớn hơn chỉ đơn giản là vấn đề phản ứng nhanh hơn và bộ nhớ lớn hơn. Ví dụ, sự lạc quan đi kèm với sự phát triển của việc chứng minh định lý phân giải, đã sớm bị giảm bớt khi các nhà nghiên cứu thất bại trong việc chứng minh các định lý liên quan đến hơn vài chục sự kiện. Thực tế là một chương trình có thể Jfind một giải pháp về nguyên tắc không có nghĩa là chương trình chứa bất kỳ cơ chế nào

cần thiết để tìm thấy nó trong thực tế.

Ảo tưởng về sức mạnh tính toán không giới hạn không chỉ giới hạn trong các chương trình giải quyết vấn đề. Các thí nghiệm ban đầu trong tiến hóa máy móc (nay được gọi là lập trình di truyền) (Fried-

Sự phát triển của máy móc

Berg, 1958; Friedberg et al., 1959) dựa trên niềm tin chắc chắn đúng đắn rằng bằng cách tạo ra một loạt các đột biến nhỏ thích hợp cho một chương trình mã máy, người ta có thể tạo ra một chương trình với hiệu suất tốt cho bất kỳ nhiệm vụ cụ thể nào. Ý tưởng là thử các đột biến ngẫu nhiên với một quá trình chọn lọc để bảo tồn các đột biến có vẻ hữu ích. Bất chấp hàng ngàn giờ CPU thời gian, hầu như không có tiến bộ nào được chứng minh.

Thất bại trong việc nắm bắt được "vụ nổ tổ hợp" là một trong những phê bình chính của AI có trong báo cáo Lighthill (Lighthill, 1973), tạo thành cơ sở cho

quyết định của chính phủ Anh chấm dứt hỗ trợ nghiên cứu AI ở tất cả trừ hai trường đại học. (Truyền thống truyền miệng về ra một bức tranh hơi khác và đầy màu sắc hơn, với tham vọng chính trị và thù địch cá nhân mà mô tả của nó nằm ngoài vấn đề.)

Khó khăn thứ ba phát sinh do một số hạn chế cơ bản về các cấu trúc cơ bản được sử dụng để tạo ra hành vi thông minh. Ví dụ, cuốn sách Perceptrons (1969) của Minsky và Papert đã chứng minh rằng, mặc dù perceptron (một dạng đơn giản của mạng nơ-ron) có thể được chứng minh để học bất cứ thứ gì chúng có khả năng đại diện, nhưng chúng có thể đại diện rất ít. Đặc biệt, một perceptron hai đầu vào không thể được đào tạo để nhận ra khi hai đầu vào của nó khác nhau. Mặc dù kết quả của họ không áp dụng cho các mạng lưới đa lớp phức tạp hơn, nhưng tài trợ nghiên cứu cho nghiên cứu mạng lưới thần kinh nhanh chóng giảm xuống gần như không có. Trớ trêu thay, các thuật toán học lan truyền mới gây ra sự hồi sinh to lớn trong nghiên cứu mạng thần kinh vào cuối những năm 1980 và một lần nữa vào những năm 2010 đã được phát triển trong các bối cảnh khác vào đầu những năm 1960 (Kelley, 1960; Bryson, 1962).

1.3.4 Hệ thống chuyên gia (1969–1986)

Bức tranh giải quyết vấn đề đã nảy sinh trong thập kỷ đầu tiên của nghiên cứu AI là một cơ chế tìm kiếm có mục đích chung cố gắng xâu chuỗi các bước lý luận cơ bản lại với nhau để tìm ra giải pháp hoàn chỉnh. Những cách tiếp cận như vậy được gọi là phương pháp yếu vi, mặc dù

Phương pháp yếu

Nói chung, chúng không mở rộng quy mô cho các trường hợp vấn đề lớn hoặc khó. Giải pháp thay thế cho các phương pháp yếu là sử dụng kiến thức mạnh mẽ hơn, theo lĩnh vực cụ thể cho phép các bước lý luận lớn hơn và có thể dễ dàng xử lý các trường hợp thường xảy ra trong các lĩnh vực chuyên môn hẹp. Người ta có thể nói rằng để giải quyết một vấn đề khó, bạn gần như phải biết câu trả lời.

Chương trình DENDRAL (Buchanan et al., 1969) là một ví dụ ban đầu về cách tiếp cận này. Nó được phát triển tại Stanford, nơi Ed Feigenbaum (cựu học trò của Herbert Simon), Bruce Buchanan (một nhà triết học trở thành nhà khoa học máy tính) và Joshua Lederberg (một nhà di truyền học Nobellaureate) hợp tác để giải quyết vấn đề suy luận cấu trúc phân tử từ thông tin được cung cấp bởi khối phổ kế. Đầu vào cho chương trình bao gồm công thức ele-mentary của phân tử (ví dụ: $C_6H_{13}NO_2$) và phổ khối lượng cho khối lượng của các mảnh khác nhau của phân tử được tạo ra khi nó bị bắn phá bởi một chùm electron. Ví dụ, phổ khối có thể chứa đỉnh ở $m = 15$, tương ứng với khối lượng của một mảnh methyl (CH_3).

Phiên bản ngây thơ của chương trình đã tạo ra tất cả các cấu trúc có thể phù hợp với công thức, và sau đó dự đoán phổ khối nào sẽ được quan sát cho mỗi cấu trúc, so sánh điều này với phổ thực tế. Như người ta có thể mong đợi, điều này là khó giải quyết đối với ngay cả những phân tử có kích thước vừa phải. Các nhà nghiên cứu DENDRAL đã tham khảo ý kiến của các nhà hóa học phân tích và phát hiện ra rằng họ làm việc bằng cách tìm kiếm các mô hình nổi tiếng của các đỉnh trong quang phổ gợi ý các cấu trúc phụ phổ biến trong phân tử. Ví dụ, quy tắc sau đây được sử dụng để nhận dạng một nhóm con xeton ($C = O$) (nặng 28):

nếu M là khối lượng của toàn bộ phân tử và có hai đỉnh ở x_1 và x_2 sao cho
(a) $x_1 + x_2 = M + 28$; (b) $x_1 - 28$ là một đỉnh cao; (c) $x_2 - 28$ là một đỉnh cao; và (d) ít nhất một trong x_1 và x_2 là cao thì có một phân nhóm xeton.

Nhận ra rằng phân tử chứa một cấu trúc phụ cụ thể làm giảm số lượng ứng cử viên có thể làm giảm rất nhiều. Theo các tác giả của nó, DENDRAL rất mạnh mẽ vì vật liệu có kiến thức liên quan về khối phổ không phải dưới dạng nguyên tắc đầu tiên mà

trong "công thức nấu ăn" hiệu quả (Feigenbaum và cộng sự, 1971). Ý nghĩa của DENDRAL là nó là hệ thống chuyên sâu tri thức thành công đầu tiên: chuyên môn của nó bắt nguồn từ một số lượng lớn các quy tắc có mục đích đặc biệt. Năm 1971, Feigenbaum và những người khác tại Stanford bắt đầu Dự án Lập trình Heuristic (HPP) để điều tra mức độ mà phương pháp mới của các hệ thống chuyên gia có thể được áp dụng cho các lĩnh vực khác.

Hệ thống chuyên gia

Nỗ lực lớn tiếp theo là hệ thống MYCIN để chẩn đoán nhiễm trùng máu. Với khoảng 450 quy tắc, MYCIN có thể hoạt động tốt như một số chuyên gia, và tốt hơn đáng kể so với các bác sĩ cấp dưới. Nó cũng chứa hai điểm khác biệt chính so với DENDRAL. Thứ nhất, không giống như các quy tắc DENDRAL, không có mô hình lý thuyết tổng quát nào tồn tại mà từ đó các quy tắc MYCIN có thể suy ra. Chúng phải được thu thập từ các cuộc phỏng vấn rộng rãi của các chuyên gia. Thứ hai, các quy tắc phải phản ánh sự không chắc chắn liên quan đến kiến thức y tế. MYCIN kết hợp tính toán của sự không chắc chắn được gọi là các yếu tố chắc chắn (xem Chương 13), dường như (vào thời điểm đó)

Yếu tố chắc chắn

để phù hợp với cách các bác sĩ đánh giá tác động của bằng chứng đối với chẩn đoán.

Hệ thống chuyên gia thương mại thành công đầu tiên, R1, bắt đầu hoạt động tại Tập đoàn Thiết bị Kỹ thuật số (McDermott, 1982). Chương trình đã giúp cấu hình các đơn đặt hàng cho các hệ thống computer mới; Đến năm 1986, nó đã tiết kiệm cho công ty khoảng 40 triệu đô la mỗi năm. Đến năm 1988, nhóm AI của DEC đã triển khai 40 hệ thống chuyên gia, với nhiều hơn nữa đang được triển khai. DuPont có 100 chiếc không được sử dụng và 500 chiếc đang được phát triển. Gần như mọi tập đoàn lớn của Mỹ đều có nhóm AI của riêng mình và đang sử dụng hoặc điều tra các hệ thống chuyên

gia quan trọng của kiến thức miền cũng rõ ràng trong lĩnh vực hiểu ngôn ngữ tự nhiên.

Bất chấp sự thành công của hệ thống SHRDLU của Winograd, các phương pháp của nó không có xu hướng thực hiện các nhiệm vụ tổng quát hơn: đối với các vấn đề như giải quyết sự mơ hồ, nó sử dụng các quy tắc đơn giản dựa trên phạm vi nhỏ của thế giới khối.

Một số nhà nghiên cứu, bao gồm Eugene Charniak tại MIT và Roger Schank tại Yale, cho rằng sự hiểu biết ngôn ngữ mạnh mẽ sẽ đòi hỏi kiến thức chung về thế giới và một phương pháp chung để sử dụng kiến thức đó. (Schank đi xa hơn, tuyên bố, "Không có cái gọi là cú pháp", điều này khiến nhiều nhà ngôn ngữ học khó chịu nhưng đã bắt đầu một cuộc thảo luận hữu ích.) Schank và các sinh viên của ông đã xây dựng một loạt các chương trình (Schank và Abelson, 1977; Wilensky, 1978; Schank và Riesbeck, 1981) rằng tất cả đều có nhiệm vụ hiểu ngôn ngữ tự nhiên. Tuy nhiên, sự nhấn mạnh ít hơn vào bản thân ngôn ngữ và nhiều hơn vào các vấn đề thể hiện và lý luận với kiến thức cần thiết để hiểu ngôn ngữ.

Sự phát triển rộng rãi của các ứng dụng cho các vấn đề trong thế giới thực đã dẫn đến sự phát triển của một loạt các công cụ biểu diễn và lý luận. Một số dựa trên logic - ví dụ, ngôn ngữ Prolog trở nên phổ biến ở châu Âu và Nhật Bản, và gia đình PLANNER ở Hoa Kỳ.

Những người khác, theo ý tưởng của Minsky về khung (1975), đã áp dụng một cấu trúc hơn

Khung

tiếp cận, tập hợp các sự kiện về các loại đối tượng và sự kiện cụ thể và sắp xếp các loại thành một hệ thống phân cấp phân loại lớn tương tự như phân loại sinh học.

Năm 1981, chính phủ Nhật Bản công bố dự án "Thế hệ thứ năm", một kế hoạch 10 năm để xây dựng các máy tính thông minh, song song hàng loạt chạy Prolog. Ngân sách đã vượt quá 1,3 tỷ đô la tiền ngày nay. Đáp lại, Hoa Kỳ đã thành lập Tập đoàn Công nghệ Máy tính và Vi điện tử (MCC), một tập đoàn được thiết kế để đảm bảo khả năng cạnh tranh quốc gia. Trong cả hai trường hợp, AI là một phần của nỗ lực rộng lớn, bao gồm thiết kế chip và nghiên cứu giao diện con người. Tại Anh, báo cáo của Alvey đã khởi phục nguồn tài trợ bị loại bỏ bởi báo cáo Lighthill. Tuy nhiên, không có dự án nào trong số này từng đạt được các mục tiêu đầy tham vọng về khả năng AI mới hoặc tác động kinh tế.

Nhìn chung, ngành công nghiệp AI đã bùng nổ từ vài triệu đô la vào năm 1980 lên hàng tỷ đô la vào năm 1988, bao gồm hàng trăm công ty xây dựng hệ thống chuyên gia, hệ thống thị giác, rô bốt và phần mềm và phần cứng chuyên dụng cho những mục đích này.

Ngay sau đó là một giai đoạn được gọi là "mùa đông AI", trong đó nhiều công ty đã thất bại khi họ không thực hiện được những lời hứa xa hoa. Hóa ra rất khó để xây dựng và duy trì các hệ thống chuyên gia cho các lĩnh vực phức tạp, một phần vì các phương pháp lý luận được sử dụng bởi các hệ thống đã bị hỏng khi đối mặt với sự không chắc chắn và một phần vì các hệ thống không thể học hỏi từ kinh nghiệm.

1.3.5 Sự trở lại của mạng nơ-ron (1986–nay)

Vào giữa những năm 1980, ít nhất bốn nhóm khác nhau đã phát minh lại thuật toán học lan truyền ngược được phát triển lần đầu tiên vào đầu những năm 1960. Thuật toán đã được áp dụng cho nhiều vấn đề học tập trong khoa học máy tính và tâm lý học, và sự phổ biến rộng rãi của các re-sults trong bộ sưu tập Xử lý phân tán song song (Rumelhart và McClelland, 1986) đã gây ra sự phấn khích lớn.

Người theo chủ nghĩa kết nối Những mô hình được gọi là kết nối này được một số người coi là đối thủ cạnh tranh trực tiếp để các mô hình biểu tượng được thúc đẩy bởi Newell và Simon và theo cách tiếp cận logic của Mc-Carthy và những người khác. Có vẻ rõ ràng rằng ở một mức độ nào đó, con người thao túng các biểu tượng - trên thực tế, cuốn sách *The Symbolic Species* (1997) của nhà nhân chủng học Terrence Deacon cho rằng đây là đặc điểm xác định của con người. Chống lại điều này, Geoff Hinton, một nhân vật hàng đầu trong sự hồi sinh của mạng nơ-ron trong những năm 1980 và 2010, đã mô tả các biểu tượng là "ete phát sáng của AI" - một tham chiếu đến môi trường không tồn tại mà qua đó nhiều nhà vật lý thế kỷ 19 tin rằng sóng điện từ lan truyền. Chắc chắn, nhiều khái niệm mà chúng ta đặt tên bằng ngôn ngữ, khi kiểm tra kỹ hơn, không có được loại điều kiện cần thiết và đủ được xác định một cách hợp lý mà các nhà nghiên cứu AI ban đầu hy vọng sẽ nắm bắt được ở dạng ax-iomatic. Có thể các mô hình kết nối hình thành các khái niệm nội bộ theo một cách trôi chảy và không chính xác hơn, phù hợp hơn với sự lộn xộn của thế giới thực. Họ cũng có khả năng học hỏi từ các ví dụ — họ có thể so sánh giá trị đầu ra dự đoán của họ với giá trị thực của một vấn đề và sửa đổi các thông số của họ để giảm sự khác biệt, làm cho họ có nhiều khả năng hoạt động tốt trên các ví dụ trong tương lai.

1.3.6 Lý luận xác suất và học máy (1987–nay)

Sự giong của các hệ thống chuyên gia đã dẫn đến một cách tiếp cận mới, khoa học hơn kết hợp xác suất hơn là logic Boolean, học máy thay vì mã hóa tay, và kết quả thực nghiệm hơn là tuyên bố triết học.¹⁴ Nó trở nên phổ biến hơn để xây dựng dựa trên các lý thuyết hiện có hơn là đề xuất những lý thuyết hoàn toàn mới, để dựa trên các định lý nghiêm ngặt hoặc phương pháp kinh nghiệm vững chắc (Cohen, 1995) hơn là trực giác, và thể hiện sự liên quan đến các ứng dụng trong thế giới thực hơn là các ví dụ về đồ chơi.

Các bộ bài toán điểm chuẩn được chia sẽ đã trở thành tiêu chuẩn để chứng minh sự tiến bộ, bao gồm kho lưu trữ UC Irvine cho các bộ dữ liệu học máy, Cạnh tranh Lập kế hoạch Quốc tế-

¹⁴ Một số người đã mô tả sự thay đổi này như một chiến thắng của những người gọn gàng - những người nghĩ rằng các lý thuyết AI nên dựa trên sự nghiêm ngặt về toán học - trước những kẻ nhếch nhác - những người thả thả nhiều ý tưởng, viết một số chương trình và sau đó đánh giá những gì dường như đang hoạt động. Cả hai cách tiếp cận đều quan trọng. Sự chuyển hướng sang gọn gàng ngụ ý rằng lĩnh vực này đã đạt đến mức độ ổn định và trưởng thành. Sự nhấn mạnh hiện tại vào học sâu có thể đại diện cho sự hồi sinh của những kẻ nhếch nhác.

tion cho các thuật toán lập kế hoạch, kho dữ liệu LibriSpeech để nhận dạng giọng nói, bộ dữ liệu MNIST để nhận dạng chữ số viết tay, ImageNet và COCO để nhận dạng đối tượng hình ảnh, SQUAD để trả lời câu hỏi ngôn ngữ tự nhiên, cuộc thi WMT cho dịch máy và Cuộc thi SAT Quốc tế cho các giải khả năng thông minh

Đã trở thành một phần như một cuộc nổi loạn chống lại những hạn chế của các lĩnh vực hiện có như lý thuyết điều khiển và thống kê, nhưng trong giai đoạn này, nó đã chấp nhận kết quả tích cực của các lĩnh vực đó. Như David McAllester (1998) đã nói:

Trong thời kỳ đầu của AI, có vẻ hợp lý rằng các hình thức tính toán biểu tượng mới, ví dụ: khung và mạng ngữ nghĩa, đã làm cho phần lớn lý thuyết cổ điển trở nên lỗi thời. Điều này dẫn đến một hình thức biệt lập trong đó AI phần lớn tách biệt với phần còn lại của khoa học máy tính. Chủ nghĩa cô lập này hiện đang bị từ bỏ. Có một sự công nhận rằng học máy không nên bị cô lập khỏi lý thuyết thông tin, rằng lý luận không chắc chắn không nên được cô lập khỏi mô hình ngẫu nhiên, rằng tìm kiếm không nên được cô lập khỏi tối ưu hóa và kiểm soát cổ điển, và rằng suy luận tự động không nên bị cô lập khỏi các phương pháp chính thức và phân tích tĩnh.

Lĩnh vực nhận dạng giọng nói minh họa mô hình. Vào những năm 1970, một loạt các kiến trúc và cách tiếp cận khác nhau đã được thử nghiệm. Nhiều trong số này khá đặc biệt và mong manh, và chỉ làm việc trên một vài ví dụ được lựa chọn cẩn thận. Vào những năm 1980, các phương pháp tiếp cận sử dụng các mô hình Markov ẩn (HMM) đã chiếm ưu thế trong khu vực. Hai khía cạnh của HMM có liên quan.

Mô hình Markov ẩn

Đầu tiên, chúng dựa trên một lý thuyết toán học nghiêm ngặt. Điều này cho phép các nhà nghiên cứu ngôn ngữ xây dựng dựa trên nhiều thập kỷ kết quả toán học được phát triển trong các lĩnh vực khác. Thứ hai, chúng được tạo ra bởi một quá trình đào tạo trên một kho dữ liệu giọng nói thực lớn. Điều này đảm bảo rằng hiệu suất là mạnh mẽ và trong các bài kiểm tra mù nghiêm ngặt, HMM đã cải thiện điểm số của họ đều đặn. Kết quả là, công nghệ giọng nói và lĩnh vực nhận dạng ký tự viết tay liên quan đã chuyển đổi sang các ứng dụng công nghiệp và tiêu dùng rộng rãi. Lưu ý rằng không có tuyên bố khoa học nào rằng con người sử dụng HMM để nhận dạng lời nói; thay vào đó, HMM cung cấp một khuôn khổ toán học-ematic để hiểu và giải quyết vấn đề. Tuy nhiên, chúng ta sẽ thấy trong Phần 1.3.8 rằng deep learning đã làm đảo lộn câu chuyện thoải mái này.

Năm 1988 là một năm quan trọng đối với sự kết nối giữa AI và các lĩnh vực khác, bao gồm thống kê, nghiên cứu hoạt động, lý thuyết quyết định và lý thuyết kiểm soát. Judea Pearl (1988) Lý luận xác suất trong các hệ thống thông minh đã dẫn đến sự chấp nhận mới về xác suất và lý thuyết quyết định trong AI. Sự phát triển của Pearl về các mạng Bayes đã mang lại một sự nghiêm ngặt và hiệu quả

Mạng Bayes

chủ nghĩa hình thức để thể hiện kiến thức không chắc chắn cũng như các thuật toán thực tế để suy luận xác suất. Các chương 12, 13, 14, 15 và 18 bao gồm lĩnh vực này, ngoài những phát triển gần đây đã làm tăng đáng kể sức mạnh biểu hiện của chủ nghĩa hình thức xác suất; Chương 21 mô tả các phương pháp học mạng Bayes và các mô hình liên quan từ dữ liệu.

Đóng góp lớn thứ hai vào năm 1988 là công trình của Rich Sutton kết nối học tăng cường - đã được sử dụng trong chương trình chơi cờ dam của Arthur Samuel vào những năm 1950 - với lý thuyết về các quá trình quyết định Markov (MDP) được phát triển trong lĩnh vực tìm kiếm lại hoạt động. Một loạt công việc sau đó kết nối nghiên cứu lập kế hoạch AI với MDP, và lĩnh vực học tăng cường đã tìm thấy các ứng dụng trong robot và kiểm soát quy trình cũng như thu thập các nền tảng lý thuyết sâu sắc.

Một hệ quả của sự đánh giá mới của AI đối với dữ liệu, mô hình thống kê, tối ưu hóa và học máy là sự thống nhất dần dần các các lĩnh vực con như thị giác máy tính, robot, nhận dạng giọng nói, hệ thống đa tác nhân và xử lý ngôn ngữ tự nhiên đã

trở nên phần nào tách biệt với AI cốt lõi. Quá trình tái hòa nhập đã mang lại những lợi ích đáng kể cả về mặt ứng dụng - ví dụ, việc triển khai robot thực tế đã mở rộng rất nhiều trong giai đoạn này - và trong sự hiểu biết lý thuyết tốt hơn về các vấn đề cốt lõi của AI.

1.3.7 Dữ liệu lớn (2001–nay)

Dữ liệu lớn

Những tiến bộ đáng kể về sức mạnh tính toán và việc tạo ra World Wide Web đã tạo điều kiện thuận lợi cho việc tạo ra các bộ dữ liệu rất lớn - một hiện tượng đôi khi được gọi là dữ liệu lớn.

Các bộ dữ liệu này bao gồm hàng nghìn tỷ từ văn bản, hàng tỷ hình ảnh và hàng tỷ giờ giọng nói và video, cũng như một lượng lớn dữ liệu gen, dữ liệu theo dõi xe, dữ liệu luồng nhấp chuột, dữ liệu mạng xã hội, v.v.

Điều này đã dẫn đến sự phát triển của các thuật toán học tập được thiết kế đặc biệt để có lợi cho các tập dữ liệu rất lớn. Thông thường, phần lớn các ví dụ trong các tập dữ liệu như vậy không được gắn nhãn; ví dụ, trong công trình có ảnh hưởng của Yarowsky (1995) về định hướng từ ngữ, sự xuất hiện của một từ như "thực vật" không được dán nhãn trong tập dữ liệu để chỉ ra xem chúng đề cập đến hệ thực vật hay nhà máy. Tuy nhiên, với các tập dữ liệu đủ lớn, các thuật toán học tập phù hợp có thể đạt được độ chính xác hơn 96% trong nhiệm vụ xác định ý nghĩa nào được dự định trong một câu. Hơn nữa, Banko và Brill (2001) lập luận rằng sự cải thiện hiệu suất do tăng kích thước của bộ dữ liệu lên hai hoặc ba bậc lớn hơn bất kỳ cải tiến nào có thể thu được từ việc điều chỉnh thuật toán.

Một hiện tượng tương tự dường như xảy ra trong các nhiệm vụ thị giác máy tính như lấp đầy các lỗ hổng trong các bức ảnh - những lỗ hổng gây ra bởi thiệt hại hoặc do loại bỏ bạn bè cũ. Hays và Efros (2007) đã phát triển một phương pháp thông minh để làm điều này bằng cách pha trộn các pixel từ các hình ảnh tương tự; Họ phát hiện ra rằng kỹ thuật này hoạt động kém với cơ sở dữ liệu chỉ có hàng nghìn hình ảnh nhưng đã vượt qua ngưỡng chất lượng với hàng triệu hình ảnh. Ngay sau đó, sự sẵn có của hàng chục triệu hình ảnh trong cơ sở dữ liệu ImageNet (Deng et al., 2009) đã chấm ngòi cho một cuộc cách mạng trong lĩnh vực thị giác máy tính.

Sự sẵn có của dữ liệu lớn và sự chuyển đổi sang học máy đã giúp AI phục hồi sức hấp dẫn thương mại (Havenstein, 2005; Halevy và cộng sự, 2009). Dữ liệu lớn là một yếu tố quan trọng trong chiến thắng năm 2011 của hệ thống Watson của IBM trước các nhà vô địch con người trong Jeopardy! quizgame, một sự kiện có tác động lớn đến nhận thức của công chúng về AI.

1.3.8 Học sâu (2011–nay)

Học sâu

Thuật ngữ deep learning đề cập đến học máy sử dụng nhiều lớp đơn giản, có thể điều chỉnh các yếu tố tính toán. Các thí nghiệm đã được thực hiện với các mạng như vậy từ những năm 1970, và dưới dạng mạng nơ-ron tích chập, họ đã tìm thấy một số thành công trong việc nhận dạng chữ số viết tay vào những năm 1990 (LeCun và cộng sự, 1995). Tuy nhiên, phải đến năm 2011, các phương pháp học sâu mới thực sự cất cánh. Điều này xảy ra đầu tiên trong nhận dạng giọng nói và sau đó là nhận dạng đối tượng trực quan trong cuộc thi ImageNet năm 2012, yêu cầu phân loại hình ảnh thành một trong các loại cát (armadillo, giá sách, nút chai, v.v.), một hệ thống học sâu được tạo ra trong nhóm của Geoffrey Hinton tại Đại học Toronto (Krizhevsky et al., 2013) đã chứng minh một sự cải tiến đáng kể so với các hệ thống trước đó, chủ yếu dựa trên các tính năng thủ công. Kể từ đó, các hệ thống deep learning đã vượt quá hiệu suất của con người trong một số nhiệm vụ vi-sion (và tụt hậu trong một số nhiệm vụ khác). Những thành tựu tương tự đã được báo cáo trong bài phát biểu

nhận dạng, dịch máy, chẩn đoán y tế và chơi trò chơi. Việc sử dụng mạng sâu để đại diện cho chức năng đánh giá đã góp phần vào chiến thắng của ALPHAGO trước những người chơi cờ vây hàng đầu (Silver và cộng sự, 2016, 2017, 2018).

Những thành công đáng chú ý này đã dẫn đến sự hồi sinh của sự quan tâm đến AI đối với sinh viên, công ty, nhà đầu tư, chính phủ, phương tiện truyền thông và công chúng. Có vẻ như mỗi tuần đều có tin tức về một ứng dụng AI mới tiếp cận hoặc vượt quá hiệu suất của con người, thường đi kèm với suy đoán về thành công nhanh chóng hoặc một mùa đông AI

Mỗi sâu phụ thuộc rất nhiều vào phần cứng mạnh mẽ. Trong khi CPU máy tính tiêu chuẩn có thể thực hiện 109 hoặc 1010 thao tác mỗi giây, một thuật toán học sâu chạy trên phần cứng chuyên dụng (ví dụ: GPU, TPU hoặc FPGA) có thể tiêu thụ từ 1014 đến 1017 phép toán mỗi giây, chủ yếu ở dạng các phép toán ma trận và vector song song cao. Tất nhiên, deep learning cũng phụ thuộc vào sự sẵn có của một lượng lớn dữ liệu đào tạo và một vài thủ thuật thuật toán (xem Chương 22).

1.4 Trạng thái của nghệ thuật

Nghiên cứu một trăm năm về AI của Đại học Stanford (còn được gọi là AI100) triệu tập các hội đồng chuyên gia để cung cấp các báo cáo về hiện trạng trong AI. Báo cáo năm 2016 của họ (Stone et al., 2016; Grosz và Stone, 2018) kết luận rằng "Có thể mong đợi sự gia tăng đáng kể trong việc sử dụng các ứng dụng AI trong tương lai, bao gồm nhiều xe tự lái hơn, chẩn đoán chăm sóc sức khỏe và điều trị nhắm mục tiêu, và hỗ trợ vật lý cho chăm sóc người cao tuổi" và rằng "Xã hội hiện đang ở một thời điểm quan trọng trong việc xác định cách triển khai các công nghệ dựa trên AI theo những cách thúc đẩy các giá trị dân chủ như tự do, bình đẳng và minh bạch." AI100 cũng tạo ra Chỉ số AI tại aindex.org để giúp theo dõi tiến độ. Một số điểm nổi bật từ năm 2018 và

Chỉ số AI

Báo cáo năm 2019 (so sánh với đường cơ sở năm 2000 trừ khi có quy định khác):

- **Ấn phẩm:** Các bài báo AI tăng gấp 20 lần từ năm 2010 đến năm 2019 lên khoảng 20.000 mỗi năm. Danh mục phổ biến nhất là học máy. (Giấy học máy arXiv.org tăng gấp đôi mỗi năm từ năm 2009 đến năm 2017.) Thị giác máy tính và xử lý ngôn ngữ tự nhiên là phổ biến tiếp theo.
- **Tâm lý:** Khoảng 70% các bài báo về AI là trung lập, nhưng các bài viết có giọng điệu tích cực tăng từ 12% năm 2016 lên 30% năm 2018. Các vấn đề phổ biến nhất là đạo đức: quyền riêng tư dữ liệu và thiên vị thuật toán.
- **Sinh viên:** Số lượng đăng ký khóa học tăng gấp 5 lần ở Hoa Kỳ và 16 lần quốc tế so với mức cơ sở năm 2010. AI là chuyên ngành phổ biến nhất trong Khoa học Máy tính.
- **Đa dạng:** Giáo sư AI trên toàn thế giới có khoảng 80% nam, 20% nữ. Con số tương tự đối với tiến sĩ sinh viên và tuyển dụng trong ngành.
- **Hội nghị:** Số người tham dự NeurIPS tăng 800% kể từ năm 2012 lên 13.500 người tham dự. Các hội nghị khác đang chứng kiến mức tăng trưởng hàng năm khoảng 30%.
- **Ngành:** Các công ty khởi nghiệp AI ở Mỹ đã tăng gấp 20 lần lên hơn 800.
- **Quốc tế hóa:** Trung Quốc xuất bản nhiều bài báo mỗi năm hơn Hoa Kỳ và nhiều như toàn bộ châu Âu. Tuy nhiên, về tác động theo trọng số trích dẫn, các tác giả Hoa Kỳ đi trước 50% so với các tác giả Trung Quốc, Singapore, Brazil, Úc, Canada và Ấn Độ là những quốc gia phát triển nhanh nhất về số lượng nhân viên AI.

- Thị giác: Tỷ lệ lỗi phát hiện đối tượng (đạt được trong LSVRC, Thử thách nhận dạng hình ảnh quy mô lớn) được cải thiện từ 28% năm 2010 lên 2% năm 2017, vượt quá hiệu suất của con người. Độ chính xác của trả lời câu hỏi trực quan kết thúc mở (VQA) được cải thiện từ 55% lên 68% kể từ năm 2015, nhưng tụt hậu so với hiệu suất của con người ở mức 83%.
- Tốc độ: Thời gian đào tạo cho nhiệm vụ nhận dạng hình ảnh giảm 100 lần chỉ trong hai năm qua. Lượng sức mạnh tính toán được sử dụng trong các ứng dụng AI hàng đầu tăng gấp đôi sau mỗi 3,4 tháng.
- Ngôn ngữ: Độ chính xác trong trả lời câu hỏi, được đo bằng điểm F1 trên Bộ dữ liệu trả lời câu hỏi Stanford (SQUAD), tăng từ 60 lên 95 từ năm 2015 đến năm 2019; trên biến thể SQUAD 2, tiến độ nhanh hơn, tăng từ 62 lên 90 chỉ trong một năm. Cả hai điểm đều vượt quá hiệu suất ở cấp độ con người.
- Tiêu chuẩn của con người: Đến năm 2019, các hệ thống AI đã đạt hoặc vượt quá hiệu suất ở cấp độ con người trong cờ vua, cờ vây, poker, Pac-Man, Jeopardy!, phát hiện đối tượng ImageNet, nhận dạng giọng nói trong một lĩnh vực hạn chế, dịch tiếng Trung sang tiếng Anh trong một lĩnh vực hạn chế, Quake III, Dota 2, StarCraft II, các trò chơi Atari khác nhau, phát hiện ung thư da, phát hiện ung thư tuyến tiền liệt, gấp protein và chẩn đoán bệnh võng mạc tiểu đường.

Khi nào (nếu có) hệ thống AI sẽ đạt được hiệu suất ở cấp độ con người trong nhiều nhiệm vụ khác nhau? Ford (2018) phỏng vấn các chuyên gia AI và tìm thấy một loạt các năm mục tiêu, từ 2029 đến 2200, với mức trung bình là 2099. Trong một cuộc khảo sát tương tự (Grace et al., 2017), 50% người được hỏi nghĩ rằng điều này có thể xảy ra vào năm 2066, mặc dù 10% nghĩ rằng nó có thể xảy ra sớm nhất là vào năm 2025 và một số ít nói "không bao giờ". Các chuyên gia cũng chia rẽ về việc liệu chúng ta cần những đột phá mới cơ bản hay chỉ cần cải tiến các cách tiếp cận hiện tại. Nhưng đừng xem xét dự đoán của họ quá nghiêm túc; như Philip Tetlock (2017) chứng minh trong lĩnh vực dự đoán các sự kiện thể giới, các chuyên gia không tốt hơn những người nghiệp dư.

Các hệ thống AI trong tương lai sẽ hoạt động như thế nào? Chúng tôi vẫn chưa thể nói. Như đã trình bày chi tiết trong phần này, lĩnh vực này đã áp dụng một số câu chuyện về chính nó - đầu tiên là ý tưởng táo bạo rằng trí thông minh của một máy móc thậm chí là có thể, sau đó nó có thể đạt được bằng cách mã hóa kiến thức chuyên môn thành logic, sau đó là các mô hình xác suất của thế giới sẽ là công cụ chính, và gần đây nhất là học máy sẽ tạo ra các mô hình có thể không dựa trên bất kỳ lý thuyết nào được hiểu rõ. Tương lai sẽ tiết lộ mô hình nào sẽ xuất hiện tiếp theo.

AI có thể làm gì ngày nay? Có lẽ không nhiều như một số bài báo truyền thông lạc quan hơn có thể khiến người ta tin tưởng, nhưng vẫn rất nhiều. Dưới đây là một số ví dụ:

Phương tiện robot: Lịch sử của xe robot kéo dài từ những năm 1920, nhưng những cuộc trình diễn đầu tiên về lái xe tự động trên đường mà không có hướng dẫn đặc biệt xảy ra vào những năm 1980 (Kanade et al., 1986; Dickmanns và Zapp, 1987). Sau những màn trình diễn thành công về lái xe trên đường đất trong Thử thách lớn DARPA dài 132 dặm vào năm 2005 (Thrun, 2006) và trên đường phố có lưu lượng truy cập trong Thử thách đô thị năm 2007, cuộc đua phát triển xe tự lái bắt đầu một cách nghiêm túc. Vào năm 2018, các phương tiện thử nghiệm của Waymo đã vượt qua mốc 10 triệu dặm lái trên đường công cộng mà không gặp tai nạn nghiêm trọng, với người lái xe chỉ tiếp quản 6.000 dặm một lần. Ngay sau đó, công ty bắt đầu cung cấp dịch vụ taxi robot thương mại.

Trên không, máy bay không người lái cánh cố định tự động đã cung cấp dịch vụ mào xuyên quốc gia ở Rwanda kể từ năm 2016. Quadcopter thực hiện các thao tác nhào lộn trên không đáng chú ý, khám phá các tòa nhà trong khi xây dựng bản đồ 3D và tự lắp ráp thành các đội hình tự động.

Chuyển động chân: BigDog, một robot bốn chân của Raibert et al. (2008), đã đảo lộn quan niệm của chúng ta về cách robot di chuyển — không còn là dáng đi chậm rãi, chân cứng, từ bên này sang bên kia của robot Hollywood, mà là một thứ gì đó gần giống với một con vật và có thể phục hồi khi bị xô đẩy khi trượt trên vùng nước băng giá. Atlas, một robot hình người, không chỉ đi trên địa hình không bằng phẳng mà còn nhảy lên hộp và lộn ngược (Ackerman và Guizzo, 2016).

Lập kế hoạch và lập kế hoạch tự động: Cách Trái đất một trăm triệu dặm, chương trình Tác nhân từ xa của NASA đã trở thành chương trình lập kế hoạch tự động trên tàu đầu tiên để kiểm soát việc lên lịch hoạt động cho tàu vũ trụ (Jonsson et al., 2000). Remote Agent tạo kế hoạch từ các mục tiêu cấp cao được chỉ định từ cơ sở và giám sát việc thực hiện các kế hoạch đó — phát hiện, chẩn đoán và phục hồi sau các sự cố khi chúng xảy ra. Ngày nay, bộ công cụ lập kế hoạch EUROPA (Barreiro et al., 2012) được sử dụng cho các hoạt động hàng ngày của Marsrovers của NASA và hệ thống SEXTANT (Winternitz, 2017) cho phép điều hướng tự động trong không gian sâu, vượt ra ngoài hệ thống GPS toàn cầu.

Trong cuộc khủng hoảng Vịnh Ba Tư năm 1991, các lực lượng Hoa Kỳ đã triển khai một Công cụ Phân tích động và Lập kế hoạch lại, DART (Cross and Walker, 1994), để lập kế hoạch và lập kế hoạch hậu cần tự động cho việc vận chuyển. Điều này liên quan đến 50.000 phương tiện, hàng hóa và con người cùng một lúc, và phải tính đến điểm xuất phát, điểm đến, tuyến đường, khả năng vận chuyển, công suất cảng và sân bay và giải quyết xung đột trong tất cả các thông số. Cơ quan Dự án Nghiên cứu Tiên tiến Quốc phòng (DARPA) tuyên bố rằng đơn đăng ký này đã hoàn trả khoản đầu tư 30 năm của DARPA vào AI.

Mỗi ngày, các công ty gọi xe như Uber và các dịch vụ bản đồ như GoogleMaps cung cấp chỉ đường lái xe cho hàng trăm triệu người dùng, nhanh chóng vạch ra một tuyến đường tối ưu có tính đến điều kiện giao thông hiện tại và dự đoán trong tương lai.

Dịch máy: Hệ thống dịch máy trực tuyến hiện cho phép đọc tài liệu bằng hơn 100 ngôn ngữ, bao gồm cả ngôn ngữ mẹ đẻ của hơn 99% con người và hiển thị hàng trăm tỷ từ mỗi ngày cho hàng trăm triệu người dùng. Mặc dù không hoàn hảo, nhưng chúng thường đủ để hiểu. Đối với các ngôn ngữ có liên quan chặt chẽ với rất nhiều dữ liệu đào tạo (chẳng hạn như tiếng Pháp và tiếng Anh) các bản dịch trong một miền hẹp gần với trình độ của con người (Wu và cộng sự, 2016b).

Nhận dạng giọng nói: Vào năm 2017, Microsoft đã chỉ ra rằng Hệ thống nhận dạng giọng nói đàm thoại của họ đã đạt tỷ lệ lỗi từ 5,1%, phù hợp với hiệu suất của con người trong nhiệm vụ Tổng đài, liên quan đến việc phiên âm các cuộc trò chuyện qua điện thoại (Xiong và cộng sự, 2017). Khoảng một phần ba tương tác máy tính trên toàn thế giới hiện được thực hiện bằng giọng nói chứ không phải bàn phím; Skype cung cấp dịch thuật chuyển giọng nói thành giọng nói theo thời gian thực bằng mười ngôn ngữ. Alexa, Siri, Cortana và Google cung cấp các trợ lý có thể trả lời câu hỏi và thực hiện các tác vụ cho người dùng; ví dụ: dịch vụ Google Duplex sử dụng tính năng nhận dạng giọng nói và tổng hợp giọng nói để đặt chỗ nhà hàng cho người dùng, thay mặt họ thực hiện một cuộc trò chuyện trôi chảy.

Đề xuất: Các công ty như Amazon, Facebook, Netflix, Spotify, YouTube, Walmart và những công ty khác sử dụng máy học để đề xuất những gì bạn có thể thích dựa trên kinh nghiệm trong quá khứ của bạn và của những người khác như bạn. Lĩnh vực hệ thống đề xuất có lịch sử lâu đời (Resnick và Varian, 1997) nhưng đang thay đổi nhanh chóng do các phương pháp học sâu mới phân tích nội dung (văn bản, âm nhạc, video) cũng như lịch sử và siêu dữ liệu (van den Oord et al., 2014; Zhang và cộng sự, 2017). Lọc thư rác cũng có thể được coi là một hình thức khuyến nghị (hoặc không đề xuất); Các kỹ thuật AI hiện tại lọc ra hơn 99,9% thư rác và các dịch vụ email cũng có thể đề xuất người nhận tiềm năng, cũng như văn bản phản hồi có thể có.

Chơi trò chơi: Khi Deep Blue đánh bại nhà vô địch cờ vua thế giới Garry Kasparov vào năm 1997, những người bảo vệ quyền tối cao của con người đã đặt hy vọng vào cờ vây. Piet Hut, một nhà vật lý thiên văn và đam mê cờ vây, dự đoán rằng sẽ mất "một trăm năm trước khi một máy tính đánh bại con người ở cờ vây - thậm chí có thể lâu hơn". Nhưng chỉ 20 năm sau, ALPHAGO đã vượt qua tất cả những người chơi trên con người (Silver et al., 2017). Ke Jie, nhà vô địch thế giới, cho biết: "Năm ngoái, nó vẫn khá giống con người khi nó chơi. Nhưng năm nay, nó trở nên giống như một vị thần cờ vây." ALPHAGO được hưởng lợi từ việc nghiên cứu hàng trăm nghìn trò chơi trong quá khứ của người chơi cờ vây và từ kiến thức chất lọc của những người chơi cờ vây chuyên nghiệp đã làm việc trong nhóm.

Một chương tiếp theo, ALPHAZERO, không sử dụng đầu vào từ con người (ngoại trừ các quy tắc của trò chơi) và có thể học thông qua tự chơi một mình để đánh bại tất cả các đối thủ, con người và máy móc, cờ vây, cờ vua và shogi (Silver et al., 2018). Trong khi đó, nhà vô địch con người đã bị đánh bại bởi các hệ thống AI trong các trò chơi đa dạng như Jeopardy! (Ferrucci và cộng sự, 2010), poker (Bowling và cộng sự, 2015; Moravčík và cộng sự, 2017; Brown và Sandholm, 2019), và trò chơi điện tử Dota 2 (Fernandez và Mahlmann, 2018), StarCraft II (Vinyals và cộng sự, 2019) và Quake III (Jaderberg và cộng sự, 2019).

Hiểu hình ảnh: Không hài lòng với độ chính xác vượt quá độ chính xác của con người trong nhiệm vụ nhận dạng đối tượng ImageNet đầy thách thức, các nhà nghiên cứu thị giác máy tính đã giải quyết vấn đề khác nhau hơn về phụ đề hình ảnh. Một số ví dụ ấn tượng bao gồm "Một người đi xe máy trên đường đất", "Hai chiếc bánh pizza ngồi trên lò nướng" và "Một nhóm thanh niên đang chơi trò chơi ném đĩa" (Vinyals và cộng sự, 2017b). Tuy nhiên, các hệ thống hiện tại còn lâu mới hoàn hảo: một "tủ lạnh chứa đầy nhiều thức ăn và đồ uống" hóa ra là biển báo đỗ xe bị che khuất một phần bởi rất nhiều nhãn dán nhỏ.

Y học: Các thuật toán AI hiện bằng hoặc vượt qua các bác sĩ chuyên môn trong việc chẩn đoán nhiều điều kiện, đặc biệt là khi chẩn đoán dựa trên hình ảnh. Ví dụ bao gồm bệnh Alzheimer (Ding và cộng sự, 2018), ung thư di căn (Liu và cộng sự, 2017; Esteva và cộng sự, 2017), bệnh nhãn khoa (Gulshan và cộng sự, 2016) và bệnh ngoài da (Liu và cộng sự, 2019c). Một đánh giá có hệ thống và phân tích tổng hợp (Liu et al., 2019a) cho thấy hiệu suất của các chương trình AI, trung bình, tương đương với các chuyên gia chăm sóc sức khỏe. Một trọng tâm hiện tại trong AI y tế là tạo điều kiện thuận lợi cho quan hệ đối tác giữa con người và máy móc. Ví dụ: hệ thống LYNA đạt được 99,6% độ chính xác tổng thể trong chẩn đoán ung thư vú di căn — tốt hơn so với một chuyên gia người không có sự trợ giúp — nhưng sự kết hợp này vẫn tốt hơn (Liu và cộng sự, 2018; Steiner và cộng sự, 2018).

Việc áp dụng rộng rãi các kỹ thuật này hiện bị giới hạn không phải bởi độ chính xác của chẩn đoán mà bởi nhu cầu chứng minh sự cải thiện trong kết quả lâm sàng và để đảm bảo tính minh bạch, không thiên vị và quyền riêng tư dữ liệu (Topol, 2019). Vào năm 2017, chỉ có hai ứng dụng AI y tế được FDA phê duyệt, nhưng con số đó đã tăng lên 12 vào năm 2018 và tiếp tục tăng.

Khoa học khí hậu: Một nhóm các nhà khoa học đã giành được Giải thưởng Gordon Bell năm 2018 cho một mô hình học sâu khám phá thông tin chi tiết về các hiện tượng thời tiết cực đoan trước đây đã được chôn vùi trong dữ liệu khí hậu. Họ đã sử dụng một siêu máy tính với phần cứng GPU chuyên dụng để vượt quá mức exaop (10¹⁸ hoạt động mỗi giây), chương trình học máy đầu tiên làm như vậy (Kurth et al., 2018). Rolnick et al. (2019) trình bày một danh mục dài 60 trang về các cách mà máy học có thể được sử dụng để giải quyết vấn đề biến đổi khí hậu.

Đây chỉ là một vài ví dụ về hệ thống trí tuệ nhân tạo tồn tại ngày nay. Không phải ma thuật hay khoa học viễn tưởng — mà là khoa học, kỹ thuật và toán học, mà cuốn sách này cung cấp một phần giới thiệu.

1.5 Rủi ro và lợi ích của AI

Francis Bacon, một triết gia được ghi nhận là người đã tạo ra phương pháp khoa học, đã lưu ý trong *The Wis-dom of the Ancients* (1609) rằng "nghệ thuật cơ khí được sử dụng mơ hồ, cũng phục vụ cho tổn thương cũng như để chữa trị." Khi AI đóng một vai trò ngày càng quan trọng trong các lĩnh vực kinh tế, xã hội, khoa học, y tế, tài chính và quân sự, chúng ta nên xem xét những tổn thương và biện pháp khắc phục - theo cách nói hiện đại, những rủi ro và lợi ích - mà nó có thể mang lại. Các chủ đề được tóm tắt ở đây được đề cập sâu hơn trong Chương 28 và 29.

Để bắt đầu với những lợi ích: nói một cách đơn giản, toàn bộ nền văn minh của chúng ta là sản phẩm của trí tuệ con người của chúng ta. Nếu chúng ta có quyền truy cập vào trí thông minh máy móc lớn hơn đáng kể, mức trần đối với tham vọng của chúng ta sẽ được nâng lên đáng kể. Tiềm năng của AI và robot để giải phóng nhân loại khỏi những công việc lặp đi lặp lại và tăng đáng kể sản xuất hàng hóa và dịch vụ có thể báo trước một kỷ nguyên hòa bình và sung túc. Khả năng đẩy nhanh nghiên cứu khoa học có thể dẫn đến các phương pháp chữa bệnh và giải pháp cho biến đổi khí hậu và thiếu hụt tài nguyên. Như DemisHassabis, Giám đốc điều hành của Google DeepMind, đã gợi ý: "Đầu tiên hãy giải quyết AI, sau đó sử dụng AI để giải mọi thứ khác."

Tuy nhiên, rất lâu trước khi chúng ta có cơ hội "giải quyết AI", chúng ta sẽ phải chịu rủi ro từ việc lạm dụng AI, vô tình hay cách khác. Một số trong số này đã rõ ràng, trong khi những người khác dường như có thể dựa trên xu hướng hiện tại:

- Vũ khí tự động gây chết người: Chúng được Liên Hợp Quốc định nghĩa là vũ khí có thể xác định, chọn lọc và loại bỏ các mục tiêu của con người mà không cần sự can thiệp của con người. Một mối quan tâm chính với những vũ khí như vậy là khả năng mở rộng của chúng: việc không có yêu cầu giám sát của con người có nghĩa là một nhóm nhỏ có thể triển khai một số lượng lớn vũ khí tùy ý chống lại các mục tiêu của con người được xác định bởi bất kỳ tiêu chí nhận biết khả thi nào. Các công nghệ cần thiết cho vũ khí tự động tương tự như những công nghệ cần thiết cho ô tô tự lái. Các cuộc thảo luận không chính thức của các chuyên gia về những rủi ro tiềm ẩn của vũ khí tự động gây chết người đã bắt đầu tại Liên Hợp Quốc vào năm 2014, chuyển sang giai đoạn chính thức trước hiệp ước của Nhóm Chuyên gia Chính phủ vào năm 2017.
- Giám sát và thuyết phục: Mặc dù việc nhân viên an ninh giám sát đường dây điện thoại, nguồn cấp dữ liệu camera video, email và các kênh nhắn tin khác tồn tại, tế nhị và đôi khi có vấn đề về mặt pháp lý, AI (nhận dạng giọng nói, thị giác máy tính và hiểu ngôn ngữ tự nhiên) có thể được sử dụng theo cách có thể mở rộng để thực hiện giám sát hàng loạt các cá nhân và phát hiện các hoạt động quan tâm. Bằng cách điều chỉnh các luồng thông tin cho người riêng thông qua phương tiện truyền thông xã hội, dựa trên các kỹ thuật học máy, hành vi chính trị có thể được sửa đổi và kiểm soát ở một mức độ nào đó - một mối quan tâm đã trở nên rõ ràng trong các cuộc bầu cử bất đầu năm 2016.
- Ra quyết định thiên vị: Việc sử dụng sai thuật toán máy học bất cần hoặc cố ý cho các nhiệm vụ như đánh giá đơn xin ăn xá và cho vay có thể dẫn đến các quyết định thiên vị theo chủng tộc, giới tính hoặc các danh mục được bảo vệ khác. Thông thường, bản thân dữ liệu phản ánh thành kiến phổ biến trong xã hội.
- Tác động đến việc làm: Những lo ngại về việc máy loại bỏ việc làm đã có từ nhiều thế kỷ. Câu chuyện không bao giờ đơn giản: máy móc thực hiện một số nhiệm vụ mà con người có thể làm, nhưng chúng cũng làm cho con người làm việc hiệu quả hơn và do đó có việc làm nhiều hơn, và làm cho các công ty có lợi nhuận cao hơn và do đó có thể trả lương cao hơn. Chúng có thể làm cho một số hoạt động khả thi về mặt kinh tế mà nếu không sẽ không thực tế. Của họ

Sử dụng thường dẫn đến sự giàu có tăng lên nhưng có xu hướng có tác động chuyển sự giàu có từ lao động sang vốn, làm trầm trọng thêm sự gia tăng bất bình đẳng. Những tiến bộ trước đây trong công nghệ - chẳng hạn như phát minh ra khung dệt cơ khí - đã dẫn đến sự gián đoạn nghiêm trọng đối với việc làm, nhưng cuối cùng mọi người tìm thấy các loại công việc mới để làm. Mặt khác, có thể AI cũng sẽ thực hiện những loại công việc mới đó. Chủ đề này đang nhanh chóng trở thành trọng tâm chính của các nhà kinh tế và chính phủ trên toàn thế giới.

- Các ứng dụng quan trọng về an toàn: Khi các kỹ thuật AI phát triển, chúng ngày càng được sử dụng trong các ứng dụng quan trọng về an toàn như lái ô tô và quản lý nguồn cung cấp nước của các thành phố. Tai nạn chết người đã xảy ra và làm nổi bật sự khó khăn của việc xác minh chính thức và phân tích rủi ro thống kê đối với các hệ thống được phát triển bằng kỹ thuật học máy. Lĩnh vực AI sẽ cần phát triển các tiêu chuẩn kỹ thuật và đạo đức, ít nhất là có thể so sánh với những tiêu chuẩn phổ biến trong các ngành kỹ thuật và chăm sóc sức khỏe khác, nơi tính mạng của con người đang bị đe dọa.
- An ninh mạng: Các kỹ thuật AI rất hữu ích trong việc bảo vệ chống lại cuộc tấn công mạng, bằng cách phát hiện các kiểu hành vi bất thường, nhưng chúng cũng sẽ góp phần vào khả năng tồn tại, khả năng sống sót và phổ biến của phần mềm độc hại. Ví dụ, các phương pháp học tập tăng cường đã được sử dụng để tạo ra các công cụ hiệu quả cao cho các cuộc tấn công tổng tiền và lừa đảo tự động, được cá nhân hóa.

Chúng ta sẽ xem xét lại các chủ đề này sâu hơn trong Phần 28.3. Khi các hệ thống AI trở nên có khả năng hơn, chúng sẽ đảm nhận nhiều vai trò xã hội hơn trước đây do con người đảm nhận. Cũng giống như con người đã sử dụng những vai trò này trong quá khứ để gây ra những trở ngại nghiêm trọng, chúng ta có thể mong đợi rằng con người có thể lạm dụng hệ thống AI trong những vai trò này để gây ra nhiều trở ngại nghiêm trọng hơn nữa. Tất cả các ví dụ được đưa ra ở trên đều chỉ ra tầm quan trọng của quản trị và cuối cùng là quy định. Hiện tại, cộng đồng tìm kiếm và các tập đoàn lớn tham gia vào nghiên cứu AI đã phát triển các nguyên tắc tự quản tự nguyện cho các hoạt động liên quan đến AI (xem Phần 28.3). Các chính phủ và các tổ chức quốc tế đang thành lập các cơ quan tư vấn để đưa ra các quy định thích hợp cho từng trường hợp sử dụng cụ thể, để chuẩn bị cho các tác động kinh tế và xã hội và tận dụng khả năng của AI để giải quyết các vấn đề xã hội lớn.

Còn về lâu dài thì sao? Liệu chúng ta có đạt được mục tiêu lâu dài: tạo ra trí tuệ tương đương hoặc có khả năng hơn trí thông minh của con người? Và, nếu chúng ta làm vậy, thì sao?

Trong phần lớn lịch sử của AI, những câu hỏi này đã bị lu mờ bởi công việc hàng ngày để các hệ thống AI làm bất cứ điều gì thậm chí thông minh từ xa. Như với bất kỳ ngành học rộng nào, đại đa số các nhà nghiên cứu AI đã chuyên về một lĩnh vực phụ cụ thể như chơi trò chơi, đại diện kiến thức, tầm nhìn hoặc hiểu ngôn ngữ tự nhiên — thường dựa trên giả định rằng tiến bộ trong các lĩnh vực phụ này sẽ đóng góp vào các mục tiêu rộng lớn hơn của AI. Nils Nilsson (1995), một trong những nhà lãnh đạo ban đầu của dự án Shakey tại SRI, đã nhắc nhở lĩnh vực này về những mục tiêu rộng lớn hơn đó và cảnh báo rằng các lĩnh vực phụ có nguy cơ trở thành mục đích trong chính chúng. Sau đó, một số nhà sáng lập có ảnh hưởng của AI, bao gồm John McCarthy (2007), Marvin Minsky (2007) và Patrick Winston (Beal và Winston, 2009), đồng ý với cảnh báo của Nilsson, gợi ý rằng thay vì tập trung vào hiệu suất có thể đo lường được trong các ứng dụng cụ thể, AI nên quay trở lại cội nguồn của nó là phần đầu, theo lời của Herb Simon, "những cố máy suy nghĩ, học hỏi và sáng tạo". Họ gọi nỗ lực này là AI cấp con người hoặc HLAI - một cỗ máy nên

AI cấp con người

Trí tuệ nhân tạo tổng quát (AGI)

có thể học cách làm bất cứ điều gì con người có thể làm. Hội nghị chuyên đề đầu tiên của họ là vào năm 2004 (Minsky et al., 2004). Một nỗ lực khác với mục tiêu tương tự, trí tuệ nhân tạo tổng quát (AGI)

phong trào (Goertzel và Pennachin, 2007), đã tổ chức hội nghị đầu tiên và tổ chức Journal of Artificial General Intelligence vào năm 2008.

Cùng lúc đó, những lo ngại đã được nêu ra rằng việc tạo ra siêu trí tuệ nhân tạo hoặc ASI - trí thông minh vượt xa khả năng của con người - có thể là một ý tưởng tồi (Yudkowsky, 2008; Omohundro, 2008). Bản thân Turing (1996) đã đưa ra quan điểm tương tự trong một bài giảng được đưa ra tại Manchester vào năm 1951, dựa trên những ý tưởng trước đó từ Samuel Butler (1863):15

Siêu trí tuệ nhân tạo (ASI)

Có vẻ như một khi phương pháp tư duy máy móc đã bắt đầu, sẽ không mất nhiều thời gian để vượt qua sức mạnh yếu ớt của chúng ta. Do đó, ở một giai đoạn nào đó, chúng ta nên mong đợi máy móc kiểm soát, theo cách được đề cập trong Erewhon của Samuel Butler. Những mối quan tâm này chỉ trở nên phổ biến hơn với những tiến bộ gần đây trong học sâu, xuất bản các cuốn sách như Siêu trí tuệ của Nick Bostrom (2014), và những tuyên bố công khai từ Stephen Hawking, Bill Gates, Martin Rees, và Elon Musk. Thử nghiệm cảm giác khó chịu chung với ý tưởng tạo ra máy siêu thông minh là điều tự nhiên. Chúng ta có thể gọi đây là vấn đề khi đột: khoảng bảy triệu năm

Vấn đề khi đột

Trước đây, một loài linh trưởng hiện đã tuyệt chủng đã tiến hóa, với một nhánh dẫn đến khi đột và một nhánh dẫn đến con người. Ngày nay, khi đột không quá hải lòng về nhánh con người; về cơ bản họ không có quyền kiểm soát tương lai của mình. Nếu đây là kết quả của sự thành công trong việc tạo ra AI siêu phàm - rằng con người từ bỏ quyền kiểm soát tương lai của họ - thì có lẽ chúng ta nên ngừng công việc về AI, và như một hệ quả tất yếu, từ bỏ những lợi ích mà nó có thể mang lại. Đây là bản chất của cảnh báo của Turing: không rõ ràng là chúng ta có thể điều khiển những cỗ máy thông minh hơn chúng ta.

Nếu AI siêu phàm là một hộp đen đến từ ngoài vũ trụ, thì thực sự sẽ là khôn ngoan khi mở hộp. Nhưng không phải vậy: chúng ta thiết kế các hệ thống AI, vì vậy nếu chúng kết thúc bằng việc "nắm quyền kiểm soát", như Turing gợi ý, đó sẽ là kết quả của một thất bại trong thiết kế.

Để tránh kết quả như vậy, chúng ta cần hiểu nguồn gốc của thất bại tiềm ẩn. Norbert Wiener (1960), người đã có động lực để xem xét tương lai lâu dài của AI sau khi chứng kiến chương trình chơi cờ đam của Arthur Samuel học cách đánh bại người tạo ra nó, đã nói như sau:

Nếu chúng ta sử dụng, để đạt được mục đích của mình, một cơ quan máy móc mà chúng ta không thể can thiệp một cách hiệu quả... Tốt hơn hết chúng ta nên chắc chắn rằng mục đích được đưa vào máy móc là mục đích mà chúng ta thực sự mong muốn. Nhiều nền văn hóa có thần thoại về con người yêu cầu các vị thần, thần đèn, pháp sư hoặc ma quỷ cho một điều gì đó. Luôn luôn, trong những câu chuyện này, họ nhận được những gì họ yêu cầu theo đúng nghĩa đen, và sau đó hối hận. Điều ước thứ ba, nếu có, là hoàn tác hai điều đầu tiên. Chúng tôi sẽ gọi đây là vấn đề Vua Midas:

Vấn đề Vua Midas

Midas, một vị vua huyền thoại trong thần thoại Hy Lạp, yêu cầu mọi thứ ông chạm vào đều phải biến thành vàng, nhưng sau đó hối hận sau khi chạm vào thức ăn, đồ uống và các thành viên trong gia đình của mình.¹⁶

Chúng tôi đã đề cập đến vấn đề này trong Phần 1.1.5, trong đó chúng tôi chỉ ra sự cần thiết phải sửa đổi đáng kể đối với mô hình tiêu chuẩn của việc đưa các mục tiêu cố định vào máy. Giải pháp cho tình trạng khó khăn của Wiener hoàn toàn không phải là có một "mục đích cụ thể được đưa vào máy". Thay vào đó, chúng ta muốn những cỗ máy cố gắng đạt được các mục tiêu của con người nhưng biết rằng chúng không biết chắc chắn chính xác những mục tiêu đó là gì.

15 Thậm chí trước đó, vào năm 1847, Richard Thornton, biên tập viên của Primitive Expounder, đã chỉ trích những người tính toán cơ học: "Tâm trí... vượt xa chính nó và loại bỏ sự cần thiết của sự tồn tại của chính nó bằng cách phát minh ra những cỗ máy để thực hiện suy nghĩ của chính nó. Nhưng ai biết được rằng những cỗ máy như vậy khi được đưa đến sự hoàn hảo hơn, có thể không nghĩ ra một kế hoạch để khắc phục tất cả các khiếm khuyết của chính chúng và sau đó nghiền nát những ý tưởng vượt quá khả năng của tâm trí phàm trần!" 16 Midas sẽ làm tốt hơn nếu ông tuân theo các nguyên tắc cơ bản về an toàn và bao gồm nút "hoàn tác" và nút "tạm dừng" trong mong muốn của mình.

Trò chơi hỗ trợ

Học tăng cường
ngịch đảo

Có lẽ thật không may là hầu hết tất cả các nghiên cứu AI cho đến nay đã được thực hiện trong mô hình tiêu chuẩn, có nghĩa là hầu hết tất cả các tài liệu kỹ thuật trong ấn bản này đều phản ánh khuôn khổ trí tuệ đó. Tuy nhiên, có một số kết quả ban đầu trong khuôn khổ mới. Trong Chương 15, chúng ta chỉ ra rằng một cỗ máy có một động lực tích cực để cho phép chính nó bị thất bại và chỉ khi nó không chắc chắn về mục tiêu của con người. Trong Chương 17, chúng ta xây dựng và nghiên cứu các trò chơi hỗ trợ, mô tả bằng toán học tình huống trong

mà con người có mục tiêu và máy móc cố gắng đạt được nó, nhưng ban đầu không chắc chắn về nó là gì. Trong Chương 23, chúng tôi giải thích các phương pháp học tăng cường nghịch đảo

cho phép máy móc tìm hiểu thêm về sở thích của con người từ việc quan sát các lựa chọn mà con người đưa ra. Trong Chương 28, chúng ta khám phá hai trong số những khó khăn chính: thứ nhất, sự lựa chọn của chúng ta phụ thuộc vào sở thích của chúng ta thông qua một cấu trúc nhận thức rất phức tạp khó đảo ngược; và thứ hai, con người chúng ta có thể không có sở thích nhất quán ngay từ đầu - cá nhân hoặc theo nhóm - vì vậy có thể không rõ các hệ thống AI nên làm gì cho chúng ta.

Tóm tắt

Chương này định nghĩa AI và thiết lập nền tảng văn hóa mà nó đã phát triển. Một số điểm quan trọng như sau:

- Những người khác nhau tiếp cận AI với các mục tiêu khác nhau. Hai câu hỏi quan trọng cần đặt ra là: Bạn có quan tâm đến suy nghĩ hay hành vi không? Bạn muốn mô hình hóa con người hay cố gắng đạt được kết quả tối ưu?
 - Theo cái mà chúng tôi gọi là mô hình tiêu chuẩn, AI chủ yếu liên quan đến hành động hợp lý. Một tác nhân thông minh lý tưởng sẽ hành động tốt nhất có thể trong một tình huống. Chúng tôi nghiên cứu vấn đề xây dựng các tác nhân thông minh theo nghĩa này.
 - Cần có hai sự tinh chỉnh cho ý tưởng đơn giản này: thứ nhất, khả năng của bất kỳ tác nhân nào, con người, để lựa chọn các hành động hợp lý bị giới hạn bởi tính toán khó giải quyết của việc làm như vậy; Thứ hai, khái niệm về một cỗ máy theo đuổi một mục tiêu xác định cần được thay thế bằng một cỗ máy theo đuổi các mục tiêu mang lại lợi ích cho con người, nhưng không chắc chắn về những mục tiêu đó là gì.
- Các nhà triết học (quay trở lại năm 400 trước Công nguyên) đã làm cho AI có thể hình dung được bằng cách gọi ý rằng tâm trí theo một cách nào đó giống như một cỗ máy, rằng nó hoạt động dựa trên kiến thức được mã hóa bằng một số ngôn ngữ nội bộ và suy nghĩ đó có thể được sử dụng để chọn những hành động sẽ thực hiện.

Các nhà toán học đã cung cấp các công cụ để thao túng các tuyên bố về sự chắc chắn logic cũng như các tuyên bố xác suất, không chắc chắn. Họ cũng đặt nền móng để hiểu tính toán và suy luận về các thuật toán.

Các nhà kinh tế chính thức hóa vấn đề đưa ra quyết định tối đa hóa tiện ích mong đợi cho người ra quyết định.

Các nhà thần kinh học đã phát hiện ra một số sự thật về cách thức hoạt động của não bộ và cách nó tương tự và khác với máy tính.

Các nhà tâm lý học đã chấp nhận ý tưởng rằng con người và động vật có thể được coi là máy xử lý thông tin. Các nhà ngôn ngữ học đã chỉ ra rằng việc sử dụng ngôn ngữ phù hợp với mô hình này.

- Các kỹ sư máy tính đã cung cấp những cỗ máy mạnh mẽ hơn bao giờ hết giúp các ứng dụng AI trở nên khả thi và các kỹ sư phần mềm làm cho chúng trở nên hữu dụng hơn.

- Lý thuyết điều khiển liên quan đến việc thiết kế các thiết bị hoạt động tối ưu trên cơ sở phản hồi từ môi trường. Ban đầu, các công cụ toán học của lý thuyết điều khiển hoàn toàn khác với những công cụ được sử dụng trong AI, nhưng các lĩnh vực đang xích lại gần nhau hơn.
- Lịch sử của AI đã có chu kỳ thành công, lạc quan sai chỗ và dẫn đến việc cắt giảm sự nhiệt tình và tài trợ. Cũng đã có những chu kỳ giới thiệu các phương pháp tiếp cận mới, sáng tạo và tinh chỉnh một cách có hệ thống những cách tốt nhất.
- AI đã trưởng thành đáng kể so với những thập kỷ đầu của nó, cả về mặt lý thuyết và phương pháp luận. Khi các vấn đề mà AI giải quyết trở nên phức tạp hơn, lĩnh vực này chuyển từ logic Boolean sang suy luận xác suất và từ kiến thức thủ công sang học máy từ dữ liệu. Điều này đã dẫn đến những cải tiến về khả năng của hệ thống thực và tích hợp nhiều hơn với các ngành khác.
- Khi các hệ thống AI được ứng dụng trong thế giới thực, cần phải xem xét một loạt các rủi ro và hậu quả đạo đức.
- Về lâu dài, chúng ta phải đối mặt với vấn đề khó khăn là kiểm soát các hệ thống AI siêu thông minh có thể phát triển theo những cách không thể đoán trước. Giải quyết vấn đề này dường như cần phải thay đổi quan niệm của chúng ta về AI.

Ghi chú thư mục và lịch sử

Một lịch sử toàn diện của AI được đưa ra bởi Nils Nilsson (2009), một trong những người tiên phong đầu tiên của lĩnh vực này. Pedro Domingos (2015) và Melanie Mitchell (2019) đưa ra cái nhìn tổng quan về máy học cho khán giả nói chung và Kai-Fu Lee (2018) mô tả cuộc đua giành vị trí lãnh đạo quốc tế trong AI. Martin Ford (2018) phỏng vấn 23 nhà nghiên cứu AI hàng đầu.

Các hiệp hội chuyên nghiệp chính về AI là Hiệp hội vì sự tiến bộ của trí tuệ nhân tạo (AAAI), Nhóm quan tâm đặc biệt của ACM về trí tuệ nhân tạo (SIGAI, trước đây là SIGART), Hiệp hội Châu Âu về AI và Hiệp hội Trí tuệ nhân tạo và Mô phỏng Hành vi (AISB). Quan hệ đối tác về AI tập hợp nhiều tổ chức thương mại và phi lợi nhuận quan tâm đến các tác động đạo đức và xã hội của AI. Tạp chí AI của AAAI chứa nhiều bài báo hướng dẫn và chuyên đề, và trang web của nó, aaai.org, chứa tin tức, hướng dẫn và thông tin cơ bản.

Công trình gần đây nhất xuất hiện trong kỷ yếu của các hội nghị AI lớn: Hội nghị chung quốc gia về AI (IJCAI), Hội nghị châu Âu hàng năm về AI (ECAI) và Hội nghị AAAI. Học máy được đề cập trong Hội nghị Quốc tế về Học máy và cuộc họp Hệ thống xử lý thông tin thần kinh (NeurIPS). Các tạp chí chính về AI nói chung là Trí tuệ nhân tạo, Trí tuệ tính toán, Giao dịch IEEE về Phân tích mẫu và Trí tuệ máy, Hệ thống thông minh IEEE và Tạp chí Nghiên cứu Trí tuệ Nhân tạo. Ngoài ra còn có nhiều hội nghị và tạp chí dành cho các lĩnh vực cụ thể, mà chúng tôi đề cập trong các chương thích hợp.

CHƯƠNG 2

ĐẠI LÝ THÔNG MINH

Trong đó chúng tôi thảo luận về bản chất của các tác nhân, hoàn hảo hay không, sự đa dạng của môi trường và kết quả là các loại tác nhân.

Chương 1 xác định khái niệm về các tác nhân hợp lý là trung tâm trong cách tiếp cận của chúng ta đối với trí tuệ nhân tạo. Trong chương này, chúng tôi làm cho khái niệm này cụ thể hơn. Chúng ta sẽ thấy rằng khái niệm hợp lý có thể được áp dụng cho nhiều tác nhân hoạt động trong bất kỳ môi trường nào có thể tưởng tượng được. Kế hoạch của chúng tôi trong cuốn sách này là sử dụng khái niệm này để phát triển một tập hợp nhỏ các nguyên tắc thiết kế để xây dựng các tác nhân thành công - các hệ thống có thể được gọi là thông minh một cách hợp lý.

Chúng ta bắt đầu bằng cách kiểm tra các tác nhân, môi trường và sự kết hợp giữa chúng. Việc quan sát rằng một số tác nhân cư xử tốt hơn những tác nhân khác dẫn đến ý tưởng về một tác nhân duy lý – một tác nhân hành xử tốt nhất có thể. Một tác nhân có thể hành xử tốt như thế nào phụ thuộc vào bản chất của môi trường; một số môi trường khó khăn hơn những môi trường khác. Chúng tôi đưa ra một phân loại thập tự chính về môi trường và cho thấy các thuộc tính của môi trường ảnh hưởng như thế nào đến việc loại bỏ dấu hiệu của các tác nhân phù hợp cho môi trường đó. Chúng tôi mô tả một số thiết kế đặc vụ "bộ xương" cơ bản, mà chúng tôi đã hoàn thiện trong phần còn lại của cuốn sách.

2.1 Tác nhân và Môi trường

Môi trườngCảm biến

Tác nhân là bất cứ thứ gì có thể được coi là nhận thức môi trường của nó thông qua các cảm biến và tác động lên môi trường đó thông qua các thiết bị truyền động. Ý tưởng đơn giản này được minh họa trong Hình 2.1.

Actuator

Một tác nhân của con người có mắt, tai và các cơ quan khác cho cảm biến và bàn tay, chân, đường thanh âm, v.v. cho bộ truyền động. Một tác nhân robot có thể có camera và máy tìm phạm vi hồng ngoại cho cảm biến và các động cơ khác nhau cho bộ truyền động. Tác nhân phần mềm nhận nội dung tệp, gói mạng và đầu vào của con người (bàn phím/chuột/màn hình cảm ứng/giọng nói) dưới dạng đầu vào cảm giác và hoạt động trên môi trường bằng cách ghi tệp, gửi gói mạng và hiển thị thông tin hoặc tạo ra âm thanh. Môi trường có thể là tất cả mọi thứ—toàn bộ vũ trụ! Trong thực tế, nó chỉ là một phần của vũ trụ mà chúng ta quan tâm đến trạng thái của nó khi thiết kế tác nhân này - phần ảnh hưởng đến những gì tác nhân nhận thức và bị ảnh hưởng bởi hành động của tác nhân.

Nhận thức

Chúng tôi sử dụng thuật ngữ nhận thức để chỉ nội dung mà cảm biến của tác nhân đang cảm nhận. Một

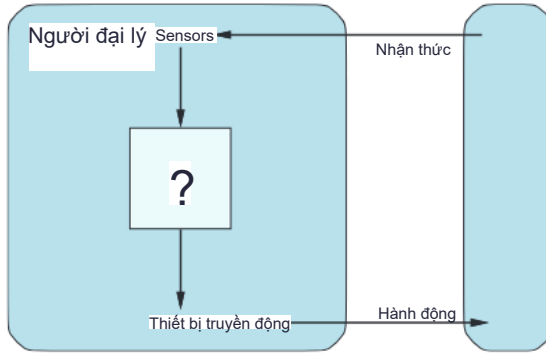
Trình tự nhận thức

Trình tự nhận thức của tác nhân là lịch sử hoàn chỉnh của mọi thứ mà tác nhân đã từng nhận thức.

Nói chung, sự lựa chọn hành động của một tác nhân tại bất kỳ thời điểm nào có thể phụ thuộc vào ranh giới tri thức tích hợp của nó và trên toàn bộ chuỗi tri giác được quan sát cho đến nay, nhưng không phải vào bất cứ điều gì mà nó không thực hiện

nhận được. Bằng cách xác định sự lựa chọn hành động của tác nhân cho mọi chuỗi tri giác có thể, chúng ta đã nói ít nhiều mọi thứ có thể nói về tác nhân. Nói về mặt toán học, chúng ta nói rằng hành vi của một tác nhân được mô tả bởi hàm tác nhân ánh xạ bất kỳ nhận thức trình tự cho một hành động.

Chức năng đại lý



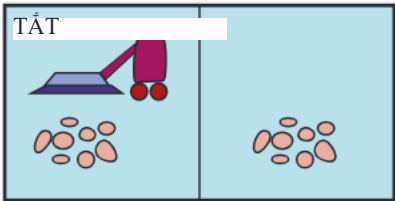
Hình 2.1 Các tác nhân tương tác với môi trường thông qua cảm biến và bộ truyền động.

Chúng ta có thể tưởng tượng việc lập bằng hàm tác nhân mô tả bất kỳ tác nhân nhất định nào; Đối với hầu hết các tác nhân, đây sẽ là một bằng rất lớn - vô hạn, trên thực tế, trừ khi chúng ta đặt một giới hạn trên độ dài của các chuỗi tri giác mà chúng ta muốn xem xét. Cho một tác nhân để thử nghiệm, về nguyên tắc, chúng ta có thể xây dựng bằng này bằng cách thử tất cả các chuỗi tri giác có thể có và ghi lại những hành động mà tác nhân thực hiện để đáp ứng.¹ Tất nhiên, bằng là một đặc điểm bên ngoài của tác nhân. Bên trong, chức năng tác nhân cho một tác nhân nhân tạo sẽ được thực hiện bởi chương trình anagent. Điều quan trọng là phải giữ cho hai ý tưởng này khác biệt. Hàm tác nhân là một mô tả toán học trừu tượng; Chương trình Agent là một triển khai cụ thể, chạy trong một hệ thống vật lý nào đó.

Để minh họa những ý tưởng này, chúng tôi sử dụng một ví dụ đơn giản - thế giới máy hút bụi, bao gồm một chất hút bụi robot trong một thế giới bao gồm các hình vuông có thể bẩn hoặc sạch. Hình 2.2 cho thấy một cấu hình chỉ có hai hình vuông, A và B. Tác nhân chân không nhận biết nó nằm trong hình vuông nào và liệu có bụi bẩn trong hình vuông hay không. Đại lý bắt đầu ở ô vuông A. Các hành động có sẵn là di chuyển sang phải, di chuyển sang trái, hút bụi bẩn hoặc không làm gì cả.² Một chức năng tác nhân rất đơn giản như sau: nếu hình vuông hiện tại bị bẩn, thì hút; nếu không, hãy di chuyển đến ô vuông khác. Một bằng một phần của chức năng thisagent được hiển thị trong Hình 2.3 và một chương trình tác nhân thực hiện nó xuất hiện trong Hình 2.8 trên trang 67.

Nhìn vào Hình 2.3, chúng ta thấy rằng các tác nhân thế giới chân không khác nhau có thể được xác định đơn giản bằng cách điền vào cột bên phải theo nhiều cách khác nhau. Câu hỏi rõ ràng, sau đó, là: Jis là cách đúng đắn để điền vào bằng? Nói cách khác, điều gì làm cho một tác nhân tốt hay xấu, Thông minh hay ngu ngốc? Chúng tôi trả lời những câu hỏi này trong phần tiếp theo.

¹ Nếu tác nhân sử dụng một số ngẫu nhiên hóa để chọn các hành động của nó, thì chúng ta sẽ phải thử từng chuỗi nhiều lần để xác định xác suất của mỗi hành động. Người ta có thể tưởng tượng rằng hành động ngẫu nhiên là khá ngớ ngẩn, nhưng chúng ta sẽ chỉ ra ở phần sau của chương này rằng nó có thể rất thông minh.² Trong một robot thực sự, nó sẽ không có những hành động như "di chuyển sang phải" và "di chuyển sang trái". Thay vào đó, các hành động sẽ là "quay bánh xe về phía trước" và "quay bánh xe lùi lại". Chúng tôi đã chọn các hành động để dễ theo dõi hơn trên trang, không phải để dễ thực hiện trong một robot thực tế.



Hình 2.2 Một thể giới máy hút bụi chỉ có hai địa điểm. Mỗi vị trí có thể sạch hoặc bẩn, và tác nhân có thể di chuyển sang trái hoặc phải và có thể làm sạch hình vuông mà nó chiếm giữ. Các phiên bản khác nhau của thể giới chặn không cho phép các quy tắc khác nhau về những gì tác nhân có thể nhận thức, liệu hành động của nó có luôn thành công hay không, v.v.

Trình tự nhận thức	Hành động
[A, Sạch sẽ]	Bên phải
[A, bẩn]	Bú
[B, Làm sạch]	Bên trái
[B, Bẩn]	Bú
[A, Sạch sẽ], [A, Sạch sẽ]	Bên phải
[A, Sạch], [A, Bẩn]	Bú
...	...
[A, Sạch sẽ], [A, Sạch], [A, Sạch sẽ]	Bên phải
[A, Sạch], [A, Sạch], [A, Bẩn]	Bú
...	...

Hình 2.3 Lập bảng một phần của một hàm tác nhân đơn giản cho thể giới máy hút bụi được hiển thị trong Hình 2.2. Tác nhân làm sạch hình vuông hiện tại nếu nó bị bẩn, nếu không nó sẽ di chuyển đến hình vuông khác. Lưu ý rằng bảng có kích thước không giới hạn trừ khi có giới hạn về độ dài của các chuỗi nhận thức có thể có.

Trước khi kết thúc phần này, chúng ta nên nhấn mạnh rằng khái niệm tác nhân có nghĩa là một công cụ để phân tích các hệ thống, không phải là một đặc điểm tuyệt đối phân chia thế giới thành các tác nhân và không tác nhân. Người ta có thể xem một máy tính cầm tay như một tác nhân chọn hành động hiển thị "4" khi cho đây tri giác " $2 + 2 =$ ", nhưng một phân tích như vậy sẽ khó giúp chúng ta hiểu được máy tính. Theo một nghĩa nào đó, tất cả các lĩnh vực kỹ thuật có thể được coi là thiết kế các hiện vật tương tác với thế giới; AI hoạt động ở mức cuối thú vị nhất của quang phổ, nơi các hiện vật có tài nguyên tính toán đáng kể và môi trường nhiệm vụ đòi hỏi việc ra quyết định không tầm thường.

2.2 Hành vi tốt: Khái niệm về tính hợp lý

Một tác nhân hợp lý là một tác nhân làm điều đúng đắn. Rõ ràng, làm điều đúng đắn là tốt hơn hơn là làm điều sai trái, nhưng làm điều đúng có nghĩa là gì?

Tác nhân hợp lý

2.2.1 Các biện pháp hiệu suất

Triết học đạo đức đã phát triển một số khái niệm khác nhau về "điều đúng đắn", nhưng AI thường gắn bó với một khái niệm gọi là chủ nghĩa hậu quả: chúng ta đánh giá hành vi của một tác nhân bằng

Chủ nghĩa hậu quả

Hậu quả. Khi một tác nhân bị lao xuống một môi trường, nó tạo ra một chuỗi hành động theo nhận thức mà nó nhận được. Chuỗi hành động này khiến môi trường trải qua một chuỗi các trạng thái. Nếu trình tự là mong muốn, thì tác nhân đã hoạt động tốt. Khái niệm về sự mong muốn này được nắm bắt bởi một thước đo hiệu suất đánh giá bất kỳ

Đo lường hiệu suất

trình tự trạng thái môi trường nhất định.

Con người có những mong muốn và sở thích của riêng họ, vì vậy khái niệm về tính hợp lý được áp dụng cho con người có liên quan đến sự thành công của họ trong việc lựa chọn các hành động tạo ra các chuỗi trạng thái môi trường được mong muốn theo quan điểm của họ. Mặt khác, máy móc không có mong muốn và sở thích của riêng chúng; Thước đo hiệu suất, ít nhất là ban đầu, nằm trong tâm trí của người thiết kế máy, hoặc trong tâm trí của người dùng mà máy được thiết kế. Chúng ta sẽ thấy rằng một số thiết kế tác nhân có một biểu diễn rõ ràng của (một phiên bản của) thước đo performance, trong khi trong các thiết kế khác, thước đo hiệu suất hoàn toàn ngầm – agent có thể làm điều đúng, nhưng nó không biết tại sao.

Nhớ lại cảnh báo của Norbert Wiener để đảm bảo rằng "mục đích được đưa vào máy là mục đích mà chúng ta thực sự mong muốn" (trang 51), lưu ý rằng có thể khá khó để đo lường hiệu suất trả một cách chính xác. Ví dụ, hãy xem xét tác nhân hút bụi từ phần trước. Chúng tôi có thể đề xuất đo lường hiệu suất bằng số lượng bụi bẩn được làm sạch trong một ca làm việc tám giờ. Với một tác nhân hợp lý, tất nhiên, những gì bạn yêu cầu là những gì bạn nhận được. Một tác nhân hợp lý có thể tối đa hóa biện pháp hiệu suất này bằng cách làm sạch bụi bẩn, sau đó đổ tất cả xuống sàn, sau đó làm sạch lại, v.v. Một biện pháp phù hợp hơn sẽ thưởng cho đại lý vì đã có một sàn nhà sạch sẽ. Ví dụ, một điểm có thể được trao cho mỗi ô vuông sạch ở mỗi bước thời gian (có thể với hình phạt cho điện tử ba được tiêu thụ và nhiễu được tạo ra). Theo nguyên tắc chung, tốt hơn là nên thiết kế các biện pháp hiệu suất theo những gì người ta thực sự muốn đạt được trong môi trường, thay vì

hơn là theo cách người ta nghĩ tác nhân nên cư xử.

Ngay cả khi tránh được những cạm bẫy rõ ràng, một số vấn đề khó khăn vẫn còn. Ví dụ, khái niệm "sàn sạch" trong đoạn trước dựa trên độ sạch trung bình ngoài giờ. Tuy nhiên, hai tác nhân khác nhau có thể đạt được độ sạch trung bình tương tự, một trong số đó luôn làm một công việc tầm thường trong khi tác nhân kia làm sạch năng lượng nhưng nghỉ ngơi lâu. Cái nào thích hợp hơn có vẻ là một điểm tốt của khoa học về sinh, nhưng trên thực tế, nó là một câu hỏi triết học sâu sắc với những hàm ý sâu rộng. Cái nào tốt hơn - một cuộc sống liều lĩnh của những thăng trầm, hay một cuộc sống an toàn nhưng khiêm tốn? Cái nào tốt hơn - một nền kinh tế mà mọi người sống trong nghèo đói vừa phải, hay một nền kinh tế mà một số người sống nhiều trong khi những người khác rất nghèo? Chúng tôi để những câu hỏi này như một bài tập cho những người đọc siêng năng.

Đối với hầu hết cuốn sách, chúng ta sẽ giả định rằng thước đo hiệu suất có thể được chỉ định chính xác. Tuy nhiên, vì những lý do được đưa ra ở trên, chúng ta phải chấp nhận khả năng rằng chúng ta có thể đặt mục đích sai lầm vào máy móc - chính xác là vấn đề King Midas được mô tả trên

trang 51. Hơn nữa, khi thiết kế một phần mềm, các bản sao của phần mềm sẽ thuộc về những người dùng khác nhau, chúng tôi không thể dự đoán được sở thích chính xác của từng người dùng. Do đó, chúng ta có thể cần xây dựng các tác nhân phản ánh sự không chắc chắn ban đầu về thước đo hiệu suất thực sự và tìm hiểu thêm về nó khi thời gian trôi qua; những tác nhân như vậy được mô tả trong Chương 15, 17 và 23.

2.2.2 Tính hợp lý

Điều gì là hợp lý tại bất kỳ thời điểm nào phụ thuộc vào bốn

điều:

- Thước đo hiệu suất xác định tiêu chí thành công.

- Kiến thức trước của đại lý về môi trường.
- Các hành động mà tác nhân có thể thực hiện.

- Trình tự nhận thức của tác nhân cho đến nay. Điều

này dẫn đến một định nghĩa về một tác nhân hợp lý:

Định nghĩa tác nhân vô lý I

Đối với mỗi trình tự tri giác có thể có, một tác nhân hợp lý nên chọn một hành động được expected để tối đa hóa thước đo hiệu suất của nó, dựa trên bằng chứng được cung cấp bởi trình tự tri giác và bất kỳ kiến thức tích hợp nào mà tác nhân có.

Hãy xem xét chất làm sạch máy hút bụi đơn giản làm sạch một hình vuông nếu nó bị bẩn và di chuyển đến hình vuông khác nếu không; đây là hàm tác nhân được lập bảng trong Hình 2.3. Đây có phải là một tác nhân hợp lý không? Điều đó phụ thuộc! Đầu tiên, chúng ta cần nói thước đo hiệu suất là gì, những gì được biết về môi trường và những cảm biến và bộ truyền động mà tác nhân có. Chúng ta hãy giả định như sau:

- Thước đo hiệu suất trao một điểm cho mỗi ô vuông sạch ở mỗi bước thời gian, trong "vòng đời" của 1000 bước thời gian.
- "Địa lý" của môi trường được biết đến tiên nghiệm (Hình 2.2) nhưng sự phân bố bụi bẩn và vị trí ban đầu của tác nhân thì không. Các ô vuông sạch sẽ luôn sạch sẽ và hút làm sạch hình vuông hiện tại. Các hành động Right và Left di chuyển tác nhân một ô vuông ngoại trừ khi điều này sẽ đưa tác nhân ra ngoài môi trường, trong trường hợp đó tác nhân vẫn ở vị trí của nó.
- Các hành động duy nhất có sẵn là Phải, Trái và Mút.
- Tác nhân nhận thức chính xác vị trí của nó và liệu vị trí đó có chứa bụi bẩn hay không. Trong những trường hợp này, tác nhân thực sự hợp lý; hiệu suất dự kiến của nó ít nhất cũng tốt như bất kỳ đại lý nào khác. Người ta có thể dễ dàng thấy rằng cùng một tác nhân sẽ không hợp lý trong những hoàn cảnh khác nhau. Ví dụ, một khi tất cả bụi bẩn được làm sạch, tác nhân sẽ dao động qua lại một cách không cần thiết; Nếu thước đo hiệu suất bao gồm một hình phạt một điểm cho mỗi chuyển động, đại lý sẽ hoạt động kém. Một đại lý tốt hơn cho trường hợp này sẽ không làm gì khi chắc chắn rằng tất cả các hình vuông đều sạch sẽ. Nếu các hình vuông sạch có thể bị bẩn trở lại, tác nhân thỉnh thoảng nên kiểm tra và làm sạch lại chúng nếu cần. Nếu không biết địa lý của môi trường, tác nhân sẽ cần phải khám phá nó. Bài tập 2.VACR yêu cầu bạn thiết kế đại lý cho những trường hợp này.

2.2.3 Toàn trí, học tập và tự chủ

Chúng ta cần phải cẩn thận để phân biệt giữa lý trí và toàn trí. Một người toàn trí tác nhân biết kết quả thực tế của hành động của mình và có thể hành động phù hợp; nhưng toàn trí là không thể trong thực tế. Hãy xem xét ví dụ sau: Một ngày nọ, tôi đang đi bộ dọc theo đại lộ ChampsElyses và tôi thấy một người bạn cũ bên kia đường. Không có giao thông gần đó và tôi

không tham gia, vì vậy, lý trí, tôi bắt đầu băng qua đường. Trong khi đó, ở độ cao 33.000 feet, một cửa chờ hàng rơi ra khỏi một chiếc máy bay chở khách đi qua, 3 và trước khi tôi đến được phía bên kia của con đường, tôi đã bị san phẳng. Tôi có vô lý khi băng qua đường không? Không có khả năng cáo phó của tôi sẽ đọc "Kể ngốc cổ gắng băng qua đường".

Ví dụ này cho thấy lý trí không giống như sự hoàn hảo. Tính hợp lý tối đa hóa hiệu suất mong đợi, trong khi sự hoàn hảo tối đa hóa hiệu suất thực tế. Rút lui khỏi yêu cầu của sự hoàn hảo không chỉ là vấn đề công bằng với các tác nhân. Vấn đề là nếu chúng ta mong đợi một tác nhân làm những gì hóa ra sau thực tế là hành động tốt nhất, thì sẽ không thể thiết kế một tác nhân để đáp ứng đặc điểm kỹ thuật này - trừ khi chúng ta cải thiện hiệu suất của các quả cầu pha lê hoặc cỗ máy thời gian.

Do đó, định nghĩa của chúng ta về tính hợp lý không đòi hỏi sự toàn trí, bởi vì sự lựa chọn hợp lý chỉ phụ thuộc vào trình tự tri giác cho đến nay. Chúng ta cũng phải đảm bảo rằng chúng ta đã vô tình cho phép tác nhân tham gia vào các hoạt động kém thông minh. Ví dụ, nếu một đặc vụ không nhìn cả hai chiều trước khi băng qua một con đường đông đúc, thì trình tự nhận thức của nó sẽ không cho nó biết rằng có một chiếc xe tải lớn đang đến với tốc độ cao. Định nghĩa của chúng ta về tính hợp lý có nói rằng bây giờ có thể băng qua đường không? Còn xa nào!

Thứ nhất, sẽ không hợp lý khi băng qua đường với trình tự nhận thức thiếu thông tin này: nguy cơ tai nạn do băng qua mà không nhìn là quá lớn. Thứ hai, một tác nhân lý trí nên chọn hành động "nhìn" trước khi bước ra đường, bởi vì nhìn giúp tối đa hóa hiệu suất mong đợi. Thực hiện các hành động để sửa đổi các nhận thức trong tương lai - đôi khi được gọi là thu thập thông tin - là một phần quan trọng của tính hợp lý và được đề cập sâu trong

Thu thập thông tin

Chương 15. Ví dụ thứ hai về thu thập thông tin được cung cấp bởi cuộc thăm dò phải được thực hiện bởi một chất làm sạch chân không trong một môi trường đông đúc chưa được biết đến.

Định nghĩa của chúng ta đòi hỏi một tác nhân hợp lý không chỉ để thu thập thông tin mà còn phải học như Học

càng nhiều càng tốt từ những gì nó nhận thức. Cấu hình ban đầu của tác nhân có thể phản ánh một số kiến thức trước đó về môi trường, nhưng khi tác nhân có được kinh nghiệm, điều này có thể được sửa đổi và tăng cường. Có những trường hợp cực đoan trong đó môi trường hoàn toàn được biết trước và hoàn toàn có thể dự đoán được. Trong những trường hợp như vậy, tác nhân không cần phải nhận thức hoặc học hỏi; nó chỉ đơn giản là hoạt động chính xác.

Tất nhiên, những tác nhân như vậy rất mong manh. Hãy xem xét loài bọ phân thấp kém. Sau khi đào tổ và đẻ trứng, nó lấy một quả bóng phân từ một đồng gần đó để bịt lối vào. Nếu quả bóng phân bị lấy ra khỏi tầm tay của nó trên đường đi, bọ cánh cứng sẽ tiếp tục nhiệm vụ của nó và kích cắm bịt tổ bằng quả bóng phân không tồn tại, không bao giờ nhận thấy rằng nó bị mất. Sự tiến hóa đã xây dựng một giả định vào hành vi của bọ cánh cứng, và khi nó bị vi phạm, hành vi không thành công sẽ dẫn đến.

Thông minh hơn một chút là ong bắp cày sphex. Con cái sẽ đào hang, đi ra ngoài và đốt sâu bướm và kéo nó vào hang, vào hang một lần nữa để kiểm tra mọi thứ tốt, kéo sâu bướm vào bên trong và đẻ trứng. Sâu bướm đóng vai trò như một nguồn thức ăn khi trứng nở. Cho đến nay vẫn tốt, nhưng nếu một nhà côn trùng học di chuyển sâu bướm ra xa vài inch trong khi phex đang thực hiện kiểm tra, nó sẽ trở lại bước "kéo sâu bướm" trong kế hoạch của nó và sẽ tiếp tục kế hoạch mà không cần sửa đổi, kiểm tra lại hang, ngay cả sau hàng chục can thiệp di chuyển sâu bướm. Sphex không thể biết rằng kế hoạch bẩm sinh của nó đang thất bại, và do đó sẽ không thay đổi nó.

3 Xem N. Henderson, "New door latches urged for Boeing 747 jumbo jets," Washington Post, ngày 24 tháng 8 năm 1989.

Quyền tự trị

Trong phạm vi mà một tác nhân dựa vào kiến thức trước của người thiết kế của nó hơn là vào nhận thức và quá trình học tập của chính nó, chúng ta nói rằng tác nhân thiếu quyền tự chủ. Một tác nhân hợp lý

nên tự chủ — nó nên học những gì nó có thể để bù đắp cho kiến thức trước đó một phần hoặc không chính xác. Ví dụ, một chất làm sạch chân không học cách dự đoán vị trí và thời điểm bụi bẩn sẽ xuất hiện sẽ hoạt động tốt hơn một chất không có. Như một vấn đề thực tế, người ta hiếm khi yêu cầu quyền tự chủ hoàn toàn ngay từ đầu: khi tác nhân có ít hoặc không có kinh nghiệm, nó sẽ phải hành động ngẫu nhiên trừ khi nhà thiết kế hỗ trợ. Cũng giống như sự tiến hóa cung cấp cho động vật đủ phản xạ tích hợp để tồn tại đủ lâu để tự học, sẽ là hợp lý để cung cấp cho một nhân viên trí tuệ nhân tạo một số kiến thức ban đầu cũng như khả năng học hỏi. Sau khi trải nghiệm đầy đủ về môi trường của nó, hành vi của một tác nhân hợp lý có thể trở nên độc lập một cách hiệu quả với kiến thức trước đó của nó. Do đó, sự kết hợp của học tập cho phép người ta thiết kế một tác nhân duy lý duy nhất sẽ thành công trong rất nhiều môi trường khác nhau.

2.3 Bản chất của môi trường

Môi trường nhiệm vụ

Bây giờ chúng ta đã có một định nghĩa về tính hợp lý, chúng ta gần như đã sẵn sàng để nghĩ về việc xây dựng các tác nhân hợp lý. Tuy nhiên, trước tiên, chúng ta phải nghĩ về môi trường nhiệm vụ, vốn là cơ sở.

Về mặt là những "vấn đề" mà các tác nhân hợp lý là "giải pháp". Chúng tôi bắt đầu bằng cách chỉ ra cách chỉ định môi trường tác vụ, minh họa quy trình với một số ví dụ. Sau đó, chúng tôi chỉ ra rằng môi trường tác vụ có nhiều hương vị khác nhau. Bản chất của môi trường nhiệm vụ ảnh hưởng trực tiếp đến thiết kế phù hợp cho chương trình tác nhân.

2.3.1 Chỉ định môi trường tác vụ

PEAS

Trong cuộc thảo luận của chúng tôi về tính hợp lý của tác nhân hút bụi đơn giản, chúng tôi phải chỉ định thước đo hiệu suất, môi trường và các thiết bị truyền động và cảm biến của tác nhân. Chúng tôi nhóm tất cả những thứ này dưới tiêu đề của môi trường tác vụ. Đối với những người có đầu óc viết tắt, chúng tôi gọi đây là mô tả PEAS (Hiệu suất, Môi trường, Thiết bị truyền động, Cảm biến). Khi thiết kế một

Agent, bước đầu tiên phải luôn là chỉ định môi trường tác vụ đầy đủ nhất có thể. Thế giới chân không là một ví dụ đơn giản; Chúng ta hãy xem xét một vấn đề phức tạp hơn: một tài xế taxi động. Hình 2.4 tóm tắt mô tả PEAS cho môi trường nhiệm vụ của taxi. Chúng ta thảo luận chi tiết hơn về từng yếu tố trong các đoạn sau. Đầu tiên, thước đo hiệu suất mà chúng ta muốn tài xế tự động của mình mong muốn là gì? Những phẩm chất mong muốn bao gồm đến đúng đích; giảm thiểu tiêu hao nhiên liệu và hao mòn; giảm thiểu thời gian hoặc chi phí chuyển đi; giảm thiểu vi phạm luật giao thông và gây rối cho người lái xe khác; tối đa hóa sự an toàn và sự thoải mái của hành khách; tối đa hóa lợi nhuận. Rõ ràng, một số mục tiêu này mâu thuẫn, vì vậy sẽ cần phải đánh đổi.

Tiếp theo, môi trường lái xe mà taxi sẽ phải đối mặt như thế nào? Bất kỳ tài xế taxi nào cũng phải đối phó với nhiều con đường khác nhau, từ làn đường nông thôn và ngõ hẻm đô thị đến đường cao tốc 12 làn xe. Những con đường có phương tiện giao thông khác, người đi bộ, động vật đi lạc, công trình đường bộ, xe cảnh sát, vũng nước và ổ gà. Taxi cũng phải tương tác với hành khách tiềm năng và thực tế. Ngoài ra còn có một số lựa chọn tùy chọn. Taxi có thể cần phải hoạt động ở Nam California, nơi tuyết hiếm khi có vấn đề, hoặc ở Alaska, nơi hiếm khi không. Nó có thể luôn luôn lái bên phải, hoặc chúng ta có thể muốn nó đủ linh hoạt để lái xe bên trái khi ở Anh hoặc Nhật Bản.

Loại đại lý	Hiệu suấtĐo lường	Môi trường	Thiết bị truyền động	Sensors
Tài xế taxi	An toàn, nhanh chóng, hợp pháp, thoải mái, tối đa hóa lợi nhuận, giảm thiểu tác động đến những người tham gia giao thông khác	Đường xá, giao thông khác, cảnh sát, người đi bộ, khách hàng, thời tiết	Hệ thống lái, chân ga, phanh, tín hiệu, còi, màn hình, giọng nói	Máy ảnh, radar, đồng hồ tốc độ, GPS, cảm biến động cơ, gia tốc kế, micro, màn hình cảm ứng

Hình 2.4 Mô tả PEAS về môi trường tác vụ cho tài xế taxi tự động.

Các bộ truyền động cho taxi tự động bao gồm những thiết bị có sẵn cho người lái xe: điều khiển động cơ thông qua chân ga và điều khiển tay lái và phanh. Ngoài ra, nó sẽ cần xuất ra màn hình hiển thị hoặc bộ tổng hợp giọng nói để nói chuyện lại với hành khách, và có thể là một số cách để giao tiếp với các phương tiện khác, lịch sự hoặc cách khác.

Các cảm biến cơ bản cho taxi sẽ bao gồm một hoặc nhiều máy quay video để nó có thể nhìn thấy, cũng như cảm biến lidar và siêu âm để phát hiện khoảng cách đến các xe khác và chướng ngại vật. Để tránh vé quá tốc độ, taxi nên có đồng hồ tốc độ và để điều khiển xe đúng cách, đặc biệt là trên các khúc cua, nó phải có gia tốc kế. Để xác định trạng thái cơ học của xe, nó sẽ cần một dây cảm biến động cơ, nhiên liệu và hệ thống điện thông thường. Giống như nhiều người lái xe, nó có thể muốn truy cập tín hiệu GPS để nó không bị lạc. Cuối cùng, nó sẽ cần màn hình cảm ứng hoặc đầu vào bằng giọng nói để hành khách yêu cầu điểm đến.

Trong Hình 2.5, chúng ta đã phân tích các yếu tố PEAS cơ bản cho một số loại chất bổ sung.

Các ví dụ khác xuất hiện trong Bài tập 2.PEAS. Các ví dụ bao gồm môi trường vật lý cũng như ảo. Lưu ý rằng môi trường tác vụ ảo có thể phức tạp như thế giới "thực": ví dụ: một tác nhân phần mềm (hoặc robot phần mềm hoặc softbot) giao dịch

Tác nhân phần mềm

trên các trang web đấu giá và bán lại giao dịch với hàng triệu người dùng khác và hàng tỷ đối tượng, nhiều người có hình ảnh thật.

Softbot

2.3.2 Thuộc tính của môi trường tác vụ

Phạm vi môi trường tác vụ có thể phát sinh trong AI rõ ràng là rất lớn. Tuy nhiên, chúng ta có thể xác định một số lượng khá nhỏ các chiều mà môi trường nhiệm vụ có thể được phân loại. Các khía cạnh này xác định, ở một mức độ lớn, thiết kế tác nhân thích hợp và khả năng áp dụng của từng họ kỹ thuật chính để triển khai tác nhân. Đầu tiên, chúng tôi liệt kê các khía cạnh, sau đó chúng tôi phân tích một số môi trường tác vụ để minh họa các ý tưởng. Các định nghĩa ở đây là không chính thức; các chương sau cung cấp các tuyên bố và ví dụ chính xác hơn về từng loại môi trường.

Có thể quan sát được hoàn toàn so với có thể quan sát được một phần: Nếu cảm biến của tác nhân cho phép nó truy

Có thể quan sát được một phầnCó thể quan sát được một phần

trạng thái hoàn chỉnh của môi trường tại mỗi thời điểm, sau đó chúng ta nói rằng môi trường nhiệm vụ hoàn toàn có thể quan sát được. Môi trường nhiệm vụ có thể quan sát được một cách hiệu quả nếu các cảm biến phát hiện tất cả các khía cạnh liên quan đến việc lựa chọn hành động; mức độ liên quan, đến lượt nó, phụ thuộc vào

Loại đại lý	Hiệu suấtĐo lường	Môi trường	Thiết bị truyền động	Sensors
Hệ thống chẩn đoán y tế	Bệnh nhân khỏe mạnh, giảm chi phí	Bệnh nhân, bệnh viện, nhân viên	Hiện thị câu hỏi, xét nghiệm, chẩn đoán, phương pháp điều trị	Màn hình cảm ứng/giọng nóinhập các triệu chứng và phát hiện
Hệ thống phân tích hình ảnh vệ tinh	Đúng phân loại đối tượng, địa hình	Vệ tinh quỹ đạo, đường xuống, thời tiết	Hiện thị phân loại cảnh	Máy ảnh kỹ thuật số độ phân giải cao
Robot chọn bộ phận	Tỷ lệ phần trăm các bộ phận trong thùng sửa chữa	Băng tải với các bộ phận; Thùng	Cánh tay và bàn tay khớp nối	Máy ảnh, cảm biến góc xúc giác và khớp
Bộ điều khiển nhà máy lọc dầu	Độ tinh khiết, năng suất, an toàn	Nhà máy lọc dầu, nguyên liệu thô, người vận hành	Van, máy bơm, lò sưởi, máy khuấy, màn hình	Cảm biến nhiệt độ, áp suất, lưu lượng, hóa học
Gia sư tiếng Anh tương tác	Bài kiểm tra điểm số của học sinh	Bộ sinh viên, cơ quan kiểm tra	Hiện thị bài tập, phản hồi, bài phát biểu	Nhập bàn phím, giọng nói

Hình 2.5 Ví dụ về các loại tác nhân và mô tả PEAS của chúng.

đo lường hiệu suất. Môi trường hoàn toàn có thể quan sát được rất thuận tiện vì tác nhân không cần duy trì bất kỳ trạng thái nội bộ nào để theo dõi thế giới. Một môi trường có thể được quan sát một phần vì các cảm biến nhiễu và không chính xác hoặc vì các phần của trạng thái chỉ đơn giản là bị thiếu trong dữ liệu cảm biến — ví dụ, một tác nhân chân không chỉ có cảm biến bụi bản cục bộ không thể biết liệu có bụi bản ở các ô vuông khác hay không và một chiếc taxi tự động không thể nhìn thấy những gì những người lái xe khác đang nghĩ. Nếu tác nhân không có cảm biến nào cả, thì môi trường không thể quan sát được. Người ta có thể nghĩ rằng trong những trường hợp như vậy, hoàn cảnh của đặc vụ là vô vọng, nhưng, như chúng ta thảo luận trong

Không thể quan sát được

Chương 4, mục tiêu của tác nhân vẫn có thể đạt được, đôi khi chắc chắn.

Một tác nhânMultiagent

Một tác nhân so với đa tác nhân: Sự khác biệt giữa một tác nhân và đa tác nhân environments có vẻ đủ đơn giản. Ví dụ: một đặc vụ tự giải ô chữ rõ ràng là trong môi trường một đặc vụ, trong khi một đặc vụ chơi cờ vua ở trong môi trường hai đặc vụ. Tuy nhiên, có một số vấn đề tinh tế. Đầu tiên, chúng tôi đã mô tả cách một thực thể có thể được xem như một tác nhân, nhưng chúng tôi đã không giải thích những thực thể nào phải được xem là tác nhân. Một tác nhân A (ví dụ như tài xế taxi) có phải coi một đối tượng B (một phương tiện khác) như một tác nhân, hay nó có thể được coi đơn thuần là một đối tượng hoạt động theo các định luật vật lý, tương tự như sóng ở bãi biển hoặc lá cây thổi trong gió? Sự khác biệt chính là liệu hành vi của B có được mô tả tốt nhất là tối đa hóa một thước đo hiệu suất mà giá trị của nó phụ thuộc vào hành vi của tác nhân A hay không.

Ví dụ, trong cờ vua, đối thủ B đang cố gắng tối đa hóa hiệu suất của nó, theo quy tắc của cờ vua, giảm thiểu thước đo hiệu suất của tác nhân A. Do đó, cờ vua là tác môi trường đa tác nhân cạnh tranh. Mặt khác, trong môi trường lái xe taxi,

Cạnh tranh

Tránh va chạm tối đa hóa thước đo hiệu suất của tất cả các tác nhân, vì vậy nó là một môi trường đa tác nhân hợp tác một phần. Nó cũng cạnh tranh một phần bởi vì, ví dụ, chỉ

Hợp tác xã

Một chiếc xe có thể chiếm một chỗ đậu xe.

Các vấn đề thiết kế tác nhân trong môi trường đa tác nhân thường khá khác so với các vấn đề trong môi trường đơn tác nhân; ví dụ, giao tiếp thường xuất hiện như một hành vi hợp lý trong môi trường đa tác nhân; Trong một số môi trường cạnh tranh, hành vi ngẫu nhiên là hợp lý vì nó tránh được những cạm bẫy của khả năng dự đoán.

Xác định so với không xác định. Nếu trạng thái tiếp theo của môi trường hoàn toàn được xác định bởi trạng thái hiện tại và hành động được thực hiện bởi (các) tác nhân, thì chúng ta nói môi trường là xác định; nếu không, nó là không xác định. Về nguyên tắc, một tác nhân không cần phải lo lắng về sự không chắc chắn trong một môi trường xác định, hoàn toàn có thể quan sát được. Tuy nhiên, nếu môi trường có thể quan sát được một phần, thì nó có thể có vẻ không xác định.

Xác địnhKhông xác định

Hầu hết các tình huống thực tế đều phức tạp đến mức không thể theo dõi tất cả các khía cạnh không quan sát; vì mục đích thực tế, chúng phải được coi là không xác định. Lái xe taxi rõ ràng là không xác định theo nghĩa này, bởi vì người ta không bao giờ có thể dự đoán chính xác hành vi của giao thông; Hơn nữa, lốp xe của một người có thể bị nổ đột ngột và động cơ của một người có thể bị giật mà không báo trước. Thế giới chân không như chúng tôi mô tả là xác định, nhưng các biến thể có thể bao gồm các yếu tố không xác định như bụi bẩn xuất hiện ngẫu nhiên và cơ chế hút không đáng tin cậy (Bài tập 2.VFIN).

Một lưu ý cuối cùng: từ ngẫu nhiên được một số người sử dụng như một từ "không xác định".

Ngẫu nhiên

nhưng chúng tôi phân biệt giữa hai thuật ngữ; Chúng ta nói rằng một mô hình môi trường là ngẫu nhiên nếu nó xử lý rõ ràng các xác suất (ví dụ: "có 25% khả năng mưa vào ngày mai") và "không xác định" nếu các khả năng được liệt kê mà không được định lượng (ví dụ: "Có khả năng mưa vào ngày mai").

Từng đợt so với tuần tự: Trong môi trường nhiệm vụ theo từng đợt, trải nghiệm của tác nhân là khác nhau được vided thành các tập nguyên tử. Trong mỗi tập, tác nhân nhận được một nhận thức và sau đó thực hiện một hành động duy nhất. Điều quan trọng là tập tiếp theo đồng nghĩa với "không xác định". Nhiều nhiệm vụ phân loại là từng đợt. Ví dụ, một đại lý phải phát hiện các bộ phận bị lỗi trên dây chuyền lắp ráp dựa trên từng quyết định dựa trên bộ phận hiện tại, bất kể các quyết định trước đó; Hơn nữa, quyết định hiện tại không ảnh hưởng đến việc phân tiếp theo có bị lỗi hay không. Mặt khác, trong môi trường tuần tự, quyết định hiện tại có thể ảnh hưởng đến tất cả các quyết định trong tương lai.⁴ Cờ vua và lái xe taxi là tuần tự: trong cả hai trường hợp, hành động ngắn hạn có thể gây ra hậu quả lâu dài. Môi trường theo từng giai đoạn đơn giản hơn nhiều so với môi trường tuần tự vì tác nhân không cần phải suy nghĩ trước.

Tuần tự theo từng tập

Tĩnh so với động: Nếu môi trường có thể thay đổi trong khi một tác nhân đang cân nhắc, thì Chúng tôi nói rằng môi trường là năng động đối với tác nhân đó; nếu không, nó là tĩnh. Môi trường tĩnh rất dễ đối phó vì tác nhân không cần phải tiếp tục nhìn vào thế giới trong khi nó đang quyết định một hành động, cũng như không cần phải lo lắng về thời gian trôi qua. Mặt khác, môi trường năng động liên tục hỏi tác nhân muốn làm gì; nếu nó chưa quyết định,

TĩnhĐộng⁹

⁴ Từ "tuần tự" cũng được sử dụng trong khoa học máy tính như là từ trái nghĩa của "song song". Hai ý nghĩa phần lớn không liên quan đến nhau.

Bán động lực

Điều đó được coi là quyết định không làm gì cả. Nếu bản thân môi trường không thay đổi theo thời gian trôi qua nhưng điểm hiệu suất của tác nhân thì có, thì chúng ta nói môi trường là bán động lực. Lái xe taxi rõ ràng là năng động: những chiếc xe khác và bản thân taxi tiếp tục di chuyển trong khi thuật toán lái xe không biết phải làm gì tiếp theo. Cờ vua, khi chơi với đồng hồ, là bán động. Trò chơi ô chữ là tĩnh.

Rời
rạcContinuous

Rời rạc so với liên tục: Sự phân biệt rời rạc / liên tục áp dụng cho trạng thái của môi trường, cách xử lý thời gian, và nhận thức và hành động của tác nhân. Ví dụ, môi trường cờ vua có một số trạng thái riêng biệt hữu hạn (không bao gồm đồng hồ). Cờ vua cũng có một tập hợp các nhận thức và hành động rời rạc. Lái xe taxi là một vấn đề trạng thái liên tục và thời gian liên tục: tốc độ và vị trí của taxi và của các phương tiện khác quét qua một loạt các giá trị liên tục và làm như vậy trơn tru theo thời gian. Các hành động lái xe taxi cũng liên tục (góc lái, v.v.). Đầu vào từ máy ảnh kỹ thuật số là rời rạc, nói một cách nghiêm ngặt, nhưng thường được coi là đại diện cho cường độ và vị trí khác nhau liên tục.

Đã biếtKhông
xác định

Đã biết so với chưa biết: Nói một cách chính xác, sự khác biệt này không đề cập đến môi trường nhưng theo trạng thái kiến thức của tác nhân (hoặc nhà thiết kế) về "định luật vật lý" của môi trường. Trong một môi trường đã biết, kết quả (hoặc xác suất kết quả nếu môi trường không xác định) cho tất cả các hành động được đưa ra. Rõ ràng, nếu môi trường không xác định, tác nhân sẽ phải tìm hiểu cách thức hoạt động của nó để đưa ra quyết định đúng đắn.

Sự khác biệt giữa môi trường đã biết và chưa biết không giống như môi trường giữa môi trường có thể quan sát được hoàn toàn và một phần. Rất có thể một môi trường đã biết có thể quan sát được một phần - ví dụ, trong các trò chơi bài solitaire, tôi biết các quy tắc nhưng vẫn không thể nhìn thấy những lá bài chưa được lật lại. Ngược lại, một môi trường chưa biết có thể được quan sát hoàn toàn — trong một trò chơi điện tử mới, màn hình có thể hiển thị trạng thái toàn bộ trò chơi nhưng tôi vẫn không biết các nút làm gì cho đến khi tôi thử chúng.

Như đã lưu ý ở trang 57, bản thân thước đo hiệu suất có thể không được biết, hoặc vì nhà thiết kế không chắc chắn làm thế nào để viết nó ra một cách chính xác hoặc vì người dùng cuối cùng - những sở thích quan trọng - không được biết. Ví dụ, một tài xế taxi thường sẽ không biết liệu một hành khách mới thích một hành trình nhàn nhã hay tốc độ, một phong cách lái xe thận trọng hay hung hăng. Một trợ lý cá nhân ảo bắt đầu không biết gì về sở thích cá nhân của chủ sở hữu mới của nó. Trong những trường hợp như vậy, tổng đài viên có thể tìm hiểu thêm về thước đo hiệu suất dựa trên các tương tác tiếp theo với nhà thiết kế hoặc người dùng. Điều này cho thấy rằng môi trường tác vụ nhất thiết phải được xem như một môi trường đã tác nhân.

Trường hợp khó nhất là có thể quan sát được một phần, đa tác nhân, không xác định, tuần tự, dynamic, liên tục và không biết. Lái xe taxi rất khó theo tất cả các nghĩa này, ngoại trừ môi trường của người lái xe hầu hết được biết đến. Lái xe thuê ở một quốc gia mới có địa lý xa lạ, luật giao thông khác nhau và hành khách lo lắng sẽ thú vị hơn rất nhiều.

Hình 2.6 liệt kê các thuộc tính của một số môi trường quen thuộc. Lưu ý rằng các vật dụng không phải lúc nào cũng được cất và sấy khô. Ví dụ, chúng tôi đã liệt kê nhiệm vụ chẩn đoán y tế là một tác nhân vì quá trình bệnh ở bệnh nhân không được mô hình hóa một cách có lợi nhuận như một tác nhân; Nhưng một hệ thống chẩn đoán y tế cũng có thể phải đối phó với những bệnh nhân ngoan cố và nhân viên hoài nghi, vì vậy môi trường có thể có khía cạnh đa tác nhân. Hơn nữa, chẩn đoán y tế là từng đợt nếu một người quan niệm nhiệm vụ là lựa chọn chẩn đoán với danh sách các triệu chứng; Vấn đề là tuần tự nếu nhiệm vụ có thể bao gồm đề xuất một loạt các xét nghiệm, đánh giá tiến độ trong quá trình điều trị, xử lý nhiều bệnh nhân, v.v.

Môi trường tác vụ	Các tác nhân có thể quan sát được			Xác định	Episodic	Tính	Rời rạc
Trò chơi ô chữ	Đầy đủ	Đơn	Tuần tự xác định			Tính	Rời rạc
Cờ vua với đồng hồ	Đầy đủ	Đa	Tuần tự xác định			Bán	Rời rạc
Poker	Một phần	Đa	Ngẫu nhiên	Sequential	Tính		Rời rạc
Backgammon	Đầy đủ	Đa	Ngẫu nhiên	Sequential	Tính		Rời rạc
Lái xe taxi	Một phần	Đa	Ngẫu nhiên	Liên tục động tuần tự			
Chẩn đoán y tế	Một phần	Đơn	Ngẫu nhiên	Liên tục động tuần tự			
Phân tích hình ảnh	Đầy đủ	Đơn	Xác định	Episodic	Bán	Liên tục	
Robot chọn bộ phận	Một phần	Đơn	Ngẫu nhiên	Episodic	Động liên tục		
Bộ điều khiển nhà máy lọc	Một phần	Đơn	Ngẫu nhiên	Liên tục động tuần tự			
Gia sư tiếng Anh	Một phần	Đa	Ngẫu nhiên	Động tuần tự			Rời rạc

Hình 2.6 Ví dụ về môi trường tác vụ và đặc điểm của chúng.

Chúng tôi đã không bao gồm một cột "đã biết/chưa biết" bởi vì, như đã giải thích trước đó, đây không hoàn toàn là một thuộc tính của môi trường. Đối với một số môi trường, chẳng hạn như cờ vua và poker, khá dễ dàng để cung cấp cho đại lý kiến thức đầy đủ về các quy tắc, nhưng dù sao cũng rất thú vị khi xem xét cách một đại lý có thể học cách chơi các trò chơi này mà không có kiến thức như vậy.

Kho lưu trữ mã được liên kết với cuốn sách này (aima.cs.berkeley.edu) bao gồm các triển khai môi trường đa dạng, cùng với trình mô phỏng môi trường mục đích chung để đánh giá hiệu suất của tác nhân. Các thí nghiệm thường được thực hiện không phải cho một môi trường duy nhất mà cho nhiều môi trường được rút ra từ một lớp môi trường. Ví dụ: để

Lớp môi trường

Đánh giá một tài xế taxi trong giao thông mô phỏng, chúng tôi muốn chạy nhiều mô phỏng với các điều kiện giao thông, ánh sáng và thời tiết khác nhau. Sau đó, chúng tôi quan tâm đến hiệu suất trung bình của tác nhân trên lớp môi trường.

2.4 Cấu trúc của các tác nhân

Cho đến nay chúng ta đã nói về các tác nhân bằng cách mô tả hành vi - hành động được thực hiện sau bất kỳ chuỗi nhận thức nhất định nào. Bây giờ chúng ta phải cân viên đạn và nói về cách thức hoạt động của nội bộ. Công việc của AI là thiết kế một chương trình tác nhân thực hiện chức năng tác nhân—

Chương trình đại lý

ánh xạ từ nhận thức đến hành động. Chúng tôi giả định chương trình này sẽ chạy trên một số loại thiết bị máy tính với các cảm biến vật lý và bộ truyền động - chúng tôi gọi đây là kiến trúc tác nhân:

Kiến trúc đại lý

đại lý = kiến trúc + chương trình.

Rõ ràng, chương trình chúng ta chọn phải là một chương trình phù hợp với kiến trúc. Nếu chương trình sẽ đề xuất các hành động như Đi bộ, kiến trúc tốt hơn nên có chân. Kiến trúc có thể chỉ là một PC thông thường, hoặc nó có thể là một chiếc ô tô robot với một số máy tính tích hợp, máy ảnh và các cảm biến khác. Nói chung, kiến trúc làm cho các nhận thức từ các cảm biến có sẵn cho chương trình, chạy chương trình và cung cấp các lựa chọn hành động của chương trình cho các thiết bị truyền động khi chúng được tạo ra. Hầu hết cuốn sách này nói về thiết kế các chương trình tác nhân, mặc dù Chương 26 và 27 đề cập trực tiếp đến các cảm biến và thiết bị truyền động.

```

function TABLE-DRIVEN-AGENT(percept) trả về một hành động
persistent: nhận thức, một trình tự, ban đầu trống rỗng
    table, một bảng các hành động, được lập chỉ mục bởi các chuỗi nhận thức, ban đầu được chỉ định đầy đủ

    thêm percept vào cuối perceptsaction ←
    LOOKUP(percepts, table) return action

```

Hình 2.7 Chương trình TABLE-DRIVEN-AGENT được gọi cho mỗi nhận thức mới và quay lại một hành động mỗi lần. Nó giữ lại trình tự tri giác hoàn chỉnh trong bộ nhớ.

2.4.1 Chương trình đại lý

Các chương trình tác nhân mà chúng tôi thiết kế trong cuốn sách này đều có cùng một khung gầm: chúng lấy nhận thức hiện tại làm đầu vào từ các cảm biến và trả lại một hành động cho các bộ truyền động. Lưu ý khác biệt giữa chương trình tác nhân, lấy nhận thức hiện tại làm đầu vào, và chức năng tác nhân, có thể phụ thuộc vào toàn bộ lịch sử nhận thức. Chương trình tác nhân không có lựa chọn nào khác ngoài việc chỉ lấy nhận thức hiện tại làm đầu vào vì không có gì hơn từ môi trường; Nếu hành động của tác nhân cần phụ thuộc vào toàn bộ chuỗi tri giác, tác nhân sẽ phải ghi nhớ các tri giác.

Chúng tôi mô tả các chương trình tác nhân bằng ngôn ngữ giả mã đơn giản được định nghĩa trong Phụ lục B. (Kho lưu trữ mã trực tuyến chứa các triển khai trong các ngôn ngữ lập trình thực.) Ví dụ, Hình 2.7 cho thấy một chương trình tác nhân khá tầm thường theo dõi chuỗi nhận thức và sau đó sử dụng nó để lập chỉ mục vào một bảng các hành động để quyết định phải làm gì. Bảng - một ví dụ được đưa ra cho thế giới chân không trong Hình 2.3 - đại diện rõ ràng chức năng tác nhân mà chương trình tác nhân thể hiện. Để xây dựng một tác nhân hợp lý theo cách này, chúng ta với tư cách là nhà thiết kế phải xây dựng một bảng chứa hành động thích hợp cho mọi trình tự nhận thức có thể.

Sẽ rất hữu ích khi xem xét lý do tại sao cách tiếp cận dựa trên bảng để xây dựng tác nhân sẽ thất bại. Giả sử P là tập hợp các nhận thức có thể xảy ra và cho T là tuổi thọ của tác nhân (tổng số nhận thức mà nó sẽ nhận được). Bảng tra cứu sẽ chứa $\sum Tt = 1 | \text{Mục nhập } P|t$. Hãy xem xét taxi tự động: đầu vào trực quan từ một máy ảnh duy nhất (tám camera là điển hình) comesin với tốc độ khoảng 70 megabyte mỗi giây (30 khung hình / giây, 1080×720 pixel với 24 bit thông tin màu). Điều này cung cấp một bảng tra cứu với hơn 10600.000.000.000 mục nhập cho một giờ lái xe. Ngay cả bảng tra cứu cho cò vua - một mảnh nhỏ, cư xử tốt của thế giới đó - có ít nhất 10150 mục nhập. Để so sánh, số lượng nguyên tử trong vũ trụ có thể quan sát được là ít hơn 1080. Kích thước đáng sợ của các bảng này có nghĩa là (a) tác nhân không vật lý trong vũ trụ này sẽ có không gian để lưu trữ bảng; (b) nhà thiết kế sẽ không có thời gian để tạo bảng; và (c) không có đại lý nào có thể học được tất cả các mục bảng phù hợp từ kinh nghiệm của mình.

Bất chấp tất cả những điều này, TABLE-DRIVEN-AGENT thực hiện những gì chúng ta muốn, giả sử bảng được điền chính xác: nó thực hiện chức năng tác nhân mong muốn.

Tô

i

5 Có những lựa chọn khác cho khung chương trình tác nhân; Ví dụ, chúng ta có thể có các chương trình tác nhân chạy không đồng bộ với môi trường. Mỗi coroutine như vậy có một cổng đầu vào và đầu ra và bao gồm một vòng lặp đọc cổng đầu vào để nhận thức và ghi hành động vào cổng đầu ra.

```
function REFLEX-VACUUM-AGENT([location,status]) trả về một hành động
  nếu status = Bắn thì trả về Suckelse nếu location =
  A thì trả về Rightelse nếu location = B thì trả về Left
```

Hình 2.8 Chương trình tác nhân cho một tác nhân phản xạ đơn giản trong môi trường chân không hai vị trí. Chương trình này thực hiện chức năng tác nhân được lập bảng trong Hình 2.3.

Thách thức chính đối với AI là tìm ra cách viết các chương trình, trong phạm vi có thể, tạo ra hành vi hợp lý từ một chương trình nhỏ hơn là từ một bảng lớn.

Chúng tôi có nhiều ví dụ cho thấy điều này có thể được thực hiện thành công trong các lĩnh vực khác: ví dụ, các bảng căn bậc hai khổng lồ được sử dụng bởi các kỹ sư và học sinh trước những năm 1970 hiện đã được thay thế bằng một chương trình năm dòng cho phương pháp Newton chạy trên máy tính điện tử. Câu hỏi đặt ra là, AI có thể làm cho hành vi thông minh nổi chung như Newton đã làm cho căn bậc hai không? Chúng tôi tin rằng câu trả lời là có.

Trong phần còn lại của phần này, chúng tôi phác thảo bốn loại chương trình tác nhân cơ bản bao gồm các nguyên tắc cơ bản của hầu hết các hệ thống thông minh:

- Các tác nhân phản xạ đơn giản;
- Tác nhân phản xạ dựa trên mô hình;
- Các tác nhân dựa trên mục tiêu; và
- Đại lý dựa trên tiện ích. Mỗi loại chương trình tác nhân kết hợp các thành phần cụ thể theo những cách cụ thể để tạo ra các hành động. Phần 2.4.6 giải thích một cách tổng quát cách chuyển đổi tất cả các tác nhân này thành các tác nhân học tập có thể cải thiện hiệu suất của các thành phần của chúng để tạo ra các hành động tốt hơn. Cuối cùng, Phần 2.4.7 mô tả nhiều cách khác nhau mà bản thân các thành phần có thể được biểu diễn trong tác nhân. Sự đa dạng này cung cấp một nguyên tắc tổ chức chính cho lĩnh vực này và cho chính cuốn sách.

2.4.2 Tác nhân phản xạ đơn giản

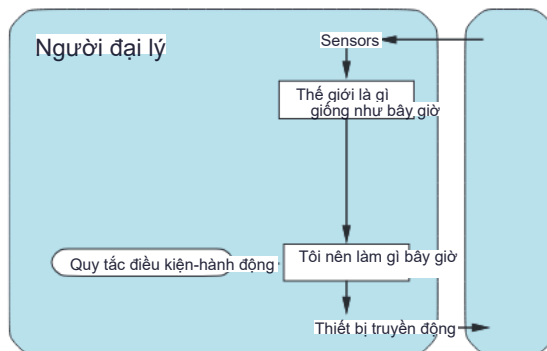
Loại tác nhân đơn giản nhất là tác nhân phản xạ đơn giản. Các tác nhân này chọn hành động trên cơ sở

Tác nhân phản xạ đơn giản

của tri giác hiện tại, bỏ qua phần còn lại của lịch sử tri giác. Ví dụ, tác nhân chân không có chức năng tác nhân được lập bảng trong Hình 2.3 là một tác nhân phản xạ đơn giản, bởi vì quyết định của nó chỉ dựa trên vị trí hiện tại và liệu vị trí đó có chứa bụi bẩn hay không. Một chương trình tác nhân cho tác nhân này được thể hiện trong Hình 2.8.

Lưu ý rằng chương trình tác nhân chân không thực sự rất nhỏ so với bảng tương ứng. Sự suy giảm rõ ràng nhất đến từ việc bỏ qua lịch sử tri giác, cắt giảm số lượng chuỗi tri giác có liên quan từ 4T xuống chỉ còn 4. Một sự suy giảm nhỏ hơn nữa đến từ thực tế là khi hình vuông hiện tại bị bắn, hành động không phụ thuộc vào vị trí. Mặc dù chúng ta đã viết chương trình tác nhân bằng cách sử dụng các câu lệnh if-then-else, nhưng nó đủ đơn giản để nó cũng có thể được thực hiện như một mạch Boolean.

Các hành vi phản xạ đơn giản xảy ra ngay cả trong môi trường phức tạp hơn. Hãy tưởng tượng bạn là tài xế của chiếc taxi tự động. Nếu xe phanh trước và đèn phanh bật sáng, thì bạn nên lưu ý điều này và bắt đầu phanh. Nói cách khác, một số quá trình xử lý được thực hiện trên



Hình 2.9 Sơ đồ của một tác nhân phản xạ đơn giản. Chúng tôi sử dụng hình chữ nhật để biểu thị trạng thái bên trong hiện tại của quá trình quyết định của tác nhân và hình bầu dục để biểu thị thông tin nền được sử dụng trong quá trình.

Điều kiện – quy tắc
hành động

đầu vào trực quan để thiết lập điều kiện mà chúng tôi gọi là "Xe phía trước đang phanh". Sau đó, điều này kích hoạt một số kết nối đã được thiết lập trong chương trình tác nhân với hành động "bắt đầu phanh". Chúng ta gọi một kết nối như vậy là một quy tắc điều kiện-hành động, được viết như

Nếu xe ở phía trước là phanh thì bắt đầu phanh.

Con người cũng có nhiều kết nối như vậy, một số trong số đó là phản ứng học được (như lái xe) và một số là phản xạ bẩm sinh (chẳng hạn như chớp mắt khi có thứ gì đó đến gần mắt). Trong cuốn sách, chúng tôi chỉ ra một số cách khác nhau mà các kết nối như vậy có thể được học và thực hiện.

Chương trình trong Hình 2.8 dành riêng cho một môi trường chân không cụ thể. Một cách tiếp cận tổng quát và linh hoạt hơn trước tiên là xây dựng một trình thông dịch có mục đích chung cho các quy tắc điều kiện-hành động và sau đó tạo ra các bộ quy tắc cho các môi trường tác vụ cụ thể. Hình 2.9 đưa ra cấu trúc của chương trình chung này ở dạng sơ đồ, cho thấy cách các quy tắc điều kiện-hành động cho phép tác nhân thực hiện kết nối từ nhận thức sang hành động. Đừng lo lắng nếu điều này có vẻ tầm thường; nó sẽ trở nên thú vị hơn trong thời gian ngắn.

Một chương trình tác nhân cho Hình 2.9 được thể hiện trong Hình 2.10. Hàm INTERPRET-INPUT tạo ra một mô tả trừu tượng về trạng thái hiện tại từ nhận thức và hàm RULE-MATCH trả về quy tắc đầu tiên trong tập hợp các quy tắc khớp với mô tả trạng thái đã cho. Lưu ý rằng mô tả về "quy tắc" và "khớp" hoàn toàn là khái niệm; như đã lưu ý ở trên, việc triển khai thực tế có thể đơn giản như một tập hợp các cổng logic thực hiện một mạch Boolean. Ngoài ra, một mạch "thần kinh có thể được sử dụng, trong đó các cổng logic được thay thế bằng các đơn vị phi tuyến của mạng nơ-ron nhân tạo (xem Chương 22).

Các tác nhân phản xạ đơn giản có đặc tính đáng ngưỡng mộ là đơn giản, nhưng chúng có trí thông minh hạn chế. Tác nhân trong Hình 2.10 sẽ chỉ hoạt động nếu quyết định chính xác có thể được đưa ra trên cơ sở của nhận thức hiện tại - nghĩa là, chỉ khi môi trường có thể quan sát được đầy đủ.

6 Còn được gọi là quy tắc tình huống-hành động, sản xuất, hoặc nếu-thì quy tắc.

```

function SIMPLE-REFLEX-AGENT(percept) trả về một hành động
persistent: quy tắc, một tập hợp các quy tắc điều kiện-hành động

state ← INTERPRET-INPUT
(percept) ← RULE-MATCH
(state, rules) hành động ← quy
tắc. ACTION trả về hành động

```

Hình 2.10 Một tác nhân phản xạ đơn giản. Nó hoạt động theo một quy tắc có điều kiện phù hợp với trạng thái hiện tại, như được xác định bởi tri giác.

Ngay cả một chút không thể quan sát được cũng có thể gây ra rắc rối nghiêm trọng. Ví dụ, quy tắc phanh được đưa ra trước đó giả định rằng tình trạng xe ở phía trước là phanh có thể được xác định từ nhận thức hiện tại - một khung hình video duy nhất. Điều này hoạt động nếu xe phía trước có đèn phanh được gắn trung tâm (và do đó có thể nhận dạng duy nhất). Thật không may, các mẫu xe cũ hơn có các cấu hình khác nhau của đèn hậu, đèn phanh và đèn xi nhan, và không phải lúc nào cũng có thể biết từ một hình ảnh duy nhất là xe đang phanh hay chỉ đơn giản là có đèn hậu. Một tác nhân phản xạ đơn giản lái xe phía sau một chiếc xe như vậy sẽ phanh liên tục và không cần thiết, hoặc tệ hơn là không bao giờ phanh chút nào.

Chúng ta có thể thấy một vấn đề tương tự phát sinh trong thế giới chân không. Giả sử rằng một tác nhân chân không phản xạ đơn giản bị tước đi cảm biến vị trí của nó và chỉ có một cảm biến bụi bẩn. Một tác nhân như vậy chỉ có hai nhận thức có thể có: [Bẩn] và [Sạch]. Nó có thể Hút để đáp lại [Bẩn]; Nó nên làm gì để đáp lại [Clean]? Di chuyển sang trái không thành công (mãi mãi) nếu nó bắt đầu ở ô vuông A và di chuyển sang phải không thành công (mãi mãi) nếu nó bắt đầu ở ô vuông B. Các vòng lặp vô hạn thường không thể tránh khỏi đối với các tác nhân phản xạ đơn giản hoạt động trong môi trường có thể quan sát được một phần.

Có thể thoát khỏi vòng lặp vô hạn nếu tác nhân có thể ngẫu nhiên hóa các hành động của nó. Đối với kỳ thi- Ngẫu nhiên

ple, nếu chất chân không nhận thức được [Sạch], nó có thể tung một đồng xu để chọn giữa Phải và Trái. Có thể dễ dàng chứng minh rằng tác nhân sẽ đến được ô vuông kia trong trung bình hai bước. Sau đó, nếu ô vuông đó bị bẩn, đại lý sẽ làm sạch nó và nhiệm vụ sẽ hoàn thành. Do đó, tác nhân phản xạ đơn giản ngẫu nhiên có thể vượt trội hơn tác nhân phản xạ đơn giản xác định.

Chúng tôi đã đề cập trong Phần 2.3 rằng hành vi ngẫu nhiên của loại phù hợp có thể là hợp lý trong một số môi trường đa tác nhân. Trong môi trường đơn tác nhân, ngẫu nhiên hóa thường là không hợp lý. Đó là một thủ thuật hữu ích giúp một tác nhân phản xạ đơn giản trong một số tình huống, nhưng trong hầu hết các trường hợp, chúng ta có thể làm tốt hơn nhiều với các tác nhân quyết định phức tạp hơn.

2.4.3 Tác nhân phản xạ dựa trên mô hình

Cách hiệu quả nhất để xử lý khả năng quan sát một phần là để tác nhân theo dõi phần thế giới mà nó không thể nhìn thấy bây giờ. Nghĩa là, tác nhân nên duy trì một số loại trạng thái bên trong phụ thuộc vào lịch sử tri giác và do đó phản ánh ít nhất một số điều chưa được quan sát

Trạng thái nội bộ

các khía cạnh của tình trạng hiện tại. Đối với vấn đề phanh, trạng thái bên trong không quá rộng — chỉ là khung hình trước đó từ camera, cho phép tác nhân phát hiện khi hai đèn đỏ ở rìa xe bật hoặc tắt đồng thời. Đối với các nhiệm vụ lái xe khác như chuyển làn, nhân viên cần theo dõi vị trí của những chiếc xe khác nếu không thể nhìn thấy tất cả chúng cùng một lúc. Và để có thể thực hiện được bất kỳ việc lái xe nào, nhân viên cần theo dõi vị trí của chìa khóa của nó.

hàm MODEL-BASED-REFLEX-AGENT(percept) trả về một hành động

persistent: trạng thái, quan niệm hiện tại của tác nhân về trạng thái thế giới

mô hình chuyển tiếp, mô tả về cách trạng thái tiếp theo phụ thuộc vào

Trạng thái hiện tại và mô hình ActionSensor, mô tả về cách trạng thái thế giới hiện tại được phản ánh

Trong PerceptsRules của Agent, một tập hợp các quy tắc điều kiện-hành động ActionAction, hành động gần đây nhất, ban đầu không có

state ← quy tắc UPDATE-STATE (state, action, percept, transition model, sensor model) ← RULE-MATCH (state, rules) action ← rule.

ACTION trả về hành động

Hình 2.12 Một tác nhân phản xạ dựa trên mô hình. Nó theo dõi tình trạng hiện tại của thế giới, sử dụng một mô hình nội bộ. Sau đó, nó chọn một hành động theo cách tương tự như tác nhân phản xạ.

có thể không nhìn thấy xung quanh chiếc xe tải lớn đã dừng lại phía trước và chỉ có thể đoán được điều gì có thể gây ra tình trạng tắc nghẽn. Do đó, sự không chắc chắn về trạng thái hiện tại có thể không thể tránh khỏi, nhưng tác nhân vẫn phải đưa ra quyết định.

2.4.4 Tác nhân dựa trên mục tiêu

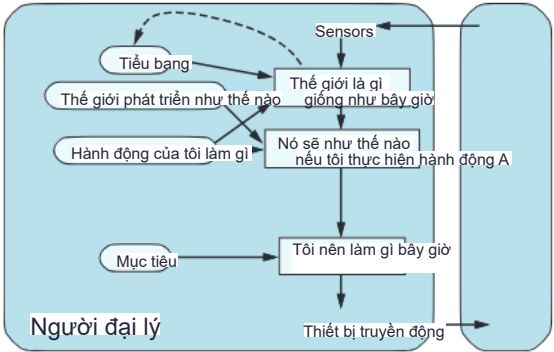
Biết điều gì đó về tình trạng hiện tại của môi trường không phải lúc nào cũng đủ để quyết định phải làm gì. Ví dụ, tại ngã ba đường, taxi có thể rẽ trái, rẽ phải hoặc đi thẳng. Quyết định chính xác phụ thuộc vào nơi taxi đang cố gắng đến. Nói cách khác, cũng như mô tả trạng thái hiện tại, tác nhân cần một số loại thông tin mục tiêu

mô tả các tình huống mong muốn—ví dụ, đang ở một điểm đến cụ thể. Chương trình tác nhân có thể kết hợp điều này với mô hình (thông tin tương tự như đã được sử dụng trong tác nhân phản xạ dựa trên mô hình) để chọn các hành động đạt được mục tiêu. Hình 2.13 cho thấy cấu trúc của tác nhân dựa trên mục tiêu.

Đôi khi việc lựa chọn hành động dựa trên mục tiêu rất đơn giản - ví dụ, khi thỏa mãn mục tiêu là kết quả ngay lập tức từ một hành động duy nhất. Đôi khi nó sẽ phức tạp hơn - ví dụ, khi tác nhân phải xem xét các chuỗi dài của những khúc quanh và ngã rẽ để tìm cách đạt được mục tiêu. Tìm kiếm (Chương 3, 4 và 6) và lập kế hoạch (Chương 11) là các lĩnh vực con của AI dành cho việc tìm kiếm các chuỗi hành động đạt được mục tiêu của tác nhân.

Lưu ý rằng việc ra quyết định kiểu này về cơ bản khác với các quy tắc điều kiện-hành động được mô tả trước đó, ở chỗ nó liên quan đến việc xem xét tương lai - cả "Điều gì sẽ xảy ra nếu tôi làm điều này và điều kia?" và "Điều đó có làm cho tôi hạnh phúc không?" Trong thiết kế tác nhân phản xạ, thông tin này không được thể hiện rõ ràng, bởi vì các quy tắc tích hợp ánh xạ trực tiếp từ nhận thức đến hành động. Tác nhân phản xạ phàn khi nhìn thấy đèn phan, khoảng thời gian. Nó không biết tại sao. Một tác nhân dựa trên mục tiêu phan khi nhìn thấy đèn phan vì đó là hành động duy nhất mà nó dự đoán sẽ đạt được mục tiêu không đâm vào những chiếc xe khác.

Mặc dù tác nhân dựa trên mục tiêu có vẻ kém hiệu quả hơn, nhưng nó linh hoạt hơn vì kiến trúc hỗ trợ các quyết định của nó được thể hiện rõ ràng và có thể được sửa đổi. Ví dụ: hành vi của một nhân viên dựa trên mục tiêu có thể dễ dàng thay đổi để đi đến một điểm đến khác,



Hình 2.13 Một tác nhân dựa trên mô hình, dựa trên mục tiêu. Nó theo dõi tình trạng thế giới cũng như một tập hợp các mục tiêu mà nó đang cố gắng đạt được, và chọn một hành động sẽ (cuối cùng) dẫn đến việc đạt được các mục tiêu của nó.

đơn giản bằng cách chỉ định điểm đến đó làm mục tiêu. Các quy tắc của tác nhân phân xạ về thời điểm rẽ và khi nào đi thẳng sẽ chỉ hiệu quả đối với một điểm đến duy nhất; tất cả chúng phải được thay thế ở một nơi nào đó mới.

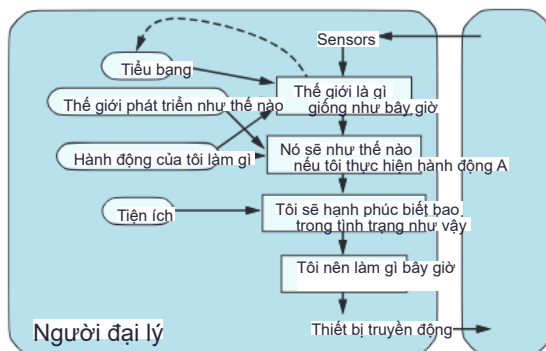
2.4.5 Tác nhân dựa trên tiện ích

Chỉ mục tiêu thôi là không đủ để tạo ra hành vi chất lượng cao trong hầu hết các môi trường. Ví dụ, nhiều cảnh hành động sẽ đưa taxi đến đích (do đó đạt được mục tiêu), nhưng một số nhanh hơn, an toàn hơn, đáng tin cậy hơn hoặc rẻ hơn những cảnh khác. Mục tiêu chỉ cung cấp sự phân biệt nhị phân giữa trạng thái "hạnh phúc" và "không hạnh phúc". Một thước đo hiệu suất tổng quát hơn nên cho phép so sánh các quốc gia khác nhau trên thế giới theo chính xác mức độ hạnh phúc mà họ sẽ làm cho tác nhân. Bởi vì "hạnh phúc" nghe có vẻ không khoa học lắm, các nhà kinh tế học và nhà khoa học máy tính sử dụng thuật ngữ tiện ích thay thế.

Chúng ta đã thấy rằng một thước đo hiệu suất gán điểm cho bất kỳ chuỗi trạng thái môi trường nhất định nào, vì vậy nó có thể dễ dàng phân biệt giữa các cách đi đến điểm đến của taxi. Chức năng tiện ích của một tác nhân về cơ bản là một sự nội tâm hóa của biện pháp hiệu suất. Với điều kiện là chức năng tiện ích bên trong và thước đo hiệu suất bên ngoài phù hợp, một tác nhân chọn các hành động để tối đa hóa tiện ích của nó sẽ hợp lý theo thước đo hiệu suất bên ngoài.

Chúng ta hãy nhấn mạnh một lần nữa rằng đây không phải là cách duy nhất để trở nên hợp lý – chúng ta đã thấy một chương trình tác nhân hợp lý cho thế giới chân không (Hình 2.8) mà không biết chức năng tiện ích của nó là gì – nhưng, giống như các tác nhân dựa trên mục tiêu, một tác nhân dựa trên tiện ích có nhiều lợi thế về tính linh hoạt và học hỏi. Hơn nữa, trong hai loại trường hợp, mục tiêu không đầy đủ nhưng tác nhân dựa trên tiện ích vẫn có thể đưa ra quyết định hợp lý. Đầu tiên, khi có các mục tiêu mâu thuẫn, chỉ một số mục tiêu có thể đạt được (ví dụ: tốc độ và an toàn), chức năng tiện ích chỉ định sự đánh đổi thích hợp. Thứ hai, khi có một số mục tiêu mà đại lý có thể

7 Từ "tiện ích" ở đây để cập đến "chất lượng hữu ích", không phải là công ty điện lực hoặc nhà máy nước.



Hình 2.14 Một tác nhân dựa trên mô hình, dựa trên tiện ích. Nó sử dụng một mô hình của thế giới, cùng với chức năng tiện ích đo lường sở thích của nó giữa các quốc gia trên thế giới. Sau đó, nó chọn hành động dẫn đến tiện ích kỳ vọng tốt nhất, trong đó tiện ích kỳ vọng được tính bằng cách tính trung bình trên tất cả các trạng thái kết quả có thể xảy ra, được trọng số bởi xác suất của kết quả.

Mục tiêu, không ai trong số đó có thể đạt được một cách chắc chắn, tiện ích cung cấp một cách mà khả năng thành công có thể được cân nhắc với tầm quan trọng của các mục tiêu.

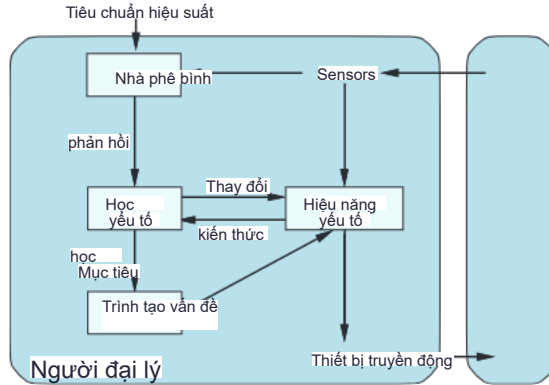
Khả năng quan sát một phần và thuyết không xác định là phổ biến trong thế giới thực, và do đó, do đó, việc ra quyết định trong điều kiện không chắc chắn. Về mặt kỹ thuật, một tác nhân dựa trên tiện ích hợp lý chọn hành động tối đa hóa tiện ích mong đợi của các kết quả hành động – đó là

Tiện ích mong đợi

là, tiện ích mà tác nhân mong đợi có được, trung bình, với xác suất và tiện ích của mỗi kết quả. (Phụ lục A định nghĩa kỳ vọng chính xác hơn.) Trong Chương 15, chúng ta chỉ ra rằng bất kỳ tác nhân duy lý nào cũng phải hành xử như thể nó sở hữu một hàm tiện ích mà giá trị mong đợi mà nó cố gắng tối đa hóa. Một tác nhân sở hữu một hàm tiện ích rõ ràng có thể thực hiện các định nghĩa hợp lý với một thuật toán mục đích chung không phụ thuộc vào hàm tiện ích cụ thể được tối đa hóa. Bằng cách này, định nghĩa "toàn cầu" về tính hợp lý – chỉ định là hợp lý những hàm tác nhân có hiệu suất cao nhất – được biến thành một ràng buộc "cục bộ" đối với các thiết kế tác nhân hợp lý có thể được biểu thị trong một chương trình đơn giản.

Cấu trúc tác nhân dựa trên tiện ích xuất hiện trong Hình 2.14. Các chương trình tác nhân dựa trên tiện ích xuất hiện trong Chương 15 và 16, nơi chúng tôi thiết kế các tác nhân ra quyết định phải xử lý sự không chắc chắn vốn có trong môi trường không xác định hoặc có thể quan sát được một phần. Đưa ra quyết định trong môi trường đa tác nhân cũng được nghiên cứu trong khuôn khổ lý thuyết tiện ích, như đã giải thích trong

Chương 17. Tại thời điểm này, người đọc có thể tự hỏi, "Nó có đơn giản như vậy không? Chúng tôi chỉ xây dựng các tác nhân tối đa hóa tiện ích mong đợi, và chúng tôi đã hoàn thành?" Đúng là những tác nhân như vậy sẽ thông minh, nhưng nó không đơn giản. Một tác nhân dựa trên tiện ích phải mô hình hóa và theo dõi môi trường của nó, các nhiệm vụ liên quan đến rất nhiều nghiên cứu về nhận thức, đại diện, lý luận và học tập. Kết quả của nghiên cứu này lấp đầy nhiều chương của cuốn sách này. Chọn quá trình hành động tối đa hóa tiện ích cũng là một nhiệm vụ khó khăn, đòi hỏi các thuật toán khéo léo lấp đầy thêm nhiều chương nữa. Ngay cả với các thuật toán này, tính hợp lý hoàn hảo thường là



Hình 2.15 Một tác nhân học tập tổng quát. Hộp "yếu tố hiệu suất" đại diện cho những gì chúng tôi đã coi là toàn bộ chương trình tác nhân. Bây giờ, hộp "yếu tố học tập" có thể sửa đổi chương trình đó để cải thiện hiệu suất của nó.

không thể đạt được trong thực tế vì độ phức tạp của tính toán, như chúng tôi đã lưu ý trong Chương 1. Chúng tôi cũng lưu ý rằng không phải tất cả các tác nhân dựa trên tiện ích đều dựa trên mô hình; chúng ta sẽ thấy trong Chương 23 và 26 rằng một tác nhân

Đại lý không có mô hình không có mô hình có thể học hành động nào là tốt nhất trong một tình huống cụ thể mà không bao giờ

tìm hiểu chính xác hành động đó thay đổi môi trường như thế nào.

Cuối cùng, tất cả những điều này giả định rằng nhà thiết kế có thể chỉ định hàm tiện ích một cách chính xác; Chương 16, 17 và 23 xem xét vấn đề các chức năng tiện ích chưa biết sâu hơn.

2.4.6 Các tác nhân học tập

Chúng tôi đã mô tả các chương trình tác nhân với nhiều phương pháp khác nhau để chọn hành động. Cho đến nay, chúng tôi vẫn chưa giải thích cách các chương trình tác nhân ra đời. Trong bài báo đầu tiên nổi tiếng của mình, Turing (1950) xem xét ý tưởng thực sự lập trình máy móc thông minh của mình bằng tay. Ông ước tính việc này có thể mất bao nhiêu công sức và kết luận, "Một số phương pháp nhanh hơn có vẻ mong muốn." Phương pháp mà ông đề xuất là xây dựng máy học và sau đó dạy chúng. Trong nhiều lĩnh vực của AI, đây hiện là phương pháp ưa thích để tạo ra các hệ thống hiện đại. Bất kỳ loại tác nhân nào (dựa trên mô hình, dựa trên mục tiêu, dựa trên tiện ích, v.v.) đều có thể được xây dựng như một tác nhân học tập (hoặc không).

Học tập có một lợi thế khác, như chúng tôi đã lưu ý trước đó: nó cho phép tác nhân vận hành các môi trường chưa biết ban đầu và trở nên có năng lực hơn so với kiến thức ban đầu của nó có thể cho phép. Trong phần này, chúng tôi giới thiệu ngắn gọn những ý tưởng chính của các tác nhân học tập. Xuyên suốt cuốn sách, chúng tôi nhận xét về các cơ hội và phương pháp học tập trong các loại tác nhân tương tác. Các chương 19, 21, 22 và 23 đi sâu hơn nhiều về bản thân các thuật toán học

Một tác nhân học tập có thể được chia thành bốn thành phần khái niệm, như thể hiện trong Hình 2.15. Sự khác biệt quan trọng nhất là giữa yếu tố học tập, đó là

sponsable để thực hiện các cải tiến và yếu tố hiệu suất, chịu trách nhiệm về chọn hành động bên ngoài. Yếu tố hiệu suất là những gì chúng tôi đã xem xét trước đây

trở thành toàn bộ tác nhân: nó nhận thức và quyết định hành động. Yếu tố học tập sử dụng phản hồi từ nhà phê bình về cách tác nhân đang làm và xác định hiệu suất như thế nào

Nhà phê bình

nên được sửa đổi để làm tốt hơn trong tương lai.

Thiết kế của yếu tố học tập phụ thuộc rất nhiều vào thiết kế của yếu tố hiệu suất. Khi cố gắng thiết kế một tác nhân học một khả năng nhất định, câu hỏi đầu tiên không phải là "Làm thế nào tôi sẽ làm cho nó học được điều này?" mà là "Đặc vụ của tôi sẽ sử dụng loại yếu tố hiệu suất nào để làm điều này một khi nó đã học được cách thức?" Với một thiết kế cho yếu tố hiệu suất, các cơ chế học tập có thể được xây dựng để cải thiện mọi phần của tác nhân.

Nhà phê bình nói với yếu tố học tập rằng tác nhân đang làm tốt như thế nào đối với tiêu chuẩn hiệu suất cố định. Người chỉ trích là cần thiết bởi vì bản thân các nhận thức không cung cấp dấu hiệu nào về sự thành công của tác nhân. Ví dụ: một chương trình cờ vua có thể nhận được một nhận thức chỉ ra rằng nó đã kiểm tra đối thủ của mình, nhưng nó cần một tiêu chuẩn hiệu suất để biết rằng đây là một điều tốt; Bản thân tri giác không nói như vậy. Điều quan trọng là tiêu chuẩn hiệu suất phải được cố định. Về mặt khái niệm, người ta nên nghĩ về nó như nằm ngoài tác nhân hoàn toàn bởi vì tác nhân không được sửa đổi nó để phù hợp với hành vi của chính nó.

Thành phần cuối cùng của tác nhân học tập là trình tạo vấn đề. Nó có trách nhiệm

Trình tạo sự cố

để đề xuất các hành động sẽ dẫn đến những trải nghiệm mới và nhiều thông tin. Nếu yếu tố hiệu suất có cách của nó, nó sẽ tiếp tục thực hiện các hành động tốt nhất, dựa trên những gì nó biết, nhưng nếu tác nhân sẵn sàng khám phá một chút và thực hiện một số hành động có lẽ không tối ưu trong ngắn hạn, nó có thể phát hiện ra các hành động tốt hơn nhiều về lâu dài. Công việc của trình tạo vấn đề là đề xuất các hành động thăm dò này. Đây là những gì các nhà khoa học làm khi họ thực hiện các thí nghiệm. Galileo không nghĩ rằng việc thả đá từ đỉnh tháp ở Pisa là có giá trị. Anh ta không cố gắng phá đá hoặc sửa đổi bộ não của những người đi bộ bất hạnh. Mục đích của anh ấy là sửa đổi bộ não của chính mình bằng cách xác định một lý thuyết tốt hơn về chuyển động của các vật thể.

Yếu tố học tập có thể thực hiện các thay đổi đối với bất kỳ thành phần "kiến thức" nào được hiển thị trong sơ đồ tác nhân (Hình 2.9, 2.11, 2.13 và 2.14). Các trường hợp đơn giản nhất liên quan đến việc học trực tiếp từ trình tự tri giác. Quan sát các cặp trạng thái liên tiếp của môi trường có thể cho phép tác nhân tìm hiểu "Hành động của tôi làm gì" và "Thế giới phát triển như thế nào" không phản ứng với hành động của nó. Ví dụ, nếu taxi tự động tạo áp lực phanh nhất định khi lái xe trên đường ướt, thì nó sẽ sớm tìm ra mức giảm tốc thực tế đạt được bao nhiêu và liệu nó có bị trượt khỏi đường hay không. Trình tạo sự cố có thể xác định một số phần nhất định của mô hình cần cải thiện và đề xuất các thí nghiệm, chẳng hạn như thử phanh trên các mặt đường khác nhau trong các điều kiện khác nhau.

Cải thiện các thành phần mô hình của một tác nhân dựa trên mô hình để chúng phù hợp hơn với thực tế hầu như luôn là một ý tưởng hay, bất kể tiêu chuẩn hiệu suất bên ngoài. (Trong một số trường hợp, tốt hơn là từ quan điểm tính toán nên có một mô hình đơn giản nhưng hơi không chính xác hơn là một mô hình hoàn hảo nhưng phức tạp một cách quái dị.) Thông tin từ tiêu chuẩn bên ngoài là cần thiết khi cố gắng tìm hiểu một thành phần phản xạ hoặc một hàm tiện ích.

Ví dụ, giả sử nhân viên lái xe taxi không nhận được tiền boa từ những hành khách đã bị rung chuyển hoàn toàn trong chuyến đi. Tiêu chuẩn hiệu suất bên ngoài phải thông báo cho tác nhân rằng mất tiền boa là một đóng góp tiêu cực vào hiệu suất tổng thể của nó; Sau đó đặc vụ có thể biết rằng các thao tác bạo lực không góp phần vào tiện ích của chính nó. Theo nghĩa là, tiêu chuẩn hiệu suất phân biệt một phần của nhận thức đến như một phần thưởng

Thường
Hình phạt

(hoặc hình phạt) cung cấp phản hồi trực tiếp về chất lượng hành vi của tác nhân. Có dây cung

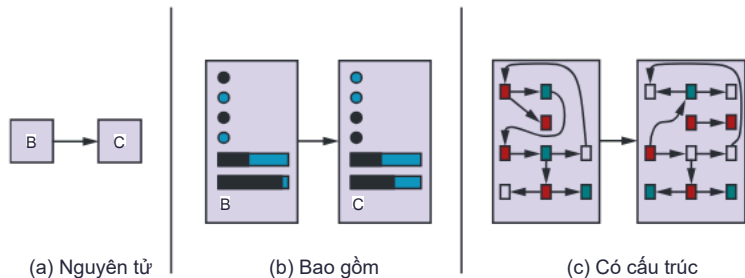
Các tiêu chuẩn hiệu suất như đau và đói ở động vật có thể được hiểu theo cách này.

Nói chung, lựa chọn của con người có thể cung cấp thông tin về sở thích của con người. Ví dụ, giả sử taxi không biết rằng mọi người thường không thích tiếng ồn lớn và quyết định với ý tưởng thổi còi liên tục như một cách để đảm bảo rằng người đi bộ biết rằng nó đang đến. Hành vi của con người - che tai, sử dụng ngôn ngữ xấu và có thể cắt dây vào còi - sẽ cung cấp bằng chứng cho tác nhân để cập nhật chức năng tiện ích của nó. Vấn đề này được thảo luận thêm trong Chương 23.

Tóm lại, các tác nhân có nhiều thành phần khác nhau và các thành phần đó có thể được phản ánh theo nhiều cách trong chương trình đại lý, vì vậy dường như có rất nhiều phương pháp học tập. Tuy nhiên, có một chủ đề thống nhất duy nhất. Học tập trong các tác nhân thông minh có thể được tóm tắt như một quá trình sửa đổi từng thành phần của tác nhân để đưa các thành phần phù hợp hơn với thông tin phản hồi có sẵn, do đó cải thiện hiệu suất tổng thể của tác nhân.

2.4.7 Cách thức hoạt động của các thành phần của chương trình tác nhân

Chúng tôi đã mô tả các chương trình tác nhân (theo thuật ngữ rất cao) bao gồm các thành phần khác nhau, có chức năng trả lời các câu hỏi như: "Thế giới bây giờ như thế nào?" "Tôi nên làm gì bây giờ?" "Hành động của tôi làm gì?" Câu hỏi tiếp theo dành cho một sinh viên của Allà, "Làm thế nào để các thành phần này hoạt động?" Phải mất khoảng một nghìn trang để bắt đầu trả lời câu hỏi đó một cách chính xác, nhưng ở đây chúng tôi muốn thu hút sự chú ý của người đọc đến một số điểm khác nhau giữa các cách khác nhau mà các thành phần có thể đại diện cho môi trường mà tác nhân sinh sống. Một cách đại khái, chúng ta có thể đặt các biểu diễn dọc theo một trục của sự phức tạp và sức mạnh biểu hiện ngày càng tăng - nguyên tử, yếu tố và cấu trúc. Để minh họa những ý tưởng này, bạn nên xem xét một thành phần tác nhân cụ thể, chẳng hạn như thành phần liên quan đến "Hành động của tôi làm gì". Thành phần này mô tả những thay đổi có thể xảy ra trong môi trường như là kết quả của các việc thực hiện một hành động và Hình 2.16 cung cấp mô tả sơ đồ về cách các chuyển đổi đó có thể được thể hiện.



Hình 2.16 Ba cách để biểu diễn các trạng thái và sự chuyển đổi giữa chúng: (a) Biểu diễn nguyên tử: trạng thái (chẳng hạn như B hoặc C) là một hộp đen không có cấu trúc bên trong; (b) Biểu diễn nhân thừa: một trạng thái bao gồm một vectơ các giá trị thuộc tính; giá trị có thể là Boolean, giá trị thực hoặc một trong một tập hợp các ký hiệu cố định. (c) Biểu diễn có cấu trúc: một trạng thái bao gồm các đối tượng, mỗi đối tượng có thể có các thuộc tính riêng cũng như các mối quan hệ với các đối tượng khác.

Trong một biểu diễn nguyên tử, mỗi trạng thái của thế giới là không thể phân chia - nó không có nội bộ

Đại diện nguyên tử

cấu trúc. Hãy xem xét nhiệm vụ tìm một tuyến đường lái xe từ đầu này của một quốc gia đến đầu kia thông qua một số chuỗi các thành phố (chúng tôi giải quyết vấn đề này trong Hình 3.1 trên trang 82). Đối với mục đích giải quyết vấn đề này, có thể đủ để giảm trạng thái của thế giới xuống chỉ còn tên của thành phố mà chúng ta đang ở - một nguyên tử duy nhất của kiến thức, một "hộp đen" mà sở thích duy nhất có thể nhận thấy là giống hoặc khác với một hộp đen khác. Các thuật toán tiêu chuẩn cơ bản cho tìm kiếm và chơi trò chơi (Chương 3, 4 và 6), các mô hình Markov ẩn (Chương 14) và các quá trình quyết định Markov (Chương 16) đều hoạt động với các biểu diễn nguyên tử.

Một biểu diễn được phân tích phân chia mỗi trạng thái thành một tập hợp các biến hoặc thuộc tính cố định, mỗi trong số đó có thể có một giá trị. Xem xét mô tả có độ trung thực cao hơn cho cùng một lần lái xe vấn đề, nơi chúng ta cần quan tâm nhiều hơn là chỉ vị trí nguyên tử ở thành phố này hay thành phố khác; chúng ta có thể cần chú ý đến lượng xăng trong bình, tọa độ GPS hiện tại của chúng ta, đèn cảnh báo dầu có hoạt động hay không, chúng ta có bao nhiêu tiền cho đường đi, đài nào trên radio, v.v. Trong khi hai trạng thái nguyên tử khác nhau không có gì lạ - chúng chỉ là những hộp đen khác nhau - hai trạng thái thừa tổ khác nhau có thể chia sẻ một số thuộc tính (chẳng hạn như ở một số vị trí GPS cụ thể) và không phải những thuộc tính khác (chẳng hạn như có nhiều khí hoặc không có khí); Điều này làm cho việc tìm ra cách biến trạng thái này thành trạng thái khác trở nên dễ dàng hơn nhiều. Nhiều lĩnh vực quan trọng của AI dựa trên các biểu diễn nhân tử, bao gồm các thuật toán thỏa mãn ràng buộc (Chương 5), logic mệnh đề (Chương 7), lập kế hoạch (Chương 11), mạng Bayes (Chương 12, 13, 14, 15 và 18) và các thuật toán học máy khác nhau.

Factored representation
VariableAttribute

Giá trị

Vì nhiều mục đích, chúng ta cần hiểu thế giới là có những thứ trong đó có liên quan đến nhau, không chỉ là các biến có giá trị. Ví dụ, chúng ta có thể nhận thấy rằng một chiếc xe tải lớn phía trước chúng ta đang lùi vào vũng lầy xe của một trang trại bò sữa, nhưng một lông leo đang chặn đường của chiếc xe tải. Một đại diện có yếu tố không có khả năng được trang bị sẵn với TruckAheadBackingIntoDairyFarmDrivewayBlockedByLooseCow với giá trị đúng hoặc sai. Thay vào đó, chúng ta sẽ cần một biểu diễn có cấu trúc, trong đó các đối tượng như bò

Đại diện có cấu trúc

và xe tải và các mối quan hệ khác nhau và khác nhau của chúng có thể được mô tả rõ ràng (xem Hình 2.16 (c)). Các biểu diễn có cấu trúc làm nền tảng cho cơ sở dữ liệu quan hệ và logic bậc nhất (Chương 8, 9 và 10), mô hình xác suất bậc nhất (Chương 18) và phần lớn hiểu biết ngôn ngữ tự nhiên (Chương 24 và 25). Trên thực tế, phần lớn những gì con người thể hiện bằng ngôn ngữ tự nhiên liên quan đến các đối tượng và mối quan hệ của chúng.

Như chúng tôi đã đề cập trước đó, trực mà dọc theo đó các nguyên tử, yếu tố và cấu trúc nằm là trục của sự biểu đạt ngày càng tăng. Nói một cách đại khái, một biểu cảm hơn

Biểu cảm

đại diện có thể nắm bắt, ít nhất là ngắn gọn, mọi thứ ít biểu cảm hơn có thể nắm bắt, cộng với một số hơn nữa. Thông thường, ngôn ngữ biểu cảm hơn ngắn gọn hơn nhiều; Ví dụ, các quy tắc của cờ vua có thể được viết bằng một hoặc hai trang của một ngôn ngữ biểu diễn có cấu trúc như logic bậc nhất nhưng yêu cầu hàng nghìn trang khi được viết bằng ngôn ngữ biểu diễn thừa số như logic mệnh đề và khoảng 1038 trang khi được viết bằng ngôn ngữ nguyên tử như của máy tự động trạng thái hữu hạn. Mặt khác, lý luận và học tập trở nên phức tạp hơn khi sức mạnh biểu đạt của đại diện tăng lên. Để đạt được lợi ích của các biểu diễn biểu đạt trong khi tránh những nhược điểm của chúng, các hệ thống thông minh cho thế giới thực có thể cần phải hoạt động đồng thời ở tất cả các điểm dọc theo trục.

Một trục khác để biểu diễn liên quan đến việc ánh xạ các khái niệm đến các vị trí trong bộ nhớ vật lý, cho dù trong máy tính hay trong não. Nếu có một ánh xạ một-một giữa các khái niệm và vị trí ký ức, chúng tôi gọi đó là biểu diễn địa phương. Mặt

Biểu diễn địa phương

Nếu biểu diễn của một khái niệm được trải rộng trên nhiều vị trí bộ nhớ và mỗi vị trí bộ nhớ được sử dụng như một phần của biểu diễn của nhiều khái niệm khác nhau, chúng ta gọi đó là biểu diễn phân tán. Các biểu diễn phân tán mạnh mẽ hơn chống nhiễu và mất thông tin. Với một biểu diễn địa phương, ánh xạ từ khái niệm đến vị trí bộ nhớ là tùy tiện, và nếu lỗi truyền đi bị cắt xén một vài bit, chúng ta có thể nhầm lẫn Truck với khái niệm không liên quan Truce. Nhưng với một biểu diễn phân tán, bạn có thể nghĩ về mỗi khái niệm đại diện cho một điểm trong không gian đa chiều, và nếu bạn cắt xén một vài bit, bạn sẽ di chuyển đến một điểm gần đó trong không gian đó, điều này sẽ có ý nghĩa tương tự.

Tóm tắt

Chương này là một chuyến tham quan về AI, mà chúng tôi đã hình thành như khoa học về thiết kế tác nhân. Những điểm chính cần nhớ lại như sau:

- Tác nhân là thứ nhận thức và hành động trong một môi trường. Chức năng tác nhân cho một tác nhân chỉ định hành động được thực hiện bởi tác nhân để phản ứng với bất kỳ chuỗi nhận thức nào.
- Thước đo hiệu suất đánh giá hành vi của tác nhân trong môi trường. Tác nhân vô lý hoạt động để tối đa hóa giá trị kỳ vọng của thước đo hiệu suất, với trình tự nhận thức mà nó đã thấy cho đến nay.
- Đặc điểm kỹ thuật môi trường tác vụ bao gồm thước đo hiệu suất, môi trường bên ngoài, bộ truyền động và cảm biến. Khi thiết kế một tác nhân, bước đầu tiên phải luôn là chỉ định môi trường tác vụ càng đầy đủ càng tốt.
- Môi trường nhiệm vụ thay đổi theo một số khía cạnh quan trọng. Chúng có thể có thể quan sát được đầy đủ hoặc riêng lẻ, tác nhân đơn hoặc đa tác nhân, xác định hoặc không xác định, tuần tự theo từng tập, tĩnh hoặc động, rời rạc hoặc liên tục, và đã biết hoặc chưa biết.
- Trong trường hợp không biết thước đo hiệu suất hoặc khó xác định chính xác, có nguy cơ đáng kể tác nhân tối ưu hóa sai mục tiêu. Trong những trường hợp như vậy, agentdesign nên phản ánh sự không chắc chắn về mục tiêu thực sự.
- Chương trình tác nhân thực hiện chức năng tác nhân. Có một loạt các thiết kế chương trình cơ bản phản ánh loại thông tin được làm rõ ràng và được sử dụng trong quá trình quyết định. Các thiết kế khác nhau về hiệu quả, nhỏ gọn và linh hoạt. Thiết kế phù hợp của chương trình đại lý phụ thuộc vào bản chất của môi trường.
- Các tác nhân phản xạ đơn giản phản ứng trực tiếp với nhận thức, trong khi các tác nhân phản xạ dựa trên mô hình duy trì trạng thái bên trong để theo dõi các khía cạnh của thế giới không rõ ràng trong nhận thức hiện tại. Các tác nhân dựa trên mục tiêu hành động để đạt được mục tiêu của họ và các tác nhân dựa trên tiện ích cố gắng tối đa hóa "hạnh phúc" mong đợi của chính họ.
- Tất cả các nhân viên có thể cải thiện hiệu suất của họ thông qua học tập.

Ghi chú thư mục và lịch sử

Vai trò trung tâm của hành động trong trí thông minh - khái niệm về lý luận thực tiễn - ít nhất là từ Đạo đức học Nicomachean của Aristotle. Lý luận thực tế cũng là chủ đề của bài báo có ảnh hưởng của McCarthy "Các chương trình với ý thức chung" (1958). Các lĩnh vực của người máy và lý thuyết điều khiển, về bản chất của chúng, chủ yếu liên quan đến các tác nhân vật lý. Các

khái niệm về một bộ điều khiển trong lý thuyết điều khiển giống hệt với khái niệm của một tác nhân trong AI. Có lẽ sur-

Điều khiển

Đáng ngạc nhiên, AI đã tập trung trong phần lớn lịch sử của nó vào các thành phần riêng lẻ của các tác nhân - hệ thống trả lời câu hỏi, người chứng minh định lý, hệ thống thị giác, v.v. - hơn là toàn bộ tác nhân. Cuộc thảo luận về các tác nhân trong văn bản của Genesereth và Nilsson (1987) là một ngoại lệ có ảnh hưởng. Quan điểm toàn tác nhân hiện được chấp nhận rộng rãi và là một chủ đề trung tâm trong các văn bản gần đây (Padgham và Winikoff, 2004; Jones, 2007; Poole và Mackworth, 2017). Chương 1 lần theo nguồn gốc của khái niệm hợp lý trong triết học và kinh tế học. Trong AI, khái niệm này được quan tâm ngoại vi cho đến giữa những năm 1980, khi nó bắt đầu tràn ngập nhiều cuộc thảo luận về nền tảng kỹ thuật thích hợp của lĩnh vực này. Một bài báo của Jon Doyle (1983) dự đoán rằng thiết kế tác nhân hợp lý sẽ được coi là sứ mệnh cốt lõi của AI, trong khi các chủ đề phổ biến khác sẽ tách ra để hình thành các lĩnh vực mới.

Sự chú ý cẩn thận đến các đặc tính của môi trường và hậu quả của chúng đối với thiết kế tác nhân ra-tional là rõ ràng nhất trong truyền thống lý thuyết điều khiển — ví dụ, các hệ thống điều khiển cổ điển (Dorf và Bishop, 2004; Kirk, 2004) xử lý hoàn toàn có thể quan sát được, môi trường xác định; kiểm soát tối ưu ngẫu nhiên (Kumar và Varaiya, 1986; Bertsekas và Shreve, 2007) xử lý các môi trường ngẫu nhiên, có thể quan sát được một phần; và kiểm soát lai (Henzinger và Sastry, 1998; Cassandras và Lygeros, 2006) liên quan đến các môi trường chứa cả các yếu tố rời rạc và liên tục. Sự khác biệt giữa môi trường có thể quan sát được hoàn toàn và một phần cũng là trung tâm trong tài liệu lập trình động được phát triển trong lĩnh vực nghiên cứu hoạt động (Puterman, 1994), mà chúng ta thảo luận trong Chương 16.

Mặc dù các tác nhân phản xạ đơn giản là trung tâm của tâm lý học hành vi (xem Chương 1), hầu hết các nhà nghiên cứu AI coi chúng là quá đơn giản để cung cấp nhiều đòn bẩy. (Rosenschein (1985) và Brooks (1986) đã đặt câu hỏi về giả định này; xem Chương 26.) Rất nhiều công việc đã được thực hiện để tìm ra các thuật toán hiệu quả để theo dõi các môi trường phức tạp (Bar-Shalom và cộng sự, 2001; Choset và cộng sự, 2005; Simon, 2006), hầu hết trong bối cảnh xác suất.

Các tác nhân dựa trên mục tiêu được giả định trước trong mọi thứ, từ quan điểm của Aristotle về reasoning thực tế đến các bài báo đầu tiên của McCarthy về AI logic. Shakey the Robot (Fikes và Nilsson, 1971; Nilsson, 1984) là hiện thân robot đầu tiên của một tác nhân logic, dựa trên mục tiêu. Phân tích logic đầy đủ của các tác nhân dựa trên mục tiêu xuất hiện trong Genesereth và Nilsson (1987), và phương pháp lập trình dựa trên mục tiêu được gọi là lập trình hướng tác nhân được phát triển bởi Shoham (1993). Cách tiếp cận dựa trên tác nhân hiện nay cực kỳ phổ biến trong kỹ thuật phần mềm (Ciancarini và Wooldridge, 2001). Nó cũng đã thâm nhập vào lĩnh vực hệ điều hành, trong đó điện toán tự động đề cập đến các hệ thống máy tính và mạng giám sát và điều khiển

Máy tính tự động

tự điều khiển bản thân bằng một vòng lặp nhận thức-hành động và các phương pháp học máy (Kephart và Chess, 2003). Lưu ý rằng một tập hợp các chương trình tác nhân được thiết kế để hoạt động tốt cùng nhau trong môi trường đa tác nhân thực sự nhất thiết phải thể hiện tính mô-đun - các chương trình không chia sẻ trạng thái nội bộ và chỉ giao tiếp với nhau thông qua môi trường - phổ biến trong lĩnh vực hệ thống đa tác nhân để thiết kế chương trình tác nhân của một tác nhân duy nhất như một tập hợp các tác nhân phụ tự trị. Trong một số trường hợp, người ta thậm chí có thể chứng minh rằng hệ thống kết quả đưa ra các giải pháp tối ưu giống như thiết kế nguyên khối.

Quan điểm dựa trên mục tiêu của các tác nhân cũng thống trị truyền thống tâm lý học nhận thức trong lĩnh vực giải quyết vấn đề, bắt đầu với Giải quyết vấn đề con người có ảnh hưởng to lớn (Newell và Simon, 1972) và chạy xuyên suốt tất cả các công trình sau này của Newell (Newell, 1990). Mục tiêu, được phân tích thêm như mong muốn (chung) và ý định (hiện đang theo đuổi), là trung tâm của lý thuyết có ảnh hưởng về các tác nhân được phát triển bởi Michael Bratman (1987).

Như đã lưu ý trong Chương 1, sự phát triển của lý thuyết tiện ích như một cơ sở cho hành vi hợp lý đã có từ hàng trăm năm trước. Trong AI, nghiên cứu ban đầu tránh các tiện ích để ủng hộ các mục tiêu, với một số ngoại lệ (Feldman và Sproull, 1977). Sự hồi sinh của sự quan tâm đến các phương pháp xác suất vào những năm 1980 đã dẫn đến việc chấp nhận tối đa hóa tiện ích dự kiến như là khuôn khổ chung nhất để ra quyết định (Horvitz et al., 1988). Văn bản của Pearl (1988) là văn bản đầu tiên trong AI đề cập sâu về xác suất và lý thuyết tiện ích; sự giải thích của nó về methods thực tế để lập luận và ra quyết định trong điều kiện không chắc chắn có lẽ là yếu tố lớn nhất trong sự chuyển đổi nhanh chóng sang các tác nhân dựa trên tiện ích trong những năm 1990 (xem Chương 15). Sự sai lầm hóa của học tăng cường trong khuôn khổ lý thuyết quyết định cũng góp phần vào sự thay đổi này (Sutton, 1988). Hơi đáng chú ý, hầu hết tất cả các nghiên cứu AI cho đến gần đây đều giả định rằng thước đo hiệu suất có thể được chỉ định chính xác và chính xác dưới dạng hàm tiện ích hoặc hàm phần thưởng (Hadfield-Menell và cộng sự, 2017a; Russell, 2019).

Thiết kế chung cho các tác nhân học tập được mô tả trong Hình 2.15 là cổ điển trong tài liệu học máy (Buchanan et al., 1978; Mitchell, 1997). Các ví dụ về thiết kế, như được thể hiện trong các chương trình, ít nhất là từ chương trình học của Arthur Samuel (1959, 1967) để chơi cờ đam. Các tác nhân học tập được thảo luận sâu trong các Chương 19, 21, 22 và 23.

Một số bài báo ban đầu về các phương pháp tiếp cận dựa trên tác nhân được thu thập bởi Huhns và Singh (1998) và Wooldridge và Rao (1999). Các văn bản về hệ thống đa tác nhân cung cấp một giới thiệu tốt về nhiều khía cạnh của thiết kế tác nhân (Weiss, 2000a; Wooldridge, 2009). Một số chuỗi hội nghị dành cho các tác nhân bắt đầu vào những năm 1990, bao gồm Hội thảo Quốc tế về Lý thuyết, Kiến trúc và Ngôn ngữ Đại lý (ATAL), Hội nghị Quốc tế về Tác nhân Tự động (AGENTS) và Hội nghị Quốc tế về Hệ thống Đa tác nhân (ICMAS). Năm 2002, ba tác nhân này hợp nhất để tạo thành Hội nghị chung quốc tế về các tác nhân tự trị và hệ thống đa tác nhân (AAMAS). Từ năm 2000 đến năm 2012 đã có các hội thảo hàng năm về Kỹ thuật phần mềm định hướng đại lý (AOSE). Tạp chí Autonomous Agents and Multi-Agent Systems được thành lập vào năm 1998. Cuối cùng, Sinh thái bộ cánh cứng phân (Hanski và Cambefort, 1991) cung cấp rất nhiều thông tin thú vị về hành vi của bộ phân. YouTube có các bản ghi video đầy cảm hứng về các hoạt động của họ.

CHƯƠNG 3

GIẢI QUYẾT VẤN ĐỀ BẰNG CÁCH TÌM KIẾM

Trong đó chúng ta thấy cách một tác nhân có thể nhìn về phía trước để tìm ra một chuỗi các hành động sẽ đạt được mục tiêu của nó.

Khi hành động chính xác cần thực hiện không rõ ràng ngay lập tức, một tác nhân có thể cần phải lên kế hoạch trước: xem xét một chuỗi các hành động tạo thành một con đường dẫn đến trạng thái mục tiêu. Một tác nhân như vậy được gọi là tác nhân giải quyết vấn đề và quá trình tính toán mà nó thực hiện được gọi là tìm kiếm.

Tác nhân giải quyết vấn đề tìm kiếm

Các tác nhân giải quyết vấn đề sử dụng các biểu diễn nguyên tử, như được mô tả trong Phần 2.4.7 — nghĩa là, các trạng thái của thế giới được coi là tổng thể, không có cấu trúc bên trong nào có thể nhìn thấy đối với các thuật toán giải quyết vấn đề. Các tác nhân sử dụng các đại diện theo yếu tố hoặc có cấu trúc của các quốc gia được gọi là các tác nhân lập kế hoạch và được thảo luận trong Chương 7 và 11.

Chúng tôi sẽ đề cập đến một số thuật toán tìm kiếm. Trong chương này, chúng ta chỉ xem xét các môi trường đơn giản nhất: từng đợt, tác nhân đơn, hoàn toàn có thể quan sát được, xác định, tĩnh, rời rạc và đã biết. Chúng tôi phân biệt giữa các thuật toán được thông báo, trong đó tác nhân có thể ước tính khoảng cách của nó từ mục tiêu và các thuật toán không có thông tin, nơi không có ước tính như vậy. Chương 4 nói lỏng các ràng buộc về môi trường và Chương 6 xem xét nhiều tác nhân. Chương này sử dụng các khái niệm về độ phức tạp tiệm cận (nghĩa là ký hiệu $O(n)$). Độ giả không quen thuộc với các khái niệm này nên tham khảo Phụ lục A.

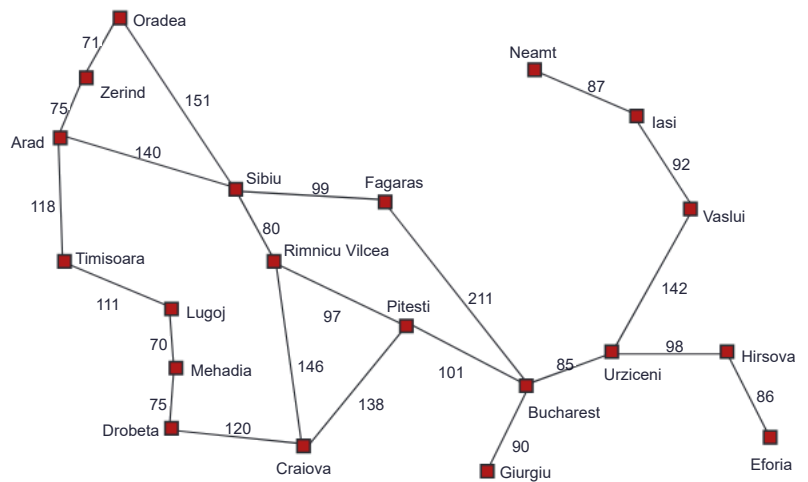
3.1 Đại lý giải quyết vấn đề

Hãy tưởng tượng một đại lý đang tận hưởng một kỳ nghỉ lưu diễn ở Romania. Đặc vụ muốn ngắm cảnh, cải thiện tiếng Romania của mình, tận hưởng cuộc sống về đêm, tránh náo náo, v.v. Vấn đề quyết định là một vấn đề phức tạp. Bây giờ, giả sử đại lý hiện đang ở thành phố Arad và có một vé không hoàn lại để bay ra khỏi Bucharest vào ngày hôm sau. Đặc vụ quan sát các biển báo đường phố và thấy rằng có ba con đường dẫn ra khỏi Arad: một đến Sibiu, một đến Timisoara và một đến Zerind. Không ai trong số này là mục tiêu, vì vậy trừ khi đặc vụ quen thuộc với địa lý của Romania, nó sẽ không biết nên đi theo con đường nào.

Nếu tác nhân không có thông tin bổ sung - nghĩa là, nếu môi trường không xác định - thì tác nhân không thể làm gì tốt hơn là thực hiện một trong các hành động một cách ngẫu nhiên. Tình huống đáng buồn này được thảo luận trong Chương 4. Trong chương này, chúng ta sẽ giả định rằng các tác nhân của chúng ta luôn có quyền truy cập vào thông tin về thế giới, chẳng hạn như bản đồ trong Hình 3.1. Với thông tin đó, tác nhân có thể làm theo quy trình giải quyết vấn đề bốn giai đoạn này:

- Xây dựng mục tiêu: Tác nhân thông qua mục tiêu đến Bucharest. Tổ chức mục tiêu xây dựng mục tiêu hành vi bằng cách giới hạn các mục tiêu và do đó các hành động cần được xem xét.

1 Chúng tôi giả định rằng hầu hết độc giả đều ở cùng một vị trí và có thể dễ dàng tưởng tượng mình là người không biết gì như tác nhân của chúng tôi. Chúng tôi xin lỗi độc giả Romania không thể tận dụng công cụ sự phạm này.



Hình 3.1 Sơ đồ đường bộ đơn giản của một phần của Romania, với khoảng cách đường bộ tính bằng dặm.

Xây dựng vấn đề	• Xây dựng vấn đề: Tác nhân đưa ra mô tả về các trạng thái và hành động cần thiết để đạt được mục tiêu - một mô hình trừu tượng của phần liên quan của thế giới. Đối với đại lý của chúng tôi, một mô hình tốt là xem xét các hành động đi từ một thành phố đến một thành phố liền kề, và do đó thực tế duy nhất về tình trạng của thế giới sẽ thay đổi do một hành động là thành phố hiện tại.
Tìm kiếm	• Tìm kiếm: Trước khi thực hiện bất kỳ hành động nào trong thế giới thực, tác nhân mô phỏng các chuỗi hành động trong mô hình của nó, tìm kiếm cho đến khi nó tìm thấy một chuỗi hành động đạt được mục tiêu. Một trình tự như vậy được gọi là giải pháp. Tác nhân có thể phải mô phỏng nhiều
Giải pháp	những điều không đạt được mục tiêu, nhưng cuối cùng nó sẽ tìm ra một giải pháp (chẳng hạn như đi từ Arad đến Sibiu đến Fagaras đến Bucharest), hoặc nó sẽ thấy rằng không có giải pháp nào là khả thi.
Thực hiện	• Thực thi: Giờ đây, tác nhân có thể thực hiện từng hành động trong giải pháp.
Vòng hở	Đó là một thuộc tính quan trọng rằng trong một môi trường hoàn toàn có thể quan sát được, xác định, đã biết, giải pháp cho bất kỳ vấn đề nào là một chuỗi cố định của các hành động: lái xe đến Sibiu, sau đó là Fagaras, sau đó Bucharest. Nếu mô hình là đúng, thì mỗi khi tác nhân đã tìm ra một giải pháp, nó có thể bỏ qua các nhận thức của nó trong khi nó đang thực hiện các hành động - có thể nói là nhắm mắt lại - bởi vì giải pháp được đảm bảo sẽ dẫn đến mục tiêu. Các nhà lý thuyết điều khiển gọi đây là hệ thống vòng hở: bỏ qua
Vòng kín	nhận thức phá vỡ vòng lặp giữa tác nhân và môi trường. Nếu có khả năng mô hình không chính xác, hoặc môi trường không xác định, thì tác nhân sẽ an toàn hơn bằng cách sử dụng cách tiếp cận vòng kín để giám sát các nhận thức (xem Phần 4.4). Trong môi trường có thể quan sát được một phần hoặc không xác định, một giải pháp sẽ là một chiến lược phân nhánh đề xuất các hành động khác nhau trong tương lai tùy thuộc vào những nhận thức nào đến. Ví dụ, người đại diện có thể lên kế hoạch lái xe từ Arad đến Sibiu nhưng có thể cần một kế hoạch dự phòng trong trường hợp nó đến Zerind một cách tình cờ hoặc tìm thấy một biển báo có nội dung "Drum "Inchis" (Đường đã đóng).

3.1.1 Tìm kiếm vấn đề và giải pháp

Một bài toán tìm kiếm có thể được định nghĩa chính thức như

sau: Một tập hợp các trạng thái có thể xảy ra mà môi trường có thể xảy ra. Chúng tôi gọi đây là không gian trạng thái.

- Trạng thái ban đầu mà tác nhân bắt đầu. Ví dụ: Arad.
- Một tập hợp một hoặc nhiều trạng thái mục tiêu. Đôi khi có một trạng thái mục tiêu (ví dụ: Bucharest), đôi khi có một tập hợp nhỏ các trạng thái mục tiêu thay thế và đôi khi mục tiêu được xác định bởi một thuộc tính áp dụng cho nhiều trạng thái (có khả năng là một số vô hạn). Ví dụ, trong thể giới máy hút bụi, mục tiêu có thể là không có bụi bẩn ở bất kỳ vị trí nào, bất kể bất kỳ sự kiện nào khác về tiểu bang. Chúng ta có thể giải thích cả ba khả năng này bằng cách chỉ định phương pháp IS-GOAL cho một vấn đề. Trong chương này, đôi khi chúng ta sẽ nói "mục tiêu" cho sự đơn giản, nhưng những gì chúng ta nói cũng áp dụng cho "bất kỳ một trong các trạng thái mục tiêu khả thi".
- Các hành động có sẵn cho đại lý. Cho một trạng thái s, ACTIONS(s) trả về một tập hợp hữu hạn2 của các hành động có thể được thực hiện trong s. Chúng tôi nói rằng mỗi hành động này có thể áp dụng trong s.

Một ví dụ:

ACTIONS (Arad) = {ToSibiu, ToTimisoara, ToZerind} .

- Một mô hình chuyển tiếp, mô tả những gì mỗi hành động làm. RESULT(s, a) trả về trạng thái là kết quả của việc thực hiện hành động a trong trạng thái s. Chẳng hạn

KẾT QUẢ (Arad, ToZerind) = Zerind

- Một hàm-chi phí hành động, được ký hiệu bằng ACTION-COST (s, a, s') khi chúng ta đang lập trình

hoặc c (s, a, s') khi chúng ta đang làm toán, điều đó đưa ra chi phí số của việc áp dụng hành động trong trạng thái s để đạt được trạng thái s'. Một tác nhân giải quyết vấn đề nên sử dụng hàm chi phí phản ánh thước đo hiệu suất của chính nó; ví dụ: đối với các tác nhân tìm tuyến đường, chi phí của một hành động có thể là độ dài tính bằng dặm (như trong Hình 3.1) hoặc có thể là thời gian cần thiết để hoàn thành hành động. Một chuỗi các hành động tạo thành một con đường và một giải pháp là một con đường từ trạng thái ban đầu đến một mục tiêu

tiểu bang. Chúng tôi giả định rằng chi phí hành động là cộng thêm; nghĩa là, tổng chi phí của một đường dẫn là tổng của chi phí hành động cá nhân. Một giải pháp tối ưu có chi phí đường dẫn thấp nhất trong số tất cả các giải pháp. Trong Chương này, chúng tôi giả định rằng tất cả các chi phí hành động sẽ là dương, để tránh một số phức tạp nhất định.3

Không gian trạng thái có thể được biểu diễn dưới dạng đồ thị trong đó các đỉnh là các trạng thái và đồ thị các cạnh có hướng giữa chúng là các hành động. Bản đồ Romania được hiển thị trong Hình 3.1 là biểu đồ như vậy, trong đó mỗi con đường chỉ ra hai hành động, một hành động theo mỗi hướng.

2 Đối với các vấn đề với vô số hành động, chúng ta sẽ cần các kỹ thuật vượt ra ngoài chương này.3 Trong bất kỳ vấn đề nào với một chu kỳ chi phí âm, giải pháp tối ưu về chi phí là đi vòng quanh chu kỳ đó vô số lần. Các thuật toán Bellman-Ford và Floyd-Warshall (không được đề cập ở đây) xử lý các hành động chi phí âm, miễn là không có chu kỳ âm. Thật dễ dàng để phù hợp với các hành động không chi phí, miễn là số lượng hành động không chi phí liên tiếp được giới hạn. Ví dụ: chúng ta có thể có một robot có chi phí di chuyển, nhưng không tốn chi phí để xoay 90o; Các thuật toán trong chương này có thể xử lý điều này miễn là không được phép quá ba lượt 90o liên tiếp. Ngoài ra còn có một sự phức tạp với các vấn đề có vô số chi phí hành động nhỏ tùy ý. Hãy xem xét một phiên bản nghịch lý của Zeno, trong đó có một hành động để di chuyển nửa chặng đường đến mục tiêu, với chi phí là một nửa so với nước đi trước đó. Vấn đề này không có giải pháp với một số lượng hành động hữu hạn, nhưng để ngăn chặn một tìm kiếm thực hiện một số lượng hành động không giới hạn mà không hoàn toàn đạt được mục tiêu, chúng ta có thể yêu cầu rằng tất cả các chi phí hành động ít nhất là , cho một số giá trị dương nhỏ .

Vấn đề

Trạng tháiKhông gian trạng thái

Trạng thái ban đầu

Trạng thái mục tiêu

Hành động

Áp dụng

Mô hình chuyển tiếp

Chức năng chi phí hành động

Con đường

Giải pháp tối ưu

3.1.2 Xây dựng vấn đề

Công thức của chúng ta về vấn đề đến Bucharest là một mô hình - một mô tả toán học trừu tượng - và không phải là điều thực tế. So sánh mô tả nhà nước nguyên tử đơn giản Arad với một chuyến đi xuyên quốc gia thực tế, nơi tình trạng của thế giới bao gồm rất nhiều thứ: những người bạn đồng hành, chương trình phát thanh hiện tại, phong cảnh ngoài cửa sổ, khoảng cách của các nhân viên thực thi pháp luật, khoảng cách đến điểm dừng chân tiếp theo, tình trạng của con đường, thời tiết, giao thông, v.v. Tất cả những cân nhắc này đều bị loại khỏi mô hình của chúng tôi vì chúng không liên quan đến vấn đề tìm kiếm một tuyến đường đến Bucharest.

Trừu tượng

Quá trình loại bỏ chi tiết khỏi một biểu diễn được gọi là trừu tượng. Một tốt Công thức vấn đề có mức độ chi tiết phù hợp. Nếu các hành động ở mức độ "di chuyển chân phải về phía trước một cm" hoặc "xoay vô lăng sang trái một độ", đặc vụ có thể sẽ không bao giờ tìm thấy đường ra khỏi bãi đậu xe, chứ đừng nói đến Bucharest.

Mức độ trừu tượng

Chúng ta có thể chính xác hơn về mức độ trừu tượng thích hợp không? Hãy nghĩ về sự trừu tượng các trạng thái và hành động mà chúng tôi đã chọn tương ứng với các tập hợp lớn các trạng thái thế giới chi tiết và chuỗi hành động chi tiết. Bây giờ hãy xem xét một giải pháp cho vấn đề trừu tượng: ví dụ, con đường từ Arad đến Sibiu đến Rimnicu Vilcea đến Pitesti đến Bucharest. Giải pháp trừu tượng này tương ứng với một số lượng lớn các đường dẫn chi tiết hơn. Ví dụ, chúng tôi có thể lái xe với radio bật giữa Sibiu và Rimnicu Vilcea, và sau đó tắt nó trong phần còn lại của chuyến đi.

Sự trừu tượng là hợp lệ nếu chúng ta có thể xây dựng bất kỳ giải pháp trừu tượng nào thành một giải pháp trong thế giới chi tiết hơn; một điều kiện đủ là đối với mỗi trạng thái chi tiết "ở Arad", có một con đường chi tiết đến một trạng thái nào đó là "ở Sibiu", v.v.4 Sự trừu tượng hóa là hữu ích nếu thực hiện mỗi hành động trong lời giải dễ dàng hơn bài toán ban đầu; trong trường hợp của chúng tôi, hành động "lái xe từ Arad đến Sibiu" có thể được thực hiện mà không cần tìm kiếm hoặc lập kế hoạch thêm bởi một tài xế có kỹ năng trung bình. Do đó, việc lựa chọn một khái niệm trừu tượng tốt liên quan đến việc loại bỏ càng nhiều chi tiết càng tốt trong khi vẫn giữ được giá trị và đảm bảo rằng các hành động trừu tượng dễ thực hiện. Nếu không có khả năng xây dựng các trừu tượng hữu ích, các tác nhân thông minh sẽ hoàn toàn bị nhấn chìm bởi thế giới thực.

3.2 Vấn đề ví dụ

Phương pháp giải quyết vấn đề đã được áp dụng cho một loạt các môi trường tác vụ. Chúng tôi liệt kê một số vấn đề được biết đến nhiều nhất ở đây, phân biệt giữa các vấn đề tiêu chuẩn hóa và các vấn đề trong thế giới thực. Một vấn đề tiêu chuẩn hóa nhằm mục đích minh họa hoặc thực hiện các phương pháp giải quyết vấn đề khác nhau-

Vấn đề tiêu chuẩn hóa

Ods. Nó có thể được đưa ra một mô tả ngắn gọn, chính xác và do đó phù hợp làm tiêu chuẩn cho các nhà nghiên cứu so sánh hiệu suất của các thuật toán. Một vấn đề trong thế giới thực, chẳng hạn như robot

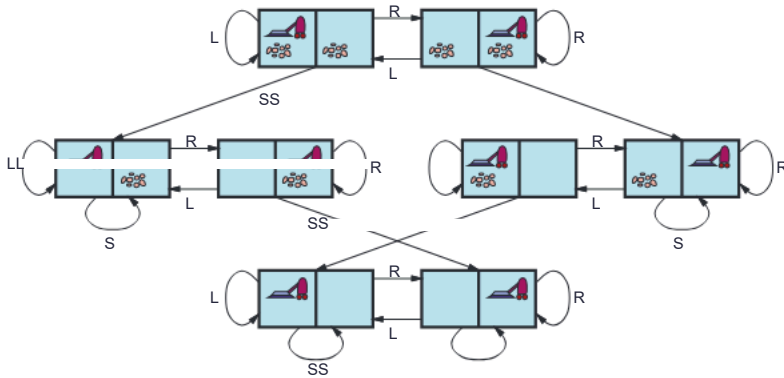
Vấn đề trong thế giới thực

Điều hướng, là một trong những giải pháp mà mọi người thực sự sử dụng và công thức của nó là đặc biệt, không được tiêu chuẩn hóa, bởi vì, ví dụ, mỗi robot có các cảm biến khác nhau tạo ra dữ liệu khác nhau.

3.2.1 Các bài toán tiêu chuẩn hóa

Thế giới lưới

Bài toán thế giới lưới là một mảng hình chữ nhật hai chiều của các ô vuông trong đó các tác nhân có thể di chuyển từ ô này sang ô khác. Thông thường, tác nhân có thể di chuyển đến bất kỳ ô liền kề nào không có chướng ngại vật — theo chiều ngang hoặc chiều dọc và trong một số vấn đề theo đường chéo. Các ô có thể chứa các đối tượng,
4 Xem Phần 11.4.



Hình 3.2 Biểu đồ không gian trạng thái cho thể giới chân không hai ô. Có 8 trạng thái và ba hành động cho mỗi trạng thái: L = Trái, R = Phải, S = Mút.

tác nhân có thể nhặt, đẩy hoặc hành động theo cách khác; Một bức tường hoặc chướng ngại vật không thể vượt qua khác trong Acell ngăn cản tác nhân di chuyển vào tế bào đó. Thể giới chân không từ Phần 2.1 có thể được xây dựng như một vấn đề thể giới lưới như sau:

- **Trạng thái:** Trạng thái của thể giới cho biết vật thể nào nằm trong tế bào nào. Đối với thể giới chân không, các vật thể là tác nhân và bất kỳ chất bẩn nào. Trong phiên bản hai ô đơn giản, tác nhân có thể nằm trong một trong hai ô, và mỗi ô có thể chứa bụi bẩn hoặc không, vì vậy có $2 \cdot 2 \cdot 2 = 8$ trạng thái (xem Hình 3.2). Nói chung, môi trường chân không với n tế bào đã $\cdot 2n$ trạng thái.
- **Trạng thái ban đầu:** Bất kỳ trạng thái nào cũng có thể được chỉ định là trạng thái ban đầu.
- **Hành động:** Trong thể giới hai ô, chúng tôi đã xác định ba hành động: Mút, di chuyển sang trái và di chuyển phải. Trong một thể giới đa tế bào hai chiều, chúng ta cần nhiều hành động chuyển động hơn. Chúng ta có thể thêm Upward và Downward, cung cấp cho chúng ta bốn hành động chuyển động tuyệt đối, hoặc chúng ta có thể chuyển sang các hành động ich kỷ, được xác định liên quan đến quan điểm của tác nhân — ví dụ: Forward, Backward, TurnRight và TurnLeft.
- **Mô hình chuyển tiếp:** Hút loại bỏ bụi bẩn khỏi tế bào của tác nhân; Chuyển tiếp tác nhân về phía trước một ô theo hướng mà nó đang đối mặt, trừ khi nó va vào tường, trong trường hợp đó hành động không có tác dụng. Backward di chuyển tác nhân theo hướng ngược lại, trong khi TurnRight và TurnLeft thay đổi hướng nó đang đối mặt với 90° .
- **Trạng thái mục tiêu:** Các trạng thái trong đó mọi tế bào đều sạch.
- **Chi phí hành động:** Mỗi hành động có giá 1. Một loại thể giới lưới khác là câu đố sokoban, trong đó mục tiêu của đặc vụ là đẩy một

Câu đố Sokoban

sổ hộp, nằm rải rác xung quanh lưới, đến các vị trí lưu trữ được chỉ định. Có thể có tối đa một hộp cho mỗi ô. Khi một tác nhân di chuyển về phía trước vào một ô chứa một hộp và có một ô trống ở phía bên kia của hộp, thì cả hộp và tác nhân đều di chuyển về phía trước.

7	2	4
5		6
8	3	1

Trạng thái bắt đầu

	1	2
34		5
67		8

Trạng thái mục tiêu

Hình 3.3 Một ví dụ điển hình của câu đố 8.

Đặc vụ không thể đẩy một chiếc hộp vào một chiếc hộp hoặc một bức tường khác. Đối với một thế giới có n ô không chương ngại vật và b hộp, có $n \times n!/(b!(n-b)!)$ Hoa Kỳ; Ví dụ, trên lưới 8×8 với hàng chục hộp, có hơn 200 nghìn tỷ tiểu bang.

Câu đố gạch trượt

Trong câu đố ô trượt, một số ô (đôi khi được gọi là khối hoặc mảnh) là ar-nằm trong một lưới với một hoặc nhiều khoảng trống để một số ô có thể trượt vào khoảng trống. Một biến thể là câu đố Giờ cao điểm, trong đó ô tô và xe tải trượt quanh lưới $a6 \times 6$ trong nỗ lực giải phóng ô tô khỏi tắc đường. Có lẽ biến thể nổi tiếng nhất là câu đố 8 (xem Hình 3.3), bao gồm một lưới 3×3 với tám ô được đánh số và một khoảng trống và 15 câu đố trên lưới 4×4 . Mục tiêu là đạt được một mục tiêu cụ thể, chẳng hạn như trạng thái được hiển thị ở bên phải của hình. Công thức tiêu chuẩn của 8puzzle như sau:

8 câu đố
15 câu đố

- **Trạng thái:** Mô tả trạng thái chỉ định vị trí của từng ô.
- **Trạng thái ban đầu:** Bất kỳ trạng thái nào cũng có thể được chỉ định là trạng thái ban đầu. Lưu ý rằng một sở thích chán lè phân chia không gian trạng thái - bất kỳ mục tiêu nhất định nào cũng có thể đạt được từ chính xác một nửa các trạng thái ban đầu có thể có (xem Bài tập 3.PHẦN).
- **Hành động:** Mặc dù trong thế giới vật lý, nó là một ô trượt, nhưng cách đơn giản nhất để mô tả một hành động là nghĩ về khoảng trống di chuyển Trái, Phải, Lên hoặc Xuống. Nếu khoảng trống ở cạnh hoặc góc thì không phải tất cả các hành động sẽ được áp dụng.
- **Mô hình chuyển tiếp:** Ảnh xạ trạng thái và hành động đến trạng thái kết quả; ví dụ, nếu chúng ta áp dụng Left cho trạng thái bắt đầu trong Hình 3.3, trạng thái kết quả có 5 và trống được chuyển đổi.
- **Trạng thái mục tiêu:** Mặc dù bất kỳ trạng thái nào cũng có thể là mục tiêu, nhưng chúng ta thường chỉ định một trạng thái với các con số theo thứ tự, như trong Hình
- **Chi phí hành động:** Mỗi hành động có chi phí 1. Lưu ý rằng mọi công thức vấn đề đều liên quan đến trừu tượng. Các hành động 8 câu đố được thực hiện ở trạng thái bắt đầu và cuối cùng của chúng, bỏ qua các vị trí trung gian nơi ô trượt. Chúng tôi đã trừu tượng hóa các hành động như lắc bằng khi gạch bị kẹt và loại trừ việc lấy gạch bằng dao và đặt chúng trở lại. Chúng ta chỉ còn lại một mô tả về các quy tắc, tránh tất cả các chi tiết của các thao tác vật lý. Bài toán tiêu chuẩn hóa cuối cùng của chúng ta được đưa ra bởi Donald Knuth (1964) và minh họa cách các không gian trạng thái vô hạn có thể phát sinh. Knuth phỏng đoán rằng bắt đầu với số 4, một dãy

của phép toán căn bậc hai, tầng và giai thừa có thể đạt được bất kỳ số nguyên dương mong muốn nào. Ví dụ, chúng ta có thể đạt được 5 từ 4 như sau:

$$\lfloor \sqrt{\sqrt{\sqrt{\sqrt{\sqrt{(4!)}}}}} \rfloor = 5.$$

Định nghĩa vấn đề rất đơn giản:

- **Tiểu bang:** Số thực dương.
- **Trạng thái ban đầu:** 4.
- **Hành động:** Áp dụng phép toán căn bậc hai, tầng hoặc giai thừa (giai thừa chỉ dành cho số nguyên).
- **Mô hình chuyển tiếp:** Như được đưa ra bởi các định nghĩa toán học của các phép toán.
- **Trạng thái mục tiêu:** Số nguyên dương
- **Chi phí hành động:** Mỗi hành động có giá 1. Không gian trạng thái cho bài toán này là vô hạn: đối với bất kỳ số nguyên nào lớn hơn 2, hoạt động giai thừa sẽ luôn mang lại số nguyên lớn hơn. Vấn đề rất thú vị vì nó khám phá những con số rất lớn: con đường ngắn nhất đến 5 đi qua $(4!)! = 620,448,401,733,239,439,360,000$. Không gian trạng thái vô hạn phát sinh thường xuyên trong các nhiệm vụ liên quan đến việc tạo ra các phép toán học, mạch, chứng minh, chương trình và các đối tượng được xác định đệ quy khác.

3.2.2 Các vấn đề trong thế giới thực

Chúng ta đã thấy bài toán tìm tuyến đường được định nghĩa như thế nào theo các vị trí và chuyển tiếp được chỉ định dọc theo các cạnh giữa chúng. Các thuật toán tìm tuyến đường được sử dụng trong nhiều ứng dụng khác nhau. Một số, chẳng hạn như các trang web và hệ thống trong xe cung cấp chỉ đường lái xe, là phần mở rộng tương đối đơn giản của ví dụ Romania. (Các phức tạp chính là chi phí khác nhau do sự chậm trễ phụ thuộc vào giao thông và định tuyến lại do ãng đường.) Những thứ khác, chẳng hạn như định tuyến các luồng video trong mạng máy tính, lập kế hoạch hoạt động quân sự và hệ thống lập kế hoạch du lịch hàng không, liên quan đến các thông số kỹ thuật phức tạp hơn nhiều. Hãy xem xét các vấn đề du lịch hàng không phải được giải quyết bằng một trang web lập kế hoạch du lịch:

- **Tiểu bang:** Mỗi tiểu bang rõ ràng bao gồm một địa điểm (ví dụ: sân bay) và thời gian hiện tại. Hơn nữa, vì chi phí của một hành động (một chặng bay) có thể phụ thuộc vào các chặng bay trước đó, cơ sở giá vé và tình trạng của chúng là trong nước hoặc quốc tế, nhà nước phải ghi lại thêm thông tin về các khía cạnh "lịch sử" này.
- **Trạng thái ban đầu:** Sân bay nhà của người dùng.
- **Hành động:** Đi bất kỳ chuyến bay nào từ vị trí hiện tại, ở bất kỳ hạng ghế nào, khởi hành sau thời gian hiện tại, chừa đủ thời gian để di chuyển trong sân bay nếu cần.
- **Mô hình chuyển tiếp:** Trạng thái kết quả của chuyến bay sẽ có quốc gia của chuyến bay là địa điểm mới và thời gian đến của chuyến bay là thời gian mới.
- **Trạng thái mục tiêu:** Thành phố đích. Đôi khi mục tiêu có thể phức tạp hơn, chẳng hạn như "đến đích trên một chuyến bay thẳng".
- **Chi phí hành động:** Kết hợp chi phí tiền tệ, thời gian chờ đợi, thời gian bay, thủ tục hải quan và nhập cảnh, chất lượng chỗ ngồi, thời gian trong ngày, loại máy bay, điểm tái hành khách thường xuyên, v.v.

Các hệ thống tự vấn du lịch thương mại sử dụng một công thức vấn đề kiểu này, với nhiều phức tạp bổ sung để xử lý cấu trúc giá vé Byzantine của các hãng hàng không. Tuy nhiên, bất kỳ du khách dày dạn kinh nghiệm nào cũng biết rằng không phải tất cả các chuyến bay đều diễn ra theo kế hoạch. Một hệ thống thực sự tốt nên bao gồm các kế hoạch dự phòng - điều gì sẽ xảy ra nếu chuyến bay này bị trì hoãn và bỏ lỡ kết nối?

Vấn đề lưu diễn Các vấn đề về chuyến lưu diễn mô tả một tập hợp các địa điểm phải được ghé thăm, thay vì một điểm đến mục tiêu. Vấn đề nhân viên bán hàng lưu động (TSP) là một vấn đề du lịch trong đó mọi thành phố trên bản đồ phải được ghé thăm. Mục đích là tìm một tour du lịch với chi phí < C (hoặc trong tối ưu hóa, để tìm một tour du lịch với chi phí thấp nhất có thể). Một lượng lớn nỗ lực đã được sử dụng để cải thiện khả năng của các thuật toán TSP. Các thuật toán cũng có thể được mở rộng để xử lý các đội xe. Ví dụ: một thuật toán tìm kiếm và tối ưu hóa để định tuyến xe buýt trường học ở Boston đã tiết kiệm 5 triệu đô la, cắt giảm giao thông và ô nhiễm không khí, đồng thời tiết kiệm thời gian cho tài xế và học sinh (Bertsimas et al., 2019). Ngoài việc lập kế hoạch cho các chuyến đi, các thuật toán tìm kiếm đã được sử dụng cho các nhiệm vụ như lập kế hoạch chuyển động của máy khoan bằng mạch tự động và máy dự trữ trên các cửa hàng.

Bổ cục VLSI Vấn đề bổ cục VLSI yêu cầu định vị hàng triệu thành phần và kết nối trên Một con chip để giảm thiểu diện tích, giảm thiểu độ trễ mạch, giảm thiểu điện dung đi lạc và tối đa hóa năng suất sản xuất. Vấn đề bổ cục xuất hiện sau giai đoạn thiết kế logic và thường được chia thành hai phần: bổ cục ô và định tuyến kênh. Trong bổ cục ô, các thành phần nguyên thủy của mạch được nhóm thành các ô, mỗi ô thực hiện một số chức năng được công nhận. Mỗi ô có một đầu chân cố định (kích thước và hình dạng) và yêu cầu một số kết nối nhất định với mỗi ô khác. Mục đích là đặt các ô trên chip sao cho chúng không chồng lên nhau và để có chỗ cho các dây kết nối được đặt giữa các ô. Channelrouting tìm một tuyến đường cụ thể cho mỗi dây qua các khoảng trống giữa các ô. Những vấn đề tìm kiếm này cực kỳ phức tạp, nhưng chắc chắn đáng để giải

Điều hướng robot QUYỀ Điều hướng robot là một khái quát hóa của vấn đề tìm tuyến đường được mô tả trước đó. Thay vì đi theo những con đường riêng biệt (chẳng hạn như những con đường ở Romania), một robot có thể đi lang thang xung quanh, trên thực tế là tạo ra những con đường của riêng nó. Đối với một robot tròn di chuyển trên một bề mặt phẳng, không gian về cơ bản là hai chiều. Khi robot có tay và chân cũng phải được điều khiển, không gian tìm kiếm trở nên nhiều chiều - một chiều cho mỗi góc khớp. Các kỹ thuật nâng cao được yêu cầu chỉ để làm cho không gian tìm kiếm liên tục về cơ bản hữu hạn (xem Chương 26). Ngoài sự phức tạp của vấn đề, robot thực sự cũng phải đối phó với các lỗi trong việc đọc cảm biến và điều khiển động cơ của chúng, với khả năng quan sát một phần và với các tác nhân khác có thể làm thay đổi môi trường.

Trình tự lắp ráp tự động Tự động lắp ráp các vật thể phức tạp (chẳng hạn như động cơ điện) bằng rô-bốt đã là thông lệ tiêu chuẩn của ngành kể từ những năm 1970. Trước tiên, các thuật toán tìm ra một trình tự lắp ráp khả thi và sau đó làm việc để tối ưu hóa quy trình. Giảm thiểu lượng lao động thủ công trên dây chuyền lắp ráp có thể tiết kiệm đáng kể về thời gian và chi phí. Trong các vấn đề lắp ráp, mục đích là tìm ra một thứ tự để lắp ráp các bộ phận của một số vật thể. Nếu chọn sai thứ tự, sẽ không có cách nào để thêm một số phần sau trong trình tự mà không hoàn tác một số công việc đã thực hiện. Kiểm tra một hành động trong trình tự về tính khả thi là một vấn đề tìm kiếm hình học khó khăn liên quan chặt chẽ đến điều hướng robot. Do đó, việc tạo ra các hành động pháp lý là phần tốn kém của trình tự lắp ráp. Bất kỳ thuật toán thực tế nào cũng phải tránh khám phá tất cả từ một phần nhỏ của không gian trạng thái. Một vấn đề lắp ráp quan trọng là thiết kế protein, trong đó mục tiêu là tìm ra một chuỗi axit amin sẽ gấp lại thành một protein ba chiều với các đặc tính phù hợp để chữa một số bệnh.

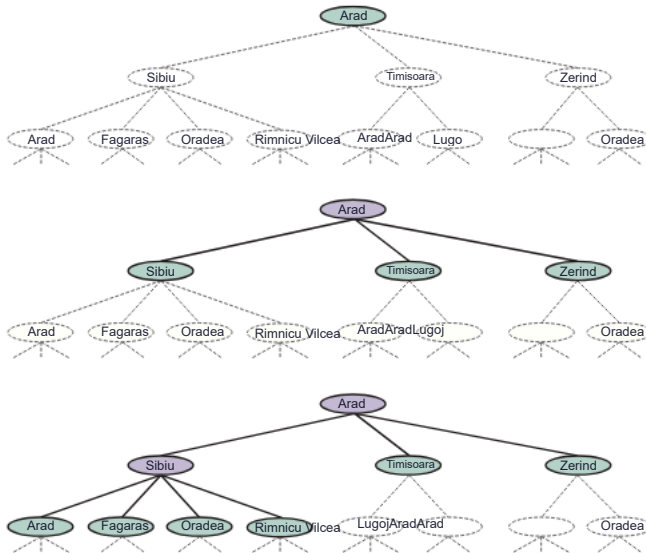
Thiết kế protein

3.3 Thuật toán tìm kiếm

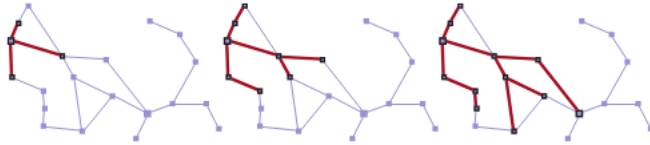
Một thuật toán tìm kiếm lấy một bài toán tìm kiếm làm đầu vào và trả về một giải pháp hoặc một chỉ báo của Thất bại. Trong chương này, chúng ta xem xét các thuật toán chống lên một cây tìm kiếm trên biểu đồ không gian trạng thái, tạo thành các đường dẫn khác nhau từ trạng thái ban đầu, cố gắng tìm một đường dẫn đạt đến trạng thái mục tiêu. Mỗi nút trong cây tìm kiếm tương ứng với một trạng thái trong không gian trạng thái và các cạnh trong cây tìm kiếm tương ứng với các hành động. Gốc của cây tương ứng với trạng thái ban đầu của vấn đề.

Điều quan trọng là phải hiểu sự khác biệt giữa không gian trạng thái và cây tìm kiếm. Không gian trạng thái mô tả tập hợp các trạng thái (có thể là vô hạn) trong thế giới và các hành động cho phép chuyển đổi từ trạng thái này sang trạng thái khác. Cây tìm kiếm mô tả các đường dẫn giữa các trạng thái này, hướng tới mục tiêu. Cây tìm kiếm có thể có nhiều đường dẫn đến (và do đó có nhiều nút cho) bất kỳ trạng thái nhất định nào, nhưng mỗi nút trong cây có một đường dẫn duy nhất trở lại gốc (như trong tất cả các cây).

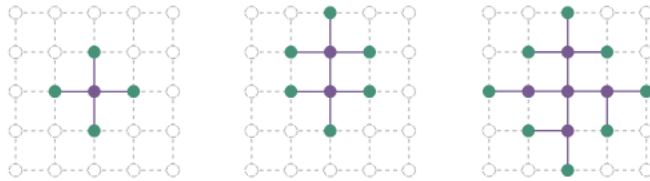
Hình 3.4 cho thấy một vài bước đầu tiên trong việc tìm một con đường từ Arad đến Bucharest. Nút gốc của cây tìm kiếm ở trạng thái ban đầu, Arad. Chúng ta có thể mở rộng nút, bằng cách xem xét



Hình 3.4 Ba cây tìm kiếm một phần để tìm tuyến đường từ Arad đến Bucharest. Các nút đã được mở rộng là hoa oải hương với các chữ in đậm; các nút trên biên giới đã được tạo ra nhưng chưa mở rộng có màu xanh lá cây; Tập hợp các trạng thái tương ứng với hai loại nút này được cho là đã đạt được. Các nút có thể được tạo tiếp theo được hiển thị bằng các đường đứt nét mờ. Lưu ý ở cây dưới cùng có một chu kỳ từ Arad đến Sibiu đến Arad; đó không thể là một con đường tối ưu, vì vậy việc tìm kiếm không nên tiếp tục từ đó.



Hình 3.5 Một chuỗi các cây tìm kiếm được tạo ra bởi một tìm kiếm đồ thị trên bài toán Romania của Hình 3.1. Ở mỗi giai đoạn, chúng ta đã mở rộng mọi nút trên biên giới, mở rộng mọi con đường với tất cả các hành động áp dụng không dẫn đến trạng thái đã đạt được. Lưu ý rằng ở giai đoạn thứ ba, thành phố trên cùng (Oradea) có hai người kế vị, cả hai đều đã được tiếp cận bằng các con đường khác, vì vậy không có con đường nào được kéo dài từ Oradea.



(một) (b) (c)

Hình 3.6 Thuộc tính phân tách của tìm kiếm đồ thị, được minh họa trên một problem-lưới hình chữ nhật. Biên giới (màu xanh lá cây) ngăn cách bên trong (hoa oải hương) với bên ngoài (đứt nét mờ). Biên giới là tập hợp các nút (và các trạng thái tương ứng) đã đạt được nhưng chưa được mở rộng; bên trong là tập hợp các nút (và các trạng thái tương ứng) đã được mở rộng; và bên ngoài là tập hợp các trạng thái chưa đạt được. Trong (a), chỉ có gốc đã được mở rộng. Trong (b), nút biên giới trên cùng được mở rộng. Trong (c), các kế thừa còn lại của gốc được mở rộng theo thứ tự theo chiều kim đồng hồ.

Tạo ra

Nút con
Nút kế thừa

Nút cha

Biên giới

Đạt

Tách

các ACTIONS có sẵn cho trạng thái đó, sử dụng hàm RESULT để xem các hành động đó dẫn đến đâu và tạo một nút mới (được gọi là nút con hoặc nút kế nhiệm) cho mỗi nút kết quả trạng thái. Mỗi nút con có Arad làm nút mẹ của nó.

Bây giờ chúng ta phải chọn nút nào trong ba nút con này để xem xét tiếp theo. Đây là bản chất của tìm kiếm - theo dõi một lựa chọn ngay bây giờ và đặt những lựa chọn khác sang một bên sau này. Giả sử chúng ta chọn mở rộng Sibiu trước. Hình 3.4 (dưới cùng) cho thấy kết quả: một tập hợp 6 nút không được mở rộng (được in đậm). Chúng tôi gọi đây là biên giới của cây tìm kiếm. Chúng tôi nói rằng bất kỳ

trạng thái đã có một nút được tạo cho nó đã đạt được (cho dù nút đó có được mở rộng).5 Hình 3.5 cho thấy cây tìm kiếm chống lên biểu đồ không gian trạng thái. Lưu ý rằng biên giới ngăn cách hai vùng của biểu đồ không gian trạng thái: một vùng bên trong nơi mọi tiểu bang đã được mở rộng, và một khu vực bên ngoài của các tiểu bang chưa được tiếp cận. Thuộc tính này được minh họa trong Hình 3.6.

5 Một số tác giả gọi biên giới là danh sách mở, vừa ít gọi lên về mặt địa lý vừa không phù hợp về mặt tính toán, bởi vì một hàng đợi hiệu quả hơn một danh sách ở đây. Những tác giả đó sử dụng thuật ngữ danh sách đóng để tham khảo tập hợp các nút được mở rộng trước đó, mà theo thuật ngữ của chúng tôi sẽ là các nút đã đạt được trừ đi biên giới.

function BEST-FIRST-SEARCH(problem, f) trả về một nút giải pháp hoặc lỗi
 nút $\leftarrow$ NODE(STATE=PROBLEM. INITIAL)frontier $\leftarrow$ một hàng đợi ưu tiên
 được sắp xếp theo f, với node là một phần tử được truy cập $\leftarrow$ một bảng
 tra cứu, với một mục nhập có vấn đề khóa. INITIAL và value node trong khi
 không phải IS-EMPTY(biên giới) làm

nút $\leftarrow$ POP (biên giới) nếu có vấn đề. IS-GOAL(nút. STATE)
 sau đó trả về node cho mỗi con trong EXPAND(vấn đề, nút) do

đưa con $\leftarrow$. STATE if s không được tiếp cận hoặc trẻ em. PATH-COST < đạt được. PATH-COST sau
 đó

đạt được [các] $\leftarrow$ đưa
 trẻ thêm đưa trẻ vào biên
 giới trở lại thất bại

chức năng EXPAND(vấn đề, nút) tạo ra các nút
 nút $\leftarrow$. STATE cho mỗi hành động trong vấn đề. (Các) HÀNH ĐỘNG

s' $\leftarrow$ vấn đề. RESULT(s, action) cost $\leftarrow$ nút. PATH-COST + vấn đề.
 ACTION-COST (s, action, s') yield NODE (STATE = s', PARENT =
 node, ACTION = action, PATH-COST = cost)

Hình 3.7 Thuật toán tìm kiếm tốt nhất đầu tiên và chức năng mở rộng nút. Các cấu trúc dữ liệu được sử dụng ở đây được mô tả trong Phần 3.3.2. Xem Phụ lục B để biết năng suất.

3.3.1 Tìm kiếm tốt nhất đầu tiên

Làm thế nào để chúng ta quyết định nút nào từ biên giới để mở rộng tiếp theo? Một cách tiếp cận rất chung được gọi là tìm kiếm tốt nhất, trong đó chúng ta chọn một nút, n, với giá trị tối thiểu là một số

Tìm kiếm tốt nhất đầu tiên

chức năng đánh giá, f (n). Hình 3.7 cho thấy thuật toán. Trên mỗi lần lặp lại, chúng tôi chọn một nút trên biên giới có giá trị f (n) tối thiểu, trả về nó nếu trạng thái của nó là trạng thái mục tiêu, và nếu không áp dụng EXPAND để tạo các nút con. Mỗi nút con được thêm vào biên giới nếu nó chưa được tiếp cận trước đây hoặc được thêm lại nếu nó hiện đang được tiếp cận với một đường dẫn có chi phí đường dẫn thấp hơn bất kỳ đường dẫn nào trước đó. Thuật toán trả về một dấu hiệu thất bại hoặc một nút đại diện cho một đường dẫn đến một mục tiêu. Bằng cách sử dụng các hàm f (n) khác nhau, chúng ta có các thuật toán cụ thể khác nhau, mà chương này sẽ đề cập.

Chức năng đánh giá

3.3.2 Tìm kiếm cấu trúc dữ liệu

Các thuật toán tìm kiếm yêu cầu cấu trúc dữ liệu để theo dõi cây tìm kiếm. Một nút trong cây được thể hiện bằng một cấu trúc dữ liệu với bốn thành phần:

- nút. STATE: trạng thái mà nút tương ứng;
- nút. PARENT: nút trong cây đã tạo nút này;
- nút. ACTION: hành động đã được áp dụng cho trạng thái của cha mẹ để tạo nút này;
- nút. PATH-COST: tổng chi phí của đường dẫn từ trạng thái ban đầu đến nút này. Trong các công thức toán học, chúng ta sử dụng g(node) như một từ đồng nghĩa với PATH-COST. Theo dõi các con trỏ PARENT trở lại từ một nút cho phép chúng ta khôi phục các trạng thái và hành động dọc theo đường dẫn đến nút đó. Làm điều này từ một nút mục tiêu cho chúng ta giải pháp.

Hàng	Chúng ta cần một cấu trúc dữ liệu để lưu trữ biên giới. Sự lựa chọn thích hợp là một hàng đợi của một số loại, bởi vì các hoạt động trên biên giới là: • IS-EMPTY(frontier) chỉ trả về true nếu không có nút trong biên giới. • POP (biên giới) loại bỏ nút trên cùng khỏi biên giới và trả về nó. • TOP(biên giới) trả về (nhưng không xóa) nút trên cùng của biên giới. • ADD (node, frontier) chèn nút vào vị trí thích hợp của nó trong hàng đợi. Ba loại hàng đợi được sử dụng trong các thuật toán tìm kiếm:
Hàng đợi ưu tiên	• Một hàng đợi ưu tiên trước tiên bật nút với chi phí tối thiểu theo một số đánh giá hàm ation, f . Nó được sử dụng trong tìm kiếm tốt nhất đầu tiên.
Hàng đợi FIFO	• Hàng đợi FIFO hoặc hàng đợi nhập trước xuất trước sẽ bật nút đã được thêm vào hàng đợi đầu tiên; Chúng ta sẽ thấy nó được sử dụng trong tìm kiếm chiều rộng đầu tiên.
Hàng đợi LIFO	• Hàng đợi LIFO hoặc hàng đợi vào trước xuất trước (còn được gọi là ngăn xếp) xuất hiện đầu tiên nhiều nhất
Ngăn xếp	nút được thêm gần đây; chúng ta sẽ thấy nó được sử dụng trong tìm kiếm đầu tiên. Các trạng thái đạt được có thể được lưu trữ dưới dạng bảng tra cứu (ví dụ: bảng băm) trong đó mỗi khóa là một trạng thái và mỗi giá trị là nút cho trạng thái đó.

3.3.3 Đường dẫn dự phòng

Trạng thái lặp lại	Cây tìm kiếm được hiển thị trong Hình 3.4 (dưới cùng) bao gồm một đường dẫn từ Arad đến Sibiu và quay lại Arad một lần nữa. Chúng tôi nói rằng Arad là một trạng thái lặp lại trong cây tìm kiếm, được tạo ra trong trường hợp này bởi
Đường dẫn CycleLoopy	một chu kỳ (còn được gọi là một con đường vòng lặp). Vì vậy, mặc dù không gian trạng thái chỉ có 20 trạng thái, nhưng
Đường dẫn dư thừa	Cây tìm kiếm hoàn chỉnh là vô hạn vì không có giới hạn về tần suất một người có thể đi qua một vòng lặp. Chu kỳ là một trường hợp đặc biệt của một đường dẫn dư thừa. Ví dụ: chúng ta có thể đến Sibiu thông qua con đường Arad–Sibiu (dài 140 dặm) hoặc con đường Arad–Zerind–Oradea–Sibiu (dài 297 dặm). Con đường thứ hai này là dư thừa - nó chỉ là một cách tồi tệ hơn để đạt được trạng thái tương tự - và không cần phải xem xét trong việc tìm kiếm những con đường tối ưu của chúng ta.

Hãy xem xét một tác nhân trong thế giới lưới 10 × 10, với khả năng di chuyển đến bất kỳ hình vuông nào trong số 8 ô vuông liền kề. Nếu không có chướng ngại vật, đặc vụ có thể đạt được bất kỳ ô vuông nào trong số 100 ô vuông trong 9 nước đi hoặc ít hơn. Nhưng số lượng đường có chiều dài 9 là gần 89 (ít hơn một chút vì các cạnh của lưới), hoặc hơn 100 triệu. Nói cách khác, ô trung bình có thể được tiếp cận bởi hơn một triệu đường dẫn dự phòng có độ dài 9, và nếu chúng ta loại bỏ các đường dẫn dư thừa, chúng ta có thể hoàn thành tìm kiếm nhanh hơn khoảng một triệu lần. Như người ta nói, các thuật toán không thể nhớ lại quá khứ sẽ lặp lại nó. Có ba cách tiếp cận cho vấn đề này.

Đầu tiên, chúng ta có thể nhớ tất cả các trạng thái đã đạt được trước đó (như tìm kiếm tốt nhất đầu tiên), cho phép chúng ta phát hiện tất cả các đường dẫn dư thừa và chỉ giữ đường dẫn tốt nhất đến mỗi trạng thái. Điều này phù hợp với không gian trạng thái nơi có nhiều đường dẫn dư thừa và là lựa chọn ưu tiên khi các trạng thái đạt được sẽ phù hợp với bộ nhớ.

Thứ hai, chúng ta không thể lo lắng về việc lặp lại quá khứ. Có một số công thức vấn đề trong đó hiếm hoặc không thể có hai con đường đạt đến cùng một trạng thái. Một ví dụ sẽ là bài toán lắp ráp trong đó mỗi hành động thêm một bộ phận vào một tập hợp đang phát triển và có thứ tự các bộ phận để có thể thêm A và sau đó là B, nhưng không phải B và sau đó là A. Đối với những vấn đề đó, chúng ta có thể tiết kiệm dung lượng bộ nhớ nếu chúng ta không theo dõi các trạng thái đã đạt được và chúng ta không kiểm tra các đường dẫn dư thừa. Chúng ta gọi thuật toán tìm kiếm là tìm kiếm đồ thị nếu nó kiểm tra dư thừa

Tìm kiếm biểu đồ	đường dẫn và tìm kiếm giống cây6 nếu nó không kiểm tra. Thuật toán TÌM KIẾM TỐT NHẤT ĐẦU TIÊN trong
Tìm kiếm giống cây	

Hình 3.7 là một thuật toán tìm kiếm đồ thị; Nếu chúng ta loại bỏ tất cả các tham chiếu đến Reached, chúng ta sẽ nhận được một tìm kiếm giống như cây sử dụng ít bộ nhớ hơn nhưng sẽ kiểm tra các đường dẫn dư thừa đến cùng một trạng thái, và do đó sẽ chạy chậm hơn.

Thứ ba, chúng ta có thể thỏa hiệp và kiểm tra các chu kỳ, nhưng không phải đối với các đường dẫn dư thừa nói chung. Vì mỗi nút có một chuỗi con trở mẹ, chúng ta có thể kiểm tra các chu kỳ mà không cần bộ nhớ bổ sung bằng cách theo dõi chuỗi cha mẹ để xem liệu trạng thái ở cuối đường dẫn có xuất hiện sớm hơn trong đường dẫn hay không. Một số triển khai tuân theo chuỗi này và do đó loại bỏ tất cả các chu kỳ; Các triển khai khác chỉ đi theo một vài liên kết (ví dụ: đến cha mẹ, ông bà và ông bà), và do đó chỉ mất một khoảng thời gian không đổi, đồng thời loại bỏ tất cả các chu kỳ ngắn (và dựa vào các cơ chế khác để đối phó với các chu kỳ dài).

3.3.4 Đo lường hiệu suất giải quyết vấn đề

Trước khi đi vào thiết kế các thuật toán tìm kiếm khác nhau, chúng tôi sẽ xem xét các tiêu chí được sử dụng để chọn trong số chúng. Chúng ta có thể đánh giá hiệu suất của thuật toán theo bốn cách:

- Tính đầy đủ: Thuật toán có đảm bảo tìm ra giải pháp khi có giải pháp không, và để Đầy đủ
báo cáo lỗi dừng khi không có?
- Tối ưu chi phí: Nó có tìm ra giải pháp có chi phí đường dẫn thấp nhất trong tất cả các giải pháp không?? Tối ưu chi phí
- Độ phức tạp về thời gian: Mất bao lâu để tìm ra giải pháp? Điều này có thể được đo bằng Độ phức tạp về thời gian
giây, hoặc trừu tượng hơn theo số lượng trạng thái và hành động được xem xét.
- Độ phức tạp của không gian: Cần bao nhiêu bộ nhớ để thực hiện tìm kiếm? Độ phức tạp của không gian

Để hiểu tính hoàn chỉnh, hãy xem xét một vấn đề tìm kiếm với một mục tiêu duy nhất. Mục tiêu đó có thể ở bất kỳ đâu trong không gian nhà nước; do đó, một thuật toán hoàn chỉnh phải có khả năng khám phá một cách có hệ thống mọi trạng thái có thể đạt được từ trạng thái ban đầu. Trong không gian trạng thái hữu hạn, điều đó dễ đạt được: miễn là chúng ta theo dõi các đường đi và cắt đứt những đường đi là chu kỳ (ví dụ: Arad đến Sibiu đến Arad), cuối cùng chúng ta sẽ đạt được mọi trạng thái có thể đạt được.

Trong không gian trạng thái vô hạn, cần phải cẩn thận hơn. Ví dụ, một thuật toán lặp đi lặp lại áp dụng toán tử "giai thừa" trong bài toán "4" của Knuth sẽ đi theo một đường dẫn vô hạn từ 4 đến 4! đến (4!)!, v.v. Tương tự, trên một lưới vô hạn không có chướng ngại vật, liên tục di chuyển về phía trước theo một đường thẳng cũng đi theo một đường vô hạn của các trạng thái mới. Trong cả hai trường hợp, thuật toán không bao giờ trở lại trạng thái mà nó đã đạt được trước đó, nhưng không hoàn chỉnh vì không gian trạng thái rộng lớn không bao giờ đạt được.

Để hoàn thành, một thuật toán tìm kiếm phải có hệ thống trong cách nó khám phá một số vô hạn Hệ thống
không gian trạng thái, đảm bảo cuối cùng nó có thể đạt đến bất kỳ trạng thái nào được kết nối với trạng thái ban đầu. Ví dụ, trên lưới vô hạn, một loại tìm kiếm có hệ thống là một đường xoắn ốc bao gồm tất cả các ô cách gốc s bước trước khi di chuyển ra các ô cách s + 1 bước. Thật không may, trong một không gian trạng thái vô hạn không có giải pháp, một thuật toán âm thanh cần phải tiếp tục tìm kiếm mãi mãi; nó không thể chấm dứt vì nó không thể biết liệu trạng thái tiếp theo có phải là mục tiêu hay không.

Độ phức tạp về thời gian và không gian được xem xét đối với một số thước đo của vấn đề. Trong khoa học máy tính lý thuyết, thước đo điển hình là kích thước của đồ thị không gian trạng thái, $|V| + |E|$, trong đó $|V|$ là số đỉnh (nút trạng thái) của đồ thị và $|E|$ Là

6 Chúng ta nói "tìm kiếm giống cây" bởi vì không gian trạng thái vẫn là cùng một biểu đồ cho dù chúng ta tìm kiếm nó như thế nào; chúng ta chỉ chọn coi nó như thể nó là một cái cây, chỉ có một đường dẫn từ mỗi nút trở lại gốc. 7 Một số tác giả sử dụng thuật ngữ "khả năng chấp nhận" cho thuộc tính tìm ra giải pháp chi phí thấp nhất, và một số chỉ sử dụng "tính tối ưu", nhưng điều đó có thể bị nhầm lẫn với các loại tối ưu khác.

Độ sâu

Hệ số phân nhánh

số lượng cạnh (các cặp trạng thái/hành động riêng biệt). Điều này thích hợp khi biểu đồ là cấu trúc dữ liệu rõ ràng, chẳng hạn như bản đồ Romania. Nhưng trong nhiều vấn đề về AI, biểu đồ chỉ được thể hiện ngầm bởi trạng thái ban đầu, hành động và mô hình chuyển tiếp. Đối với một không gian ngầm, độ phức tạp có thể được đo bằng d , độ sâu hoặc số lượng hành động trong một giải pháp tối ưu; m , số lượng hành động tối đa trong bất kỳ đường dẫn nào; và b , yếu tố phân nhánh hoặc số lượng người kế thừa của một nút cần được xem xét.

3.4 Chiến lược tìm kiếm thiếu thông tin

Một thuật toán tìm kiếm thiếu thông tin không được cung cấp manh mối về mức độ gần của một trạng thái với (các) mục tiêu. Ví dụ, hãy xem xét đại lý của chúng tôi ở Arad với mục tiêu đến Bucharest. Một người không có thông tin không biết về địa lý Romania không biết liệu đến Zerind hay Sibiu có phải là bước đầu tiên tốt hơn hay không. Ngược lại, một đặc vụ có hiểu biết (Phần 3.5) biết vị trí của mỗi thành phố biết rằng Sibiu gần Bucharest hơn nhiều và do đó có nhiều khả năng đi trên con đường ngắn nhất.

3.4.1 Tìm kiếm chiều rộng đầu tiên

Tìm kiếm theo chiều rộng đầu tiên

Khi tất cả các hành động có cùng chi phí, một chiến lược thích hợp là tìm kiếm trên toàn cầu, trong đó nút gốc được mở rộng trước, sau đó tất cả các nút kế thừa của nút gốc được mở rộng tiếp theo, sau đó là các nút kế thừa của chúng, v.v. Đây là một chiến lược tìm kiếm có hệ thống do đó hoàn chỉnh ngay cả trên các không gian trạng thái vô hạn. Chúng ta có thể thực hiện tìm kiếm chiều rộng đầu tiên như một lệnh gọi đến BEST-FIRST-SEARCH trong đó hàm đánh giá $f(n)$ là độ sâu của nút — nghĩa là số lượng hành động cần thiết để tiếp cận nút.

Tuy nhiên, chúng ta có thể đạt được hiệu quả bổ sung với một vài thủ thuật. Một hàng đợi đầu tiên trong đầu tiên sẽ nhanh hơn hàng đợi ưu tiên và sẽ cung cấp cho chúng ta thứ tự chính xác của các nút: các nút mới (luôn sâu hơn cha mẹ của chúng) đi đến phía sau của hàng đợi và các nút cũ, nông hơn các nút mới, được mở rộng trước. Ngoài ra, đạt được có thể là một tập hợp các trạng thái chứ không phải là ánh xạ từ trạng thái sang nút, bởi vì một khi chúng ta đã đạt đến một trạng thái, chúng ta không bao giờ có thể tìm thấy một con đường tốt hơn đến trạng thái. Điều đó cũng có

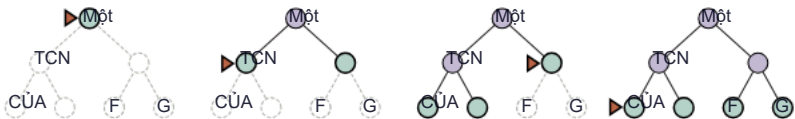
Kiểm tra mục tiêu sớm

Kiểm tra mục tiêu muộn

nghĩa là chúng ta có thể thực hiện một bài kiểm tra mục tiêu sớm,

kiểm tra xem một nút có phải là một giải pháp ngay khi nó được tạo hay không, thay vì kiểm tra mục tiêu muộn tìm kiếm đầu tiên tốt nhất đó sử dụng, đợi cho đến khi một nút được bật ra khỏi hàng đợi. Hình 3.8 cho thấy tiến trình tìm kiếm theo chiều rộng đầu tiên trên cây nhị phân và Hình 3.9 cho thấy thuật toán với các cải tiến hiệu quả mục tiêu ban đầu.

Tìm kiếm chiều rộng đầu tiên luôn tìm thấy một giải pháp với số lượng hành động tối thiểu, bởi vì khi nó tạo ra các nút ở độ sâu d , nó đã tạo ra tất cả các nút ở độ sâu $d - 1$, vì vậy nếu một trong số chúng là một giải pháp, nó sẽ được tìm thấy. Điều đó có nghĩa là nó là tối ưu về chi phí



Hình 3.8 Tìm kiếm theo chiều rộng đầu tiên trên một cây nhị phân đơn giản. Ở mỗi giai đoạn, nút được ex-panded tiếp theo được biểu thị bằng điểm đánh dấu hình tam giác.

```

function BREADTH-FIRST-SEARCH(problem) trả về một nút giải pháp hoặc lỗi
    nút ← NODE(vấn đề, INITIAL) nếu có vấn đề.
    IS-GOAL(nút, STATE) sau đó trả về
    nodefrontier ← hàng đợi FIFO, với node là một
    phần tử đạt được ← {vấn đề, BAN ĐẦU}
    trong khi không phải IS-EMPTY (biên giới) làm
        nút ← POP (biên giới) cho mỗi con trong EXPAND (vấn đề, nút) làm
            đưa con ←, STATE nếu vấn đề, IS-GOAL(s)
            sau đó trả lại đưa trên nếu s không đạt được thì
                thêm s vào đạt được thêm con
                vào biên giới trả lại thất bại

```

```

function UNIFORM-COST-SEARCH(problem) trả về một nút giải pháp hoặc lỗi
    trả về BEST-FIRST-SEARCH(vấn đề, PATH-COST)

```

Hình 3.9 Các thuật toán tìm kiếm theo chiều rộng đầu tiên và tìm kiếm chi phí đồng nhất.

Đối với các vấn đề mà tất cả các hành động đều có cùng chi phí, nhưng không phải đối với các vấn đề không có thuộc tính đó. Nó là hoàn chỉnh trong cả hai trường hợp. Về mặt thời gian và không gian, hãy tưởng tượng tìm kiếm một cây đồng nhất nơi mọi tiểu bang đều có những người kế nhiệm. Gốc của cây tìm kiếm tạo ra các bnode, mỗi node tạo ra b node hơn, với tổng số b² ở cấp độ thứ hai. Mỗi trong số này tạo ra b nút hơn, tạo ra b³ nút ở cấp độ thứ ba, v.v. Bây giờ giả sử rằng lời giải ở độ sâu d. Sau đó, tổng số nút được tạo ra là

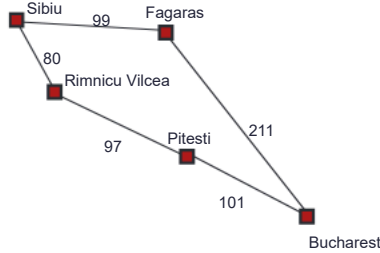
$1 + b + b^2 + b^3 + \dots + b^d = O(b^d)$ Tất cả các nút vẫn còn trong bộ nhớ, vì vậy cả độ phức tạp về thời gian và không gian đều là $O(b^d)$. Những giới hạn theo cấp số nhân như vậy thật đáng sợ. Như một ví dụ điển hình trong thế giới thực, hãy xem xét một vấn đề với hệ số phân nhánh $b = 10$, tốc độ xử lý 1 triệu nút / giây và yêu cầu bộ nhớ là 1Kbyte / nút. Một tìm kiếm đến độ sâu $d = 10$ sẽ mất chưa đầy 3 giờ, nhưng sẽ yêu cầu 10terabyte bộ nhớ. Yêu cầu bộ nhớ là một vấn đề lớn hơn đối với tìm kiếm chiều rộng đầu tiên. Thời gian thực hiện. Nhưng thời gian vẫn là một yếu tố quan trọng. Ở độ sâu $d = 14$, ngay cả với bộ nhớ vô hạn, việc tìm kiếm sẽ mất 3,5 năm. Nói chung, các vấn đề J tìm kiếm phức tạp theo cấp số nhân không thể được giải quyết bằng cách tìm kiếm không có thông tin cho bất kỳ trường hợp nào ngoại trừ các trường hợp nhỏ nhất.

3.4.2 Thuật toán của Dijkstra hoặc tìm kiếm chi phí đồng nhất

Khi các hành động có chi phí khác nhau, một lựa chọn rõ ràng là sử dụng tìm kiếm tốt nhất đầu tiên trong đó hàm đánh giá là chi phí của đường dẫn từ gốc đến nút hiện tại. Đây được gọi là thuật toán Dijkstra bởi cộng đồng khoa học máy tính lý thuyết và tìm kiếm chi phí đồng nhất

Tìm kiếm chi phí đồng
nhất

bởi cộng đồng AI. Ý tưởng là trong khi tìm kiếm chiều rộng đầu tiên trải rộng trong các đợt độ sâu đơn dạng - độ sâu đầu tiên là 1, sau đó là độ sâu 2, v.v. - tìm kiếm chi phí đồng nhất trải rộng trong các đợt chi phí đường đi đồng nhất. Thuật toán có thể được triển khai như một cuộc gọi đến BEST-FIRST-SEARCH với PATH-COST làm hàm đánh giá, như thể hiện trong Hình 3.9.



Hình 3.10 Một phần của không gian nhà nước Romania, được chọn để minh họa tìm kiếm chi phí đồng đều.

Hãy xem Hình 3.10, trong đó vấn đề là đi từ Sibiu đến Bucharest. Thành công của Sibiu là Rimnicu Vilcea và Fagaras, với giá lần lượt là 80 và 99. Nút có chi phí thấp nhất, Rimnicu Vilcea, được mở rộng tiếp theo, thêm Pitesti với chi phí $80 + 97 = 177$. Nút chi phí thấp nhất bây giờ là Fagaras, vì vậy nó được mở rộng, thêm Bucharest với chi phí $99 + 211 = 310$. Bucharest là mục tiêu, nhưng thuật toán chỉ kiểm tra mục tiêu khi nó mở rộng một nút, không phải khi nó tạo ra một nút, vì vậy nó vẫn chưa phát hiện ra rằng đây là một đường dẫn đến mục tiêu.

Thuật toán tiếp tục, chọn Pitesti để mở rộng tiếp theo và thêm con đường thứ hai đến Bucharest với chi phí $80 + 97 + 101 = 278$. Nó có chi phí thấp hơn, vì vậy nó thay thế con đường trước đó đã đạt được và được thêm vào biên giới. Hóa ra nút này hiện có chi phí thấp nhất, vì vậy nó được coi là tiếp theo, được tìm thấy là mục tiêu và trả lại. Lưu ý rằng nếu chúng ta đã kiểm tra một mục tiêu khi tạo một nút thay vì khi mở rộng nút có chi phí thấp nhất, thì chúng ta sẽ trả về một đường dẫn có chi phí cao hơn (đường dẫn qua Fagaras).

Độ phức tạp của tìm kiếm chi phí đồng nhất được đặc trưng bằng C^* , chi phí của giải pháp tối ưu, b và c , giới hạn thấp hơn về chi phí của mỗi hành động, với $b > 0$. Sau đó độ phức tạp về thời gian và không gian trong trường hợp xấu nhất của thuật toán là $O(b(1+bC^*/c))$, có thể lớn hơn nhiều so với bd . Điều này là do tìm kiếm chi phí đồng nhất có thể khám phá các cây hành động lớn với chi phí thấp trước khi khám phá các đường dẫn liên quan đến một hành động có chi phí cao và có thể hữu ích. Khi tất cả các chi phí hành động bằng nhau, $b + bC^*/c$ chỉ là $bd + 1$ và tìm kiếm chi phí đồng nhất tương tự như tìm kiếm chiều rộng đầu tiên.

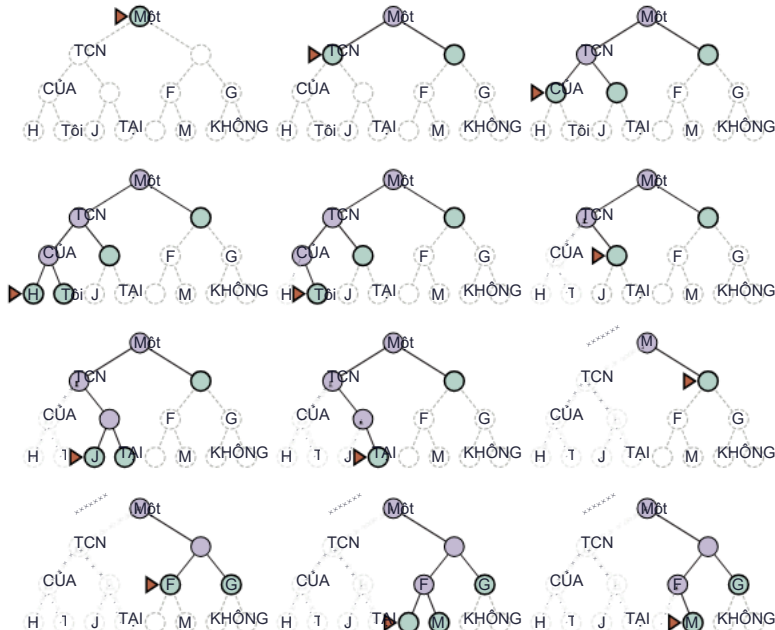
Tìm kiếm chi phí đồng nhất đã hoàn thành và tối ưu về chi phí, bởi vì giải pháp đầu tiên mà nó tìm thấy sẽ có chi phí ít nhất là thấp bằng chi phí của bất kỳ nút nào khác trong biên giới. Tìm kiếm chi phí đồng nhất xem xét tất cả các đường dẫn một cách có hệ thống theo thứ tự tăng chi phí, không bao giờ bị bắt theo một con đường vô hạn duy nhất (giả sử rằng tất cả chi phí hành động đều > 0).

3.4.3 Tìm kiếm độ sâu đầu tiên và vấn đề về bộ nhớ

Tìm kiếm độ sâu đầu tiên luôn mở rộng nút sâu nhất trong biên giới trước. Nó có thể là implemented như một lời kêu gọi đến BEST-FIRST-SEARCH trong đó hàm đánh giá f là âm của độ sâu. Tuy nhiên, nó thường được thực hiện không phải như một tìm kiếm đồ thị mà là một tìm kiếm giống như cây không giữ một bảng các trạng thái đã đạt được. Tiến trình tìm kiếm được minh họa trong Hình 3.11; Tìm kiếm tiến hành ngay lập tức đến cấp độ sâu nhất của cây tìm kiếm, nơi các nút không có người kế nhiệm. Tìm kiếm sau đó "sao lưu" đến nút sâu nhất tiếp theo vẫn còn

Tìm kiếm độ sâu đầu tiên

Ở đây, và trong suốt cuốn sách, "ngôi sao" trong C^* có nghĩa là một giá trị tối ưu cho C .



Hình 3.11 Hàng chục bước (từ trái sang phải, từ trên xuống dưới) trong tiến trình tìm kiếm độ sâu đầu tiên trên cây nhị phân từ trạng thái bắt đầu A đến mục tiêu M. Biên giới có màu xanh lá cây, với một hình tam giác đánh dấu nút sẽ được mở rộng tiếp theo. Các nút mở rộng trước đây là hoa oải hương và các nút tương lai tiềm năng có các đường đứt nét mờ. Các nút mở rộng không có hậu duệ trong biên giới (các đường rất mờ) có thể bị loại bỏ.

những người kế nhiệm chưa mở rộng. Tìm kiếm ưu tiên độ sâu không phải là tối ưu về chi phí; Nó trả về giải pháp đầu tiên mà nó tìm thấy, ngay cả khi nó không rẻ nhất.

Đối với các không gian trạng thái hữu hạn là cây, nó hiệu quả và đầy đủ; Đối với không gian trạng thái không tuần hoàn Nó có thể mở rộng cùng một trạng thái nhiều lần thông qua các đường dẫn khác nhau, nhưng (cuối cùng) sẽ khám phá toàn bộ không gian một cách có hệ thống.

Trong không gian trạng thái tuần hoàn, nó có thể bị mắc kẹt trong một vòng lặp vô hạn; do đó, một số triển khai của tìm kiếm độ sâu đầu tiên kiểm tra từng nút mới cho các chu kỳ. Cuối cùng, trong không gian trạng thái vô hạn, tìm kiếm độ sâu đầu tiên không có hệ thống: nó có thể bị mắc kẹt khi đi xuống một con đường vô hạn, ngay cả khi không có chu kỳ. Do đó, tìm kiếm độ sâu đầu tiên là không đầy

Vì tất cả những tin xấu này, tại sao mọi người lại cân nhắc sử dụng tìm kiếm độ sâu đầu tiên thay vì chiều rộng đầu tiên hoặc tốt nhất đầu tiên? Câu trả lời là đối với các vấn đề mà tìm kiếm giống như cây là khả thi, tìm kiếm độ sâu đầu tiên có nhu cầu bộ nhớ nhỏ hơn nhiều. Chúng ta hoàn toàn không giữ một bảng đạt được, và biên giới rất nhỏ: hãy nghĩ về biên giới trong tìm kiếm chiều rộng đầu tiên như bề mặt của một quả cầu mở rộng không bao giờ, trong khi biên giới trong tìm kiếm độ sâu đầu tiên chỉ là bán kính của quả cầu.

Đối với một không gian trạng thái hình cây hữu hạn như trong Hình 3.11, một tìm kiếm giống như cây có độ sâu đầu tiên mất thời gian tỷ lệ thuận với số lượng trạng thái và có độ phức tạp của bộ nhớ chỉ là $O(bm)$, trong đó b là hệ số phân nhánh và m là độ sâu tối đa của cây. Một số vấn đề yêu cầu exabyte bộ nhớ với tìm kiếm chiều rộng đầu tiên có thể được xử lý chỉ với kilobyte bằng cách sử dụng tìm kiếm độ sâu đầu tiên. Do sử dụng bộ nhớ một cách tiết kiệm, tìm kiếm giống như cây có độ sâu đầu tiên đã được chấp nhận làm công việc cơ bản của nhiều lĩnh vực của AI, bao gồm thỏa mãn ràng buộc (Chương 5), khả năng thỏa mãn mệnh đề (Chương 7) và lập trình logic (Chương 9).

Tìm kiếm quay ngược

Một biến thể của tìm kiếm độ sâu đầu tiên được gọi là tìm kiếm quay ngược thậm chí còn sử dụng ít bộ nhớ hơn. (Xem

Chương 5 để biết thêm chi tiết.) Trong backtrack, chỉ có một người kế nhiệm được tạo ra tại một thời điểm thay vì tất cả những người kế nhiệm; mỗi nút được mở rộng một phần ghi nhớ nút kế thừa nào để generateNext. Ngoài ra, các trình kế thừa được tạo bằng cách sửa đổi trực tiếp mô tả trạng thái hiện tại thay vì phân bổ bộ nhớ cho một trạng thái hoàn toàn mới. Điều này làm giảm yêu cầu bộ nhớ chỉ còn một mô tả trạng thái và một đường dẫn của các hành động $O(m)$; tiết kiệm đáng kể so với trạng thái $O(bm)$ để tìm kiếm độ sâu đầu tiên. Với tính năng quay trở lại, chúng tôi cũng có tùy chọn duy trì cấu trúc dữ liệu tập hợp hiệu quả cho các trạng thái trên đường dẫn hiện tại, cho phép chúng tôi kiểm tra một đường đi theo chu kỳ theo thời gian $O(1)$ thay vì $O(m)$. Để backtrack hoạt động, chúng ta phải có khả năng hoàn tác từng hành động khi chúng ta quay lại. Quay trở lại là rất quan trọng đối với sự thành công của nhiều vấn đề với mô tả trạng thái lớn, chẳng hạn như lắp ráp robot.

3.4.4 Tìm kiếm sâu giới hạn độ sâu và lặp đi lặp lại

Tìm kiếm giới hạn độ sâu

Để giữ cho tìm kiếm độ sâu đầu tiên không đi lang thang trên một con đường vô hạn, chúng ta có thể sử dụng tìm kiếm giới hạn độ sâu, một phiên bản của tìm kiếm độ sâu đầu tiên, trong đó chúng ta cung cấp giới hạn độ sâu, l , và xử lý tất cả các nút ở độ sâu l như thể họ không có người kế thừa (xem Hình 3.12). Độ phức tạp thời gian là $O(b^l)$ và độ phức tạp không gian là $O(b^l)$. Thật không may, nếu chúng ta đưa ra một lựa chọn sai cho l thuật toán sẽ không đạt được giải pháp, khiến nó không hoàn chỉnh một lần nữa.

Vì tìm kiếm độ sâu đầu tiên là một tìm kiếm giống như cây, chúng ta không thể giữ cho nó không lãng phí thời gian trên các đường dẫn lặp lại nói chung, nhưng chúng ta có thể loại bỏ các chu kỳ với chi phí của một số thời gian tính toán. Nếu chúng ta chỉ nhìn vào một vài liên kết trong chuỗi mẹ, chúng ta có thể nắm bắt hầu hết các chu kỳ; chu kỳ dài hơn được xử lý bởi giới hạn độ sâu.

Đường kính

Đôi khi một giới hạn độ sâu tốt có thể được chọn dựa trên kiến thức về vấn đề. Ví dụ, trên bản đồ Romania có 20 thành phố. Do đó, $l = 19$ là một giới hạn hợp lệ. Nhưng nếu chúng ta nghiên cứu bản đồ một cách cẩn thận, chúng ta sẽ phát hiện ra rằng bất kỳ thành phố nào cũng có thể đến được từ bất kỳ thành phố nào khác trong tối đa 9 hành động. Con số này, được gọi là đường kính của đồ thị không gian trạng thái, cho

chúng tôi có giới hạn độ sâu tốt hơn, dẫn đến tìm kiếm giới hạn độ sâu hiệu quả hơn. Tuy nhiên, đối với hầu hết các vấn đề, chúng ta sẽ không biết giới hạn độ sâu tốt cho đến khi chúng ta giải quyết được vấn đề.

Tìm kiếm sâu lặp đi lặp lại

Tìm kiếm sâu sắc lặp đi lặp lại giải quyết vấn đề chọn một giá trị tốt cho l bằng cách cố gắng Tất cả các giá trị: đầu tiên là 0, sau đó là 1, sau đó là 2, v.v. — cho đến khi tìm thấy giải pháp hoặc tìm kiếm giới hạn độ sâu trả về giá trị thất bại thay vì giá trị cắt. Thuật toán được thể hiện trong Hình 3.12. Đào sâu lặp đi lặp lại kết hợp nhiều lợi ích của tìm kiếm độ sâu đầu tiên và độ rộng đầu tiên. Giống như tìm kiếm độ sâu đầu tiên, yêu cầu bộ nhớ của nó là khiêm tốn: $O(bd)$ khi có một lời giải, hoặc $O(bm)$ trên không gian trạng thái hữu hạn không có lời giải. Giống như tìm kiếm chiều rộng đầu tiên, sâu lặp đi lặp lại là tối ưu cho các vấn đề trong đó tất cả các hành động có cùng chi phí và hoàn chỉnh không gian trạng thái vô tuần hữu hạn, hoặc trên bất kỳ không gian trạng thái hữu hạn nào khi chúng ta kiểm tra các nút cho các chu kỳ trên đường đi.

```
function ITERATIVE-DEEPENING-SEARCH(problem) trả về một nút giải pháp hoặc lỗi
  for depth = 0 đến ∞ do
    result ← DEPTH-LIMITED-SEARCH(vấn đề, độ sâu) nếu kết quả ≠ cutoff thì trả về kết quả
```

```
function DEPTH-LIMITED-SEARCH(problem, ' ) trả về một nút hoặc lỗi hoặc cutoff
  frontier ← một hàng đợi LIFO (ngăn xếp) với NODE(vấn đề, INITIAL) như một
  phần tử kết quả ← thất bại trong khi không phải IS-EMPTY (biên giới) là
```

```
  nút ← POP (biên giới) nếu có vấn đề. IS-GOAL(nút.
  STATE) sau đó trả về node if DEPTH(node) > ' thì
```

```
  kết quả ← cutoff else nếu không phải IS-CYCLE(node) do
```

```
    đối với mỗi con trong EXPAND(vấn đề, nút) làm
    Thêm con vào Frontier Trả lời kết quả
```

Hình 3.12 Tìm kiếm giống cây sâu lặp đi lặp lại và giới hạn độ sâu. Sâu lặp đi lặp lại áp dụng tìm kiếm giới hạn độ sâu với giới hạn tăng dần. Nó trả về một trong ba loại giá trị khác nhau: một nút giải pháp; hoặc thất bại, khi nó đã cạn kiệt tất cả các nút và chứng minh không có giải pháp ở bất kỳ độ sâu nào; hoặc cutoff, có nghĩa là có thể có một giải pháp ở độ sâu sâu hơn'. Đây là một thuật toán tìm kiếm giống cây không theo dõi các trạng thái đã đạt được và do đó sử dụng ít bộ nhớ hơn nhiều so với tìm kiếm tốt nhất, nhưng có nguy cơ truy cập cùng một trạng thái nhiều lần trên các đường dẫn khác nhau. Ngoài ra, nếu kiểm tra IS-CYCLE không kiểm tra tất cả các chu kỳ, thì thuật toán có thể bị mắc kẹt trong một vòng lặp.

Độ phức tạp thời gian là $O(bd)$ khi có lời giải, hoặc $O(bm)$ khi không có. Mỗi lần lặp lại của tìm kiếm sâu lặp đi lặp lại tạo ra một cấp độ mới, giống như cách tìm kiếm chiều rộng đầu tiên, nhưng chiều rộng đầu tiên thực hiện điều này bằng cách lưu trữ tất cả các nút trong bộ nhớ, trong khi sâu lặp lại làm điều đó bằng cách lặp lại các cấp độ trước đó, do đó tiết kiệm bộ nhớ với chi phí nhiều thời gian hơn. Hình 3.13 cho thấy bốn lần lặp lại của tìm kiếm sâu lặp đi lặp lại trên cây tìm kiếm nhị phân, trong đó giải pháp được tìm thấy trong lần lặp thứ tư.

Tìm kiếm sâu lặp đi lặp lại có vẻ lãng phí vì các trạng thái gần đầu cây tìm kiếm được tạo lại nhiều lần. Nhưng đối với nhiều không gian trạng thái, hầu hết các nút đều ở cấp dưới cùng, vì vậy việc các cấp trên được lặp lại không quan trọng lắm. Trong tìm kiếm sâu lặp đi lặp lại, các nút ở cấp dưới cùng (độ sâu d) được tạo một lần, các nút ở cấp độ tiếp theo xuống dưới được tạo hai lần, v.v., cho đến các con của gốc, được tạo ra d lần. Vì vậy, tổng số nút được tạo ra trong trường hợp xấu nhất là

$N(IDS) = (d-1)b + (d-2)b^2 + \dots + b^{d-1} + b^d$, cho độ phức tạp thời gian của $O(b^d)$ - tiệm cận giống như tìm kiếm chiều rộng đầu tiên. Ví dụ: nếu $b = 10$ và $d = 5$, các con số là

$N(IDS) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$
 $N(BFS) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,110$. Nếu bạn thực sự quan tâm đến sự lặp lại, bạn có thể sử dụng cách tiếp cận kết hợp chạy

hàm BIBF-SEARCH(problemF, fF, problemB, fB) trả về một nút giải pháp hoặc thất bại

nodeF $\leftarrow$ NODE(vấn đềF, BAN ĐẦU) Nút cho trạng thái bắt đầu

nodeB $\leftarrow$ NODE(PROBLEMB.INITIAL) Nút cho trạng thái mục tiêu

frontierF $\leftarrow$ một hàng đợi ưu tiên được sắp xếp theo fF, với nodeF là một phần tử

frontierB $\leftarrow$ một hàng đợi ưu tiên được sắp xếp theo fB, với nodeB là một phần tử

được sắp xếp bởi fB, với nodeB là một phần tử được sắp xếp F

$\leftarrow$ một bảng tra cứu, với một khóa nodeF.STATE và giá trị nodeF.reachedB

$\leftarrow$ một bảng tra cứu, với một khóa nodeB.STATE và giá trị nodeB.giải pháp

$\leftarrow$ thất bại trong khi không TERMINATED(solution, frontierF, frontierB) do

nếu fF (TOP(frontierF)) < fB(TOP(frontierB)) thì

giải pháp $\leftarrow$ PROCEED (F, vấn đề F, biên giới F, đạt được F, đạt được B, giải pháp) giải pháp khác $\leftarrow$ PROCEED (B, vấn đề B, biên giới B, đạt được B, đạt đến F, giải pháp) giải pháp trả về

hàm PROCEED(dir, problem, frontier, reached, reached2, solution) trả về một giải pháp

Mở rộng nút trên biên giới; kiểm tra biên giới khác trong reached2.// Biến "dir" là hướng: F cho chuyển tiếp hoặc B cho lùi.node $\leftarrow$ POP(frontier) cho mỗi con trong EXPAND(problem, node) do

đứa con $\leftarrow$. STATE nếu không đạt được hoặc PATH-COST (child) < PATH-COST (đã đạt [s]) thì

đạt đến $\leftarrow$ trêthêm

con vào biên giớinếu

s đã đạt được2 thì

solution2 $\leftarrow$ JOIN-NODES(dir, con, reached2[s])nếu

PATH-COST(solution2) < PATH-COST(solution) thì

Giải pháp $\leftarrow$ giải pháp2Giải pháp trả lại

Hình 3.14 Tìm kiếm tốt nhất đầu tiên hai chiều giữ hai biên giới và hai bảng các trạng thái đạt được. Khi một đường dẫn trong một biên giới đạt đến trạng thái cũng đã đạt được trong nửa còn lại của tìm kiếm, hai đường dẫn được nối (bằng hàm JOIN-NODES) để tạo thành một giải pháp. Giải pháp đầu tiên chúng tôi nhận được không đảm bảo là tốt nhất; chức năng TERMINATED xác định khi nào ngừng tìm kiếm giải pháp mới.

Để điều này hoạt động, chúng ta cần theo dõi hai biên giới và hai bảng các trạng thái đã đạt được, và chúng ta cần có khả năng suy luận ngược: nếu trạng thái s là người kế thừa của s theo hướng tiến, thì chúng ta cần biết rằng s là người kế thừa của s theo hướng lùi. Chúng ta có một giải pháp khi hai biên giới va chạm.⁹

Có nhiều phiên bản khác nhau của tìm kiếm hai chiều, cũng như có nhiều thuật toán tìm kiếm một chiều khác nhau. Trong phần này, chúng tôi mô tả tìm kiếm tốt nhất đầu tiên hai chiều. Mặc dù có hai biên giới riêng biệt, nhưng nút được mở rộng tiếp theo luôn là một nút có giá trị tối thiểu của hàm đánh giá, trên một trong hai biên giới. Khi đánh giá

⁹ Trong quá trình triển khai của chúng tôi, cấu trúc dữ liệu đạt được hỗ trợ một truy vấn hỏi liệu một trạng thái nhất định có phải là một thành viên hay không, và cấu trúc dữ liệu biên giới (hàng đợi ưu tiên) thì không, vì vậy chúng tôi kiểm tra xung đột bằng cách sử dụng đã đạt được; Nhưng ngay lập tức, chúng tôi đang hỏi liệu hai biên giới đã gặp nhau chưa. Việc triển khai có thể được mở rộng để xử lý các trạng thái nhiều mục tiêu bằng cách tải nút cho mỗi trạng thái mục tiêu vào biên giới lùi và bảng đạt được ngược.

hàm là chi phí đường dẫn, chúng ta nhận được tìm kiếm chi phí đồng nhất hai chiều và nếu chi phí của đường dẫn tối ưu là C^* , thì không có nút nào có chi phí $> C^*2$ sẽ được mở rộng. Điều này có thể dẫn đến tăng tốc đáng kể.

Thuật toán tìm kiếm hai chiều tốt nhất đầu tiên được thể hiện trong Hình 3.14. Chúng tôi chuyển hai phiên bản của bài toán và hàm đánh giá, một theo hướng tiến (chỉ số dưới F) và một theo hướng lùi (chỉ số dưới B). Khi hàm đánh giá là chi phí đường dẫn, chúng ta biết rằng giải pháp đầu tiên được tìm thấy sẽ là một giải pháp tối ưu, nhưng với các chức năng đánh giá khác nhau không nhất thiết phải đúng. Do đó, chúng tôi theo dõi giải pháp tốt nhất được tìm thấy cho đến nay và có thể phải cập nhật nhiều lần trước khi TERMINATED test chứng minh rằng không còn giải pháp nào tốt hơn.

3.4.6 So sánh các thuật toán tìm kiếm thiếu thông tin

Hình 3.15 so sánh các thuật toán tìm kiếm thiếu thông tin về bốn tiêu chí đánh giá được nêu trong Phần 3.3.4. So sánh này dành cho các phiên bản tìm kiếm giống cây không kiểm tra các trạng thái lặp lại. Đối với các tìm kiếm đồ thị kiểm tra, sự khác biệt chính là tìm kiếm độ sâu đầu tiên hoàn thành đối với không gian trạng thái hữu hạn và độ phức tạp của không gian và thời gian bị giới hạn bởi kích thước của không gian trạng thái (số lượng đỉnh và cạnh, $|V| + |E|$).

Tiêu chuẩn	Chiều rộng- Chi phí đầu tiên	Đồng phục- Chi phí đầu tiên	Độ sâu-Độ sâu- Đầu tiên Hạn chế	Lặp Sâu sắc	Hai chiều (nếu có)
Hoàn thành?	Có1	Có1,2	Sức mạnh	Có1	Có1,4
Chi phí tối ưu?	Có3	Có	Sức mạnh	Có3	Có3,4
Thời gian	$O(bd)$	$O(b(1 + bC^*) / \epsilon)$	$O(bm)$ $O(b')$	$O(bd)$	$O(bd / 2)$
Không gian	$O(bd)$	$O(b(1 + bC^*) / \epsilon)$	$O(bm)$ $O(b')$	$O(bd)$	$O(bd / 2)$

Hình 3.15 Đánh giá các thuật toán tìm kiếm. b là hệ số phân nhánh; m là độ sâu tối đa của cây tìm kiếm; d là độ sâu của dung dịch nông nhất, hoặc là m khi không có dung dịch; ϵ là giới hạn độ sâu. Các cảnh báo chỉ số trên như sau: 1 hoàn thành nếu b là hữu hạn, và không gian trạng thái có một nghiệm hoặc là hữu hạn. 2 hoàn thành nếu tất cả các chi phí hành động là ≥ 0 ; 3 chi phí tối ưu nếu chi phí hành động đều giống hệt nhau; 4 nếu cả hai hướng đều là chiều rộng đầu tiên hoặc chi phí đồng đều.

3.5 Chiến lược tìm kiếm được thông tin (heuristic)

Tìm kiếm sáng suốt

Phản này cho thấy cách một chiến lược tìm kiếm được thông báo - một chiến lược sử dụng các gợi ý dành riêng cho miền

Chức năng phỏng đoán

về vị trí của các mục tiêu — có thể tìm ra giải pháp hiệu quả hơn so với một chiến lược thiếu thông tin. Các gợi ý đến dưới dạng một hàm heuristic, được ký hiệu là $h(n)$: 10

$h(n)$ = chi phí ước tính của đường dẫn rẻ nhất từ trạng thái tại nút n đến trạng thái mục tiêu. Ví dụ, trong các bài toán tìm đường, chúng ta có thể ước tính khoảng cách từ trạng thái hiện tại đến mục tiêu bằng cách tính khoảng cách đường thẳng trên bản đồ giữa hai điểm. Chúng tôi nghiên cứu heuristics và nguồn gốc của chúng chi tiết hơn trong Phần 3.6.

10 Có vẻ kỳ lạ khi hàm heuristic hoạt động trên một nút, trong khi tất cả những gì nó thực sự cần là trạng thái của nút. Theo truyền thống, sử dụng $h(n)$ thay vì $h(s)$ để phù hợp với hàm đánh giá $f(n)$ và chi phí đường dẫn $g(n)$.

Arad	366	Mehadia	241
Bucharest	0160	NeamtOradea	234
Craiova	2421		380
Drobeta	61	PitestiRimnicu Vilcea	100193
Eforia			
Fagaras	176	Sibi	253329
GiurgiuHirsova	77151	Timisoara	
		Urziceni	80199
Iasi	226	Vaslui	
Lugoj	244	Zerind	374

Hình 3.16 Giá trị của hSLD — khoảng cách đường thẳng đến Bucharest.

3.5.1 Tìm kiếm tốt nhất đầu tiên tham lam

Tìm kiếm tốt nhất đầu tiên tham lam là một hình thức tìm kiếm tốt nhất đầu tiên mở rộng đầu tiên nút với giá trị $H(n)$ thấp nhất - nút đường như gần nhất với mục tiêu - với lý do điều này có khả năng dẫn đến một giải pháp nhanh chóng. Vì vậy, hàm đánh giá $f(n) = h(n)$.

Hãy để chúng tôi xem điều này hoạt động như thế nào đối với các vấn đề tìm đường ở Romania; chúng tôi sử dụng phỏng đoán khoảng cách đường thẳng, mà chúng tôi sẽ gọi là hSLD. Nếu mục tiêu là Bucharest, chúng ta cần biết

khoảng cách đường thẳng đến Bucharest, được thể hiện trong Hình 3.16. Ví dụ, hSLD (Arad) = 366. Lưu ý rằng các giá trị của hSLD không thể được tính toán từ chính mô tả vấn đề (nghĩa là các hàm ACTIONS và RESULT). Hơn nữa, cần một lượng kiến thức thế giới nhất định để biết rằng hSLD tương quan với khoảng cách đường thực tế và do đó, là một phương pháp phỏng đoán hữu ích.

Hình 3.17 cho thấy tiến trình của một tìm kiếm tham lam tốt nhất đầu tiên sử dụng hSLD để tìm một con đường từ Arad đến Bucharest. Nút đầu tiên được mở rộng từ Arad sẽ là Sibiu vì heuristic nói rằng nó gần Bucharest hơn là Zerind hoặc Timisoara. Nút tiếp theo được mở rộng sẽ là Fagaras vì nó hiện gần nhất theo phỏng đoán. Fagaras lần lượt tạo ra Bucharest, đó là mục tiêu. Đối với vấn đề cụ thể này, tìm kiếm tốt nhất đầu tiên tham lam sử dụng hSLD tìm ra một giải pháp mà không bao giờ mở rộng một nút không nằm trên đường dẫn giải pháp. Tuy nhiên, giải pháp mà họ tìm thấy không có chi phí tối ưu: con đường qua Sibiu và Fagaras đến Bucharest dài hơn 32 dặm so với con đường qua Rimnicu Vilcea và Pitesti. Đây là lý do tại sao thuật toán được gọi là "tham lam" - trong mỗi lần lặp lại, nó cố gắng đến gần mục tiêu nhất có thể, nhưng lòng tham có thể dẫn đến kết quả tồi tệ hơn là cần thận.

Tìm kiếm đồ thị tốt nhất đầu tiên tham lam hoàn thành trong không gian trạng thái hữu hạn, nhưng không phải trong không gian trạng thái vô hạn. Độ phức tạp về thời gian và không gian trong trường hợp xấu nhất là $O(|V|)$. Tuy nhiên, với một hàm heuristic tốt, độ phức tạp có thể được giảm đáng kể, đối với một số vấn đề đạt đến $O(bm)$.

3.5.2 Tìm kiếm A^*

Thuật toán tìm kiếm thông tin phổ biến nhất là tìm kiếm A^* (phát âm là "tìm kiếm sao A "), một A^* tìm kiếm

Tìm kiếm tốt nhất đầu tiên sử dụng chức năng đánh giá

$f(n) = g(n) + h(n)$ trong đó $g(n)$ là chi phí đường dẫn từ trạng thái ban đầu đến nút n và $h(n)$ là chi phí ước tính của đường đi ngắn nhất từ n đến trạng thái mục tiêu, vì vậy chúng ta có

$f(n)$ = chi phí ước tính của đường dẫn tốt nhất tiếp tục từ n đến một mục tiêu.

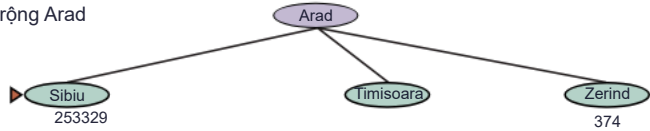
Tham lam tìm kiếm tốt nhất đầu tiên

Khoảng cách đường thẳng

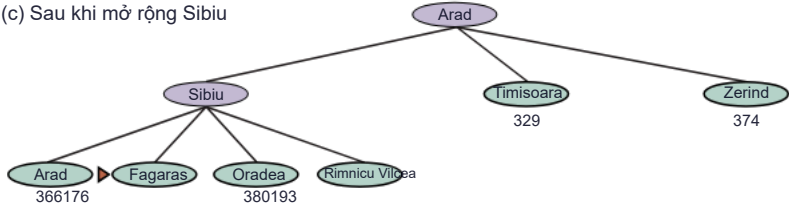
(a) Trạng thái ban đầu



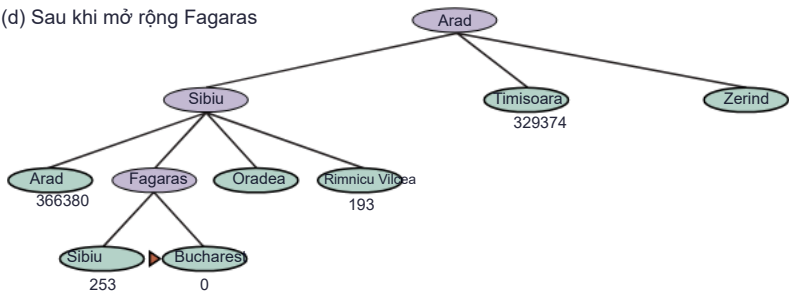
(b) Sau khi mở rộng Arad



(c) Sau khi mở rộng Sibiu



(d) Sau khi mở rộng Fagaras



Hình 3.17 Các giai đoạn trong một cuộc tìm kiếm giống như cây tốt nhất đầu tiên tham lam cho Bucharest với hSLD heuristic khoảng cách đường thẳng. Các nút được dán nhãn bằng giá trị h của chúng.

Trong Hình 3.18, chúng tôi cho thấy tiến trình tìm kiếm A* với mục tiêu đến Bucharest. Các giá trị của g được tính từ chi phí hành động trong Hình 3.1 và các giá trị của hSLD được đưa ra trong Hình 3.16. Lưu ý rằng Bucharest lần đầu tiên xuất hiện trên biên giới ở bước (e), nhưng nó không được chọn để mở rộng (và do đó không được phát hiện như một giải pháp) bởi vì ở $f = 450$ nó không phải là nút có chi phí thấp nhất trên biên giới - đó sẽ là Pitesti, ở $f = 417$. Một cách khác để nói điều này là có thể có một giải pháp thông qua Pitesti có chi phí thấp tới 417, vì vậy thuật toán sẽ không giải quyết cho một giải pháp có giá 450. Ở bước (f), một con đường khác đến Bucharest hiện là nút có chi phí thấp nhất, ở $f = 418$, vì vậy nó được chọn và phát hiện là giải pháp tối ưu.

A* tìm kiếm đã hoàn tất. 11 A* có tối ưu về chi phí hay không phụ thuộc vào một số thuộc tính nhất định của phỏng đoán. Một thuộc tính quan trọng là khả năng chấp nhận: một heuristic được chấp nhận là một phương pháp không bao giờ

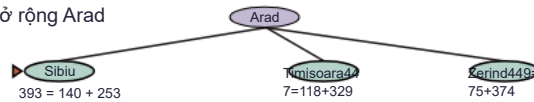
đánh giá quá cao chi phí để đạt được mục tiêu. (Do đó, một heuristic được chấp nhận là lạc quan.) Với

11 Một lần nữa, giả sử tất cả các chi phí hành động là > 0 , và không gian trạng thái có một giải pháp hoặc là hữu hạn.

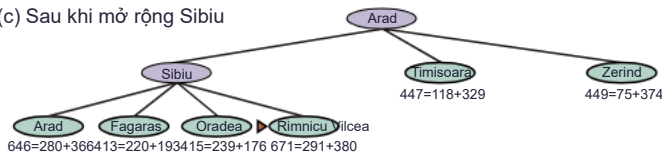
(a) Trạng thái ban đầu



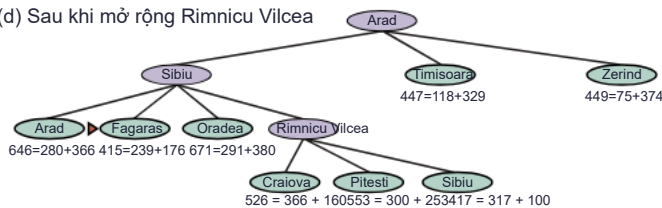
(b) Sau khi mở rộng Arad



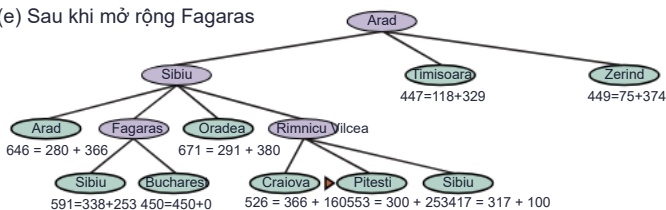
(c) Sau khi mở rộng Sibiu



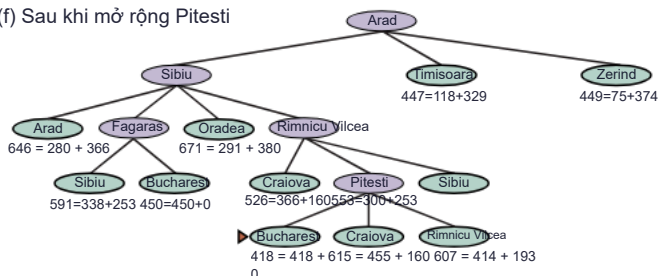
(d) Sau khi mở rộng Rimnicu Vilcea



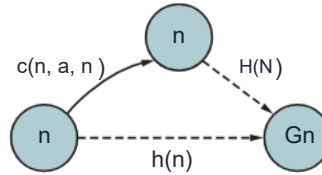
(e) Sau khi mở rộng Fagaras



(f) Sau khi mở rộng Pitesti



Hình 3.18 Các giai đoạn trong tìm kiếm A* cho Bucharest. Các nút được dán nhãn $f = g + h$. Các hvalues là khoảng cách đường thẳng đến Bucharest được lấy từ Hình 3.16.



Hình 3.19 Bất đẳng thức tam giác: Nếu h heuristic nhất quán, thì số duy nhất $h(n)$ sẽ nhỏ hơn tổng chi phí $c(n, a, n')$ của hành động từ n đến n' cộng với ước tính heuristic $h(n')$.

một phỏng đoán có thể chấp nhận được, A^* là tối ưu về chi phí, mà chúng ta có thể chỉ ra bằng một chứng minh bằng mâu thuẫn. Giả sử đường dẫn tối ưu có chi phí C^* , nhưng thuật toán trả về một đường dẫn có chi phí $C > C^*$. Sau đó, phải có một số nút n nằm trên đường dẫn tối ưu và không được mở rộng (bởi vì nếu tất cả các nút trên đường dẫn tối ưu đã được mở rộng, thì chúng ta sẽ trả về giải pháp tối ưu đó). Vì vậy, sau đó, sử dụng ký hiệu $g^*(n)$ để có nghĩa là chi phí của đường dẫn tối ưu từ đầu đến n và $h^*(n)$ để có nghĩa là chi phí của đường dẫn tối ưu từ n đến mục tiêu gần nhất, chúng ta có:

$f(n) > C^*$ (nếu không n sẽ được mở rộng) $f(n) = g(n) + h(n)$ (theo định nghĩa) $f(n) = g^*(n) + h(n)$ (vì n đang đi trên con đường tối ưu) $f(n) \leq g^*(n) + h^*(n)$ (vì khả năng chấp nhận, $h(n) \leq h^*(n)$) $f(n) \leq C^*$ (theo định nghĩa, $C^* = g^*(n) + h^*(n)$) Các dòng đầu tiên và cuối cùng tạo thành một mâu thuẫn, vì vậy giả định rằng thuật toán có thể trả về một đường dẫn dưới mức tối ưu phải là sai - nó phải là A^* chỉ trả về các đường dẫn tối ưu về chi phí.

Tính nhất quán

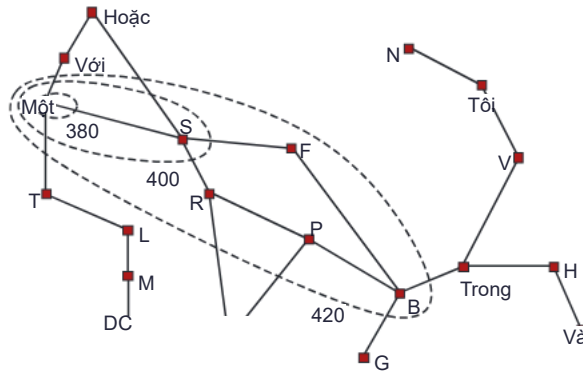
Một thuộc tính mạnh hơn một chút được gọi là tính nhất quán. $H(n)$ heuristic nhất quán nếu, cho mỗi nút n và mỗi nút kế thừa n' của n được tạo ra bởi một hành động A , chúng ta có:

$h(n) \leq c(n, a, n') + h(n')$. Đây là một dạng bất đẳng thức tam giác, quy định rằng một cạnh của tam giác không thể

Bất đẳng thức tam giác

dài hơn tổng của hai cạnh còn lại (xem Hình 3.19). Một ví dụ về phương pháp phỏng đoán nhất quán là khoảng cách đường thẳng hSLD mà chúng tôi đã sử dụng để đến Bucharest.

Mọi heuristic nhất quán đều có thể chấp nhận được (nhưng không phải ngược lại), vì vậy với một heuristic nhất quán, A^* là tối ưu về chi phí. Ngoài ra, với một phương pháp phỏng đoán nhất quán, lần đầu tiên chúng ta đạt đến một trạng thái, nó sẽ đi trên một con đường tối ưu, vì vậy chúng ta không bao giờ phải thêm lại một trạng thái vào biên giới và không bao giờ phải thay đổi một mục trong đã đạt được. Nhưng với một phương pháp phỏng đoán không nhất quán, chúng ta có thể kết thúc với nhiều đường dẫn đạt đến cùng một trạng thái và nếu mỗi đường dẫn mới có chi phí đường dẫn thấp hơn đường dẫn trước, thì chúng ta sẽ kết thúc với nhiều nút cho trạng thái đó ở biên giới, khiến chúng ta tốn cả thời gian và không gian. Do đó, một số triển khai của A^* chỉ quan tâm đến việc nhập một trạng thái vào biên giới một lần và nếu tìm thấy một đường dẫn tốt hơn đến trạng thái, tất cả các trình kế thừa của trạng thái sẽ được cập nhật (điều này yêu cầu các nút phải có con trỏ con cũng như con trỏ mẹ). Những phức tạp này đã khiến nhiều người thực hiện tránh các phương pháp phỏng đoán không nhất quán, nhưng Felner et al. (2011) lập luận rằng các tác động tiêu cực nhất hiềm khi xảy ra trong thực tế và người ta không nên sợ các phương pháp phỏng đoán không nhất quán.



Hình 3.20 Bản đồ Romania hiển thị các đường viền tại $f = 380$, $f = 400$ và $f = 420$, với Arad là trạng thái bắt đầu. Các nút bên trong một đường viền nhất định có giá $f = g + h$ nhỏ hơn hoặc bằng giá trị đường viền.

Với một phương pháp phỏng đoán không thể chấp nhận được, A* có thể tối ưu hoặc không có thể tối ưu về chi phí. Dưới đây là hai trường hợp: Thứ nhất, nếu có thậm chí một đường dẫn tối ưu về chi phí mà $h(n)$ được chấp nhận cho tất cả các nút n trên đường dẫn, thì đường dẫn đó sẽ được tìm thấy, bất kể heuristic nói gì cho các trạng thái ngoài đường dẫn. Thứ hai, nếu giải pháp tối ưu có chi phí C^* và giải pháp tốt thứ hai có chi phí C_2 , và nếu $h(n)$ ước tính quá cao một số chi phí, nhưng không bao giờ nhiều hơn $C_2 - C^*$, thì A* được đảm bảo trả lại các giải pháp tối ưu về chi phí.

3.5.3 Tìm kiếm đường viền

Một cách hữu ích để trực quan hóa tìm kiếm là vẽ các đường viền trong không gian trạng thái, giống như các đường viền Đường nét

trong bản đồ địa hình. Hình 3.20 cho thấy một ví dụ. Bên trong đường viền được dán nhãn 400, tất cả các nút có $f(n) = g(n) + h(n) \leq 400$, v.v. Sau đó, vì A* mở rộng nút biên giới của chi phí f thấp nhất, chúng ta có thể thấy rằng tìm kiếm A* quạt ra từ nút bắt đầu, thêm các nút các dải không đồng tâm tăng chi phí f .

Với tìm kiếm chi phí đồng đều, chúng ta cũng có đường viền, nhưng chi phí g , không phải $g + h$. Các đường viền với tìm kiếm chi phí đồng đều sẽ là "hình tròn" xung quanh trạng thái bắt đầu, trải đều theo mọi hướng mà không có ưu tiên về mục tiêu. Với tìm kiếm A* sử dụng một phương pháp phỏng đoán tốt, dải $g + h$ sẽ kéo dài về trạng thái mục tiêu (như trong Hình 3.20) và trở nên tập trung hẹp hơn xung quanh một con đường tối ưu.

Cần rõ ràng rằng khi bạn mở rộng một con đường, chi phí g là đơn điệu: chi phí đường dẫn Đơn điệu luôn tăng lên khi bạn đi dọc theo một con đường, bởi vì chi phí hành động luôn dương.¹² Do đó, bạn có được các đường viền đồng tâm không cắt ngang nhau, và nếu bạn chọn vẽ các đường thẳng đủ tốt, bạn có thể đặt một đường thẳng giữa hai nút bất kỳ trên bất kỳ con đường nào.

¹² Về mặt kỹ thuật, chúng tôi nói "đơn điệu nghiêm ngặt" cho các chi phí luôn tăng, và "đơn điệu" cho các chi phí không bao giờ giảm, nhưng có thể vẫn giữ nguyên.

Nhưng không rõ liệu chi phí $f = g + h$ có tăng đơn điệu hay không. Khi bạn thực hiện một con đường từ n đến n' , chi phí sẽ tăng từ $g(n) + h(n)$ đến $g(n) + c(n, a, n') + h(n')$. Hủy bỏ số hạng $g(n)$, chúng ta thấy rằng chi phí của đường dẫn sẽ tăng lên một cách đơn điệu nếu và chỉ nếu, nếu $(n) \leq c(n, a, n') + h(n')$; nói cách khác nếu và chỉ khi heuristic nhất quán.¹³ Nhưng lưu ý rằng một đường dẫn có thể đóng góp một số nút trong một liên tiếp với cùng điểm $g(n) + h(n)$; Điều này sẽ xảy ra bất cứ khi nào sự giảm H chính xác bằng chi phí hành động vừa thực hiện (ví dụ: trong một bài toán lưới, khi n ở cùng hàng với mục tiêu và bạn bước một bước về phía mục tiêu, g tăng 1 và h giảm 1). Nếu C^* là chi phí của đường dẫn giải tối ưu, thì chúng ta có thể nói như sau:

Chắc chắn các nút mở rộng

- A^* mở rộng tất cả các nút có thể đạt được từ trạng thái ban đầu trên đường dẫn mà mọi nút trên đường dẫn đều có $f(n) < C^*$. Chúng tôi nói rằng đây chắc chắn là các nút mở rộng.
- A^* sau đó có thể mở rộng một số nút ngay trên "đường viền mục tiêu" (trong đó $f(n) = C^*$) trước khi chọn nút mục tiêu.

Hiệu quả tối ưu

- A^* không mở rộng các nút nào với $f(n) > C^*$. Chúng tôi nói rằng A^* với một heuristic nhất quán là hiệu quả tối ưu theo nghĩa là bất kỳ thuật toán nào

mở rộng các đường dẫn tìm kiếm từ trạng thái ban đầu và sử dụng cùng một thông tin heuristic, phải mở rộng tất cả các nút chắc chắn được mở rộng bởi A^* (bởi vì bất kỳ một trong số chúng đều có thể là một phần của giải pháp tối ưu). Trong số các nút có $f(n) = C^*$, một thuật toán có thể gặp may mắn và chọn thuật toán tối ưu trước trong khi thuật toán khác không may mắn; Chúng tôi không xem xét sự khác biệt này trong việc xác định hiệu quả tối ưu.

Tia

A^* hiệu quả vì nó cắt bỏ các nút cây tìm kiếm không cần thiết để tìm kiếm một giải pháp tối ưu. Trong Hình 3.18 (b) chúng ta thấy rằng Timisoara có $f = 447$ và Zerind có $f = 449$. Mặc dù chúng là con của gốc và sẽ nằm trong số các nút đầu tiên được mở rộng bằng tìm kiếm chi phí đồng nhất hoặc chiều rộng đầu tiên, chúng không bao giờ được mở rộng bởi tìm kiếm A^* vì giải pháp có $f = 418$ được tìm thấy đầu tiên. Khái niệm cắt tia — loại bỏ các khả năng khỏi việc xem xét mà không cần phải kiểm tra chúng — rất quan trọng đối với nhiều lĩnh vực của AI. Việc tìm kiếm A^* là hoàn chỉnh, tối ưu về chi phí và hiệu quả tối ưu trong số tất cả các thuật toán như vậy là khá thỏa mãn. Thật không may, điều đó không có nghĩa là A^* là câu trả lời cho tất cả các nhu cầu tìm kiếm của chúng ta. Điểm mấu chốt là đối với nhiều vấn đề, số lượng nút mở rộng có thể theo cấp số nhân trong độ dài của lời giải. Ví dụ, hãy xem xét một phiên bản của thế giới chân không với một máy hút siêu mạnh có thể dọn dẹp bất kỳ hình vuông nào với chi phí 1 đơn vị, thậm chí không cần phải đến thăm quảng trường; Trong trường hợp đó, các ô vuông có thể được làm sạch theo bất kỳ thứ tự nào. Với N hình vuông ban đầu, có $2N$ trạng thái mà một số tập hợp con đã được làm sạch; tất cả các trạng thái đó đều nằm trên một con đường giải pháp tối ưu, và do đó thỏa mãn $f(n) < C^*$, vì vậy tất cả chúng sẽ được truy cập bởi A^* .

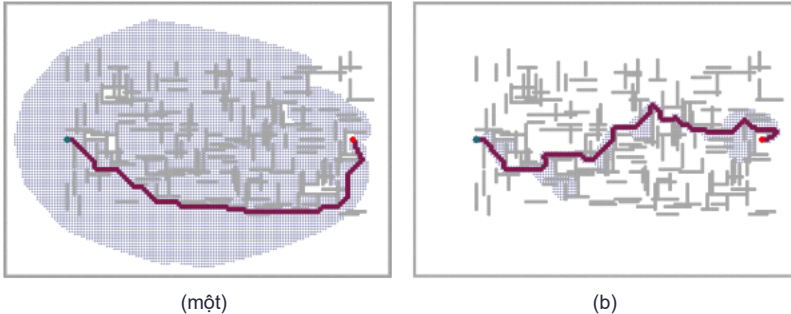
3.5.4 Tìm kiếm thỏa đáng: Phỏng đoán không thể chấp nhận được và A có trọng số*

Không thể chấp nhận được

A^* tìm kiếm có nhiều phẩm chất tốt, nhưng nó mở rộng rất nhiều nút. Chúng ta có thể khám phá ít nút hơn (mất ít thời gian và không gian hơn) nếu chúng ta sẵn sàng chấp nhận các giải pháp không tối ưu, nhưng "đủ tốt" - điều mà chúng ta gọi là giải pháp thỏa đáng. Nếu chúng ta cho phép tìm kiếm A^* sử dụng một phỏng đoán không thể chấp nhận được - một phương pháp có thể đánh giá quá cao - thì chúng ta có nguy cơ bỏ lỡ phương pháp tối ưu

nhưng heuristic có khả năng chính xác hơn, do đó giảm số lượng

¹³ Trên thực tế, thuật ngữ "phỏng đoán đơn điệu" là một từ đồng nghĩa với "phỏng đoán nhất quán". Hai ý tưởng được phát triển độc lập, và sau đó nó được chứng minh rằng chúng tương đương nhau (Pearl, 1984).



Hình 3.21 Hai tìm kiếm trên cùng một lưới: (a) tìm kiếm A* có trọng số với trọng số $W = 2$. Các thanh màu xám là chướng ngại vật, đường màu tím là đường đi từ greenstart đến mục tiêu màu đỏ và các chấm nhỏ là trạng thái đạt được bởi mỗi lần tìm kiếm. Về vấn đề cụ thể này, trọng số A* khám phá các trạng thái ít hơn 7 lần và tìm ra một con đường tốn kém hơn 5%.

các nút được mở rộng. Ví dụ, các kỹ sư đường bộ biết khái niệm về chỉ số đường vòng, đó là Chỉ số đường vòng một hệ số nhân được áp dụng cho khoảng cách đường thẳng để giải thích độ cong điển hình của đường. Chỉ số đường vòng là 1,3 có nghĩa là nếu hai thành phố cách nhau 10 dặm trong khoảng cách đường thẳng, ước tính tốt về con đường tốt nhất giữa chúng là 13 dặm. Đối với hầu hết các địa phương, chỉ số đường vòng nằm trong khoảng từ 1,2 đến

đường ta có thể áp dụng ý tưởng này cho bất kỳ vấn đề nào, không chỉ những vấn đề liên quan đến đường xá, với một cách tiếp cận được gọi là tìm kiếm A* trọng số, trong đó chúng ta có trọng số giá trị heuristic nặng hơn, cho chúng ta

Tìm kiếm A* có trọng số

hàm đánh giá $f(n) = g(n) + W \times h(n)$, đối với một số $W >$

Hình 3.21 cho thấy một bài toán tìm kiếm trên thế giới lưới. Trong (a), tìm kiếm A* tìm ra giải pháp tối ưu, nhưng phải khám phá một phần lớn không gian trạng thái để tìm ra nó. Trong (b), tìm kiếm A* có trọng số tìm thấy một giải pháp tốn kém hơn một chút, nhưng thời gian tìm kiếm nhanh hơn nhiều. Chúng ta thấy rằng tìm kiếm có trọng số tập trung đường viền của các trạng thái đạt được hướng tới một mục tiêu. Điều đó có nghĩa là ít trạng thái được khám phá, nhưng nếu con đường tối ưu đi lạc ra ngoài đường viền của tìm kiếm có trọng số (như trong trường hợp này), thì con đường tối ưu sẽ không được tìm thấy. Nói chung, nếu giải pháp tối ưu có giá C^* , thì tìm kiếm A* có trọng số sẽ tìm thấy một giải pháp có giá ở đâu đó giữa C^* và $W \times C^*$; nhưng trong thực tế, chúng ta thường nhận được kết quả gần với C^* hơn $W \times C^*$.

Chúng tôi đã xem xét các tìm kiếm đánh giá trạng thái bằng cách kết hợp g và h theo nhiều cách khác nhau; trọng số A* có thể được coi là một khái quát hóa của những người khác:

A* tìm kiếm: $g(n) + h(n)$ ($W = 1$)

Tìm kiếm chi phí đồng phục: $g(n)$ ($W = 0$)

Tìm kiếm tốt nhất đầu tiên tham lam: $h(n)$ ($W = \infty$)

Tìm kiếm A* có trọng số: $g(n) + W \times h(n)$ ($1 < W < \infty$)

Bạn có thể gọi trọng số A* "tìm kiếm có phần tham lam": giống như tìm kiếm tham lam tốt nhất đầu tiên, nó tập trung tìm kiếm hướng tới một mục tiêu; Mặt khác, nó sẽ không bỏ qua hoàn toàn chi phí đường dẫn và sẽ định chỉ một con đường đang đạt được ít tiến bộ với chi phí lớn.

Tìm kiếm dưới tối ưu có giới hạn

Tìm kiếm chi phí giới hạn
Chi phí không giới hạn tìm kiếm

Có nhiều thuật toán tìm kiếm dưới mức tối ưu, có thể được đặc trưng bởi các tiêu chí cho những gì được coi là "dù tốt". Trong tìm kiếm dưới tối ưu có giới hạn, chúng ta tìm kiếm một giải pháp được đảm bảo nằm trong hệ số không đổi W của chi phí tối ưu. Weighted A* cung cấp đảm bảo này. Trong tìm kiếm chi phí giới hạn, chúng tôi tìm kiếm một giải pháp có chi phí thấp hơn hơn một số C hằng số. Và trong tìm kiếm chi phí không giới hạn, chúng tôi chấp nhận một giải pháp của bất kỳ chi phí nào, như

miễn là chúng ta có thể tìm thấy nó một cách nhanh chóng.

Tìm kiếm nhanh chóng

Một ví dụ về thuật toán tìm kiếm chi phí không giới hạn là tìm kiếm nhanh, là một phiên bản của tìm kiếm tốt nhất đầu tiên tham lam sử dụng như một phỏng đoán số lượng hành động ước tính cần thiết để đạt được mục tiêu, bất kể chi phí của những hành động đó. Do đó, đối với các vấn đề mà tất cả các hành động đều có cùng chi phí thì nó cũng giống như tìm kiếm tốt nhất đầu tiên tham lam, nhưng khi hành động có chi phí khác nhau, nó có xu hướng dẫn dắt tìm kiếm để tìm ra giải pháp một cách nhanh chóng, ngay cả khi nó có thể có chi phí cao.

3.5.5 Tìm kiếm giới hạn bộ nhớ

Vấn đề chính với A* là việc sử dụng bộ nhớ. Trong phần này, chúng ta sẽ đề cập đến một số thủ thuật triển khai giúp tiết kiệm dung lượng và sau đó là một số thuật toán hoàn toàn mới tận dụng tốt hơn không gian có sẵn.

Kỷ ức được phân chia giữa biên giới và các trạng thái đạt được. Trong việc triển khai tìm kiếm tốt nhất đầu tiên của chúng tôi, một trạng thái nằm trên biên giới được lưu trữ ở hai nơi: như một nút trong biên giới (vì vậy chúng ta có thể quyết định những gì sẽ mở rộng tiếp theo) và như một mục nhập trong bảng các trạng thái đã đạt được (vì vậy chúng ta biết liệu chúng ta đã truy cập trạng thái trước đó hay chưa). Đối với nhiều vấn đề (chẳng hạn như khám phá một lưới), sự trùng lặp này không phải là một vấn đề đáng lo ngại, bởi vì kích thước của biên giới nhỏ hơn nhiều so với mức đạt được, vì vậy việc sao chép các trạng thái trong biên giới đòi hỏi một lượng bộ nhớ tương đối nhỏ. Tuy nhiên, một số triển khai chỉ giữ trạng thái ở một trong hai nơi, tiết kiệm một chút không gian với chi phí làm phức tạp (và có thể làm chậm lại) thuật toán.

Một khả năng khác là loại bỏ các trạng thái khỏi phạm vi tiếp cận khi chúng ta có thể chứng minh rằng chúng không còn cần thiết nữa. Đối với một số vấn đề, chúng ta có thể sử dụng thuộc tính tách (Hình 3.6 trên trang 90), cùng với việc cấm các hành động quay đầu, để đảm bảo rằng tất cả các hành động hoặc di chuyển ra ngoài khỏi biên giới hoặc sang một trạng thái biên giới khác. Trong trường hợp đó, chúng ta chỉ cần kiểm tra biên giới để tìm các đường dẫn dư thừa và chúng ta có thể loại bỏ bảng đã đạt được.

Số lượng tham chiếu

Đối với các vấn đề khác, chúng ta có thể giữ số lần tham chiếu về số lần một trạng thái có đã đạt được và xóa nó khỏi bảng đạt được khi không còn cách nào để tiếp cận trạng thái. Ví dụ, trên một thế giới lưới nơi mỗi trạng thái chỉ có thể đạt được từ bốn người lân cận của nó, một khi chúng ta đã đạt đến một trạng thái bốn lần, chúng ta có thể xóa nó khỏi bảng.

Tìm kiếm chùm tia

Bây giờ chúng ta hãy xem xét các thuật toán mới được thiết kế để tiết kiệm việc sử dụng bộ nhớ. Tìm kiếm chùm tia giới hạn kích thước của biên giới. Cách tiếp cận dễ nhất là chỉ giữ k các nút có điểm f tốt nhất, loại bỏ bất kỳ nút mở rộng nào khác. Tất nhiên điều này làm cho tìm kiếm không hoàn chỉnh và không tối ưu, nhưng chúng ta có thể chọn k để sử dụng tốt bộ nhớ có sẵn và thuật toán thực thi nhanh vì nó mở rộng ít nút hơn. Đối với nhiều vấn đề, nó có thể tìm thấy các giải pháp tốt gần như tối ưu. Bạn có thể nghĩ về tìm kiếm chi phí đồng nhất hoặc A* trải rộng khắp mọi nơi trong các đường viền đồng tâm và nghĩ về tìm kiếm chùm tia là khám phá chỉ một phần tập trung của các đường viền đó, phần chứa k ứng cử viên tốt nhất.

Một phiên bản thay thế của tìm kiếm chùm tia không giữ giới hạn nghiêm ngặt về kích thước của biên giới mà thay vào đó giữ mọi nút có điểm f nằm trong δ điểm f tốt nhất. Bằng cách đó, khi có một vài nút ghi điểm mạnh, chỉ một vài nút sẽ được giữ lại, nhưng nếu không có nút mạnh thì sẽ giữ lại nhiều hơn cho đến khi một nút mạnh xuất hiện.

Tìm kiếm A* sâu lặp đi lặp lại (IDA*) đối với A* tìm kiếm sâu lặp đi lặp lại là gì độ sâu đầu tiên: IDA* mang lại cho chúng ta những lợi ích của A* mà không yêu cầu giữ tất cả các trạng thái đã đạt được trong bộ nhớ, với chi phí phải ghé thăm một số tiểu bang nhiều lần. Nó là một thuật toán rất quan trọng và thường được sử dụng cho các vấn đề không vừa với bộ nhớ.

Lặp đi lặp lại-làm sâu
Tìm kiếm*

Trong độ sâu lặp đi lặp lại tiêu chuẩn, ngưỡng là độ sâu, được tăng lên một lần mỗi lần lặp. Trong IDA* ngưỡng là f -chi phí ($g + h$); Tại mỗi lần lặp, giá trị giới hạn là chi phí f nhỏ nhất của bất kỳ nút nào vượt quá mức giới hạn trong lần lặp trước đó. Nói cách khác, mỗi lần lặp lại tìm kiếm một đường viền f một cách triệt để, tìm một nút ngay bên ngoài đường viền đó và sử dụng chi phí f của nút đó làm đường viền tiếp theo. Đối với các vấn đề như câu đố 8 trong đó mỗi con ~~đồng~~ là một số nguyên, điều này hoạt động rất tốt, dẫn đến tiến trình ổn định hướng tới mục tiêu mỗi lần lặp. Nếu giải pháp tối ưu có chi phí C^* , thì không thể có nhiều hơn C^* lần lặp lại (ví dụ: không quá 31 lần lặp lại cho các bài toán 8 câu đố khó nhất). Nhưng đối với một vấn đề mà mỗi nút có một f -cost khác nhau, mỗi đường viền mới có thể chỉ chứa một nút mới và số lần lặp lại có thể bằng số trạng thái.

Tìm kiếm tốt nhất đệ quy (RBFS) (Hình 3.22) cố gắng bắt chước hoạt động của tìm kiếm tốt nhất đầu tiên tiêu chuẩn, nhưng chỉ sử dụng không gian tuyến tính. RBFS giống như tìm kiếm độ sâu đệ quy, nhưng thay vì tiếp tục vô thời hạn xuống đường dẫn hiện tại, nó sử dụng biến f limit để theo dõi giá trị f của đường dẫn thay thế tốt nhất có sẵn từ bất kỳ tổ tiên nào của nút hiện tại. Nếu nút hiện tại vượt quá giới hạn này, đệ quy sẽ rút gọn trở lại đường dẫn thay thế. Khi đệ quy được tháo ra, RBFS thay thế giá trị f của mỗi nút dọc theo đường dẫn bằng một giá trị được sao lưu — giá trị f tốt nhất của các con của nó. Bằng cách này, RBFS ghi nhớ

Tìm kiếm tốt nhất đệ quy

giá trị f của chiếc lá tốt nhất trong cây con bị lãng quên và do đó có thể quyết định xem nó có đáng để mở rộng cây con vào một thời điểm nào đó sau này hay không. Hình 3.23 cho thấy RBFS đến Bucharest như thế nào. RBFS có phần hiệu quả hơn IDA*, nhưng vẫn bị tái tạo nút quá mức. Trong ví dụ trong Hình 3.23, RBFS đi theo đường dẫn qua Rimnicu Vilcea, sau đó

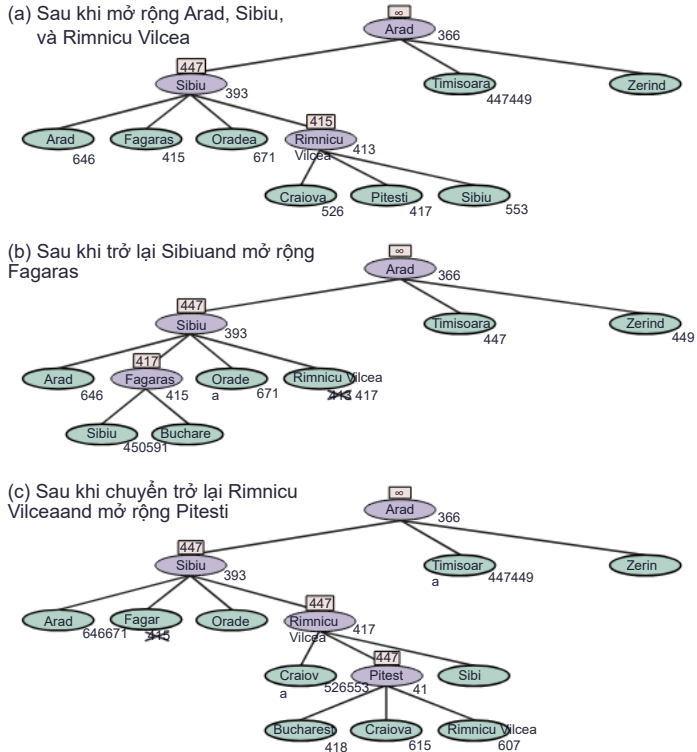
Giá trị dự phòng

hàm RECURSIVE-BEST-FIRST-SEARCH(problem) trả về một giải pháp hoặc thất bại
 solution, fvalue $\leftarrow$ RBFS(vấn đề, NODE(vấn đề, INITIAL), ∞) giải pháp trở lại

hàm RBFS(vấn đề, nút, giới hạn f) trả về một giải pháp hoặc thất bại, và một giới hạn chi phí f mới nếu có vấn đề. IS-GOAL(nút, STATE) sau đó trả về nodesuccessors
 $\leftarrow$ LIST(EXPAND(node)) nếu successors trống thì trả về failure, ∞ đối
 với mỗi s trong successors do// cập nhật f với giá trị từ tìm kiếm trước
 đó
 $s.f \leftarrow \max(s.PATH-COST + h(s), node.f)$ trong khi true do

tốt nhất $\leftarrow$ nút trong các nút kế thừa có giá trị f thấp nhất nếu
 tốt nhất. $f >$ giới hạn f sau đó trả về thất bại, tốt nhất. thay thế
 $\leftarrow$ giá trị f thấp thứ hai trong số những người kế nhiệm kết
 quả, tốt nhất. $f \leftarrow$ RBFS(vấn đề, tốt nhất, tối thiểu (giới hạn f ,
 thay thế)) nếu kết quả $\neq$ thất bại thì trả về kết quả, tốt nhất. f

Hình 3.22 Thuật toán tìm kiếm đệ quy tốt nhất đầu tiên.



Hình 3.23 Các giai đoạn trong RBFS tìm kiếm tuyến đường ngắn nhất đến Bucharest. Giá trị f -limit cho mỗi cuộc gọi đệ quy được hiển thị trên đầu mỗi nút hiện tại và mọi nút được gắn nhãn với f -cost của nó. (a) Con đường qua Rimnicu Vilcea được đi theo cho đến khi lá tốt nhất hiện tại (Pitesti) có giá trị kém hơn con đường thay thế tốt nhất (Fagaras). (b) Sự đệ quy được giải phóng và giá trị lá tốt nhất của cây con bị lãng quên (417) được sao lưu cho Rimnicu Vilcea; sau đó Fagaras được mở rộng, cho thấy giá trị lá tốt nhất là 450. (c) Đệ quy được giải phóng và giá trị lá tốt nhất của cây con bị lãng quên (450) được sao lưu cho Fagaras; sau đó Rimnicu Vilcea được mở rộng. Lần này, vì con đường thay thế tốt nhất (qua Timisoara) có giá ít nhất 447, việc mở rộng tiếp tục đến Bucharest.

"thay đổi suy nghĩ" và thử Fagaras, và sau đó lại thay đổi ý định. Những thay đổi tư duy này xảy ra bởi vì mỗi khi con đường tốt nhất hiện tại được mở rộng, giá trị f của nó có khả năng tăng lên - h thường kém lạc quan hơn đối với các nút gần với mục tiêu. Khi điều này xảy ra, con đường tốt thứ hai có thể trở thành con đường tốt nhất, vì vậy việc tìm kiếm phải quay lại để đi theo nó. Mỗi thay đổi tâm trí tương ứng với một lần lặp lại của IDA* và có thể yêu cầu nhiều lần mở rộng lại các nút bị lãng quên để tạo lại con đường tốt nhất và mở rộng nó thêm một nút nữa.

RBFS là tối ưu nếu hàm heuristic $h(n)$ được chấp nhận. Độ phức tạp không gian của nó là tuyến tính theo chiều sâu của giải pháp tối ưu sâu nhất, nhưng độ phức tạp thời gian của nó khá khó để mô tả: nó phụ thuộc vào cả độ chính xác của hàm phỏng đoán và tần suất đường dẫn tốt nhất thay đổi khi các nút được mở rộng. Nó mở rộng các nút theo thứ tự tăng điểm f , ngay cả khi f không đơn điệu.

IDA* và RBFS bị ảnh hưởng bởi việc sử dụng quá ít bộ nhớ. Giữa các lần lặp, IDA* chỉ giữ lại một con số duy nhất: giới hạn chi phí f hiện tại. RBFS giữ lại nhiều thông tin hơn trong bộ nhớ, nhưng nó chỉ sử dụng không gian tuyến tính: ngay cả khi có nhiều bộ nhớ hơn, RBFS không có cách nào để sử dụng nó. Bởi vì họ quên hầu hết những gì họ đã làm, cả hai thuật toán có thể cuối cùng khám phá lại các trạng thái giống nhau nhiều lần.

Do đó, có vẻ hợp lý để xác định chúng ta có bao nhiêu bộ nhớ và cho phép một thuật toán sử dụng tất cả chúng. Hai thuật toán thực hiện điều này là MA* (giới hạn bộ nhớ NHƯNG* A*) và SMA* (MA đơn giản *). SMA* đơn giản hơn, vì vậy chúng tôi sẽ mô tả nó. SMA* SMA* tiến hành giống như A*, mở rộng chiếc lá tốt nhất cho đến khi bộ nhớ đầy. Tại thời điểm này, nó không thể thêm một nút mới vào cây tìm kiếm mà không bỏ một nút cũ. SMA* luôn loại bỏ nút lá tồi tệ nhất — nút có giá trị f cao nhất. Giống như RBFS, SMA* sau đó sao lưu giá trị của nút bị lãng quên vào nút mẹ của nó. Bằng cách này, tổ tiên của một cây con bị lãng quên biết chất lượng của con đường tốt nhất trong cây con đó. Với thông tin này, SMA* chỉ tái tạo cây con khi tất cả các đường dẫn khác đã được chứng minh là trông kém hơn đường dẫn mà nó đã quên. Một cách khác để nói điều này là nếu tất cả các hậu duệ của một nút n bị lãng quên, thì chúng ta sẽ không biết nên đi theo con đường nào từ n , nhưng chúng ta vẫn sẽ có ý tưởng về việc đi bất cứ đâu từ n đáng giá như thế nào.

Thuật toán hoàn chỉnh được mô tả trong kho lưu trữ mã thực tuyến đi kèm với cuốn sách này. Có một sự tinh tế đáng nói. Chúng tôi đã nói rằng SMA* mở rộng lá tốt nhất và xóa lá xấu nhất. Điều gì sẽ xảy ra nếu tất cả các nút lá có cùng giá trị f ? Để tránh chọn cùng một nút để xóa và mở rộng, SMA* mở rộng lá tốt nhất mới nhất và xóa lá xấu nhất cũ nhất. Những điều này trùng khớp khi chỉ có một lá, nhưng trong trường hợp đó, cây tìm kiếm hiện tại phải là một đường dẫn duy nhất từ gốc này sang lá khác lấp đầy tất cả bộ nhớ. Nếu lá không phải là một nút mục tiêu, thì ngay cả khi nó nằm trên một đường dẫn giải pháp tối ưu, giải pháp đó không thể truy cập được với bộ nhớ có sẵn. Do đó, nút có thể bị loại bỏ chính xác như thể nó không có người kế nhiệm.

SMA* là hoàn chỉnh nếu có bất kỳ giải pháp nào có thể đạt được — nghĩa là, nếu d , độ sâu của nút mục tiêu thấp nhất, nhỏ hơn kích thước bộ nhớ (được biểu thị bằng các nút). Sẽ là tối ưu nếu có thể đạt được bất kỳ giải pháp tối ưu nào; nếu không, nó trả về giải pháp có thể truy cập tốt nhất. Về mặt thực tế, SMA* là một lựa chọn khá mạnh mẽ để tìm ra các giải pháp tối ưu, đặc biệt là khi không gian trạng thái là đồ thị, chi phí hành động không đồng nhất và việc tạo nút tốn kém so với chi phí duy trì biên giới và tập hợp đạt được.

Tuy nhiên, đối với các vấn đề rất khó, thường sẽ xảy ra trường hợp SMA* buộc phải chuyển đổi liên tục giữa nhiều đường dẫn giải pháp ứng cử viên, chỉ một tập hợp con nhỏ trong số đó có thể phù hợp với bộ nhớ. (Điều này giống như vấn đề đập trong hệ thống phân trang đĩa.) Sau đó

Đánh đập

thêm thời gian cần thiết để tái tạo lặp đi lặp lại của cùng một nút có nghĩa là các vấn đề thực tế có thể được giải quyết bằng A*, với bộ nhớ không giới hạn, trở nên khó giải quyết đối với SMA*. Điều đó có nghĩa là, giới hạn bộ nhớ có thể làm cho một vấn đề trở nên khó giải quyết từ quan điểm Jof về thời gian tính toán. Mặc dù không có lý thuyết hiện tại nào giải thích sự đánh đổi giữa thời gian

và trí nhớ, dường như đây là một vấn đề không thể tránh khỏi. Cách duy nhất là bỏ yêu cầu tối ưu.

3.5.6 Tìm kiếm phỏng đoán hai chiều

Với tìm kiếm tốt nhất đầu tiên một chiều, chúng ta thấy rằng việc sử dụng $f(n) = g(n) + h(n)$ làm hàm đánh giá cho chúng ta một tìm kiếm A^* được đảm bảo tìm ra các giải pháp chi phí tối ưu (giả sử h có thể chấp nhận được) trong khi vẫn hiệu quả tối ưu về số lượng nút được mở rộng.

Với tìm kiếm tốt nhất đầu tiên hai chiều, chúng ta cũng có thể thử sử dụng $f(n) = g(n) + h(n)$, nhưng không may là không có gì đảm bảo rằng điều này sẽ dẫn đến một giải pháp chi phí tối ưu, cũng như nó sẽ hiệu quả tối ưu, ngay cả với một phỏng đoán có thể chấp nhận được. Với tìm kiếm hai chiều, hóa ra không phải các nút riêng lẻ mà là các cặp nút (một từ mỗi biên giới) có thể được chứng minh là chắc chắn được mở rộng, vì vậy bất kỳ bằng chứng nào về hiệu quả sẽ phải xem xét các cặp nút (Eckerle và cộng sự, 2017).

Chúng ta sẽ bắt đầu với một số ký hiệu mới. Chúng tôi sử dụng $fF(n) = gF(n) + hF(n)$ cho các nút đi theo hướng tiến (với trạng thái ban đầu là gốc) và $fB(n) = gB(n) + hB(n)$ cho các nút theo hướng ngược (với trạng thái mục tiêu là gốc). Mặc dù cả tìm kiếm tiến và lùi đều giải quyết cùng một vấn đề, nhưng chúng có các chức năng đánh giá khác nhau bởi vì, ví dụ, các phỏng đoán khác nhau tùy thuộc vào việc bạn đang phấn đấu cho mục tiêu hay cho trạng thái ban đầu. Chúng tôi sẽ giả định các phỏng đoán có thể chấp nhận được.

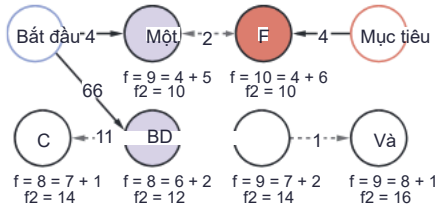
Xem xét một đường dẫn chuyển tiếp từ trạng thái ban đầu đến một nút m và một đường dẫn lùi từ mục tiêu đến một nút n . Chúng ta có thể xác định giới hạn dưới về chi phí của một giải pháp đi theo đường dẫn từ trạng thái ban đầu đến m , sau đó bằng cách nào đó đạt được n , sau đó đi theo đường dẫn đến mục tiêu như

$lb(m, n) = \max(gF(m) + gB(n), fF(m), fB(n))$ Nói cách khác, chi phí của một con đường như vậy ít nhất phải lớn bằng tổng chi phí đường dẫn của hai phần (vì kết nối còn lại giữa chúng phải có chi phí không âm), và chi phí cũng phải ít nhất bằng chi phí ước tính của một trong hai phần (vì ước tính heuristic là lạc quan). Cho rằng, định lý là đối với bất kỳ cặp nút nào m, n , với $lb(m, n)$ nhỏ hơn chi phí tối ưu C^* , chúng ta phải mở rộng m hoặc n , bởi vì đường đi qua cả hai là một giải pháp tối ưu tiềm năng. Khó khăn là chúng ta không biết chắc nút nào là tốt nhất để mở rộng, và do đó không có thuật toán tìm kiếm hai chiều nào có thể được đảm bảo là hiệu quả tối ưu — bất kỳ thuật toán nào cũng có thể mở rộng lên đến gấp đôi số nút tối thiểu nếu nó luôn chọn sai thành viên của một cặp để mở rộng trước. Một số thuật toán tìm kiếm phỏng đoán hai chiều quản lý rõ ràng một hàng đợi gồm các cặp (m, n) , nhưng chúng ta sẽ gắn bó với tìm kiếm tốt nhất đầu tiên hai chiều (Hình 3.14), có hai hàng đợi ưu tiên biên giới và cung cấp cho nó một chức năng đánh giá bất chước tiêu chỉ lb :

$f2(n) = \max(2g(n), g(n) + h(n))$ Nút cần mở rộng tiếp theo sẽ là nút giảm thiểu giá trị $f2$ này; nút có thể đến từ một trong hai biên giới. Hàm $f2$ này đảm bảo rằng chúng ta sẽ không bao giờ mở rộng một nút (từ một trong hai biên giới) với $g(n) > C^*2$. Chúng ta nói hai nửa của tìm kiếm "gặp nhau ở giữa" theo nghĩa là khi hai biên giới chạm nhau, không có nút nào bên trong một trong hai biên giới có chi phí đường đi lớn hơn C^*2 bị ràng buộc—Hình 3.24 hoạt động thông qua một ví dụ tìm kiếm hai chiều.

Chúng tôi đã mô tả một cách tiếp cận trong đó heuristic ước tính khoảng cách đến mục tiêu (hoặc, khi vấn đề có nhiều trạng thái mục tiêu, khoảng cách đến mục tiêu gần nhất) và hB ước tính khoảng cách đến điểm bắt đầu. Đây được gọi là tìm kiếm từ đầu đến cuối. Một giải pháp thay thế, được gọi là

tìm kiếm từ đầu đến trước, cố gắng ước tính khoảng cách đến biên giới khác. Rõ ràng, nếu một Frontier có hàng triệu nút, sẽ không hiệu quả nếu áp dụng hàm heuristic cho mọi



Hình 3.24 Tìm kiếm hai chiều duy trì hai ranh giới: ở bên trái, các nút A và B là các nút kế thừa của Start; ở bên phải, nút F là sự kế thừa nghịch đảo của Goal. Mỗi nút được dán nhãn với giá trị $f = g + h$ và giá trị $f_2 = \max(2g, g + h)$. (Các giá trị g là tổng của chi phí hành động như được hiển thị trên mỗi mũi tên; các giá trị h là tùy ý và không thể được lấy từ bất kỳ thứ gì trong hình.) Giải pháp tối ưu, Start-A-F-Goal, có giá $C^* = 4 + 2 + 4 = 10$, vì vậy điều đó có nghĩa là một thuật toán hai chiều gặp gỡ ở giữa không được mở rộng bất kỳ nút nào với $g > C^* / 2 = 5$; và thực sự nút tiếp theo được mở rộng sẽ là A hoặc F (mỗi nút có $g = 4$), dẫn chúng ta đến một giải pháp tối ưu. Nếu chúng ta mở rộng nút có chi phí f thấp nhất trước, thì Bảng tần C sẽ đến tiếp theo, và D và E sẽ được gắn với A, nhưng tất cả chúng đều có $g > C^* / 2$ và do đó không bao giờ mở rộng khi f_2 là hàm đánh giá.

một trong số đó và lấy mức tối thiểu. Nhưng nó có thể hoạt động để lấy mẫu một vài nút từ biên giới. Trong một số lĩnh vực vấn đề cụ thể, có thể tóm tắt biên giới — ví dụ, trong bài toán tìm kiếm lưới, chúng ta có thể tính toán dần một hộp giới hạn của biên giới và sử dụng như một phỏng đoán khoảng cách đến hộp giới hạn.

Tìm kiếm hai chiều đôi khi hiệu quả hơn tìm kiếm một chiều, đôi khi không. Nói chung, nếu chúng ta có một phương pháp phỏng đoán rất tốt, thì tìm kiếm A^* tạo ra các đường viền tìm kiếm tập trung vào mục tiêu và việc thêm tìm kiếm hai chiều không giúp ích nhiều. Với phỏng đoán trung bình, tìm kiếm hai chiều gặp nhau ở giữa có xu hướng mở rộng ít nút hơn và được ưu tiên hơn. Trong trường hợp xấu nhất của một heuristic kém, tìm kiếm không còn tập trung vào mục tiêu và tìm kiếm hai chiều có cùng độ phức tạp tiệm cận như A^* . Tìm kiếm hai chiều với chức năng đánh giá f_2 và h phỏng đoán được chấp nhận là hoàn chỉnh và tối ưu.

3.6 Chức năng heuristic

Trong phần này, chúng ta xem xét độ chính xác của heuristic ảnh hưởng như thế nào đến hiệu suất tìm kiếm và cũng xem xét cách heuristic có thể được phát minh. Ví dụ chính của chúng ta, chúng ta sẽ quay trở lại câu đố 8. Như đã đề cập trong Phần 3.2, mục tiêu của câu đố là trượt các ô theo chiều ngang hoặc chiều dọc vào không gian trống cho đến khi cấu hình khớp với cấu hình mục tiêu (Hình 3.25).

Có $9!/2 = 181,400$ trạng thái có thể tiếp cận trong một câu đố 8, vì vậy một tìm kiếm có thể dễ dàng giữ tất cả chúng trong bộ nhớ. Nhưng đối với câu đố 15, có $16!/2$ trạng thái - hơn 10 nghìn tỷ - vì vậy để tìm kiếm không gian đó, chúng ta sẽ cần sự trợ giúp của một hàm phỏng đoán tốt có thể chấp nhận được. Có cùng lịch sử của các phương pháp phỏng đoán như vậy cho câu đố 15; Dưới đây là hai ứng cử viên thường được sử dụng:

- $H1$ = số lượng ô đặt sai vị trí (không bao gồm chỗ trống). Đối với Hình 3.25, tất cả tám ô đều nằm ngoài vị trí, vì vậy trạng thái bắt đầu có $h1 = 8$. $H1$ là một phương pháp phỏng đoán được chấp nhận bởi vì bất kỳ ô nào bị lệch vị trí sẽ yêu cầu ít nhất một lần di chuyển để đưa nó đến đúng vị trí.

7	2	4
5		6
8	3	1

Trạng thái bắt đầu

	1	2
34		5
67		8

Trạng thái mục tiêu

Hình 3.25 Một ví dụ điển hình của câu đố 8. Giải pháp ngắn nhất là 26 hành động dài.

• $h2$ = tổng khoảng cách của các ô từ vị trí mục tiêu của chúng. Bởi vì gạch không thể di chuyển dọc theo đường chéo, khoảng cách là tổng của khoảng cách ngang và dọc — đôi khi được gọi là khoảng cách khối thành phố hoặc khoảng cách Manhattan. $H2$ cũng được chấp nhận

Bởi vì tất cả những gì bất kỳ động thái nào có thể làm là di chuyển một ô gần hơn một bước đến mục tiêu. Các ô từ 1 đến 8 ở trạng thái bắt đầu của Hình 3.25 cho khoảng cách Manhattan là

$$h2 = 3 + 1 + 2 + 2 + 2 + 3 + 3 + 2 = 18.$$

Đúng như dự đoán, cả hai đều không đánh giá quá cao chi phí giải pháp thực sự, tức là 26.

3.6.1 Ảnh hưởng của độ chính xác phỏng đoán đối với hiệu suất

Một cách để mô tả chất lượng của heuristic là yếu tố phân nhánh hiệu quả b^* . Nếu tổng số nút được tạo ra bởi A^* cho một bài toán cụ thể là N và độ sâu dung dịch isd , thì b^* là hệ số phân nhánh mà một cây đồng nhất có độ sâu d sẽ phải có để chứa $N + 1$ nút. Vậy

$N + 1 = 1 + b^* + (b^*)^2 + \dots + (b^*)^d$. Ví dụ: nếu A^* tìm thấy một nghiệm ở độ sâu 5 bằng cách sử dụng 52 nút, thì hệ số phân nhánh hiệu quả là 1,92. Hệ số phân nhánh hiệu quả có thể khác nhau giữa các trường hợp vấn đề, nhưng thông thường đối với một miền cụ thể (chẳng hạn như 8 câu đố), nó khá ổn định trên tất cả các vị trí vấn đề không tầm thường. Do đó, các phép đo thực nghiệm của b^* trên một tập hợp các vấn đề nhỏ có thể cung cấp hướng dẫn tốt về tính hữu ích tổng thể của heuristic. Một heuristic được thiết kế tốt sẽ có giá trị b^* gần 1, cho phép giải quyết các vấn đề khá lớn với chi phí tính toán hợp lý.

Korf và Reid (1998) lập luận rằng một cách tốt hơn để mô tả hiệu ứng của cắt tỉa A^* với một h heuristic nhất định là nó làm giảm độ sâu hiệu quả bằng một kh không đổi so với chiều sâu thực sự. Điều này có nghĩa là tổng chi phí tìm kiếm là $O(bd - kh)$ so với $O(bd)$ đối với tìm kiếm không có thông tin. Các thí nghiệm của họ về khối Rubik và các bài toán n -puzzle cho thấy công thức này đưa ra dự đoán chính xác cho tổng chi phí tìm kiếm cho các trường hợp vấn đề được lấy mẫu trên một loạt các độ dài lời giải — ít nhất là đối với độ dài giải lớn hơn kh .

Đối với Hình 3.26, chúng tôi đã tạo ra các câu đố 8 câu đố ngẫu nhiên và giải chúng bằng tìm kiếm chiều rộng đầu tiên không được định hình và với tìm kiếm A^* sử dụng cả $h1$ và $h2$, báo cáo số lượng nút trung bình được tạo ra và hệ số phân nhánh hiệu quả tương ứng cho mỗi chiến lược tìm kiếm và cho mỗi độ dài giải pháp. Kết quả cho thấy $h2$ tốt hơn $h1$ và cả hai đều tốt hơn là không có heuristic.

Khoảng cách Manhattan

Hệ số phân nhánh hiệu quả

Độ sâu hiệu quả

d	Chi phí tìm kiếm (các nút được tạo)			Hệ số phân nhánh hiệu quả		
	BFS	A*(h1)	Một * (h2)	BFS	A*(h1)	Một * (h2)
6	128	2419		2.01	1.42	1.34
8	368	4831		1.91	1.40	1.30
10	1033	116	48	1.85	1.43	1.27
12	2672	279	84	1.80	1.45	1.28
14	6783	678	174	1.77	1.47	1.31
16	17270	1683	364	1.74	1.48	1.32
18	41558	4102	751	1.72	1.49	1.34
20	91493	9905	1318	1.69	1.50	1.34
22	175921	22955	2548	1.66	1.50	1.34
24	290082	53039	5733	1.62	1.50	1.36
26	395355	110372	10080	1.58	1.50	1.35
28	463234	202565	22055	1.53	1.49	1.36

Hình 3.26 So sánh chi phí tìm kiếm và các yếu tố phân nhánh hiệu quả cho 8 câu đố bằng cách sử dụng tìm kiếm chiều rộng đầu tiên, A* với h1 (ô đặt sai) và A* với h2 (Manhattan distance). Dữ liệu được tính trung bình trên 100 câu đố cho mỗi giải có độ dài d từ 6 đến 28.

Người ta có thể hỏi liệu h2 có luôn tốt hơn h1 hay không. Câu trả lời là "Về cơ bản, có." Thật dễ dàng để thấy từ các định nghĩa của hai heuristics rằng đối với bất kỳ nút n nào, $h2(n) \geq h1(n)$. Do đó, chúng ta nói rằng h2 thống trị h1. Sự thống trị chuyển trực tiếp thành hiệu quả: A* sử dụng h2 Domination sẽ không bao giờ mở rộng nhiều nút hơn A* sử dụng h1 (ngoại trừ trường hợp phá vỡ mối quan hệ không may mắn). Lập luận rất đơn giản. Hãy nhớ lại quan sát ở trang 108 rằng mọi nút có $f(n) < C^*$ chắc chắn sẽ được mở rộng. Điều này cũng giống như nói rằng mọi nút có $h(n) < C^* - g(n)$ chắc chắn được mở rộng khi h nhất quán. Nhưng bởi vì h2 ít nhất lớn bằng h1 cho tất cả các nút, mọi nút chắc chắn được mở rộng bởi A* tìm kiếm với h2 cũng chắc chắn được mở rộng với h1, và h1 cũng có thể khiến các nút khác cũng được mở rộng. Do đó, nói chung tốt hơn là sử dụng hàm aheuristic với các giá trị cao hơn, miễn là nó nhất quán và thời gian tính toán cho heuristic không quá dài.

3.6.2 Tạo heuristic từ các vấn đề thư giãn

Chúng ta đã thấy rằng cả h1 (ô đặt sai) và h2 (khoảng cách Manhattan) đều là những phương pháp khá tốt cho câu đố 8 và h2 tốt hơn. Làm thế nào người ta có thể nghĩ ra h2? Liệu máy tính có thể phát minh ra một phương pháp phỏng đoán như vậy một cách cơ học không?

H1 và H2 là ước tính độ dài đường đi còn lại cho câu đố 8, nhưng chúng cũng là độ dài đường dẫn hoàn toàn chính xác cho các phiên bản đơn giản hóa của câu đố. Nếu các quy tắc của câu đố được thay đổi để một ô có thể di chuyển đến bất cứ đâu thay vì chỉ đến ô trống liền kề, thì h1 sẽ đưa ra độ dài chính xác của giải pháp ngắn nhất. Tương tự, nếu một ô có thể di chuyển một hình vuông theo bất kỳ hướng nào, ngay cả trên một hình vuông bị chiếm đóng, thì h2 sẽ cho độ dài chính xác của giải pháp ngắn nhất. Một vấn đề với ít hạn chế hơn đối với các hành động được gọi là một vấn đề nói lỏng. Biểu đồ không gian trạng thái của bài toán thư giãn là biểu đồ trên của trạng thái ban đầu

Bài toán thư giãn

không gian vì việc loại bỏ các hạn chế tạo ra các cạnh bổ sung trong biểu đồ.

Bởi vì bài toán thư giãn thêm các cạnh vào đồ thị không gian trạng thái, bất kỳ giải pháp tối ưu nào trong bài toán ban đầu, theo định nghĩa, cũng là một giải pháp trong bài toán thư giãn; nhưng thư giãn

vấn đề có thể có giải pháp tốt hơn nếu các cạnh được thêm vào cung cấp các phẩm tất. Do đó, chi phí của một giải pháp tối ưu cho một bài toán thư giãn là một phương pháp phỏng đoán có thể chấp nhận được cho vấn đề ban đầu.

Hơn nữa, bởi vì phỏng đoán dẫn xuất là một chi phí chính xác cho bài toán thư giãn, nó phải tuân theo bất đẳng thức tam giác và do đó nhất quán (xem trang 106).

Nếu một định nghĩa vấn đề được viết ra bằng một ngôn ngữ chính thức, có thể tự động xây dựng các vấn đề thư giãn.¹⁴ Ví dụ, nếu các hành động 8 câu đố được mô tả là

Một ô có thể di chuyển từ hình vuông X sang hình vuông Y nếu X liền kề với Y và Y trống, chúng ta có thể tạo ra ba vấn đề thư giãn bằng cách loại bỏ một hoặc cả hai điều kiện:

(a) Một ô có thể di chuyển từ hình vuông X đến ô vuông Y nếu X liền kề với Y. (b) Một ô có thể di chuyển từ hình vuông X sang hình vuông Y nếu Y trống. (c) Một ô có thể di chuyển từ hình vuông X đến hình vuông Y. Từ (a), chúng ta có thể suy ra h_2 (khoảng cách Manhattan). Lý do là h_2 sẽ là điểm thích hợp nếu chúng ta lần lượt di chuyển từng ô đến đích của nó. Phép phỏng đoán bắt nguồn từ (b) được thảo luận trong Bài tập 3.GASC. Từ (c), chúng ta có thể suy ra h_1 (ô đặt sai) vì nó sẽ là điểm thích hợp nếu các ô có thể di chuyển đến đích dự định của chúng trong một hành động. Lưu ý rằng điều quan trọng là các vấn đề thư giãn được tạo ra bởi kỹ thuật này có thể được giải quyết về cơ bản mà không cần tìm kiếm, bởi vì các quy tắc nói lỏng cho phép vấn đề được phân tách thành tám bài toán con độc lập. Nếu vấn đề thư giãn khó giải quyết, thì các giá trị của heuristic tương ứng sẽ rất tốn kém để có được.

Một chương trình gọi là ABSOLVER có thể tạo ra các phương pháp phỏng đoán tự động từ các xác định vấn đề, sử dụng phương pháp "bài toán thư giãn" và nhiều kỹ thuật khác (Prieditis, 1993). ABSOLVER đã tạo ra một phương pháp phỏng đoán mới cho câu đố 8 tốt hơn bất kỳ phương pháp phỏng đoán nào đã tồn tại trước đó và tìm thấy phương pháp phỏng đoán hữu ích đầu tiên cho câu đố Rubik's Cube nổi tiếng.

Nếu một tập hợp các phương pháp phỏng đoán có thể chấp nhận được $h_1 \dots H_M$ sẵn sàng cho một vấn đề và không có vấn đề nào rõ ràng tốt hơn những vấn đề khác, chúng ta nên chọn cái nào? Hóa ra, chúng ta có thể có những gì tốt nhất của tất cả các thế giới, bằng cách xác định

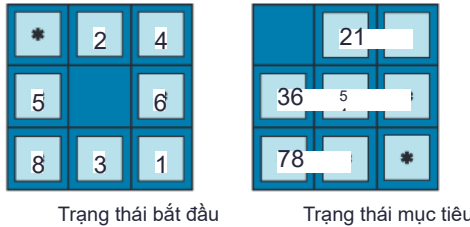
$h(n) = \max\{h_1(n), \dots, h_M(n)\}$. Phỏng đoán tổng hợp này chọn bất kỳ hàm nào chính xác nhất trên nút được đề cập. Bởi vì các thành phần h_i có thể chấp nhận được, h được chấp nhận (và nếu h_i đều nhất quán, h là nhất quán). Hơn nữa, h thống trị tất cả các phỏng đoán thành phần của nó. Hạn chế duy nhất là $h(n)$ mất nhiều thời gian hơn để tính toán. Nếu đó là một vấn đề, một giải pháp thay thế là chọn ngẫu nhiên một trong các heuristic ở mỗi đánh giá hoặc sử dụng thuật toán máy học để dự đoán heuristic nào sẽ tốt nhất. Làm điều này có thể dẫn đến một heuristic không nhất quán (ngay cả khi mọi h_i nhất quán), nhưng trong thực tế, nó thường dẫn đến việc giải quyết vấn đề nhanh hơn.

3.6.3 Tạo heuristic từ các bài toán phụ: Cơ sở dữ liệu mẫu

Vấn đề phụ

Các phương pháp phỏng đoán được chấp nhận cũng có thể được rút ra từ chi phí giải pháp của một vấn đề con của một bài toán. Ví dụ, Hình 3.27 cho thấy một bài toán phụ của trường hợp 8 câu đố trong Hình 3.25. Vấn đề phụ liên quan đến việc đưa các ô 1, 2, 3, 4 và khoảng trống vào vị trí chính xác của chúng. Rõ ràng, chi phí của giải pháp tối ưu của vấn đề con này là giới hạn thấp hơn trên

¹⁴ Trong Chương 8 và 11, chúng tôi mô tả các ngôn ngữ chính thức phù hợp với nhiệm vụ này; Với các mô tả chính thức có thể được thao túng, việc xây dựng các vấn đề thư giãn có thể được tự động hóa. Hiện tại, chúng tôi sử dụng tiếng Anh.



Hình 3.27 Một bài toán con của thực thể 8 câu đố được đưa ra trong Hình 3.25. Nhiệm vụ là đưa các ô 1, 2, 3, 4 và khoảng trống vào đúng vị trí của chúng mà không phải lo lắng về điều gì sẽ xảy ra với các ô khác.

chi phí của vấn đề hoàn chỉnh. Nó hóa ra chính xác hơn khoảng cách Manhattan trong một số trường hợp.

Ý tưởng đằng sau cơ sở dữ liệu mẫu là lưu trữ các chi phí giải pháp chính xác này cho mọi khả năng

Cơ sở dữ liệu mẫu

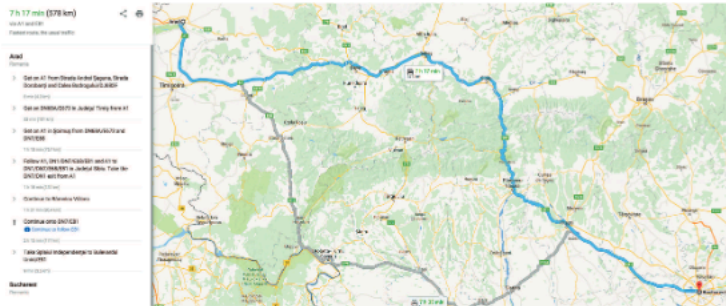
subproblem—trong ví dụ của chúng ta, mọi cấu hình có thể có của bốn ô và theblank. (Sẽ có $9 \times 8 \times 7 \times 6 \times 5 = 15,120$ mẫu trong cơ sở dữ liệu. Danh tính của bốn ô còn lại không liên quan đến mục đích giải quyết vấn đề phụ, nhưng di chuyển của các ô đó được tính vào chi phí giải pháp của bài toán phụ.) Sau đó, chúng tôi tính toán hDB heuristic có thể chấp nhận được cho mỗi trạng thái gặp phải trong quá trình tìm kiếm đơn giản bằng cách tra cứu cấu trúc hình vấn đề con tương ứng trong cơ sở dữ liệu. Bản thân cơ sở dữ liệu được xây dựng bằng cách tìm kiếm lại từ mục tiêu và ghi lại chi phí của mỗi mẫu mới gặp phải; 15 Chi phí của việc tìm kiếm này được khấu hao cho các trường hợp vấn đề tiếp theo, và vì vậy có ý nghĩa nếu chúng ta mong đợi được yêu cầu giải quyết nhiều vấn đề.

Việc lựa chọn các ô 1-2-3-4 để đi kèm với khoảng trống là khá tùy tiện; chúng ta cũng có thể xây dựng cơ sở dữ liệu cho 5-6-7-8, cho 2-4-6-8, v.v. Mỗi cơ sở dữ liệu mang lại một heuristic có thể chấp nhận được và các heuristic này có thể được kết hợp, như đã giải thích trước đó, bằng cách lấy giá trị lớn nhất. Một heuristic kết hợp loại này chính xác hơn nhiều so với khoảng cách Manhattan; Số lượng nút được tạo ra khi giải 15 câu đố ngẫu nhiên có thể được giảm xuống một hệ số

1000. Tuy nhiên, với mỗi cơ sở dữ liệu bổ sung, lợi nhuận giảm dần và chi phí bộ nhớ và tính toán tăng lên. Người ta có thể tự hỏi liệu các phương pháp phỏng đoán thu được từ cơ sở dữ liệu 1-2-3-4 và 5-6-7-8 có thể được thêm vào hay không, vì hai vấn đề phụ dường như không trùng lặp. Điều này có còn mang lại một phỏng đoán có thể chấp nhận được không? Câu trả lời là không, bởi vì các giải pháp của bài toán con 1-2-3-4 và bài toán con 5-6-7-8 cho một trạng thái nhất định gần như chắc chắn sẽ chia sẻ một số động thái — không có khả năng 1-2-3-4 có thể được di chuyển vào vị trí mà không chạm vào 5-6-7-8, và ngược lại. Nhưng điều gì sẽ xảy ra nếu chúng ta không đếm những bước di chuyển đó — điều gì sẽ xảy ra nếu chúng ta không trừu tượng hóa các ô khác thành các ngôi sao, mà thay vào đó làm cho chúng biến mất? Nghĩa là, chúng tôi không ghi lại tổng chi phí để giải bài toán phụ 1-2-3-4, mà chỉ ghi lại số lượng nước đi liên quan đến 1-2-3-4. Sau đó, tổng của hai chi phí vẫn là giới hạn thấp hơn về chi phí giải quyết toàn bộ vấn đề. Đây là ý tưởng đằng sau cơ sở dữ liệu rời rạc. Với cơ sở dữ liệu như vậy, có thể giải 15 câu đố ngẫu nhiên trong một vài câu đố

Cơ sở dữ liệu mẫu rời rạc

15 Bằng cách làm việc ngược lại từ mục tiêu, chi phí giải pháp chính xác của mọi trường hợp gặp phải có sẵn ngay lập tức. Đây là một ví dụ về lập trình động, mà chúng ta thảo luận thêm trong Chương 16.



Hình 3.28 Một dịch vụ Web cung cấp chỉ đường lái xe, được tính toán bằng thuật toán tìm kiếm.

mili giây — số lượng nút được tạo ra giảm 10.000 hệ số so với việc sử dụng khoảng cách Manhattan. Đối với 24 câu đố, có thể đạt được tốc độ khoảng một triệu lần. Cơ sở dữ liệu mẫu rời rạc hoạt động cho các câu đố ô trượt vì vấn đề có thể được phân chia theo cách mà mỗi bước di chuyển chỉ ảnh hưởng đến một vấn đề phụ — bởi vì chỉ có một ô được di chuyển tại một thời điểm.

3.6.4 Tạo phỏng đoán với các mốc

Có những dịch vụ trực tuyến lưu trữ bản đồ với hàng chục triệu đỉnh và tìm hướng lái xe tối ưu về chi phí trong mili giây (Hình 3.28). Làm thế nào họ có thể làm điều đó, khi các thuật toán tìm kiếm tốt nhất mà chúng tôi đã xem xét cho đến nay chậm hơn khoảng một triệu lần? Có rất nhiều thủ thuật, nhưng thủ thuật quan trọng nhất là tính toán trước một số chi phí đường dẫn tối ưu. Mặc dù tính toán trước có thể tốn thời gian, nó chỉ cần được thực hiện một lần và sau đó có thể được khấu hao trên hàng tỷ yêu cầu tìm kiếm của người dùng.

Chúng ta có thể tạo ra một heuristic hoàn hảo bằng cách tính toán trước và lưu trữ chi phí của đường dẫn tối ưu giữa mỗi cặp đỉnh. Điều đó sẽ mất $O(|V|^2)$ không gian và $O(|E|^3)$ thời gian — thực tế đối với đồ thị có 10 nghìn đỉnh, nhưng không phải 10 triệu.

Một cách tiếp cận tốt hơn là chọn một vài (có thể là 10 hoặc 20) điểm mốc từ Đỉnh. Sau đó, đối với mỗi cột mốc L và cho mỗi đỉnh v khác trong đồ thị, chúng ta tính toán và lưu trữ $C^*(v, L)$, chi phí chính xác của đường dẫn tối ưu từ v đến L . (Chúng ta cũng cần $C^*(L, v)$; trên đồ thị không có hướng, điều này giống như $C^*(v, L)$; Trên một đồ thị có hướng - ví dụ, với đường một chiều - chúng ta cần tính toán điều này một cách riêng biệt.) Với các bảng C^* được lưu trữ, chúng ta có thể dễ dàng tạo ra một phỏng đoán hiệu quả (mặc dù không thể chấp nhận được): tối thiểu, trên tất cả các mốc mốc, chi phí đi từ nút hiện tại đến mốc và sau đó đến mục tiêu:

$$hL(n) = \min_{L \in \text{Landmarks}} C^*(n, L) + C^*(L, \text{mục tiêu})$$

Nếu con đường tối ưu tình cờ đi qua một cột mốc, heuristic này sẽ chính xác; nếu không, nó không thể chấp nhận được - nó đánh giá quá cao chi phí cho mục tiêu. Trong tìm kiếm A^* , nếu bạn có exactheuristics, thì một khi bạn đạt được một nút nằm trên con đường tối ưu, mọi nút bạn sẽ mở rộng

16 Điểm mốc đôi khi được gọi là "trục" hoặc "neo".

Từ đó trở đi sẽ đi trên con đường tối ưu. Hãy nghĩ về các đường viền như đi theo con đường tối ưu này. Tìm kiếm sẽ theo dõi dọc theo đường dẫn tối ưu, trên mỗi lần lặp thêm một hành động với chi phí c để đạt được trạng thái kết quả có giá trị h sẽ nhỏ hơn c , có nghĩa là tổng $f = g + h$ score sẽ không đổi ở C^* dọc theo đường dẫn.

Một số thuật toán tìm tuyến đường thậm chí còn tiết kiệm nhiều thời gian hơn bằng cách thêm các phẩm tất — các Phẩm tất

trong biểu đồ xác định đường dẫn đa hành động tối ưu. Ví dụ, nếu có các lối tắt được xác định trước giữa tất cả 100 thành phố lớn nhất ở Hoa Kỳ và chúng tôi đang cố gắng điều hướng từ khuôn viên Berkeley ở California đến NYU ở New York, chúng tôi có thể đi tắt giữa Sacramento và Manhattan và bao phủ 90% con đường trong một hành động. hiệu quả nhưng không được chấp nhận. Nhưng với một chút cẩn thận, chúng ta có thể đưa ra aheuristic vừa hiệu quả vừa có thể chấp nhận được:

$$hDH(n) = \max_{L \in \text{Địa danh}} |C^*(n, L) - C^*(\text{mục tiêu}, L)|$$

Đây được gọi là phương pháp phỏng đoán vi sai (vi phép trừ). Hãy nghĩ về điều này với một cột mốc ở đâu đó ngoài mục tiêu. Nếu mục tiêu tình cờ nằm trên con đường tối ưu từ n đến cột mốc, thì điều này nói rằng "hãy xem xét toàn bộ con đường từ n đến L , sau đó trừ đi phần cuối cùng của con đường đó, từ mục tiêu đến L , cho chúng ta chi phí chính xác của con đường từ n đến mục tiêu." Trong phạm vi mục tiêu hơi lệch khỏi con đường tối ưu đến cột mốc, heuristic sẽ không chính xác, nhưng vẫn có thể chấp nhận được. Các mốc không nằm ngoài mục tiêu sẽ không hữu ích; một mốc nằm chính xác giữa n và mục tiêu sẽ cho $hDH = 0$, điều này không hữu ích. Một số cách để chọn các điểm mốc. Chọn điểm ngẫu nhiên rất nhanh, nhưng chúng ta sẽ nhận được kết quả tốt hơn nếu chúng ta cẩn thận trải các mốc ra ngoài để chúng không quá gần nhau. Một cách tiếp cận tham lam là chọn một mốc đầu tiên một cách ngẫu nhiên, sau đó tìm điểm xa nhất từ điểm đó và thêm nó vào tập hợp các mốc và tiếp tục, ở mỗi lần lặp lại, thêm điểm tối đa hóa khoảng cách đến cột mốc gần nhất. Nếu bạn có nhật ký về các yêu cầu tìm kiếm trước đây của người dùng, thì bạn có thể chọn các mốc được yêu cầu thường xuyên trong các tìm kiếm. Đối với phỏng đoán vi sai, sẽ tốt nếu các mốc được trải xung quanh chu vi của biểu đồ. Do đó, một kỹ thuật tốt là tìm tâm của đồ thị, sắp xếp các nêm hình tròn xung quanh tâm và trong mỗi nêm chọn đỉnh xa tâm nhất.

Các địa danh hoạt động đặc biệt tốt trong các vấn đề tìm đường vì cách đường được bố trí trên thế giới: rất nhiều phương tiện giao thông thực sự muốn di chuyển giữa các địa danh, vì vậy các kỹ sư dân dụng xây dựng những con đường rộng nhất và nhanh nhất dọc theo các tuyến đường này; Tìm kiếm cột mốc giúp khôi phục các tuyến đường này dễ dàng hơn.

3.6.5 Học cách tìm kiếm tốt hơn

Chúng tôi đã trình bày một số chiến lược tìm kiếm cố định - chiều rộng đầu tiên, A^* , v.v. - đã được các nhà khoa học máy tính thiết kế và lập trình cẩn thận. Một nhân viên có thể học cách tìm kiếm tốt hơn không? Câu trả lời là có, và phương thức dựa trên một khái niệm quan trọng được gọi là không gian trạng thái siêu cấp. Mỗi trạng thái trong không gian trạng thái metalevel nằm bất nội bộ (computational) của một chương trình đang tìm kiếm trong một không gian trạng thái thông thường như bản đồ của Romania. (Để giữ cho hai khái niệm riêng biệt, chúng tôi gọi bản đồ Romania là một không gian trạng thái cấp đối tượng.) Ví dụ: trạng thái bên trong của thuật toán A^* bao gồm tìm kiếm hiện tại cây. Mỗi hành động trong không gian trạng thái metalevel là một bước tính toán thay đổi nội bộ

Không gian trạng thái metalevel

Không gian trạng thái cấp đối tượng

tiểu bang; ví dụ: mỗi bước tính toán trong A* mở rộng một nút lá và thêm các nút kế thừa của nó vào cây. Do đó, Hình 3.18, cho thấy một chuỗi các cây tìm kiếm ngày càng lớn hơn, có thể được coi là mô tả một đường dẫn trong không gian trạng thái metalevel trong đó mỗi trạng thái trên đường dẫn là một cây tìm kiếm cấp đối tượng.

Học metalevel

Hãy giả sử, con đường trong Hình 3.18 có năm bước, bao gồm một bước, sự mở rộng của Fagaras, điều đó không đặc biệt hữu ích. Đối với các vấn đề khó hơn, sẽ có nhiều sai lầm như vậy và thuật toán học ở cấp độ khác có thể học hỏi từ những kinh nghiệm này để tránh khám phá những điều không hứa hẹn-
ing cây phụ. Các kỹ thuật được sử dụng cho loại hình học tập này được mô tả trong Chương 23. Mục tiêu của việc học là giảm thiểu tổng chi phí giải quyết vấn đề, đánh đổi chi phí tính toán và chi phí đường dẫn.

3.6.6 Học heuristic từ kinh nghiệm

Chúng ta đã thấy rằng một cách để phát minh ra một phương pháp phỏng đoán là nghĩ ra một vấn đề thoải mái mà một giải pháp tối ưu có thể được tìm thấy một cách dễ dàng. Một giải pháp thay thế là học hỏi từ kinh nghiệm. Ví dụ, "Kinh nghiệm" ở đây có nghĩa là giải rất nhiều câu đố 8. Mỗi giải pháp tối ưu cho một câu đố 8 cung cấp một cặp ví dụ (mục tiêu, đường dẫn). Từ những ví dụ này, một thuật toán học có thể được sử dụng để xây dựng một hàm h có thể (nếu may mắn) ước tính chi phí đường dẫn thực cho các trạng thái khác phát sinh trong quá trình tìm kiếm. Hầu hết các cách tiếp cận này học được một xấp xỉ không hoàn hảo đối với hàm heuristic, và do đó có nguy cơ không thể chấp nhận được. Điều này dẫn đến sự đánh đổi không thể tránh khỏi giữa thời gian học tập, thời gian chạy tìm kiếm và chi phí giải pháp. Các kỹ thuật học máy được trình bày trong Chương 19. Các phương pháp học tăng cường được mô tả trong Chương 23 cũng có thể áp dụng cho tìm kiếm.

Tính năng

Một số kỹ thuật học máy hoạt động tốt hơn khi được cung cấp các tính năng của một trạng thái có liên quan đến việc dự đoán giá trị heuristic của trạng thái, chứ không phải chỉ với mô tả trạng thái thô. Ví dụ: tính năng "số ô đặt sai" có thể hữu ích trong việc xác định trước khoảng cách thực tế của trạng thái 8 câu đố từ mục tiêu. Hãy gọi tính năng này là $x_1(n)$. Chúng tôi có thể lấy 100 cấu hình 8 câu đố được tạo ngẫu nhiên và thu thập số liệu thống kê về chi phí giải pháp thực tế của chúng. Chúng ta có thể thấy rằng khi $x_1(n)$ là 5, chi phí giải pháp trung bình là khoảng 14, v.v. Tất nhiên, chúng ta có thể sử dụng nhiều tính năng. Tính năng thứ hai $x_2(n)$ có thể là "số cặp ô liền kề không liền kề trong trạng thái mục tiêu". $x_1(n)$ và $x_2(n)$ nên được kết hợp như thế nào để dự đoán $h(n)$? Một cách tiếp cận phổ biến là sử dụng kết hợp tuyến tính:

$h(n) = c_1 x_1(n) + c_2 x_2(n)$. Các hằng số c_1 và c_2 được điều chỉnh để phù hợp nhất với dữ liệu thực tế trên các cấu hình được tạo ngẫu nhiên. Người ta mong đợi cả c_1 và c_2 đều dương vì các ô đặt sai vị trí và các cặp liền kề không chính xác khiến vấn đề khó giải quyết hơn. Lưu ý rằng heuristic này đáp ứng điều kiện $h(n) = 0$ cho các trạng thái mục tiêu, nhưng nó không nhất thiết phải được chấp nhận hoặc nhất quán.

Tóm tắt

Chương này đã giới thiệu các thuật toán tìm kiếm mà một tác nhân có thể sử dụng để chọn chuỗi hành động trong nhiều môi trường khác nhau — miễn là chúng là từng đợt, tác nhân đơn, hoàn toàn có thể quan sát, xác định, tĩnh, rời rạc và hoàn toàn được biết đến. Có những sự đánh đổi được thực hiện giữa lượng thời gian tìm kiếm, dung lượng bộ nhớ có sẵn và chất lượng của giải pháp. Chúng ta có thể hiệu quả hơn nếu chúng ta có kiến thức phụ thuộc vào lĩnh vực trong

dạng của một hàm phỏng đoán ước tính khoảng cách của một trạng thái nhất định so với mục tiêu hoặc nếu chúng ta tính toán trước các giải pháp từng phần liên quan đến các mẫu hoặc mốc.

- Trước khi một tác nhân có thể bắt đầu tìm kiếm, một vấn đề được xác định rõ ràng phải được xây dựng.
- Một vấn đề bao gồm năm phần: trạng thái ban đầu, một tập hợp các hành động, một mô hình chuyển tiếp mô tả kết quả của các hành động đó, một tập hợp các trạng thái mục tiêu và một hàm chi phí hành động.

Môi trường của vấn đề được biểu thị bằng biểu đồ không gian trạng thái. Một đường dẫn xuyên qua không gian trạng thái (một chuỗi các hành động) từ trạng thái ban đầu đến trạng thái mục tiêu là một giải pháp.

- Các thuật toán tìm kiếm thường coi các trạng thái và hành động là nguyên tử, không có bất kỳ cấu trúc nội bộ nào (mặc dù chúng tôi đã giới thiệu các tính năng của trạng thái khi đến lúc học tập).

- Các thuật toán tìm kiếm được đánh giá trên cơ sở tính đầy đủ, tối ưu chi phí, tính phức tạp về thời gian và độ phức tạp của không gian.

• Các phương pháp tìm kiếm thiếu thông tin chỉ có quyền truy cập vào định nghĩa vấn đề. Thuật toán xây dựng một cây tìm kiếm để cố gắng tìm ra giải pháp. Các thuật toán khác nhau dựa trên nút mà chúng mở rộng trước:— Tìm kiếm tốt nhất đầu tiên chọn các nút để mở rộng bằng cách sử dụng chức năng đánh giá.— Tìm kiếm chiều rộng đầu tiên mở rộng các nút nông nhất trước; nó hoàn chỉnh, tối ưu cho chi phí hành động đơn vị, nhưng có độ phức tạp không gian theo cấp số nhân.— Tìm kiếm chi phí đồng nhất mở rộng nút với chi phí đường dẫn thấp nhất, $g(n)$ và tối ưu cho chi phí hành động chung.— Tìm kiếm độ sâu đầu tiên mở rộng nút chưa mở rộng sâu nhất trước. Nó không hoàn chỉnh cũng không tối ưu, nhưng có độ phức tạp không gian tuyến tính. Tìm kiếm giới hạn độ sâu bổ sung giới hạn độ sâu.— Tìm kiếm sâu lặp đi lặp lại gọi tìm kiếm độ sâu đầu tiên với giới hạn độ sâu tăng lên cho đến khi tìm thấy mục tiêu. Nó hoàn thành khi kiểm tra chu kỳ đầy đủ được thực hiện, chi phí hành động đơn vị tối ưu, có độ phức tạp thời gian tương đương với tìm kiếm chiều rộng đầu tiên và có độ phức tạp không gian tuyến tính.— Tìm kiếm hai chiều mở rộng hai biên giới, một xung quanh trạng thái ban đầu và một xung quanh mục tiêu, dừng lại khi hai biên giới gặp nhau.

- Các phương pháp tìm kiếm được thông tin có quyền truy cập vào một hàm phỏng đoán $h(n)$ ước tính chi phí của một giải pháp từ n . Họ có thể có quyền truy cập vào thông tin bổ sung như cơ sở dữ liệu mẫu với chi phí giải pháp.— Tìm kiếm tốt nhất đầu tiên mở rộng các nút với $h(n)$ tối thiểu. Nó không phải là tối ưu nhưng thường hiệu quả.— A^* tìm kiếm mở rộng các nút với $f(n) = g(n) + h(n)$ tối thiểu. A^* là hoàn chỉnh và tối ưu, miễn là $h(n)$ được chấp nhận. Độ phức tạp không gian của A^* vẫn là một vấn đề cho nhiều vấn đề.— Tìm kiếm hai chiều A^* đôi khi hiệu quả hơn chính A^* .— IDA* (tìm kiếm A^* sâu lặp đi lặp lại) là một phiên bản đào sâu lặp đi lặp lại của A^* , và do đó giải quyết vấn đề độ phức tạp của không gian.— RBFS (tìm kiếm đệ quy tốt nhất đầu tiên) và SMA* (A giới hạn bộ nhớ đơn giản hóa*) là các thuật toán tìm kiếm mạnh mẽ, tối ưu sử dụng lượng bộ nhớ hạn chế; nếu có đủ thời gian, họ có thể giải quyết các vấn đề mà A^* hết bộ nhớ.

– Tìm kiếm chùm tia đặt giới hạn về kích thước của biên giới; điều đó làm cho nó không đầy đủ và không tối ưu, nhưng nó thường tìm thấy các giải pháp hợp lý tốt và chạy nhanh hơn tìm kiếm hoàn chỉnh. – Tìm kiếm có trọng số A^* tập trung tìm kiếm hướng tới một mục tiêu, mở rộng ít nút hơn, nhưng hy sinh tính tối ưu.

- Hiệu suất của các thuật toán tìm kiếm heuristic phụ thuộc vào chất lượng của chức năng heuristic. Đôi khi người ta có thể xây dựng các phương pháp phỏng đoán tốt bằng cách nói lỏng định nghĩa vấn đề, bằng cách lưu trữ chi phí giải pháp được tính toán trước cho các vấn đề phụ trong cơ sở dữ liệu mẫu, bằng cách xác định các mốc hoặc bằng cách học hỏi từ kinh nghiệm với lớp vấn đề.

Ghi chú thư mục và lịch sử

Chủ đề tìm kiếm không gian nhà nước bắt nguồn từ những năm đầu của AI. Nghiên cứu của Newell và Simon về Nhà lý thuyết logic (1957) và GPS (1961) đã dẫn đến việc thiết lập các thuật toán tìm kiếm như là công cụ chính cho các nhà nghiên cứu AI những năm 1960 và thiết lập giải quyết vấn đề như một nhiệm vụ AI kinh điển. Nghiên cứu hoạt động của Richard Bellman (1957) cho thấy tầm quan trọng của chi phí đường dẫn cộng trong việc đơn giản hóa các thuật toán tối ưu hóa. Văn bản của Nils Nilsson (1971) đã thiết lập khu vực trên một nền tảng lý thuyết vững chắc.

Câu đố 8 là một người anh em họ nhỏ hơn của câu đố 15, có lịch sử được kể lại bởi Slocum và Sonneveld (2006). Năm 1880, câu đố 15 đã thu hút sự chú ý rộng rãi từ công chúng và các nhà toán học (Johnson và Story, 1879; Tait, 1880). Các biên tập viên của Tạp chí Toán học Hoa Kỳ tuyên bố, "Câu đố '15' trong vài tuần qua đã nổi bật trước công chúng Mỹ, và có thể nói một cách an toàn là đã thu hút sự chú ý của chính trong số mười người ở cả hai giới và mọi lứa tuổi và điều kiện của cộng đồng," trong khi Weekly News-Democrat của Emporia, Kansas viết vào ngày 12 tháng 3, Năm 1880 rằng "Nó đã trở thành một dịch bệnh trên toàn quốc."

Nhà thiết kế trò chơi nổi tiếng người Mỹ Sam Loyd tuyên bố sai sự thật là người đã phát minh ra câu đố 15 (Loyd, 1959); thực ra nó được phát minh bởi Noyes Chapman, một giám đốc bưu điện ở Canastota, New York, vào giữa những năm 1870 (mặc dù một bằng sáng chế chung bao gồm các khối trượt đã được cấp cho Ernest Kinsey vào năm 1878). Ratner và Warmuth (1986) đã chỉ ra rằng phiên bản chung $n \times n$ của 15 câu đố thuộc lớp bài toán hoàn thành NP.

Khối Rubik tất nhiên được phát minh vào năm 1974 bởi Ernő Rubik, người cũng đã phát hiện ra một thuật toán để tìm ra các giải pháp tốt, nhưng không phải là tối ưu. Korf (1997) đã tìm ra các giải pháp tối ưu cho một số trường hợp vấn đề ngẫu nhiên bằng cách sử dụng cơ sở dữ liệu mẫu và tìm kiếm $*IDA$. Rokicki et al. (2014) đã chứng minh rằng bất kỳ trường hợp nào cũng có thể được giải quyết trong 26 nước đi (nếu bạn coi một vòng xoắn 180° là hai nước đi; 20 nếu nó được tính là một). Bằng chứng đã tiêu tốn 35 năm tính toán CPU; nó không dẫn đến ngay lập tức một thuật toán hiệu quả. Agostinelli et al. (2019) đã sử dụng học tăng cường, mạng học sâu và tìm kiếm cây Monte Carlo để tìm hiểu một trình giải hiệu quả hơn nhiều cho khối Rubik. Nó không được đảm bảo để tìm ra một giải pháp tối ưu về chi phí, nhưng làm như vậy khoảng 60% thời gian và thời gian giải pháp điển hình là chưa đầy một giây.

Mỗi vấn đề tìm kiếm trong thế giới thực được liệt kê trong chương này đều là chủ đề của rất nhiều nỗ lực nghiên cứu. Các phương pháp để lựa chọn các chuyển bay hàng không tối ưu vẫn là một thách thức trong hầu hết các phần, nhưng Carl de Marcken đã chỉ ra bằng cách giảm bớt các vấn đề quyết định của Diophantine rằng giá vé máy bay và các hạn chế đã trở nên phức tạp đến mức khả năng-

lem của việc lựa chọn một chuyến bay tối ưu là không thể quyết định chính thức (Robinson, 2002). Bài toán nhân viên bán hàng du lịch (TSP) là một bài toán tổ hợp tiêu chuẩn trong khoa học máy tính lý thuyết (Lawler et al., 1992). Karp (1972) đã chứng minh bài toán quyết định TSP là NP-hard, nhưng các phương pháp xấp xỉ heuristic hiệu quả đã được phát triển (Lin và Kernighan, 1973). Arora (1998) đã nghĩ ra một sơ đồ xấp xỉ đa thức hoàn toàn cho các TSP Euclid. Các phương pháp bố trí VLSI được khảo sát bởi LaPaugh (2010), và nhiều bài báo tối ưu hóa bố cục xuất hiện trong các tạp chí VLSI. Điều hướng bằng robot được thảo luận trong Chương 26. Tự động lắp ráp tự động lần đầu tiên được chứng minh bởi FREDDY (Michie, 1972); một đánh giá toàn diện được đưa ra bởi (Bahubalendruni và Biswal, 2016).

Các thuật toán tìm kiếm thiếu thông tin là một chủ đề trung tâm của khoa học máy tính (Cormen et al., 2009) và nghiên cứu hoạt động (Dreyfus, 1969). Tìm kiếm chiều rộng đầu tiên được xây dựng cho các mê cung giải quyết bởi Moore (1959). Phương pháp lập trình động (Bellman, 1957; Bellman và Dreyfus, 1962), ghi lại các giải pháp một cách có hệ thống cho tất cả các vấn đề phụ có độ dài tăng dần, có thể được coi là một hình thức tìm kiếm chiều rộng đầu tiên.

Thuật toán của Dijkstra ở dạng nó thường được trình bày (Dijkstra, 1959) có thể áp dụng cho đồ thị hữu hạn rõ ràng. Nilsson (1971) đã giới thiệu một phiên bản thuật toán của Dijkstra mà ông gọi là tìm kiếm chi phí đồng nhất (bởi vì thuật toán "trải rộng dọc theo các đường viền có chi phí đường đi bằng nhau") cho phép các đồ thị vô hạn, được xác định ngầm. Công trình của Nilsson cũng giới thiệu ý tưởng về danh sách đóng và mở, và thuật ngữ "tìm kiếm đồ thị". Cái tên BEST-FIRST-SEARCH đã được giới thiệu trong Sổ tay AI (Barr và Feigenbaum, 1981). Floyd-Warshall (Floyd, 1962) và Bellman-Ford (Bellman, 1958; Ford, 1956) cho phép chi phí bước âm (miễn là không có chu kỳ âm).

Một phiên bản đào sâu lặp đi lặp lại được thiết kế để sử dụng hiệu quả đồng hồ cờ vua lần đầu tiên được sử dụng lần đầu tiên bởi Slate và Atkin (1977) trong chương trình chơi cờ CHESS 4.5. Thuật toán của MartelliB (1977) cũng bao gồm một khía cạnh đào sâu lặp đi lặp lại. Kỹ thuật đào sâu lặp đi lặp lại được giới thiệu bởi Bertram Raphael (1976) và nổi lên hàng đầu trong công trình của Korf (1985a).

Việc sử dụng thông tin heuristic trong giải quyết vấn đề xuất hiện trong một bài báo đầu tiên của Simon và Newell (1958), nhưng cụm từ "tìm kiếm heuristic" và việc sử dụng các hàm heuristic ước tính khoảng cách đến mục tiêu đến muộn hơn một chút (Newell và Ernst, 1965; Lin, 1965). Doran và Michie (1966) đã tiến hành các nghiên cứu thực nghiệm sâu rộng về tìm kiếm phỏng đoán. Mặc dù họ đã phân tích độ dài đường dẫn và "độ thâm nhập" (tỷ lệ giữa chiều dài đường dẫn với tổng số nút được kiểm tra cho đến nay), họ dường như đã bỏ qua thông tin được cung cấp bởi chi phí đường dẫn $g(n)$. Thuật toán A^* , kết hợp chi phí đường dẫn hiện tại vào tìm kiếm phỏng đoán, được phát triển bởi Hart, Nilsson và Raphael (1968). Dechter và Pearl (1985) đã nghiên cứu các điều kiện trong đó A^* có hiệu quả tối ưu (về số lượng nút được mở rộng). Bài báo A^* ban đầu (Hart et al., 1968) đã giới thiệu điều kiện nhất quán trên các hàm heuristic. Điều kiện đơn điệu được Pohl (1977) giới thiệu như một sự thay thế đơn giản hơn, nhưng Pearl (1984) cho thấy rằng cả hai là tương đương.

Pohl (1977) đi tiên phong trong việc nghiên cứu mối quan hệ giữa lỗi trong các chức năng heuristic và độ phức tạp về thời gian của A^* . Kết quả cơ bản thu được cho tìm kiếm giống cây với chi phí đơn vị và một trạng thái mục tiêu duy nhất (Pohl, 1977; Gaschnig, 1979; Huyn và cộng sự, 1980; Pearl, 1984) và với nhiều trạng thái mục tiêu (Đinh và cộng sự, 2007). Korf và Reid (1998) đã chỉ ra cách dự đoán chính xác số lượng nút mở rộng (không chỉ là xấp xỉ tiệm cận) trên nhiều miền vấn đề thực tế. "Hệ số phân nhánh hiệu quả" được Nils-son (1971) đề xuất như một thước đo thực nghiệm về hiệu quả. Đối với tìm kiếm đồ thị, Helmert và Röger (2008)

lưu ý rằng một số vấn đề nổi tiếng chứa nhiều nút theo cấp số nhân trên các đường dẫn giải pháp chi phí tối ưu, ngụ ý độ phức tạp thời gian theo cấp số nhân đối với A^* .

Có nhiều biến thể trên thuật toán A^* . Pohl (1970) giới thiệu A^* -search có trọng số, và sau đó là phiên bản động (1973), trong đó trọng lượng thay đổi theo độ sâu của cây. Ebenadt và Drechsler (2009) tổng hợp kết quả và kiểm tra một số ứng dụng. Hatem và Ruml (2014) cho thấy một phiên bản đơn giản hóa và cải tiến của A^* có trọng số* để thực hiện hơn. Wilt và Ruml (2014) giới thiệu tìm kiếm tốc độ như một giải pháp thay thế cho tìm kiếm tham lam tập trung vào việc giảm thiểu thời gian tìm kiếm, và Wilt và Ruml (2016) chỉ ra rằng các phương pháp phỏng đoán tốt nhất để tìm kiếm thỏa mãn khác với các phương pháp cho tìm kiếm tối ưu. Burnset al. (2012) đưa ra một số thủ thuật triển khai để viết mã tìm kiếm nhanh và Felner (2018) xem xét cách triển khai thay đổi khi sử dụng kiểm tra mục tiêu sớm.

Pohl (1971) giới thiệu tìm kiếm hai chiều. Holte et al. (2016) mô tả phiên bản tìm kiếm hai chiều được đảm bảo đáp ứng ở giữa, làm cho nó có thể ứng dụng rộng rãi hơn. Eckerle et al. (2017) mô tả tập hợp các cặp nút mở rộng chắc chắn và cho thấy rằng không có tìm kiếm hai chiều nào có thể hiệu quả tối ưu. Thuật toán NBS (Chen và cộng sự, 2017) sử dụng rõ ràng hàng đợi các cặp nút.

Sự kết hợp của A^* hai chiều và các mốc đã biết đã được sử dụng để tìm các tuyến đường lái xe hiệu quả cho dịch vụ bản đồ trực tuyến của Microsoft (Goldberg et al., 2006). Sau khi lưu trữ các đường dẫn giữa các mốc, thuật toán có thể tìm thấy một đường dẫn chi phí tối ưu giữa bất kỳ cặp điểm nào trong biểu đồ 24 triệu điểm của Hoa Kỳ, tìm kiếm ít hơn 0,1% biểu đồ. Korf (1987) chỉ ra cách sử dụng các mục tiêu phụ, toán tử ví mô và trừu tượng hóa để đạt được tốc độ đáng kể so với các kỹ thuật trước đây. Dellling et al. (2009) mô tả cách sử dụng tìm kiếm hai chiều, mốc, cấu trúc phân cấp và các thủ thuật khác để tìm các tuyến đường lái xe. Anderson et al. (2008) mô tả một kỹ thuật liên quan, được gọi là tìm kiếm từ thô đến mịn,

Tìm kiếm từ thô đến mịn

có thể được coi là xác định các mốc ở các cấp độ trừu tượng phân cấp khác nhau. Korf (1987) mô tả các điều kiện trong đó tìm kiếm từ thô đến mịn cung cấp tốc độ theo cấp số nhân. Knoblock (1991) cung cấp các kết quả thí nghiệm và phân tích để định lượng các lợi thế của tìm kiếm phân cấp.

Nhánh và ràng buộc

A^* và các thuật toán tìm kiếm không gian trạng thái khác có liên quan chặt chẽ đến nhánh và ràng buộc

các kỹ thuật được sử dụng rộng rãi trong nghiên cứu hoạt động (Lawler và Wood, 1966; Rayward-Smith và cộng sự, 1996). Kumar và Kanai (1988) cố gắng "thống nhất lớn" của tìm kiếm phỏng đoán, lập trình động và các kỹ thuật nhánh và ràng buộc dưới tên CDP - "quá trình quyết định tổng hợp".

Bởi vì hầu hết các máy tính trong những năm 1960 chỉ có vài nghìn từ bộ nhớ chính, tìm kiếm phỏng đoán giới hạn bộ nhớ là một chủ đề nghiên cứu ban đầu. Graph Traverser (Doranand Michie, 1966), một trong những chương trình tìm kiếm sớm nhất, cam kết thực hiện một hành động sau khi tìm kiếm tốt nhất trước cho đến giới hạn bộ nhớ. IDA* (Korf, 1985b) là thuật toán tìm kiếm phỏng đoán có giới hạn bộ nhớ, tối ưu chiều dài đầu tiên được sử dụng rộng rãi và một số lượng lớn các biến thể đã được phát triển. Một phân tích về hiệu quả của IDA* và những khó khăn của nó với các giá trị thực xuất hiện trong Patrick et al. (1992).

Phiên bản gốc của RBFS (Korf, 1993) thực sự phức tạp hơn một chút so với thuật toán được hiển thị trong Hình 3.22, thực sự gần với một thuật toán được phát triển độc lập được gọi là mở rộng lặp lại hoặc IE (Russell, 1992). RBFS sử dụng giới hạn dưới như cũng như giới hạn trên; hai thuật toán hoạt động giống hệt nhau với phỏng đoán được chấp nhận, nhưng RBFS mở rộng các nút theo thứ tự tốt nhất ngay cả với một phỏng đoán không thể chấp nhận được. Ý tưởng về

Mở rộng lặp đi lặp lại

theo dõi con đường thay thế tốt nhất đã xuất hiện trước đó trong Prologimplementation tao nhâ của Bratko (2009) về A^* và trong thuật toán DTA* (Russell và Wefald, 1991). Tác phẩm thứ hai cũng thảo luận về không gian trạng thái metalevel và học metalevel. Thuật toán MA^* xuất hiện trong Chakrabarti et al. (1989). SMA^* , hoặc MA^* đơn giản, xuất hiện từ một nỗ lực để thực hiện MA^* (Russell, 1992). Kaïndl và Khorsand (1994) đã áp dụng SMA^* để tạo ra một thuật toán tìm kiếm hai chiều nhanh hơn đáng kể so với các thuật toán trước đó. Korf và Zhang (2000) mô tả cách tiếp cận phân chia và chinh phục, và Zhou và Hansen (2002) giới thiệu tìm kiếm đồ thị A^* giới hạn bộ nhớ và chiến lược chuyển sang tìm kiếm chiều rộng đầu tiên để tăng hiệu quả bộ nhớ (Zhou và Hansen, 2006).

Tương tự cho rằng các phương pháp phỏng đoán có thể được chấp nhận có thể được rút ra bằng cách thư giãn vấn đề xuất hiện trong bài báo này của Held và Karp (1970), những người đã sử dụng phương pháp phỏng đoán cây kéo dài tối thiểu để giải TSP. (Xem Bài tập 3.MSTR.) Việc tự động hóa quá trình thư giãn đã được thực hiện thành công bởi Frieditis (1993). Ngày càng có nhiều tài liệu về ứng dụng học máy để khám phá các chức năng heuristic (Samadi và cộng sự, 2008; Arfaee và cộng sự, 2010; Thayer và cộng sự, 2011; Lelis và cộng sự, 2012).

Việc sử dụng cơ sở dữ liệu mẫu để rút ra các phương pháp phỏng đoán được chấp nhận là do Gasser (1995) và Culberson và Schaeffer (1996, 1998); cơ sở dữ liệu mẫu rời rạc được mô tả bởi Korf và Felner (2002); một phương pháp tương tự sử dụng các mẫu biểu tượng là do Edelkamp (2009). Felner et al. (2007) chỉ ra cách nén cơ sở dữ liệu mẫu để tiết kiệm dung lượng. Việc giải thích xác suất của phỏng đoán đã được nghiên cứu bởi Pearl (1984) và Hansson và Mayer (1989).

Pearl (1984) Heuristics và Edelkamp và Schrödl's (2012) Heuristic Search là những cuốn sách giáo khoa có ảnh hưởng về tìm kiếm. Các bài báo về các thuật toán tìm kiếm mới xuất hiện tại Hội nghị chuyên đề quốc tế về Tìm kiếm Kết hợp (SoCS) và Hội nghị Quốc tế về Lập kế hoạch và Lập kế hoạch Tự động (ICAPS), cũng như trong các hội nghị AI nói chung như AAAI và IJCAI, và các tạp chí như Trí tuệ nhân tạo và Tạp chí ACM.

CHƯƠNG 4

TÌM KIẾM TRONG MÔI TRƯỜNG PHỨC TẠP

Trong đó chúng ta nói lòng các giả định đơn giản hóa của chương trước, để đến gần hơn với thế giới đó.

Chương 3 đề cập đến các vấn đề trong môi trường hoàn toàn có thể quan sát được, xác định, tĩnh, đã biết, nơi giải pháp là một chuỗi các hành động. Trong chương này, chúng ta nói lòng những ràng buộc đó. Chúng ta bắt đầu với vấn đề tìm kiếm một trạng thái tốt mà không phải lo lắng về con đường để đạt được điều đó, bao gồm cả trạng thái rời rạc (Phần 4.1) và liên tục (Phần 4.2). Sau đó, chúng ta nói lòng các giả định về quyết định luận (Phần 4.3) và khả năng quan sát (Phần 4.4). Trong một thế giới không xác định, tác nhân sẽ cần một kế hoạch có điều kiện và thực hiện các hành động khác nhau tùy thuộc vào những gì nó quan sát được - ví dụ, dừng lại nếu đèn màu đỏ và đi nếu nó có màu xanh lá cây. Với khả năng quan sát một phần, tác nhân cũng sẽ cần theo dõi các trạng thái có thể xảy ra. Cuối cùng, Phần 4.5 hướng dẫn tác nhân qua một không gian chưa biết mà nó phải học khi nó đi, sử dụng tìm kiếm trực tuyến.

4.1 Vấn đề tìm kiếm và tối ưu hóa cục bộ

Trong các bài toán tìm kiếm của Chương 3, chúng tôi muốn tìm các con đường xuyên qua không gian tìm kiếm, chẳng hạn như một con đường từ Arad đến Bucharest. Nhưng đôi khi chúng ta chỉ quan tâm đến trạng thái cuối cùng, không quan tâm đến con đường để đạt được điều đó. Ví dụ, trong bài toán 8 ong chúa (Hình 4.3), chúng ta chỉ quan tâm đến việc tìm một cấu hình cuối cùng hợp lệ của 8 nữ hoàng (bởi vì nếu bạn biết cấu hình, nó rất tầm thường để xây dựng lại các bước đã tạo ra nó). Điều này cũng đúng với nhiều ứng dụng quan trọng như thiết kế mạch tích hợp, bố trí sàn nhà máy, lập lịch xưởng, lập trình tự động, tối ưu hóa mạng viễn thông, lập kế hoạch cây trồng và quản lý danh mục đầu tư.

Tìm kiếm địa phương

Các thuật toán tìm kiếm cục bộ hoạt động bằng cách tìm kiếm từ trạng thái bắt đầu đến các trạng thái lân cận,

mà không theo dõi các con đường, cũng như tập hợp các trạng thái đã đạt được. Điều đó có nghĩa là chúng không có hệ thống - chúng có thể không bao giờ khám phá một phần của không gian tìm kiếm nơi giải pháp thực sự cư trú. Tuy nhiên, chúng có hai ưu điểm chính: (1) chúng sử dụng rất ít bộ nhớ; và (2) họ thường có thể tìm thấy các giải pháp hợp lý trong không gian trạng thái lớn hoặc vô hạn mà các thuật toán hệ thống không phù hợp. Các thuật toán tìm kiếm cục bộ cũng có thể giải quyết các vấn đề tối ưu hóa, trong đó mục đích là

Vấn đề tối ưu hóa

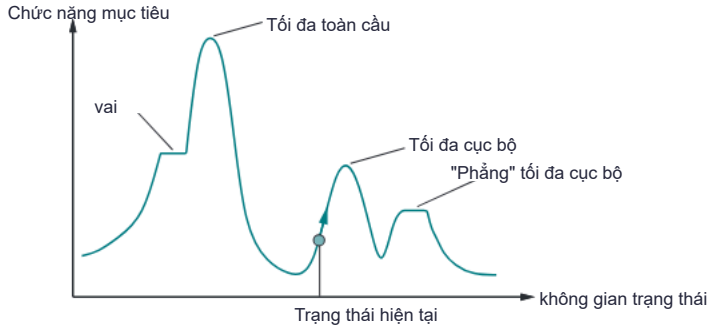
Chức năng mục tiêu

Tìm trạng thái tốt nhất theo một hàm khách

Cảnh quan không gian nhà nước

để giúp tìm kiếm cục bộ, hãy xem xét các trạng thái của một vấn đề được đặt ra trong bối cảnh không gian trạng thái, như thể hiện trong Hình 4.1. Mỗi điểm (trạng thái) trong cảnh quan có một "độ cao", de-

bị phạt bởi giá trị của hàm mục tiêu. Nếu độ cao tương ứng với một hàm mục tiêu,



Hình 4.1 Cảnh quan trạng thái-không gian một chiều trong đó độ cao tương ứng với hàm khách quan. Mục đích là tìm ra mức tối đa toàn cầu.

hàm HILL-CLIMBING(problem) trả về một trạng thái là mức tối đa cục bộ
vấn đề $\leftarrow$ hiện tại. INITIAL trong khi true làm

neighbor $\leftarrow$ trạng thái kế thừa có giá trị cao nhất của current if VALUE
(neighbor) $\leq$ VALUE (current) sau đó trả về current current $\leftarrow$ neighbor

Hình 4.2 Thuật toán tìm kiếm leo đồi, là công nghệ tìm kiếm cục bộ cơ bản nhất. Ở mỗi bước, nút hiện tại được thay thế bằng hàng xóm tốt nhất.

Sau đó, mục đích là tìm ra đỉnh cao nhất - một cực đại toàn cầu - và chúng ta gọi là quá trình leo đồi. Nếu độ cao tương ứng với chi phí, thì mục đích là tìm thung lũng thấp nhất - mức tối thiểu toàn cầu - và chúng tôi gọi nó là gradient descent.

Tối đa toàn cầu

Tối thiểu toàn cầu

4.1.1 Tìm kiếm leo đồi

Thuật toán tìm kiếm leo đồi được thể hiện trong Hình 4.2. Nó theo dõi một trạng thái hiện tại và trên mỗi lần lặp lại di chuyển đến trạng thái lân cận với giá trị cao nhất — nghĩa là, nó đi theo hướng cung cấp độ dốc nhất. Nó kết thúc khi nó đạt đến "đỉnh" trong đó

Leo đồi

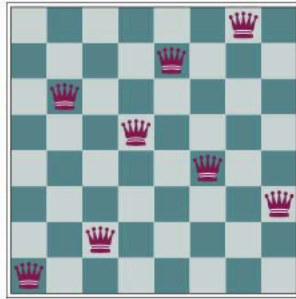
Leo dốc nhất

không có hàng xóm nào có giá trị cao hơn. Leo đồi không nhìn về phía trước vượt ra ngoài những người hàng xóm trực tiếp của trạng thái hiện tại. Điều này giống như cố gắng tìm đỉnh Everest trong sương mù dày đặc trong khi bị mất trí nhớ. Lưu ý rằng một cách để sử dụng tìm kiếm leo đồi là sử dụng số âm của hàm chi phí heuristic làm hàm mục tiêu; điều đó sẽ leo lên cục bộ đến tiểu bang có khoảng cách heuristic nhỏ nhất đến mục tiêu.

Để minh họa cho việc leo đồi, chúng ta sẽ sử dụng bài toán 8 chúa (Hình 4.3). Chúng ta sẽ sử dụng một công thức trạng thái hoàn chỉnh, có nghĩa là mọi trạng thái đều có tất cả các thành phần của một

Công thức trạng thái hoàn chỉnh

giải pháp, nhưng có thể không phải tất cả chúng đều ở đúng chỗ. Trong trường hợp này, mọi tiểu bang đều có 8 nữ hoàng



(a)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	13	16	13	16	
13	14	17	15	13	14	16	16
17	13	16	18	15	13	15	13
18	14	13	15	15	14	13	16
14	14	13	17	12	14	12	18

(b)

Hình 4.3 (a) Bài toán 8 nữ hoàng: đặt 8 quân hậu trên bàn cờ để không có quân hậu nào tấn công một quân khác. (Một nữ hoàng tấn công bất kỳ quân cờ nào trong cùng một hàng, cột hoặc đường chéo.) Vị trí này gần như là một giải pháp, ngoại trừ hai nữ hoàng ở cột thứ tư và thứ bảy tấn công nhau dọc theo đường chéo. (b) Một trạng thái 8 queens với ước tính chi phí heuristic = 17. Bảng hiển thị giá trị của h cho mỗi người kế thừa có thể có được bằng cách di chuyển một nữ hoàng trong cột của nó. Có 8 nước đi được xếp hạng tốt nhất, với $h = 12$. Thuật toán leo đồi sẽ chọn một trong số này.

trên bảng, mỗi cột. Trạng thái ban đầu được chọn ngẫu nhiên và các trạng thái kế thừa của trạng thái là tất cả các trạng thái có thể được tạo ra bằng cách di chuyển một nữ hoàng đến một ô vuông khác trong cùng một cột (vì vậy mỗi trạng thái có $8 \times 7 = 56$ trạng thái kế tiếp). Hàm chi phí heuristic h là số cặp ong chúa đang tấn công lẫn nhau; Điều này sẽ chỉ bằng 0 đối với các giải pháp. (Nó được tính là một cuộc tấn công nếu hai quân cờ nằm trong cùng một dòng, ngay cả khi có một quân cờ xen kẽ giữa chúng.) Hình 4.3 (b) cho thấy một trạng thái có $h = 17$. Hình này cũng cho thấy giá trị h của tất cả những người kế nhiệm.

Tìm kiếm địa phương tham Leo đồi đôi khi được gọi là tìm kiếm địa phương tham lam vì nó chiếm được một người hàng xóm tốt

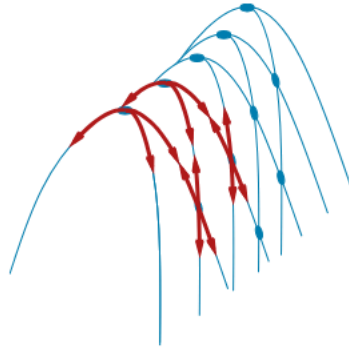
trạng thái mà không suy nghĩ trước về nơi tiếp theo. Mặc dù tham lam được coi là một trong bảy tội lỗi chết người, nhưng hóa ra các thuật toán tham lam thường hoạt động khá tốt. Leo đồi có thể tiến bộ nhanh chóng hướng tới một giải pháp vì nó thường khá dễ dàng để cải thiện một trạng thái xấu. Ví dụ, từ trạng thái trong Hình 4.3 (b), chỉ mất năm bước để đạt được trạng thái trong Hình 4.3 (a), có $h = 1$ và rất gần là một lời giải. Thật không may, leo đồi có thể bị mắc kẹt vì bất kỳ lý do nào sau đây:

Tối đa cục bộ

- **Tối đa cục bộ:** Mức tối đa cục bộ là một đỉnh cao hơn mỗi đỉnh lân cận của nó nhưng thấp hơn mức tối đa toàn cầu. Các thuật toán leo đồi đạt đến vùng lân cận của mức tối đa cục bộ sẽ được kéo lên trên về phía đỉnh nhưng sau đó sẽ bị mắc kẹt mà không có nơi nào khác để đi. Hình 4.1 minh họa vấn đề một cách sơ đồ. Cụ thể hơn, trạng thái trong Hình 4.3 (a) là mức tối đa cục bộ (tức là mức tối thiểu cục bộ cho chi phí h); Mỗi động thái của một nữ hoàng duy nhất đều làm cho tình hình trở nên tồi tệ hơn.

Nóc

- **Ridges:** Một sườn núi được thể hiện trong Hình 4.4. Các đường gờ dẫn đến một chuỗi cực đại cục bộ. Điều đó rất khó đối với các thuật toán tham lam để điều hướng.



Hình 4.4 Minh họa lý do tại sao các rặng núi gây khó khăn cho việc leo đồi. Lưới các trạng thái (quảng tối) được chồng lên một sườn núi mọc lên từ trái sang phải, tạo ra một dãy cực đại cục bộ không được kết nối trực tiếp với nhau. Từ mỗi mức tối đa cục bộ, tất cả các hành động có sẵn đều hướng xuống dốc. Các cấu trúc liên kết như thế này phổ biến trong các không gian trạng thái chiều thấp, chẳng hạn như các điểm trong mặt phẳng hai chiều. Nhưng trong không gian nhà nước với hàng trăm hoặc hàng ngàn chiều, bức tranh trực quan này không giữ được, và thường có ít nhất một vài chiều giúp nó có thể thoát khỏi các rặng núi và cao nguyên.

• Cao nguyên: Cao nguyên là một khu vực bằng phẳng của cảnh quan không gian nhà nước. Nó có thể là một địa

Cao nguyên

tối đa, từ đó không tồn tại lối ra lên dốc, hoặc một vai, từ đó tiến bộ

Vai

Có thể. (Xem Hình 4.1.) Một cuộc tìm kiếm leo đồi có thể bị lạc khi lang thang trên cao nguyên. Trong mỗi trường hợp, thuật toán đạt đến điểm mà tại đó không có tiến bộ nào được thực hiện. Bắt đầu từ trạng thái 8 nữ hoàng được tạo ngẫu nhiên, leo đồi dốc nhất bị mắc kẹt 86% thời gian, chỉ giải quyết 14% trường hợp có vấn đề. Mặt khác, nó hoạt động nhanh chóng, chỉ thực hiện trung bình 4 bước khi thành công và 3 bước khi gặp khó khăn - không tệ đối với một không gian tiểu bang với $88 \approx 17$ triệu tiểu bang.

Làm thế nào chúng ta có thể giải quyết nhiều vấn đề hơn? Một câu trả lời là tiếp tục đi khi chúng ta đạt đến cao nguyên - cho phép đi chuyển sang một bên với hy vọng rằng cao nguyên thực sự là một vai, như đã hiển thị

Di chuyển ngang

trong Hình 4.1. Nhưng nếu chúng ta thực sự ở mức tối đa cục bộ bằng phẳng, thì cách tiếp cận này sẽ lang thang trên cao nguyên mãi mãi. Do đó, chúng ta có thể giới hạn số lần di chuyển đi ngang liên tiếp, dừng lại sau 100 lần đi ngang liên tiếp. Điều này nâng tỷ lệ các trường hợp vấn đề được giải quyết bằng cách leo đồi từ 14% lên 94%. Thành công đi kèm với cái giá: thuật toán trung bình khoảng 21 bước cho mỗi phiên bản thành công và 64 bước cho mỗi lần thất bại.

Nhiều biến thể của leo đồi đã được phát minh. Leo đồi ngẫu nhiên chọn tại ngẫu nhiên trong số các động tác lên dốc; Xác suất lựa chọn có thể thay đổi theo độ dốc của động tác lên dốc. Điều này thường hội tụ chậm hơn so với leo dốc nhất, nhưng ở một số cảnh quan tiểu bang, nó tìm ra các giải pháp tốt hơn. Dụng cụ leo đồi ngẫu nhiên được lựa chọn hàng đầu

Leo đồi ngẫu nhiên

leo đồi bằng cách tạo ra những người kế nhiệm một cách ngẫu nhiên cho đến khi một người được tạo ra tốt hơn trạng thái hiện tại. Đây là một chiến lược tốt khi một quốc gia có nhiều (ví dụ: hàng nghìn) người kế nhiệm.

Lựa chọn đầu tiên leo đồi

Một biến thể khác là leo đồi khởi động lại ngẫu nhiên, áp dụng câu ngạn ngữ, "Nếu lúc đầu bạn Đứng thành công, hãy thử, thử lại." Nó tiến hành một loạt các tìm kiếm leo đồi ngẫu nhiên

Leo đồi khởi động lại ngẫu nhiên

tạo trạng thái ban đầu, cho đến khi tìm thấy mục tiêu. Nó hoàn thành với xác suất 1, bởi vì cuối cùng nó sẽ tạo ra trạng thái mục tiêu như trạng thái ban đầu. Nếu mỗi lần tìm kiếm leo đồi có xác suất p thành công, thì số lần khởi động lại dự kiến cần thiết là $1/p$. Đối với các trường hợp 8 queen không được phép di chuyển ngang, $p \approx 0,14$, vì vậy chúng ta cần khoảng 7 lần lặp lại để tìm mục tiêu (6 thất bại và 1 thành công). Số bước dự kiến là chi phí của một lần lặp lại thành công cộng với $(1 - p)/p$ nhân với chi phí thất bại, hoặc khoảng 22 bước trong tất cả. Khi chúng ta cho phép di chuyển ngang, trung bình cần $1 / 0,94 \approx 1,06$ lần lặp lại và $(1 \times 21) + (0,06 / 0,94) \times 64 \approx 25$ bước. Đối với 8 chúa, leo đồi khởi động lại ngẫu nhiên thực sự rất hiệu quả. Ngay cả đối với ba triệu con chúa, cách tiếp cận này có thể tìm ra giải pháp trong vài giây.¹

Sự thành công của việc leo đồi phụ thuộc rất nhiều vào hình dạng của cảnh quan đất đai không gian nhà nước: nếu có ít cực đại và cao nguyên cục bộ, leo đồi khởi động lại ngẫu nhiên sẽ tìm ra giải pháp tốt rất nhanh. Mặt khác, nhiều vấn đề thực tế có cảnh quan giống như một gia đình nhím hói rải rác trên sàn phẳng, với những con nhím thu nhỏ sống trên đầu mỗi cây kim nhím. Các vấn đề cứng NP (xem Phụ lục A) thường có một số lượng cực đại cục bộ theo cấp số nhân để mắc kẹt. Mặc dù vậy, mức tối đa cục bộ tốt hợp lý thường có thể được tìm thấy sau một số lần khởi động lại.

4.1.2 Ủ mô phỏng

Một thuật toán leo đồi không bao giờ thực hiện các động thái "xuống dốc" về phía các tiểu bang có giá trị thấp hơn (hoặc chi phí cao hơn) luôn dễ bị mắc kẹt trong mức tối đa cục bộ. Ngược lại, một bước đi hoàn toàn ngẫu nhiên chuyển sang trạng thái kế thừa mà không quan tâm đến giá trị cuối cùng sẽ vấp phải mức tối đa toàn cầu, nhưng có vẻ kỳ diệu hiệu quả. Do đó, có vẻ hợp lý khi cố gắng kết hợp leo đồi với đi bộ ngẫu nhiên theo cách mang lại cả hiệu quả và hoàn chỉnh.

Ủ mô phỏng

Ủ mô phỏng là một thuật toán như vậy. Trong luyện kim, ủ là quy trình được sử dụng để ủ hoặc làm cứng kim loại và thủy tinh bằng cách nung nóng chúng đến nhiệt độ cao và sau đó làm nguội dần chúng, do đó cho phép vật liệu đạt đến trạng thái tinh thể năng lượng thấp. Để giải thích ủ mô phỏng, chúng tôi chuyển quan điểm của mình từ leo đồi sang xuống dốc (tức là giảm thiểu chi phí) và tưởng tượng nhiệm vụ đưa một quả bóng bàn vào kẻ hở sâu nhất trong bề mặt rất gập ghềnh. Nếu chúng ta chỉ để quả bóng lăn, nó sẽ dừng lại ở mức tối thiểu cục bộ. Nếu chúng ta lắc bề mặt, chúng ta có thể nảy quả bóng ra khỏi mức tối thiểu cục bộ — có thể vào mức tối thiểu cục bộ sâu hơn, nơi nó sẽ dành nhiều thời gian hơn. Bí quyết là lắc vừa đủ mạnh để nảy bóng ra khỏi mức tối thiểu cục bộ nhưng không đủ mạnh để đánh bật nó khỏi mini-mum toàn cầu. Dung dịch ủ mô phỏng là bắt đầu bằng cách lắc mạnh (tức là ở nhiệt độ cao) và sau đó giảm dần cường độ lắc (tức là hạ nhiệt độ).

Cấu trúc tổng thể của thuật toán ủ mô phỏng (Hình 4.5) tương tự như leo đồi. Tuy nhiên, thay vì chọn nước đi tốt nhất, nó chọn một nước đi ngẫu nhiên. Nếu động thái cải thiện tình hình, nó luôn được chấp nhận. Nếu không, thuật toán chấp nhận di chuyển với một số xác suất nhỏ hơn 1. Xác suất giảm theo cấp số nhân với "độ xấu" của nước đi - số lượng ΔE mà đánh giá trở nên tồi tệ hơn. Xác suất cũng giảm khi "nhiệt độ" T giảm xuống: các nước đi "xấu" có nhiều khả năng được cho phép khi bắt đầu khi T cao và chúng trở nên khó xảy ra hơn khi T giảm. Nếu lịch trình hạ T xuống 0 đủ chậm, thì một thuộc tính của phân phối Boltzmann, $e^{\Delta E/T}$, là

¹ Luby et al. (1993) đề xuất khởi động lại sau một số bước cố định và chỉ ra rằng điều này có thể hiệu quả hơn nhiều so với việc để mỗi tìm kiếm tiếp tục vô thời hạn.

hàm SIMULATED-ANNEALING(problem, schedule) trả về trạng thái giải pháp
vấn đề $\leftarrow$ hiện tại. INITIAL cho $t = 1$ đến ∞ do

$T \leftarrow$ lịch trình (t) nếu $T = 0$ thì trả về hiện
tại tiếp theo $\leftarrow$ một người kế thừa được
chọn ngẫu nhiên của hiện tại $\Delta E \leftarrow$ VALUE
(hiện tại) - VALUE (tiếp theo) nếu $\Delta E > 0$
thì đồng điện $\leftarrow$ tiếp theo chỉ $\leftarrow$ tiếp theo
với xác suất $e^{\Delta E / T}$

Hình 4.5 Thuật toán ủ mô phỏng, một phiên bản leo đồi ngẫu nhiên cho phép một số di chuyển xuống dốc. Đầu vào lịch trình xác định giá trị của "nhiệt độ" T như một hàm của thời gian.

Tất cả xác suất tập trung vào cực đại toàn cầu, mà thuật toán sẽ tìm thấy với xác suất gần 1.

Ủ mô phỏng được sử dụng để giải quyết các vấn đề bố trí VLSI bắt đầu từ những năm 1980. Nó đã được áp dụng rộng rãi cho lập lịch trình nhà máy và các nhiệm vụ tối ưu hóa quy mô lớn khác.

4.1.3 Tìm kiếm chùm tia cục bộ

Chỉ giữ một nút trong bộ nhớ có vẻ như là một phản ứng cực đoan đối với vấn đề giới hạn bộ nhớ. Thuật toán tìm kiếm chùm tia cục bộ theo dõi các trạng thái k thay vì

Tìm kiếm chùm tia cục bộ

chỉ một. Nó bắt đầu với k trạng thái được tạo ngẫu nhiên. Ở mỗi bước, tất cả các kế thừa của tất cả các k state được tạo ra. Nếu bất kỳ mục tiêu nào là mục tiêu, thuật toán sẽ dừng lại. Nếu không, nó chọn k người kế nhiệm tốt nhất từ danh sách đầy đủ và lặp lại.

Thoạt nhìn, tìm kiếm chùm tia cục bộ với k trạng thái có vẻ không gì khác hơn là chạy k khởi động lại ngẫu nhiên song song thay vì theo trình tự. Trên thực tế, hai thuật toán khá khác nhau. Trong tìm kiếm khởi động lại ngẫu nhiên, mỗi quá trình tìm kiếm chạy độc lập với các quy trình khác. Trong tìm kiếm chùm tia cục bộ, thông tin hữu ích được truyền giữa các Jthread tìm kiếm song song. Trên thực tế, các quốc gia tạo ra những người kế nhiệm tốt nhất nói với những người khác, "Hãy đến

ở đây, cô xanh hơn!" Thuật toán nhanh chóng từ bỏ các tìm kiếm không hiệu quả và di chuyển tài nguyên đến nơi đang đạt được nhiều tiến bộ nhất.

Tìm kiếm chùm tia cục bộ có thể bị thiếu sự đa dạng giữa các tiểu bang k - chúng có thể tập trung trong một khu vực nhỏ của không gian tiểu bang, làm cho việc tìm kiếm chậm hơn một chút so với phiên bản leo đồi chậm hơn. Một biến thể được gọi là tìm kiếm chùm tia ngẫu nhiên, analo-

Tìm kiếm chùm tia ngẫu nhiên

Đi đến Stochastic Hill Climbing, giúp giảm bớt vấn đề này. Thay vì chọn những người kế nhiệm hàng đầu, tìm kiếm chùm tia ngẫu nhiên chọn những người kế nhiệm với xác suất tỷ lệ thuận với giá trị của người kế nhiệm, do đó làm tăng sự đa dạng.

4.1.4 Thuật toán tiến hóa

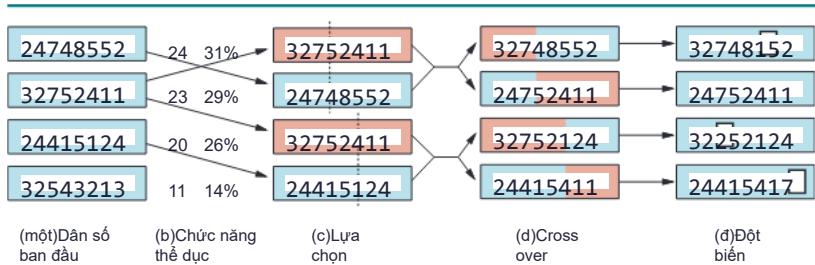
Các thuật toán tiến hóa có thể được coi là các biến thể của tìm kiếm chùm tia ngẫu nhiên rõ ràng

Thuật toán tiến hóa

Được thúc đẩy bởi phép ẩn dụ về chọn lọc tự nhiên trong sinh học: có một quần thể các cá nhân (trạng thái), trong đó các cá thể phù hợp nhất (giá trị cao nhất) tạo ra con cái (trạng thái kế thừa) tạo ra thế hệ tiếp theo, một quá trình được gọi là tái tổ hợp. Có vô số hình thức

Tái tổ hợp

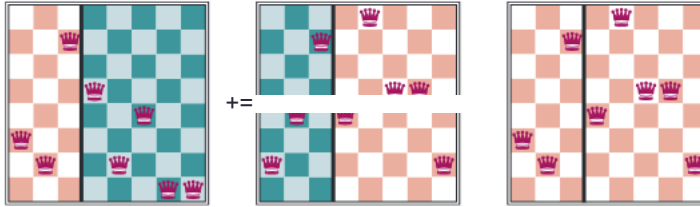
của các thuật toán tiến hóa, thay đổi theo những cách sau:



Hình 4.6 Một thuật toán di truyền, được minh họa cho các chuỗi chữ số đại diện cho các trạng thái 8 chữ. Quần thể ban đầu trong (a) được xếp hạng theo hàm phù hợp trong (b) dẫn đến các cặp để giao phối trong (c). Chúng sinh ra con cái trong (d), có thể bị đột biến trong (e).

Thuật toán di truyền	Quy mô dân số.
Chiến lược phát triển Lập trình di truyền	- Đại diện của từng cá nhân. Trong các thuật toán di truyền, mỗi cá nhân là một chuỗi trên một bảng chữ cái hữu hạn (thường là một chuỗi Boolean), giống như DNA là một chuỗi trên alphabet ACGT. Trong các chiến lược tiến hóa, một cá nhân là một dãy các số thực, và trong Lập trình di truyền một cá nhân là một chương trình máy tính. - Số trộn, p, là số bố mẹ kết hợp với nhau để hình thành con cái. Trường hợp phổ biến nhất là $p = 2$: hai bố mẹ kết hợp "gen" của chúng (các phần đại diện của chúng) để tạo thành con cái. Khi $p = 1$, chúng ta có tìm kiếm chùm ngẫu nhiên (có thể được coi là sinh sản vô tính). Có thể có $p > 2$, điều này hiếm khi xảy ra trong tự nhiên nhưng đủ để dùng để mô phỏng trên máy tính.
Lựa chọn	Quy trình lựa chọn các cá nhân sẽ trở thành cha mẹ của thế hệ tiếp theo: Một khả năng là chọn từ tất cả các cá nhân có xác suất theo tỷ lệ phù hợp với điểm thể lực của họ. Một khả năng khác là chọn ngẫu nhiên n cá thể ($n > p$), sau đó chọn p phù hợp nhất làm cha mẹ.
Điểm giao nhau	Quy trình tái tổ hợp. Một cách tiếp cận phổ biến (giả sử $p = 2$), là chọn ngẫu nhiên một điểm giao nhau để tách từng chuỗi cha và kết hợp lại các phần thành tạo thành hai con, một với phần đầu tiên của cha mẹ 1 và phần thứ hai của cha mẹ 2; phần còn lại với phần thứ hai của cha mẹ 1 và phần đầu tiên của cha mẹ 2.
Tỷ lệ đột biến	Tỷ lệ đột biến, xác định tần suất con cái có đột biến ngẫu nhiên đại diện của họ. Một khi một con cái đã được tạo ra, mọi bit trong thành phần của nó đều bị lật với xác suất bằng với tỷ lệ đột biến.
Chủ nghĩa tinh hoa	Trang điểm của thế hệ tiếp theo. Đây có thể chỉ là những đứa con mới hình thành, hoặc nó có thể bao gồm một vài bậc cha mẹ đạt điểm cao nhất từ thế hệ trước (một thực hành được gọi là chủ nghĩa tinh hoa, đảm bảo rằng thế lực tổng thể sẽ không bao giờ giảm theo thời gian). Thực hành-

loại bỏ, trong đó tất cả các cá thể dưới một ngưỡng nhất định đều bị loại bỏ, có thể dẫn đến tăng tốc độ (Baum et al., 1995). Hình 4.6 (a) cho thấy một quần thể gồm bốn chuỗi 8 chữ số, mỗi chuỗi đại diện cho một trạng thái của câu đố 8 nữ hoàng: chữ số thứ c đại diện cho số hàng của nữ hoàng trong cột c . Trong (b), mỗi trạng thái được đánh giá bởi hàm thể dục. Giá trị thể dục cao hơn là tốt hơn, vì vậy đối với 8 nữ hoàng



Hình 4.7 Các trạng thái 8 ong chứa tương ứng với hai bộ mẹ đầu tiên trong Hình 4.6 (c) và con cái đầu tiên trong Hình 4.6 (d). Các cột màu xanh lá cây bị mất trong bước giao nhau và các cột ở đó được giữ lại. (Để giải thích các con số trong Hình 4.6: hàng 1 là hàng dưới cùng và 8 là hàng trên cùng.)

Vấn đề chúng ta sử dụng số cặp ong chứa không tấn công, có giá trị là $8 \times 7/2 = 28$ cho một lời giải. Giá trị của bốn trạng thái trong (b) là 24, 23, 20 và 11. Điểm độ phù hợp sau đó được chuẩn hóa thành xác suất và các giá trị kết quả được hiển thị bên cạnh các giá trị phù hợp trong (b).

Trong (c), hai cặp cha mẹ được chọn, phù hợp với xác suất trong (b). Lưu ý rằng một cá nhân được chọn hai lần và một cá nhân hoàn toàn không. Đối với mỗi cặp đã chọn, một điểm giao nhau (đường chấm) được chọn ngẫu nhiên. Trong (d), chúng ta vượt qua các dây mẹ tại các điểm giao nhau, tạo ra con cái mới. Ví dụ: con đầu tiên của cặp đầu tiên nhận được ba chữ số đầu tiên (327) từ cha mẹ đầu tiên và các chữ số còn lại (48552) từ cha thứ hai. Các trạng thái 8 nữ hoàng liên quan đến bước tái tổ hợp này được thể hiện trong Hình 4.7.

Cuối cùng, trong (e), mỗi vị trí trong mỗi chuỗi có thể bị đột biến ngẫu nhiên với xác suất độc lập nhỏ. Một chữ số đã bị đột biến ở con thứ nhất, thứ ba và thứ tư. Trong bài toán 8 nữ hoàng, điều này tương ứng với việc chọn một nữ hoàng một cách ngẫu nhiên và di chuyển nó đến một hình vuông ngẫu nhiên trong cột của nó. Thường là trường hợp quần thể đa dạng ngay từ đầu trong quá trình, vì vậy sự giao thoa thường thực hiện các bước lớn trong không gian trạng thái sớm trong quá trình tìm kiếm (như ủ mô phỏng). Sau nhiều thế hệ lựa chọn hướng tới khả năng phù hợp cao hơn, dân số trở nên ít đa dạng hơn và các bước nhỏ hơn là điển hình. Hình 4.8 mô tả một thuật toán thực hiện tất cả các bước này.

Các thuật toán di truyền tương tự như tìm kiếm chùm tia ngẫu nhiên, nhưng có thêm hoạt động chéo. Điều này có lợi nếu có các khối thực hiện các chức năng hữu ích. Ví dụ, có thể đặt ba nữ hoàng đầu tiên ở vị trí 2, 4 và 6 (nơi chúng không tấn công nhau) tạo thành một khối hữu ích có thể được kết hợp với các khối hữu ích khác xuất hiện ở các cá nhân khác để xây dựng một giải pháp. Nó có thể được chứng minh bằng toán học rằng, nếu các khối không phục vụ một mục đích - ví dụ nếu các vị trí của mã di truyền được hoán vị ngẫu nhiên - thì sự giao nhau không mang lại lợi thế.

Lý thuyết về các thuật toán di truyền giải thích cách thức hoạt động của điều này bằng cách sử dụng ý tưởng về một Schema là một chuỗi con trong đó một số vị trí có thể không được xác định. Ví dụ, lược đồ 246***** mô tả tất cả các trạng thái 8 chúa trong đó ba nữ hoàng đầu tiên lần lượt là các vị trí 2, 4 và 6. Các chuỗi khớp với lược đồ (chẳng hạn như 24613578) được gọi là các phiên bản của lược đồ. Có thể chỉ ra rằng nếu mức độ phù hợp trung bình của các trường hợp

Schema

Ví dụ

Một lược đồ cao hơn giá trị trung bình, thì số lượng thực thể của lược đồ sẽ tăng lên theo thời gian.

Tiến hóa và tìm kiếm

Lý thuyết tiến hóa được phát triển bởi Charles Darwin trong *On the Origin of Species by Means of Natural Selection* (1859) và độc lập bởi Alfred Russel Wallace (1858). Ý tưởng trung tâm rất đơn giản: các biến thể xảy ra trong sinh sản và sẽ được bảo tồn trong các thế hệ kế tiếp xấp xỉ tỷ lệ với ảnh hưởng của chúng đối với sức khỏe sinh sản.

Lý thuyết của Darwin được phát triển mà không có kiến thức về cách các đặc điểm của chủ nghĩa cơ quan có thể được kế thừa và sửa đổi. Các quy luật xác suất chi phối các pro-cess này lần đầu tiên được xác định bởi Gregor Mendel (1866), một tu sĩ đã thử nghiệm với đậu ngọt. Rất lâu sau đó, Watson và Crick (1953) đã xác định cấu trúc của phân tử DNA và bảng chữ cái của nó, AGTC (adenine, guanine, thymine, cytosine). Trong mô hình tiêu chuẩn, sự biến đổi xảy ra cả bởi các đột biến điểm trong trình tự chữ cái và bởi "sự giao thoa" (trong đó DNA của con cái được tạo ra bằng cách kết hợp các phần dài của DNA từ mỗi cha mẹ).

Sự tương tự với các thuật toán tìm kiếm cục bộ đã được mô tả; Sự khác biệt chính giữa tìm kiếm chùm tia ngẫu nhiên và tiến hóa là việc sử dụng sinh sản hữu tính, trong đó những người kế thừa được tạo ra từ nhiều cá nhân chứ không chỉ một. Tuy nhiên, các cơ chế tiến hóa thực sự phong phú hơn nhiều so với hầu hết các thuật toán di truyền cho phép. Ví dụ, đột biến có thể liên quan đến sự đảo ngược, duplications và chuyển động của các khối DNA lớn; một số virus mượn DNA từ một sinh vật và chèn nó vào một sinh vật khác; và có những gen có thể chuyển đổi không làm gì ngoài việc sao chép hàng ngàn lần trong bộ gen.

Thậm chí còn có những gen đầu độc tế bào từ những người bạn tình tiềm năng không mang gen, do đó làm tăng cơ hội sao chép của chính chúng. Quan trọng nhất là thực tế là bản thân các gen mã hóa các cơ chế mà theo đó bộ gen được tái tạo và dịch thành một sinh vật. Trong các thuật toán di truyền, các cơ chế đó là một chương trình riêng biệt không được biểu diễn trong các chuỗi được thao tác.

Sự tiến hóa của Darwin có thể tỏ ra không hiệu quả, đã tạo ra một cách mù quáng một số sinh vật hoặc lâu hơn mà không cải thiện một chút phỏng đoán tìm kiếm của nó. Nhưng học tập đóng một vai trò trong quá trình tiến hóa. Mặc dù nhà tự nhiên học vĩ đại người Pháp Jean Lamarck (1809) đã sai khi đề xuất rằng các đặc điểm có được bằng cách thích nghi trong suốt cuộc đời của một sinh vật sẽ được truyền lại cho con cái của nó, lý thuyết tương tự bề ngoài của James Baldwin (1896) là đúng: học tập có thể thư giãn hiệu quả bối cảnh phù hợp, dẫn đến sự tăng tốc trong tốc độ tiến hóa. Một sinh vật có đặc điểm không hoàn toàn thích nghi với môi trường của nó sẽ truyền lại đặc điểm đó nếu nó cũng có đủ độ dẻo để học cách thích nghi với môi trường theo cách có lợi. Mô phỏng máy tính (Hinton và Nowlan, 1987) xác nhận rằng hiệu ứng Baldwin effect này là có thật và hậu quả là những thứ khó học cuối cùng sẽ nằm trong bộ gen, nhưng những thứ dễ học không cần phải nằm ở đó (Morgan và Griffiths, 2015).

hàm GENETIC-ALGORITHM(population, fitness) trả về một cá nhân
lặp lại

trọng lượng $\leftarrow$ WEIGHTED-BY(dân số, thể chất)dân
số2 $\leftarrow$ danh sách trống cho $i = 1$ đến SIZE(dân số) do

parent1, parent2 $\leftarrow$ LỰA CHỌN NGẪU NHIÊN CÓ TRỌNG SỐ(quần thể,
trọng số, 2)con $\leftarrow$ TÁI TẠO (parent1, parent2)nếu (xác suất ngẫu nhiên
nhỏ) thì con $\leftarrow$ ĐỘT BIẾN (con) thêm con vào dân số2quần thể $\leftarrow$ dân
số2cho đến khi một số cá thể đủ phù hợp, hoặc đủ thời gian đã trôi qua trả
về cá thể tốt nhất trong dân số, theo sự phù hợp

hàm REPRODUCE(parent1, parent2) trả về một cá nhân
 $n \leftarrow$ LENGTH(parent1) $c \leftarrow$ số ngẫu nhiên từ 1 đến n return
APPEND(SUBSTRING(parent1, 1, c), SUBSTRING(parent2, c + 1,
n))

Hình 4.8 Một thuật toán di truyền. Trong hàm, dân số là một danh sách có thứ tự của các cá nhân, trọng số là một danh sách các giá trị phù hợp tương ứng cho mỗi cá nhân và phù hợp là một chức năng để tính toán các giá trị này.

Rõ ràng, hiệu ứng này không có khả năng đáng kể nếu các bit liên kết hoàn toàn không liên quan đến nhau, bởi vì khi đó sẽ có một số khối liên kết cung cấp một lợi ích nhất quán. Các thuật toán di truyền hoạt động tốt nhất khi các lược đồ tương ứng với các thành phần có ý nghĩa của dung dịch. Ví dụ, nếu chuỗi là đại diện của ống-ten, thì các lược đồ có thể đại diện cho các thành phần của ống-ten, chẳng hạn như gương phản xạ và bộ làm lệch hướng. Một thành phần tốt có thể tốt trong nhiều thiết kế khác nhau. Điều này cho thấy rằng việc sử dụng thành công các thuật toán di truyền đòi hỏi phải có kỹ thuật biểu diễn cẩn thận.

Trong thực tế, các thuật toán di truyền có vị trí của chúng trong bối cảnh rộng lớn của các phương pháp tối ưu hóa (Marler và Arora, 2004), đặc biệt là đối với các vấn đề có cấu trúc phức tạp như bố trí mạch hoặc lập lịch trình công việc, và gần đây hơn là để phát triển kiến trúc của mạng nơ-ron sâu (Miikkulainen và cộng sự, 2019). Không rõ bao nhiêu sức hấp dẫn của các thuật toán di truyền phát sinh từ sự vượt trội của chúng trong các nhiệm vụ cụ thể, và bao nhiêu từ phép ẩn dụ hấp dẫn của sự tiến hóa.

4.2 Tìm kiếm cục bộ trong không gian liên tục

Trong Chương 2, chúng tôi đã giải thích sự khác biệt giữa môi trường rời rạc và liên tục, chỉ ra rằng hầu hết các môi trường trong thế giới thực là liên tục. Một không gian hành động liên tục có một hệ số phân nhánh vô hạn, và do đó không thể được xử lý bởi hầu hết các thuật toán mà chúng ta đã đề cập cho đến nay (ngoại trừ leo đồi lựa chọn đầu tiên và ủ mô phỏng).

Phản ứng cung cấp một giới thiệu rất ngắn gọn về một số kỹ thuật tìm kiếm cục bộ cho các không gian liên tục. Tài liệu về chủ đề này rất lớn; Nhiều kỹ thuật cơ bản bắt nguồn

vào thế kỷ 17, sau sự phát triển của giải tích bởi Newton và Leibniz. Chúng tôi tìm thấy cách sử dụng các kỹ thuật này ở một số nơi trong cuốn sách này, bao gồm các chương về học tập, thị giác và robot.

Chúng ta bắt đầu với một ví dụ. Giả sử chúng ta muốn đặt ba sân bay mới ở bất kỳ đâu ở Romania, sao cho tổng khoảng cách đường thẳng bình phương từ mỗi thành phố trên bản đồ đến sân bay gần nhất được giảm thiểu. (Xem Hình 3.1 để biết bản đồ của Romania.) Không gian trạng thái sau đó được xác định bởi tọa độ của ba sân bay: (x_1, y_1) , (x_2, y_2) và (x_3, y_3) . Đây là một không gian sáu chiều; Chúng tôi cũng nói rằng các trạng thái được xác định bởi sáu biến. Trong

Biến số

Nói chung, các trạng thái được xác định bởi một vector N chiều của các biến, x . Di chuyển xung quanh trong không gian này tương ứng với việc di chuyển một hoặc nhiều sân bay trên bản đồ. Chức năng mục tiêu

$f(x) = f(x_1, y_1, x_2, y_2, x_3, y_3)$ tương đối dễ tính toán cho bất kỳ tiểu bang cụ thể nào sau khi chúng tôi tính toán các thành phố gần nhất. Giả sử C_i là tập hợp các thành phố có sân bay gần nhất (trong trạng thái x) lân cận bay i . Sau đó, chúng ta có

$$f(x) = f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_{C \in C_i} \sum_{j=1}^3 (x_i - x_c)^2 + (y_i - y_c)^2. \quad (4.1)$$

Phương trình này không chỉ đúng đối với trạng thái x mà còn đối với các tiểu bang trong khu vực lân cận cục bộ x . Tuy nhiên, nó không đúng trên toàn cầu; nếu chúng ta đi quá xa X (bằng cách thay đổi vị trí của một hoặc nhiều sân bay một lượng lớn) thì tập hợp các thành phố gần nhất cho sân bay đó sẽ thay đổi và chúng ta cần tính toán lại C_i .

Rời rạc hóa

Một cách để đối phó với một không gian trạng thái liên tục là rời rạc nó. Ví dụ: thay vì Cho phép các vị trí (x_i, y_i) là bất kỳ điểm nào trong không gian hai chiều liên tục, chúng ta có thể giới hạn chúng thành các điểm cố định trên lưới hình chữ nhật với khoảng cách kích thước δ (delta). Sau đó, thay vì có một số lượng vô hạn các biến kế nhiệm, mỗi trạng thái trong không gian sẽ chỉ có 12 thành công, tương ứng với việc tăng một trong 6 biến lên $\pm\delta$. Sau đó, chúng ta có thể áp dụng bất kỳ thuật toán tìm kiếm cục bộ nào của chúng ta cho không gian rời rạc này. Ngoài ra, chúng ta có thể làm cho hệ số phân nhánh hữu hạn bằng cách lấy mẫu các trạng thái kế thừa một cách ngẫu nhiên, di chuyển theo một hướng ngẫu nhiên với một lượng nhỏ, δ . Các phương pháp đo lường tiến độ bằng sự thay đổi giá trị của chức năng khách quan giữa hai điểm lân cận được gọi là phương pháp gradient thực nghiệm. Gradient thực nghiệm

Gradient thực nghiệm

Tìm kiếm cũng giống như leo đồi dốc nhất trong một phiên bản rời rạc của không gian tiểu bang. Giảm giá trị của δ theo thời gian có thể đưa ra một giải pháp chính xác hơn, nhưng không nhất thiết hội tụ đến một tối ưu toàn cầu trong giới hạn.

Thường thì chúng ta có một hàm khách quan được biểu thị dưới dạng toán học để chúng ta có thể sử dụng giải tích để giải quyết vấn đề bằng cách phân tích hơn là theo kinh nghiệm. Nhiều phương pháp cố gắng sử dụng độ dốc của phong cảnh để tìm mức tối đa. Độ dốc của hàm mục tiêu là một vector ∇f cho độ lớn và hướng của độ dốc nhất. Đối với vấn đề của chúng tôi, chúng tôi có

Gradient

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial y_3} \right).$$

Trong một số trường hợp, chúng ta có thể tìm thấy mức tối đa bằng cách giải phương trình $\nabla f = 0$. (Điều này có thể được thực hiện, ví dụ, nếu chúng ta chỉ đặt một sân bay; giải pháp là giá trị trung bình số học của tất cả các tọa độ của các thành phố.) Tuy nhiên, trong nhiều trường hợp, phương trình này không thể được giải ở dạng đóng. Ví dụ: với ba sân bay, biểu thức cho gradient phụ thuộc vào thành phố

2 Kiến thức về vector, ma trận và đạo hàm rất hữu ích cho phần này (xem Phụ lục A).

gần mỗi sân bay nhất trong trạng thái hiện tại. Điều này có nghĩa là chúng ta có thể tính toán gradient cục bộ (nhưng không phải trên toàn cầu); chẳng hạn

$$\frac{\partial f}{\partial x_1} = \sum_{C \in C_1} (x_1 - x_C) \quad (4.2)$$

Với một biểu thức chính xác cục bộ cho độ dốc, chúng ta có thể thực hiện leo đồi dốc nhất bằng cách cập nhật trạng thái hiện tại theo công thức

$x \leftarrow x + \alpha \nabla f(x)$, trong đó α (alpha) là một hằng số nhỏ thường được gọi là kích thước bước. Có rất nhiều loại

Kích thước bước

các phương pháp điều chỉnh α . Vấn đề cơ bản là nếu α quá nhỏ, cần quá nhiều bước; Nếu α quá lớn, tìm kiếm có thể vượt quá mức tối đa. Kỹ thuật tìm kiếm dòng cố gắng khắc phục tình thế tiến thoái lưỡng nan này bằng cách mở rộng hướng gradient hiện tại — thường là bằng cách liên tục tăng gấp đôi α — cho đến khi f bắt đầu giảm trở lại. Điểm mà điều này xảy ra trở thành trạng thái hiện tại mới. Có một số trường phái suy nghĩ về cách chọn hướng đi mới vào thời điểm này.

Tìm kiếm dòng

Đối với nhiều vấn đề, thuật toán hiệu quả nhất là Newton-Raphson đáng kính phương pháp. Đây là một kỹ thuật chung để tìm gốc của các hàm — nghĩa là giải các phương trình ở dạng $g(x) = 0$. Nó hoạt động bằng cách tính toán một ước tính mới cho gốc x theo công thức Newton

Newton-Raphson

$x \leftarrow x - g(x) / g'(x)$. Để tìm cực đại hoặc tối thiểu của f , chúng ta cần tìm x sao cho gradient là một vector bằng không (tức là $\nabla f(x) = 0$). Do đó, $g(x)$ trong công thức Newton trở thành $\nabla f(x)$ và phương trình cập nhật có thể được viết ở dạng ma trận-vector như

$x \leftarrow x - H^{-1} \nabla f(x)$, trong đó H là ma trận Hessian của đạo hàm thứ hai, có các phần tử H_{ij} được cho bởi

Hessian

$\partial^2 f / \partial x_i \partial x_j$. Đối với ví dụ sân bay của chúng ta, chúng ta có thể thấy từ Phương trình (4.2) rằng H rất đơn giản: các phần tử ngoài đường chéo là không và các phần tử đường chéo cho sân bay i chỉ gấp đôi số thành phố trong C_i . Một phép tính của một khoảng khắc cho thấy rằng một bước của bản cập nhật di chuyển sân bay i trực tiếp đến tâm của C_i , là tối thiểu của biểu thức cục bộ cho f từ Phương trình (4.1).³ Tuy nhiên, đối với các bài toán chiều cao, việc tính toán n^2 en-try của Hessian và đảo ngược nó có thể tốn kém, vì vậy nhiều phiên bản gần đúng của phương pháp Newton-Raphson đã được phát triển.

Các phương pháp tìm kiếm cục bộ bị ảnh hưởng bởi cực đại cục bộ, rặng và cao nguyên trong không gian trạng thái liên tục cũng như trong các không gian rời rạc. Khởi động lại ngẫu nhiên và ủ mô phỏng thường hữu ích. Tuy nhiên, không gian liên tục chiều cao là những nơi lớn mà rất dễ bị lạc.

Chủ đề cuối cùng là tối ưu hóa hạn chế. Một bài toán tối ưu hóa bị hạn chế nếu các giải pháp phải đáp ứng một số ràng buộc cứng về giá trị của các biến. Ví dụ, trong vấn đề vị trí sân bay của chúng tôi, chúng tôi có thể hạn chế các địa điểm ở bên trong Romania và trên đất khô (thay vì ở giữa hồ). Độ khó của các bài toán tối ưu hóa bị ràng buộc phụ thuộc vào bản chất của các ràng buộc và hàm mục tiêu. Loại nổi tiếng nhất là các bài toán lập trình tuyến tính, trong đó các ràng buộc phải là bất đẳng thức tuyến tính

Tối ưu hóa hạn chế

Lập trình tuyến tính

³ Nói chung, bản cập nhật Newton-Raphson có thể được coi là phù hợp với một bề mặt bậc hai với f tại x và sau đó di chuyển trực tiếp đến mức tối thiểu của bề mặt đó — cũng là giá trị tối thiểu của f nếu f là bậc hai.

Bộ lời	tạo thành một tập lời 4 và hàm mục tiêu cũng là tuyến tính. Độ phức tạp về thời gian của tuyến tính Lập trình là đa thức về số biến.
Tối ưu hóa lời	Lập trình tuyến tính có lẽ là phương pháp được nghiên cứu rộng rãi nhất và hữu ích rộng rãi nhất để tối ưu hóa. Đây là một trường hợp đặc biệt của vấn đề tổng quát hơn là tối ưu hóa lời, đó là Cho phép vùng ràng buộc là bất kỳ vùng lời nào và mục tiêu là bất kỳ hàm nào lời trong vùng ràng buộc. Trong một số điều kiện nhất định, các bài toán tối ưu hóa lời cũng có thể giải được đa từ và có thể khả thi trong thực tế với hàng nghìn biến. Một số vấn đề quan trọng trong học máy và lý thuyết điều khiển có thể được xây dựng như các bài toán tối ưu hóa lời (xem Chương 21).

4.3 Tìm kiếm với các hành động không xác định

Trong Chương 3, chúng ta giả định một môi trường đã biết, xác định, hoàn toàn có thể quan sát được. Do đó, một tác nhân có thể quan sát trạng thái ban đầu, tính toán một chuỗi các hành động đạt được mục tiêu và thực hiện các hành động với "đôi mắt nhắm nghiền" của nó, không bao giờ phải sử dụng nhận thức của nó.

Tuy nhiên, khi môi trường có thể quan sát được một phần, tác nhân không biết chắc chắn nó ở trạng thái nào; Và khi môi trường không xác định, tác nhân không biết nó chuyển sang trạng thái nào sau khi thực hiện một hành động. Điều đó có nghĩa là thay vì nghĩ rằng "Tôi đang ở trạng thái s1 và nếu tôi thực hiện hành động a, tôi sẽ kết thúc ở trạng thái s2", một tác nhân bây giờ sẽ nghĩ rằng "Tôi đang ở trạng thái s1 hoặc s3, và nếu tôi thực hiện hành động a, tôi sẽ kết thúc ở trạng thái s2, s4 hoặc s5." Chúng ta gọi một tập hợp các trạng thái vật lý mà tác nhân tin là có thể là trạng thái niềm tin.

Trạng thái niềm tin

Trong môi trường có thể quan sát một phần và không xác định, giải pháp cho một vấn đề không còn là một trình tự, mà là một kế hoạch có điều kiện (đôi khi được gọi là kế hoạch dự phòng hoặc chiến lược) chỉ định những gì cần làm tùy thuộc vào những nhận thức mà tác nhân nhận được trong khi thực hiện kế hoạch. Chúng tôi xem xét tính không xác định trong phần này và khả năng quan sát một phần trong phần tiếp theo.

Kế hoạch có điều kiện

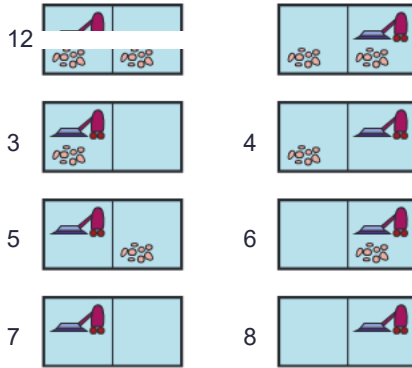
4.3.1 Thế giới chân không thất thường

Thế giới chân không từ Chương 2 có tám trạng thái, như thể hiện trong Hình 4.9. Có ba hành động - Phải, Trái và Hút - và mục tiêu là dọn dẹp tất cả bụi bẩn (trạng thái 7 và 8). Nếu môi trường hoàn toàn có thể quan sát được, xác định và hoàn toàn được biết đến, thì vấn đề rất dễ giải quyết bằng bất kỳ thuật toán nào trong Chương 3 và giải pháp là một chuỗi hành động. Ví dụ: nếu trạng thái ban đầu là 1, thì chuỗi hành động [Suck, Right, Suck] sẽ đạt đến trạng thái mục tiêu, 8.

Bây giờ giả sử rằng chúng ta giới thiệu thuyết không xác định dưới dạng một máy hút bụi mạnh mẽ nhưng thất thường. Trong thế giới chân không thất thường, hành động Suck hoạt động như sau:

- Khi áp dụng cho một hình vuông bẩn, hành động sẽ làm sạch hình vuông và đôi khi cũng làm sạch bụi bẩn ở một hình vuông liền kề.
- Khi áp dụng cho một hình vuông sạch, hành động đôi khi lắng đọng bụi bẩn trên thảm.5 Để cung cấp một công thức chính xác cho vấn đề này, chúng ta cần khái quát hóa khái niệm về mô hình chuyển đổi từ Chương 3. Thay vì xác định mô hình chuyển tiếp bằng hàm RESULT

4 Một tập hợp các điểm S là lời nếu đường nối hai điểm bất kỳ trong S cũng được chứa trong S. Một hàm lời là một hàm mà không gian "phía trên" nó tạo thành một tập lời; Theo định nghĩa, các hàm lời không có mức tối thiểu cục bộ (trái ngược với toàn cục).5 Chúng tôi giả định rằng hầu hết độc giả phải đối mặt với các vấn đề tương tự và có thể thông cảm với tác nhân của chúng tôi. Chúng tôi xin lỗi cho sự hữu ích của thiết bị làm sạch hiện đại, hiệu quả, những người không thể tận dụng thiết bị sự phạm này.



Hình 4.9 Tám trạng thái có thể có của thế giới chân không; Trạng thái 7 và 8 là trạng thái mục tiêu.

trả về một trạng thái kết quả duy nhất, chúng ta sử dụng hàm RESULTS trả về một tập hợp các trạng thái kết quả có thể xảy ra. Ví dụ: trong thế giới chân không thất thường, hành động Suck ở trạng thái 1 chỉ làm sạch vị trí hiện tại hoặc cả hai vị trí:

KẾT QUẢ (1, Mút) = {5, 7}

Nếu chúng ta bắt đầu ở trạng thái 1, không có chuỗi hành động nào giải quyết được vấn đề, nhưng kế hoạch có điều kiện sau đây:

[Hút, nếu State = 5 thì [Đúng, Mút] nếu không []]. (4.3)

Ở đây chúng ta thấy rằng một kế hoạch có điều kiện có thể chứa các bước nếu-thì-khác; Điều này có nghĩa là các giải pháp là cây chứ không phải trình tự. Ở đây điều kiện trong câu lệnh if kiểm tra để xem trạng thái hiện tại là gì; Đây là điều mà nhân viên sẽ có thể quan sát được trong thời gian chạy, nhưng không biết tại thời điểm lập kế hoạch. Ngoài ra, chúng ta có thể có một công thức kiểm tra tri giác hơn là trạng thái. Nhiều vấn đề trong thế giới thực, vật lý là các vấn đề ngẫu nhiên, bởi vì dự đoán chính xác về tương lai là không thể. Vì lý do này, nhiều người luôn mở mắt khi đi dạo xung quanh.

4.3.2 Cây tìm kiếm AND-OR

Làm thế nào để chúng ta tìm ra những giải pháp ngẫu nhiên này cho các vấn đề không xác định? Như trong Chương 3, chúng ta bắt đầu bằng cách xây dựng cây tìm kiếm, nhưng ở đây các cây có một đặc điểm khác. Trong một môi trường phi thuật ngữ, sự phân nhánh duy nhất được giới thiệu bởi các lựa chọn riêng của tác nhân trong mỗi trạng thái: Tôi có thể thực hiện hành động này hoặc hành động kia. Chúng tôi gọi các nút này là nút OR. Trong thế giới chân không,

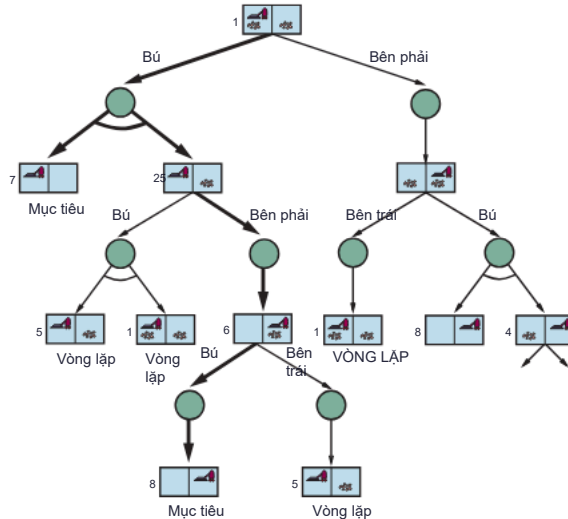
Hoặc nút

ví dụ: tại một nút OR, tác nhân chọn Left hoặc Right hoặc Suck. Trong môi trường không xác định, phân nhánh cũng được giới thiệu bởi sự lựa chọn kết quả của môi trường cho mỗi hành động. Chúng tôi gọi các nút này là các nút AND. Ví dụ: hành động Suck ở trạng thái 1 dẫn đến

Và nút

trạng thái niềm tin {5, 7}, vì vậy tác nhân sẽ cần tìm một kế hoạch cho trạng thái 5 và cho trạng thái 7. Hai loại nút này xen kẽ, dẫn đến cây AND-OR như minh họa trong Hình 4.10.

Và-hoặc cây



Hình 4.10 Hai cấp độ đầu tiên của cây tìm kiếm cho thể giới chân không thất thường. Các nút trạng thái là các nút HOẶC mà một số hành động phải được chọn. Tại các nút AND, được hiển thị dưới dạng vòng tròn, mọi kết quả phải được xử lý, như được biểu thị bằng vòng cung liên kết các nhánh đi. Giải pháp được tìm thấy được thể hiện bằng các dòng in đậm.

Một giải pháp cho vấn đề tìm kiếm AND-OR là một cây con của cây tìm kiếm hoàn chỉnh mà (1) có một nút mục tiêu ở mỗi lá, (2) chỉ định một hành động ở mỗi nút OR của nó và (3) bao gồm mọi nhánh kết quả ở mỗi nút AND của nó. Giải pháp được thể hiện bằng các đường in đậm trong hình; nó tương ứng với kế hoạch được đưa ra trong Phương trình (4.3).

Hình 4.11 đưa ra một thuật toán đệ quy, độ sâu đầu tiên để tìm kiếm đồ thị AND-OR. Một khía cạnh quan trọng của thuật toán là cách nó xử lý các chu kỳ, thường phát sinh trong các vấn đề không liên quan (ví dụ: nếu một hành động đôi khi không có tác dụng hoặc nếu một hiệu ứng ngoài ý muốn có thể được sửa chữa). Nếu trạng thái hiện tại giống hệt với trạng thái trên đường dẫn từ gốc, thì nó sẽ quay trở lại với sự thất bại. Điều này không có nghĩa là không có giải pháp từ trạng thái hiện tại; Nó chỉ đơn giản có nghĩa là nếu có một giải pháp không tuân hoàn, nó phải có thể đạt được từ hóa thân trước đó của trạng thái hiện tại, vì vậy hóa thân mới có thể bị loại bỏ. Với kiểm tra này, chúng tôi đảm bảo rằng thuật toán kết thúc trong mọi không gian trạng thái hữu hạn, bởi vì mọi đường dẫn phải đạt được mục tiêu, ngõ cụt hoặc trạng thái lặp lại. Lưu ý rằng thuật toán không kiểm tra xem trạng thái hiện tại có phải là sự lặp lại của một trạng thái trên một số đường dẫn khác từ gốc hay không, điều này rất quan trọng đối với hiệu quả.

Đồ thị AND-OR có thể được khám phá chiều rộng đầu tiên hoặc tốt nhất đầu tiên. Khái niệm về hàm heuristics phải được sửa đổi để ước tính chi phí của một giải pháp ngẫu nhiên hơn là một kết quả, nhưng khái niệm về khả năng chấp nhận được vẫn tiếp tục và có một sự tương tự của thuật toán A* để tìm ra các giải pháp tối ưu. (Xem ghi chú thư mục ở cuối chương.)

```
function AND-OR-SEARCH(problem) trả về một kế hoạch có điều kiện hoặc thất bại
trả về OR-SEARCH(vấn đề, vấn đề. BAN ĐẦU, [])
```

```
function OR-SEARCH(problem, state, path) trả về một kế hoạch có điều kiện hoặc thất bại
nếu có vấn đề. IS-GOAL(state) sau đó trả về planif IS-CYCLE(state, path)
sau đó trả về failurecho mỗi hành động có vấn đề. ACTIONS(state) làm
```

```
plan ← AND-SEARCH(problem, RESULTS(state, action), [state] +
[path])nếu plan 6= thất bại thì trả về [action] + [plan]return failure
```

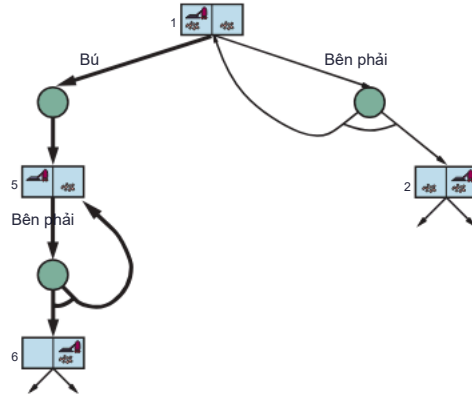
```
function AND-SEARCH(problem, states, path) trả về một kế hoạch có điều kiện hoặc thất bại
Đối với mỗi SI ở các tiểu bang làm
plan1 ← OR-SEARCH(problem, si, path)if plan1 = failure thì trả về
failurereturn [if s1 then plan1 else if s2 then plan2 else . . . nếu sn-1 thì
plann-1 else plann]
```

Hình 4.11 Một thuật toán để tìm kiếm đồ thị AND-OR được tạo ra bởi môi trường không xác định. Giải pháp là một kế hoạch có điều kiện xem xét mọi kết quả không xác định và lập kế hoạch cho từng kết quả.

4.3.3 Thử, thử lại

Hãy xem xét một thể giới chân không trơn trượt, giống hệt với thể giới chân không thông thường (không thất thường) ngoại trừ các hành động chuyển động đôi khi thất bại, để lại tác nhân ở cùng một vị trí. Ví dụ, di chuyển sang phải ở trạng thái 1 dẫn đến trạng thái niềm tin {1, 2}. Hình 4.12 cho thấy một phần của biểu đồ tìm kiếm; rõ ràng, không còn bất kỳ giải pháp không tuần hoàn nào từ trạng thái 1 và AND-OR-SEARCH sẽ trở lại với thất bại. Tuy nhiên, có một giải pháp tuần hoàn, đó là Giải pháp tuần hoàn để tiếp tục cố gắng cho đến khi nó hoạt động. Chúng ta có thể diễn đạt điều này bằng một cấu trúc while mới:
[Suck, trong khi State = 5 do Right, Suck] hoặc bằng cách thêm một nhãn để biểu thị một số phần của kế hoạch và tham khảo nhãn đó sau:

[Suck, L1 : Đúng, nếu State = 5 thì L1 else Suck] . Khi nào một kế hoạch tuần hoàn là một giải pháp? Điều kiện tối thiểu là mỗi chiếc lá là một trạng thái mục tiêu và một chiếc lá có thể đạt được từ mọi điểm trong kế hoạch. Thêm vào đó, chúng ta cần xem xét nguyên nhân của thuyết không xác định. Nếu thực sự là trường hợp cơ chế truyền động của robot hút bụi hoạt động trong một số thời gian, nhưng trượt ngẫu nhiên và độc lập vào những trường hợp khác, thì tác nhân có thể tự tin rằng nếu hành động được lặp lại đủ số lần, cuối cùng nó sẽ hoạt động và kế hoạch sẽ thành công. Nhưng nếu tính không xác định là do một số thực thể không được quan sát thấy về robot hoặc môi trường - có lẽ dây đai truyền động đã bị đứt và robot sẽ không bao giờ di chuyển - thì việc lặp lại hành động sẽ không giúp ích gì. Một cách để hiểu quyết định này là nói rằng công thức vấn đề ban đầu (có thể quan sát được hoàn toàn, không xác định) bị bỏ rơi để ủng hộ một công thức khác (có thể quan sát được một phần, xác định) trong đó sự thất bại của kế hoạch tuần hoàn được quy cho một thuộc tính không quan sát được của đai truyền động. Trong Chương 12, chúng ta thảo luận về cách quyết định khả năng nào trong số một số khả năng không chắc chắn có nhiều khả năng hơn.



Hình 4.12 Một phần của biểu đồ tìm kiếm cho một thế giới chân không tron trượt, nơi chúng ta đã chỉ ra (một số) chu kỳ một cách rõ ràng. Tất cả các giải pháp cho vấn đề này là các kế hoạch theo chu kỳ vì không có cách nào để di chuyển một cách đáng tin cậy.

4.4 Tìm kiếm trong môi trường có thể quan sát được một phần

Bây giờ chúng ta chuyển sang vấn đề về khả năng quan sát một phần, trong đó nhận thức của tác nhân không đủ để xác định trạng thái chính xác. Điều đó có nghĩa là một số hành động của tác nhân sẽ nhằm mục đích giảm bớt sự không chắc chắn về trạng thái hiện tại.

4.4.1 Tìm kiếm mà không quan sát

Không cảm biến
Theo

Khi nhận thức của tác nhân không cung cấp thông tin gì cả, chúng ta có cái được gọi là cảm biến vấn đề (hoặc một vấn đề phù hợp). Lúc đầu, bạn có thể nghĩ rằng tác nhân không cảm biến không có hy vọng giải quyết một vấn đề nếu nó không biết nó bắt đầu ở trạng thái nào, nhưng các giải pháp không cảm biến phổ biến và hữu ích một cách đáng ngạc nhiên, chủ yếu là vì chúng không dựa vào các cảm biến hoạt động đúng cách. Ví dụ, trong các hệ thống sản xuất, nhiều phương pháp khéo léo đã được phát triển để định hướng các bộ phận một cách chính xác từ một vị trí ban đầu không xác định bằng cách sử dụng một chuỗi các hành động mà không có cảm biến nào cả. Đôi khi một kế hoạch không cảm biến sẽ tốt hơn ngay cả khi có sẵn một kế hoạch có điều kiện có cảm biến. Ví dụ, các bác sĩ thường kê đơn một loại kháng sinh phổ rộng thay vì sử dụng kế hoạch có điều kiện là làm xét nghiệm máu, sau đó đợi kết quả trở lại, và sau đó kê đơn một loại kháng sinh đặc hiệu hơn. Kế hoạch không cảm biến giúp tiết kiệm thời gian và tiền bạc, đồng thời tránh nguy cơ nhiễm trùng trở nên tồi tệ hơn trước khi có kết quả xét nghiệm.

Hãy xem xét một phiên bản không cảm biến của thế giới chân không (xác định). Giả sử rằng tác nhân biết địa lý của thế giới của nó, nhưng không biết vị trí của chính nó hoặc sự phân bố của bụi bẩn. Trong trường hợp đó, trạng thái niềm tin ban đầu của nó là $\{1, 2, 3, 4, 5, 6, 7, 8\}$ (xem Hình 4.9). Bây giờ, nếu đặc vụ di chuyển Right, nó sẽ ở một trong các trạng thái $\{2, 4, 6, 8\}$ — đặc vụ đã thu được thông tin mà không nhận thức được bất cứ điều gì! Sau [Right, Suck] agent sẽ luôn kết thúc ở một trong các trạng thái $\{4, 8\}$. Cuối cùng, sau [Right, Suck, Left, Suck], agent được đảm bảo đạt được trạng thái mục tiêu 7, bất kể trạng thái bắt đầu là gì. Chúng ta nói rằng tác nhân có thể ép buộc thế giới vào trạng thái 7.

Cưỡng bức

Giải pháp cho một vấn đề không cảm biến là một chuỗi các hành động, không phải một kế hoạch có điều kiện (vì không có nhận thức). Nhưng chúng ta tìm kiếm trong không gian của các trạng thái niềm tin hơn là các trạng thái vật lý.⁶ Trong không gian trạng thái niềm tin, vấn đề hoàn toàn có thể quan sát được bởi vì tác nhân luôn biết trạng thái niềm tin của chính mình. Hơn nữa, giải pháp (nếu có) cho một vấn đề không có cảm biến luôn là một chuỗi các hành động. Điều này là bởi vì, như trong các vấn đề thông thường của Chương 3, các nhận thức nhận được sau mỗi hành động là hoàn toàn có thể dự đoán được - chúng luôn trống rỗng! Vì vậy, không có dự phòng nào để lên kế hoạch. Điều này đúng ngay cả khi môi trường không xác định.

Chúng ta có thể giới thiệu các thuật toán mới cho các vấn đề tìm kiếm không cảm biến. Nhưng thay vào đó, chúng ta có thể sử dụng các thuật toán hiện có từ Chương 3 nếu chúng ta chuyển đổi vấn đề vật lý cơ bản thành một vấn đề trạng thái niềm tin, trong đó chúng ta tìm kiếm các trạng thái niềm tin hơn là trạng thái vật lý. Bài toán ban đầu, P , có các thành phần $ActionsP$, $ResultP$, v.v., và bài toán trạng thái niềm tin có các thành phần sau:

- Trạng thái: Không gian trạng thái niềm tin chứa mọi tập hợp con có thể có của các trạng thái vật lý. Nếu $Phas\ N$ trạng thái, thì vấn đề trạng thái niềm tin có $2N$ trạng thái niềm tin, mặc dù nhiều trạng thái trong số đó có thể không thể đạt được từ trạng thái ban đầu.

• Trạng thái ban đầu: Thông thường trạng thái niềm tin bao gồm tất cả các trạng thái trong P , mặc dù trong một số trường hợp, tác nhân sẽ có nhiều kiến thức hơn điều này.

- Hành động: Điều này hơi phức tạp. Giả sử tác nhân ở trạng thái tin tưởng $b = \{s_1, s_2\}$, nhưng $ACTIONS(b) = \{a_1, a_2\}$; thì người đại diện không chắc chắn về hành động nào là hợp pháp. Nếu chúng ta giả định rằng các hành động bất hợp pháp không có ảnh hưởng đến môi trường, thì sẽ an toàn khi thực hiện sự kết hợp của tất cả các hành động ở bất kỳ trạng thái vật lý nào trong trạng thái niềm tin hiện tại: $ACTIONS(b) = U$

$$S \in b$$

Mặt khác, nếu một hành động bất hợp pháp có thể dẫn đến thảm họa, sẽ an toàn hơn nếu chỉ cho phép giao lộ, tức là tập hợp các hành động hợp pháp ở tất cả các tiểu bang. Đối với thế giới chân không, mọi quốc gia đều có các hành động pháp lý giống nhau, vì vậy cả hai phương pháp đều cho kết quả giống nhau.

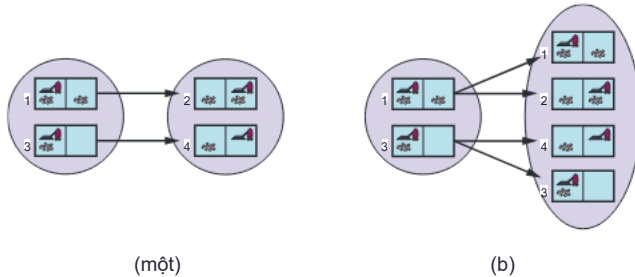
- Mô hình chuyển tiếp: Đối với các hành động xác định, trạng thái niềm tin mới có một trạng thái kết quả cho mỗi trạng thái có thể hiện tại (mặc dù một số trạng thái kết quả có thể giống nhau): $b' = RESULT(b, a) = \{s' : s' = RESULTP(s, a) \text{ và } s \in b\}$. (4.4) Với thuyết không xác định, trạng thái niềm tin mới bao gồm tất cả các kết quả có thể có của việc áp dụng hành động cho bất kỳ trạng thái nào trong trạng thái niềm tin hiện tại: $b' = RESULT(b, a) = \{s' : s' \in RESULTSP(s, a) \text{ và } s \in b\}$

$$= U \text{ KẾT QUẢ } (s, a), \\ S \in b$$

Kích thước của b' sẽ bằng hoặc nhỏ hơn b đối với các hành động xác định, nhưng có thể lớn hơn b với các hành động không xác định (xem Hình 4.13).

- Kiểm tra mục tiêu: Tác nhân có thể đạt được mục tiêu nếu bất kỳ trạng thái nào trong trạng thái niềm tin thỏa mãn kiểm tra mục tiêu của vấn đề cơ bản, IS-GOALP. Tác nhân nhất thiết phải đạt được mục tiêu nếu mọi trạng thái thỏa mãn (các) IS-GOALP. Chúng tôi mong muốn đạt được mục tiêu.
- Chi phí hành động: Điều này cũng rất phức tạp. Nếu cùng một hành động có thể có những chi phí khác nhau ở các trạng thái khác nhau, thì chi phí của việc thực hiện một hành động trong một trạng thái niềm tin nhất định có thể là một trong những

⁶ Trong một môi trường hoàn toàn có thể quan sát được, mỗi trạng thái niềm tin chứa đựng một trạng thái vật lý. Do đó, chúng ta có thể xem các thuật toán trong Chương 3 như đang tìm kiếm trong một không gian trạng thái niềm tin của các trạng thái niềm tin đơn lẻ.



Hình 4.13 (a) Dự đoán trạng thái niềm tin tiếp theo cho thế giới chân không không cảm biến với hành động xác định, Phải. (b) Dự đoán cho cùng một trạng thái niềm tin và hành động trong phiên bản trơn trượt của thế giới chân không không cảm biến.

một số giá trị. (Điều này làm nảy sinh một lớp vấn đề mới, mà chúng tôi khám phá trong Exer-cise 4.MVAL.) Bây giờ chúng ta giả định rằng chi phí của một hành động là như nhau ở tất cả các trạng thái và do đó có thể được chuyển trực tiếp từ vấn đề vật lý cơ bản.

Hình 4.14 cho thấy không gian trạng thái niềm tin có thể tiếp cận được cho thế giới vac-uum xác định, không cảm biến. Chỉ có 12 trạng thái niềm tin có thể đạt được trong số $28 = 256$ trạng thái niềm tin có thể có.

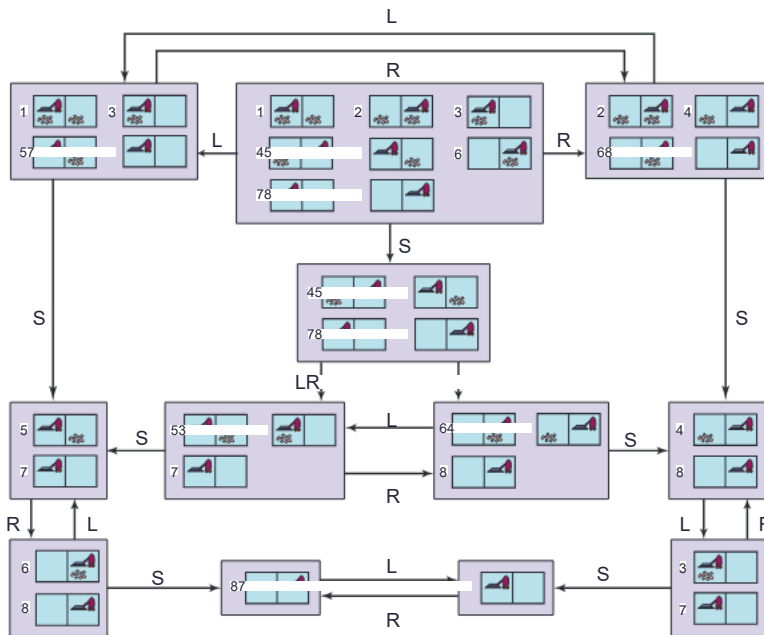
Các định nghĩa trước cho phép xây dựng tự động công thức vấn đề trạng thái niềm tin từ định nghĩa của vấn đề vật lý cơ bản. Khi điều này được thực hiện, chúng ta có thể giải quyết các vấn đề không cảm biến bằng bất kỳ thuật toán tìm kiếm thông thường nào của Chương 3.

Trong tìm kiếm đồ thị thông thường, các trạng thái mới đạt được được kiểm tra để xem chúng đã đạt được trước đó hay chưa. Điều này cũng hiệu quả đối với các trạng thái niềm tin; ví dụ, xem Hình 4.14, chuỗi hành động [Suck, Left, Suck] bắt đầu ở trạng thái ban đầu đạt đến trạng thái tin tưởng tương tự như [Right, Left, Suck], cụ thể là {5, 7}. Bây giờ, hãy xem xét trạng thái niềm tin đạt được bởi [Trái], cụ thể là, {1, 3, 5, 7}. Rõ ràng, điều này không giống với {5, 7}, nhưng nó là một siêu tập. Chúng ta có thể loại bỏ (cắt tia) bất kỳ trạng thái niềm tin siêu tập nào như vậy. Tại sao? Bởi vì một lời giải từ {1, 3, 5, 7} phải là một lời giải cho mỗi trạng thái riêng lẻ 1, 3, 5 và 7, và do đó nó là một lời giải cho bất kỳ sự kết hợp nào của các trạng thái riêng lẻ này, chẳng hạn như {5, 7}; Do đó, chúng ta không cần phải cố gắng giải {1, 3, 5, 7}, chúng ta có thể tập trung vào việc cố gắng giải quyết trạng thái niềm tin hoàn toàn dễ dàng hơn {5, 7}.

Ngược lại, nếu {1, 3, 5, 7} đã được tạo ra và được tìm thấy là có thể giải được, thì bất kỳ tập con nào, chẳng hạn như {5, 7}, được đảm bảo là có thể giải được. (Nếu tôi có một giải pháp hoạt động khi tôi rất bối rối về trạng thái của mình, nó vẫn sẽ hoạt động khi tôi bớt bối rối.) Mức độ cắt tia bổ sung này có thể cải thiện đáng kể hiệu quả của việc giải quyết vấn đề không cảm biến.

Tuy nhiên, ngay cả với sự cải tiến này, việc giải quyết vấn đề không cảm biến như chúng tôi đã mô tả hiếm khi khả thi trong thực tế. Một vấn đề là sự rộng lớn của không gian trạng thái niềm tin - chúng ta đã thấy trong chương trước rằng không gian tìm kiếm có kích thước N thường quá lớn, và bây giờ chúng ta có không gian tìm kiếm có kích thước $2N$. Hơn nữa, mỗi phần tử của không gian tìm kiếm là một tập hợp tối đa N phần tử. Đối với N lớn, chúng ta sẽ không thể biểu diễn dù chỉ một trạng thái niềm tin mà không hết không gian bộ nhớ.

Một giải pháp là thể hiện trạng thái niềm tin bằng một số mô tả ngắn gọn hơn. Trong English, chúng ta có thể nói tác nhân không biết "Không có gì" ở trạng thái ban đầu; sau khi di chuyển sang trái, chúng ta có thể



Hình 4.14 Phần có thể tiếp cận được của không gian trạng thái niềm tin cho thể giới chân không xác định, không cần biên. Mỗi hộp hình chữ nhật tương ứng với một trạng thái niềm tin duy nhất. Tại bất kỳ thời điểm nào, tác nhân có trạng thái niềm tin nhưng không biết nó đang ở trạng thái vật lý nào. Trạng thái niềm tin ban đầu (hoàn toàn thiếu hiểu biết) là hộp trung tâm trên cùng.

nói, "Không ở cột ngoài cùng bên phải", v.v. Chương 7 giải thích cách làm điều này trong một sơ đồ biểu diễn chính thức.

Một cách tiếp cận khác là trình các thuật toán tìm kiếm tiêu chuẩn, coi trạng thái niềm tin như hộp đen giống như bất kỳ trạng thái vấn đề nào khác. Thay vào đó, chúng ta có thể nhìn vào bên trong các trạng thái niềm tin và phát triển các thuật toán tìm kiếm trạng thái niềm tin gia tăng để xây dựng giải pháp một vật lý.

ical tại một thời điểm. Ví dụ, trong thế giới chân không không cảm biến, trạng thái niềm tin ban đầu là $\{1, 2, 3, 4, 5, 6, 7, 8\}$, và chúng ta phải tìm một chuỗi hành động hoạt động trong tất cả 8 trạng thái. Chúng ta có thể làm điều này bằng cách trước tiên tìm ra một giải pháp phù hợp với trạng thái 1; sau đó chúng ta kiểm tra xem nó có hoạt động cho trạng thái 2 hay không; nếu không, hãy quay lại và tìm một giải pháp khác cho trạng thái 1, v.v.

Cũng giống như tìm kiếm AND-OR phải tìm một giải pháp cho mọi nhánh tại một nút AND, thuật toán này phải tìm một giải pháp cho mọi trạng thái trong trạng thái niềm tin; sự khác biệt là tìm kiếm AND-OR có thể tìm thấy một giải pháp khác nhau cho mỗi nhánh, trong khi tìm kiếm trạng thái niềm tin gia tăng phải tìm một giải pháp phù hợp với tất cả các trạng thái.

Ưu điểm chính của cách tiếp cận gia tăng là nó thường có thể phát hiện thất bại một cách nhanh chóng — khi một trạng thái niềm tin không thể giải quyết được, thường là trường hợp một tập hợp con nhỏ của

Tìm kiếm trạng thái tín
ngưỡng gia tăng

Trạng thái niềm tin, bao gồm một vài trạng thái đầu tiên được kiểm tra, cũng không thể giải quyết được. Trong một số trường hợp, điều này dẫn đến tốc độ tỷ lệ thuận với kích thước của các trạng thái niềm tin, bản thân chúng có thể lớn như chính không gian trạng thái vật lý.

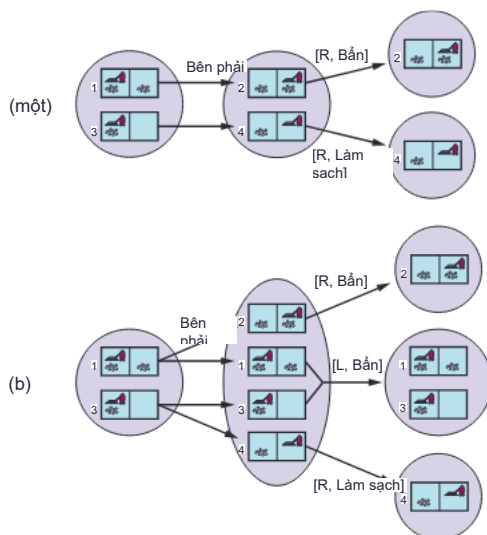
4.4.2 Tìm kiếm trong môi trường có thể quan sát được một phần

Nhiều vấn đề không thể được giải quyết nếu không cảm nhận. Ví dụ, câu đố 8 không cảm biến là không thể. Mặt khác, một chút cảm nhận có thể đi một chặng đường dài: chúng ta có thể giải 8 câu đố nếu chúng ta có thể chỉ nhìn thấy ô vuông góc trên bên trái. Giải pháp liên quan đến việc lần lượt di chuyển từng ô vào hình vuông có thể quan sát được và theo dõi vị trí của nó từ đó trở đi.

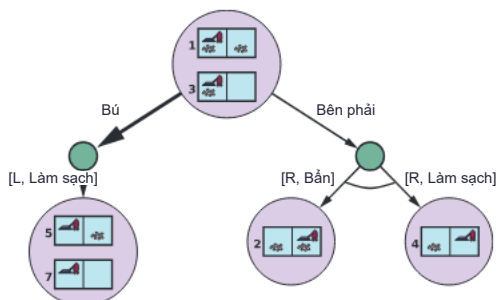
Đối với một vấn đề có thể quan sát được một phần, đặc tả vấn đề sẽ chỉ định một hàm PERCEPT trả về nhận thức mà tác nhân nhận được trong một trạng thái nhất định. Nếu cảm nhận là không xác định, thì chúng ta có thể sử dụng hàm PERCEPTS trả về một tập hợp các nhận thức có thể có. Đối với các vấn đề hoàn toàn có thể quan sát được, $PERCEPT(s) = s$ cho mọi trạng thái s và đối với các vấn đề không cảm biến $PERCEPT(s) = \text{null}$.

Hãy xem xét một thế giới chẵn không cảm biến cục bộ, trong đó tác nhân có một cảm biến vị trí tạo ra nhận thức L ở ô vuông bên trái và R ở ô vuông bên phải, và một cảm biến bụi bẩn tạo ra Bẩn khi hình vuông hiện tại bị bẩn và Sạch khi nó sạch. Do đó, PERCEPT trong trạng thái 1 là [L, Dirty]. Với khả năng quan sát một phần, thường sẽ là trường hợp một số trạng thái tạo ra cùng một nhận thức; trạng thái 3 cũng sẽ tạo ra [L, Dirty]. Do đó, với nhận thức ban đầu này, trạng thái niềm tin ban đầu sẽ là {1, 3}. Chúng ta có thể nghĩ về mô hình chuyển tiếp giữa các trạng thái niềm tin đối với các vấn đề có thể quan sát được một phần xảy ra trong ba giai đoạn, như thể hiện trong Hình 4.15:

- Giai đoạn dự đoán tính toán trạng thái niềm tin do hành động, $RESULT(b, a)$, chính xác như chúng ta đã làm với các vấn đề không cảm biến. Để nhấn mạnh rằng đây là một dự đoán, chúng tôi sử dụng ký hiệu $\hat{b} = RESULT(b, a)$, trong đó "mũ" trên b có nghĩa là "ước tính", và chúng tôi cũng sử dụng $PREDICT(b, a)$ như một từ đồng nghĩa với $RESULT(b, a)$.
- Giai đoạn tri giác có thể tính toán tập hợp các nhận thức có thể được quan sát trong trạng thái niềm tin được dự đoán (sử dụng chữ o để quan sát): $POSSIBLE-PERCEPTS(\hat{b}) = \{o : o = NHẬN THỨC \text{ và } s \in \hat{b}\}$.
- Giai đoạn cập nhật tính toán, đối với mỗi nhận thức có thể xảy ra, trạng thái niềm tin sẽ là kết quả của tri giác. Trạng thái niềm tin được cập nhật bo là tập hợp các trạng thái trong $\hat{b}$ có thể tạo ra nhận thức: $bo = UPDATE(\hat{b}, o) = \{s : o = PERCEPT(s) \text{ và } s \in \hat{b}\}$. Tác nhân cần phải đối phó với các nhận thức có thể xảy ra tại thời điểm lập kế hoạch, bởi vì nó sẽ không biết các nhận thức thực tế cho đến khi nó thực hiện kế hoạch. Lưu ý rằng thuyết không xác định trong môi trường vật lý có thể mở rộng trạng thái niềm tin trong giai đoạn dự đoán, nhưng mỗi trạng thái niềm tin được cập nhật bo không thể lớn hơn trạng thái niềm tin dự đoán $\hat{b}$; quan sát chỉ có thể giúp giảm bớt sự không chắc chắn. Hơn nữa, đối với cảm giác xác định, các trạng thái niềm tin cho các nhận thức có thể khác nhau sẽ rời rạc, tạo thành một phân vùng của trạng thái niềm tin dự đoán ban đầu. Đặt ba giai đoạn này lại với nhau, chúng ta thu được các trạng thái niềm tin có thể xảy ra từ một hành động nhất định và các nhận thức có thể xảy ra sau đó: $RESULTS(b, a) = \{bo : bo = UPDATE(PREDICT(b, a), o) \text{ và } o \in \text{KHẢ NĂNG-NHẬN THỨC (DỰ ĐOÁN (b, a))}\}$. (4.5)



Hình 4.15 Hai ví dụ về sự chuyển đổi trong thế giới chân không cảm biến cục bộ. (a) Trong thế giới rắn đuôi, Quyền được áp dụng trong trạng thái niềm tin ban đầu, dẫn đến một trạng thái niềm tin mới được dự đoán với hai trạng thái vật lý có thể xảy ra; đối với những trạng thái đó, các nhận thức có thể có là [R, Bắn] và [R, Sạch], dẫn đến hai trạng thái niềm tin, mỗi trạng thái là một trạng thái đơn. (b) Trong thế giới trơn trượt, Quyền được áp dụng trong trạng thái niềm tin ban đầu, đưa ra một trạng thái niềm tin mới với bốn trạng thái vật lý; đối với những trạng thái đó, các nhận thức có thể là [L, Bắn], [R, Bắn] và [R, Sạch], dẫn đến ba trạng thái niềm tin như được hiển thị.



Hình 4.16 Cấp độ đầu tiên của cây tìm kiếm AND-OR cho một vấn đề trong thế giới chân không cảm biến cục bộ; Hút là hành động đầu tiên trong giải pháp.

4.4.3 Giải quyết các vấn đề có thể quan sát được một phần

Phần trước đã chỉ ra cách suy ra hàm RESULTS cho một vấn đề trạng thái niềm tin không xác định từ một vấn đề vật lý cơ bản, cho hàm PERCEPT. Với công thức này, thuật toán tìm kiếm AND-OR của Hình 4.11 có thể được áp dụng trực tiếp để suy ra một giải pháp. Hình 4.16 cho thấy một phần của cây tìm kiếm cho thể giới chân không cảm nhận cục bộ, giả định một nhận thức ban đầu [L, Bẩn]. Giải pháp là kế hoạch có điều kiện

[Mút, đúng, nếu Rstate = {6} thì Suck else []] .

Lưu ý rằng, vì chúng tôi đã cung cấp một bài toán trạng thái niềm tin cho thuật toán tìm kiếm AND-OR, nó trả về một kế hoạch có điều kiện kiểm tra trạng thái niềm tin hơn là trạng thái thực tế. Điều này đúng như vậy: trong một môi trường có thể quan sát được một phần, tác nhân sẽ không biết trạng thái thực tế.

Như trong trường hợp các thuật toán tìm kiếm tiêu chuẩn được áp dụng cho các vấn đề không cảm biến, thuật toán tìm kiếm AND-OR coi trạng thái niềm tin là hộp đen, giống như bất kỳ trạng thái nào khác. Người ta có thể cải thiện điều này bằng cách kiểm tra các trạng thái niềm tin được tạo ra trước đó là tập con hoặc siêu tập hợp của trạng thái hiện tại, cũng như đối với các vấn đề không có cảm biến. Người ta cũng có thể rút ra các thuật toán tìm kiếm gia tăng, tương tự như những thuật toán được mô tả cho các vấn đề không cảm biến, cung cấp tốc độ đáng kể so với cách tiếp cận hợp đen.

4.4.4 Một tác nhân cho các môi trường có thể quan sát được một phần

Một tác nhân cho các môi trường có thể quan sát được một phần xây dựng một vấn đề, gọi một thuật toán tìm kiếm (chẳng hạn như AND-OR-SEARCH) để giải quyết nó và thực hiện giải pháp. Có hai điểm khác biệt chính giữa tác nhân này và tác nhân đối với môi trường xác định hoàn toàn có thể quan sát được. Đầu tiên, giải pháp sẽ là một kế hoạch có điều kiện chứ không phải là một trình tự; Để thực hiện một if-then-else expression, agent sẽ cần kiểm tra điều kiện và thực hiện nhánh thích hợp của the conditional. Thứ hai, tác nhân sẽ cần duy trì trạng thái niềm tin của mình khi thực hiện các hành động và nhận nhận thức. Quá trình này giống với quá trình dự đoán-quan sát-cập nhật trong Phương trình (4.5) nhưng thực sự đơn giản hơn vì nhận thức được đưa ra bởi môi trường chứ không phải do tác nhân tính toán. Với trạng thái niềm tin ban đầu b, một hành động a và một nhận thức o, trạng thái niềm tin mới là:

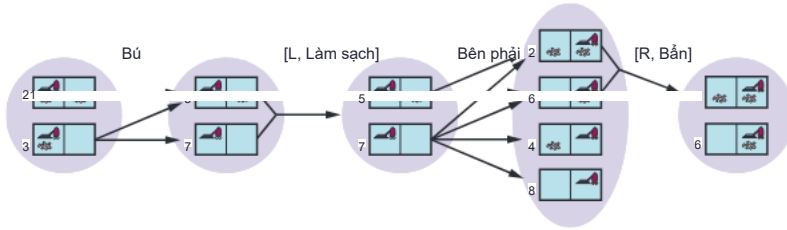
$$b' = \text{CẬP NHẬT (DỰ ĐOÁN (B, A), O)} . \quad (4.6)$$

Hãy xem xét một thể giới chân không mẫu giáo, trong đó các tác nhân chỉ cảm nhận trạng thái của hình vuông hiện tại của họ, và bất kỳ hình vuông nào cũng có thể trở nên bẩn bất cứ lúc nào trừ khi tác nhân đặc tích cực làm sạch nó tại thời điểm đó. 7 Hình 4.17 cho thấy trạng thái niềm tin được duy trì trong môi trường này.

Trong môi trường có thể quan sát được một phần - bao gồm phần lớn môi trường trong thể giới thực - duy trì trạng thái niềm tin của một người là chức năng cốt lõi của bất kỳ hệ thống thông minh nào. Chức năng này có nhiều tên khác nhau, bao gồm giám sát, lọc và ước tính trạng thái-

Tion. Phương trình (4.6) được gọi là công cụ ước lượng trạng thái đệ quy vì nó tính toán niềm tin mới trạng thái từ cái trước hơn là bằng cách kiểm tra toàn bộ trình tự nhận thức. Nếu tác nhân không bị "tụt lại phía sau", thì việc tính toán phải diễn ra nhanh như các nhận thức đang đi vào. Khi môi trường trở nên phức tạp hơn, tác nhân sẽ chỉ có thời gian để tính toán một trạng thái niềm tin gần gũi, có lẽ tập trung vào các hàm ý của nhận thức đối với các khía cạnh của môi trường đang được quan tâm hiện tại. Hầu hết công việc về vấn đề này đã được thực hiện cho

7 Lời xin lỗi thông thường đối với những người không quen thuộc với ảnh hưởng của trẻ nhỏ đối với môi trường.



Hình 4.17 Hai chu kỳ dự đoán-cập nhật của việc duy trì trạng thái niềm tin trong thế giới chân không mẫu giáo với cảm biến cục bộ.

Môi trường trạng thái ngẫu nhiên, liên tục với các công cụ của lý thuyết xác suất, như đã giải thích trong Chương 14.

Trong phần này, chúng tôi sẽ chỉ ra một ví dụ trong môi trường rời rạc với các cảm biến xác định và các hành động không xác định. Ví dụ liên quan đến một robot với một nhiệm vụ ước tính trạng thái cụ thể được gọi là bản địa hóa: tìm ra vị trí của nó, cho một bản đồ thế giới và

Localization

một chuỗi các nhận thức và hành động. Robot của chúng ta được đặt trong môi trường giống như mê cung của Hình 4.18. Robot được trang bị bốn cảm biến sonar cho biết liệu có chướng ngại vật - bức tường bên ngoài hoặc một hình vuông bóng tối trong hình - ở mỗi hướng trong số bốn hướng la bàn. Nhận thức ở dạng vector bit, một bit cho mỗi hướng bắc, đông, nam và tây theo thứ tự đó, vì vậy 1011 có nghĩa là có chướng ngại vật ở phía bắc, nam và tây, nhưng không phải hướng đông.

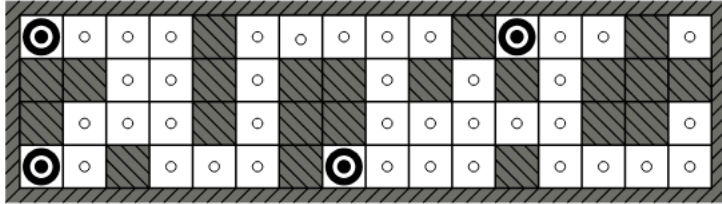
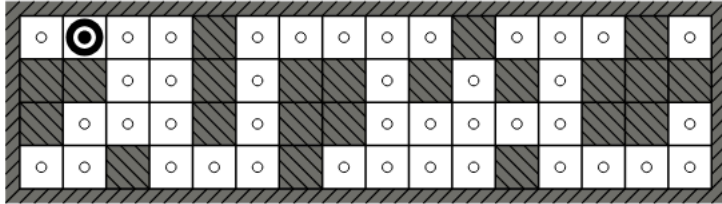
Chúng tôi giả định rằng các cảm biến cung cấp dữ liệu hoàn toàn chính xác và robot có bản đồ môi trường chính xác. Nhưng thật không may, hệ thống điều hướng của robot bị hỏng, vì vậy khi nó thực hiện hành động Right, nó di chuyển ngẫu nhiên đến một trong các ô vuông liền kề. Nhiệm vụ của robot là xác định vị trí hiện tại của nó.

Giả sử robot vừa được bật và nó không biết nó đang ở đâu - trạng thái niềm tin ban đầu b của nó bao gồm tập hợp tất cả các vị trí. Sau đó, robot nhận được nhận thức 1011 và thực hiện cập nhật bằng cách sử dụng phương trình $bo = \text{UPDATE}(1011)$, mang lại 4 vị trí được hiển thị trong Hình 4.18 (a). Bạn có thể kiểm tra mê cung để thấy rằng đó là bốn vị trí duy nhất mang lại nhận thức 1011.

Tiếp theo, robot thực hiện một hành động Đứng, nhưng kết quả là không xác định. Trạng thái niềm tin mới, $ba = \text{PREDICT}(bo, \text{Right})$, chứa tất cả các vị trí cách một bước so với các vị trí trong bo . Khi cảm giác thứ hai, 1010, đến, robot thực hiện $\text{UPDATE}(ba, 1010)$ và phát hiện ra rằng trạng thái niềm tin đã sụp đổ xuống vị trí duy nhất được hiển thị trong Hình 4.18 (b). Đó là vị trí duy nhất có thể là kết quả của

$\text{UPDATE}(\text{PREDICT}(\text{UPDATE}(b, 1011), \text{Right}), 1010)$.

Với các hành động không xác định, bước PREDICT phát triển trạng thái niềm tin, nhưng bước UPDATE thu nhỏ nó trở lại - miễn là các nhận thức cung cấp một số thông tin nhận dạng hữu ích. Đôi khi các nhận thức không giúp ích nhiều cho việc định vị hóa: Nếu có một hoặc nhiều hành lang đồng-tây dài, thì một robot có thể nhận được một chuỗi dài gồm 1010 tri giác, nhưng không bao giờ biết

(a) Vị trí có thể có của rô-bốt sau $E1 = 1011$ (b) Vị trí có thể có của rô-bốt sau $E1 = 1011$, $E2 = 1010$

Hình 4.18 Các vị trí có thể có của robot, (a) sau một quan sát, $E1 = 1011$, và (b) sau khi di chuyển một ô vuông và thực hiện quan sát thứ hai, $E2 = 1010$. Khi cảm biến không có tiếng ồn và mô hình chuyển tiếp chính xác, chỉ có một vị trí có thể cho robot phù hợp với chuỗi hai quan sát này.

nó ở đâu trong (các) hành lang. Nhưng đối với các môi trường có sự thay đổi hợp lý về địa lý, bản đồ hóa thường hội tụ nhanh chóng đến một điểm duy nhất, ngay cả khi các hành động không xác định.

Điều gì xảy ra nếu cảm biến bị lỗi? Nếu chúng ta chỉ có thể suy luận với logic Boolean, thì chúng ta phải coi mọi bit cảm biến là đúng hoặc sai, điều này cũng giống như không có thông tin tri giác nào cả. Nhưng chúng ta sẽ thấy rằng suy luận xác suất (Chương 12), cho phép chúng ta trích xuất thông tin hữu ích từ một cảm biến bị lỗi miễn là nó sai ít hơn một nửa thời gian.

4.5 Đại lý tìm kiếm trực tuyến và môi trường không xác định

Tìm kiếm ngoại tuyến Cho đến nay, chúng tôi đã tập trung vào các đại lý sử dụng thuật toán tìm kiếm ngoại tuyến. Họ tính toán một giải pháp hoàn chỉnh trước khi thực hiện hành động đầu tiên của họ. Ngược lại, một đại lý tìm kiếm trực tuyến **Tìm kiếm trực tuyến** xen kẽ tính toán và hành động: đầu tiên nó thực hiện một hành động, sau đó nó quan sát môi trường và tính toán hành động tiếp theo. Tìm kiếm trực tuyến là một ý tưởng hay trong môi trường động hoặc bán động, nơi có hình phạt cho việc ngồi xung quanh và tính toán quá lâu. Trực tuyến

8 Thuật ngữ "trực tuyến" ở đây đề cập đến các thuật toán phải xử lý đầu vào khi nó được nhận thay vì chờ đợi toàn bộ bộ dữ liệu đầu vào có sẵn. Việc sử dụng "trực tuyến" này không liên quan đến khái niệm "có kết nối Internet".

Tìm kiếm cũng hữu ích trong các lĩnh vực không xác định vì nó cho phép tác nhân tập trung nỗ lực tính toán của mình vào các tình huống ngẫu nhiên thực sự phát sinh hơn là những điều có thể xảy ra nhưng có thể sẽ không.

Tất nhiên, có một sự đánh đổi: một đại lý càng lên kế hoạch trước, nó càng ít thường xuyên thấy mình lên con lạch mà không có mái chèo. Trong môi trường không xác định, nơi tác nhân không biết trạng thái nào tồn tại hoặc hành động của nó làm gì, tác nhân phải sử dụng hành động của mình như các thí nghiệm để tìm hiểu về môi trường.

Một ví dụ điển hình của tìm kiếm trực tuyến là vấn đề lập bản đồ: một robot được đặt trong một tòa nhà không xác định và phải khám phá để xây dựng một bản đồ mà sau này có thể được sử dụng để đi từ A đến B. Phương pháp thoát khỏi mê cung — kiến thức cần thiết cho các anh hùng đầy tham vọng của thời cổ đại — cũng là những ví dụ về thuật toán tìm kiếm trực tuyến. Tuy nhiên, khám phá không gian không phải là hình thức khám phá trực tuyến duy nhất. Hãy xem xét một em bé sơ sinh: nó có nhiều hành động có thể xảy ra nhưng không biết kết quả của bất kỳ hành động nào trong số đó, và nó chỉ trải qua một vài trạng thái có thể mà nó có thể đạt được.

4.5.1 Vấn đề tìm kiếm trực tuyến

Một vấn đề tìm kiếm trực tuyến được giải quyết bằng cách xen kẽ tính toán, cảm biến và hành động. Chúng ta sẽ bắt đầu bằng cách giả định một môi trường xác định và hoàn toàn có thể quan sát được (Chương 16 nói lỏng các giả định này) và quy định rằng tác nhân chỉ biết những điều sau:

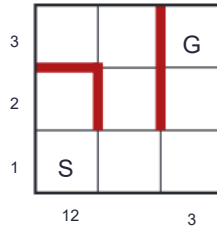
- (Các) HÀNH ĐỘNG, các hành động pháp lý tại các tiểu bang;
- $c(s, a, s')$, chi phí áp dụng hành động A ở trạng thái S để đạt được trạng thái S'. Lưu ý rằng điều này không thể được sử dụng cho đến khi tác nhân biết rằng s' là kết quả.
- IS-GOAL(s), bài kiểm tra mục tiêu. Đặc biệt lưu ý rằng tác nhân không thể xác định (các, a) KẾT QUẢ ngoại trừ bằng cách thực sự ở trong cát làm a. Ví dụ, trong bài toán mê cung được hiển thị trong Hình 4.19, tác nhân không biết rằng đi lên từ (1,1) dẫn đến (1,2); cũng như vậy, sau khi làm điều đó, nó không biết rằng đi xuống sẽ đưa nó trở lại (1,1). Mức độ thiếu hiểu biết này có thể được giảm bớt trong một số ứng dụng — ví dụ, một nhà thám hiểm robot có thể biết các hành động chuyển động của nó hoạt động như thế nào và chỉ không biết vị trí của chúng ngay vật. Cuối cùng, tác nhân có thể có quyền truy cập vào một hàm phỏng đoán có thể chấp nhận h (các) ước tính khoảng cách từ trạng thái hiện tại đến trạng thái mục tiêu. Ví dụ, trong Hình 4.19, tác nhân có thể biết vị trí của mục tiêu và có thể sử dụng phương pháp phỏng đoán khoảng cách Manhattan (trang 116). Thông thường, mục tiêu của tác nhân là đạt được trạng thái mục tiêu trong khi giảm thiểu chi phí. (Một mục tiêu khác có thể chỉ đơn giản là khám phá toàn bộ môi trường.) Chi phí là tổng chi phí đường dẫn mà tổng nhân viên phải chịu khi nó di chuyển. Người ta thường so sánh chi phí này với chi phí đường dẫn mà tác nhân sẽ phải chịu nếu nó biết trước không gian tìm kiếm — nghĩa là đường dẫn tối ưu trong môi trường đã biết. Trong ngôn ngữ của các thuật toán trực tuyến, sự so sánh này được gọi là tỷ lệ cạnh tranh; chúng tôi muốn nó càng nhỏ càng tốt.

Tỷ lệ cạnh tranh

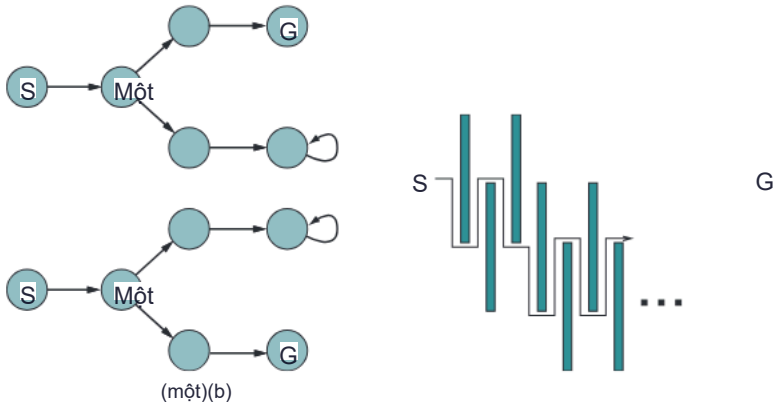
Các nhà thám hiểm trực tuyến dễ bị tổn thương bởi ngõ cụt: các trạng thái mà từ đó không đạt được trạng thái mục tiêu.

Ngõ cụt

khả. Nếu tác nhân không biết mỗi hành động làm gì, nó có thể thực hiện hành động "nhảy vào hồ không bot" và do đó không bao giờ đạt được mục tiêu. Nói chung, không có thuật toán nào có thể tránh được các Jend chết trong tất cả các không gian trạng thái. Hãy xem xét hai không gian trạng thái ngõ cụt trong Hình 4.20 (a). Một trực tuyến thuật toán tìm kiếm đã truy cập các trạng thái S và A không thể biết nó nằm trong không gian trạng thái trên cùng hay không gian dưới cùng; Cả hai trông giống hệt nhau dựa trên những gì đặc vụ đã thấy. Do đó, có



Hình 4.19 Một bài toán mê cung đơn giản. Đặc vụ bắt đầu từ S và phải đến G nhưng không biết gì về môi trường.



Hình 4.20 (a) Hai không gian trạng thái có thể dẫn một đại lý tìm kiếm trực tuyến vào ngõ cụt. Bất kỳ tác nhân nào sẽ thất bại trong ít nhất một trong những không gian này. (b) Môi trường hai chiều có thể khiến tác nhân tìm kiếm trực tuyến đi theo một lộ trình không hiệu quả tùy ý đến mục tiêu. Cho dù đặc vụ đưa ra lựa chọn nào, kẻ thù sẽ chặn tuyến đường đó bằng một bức tường mỏng và dài khác, do đó con đường đi theo dài hơn nhiều so với con đường tốt nhất có thể.

Lập luận của đối thủ

Hành động không thể
đảo ngược

Có thể khám phá an toàn

không có cách nào nó có thể biết cách chọn hành động chính xác trong cả hai không gian trạng thái. Đây là một ví dụ về một lập luận đối nghịch – chúng ta có thể tưởng tượng một kẻ thù xây dựng nhà nước trong khi tác nhân khám phá nó và đặt các mục tiêu và ngõ cụt ở bất cứ đâu nó chọn, như trong Hình 4.20 (b).

Ngô cụt là một khó khăn thực sự đối với việc khám phá robot — cầu thang, đường dốc, vách đá, đường một chiều và thậm chí cả địa hình tự nhiên, tất cả đều hiện tại trạng thái mà từ đó một số hành động thông thường sẽ đảo ngược — không có cách nào để trở lại trạng thái trước đó. Thuật toán khám phá mà chúng tôi sẽ trình bày chỉ được đảm bảo hoạt động trong không gian trạng thái có thể khám phá một cách an toàn — nghĩa là một trạng thái mục tiêu nào đó có thể truy cập được từ mọi trạng thái có thể tiếp cận. Không gian trạng thái chỉ có các hành động có thể đảo ngược, chẳng hạn như

function ONLINE-DFS-AGENT(problem, s') trả về một hành động

s, a, trạng thái và hành động trước đó, ban đầu nullresult, ánh xạ bảng (s, a) thành s', ban đầu trống không thử, bảng ánh xạ s đến danh sách các hành động chưa được thử unbacktracked, bảng ánh xạ s đến danh sách các trạng thái không bao giờ quay ngược đến

nếu có vấn đề. IS-GOAL (s') sau đó trả lại stopif s' là một trạng thái mới (không được thử) sau đó chưa thử [s'] ← vấn đề. ACTIONS

(s') nếu s không rỗng thì

kết quả [s, a] ← s'thêm s vào mặt

trước của unbacktracked[s'] nếu

untried[s'] trống thì

Nếu unbacktracked[s'] trống thì hãy quay lại điểm dừng

a ← một hành động b sao cho result[s', b] =

POP(unbacktracked[s'])s' ← nullelse a ← POP(untried[s'])s ←

s'return a

Hình 4.21 Một tác nhân tìm kiếm trực tuyến sử dụng khám phá độ sâu đầu tiên. Tác nhân chỉ có thể khám phá một cách an toàn trong không gian trạng thái trong đó mọi hành động có thể được "hoàn tác" bởi một số hành động khác.

mê cung và 8 câu đố, rõ ràng có thể khám phá một cách an toàn (nếu chúng có bất kỳ giải pháp nào). Chúng tôi sẽ đề cập sâu hơn về chủ đề thăm dò an toàn trong Phần 23.3.2.

Ngay cả trong môi trường có thể khám phá an toàn, không có tỷ lệ cạnh tranh giới hạn nào có thể được đảm bảo nếu có những con đường chi phí không giới hạn. Điều này rất dễ thể hiện trong môi trường có các hành động không thể đảo ngược, nhưng trên thực tế nó vẫn đúng đối với trường hợp có thể đảo ngược, như Hình 4.20 (b) cho thấy. Vì lý do này, người ta thường mô tả hiệu suất của các thuật toán tìm kiếm trực tuyến về kích thước của toàn bộ không gian trạng thái hơn là chỉ độ sâu của mục tiêu nông nhất.

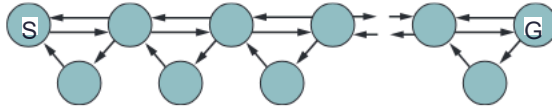
4.5.2 Đại lý tìm kiếm trực tuyến

Sau mỗi hành động, một tác nhân trực tuyến trong một môi trường có thể quan sát được nhận được một nhận thức cho nó biết nó đã đạt đến trạng thái nào; Từ thông tin này, nó có thể tăng cường bản đồ môi trường của nó. Bản đồ cập nhật sau đó được sử dụng để lập kế hoạch đi tiếp theo. Sự xen kẽ giữa lập kế hoạch và hành động này có nghĩa là các thuật toán tìm kiếm trực tuyến khá khác so với thuật toán tìm kiếm ngoại tuyến mà chúng ta đã thấy trước đây: các thuật toán ngoại tuyến khám phá thể giới thực. Ví dụ, A* có thể mở rộng một nút trong một phần của không gian và sau đó ngay lập tức mở rộng một nút ở một phần xa của không gian, bởi vì mở rộng nút liên quan đến các hành động mô phỏng hơn là thực tế.

Mặt khác, một thuật toán trực tuyến chỉ có thể khám phá những người kế nhiệm cho một tiểu bang mà nó chiếm giữ. Để tránh di chuyển đến một trạng thái xa để mở rộng nút tiếp theo, có vẻ như tốt hơn là nên mở rộng các nút theo thứ tự cục bộ.

Tìm kiếm độ sâu đầu tiên có chính xác thuộc tính này bởi vì (ngoại trừ khi thuật toán đang quay lại) nút tiếp theo được mở rộng là con của nút trước đó được mở rộng.

Một tác nhân thăm dò độ sâu trực tuyến (cho các hành động xác định nhưng chưa biết) được hiển thị trong Hình 4.21. Tác nhân này lưu trữ bản đồ của nó trong một bảng, result[s, a], ghi lại trạng thái kết quả từ việc thực hiện hành động a trong trạng thái s. (Đối với các hành động không xác định, tác nhân có thể ghi lại một tập hợp



Hình 4.22 Một môi trường trong đó một cuộc đi bộ ngẫu nhiên sẽ thực hiện theo cấp số nhân nhiều bước để tìm mục tiêu.

của các tiểu bang theo kết quả [s, a].) Bất cứ khi nào trạng thái hiện tại có các hành động chưa được khám phá, tác nhân sẽ thử một trong những hành động đó. Khó khăn xảy ra khi đặc vụ đã thử tất cả các hành động trong một trạng thái. Trong tìm kiếm độ sâu đầu tiên ngoại tuyến, trạng thái chỉ đơn giản là bị loại khỏi hàng đợi; Trong một tìm kiếm trực tuyến, đại lý phải quay lại trong thế giới thực. Trong tìm kiếm chuyên sâu, điều này có nghĩa là quay trở lại trạng thái mà từ đó tác nhân đã vào trạng thái hiện tại gần đây nhất. Để đạt được điều đó, thuật toán giữ một bảng khác liệt kê, đối với mỗi trạng thái, các trạng thái tiền nhiệm mà tác nhân chưa quay trở lại. Nếu tác nhân đã hết trạng thái mà nó có thể quay trở lại, thì việc tìm kiếm của nó đã hoàn tất.

Chúng tôi khuyên người đọc nên theo dõi tiến trình của ONLINE-DFS-AGENT khi áp dụng cho mê cung được đưa ra trong Hình 4.19. Khá dễ dàng để thấy rằng tác nhân, trong trường hợp xấu nhất, sẽ đi qua mọi liên kết trong không gian trạng thái chính xác hai lần. Đối với khám phá, đây là tối ưu; Mặt khác, để tìm kiếm một mục tiêu, tỷ lệ cạnh tranh của đại lý có thể xấu một cách tùy ý nếu nó đi trong một chuyến du ngoạn dài khi có một mục tiêu ngay bên cạnh trạng thái ban đầu. Một biến thể trực tuyến của đào sâu lặp đi lặp lại giải quyết vấn đề này; Đối với một môi trường là cây đồng nhất, tỷ lệ cạnh tranh của một tác nhân như vậy là một hằng số nhỏ.

Do phương pháp quay trở lại, ONLINE-DFS-AGENT chỉ hoạt động trong không gian trạng thái, nơi các hành động có thể đảo ngược. Có những thuật toán phức tạp hơn một chút hoạt động trong các không gian trạng thái nói chung, nhưng không có thuật toán nào như vậy có tỷ lệ cạnh tranh có giới hạn.

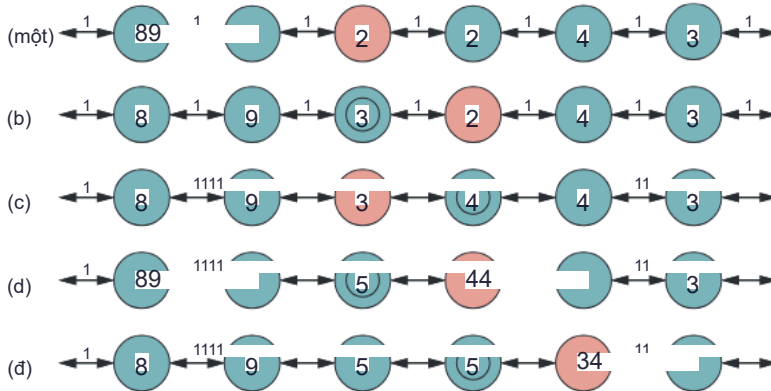
4.5.3 Tìm kiếm địa phương trực tuyến

Giống như tìm kiếm độ sâu đầu tiên, tìm kiếm leo đồi có đặc tính địa phương trong phần mở rộng nút của nó. Trên thực tế, bởi vì nó chỉ giữ một trạng thái hiện tại trong bộ nhớ, tìm kiếm leo đồi đã là một thuật toán tìm kiếm trực tuyến! Thật không may, thuật toán cơ bản không tốt lắm cho việc khám phá vì nó khiến tác nhân ngồi ở mức tối đa cục bộ không có nơi nào để đi. Hơn nữa, không thể sử dụng khởi động lại ngẫu nhiên, vì tác nhân không thể tự dịch chuyển tức thời sang trạng thái bất đầu mới.

Đi bộ ngẫu nhiên

Thay vì khởi động lại ngẫu nhiên, người ta có thể cân nhắc sử dụng đi bộ ngẫu nhiên để khám phá vironment. Một cuộc đi bộ ngẫu nhiên chỉ đơn giản là chọn ngẫu nhiên một trong các hành động có sẵn từ trạng thái hiện tại; Ưu tiên có thể được dành cho các hành động chưa được thử. Thật dễ dàng để chứng minh rằng một cuộc đi bộ ngẫu nhiên cuối cùng sẽ tìm thấy một mục tiêu hoặc hoàn thành việc khám phá của nó, miễn là không gian là hữu hạn và có thể khám phá một cách an toàn.⁹ Mặt khác, quá trình này có thể rất chậm. Hình 4.22 cho thấy một môi trường trong đó một cuộc đi bộ ngẫu nhiên sẽ mất theo cấp số nhân nhiều bước để tìm mục tiêu, bởi vì, đối với mỗi trạng thái ở hàng trên cùng ngoại trừ S, tiến trình lùi có khả năng gấp đôi so với tiến trình tiến. Tất nhiên, ví dụ này là bịa đặt, nhưng có nhiều không gian trạng thái thực mà tổ pô gây ra các loại "bẫy" này cho các bước đi ngẫu nhiên.

⁹ Đi bộ ngẫu nhiên hoàn thành trên lưới một chiều và hai chiều vô hạn. Trên lưới ba chiều, xác suất đi bộ trở lại điểm xuất phát chỉ khoảng 0,3405 (Hughes, 1995).



Hình 4.23 Năm lần lặp lại LRTA* trên không gian trạng thái một chiều. Mỗi trạng thái được gắn nhãn H(các), ước tính chi phí hiện tại để đạt được mục tiêu và mọi liên kết có chi phí hành động là 1. Trạng thái màu đỏ đánh dấu vị trí của tổng đài viên và ước tính chi phí cập nhật tại mỗi lần lặp lại có một vòng tròn kép.

Tăng cường leo đồi bằng trí nhớ thay vì ngẫu nhiên hóa ra là một cách tiếp cận hiệu quả hơn. Ý tưởng cơ bản là lưu trữ "ước tính tốt nhất hiện tại" H (các) chi phí để đạt được mục tiêu từ mỗi tiểu bang đã được truy cập. $H(s)$ bắt đầu chỉ là heuristic estimate $h(s)$ và được cập nhật khi tác nhân có được kinh nghiệm trong không gian trạng thái.

Hình 4.23 cho thấy một ví dụ đơn giản trong không gian trạng thái một chiều. Trong (a), tác nhân dường như bị mắc kẹt trong một mức tối thiểu cục bộ phẳng ở trạng thái màu đỏ. Thay vì ở lại vị trí hiện tại, đại lý nên đi theo những gì dường như là con đường tốt nhất để đạt được mục tiêu dựa trên ước tính chi phí hiện tại cho các nước láng giềng của nó. Chi phí ước tính để đạt được mục tiêu thông qua một người hàng xóm là chi phí để đạt được s cộng với chi phí ước tính để đạt được mục tiêu từ đó — tức là $c(s, a, s') + H(s')$. Trong ví dụ, có hai hành động, với chi phí ước tính $1 + 9$ sang trái và $1 + 2$ sang phải, vì vậy có vẻ tốt nhất là di chuyển sang phải.

Trong (b) rõ ràng là ước tính chi phí 2 cho trạng thái đỏ trong (a) là quá lạc quan. Vì nước đi tốt nhất có giá 1 và dẫn đến trạng thái cách mục tiêu ít nhất 2 bước, trạng thái đỏ phải cách mục tiêu ít nhất 3 bước, vì vậy H của nó phải được cập nhật cho phù hợp, như thể hiện trong Hình 4.23 (b). Tiếp tục quá trình này, tác nhân sẽ di chuyển qua lại hai lần nữa, cập nhật H mỗi lần và "lạm phẳng" mức tối thiểu cục bộ cho đến khi nó thoát ra bên phải.

Một tác nhân thực hiện sơ đồ này, được gọi là học A^* thời gian thực (LRTA*), là được hiển thị trong Hình 4.24. Giống như ONLINE-DFS-AGENT, nó xây dựng một bản đồ của môi trường trong bảng kết quả. Nó cập nhật ước tính chi phí cho tiểu bang mà nó vừa rời đi và sau đó chọn động thái "rõ ràng là tốt nhất" theo ước tính chi phí hiện tại của nó. Một chi tiết quan trọng là các hành động chưa được thử nghiệm trong một trạng thái s luôn được giả định là dẫn đến ngay lập tức mục tiêu với chi phí thấp nhất có thể, cụ thể là $h(s)$. Sự lạc quan trong điều kiện không chắc chắn này khuyến khích

Lạc quan dưới sự không chắc chắn

tác nhân để khám phá những con đường mới, có thể đầy hứa hẹn.

hàm $LRTA^*-AGENT(problem, s', h)$ trả về một hành động

s, a , trạng thái và hành động trước đó, ban đầu nullresult, ánh xạ bằng (s, a) đến s' , ban đầu trống, ánh xạ bằng s với ước tính chi phí, ban đầu trống

nếu $IS-GOAL(s')$ thì trả về stopif s' là một

trạng thái mới (không phải trong H) thì

$H[s'] \leftarrow h(s')$ nếu s không rỗng thì

kết quả $[s, a] \leftarrow s'H[s] \leftarrow \min_{b \in ACTIONS(s)} LRTA^*-$

$COST(problem, s, b, result[s, b], H)a \leftarrow \argmin$

$LRTA^*-COST(vấn đề, s', b, kết quả [s', b], H)$

$b \in$ HÀNH ĐỘNG

$s \leftarrow s'$ return hàm $LRTA^*-COST(problem, s, a, s', H)$ trả về

ước tính chi phí

nếu s' không được xác định thì trả về $h(s)$ else trả

về vấn đề. CHI PHÍ HÀNH ĐỘNG $(S, A, S') + H[s']$

Hình 4.24 $LRTA^*-AGENT$ chọn một hành động theo giá trị của các trạng thái lân cận, được cập nhật khi tác nhân di chuyển xung quanh không gian trạng thái.

Một nhân viên $LRTA^*$ được đảm bảo sẽ tìm thấy mục tiêu trong bất kỳ môi trường hữu hạn, an toàn nào có thể khám phá. Tuy nhiên, không giống như A^* , nó không hoàn chỉnh đối với không gian trạng thái vô hạn - có những trường hợp nó có thể đi lạc vô hạn. Nó có thể khám phá một môi trường của n trạng thái trong các bước $O(n^2)$ trong trường hợp xấu nhất, nhưng thường làm tốt hơn nhiều. Tác nhân $LRTA^*$ chỉ là một trong một trong một nhóm lớn các tác nhân trực tuyến mà người ta có thể xác định bằng cách chỉ định quy tắc lựa chọn hành động và quy tắc cập nhật theo nhiều cách khác nhau. Chúng ta thảo luận về họ này, được phát triển ban đầu cho môi trường ngẫu nhiên, trong Chương 23.

4.5.4 Học trong tìm kiếm trực tuyến

Sự thiếu hiểu biết ban đầu của các đại lý tìm kiếm trực tuyến cung cấp một số cơ hội để học hỏi. Đầu tiên, các tác nhân học một "bản đồ" của môi trường - chính xác hơn là kết quả của mỗi hành động trong mỗi trạng thái - chỉ bằng cách ghi lại từng trải nghiệm của họ. Thứ hai, các đại lý tìm kiếm địa phương có được ước tính chính xác hơn về chi phí của mỗi tiểu bang bằng cách sử dụng các quy tắc cập nhật địa phương, như trong $LRTA^*$. Trong Chương 23, chúng tôi chỉ ra rằng những cập nhật này cuối cùng hội tụ đến các giá trị chính xác cho mọi trạng thái, miễn là tác nhân khám phá không gian trạng thái theo đúng cách. Một khi các giá trị chính xác được biết, các quyết định tối ưu có thể được đưa ra đơn giản bằng cách chuyển sang người kế nhiệm có chi phí thấp nhất - nghĩa là, leo đồi thuần túy sau đó là một chiến lược tối ưu.

Nếu bạn làm theo gợi ý của chúng tôi để theo dõi hành vi của $ONLINE-DFS-AGENT$ trong môi trường của Hình 4.19, bạn sẽ nhận thấy rằng tác nhân không sáng lắm. Ví dụ, sau khi nó đã thấy rằng hành động Lên đi từ $(1,1)$ đến $(1,2)$, tác nhân vẫn không biết rằng hành động Xuống quay trở lại $(1,1)$ hoặc hành động Lên cũng đi từ $(2,1)$ đến $(2,2)$, từ $(2,2)$ đến $(2,3)$, v.v. Nói chung, chúng ta muốn tác nhân biết rằng Uptăng tọa độ y trừ khi có một bức tường cản đường, Down giảm nó, v.v.

Để điều này xảy ra, chúng ta cần hai điều. Đầu tiên, chúng ta cần một đại diện chính thức và rõ ràng có thể thao túng cho các loại quy tắc chung này; Cho đến nay, chúng tôi đã giấu thông tin bên trong

hộp đen được gọi là hàm RESULT. Các chương 8 đến 11 được dành cho vấn đề này. Thứ hai, chúng ta cần các thuật toán có thể xây dựng các quy tắc chung phù hợp từ các quan sát cụ thể do tác nhân thực hiện. Những điều này được đề cập trong Chương 19.

Nếu chúng ta dự đoán rằng chúng ta sẽ được kêu gọi giải quyết nhiều vấn đề tương tự trong tương lai thì sẽ có ý nghĩa khi đầu tư thời gian (và bộ nhớ) để làm cho những tìm kiếm trong tương lai dễ dàng hơn. Có một số cách để làm điều này, tất cả đều thuộc tiêu đề tìm kiếm gia tăng. Chúng ta

Tìm kiếm gia tăng

có thể giữ cây tìm kiếm trong bộ nhớ và sử dụng lại các phần của nó không thay đổi trong NewProblem. Chúng ta có thể giữ các giá trị h phỏng đoán và cập nhật chúng khi chúng ta có được thông tin mới - vì thế giới đã thay đổi hoặc vì chúng ta đã tính toán một ước tính tốt hơn. Hoặc chúng ta có thể giữ các giá trị g tốt nhất, sử dụng chúng để ghép lại với nhau một giải pháp mới và cập nhật chúng khi thế giới thay đổi.

Tóm tắt

Chương này đã xem xét các thuật toán tìm kiếm các vấn đề trong môi trường có thể quan sát được một phần, không xác định, không biết và liên tục.

- Các phương pháp tìm kiếm địa phương như leo đồi chỉ giữ một số lượng nhỏ các tiểu bang trong bộ nhớ. Chúng đã được áp dụng cho các bài toán tối ưu hóa, trong đó ý tưởng là tìm trạng thái điểm cao, mà không phải lo lắng về đường dẫn đến trạng thái. Một số thuật toán tìm kiếm cục bộ ngẫu nhiên đã được phát triển, bao gồm cả ủ mô phỏng, trả về các giải pháp tối ưu khi được đưa ra một lịch trình làm mát thích hợp.
- Nhiều phương pháp tìm kiếm cục bộ cũng áp dụng cho các vấn đề trong không gian liên tục. Các bài toán pro-gramming tuyến tính và tối ưu hóa lồi tuân theo những hạn chế nhất định về hình dạng của không gian trạng thái và bản chất của hàm mục tiêu, và thừa nhận các thuật toán đa thức thời gian thường cực kỳ hiệu quả trong thực tế. Đối với một số bài toán được hình thành tốt về mặt toán học, chúng ta có thể tìm mức tối đa bằng cách sử dụng giải tích để tìm nơi gradient bằng không; Đối với các vấn đề khác, chúng ta phải làm với gradient thực nghiệm, đo lường sự khác biệt về độ phù hợp giữa hai điểm gần đó.
- Thuật toán tiến hóa là một tìm kiếm leo đồi ngẫu nhiên trong đó một quần thể các quốc gia được duy trì. Các trạng thái mới được tạo ra bởi đột biến và bởi sự giao thoa, kết hợp các cặp trạng thái.
- Trong môi trường không xác định, các tác nhân có thể áp dụng tìm kiếm AND-OR để tạo ra các kế hoạch liên tục đạt được mục tiêu bất kể kết quả nào xảy ra trong quá trình thực hiện. Khi môi trường có thể quan sát được một phần, trạng thái niềm tin đại diện cho tập hợp các trạng thái có thể xảy ra mà tác nhân có thể ở trong.
- Các thuật toán tìm kiếm tiêu chuẩn có thể được áp dụng trực tiếp cho không gian trạng thái niềm tin để giải quyết các vấn đề không có cảm biến và tìm kiếm trạng thái tin tưởng VÀ-OR có thể giải quyết các vấn đề chung có thể quan sát được một phần. Các thuật toán gia tăng xây dựng các giải pháp theo trạng thái trong một trạng thái tín ngưỡng thường hiệu quả hơn.
- Các vấn đề thăm dò phát sinh khi tác nhân không có ý tưởng về trạng thái và hành động của môi trường của nó. Đối với môi trường có thể khám phá an toàn, các đại lý tìm kiếm trực tuyến có thể xây dựng bản đồ và tìm mục tiêu nếu có. Cập nhật các ước tính heuristic từ kinh nghiệm cung cấp phương pháp hiệu quả để thoát khỏi mức tối thiểu cục bộ.

Ghi chú thư mục và lịch sử

Kỹ thuật tìm kiếm cục bộ có lịch sử lâu đời trong toán học và khoa học máy tính. Thật vậy, phương pháp Newton-Raphson (Newton, 1671; Raphson, 1690) có thể được coi là một phương pháp tìm kiếm cục bộ rất hiệu quả cho các không gian liên tục trong đó có thông tin gradient. Brent (1973) là một tài liệu tham khảo cổ điển cho các thuật toán tối ưu hóa không yêu cầu thông tin như vậy. Tìm kiếm chùm tia, mà chúng tôi đã trình bày như một thuật toán tìm kiếm cục bộ, bắt nguồn như một biến thể có chiều rộng giới hạn của lập trình động để nhận dạng giọng nói trong hệ thống HARPY (Lowerre, 1976). Một thuật toán liên quan được phân tích sâu bởi Pearl (1984, Ch. 5).

Chủ đề tìm kiếm cục bộ đã được tiếp thêm sinh lực vào đầu những năm 1990 bởi kết quả tốt đáng ngạc nhiên cho các vấn đề thỏa mãn ràng buộc lớn như n -queens (Minton et al., 1992) và Booleansatisfiability (Selman et al., 1992) và bằng cách kết hợp tính ngẫu nhiên, nhiều tìm kiếm đồng thời và các cải tiến khác. Sự phục hưng này của những gì Christos Papadimitriou gọi là các thuật toán "Thời đại mới" cũng làm dấy lên sự quan tâm ngày càng tăng của các nhà khoa học máy tính lý thuyết (Koutsoupias và Papadimitriou, 1992; Aldous và Vazirani, 1994).

Tìm kiếm Tabu

Trong lĩnh vực nghiên cứu hoạt động, một biến thể của leo đồi được gọi là tìm kiếm tabu đã đạt được phổ biến (Glover và Laguna, 1997). Thuật toán này duy trì một danh sách cấm k, các trạng thái đã truy cập trước đó không thể xem lại; Cũng như cải thiện hiệu quả khi tìm kiếm đồ thị, danh sách này có thể cho phép thuật toán thoát khỏi một số mức tối thiểu cục bộ.

Một cải tiến hữu ích khác về leo đồi là thuật toán STAGE (Boyan và Moore, 1998). Ý tưởng là sử dụng mức tối đa cục bộ được tìm thấy bằng cách leo đồi khởi động lại ngẫu nhiên để có được ý tưởng về hình dạng tổng thể của cảnh quan. Thuật toán phù hợp với một bề mặt bậc hai mịn với tập hợp của cực đại cục bộ và sau đó tính toán cực đại toàn cầu của bề mặt đó bằng cách phân tích. Đây trở thành điểm khởi động lại mới. Gomes et al. (1998) đã chỉ ra rằng thời gian chạy của

Phân phối đuôi nặng

các thuật toán quay ngược hệ thống thường có một phân bố đuôi nặng, có nghĩa là

Xác suất của một thời gian chạy rất dài là nhiều hơn dự đoán nếu thời gian chạy được phân phối theo chủ thể. Khi phân phối thời gian chạy bị đuôi nặng, khởi động lại ngẫu nhiên sẽ tìm thấy giải pháp nhanh hơn, trung bình, so với một lần chạy duy nhất để hoàn thành. Hoos và Stützle (2004) cung cấp một cuốn sách bao quát về chủ đề này.

Ủ mô phỏng lần đầu tiên được mô tả bởi Kirkpatrick et al. (1983), người vay mượn trực tiếp từ thuật toán Metropolis (được sử dụng để mô phỏng inphysics các hệ thống phức tạp (Metropolis et al., 1953) và được cho là được phát minh tại một bữa tiệc tối ở Los Alamos). Ủ mô phỏng hiện là một lĩnh vực, với hàng trăm bài báo được xuất bản mỗi năm.

Tìm ra các giải pháp tối ưu trong không gian liên tục là chủ đề của một số lĩnh vực, bao gồm lý thuyết tối ưu hóa, lý thuyết điều khiển tối ưu và phép tính các biến thể. Các kỹ thuật cơ bản được giải thích tốt bởi Bishop (1995); Press et al. (2007) bao gồm một loạt các thuật toán và cung cấp phần mềm hoạt động.

Các nhà nghiên cứu đã lấy cảm hứng cho các thuật toán tìm kiếm và tối ưu hóa từ nhiều lĩnh vực nghiên cứu: luyện kim (ủ mô phỏng); sinh học (thuật toán di truyền); khoa học thần kinh (mạng thần kinh); leo núi (leo đồi); kinh tế học (thuật toán dựa trên thị trường (Dias et al., 2006)); vật lý (bầy hạt (Li và Yao, 2012) và kính quay (Mézard et al., 1987)); hành vi động vật (học tăng cường, tối ưu hóa sói xám (Mir-jalili và Lewis, 2014)); điều học (tìm kiếm chim cu gáy (Yang và Deb, 2014)); côn trùng học (đàn kiến (Dorigo và cộng sự, 2008), đàn ong (Karaboga và Basturk, 2007), đom đóm (Yang, 2009) và giun phát sáng (Krishnanand và Ghose, 2009) tối ưu hóa); và những người khác.

Lập trình tuyến tính (LP) lần đầu tiên được nghiên cứu một cách có hệ thống bởi nhà toán học Leonid Kantorovich (1939). Đó là một trong những ứng dụng đầu tiên của máy tính; algo-rithm đơn giản (Dantzig, 1949) vẫn được sử dụng mặc dù độ phức tạp theo cấp số nhân trong trường hợp xấu nhất. Karmarkar (1984) đã phát triển họ phương pháp điểm nội thất hiệu quả hơn nhiều, được chứng minh là có độ phức tạp đa thức cho lớp tổng quát hơn của các bài toán tối ưu hóa lồi bởi Nesterov và Nemirovski (1994). Những giới thiệu tuyệt vời về tối ưu hóa lồi được cung cấp bởi Ben-Tal và Nemirovski (2001) và Boyd và Vandenberghe (2004).

Công trình của Sewall Wright (1931) về khái niệm cảnh quan thể dục là tiền thân quan trọng cho sự phát triển của các thuật toán di truyền. Trong những năm 1950, một số nhà thống kê, bao gồm Box (1957) và Friedman (1959), đã sử dụng các kỹ thuật tiến hóa cho các vấn đề tối ưu hóa, nhưng phải đến khi Rechenberg (1965) giới thiệu các chiến lược tiến hóa để giải quyết các vấn đề optimization cho cánh máy bay, cách tiếp cận này mới trở nên phổ biến. Trong những năm 1960 và 1970, John Holland (1975) đã ủng hộ các thuật toán di truyền, cả hai đều là một công cụ tối ưu hóa hữu ích và một phương pháp để mở rộng hiểu biết của chúng ta về sự thích nghi (Holland, 1995).

Phong trào sự sống nhân tạo (Langton, 1995) đã đưa ý tưởng này tiến thêm một bước nữa, xem các sản phẩm của thuật toán di truyền là sinh vật hơn là giải pháp cho các vấn đề. Hiệu ứng Baldwin được thảo luận trong chương được đề xuất gần như đồng thời bởi Conwy Lloyd Morgan (1896) và James (Baldwin, 1896). Mô phỏng máy tính đã giúp làm rõ ý nghĩa của nó (Hinton và Nowlan, 1987; Ackley và Littman, 1991; Morgan và Grif-fiths, 2015). Smith và Szathm'ary (1999), Ridley (2004) và Carroll (2007) cung cấp nền tảng chung về tiến hóa.

Hầu hết các so sánh các thuật toán di truyền với các phương pháp tiếp cận khác (đặc biệt là leo đồi ngẫu nhiên) đã phát hiện ra rằng các thuật toán di truyền hội tụ chậm hơn (O'Reilly và Op-pacher, 1994; Mitchell và cộng sự, 1996; Juels và Wattenberg, 1996; Baluja, 1997). Những phát hiện như vậy không phổ biến rộng rãi trong cộng đồng GA, nhưng những nỗ lực gần đây trong cộng đồng đó để hiểu tìm kiếm dựa trên dân số như một hình thức gần đúng của học Bayes (xem Chương 21) có thể giúp thu hẹp khoảng cách giữa lĩnh vực này và những người chỉ trích nó (Pelikan et al., 1999). Lý thuyết về các hệ động lực học bậc hai cũng có thể giải thích hiệu suất của GA (Rabani và cộng sự, 1998). Có một số ứng dụng thực tế ấn tượng của GA, trong các lĩnh vực đa dạng như thiết kế ăng-ten (Lohn và cộng sự, 2001), thiết kế có sự hỗ trợ của máy tính (Renner và Ekart, 2003), mô hình khí hậu (Stanislawska và cộng sự, 2015), y học (Ghaheeri và cộng sự, 2015) và khử ký kết mạng nơ-ron sâu (Miikkulainen và cộng sự, 2019).
 Lĩnh vực lập trình di truyền là một lĩnh vực con của các thuật toán di truyền trong đó các đại diện là các chương trình chứ không phải chuỗi bit. Các chương trình được thể hiện dưới dạng cây cú pháp, bằng ngôn ngữ lập trình tiêu chuẩn hoặc ở các định dạng được thiết kế đặc biệt để đại diện cho các mạch điện tử, bộ điều khiển rô-bốt, v.v. Sự giao thoa liên quan đến việc nối các cây con để lấy theo cách mà con cái được đảm bảo là những biểu hiện được hình thành tốt.

Sự quan tâm đến lập trình di truyền được thúc đẩy bởi công trình của John Koza (1992, 1994), nhưng ít nhất nó quay trở lại các thí nghiệm ban đầu với mã máy của Friedberg (1958) và với máy tự động trạng thái hữu hạn của Fogel et al. (1966). Cũng như các thuật toán di truyền, có cuộc tranh luận về hiệu quả của kỹ thuật này. Koza et al. (1999) mô tả các thí nghiệm trong việc sử dụng lập trình di truyền để thiết kế các thiết bị mạch.
 Các tạp chí *Evolutionary Computation* và *IEEE Transactions on Evolutionary Computation* bao gồm các thuật toán tiến hóa; các bài viết cũng được tìm thấy trong *Hệ thống phức tạp*, *Hành vi thích ứng* và *Cuộc sống nhân tạo*. Hội nghị chính là Ủy ban Di truyền và Tiến hóa

Hội nghị trình diễn (GECCO). Các văn bản tổng quan tốt về các thuật toán di truyền bao gồm những văn bản của Mitchell (1996), Fogel (2000), Langdon and Poli (2002) và Poli et al. (2008).

Tính không thể đoán trước và khả năng quan sát một phần của môi trường thực đã được công nhận sớm trong các dự án robot sử dụng các kỹ thuật lập kế hoạch, bao gồm Shakey (Fikes et al., 1972) và FREDDY (Michie, 1972). Các vấn đề đã nhận được nhiều sự chú ý hơn sau khi xuất bản bài báo có ảnh hưởng của McDermott (1978a) Quy hoạch và Hành động.

Công trình đầu tiên sử dụng rõ ràng các cây AND-OR đường như là chương trình SAINT của Slagle để tích hợp biểu tượng, được đề cập trong Chương 1. Amarel (1967) đã áp dụng ý tưởng để chứng minh định lý mệnh đề, một chủ đề được thảo luận trong Chương 7, và giới thiệu một thuật toán tìm kiếm tương tự như AND-OR-GRAPH-SEARCH. Thuật toán được phát triển thêm bởi Nilsson (1971), người cũng mô tả AO* — như tên gọi của nó, tìm thấy các giải pháp tối ưu. AO* được cải thiện hơn nữa bởi Martelli và Montanari (1973).

AO* là một thuật toán từ trên xuống; tổng quát hóa từ dưới lên của A* là A*LD, cho A* Light-est Derivation (Felzenszwalb và McAllester, 2007). Sự quan tâm đến tìm kiếm AND-OR đã phục hồi vào đầu những năm 2000, với các thuật toán mới để tìm ra các giải pháp tương hoàn (Jimenez và Torras, 2000; Hansen và Zilberstein, 2001) và các kỹ thuật mới lấy cảm hứng từ chương trình năng động (Bonet và Geffner, 2005).

Ý tưởng chuyển đổi các vấn đề có thể quan sát được một phần thành các vấn đề trạng thái niềm tin ban đầu với Astrom (1965) cho trường hợp phức tạp hơn nhiều của sự không chắc chắn xác suất (xem Chương 16). Erdmann và Mason (1988) đã nghiên cứu vấn đề thao tác robot mà không có cảm biến, sử dụng một hình thức tìm kiếm trạng thái niềm tin liên tục. Họ chỉ ra rằng có thể định hướng một bộ phận trên bàn từ một vị trí ban đầu tùy ý bằng một chuỗi các hành động nghiên cứu được thiết kế tốt. Các phương pháp thực tế hơn, dựa trên một loạt các rào cản chèo được định hướng chính xác trên băng tải, sử dụng cùng một hiểu biết về thuật toán (Wiegley et al., 1996).

Cách tiếp cận trạng thái niềm tin đã được phát minh lại trong bối cảnh các vấn đề tìm kiếm không có cảm biến và có thể quan sát được một phần bởi Genesereth và Nourbakhsh (1993). Công việc bổ sung đã được thực hiện về các vấn đề không cảm biến trong cộng đồng lập kế hoạch dựa trên logic (Goldman và Boddy, 1996; Smith và Weld, 1998). Tác phẩm này đã nhấn mạnh các đại diện ngắn gọn cho các trạng thái niềm tin, như đã giải thích trong Chương 11. Bonet và Geffner (2000) đã giới thiệu các phương pháp phỏng đoán hiệu quả đầu tiên để tìm kiếm trạng thái niềm tin; chúng đã được tinh chỉnh bởi Bryce et al. (2006). Cách tiếp cận gia tăng để tìm kiếm trạng thái niềm tin, trong đó các giải pháp được xây dựng dần dần cho các tập hợp con của các trạng thái trong mỗi trạng thái niềm tin, đã được nghiên cứu trong tài liệu lập kế hoạch bởi Kurien et al. (2002); một số thuật toán gia tăng mới đã được giới thiệu cho các xác suất không xác định, có thể quan sát được một phần bởi Russell và Wolfe (2005). Các tài liệu tham khảo bổ sung cho việc lập kế hoạch trong môi trường ngẫu nhiên, có thể quan sát một phần xuất hiện trong Chương 16.

Các thuật toán để khám phá các không gian trạng thái chưa biết đã được quan tâm trong nhiều thế kỷ. Tìm kiếm đồ sộ đầu tiên trong mê cung có thể đảo ngược có thể được thực hiện bằng cách giữ tay trái của một người trên tường; có thể tránh các vòng lặp bằng cách đánh dấu từng điểm nổi. Vấn đề chung hơn của việc khám phá đồ thị Eulerian (tức là đồ thị trong đó mỗi nút có số lượng đến bằng nhau

và các cạnh đi) đã được giải quyết bằng một thuật toán do Hierholzer

(1873) cứu thuật toán kỹ lưỡng đầu tiên về vấn đề thăm dò cho đồ thị tùy ý được thực hiện bởi Deng và Papadimitriou (1990), những người đã phát triển một thuật toán hoàn toàn quát nhưng chỉ ra rằng không có tỷ lệ cạnh tranh giới hạn nào có thể để khám phá một đồ thị tổng quát. Papadimitriou và Yannakakis (1991) đã xem xét câu hỏi tìm ra con đường dẫn đến một môi trường lập kế hoạch đường đi trong mục tiêu (nơi tất cả các hành động đều có thể đảo ngược). Họ đã cho thấy rằng

Một tỷ lệ cạnh tranh nhỏ có thể đạt được với các chương ngại vật vuông, nhưng với các chương ngại vật hình chữ nhật nói chung, không thể đạt được tỷ lệ giới hạn. (Xem Hình 4.20.)

Trong một môi trường năng động, trạng thái của thế giới có thể thay đổi một cách tự nhiên mà không cần bất kỳ hành động nào của tác nhân. Ví dụ: đại lý có thể lên kế hoạch cho một tuyến đường lái xe tối ưu từ A đến B, nhưng một tai nạn hoặc giao thông giờ cao điểm xấu bất thường có thể làm hỏng kế hoạch. Các thuật toán tìm kiếm gia tăng như Lập kế hoạch trọn đời A* (Koenig et al., 2004) và D* Lite (Koenig và Likhachev, 2002) giải quyết tình huống này.

Thuật toán LRTA* được phát triển bởi Korf (1990) như một phần của cuộc điều tra tìm kiếm thời gian thực cho các môi trường trong đó tác nhân phải hành động sau khi tìm kiếm chỉ trong một khoảng thời gian cố định (một tình huống phổ biến trong trò chơi hai người chơi). LRTA* trên thực tế là một trường hợp đặc biệt của các thuật toán học tăng cường cho môi trường ngẫu nhiên (Barto et al., 1995). Chính sách lạc quan của nó trong điều kiện không chắc chắn - luôn hướng đến trạng thái chưa được truy cập gần nhất - có thể dẫn đến một mô hình thăm dò kém hiệu quả hơn trong trường hợp thiếu thông tin so với tìm kiếm độ sâu đầu tiên đơn giản (Koenig, 2000). Dasgupta et al. (1994) chỉ ra rằng tìm kiếm sâu lặp đi lặp lại trực tuyến là hiệu quả tối ưu để tìm mục tiêu trong một cây thống nhất không có thông tin phỏng đoán.

Một số biến thể được thông báo về chủ đề LRTA* đã được phát triển với các phương pháp khác nhau để tìm kiếm và cập nhật trong phần đã biết của biểu đồ (Pemberton và Korf, 1992). Cho đến nay, vẫn chưa có sự hiểu biết lý thuyết tốt về cách tìm mục tiêu với hiệu quả tối ưu khi sử dụng thông tin phỏng đoán. Sturtevant và Bulitko (2016) cung cấp phân tích về một số cạm bẫy xảy ra trong thực tế.

CHƯƠNG 5

SỰ THỎA MÃN RÀNG BUỘC vấn đề

Trong đó chúng ta thấy cách coi các trạng thái không chỉ là hộp đen nhỏ dẫn đến các phương pháp tìm kiếm mới và hiểu sâu hơn về cấu trúc vấn đề.

Chương 3 và 4 khám phá ý tưởng rằng các vấn đề có thể được giải quyết bằng cách tìm kiếm không gian trạng thái: một đồ thị trong đó các nút là trạng thái và các cạnh giữa chúng là các hành động. Chúng tôi thấy rằng các phương pháp phỏng đoán theo tên miền cụ thể có thể ước tính chi phí đạt được mục tiêu từ một trạng thái nhất định, nhưng từ quan điểm của thuật toán tìm kiếm, mỗi trạng thái là nguyên tử, hoặc không thể phân chia - một hộp đen không có cấu trúc bên trong. Đối với mỗi vấn đề, chúng ta cần mã miền cụ thể để mô tả quá trình chuyển đổi giữa các trạng thái.

Trong chương này, chúng ta mở hộp đen bằng cách sử dụng một biểu diễn thừa số cho mỗi trạng thái: một tập hợp các biến, mỗi biến có một giá trị. Một bài toán được giải quyết khi mỗi biến có một giá trị thỏa mãn tất cả các ràng buộc đối với biến. Một vấn đề được mô tả theo cách này được gọi là vấn đề thỏa mãn ràng buộc, hoặc CSP.

Các thuật toán tìm kiếm CSP tận dụng cấu trúc của trạng thái và sử dụng các phương pháp phỏng đoán chung chứ không phải miền cụ thể để cho phép giải quyết các vấn đề phức tạp. Ý tưởng chính là loại bỏ tất cả các phần lớn của không gian tìm kiếm cùng một lúc bằng cách xác định các kết hợp biến / giá trị vi phạm các ràng buộc. CSP có thêm lợi thế là hành động và mô hình chuyển đổi có thể được suy ra từ mô tả vấn đề.

5.1 Xác định các vấn đề thỏa mãn ràng buộc

Bài toán thỏa mãn ràng buộc bao gồm ba thành phần, X , D và C :

X là một tập hợp các biến, $\{X_1, \dots, X_n\}$. D là một tập hợp các miền, $\{D_1, \dots, D_n\}$, một cho mỗi biến. C là một tập hợp các ràng buộc chỉ định các kết hợp giá trị cho phép. Một miền, D_i , bao gồm một tập hợp các giá trị cho phép, $\{v_1, \dots, v_k\}$, cho biến X_i . Ví dụ, một biến Boolean sẽ có miền $\{\text{true}, \text{false}\}$. Các biến khác nhau có thể có các miền khác nhau với kích thước khác nhau. Mỗi ràng buộc C_j bao gồm một cặp $\langle \text{scope}, \text{rel} \rangle$, trong đó scope là một bộ các biến tham gia vào ràng buộc và rel là một mối quan hệ de-

phạt các giá trị mà các biến đó có thể đảm nhận. Một mối quan hệ có thể được biểu diễn dưới dạng một tập hợp ví dụ của tất cả các bộ giá trị thỏa mãn ràng buộc hoặc như một hàm có thể tính toán xem một bộ có phải là một thành viên của mối quan hệ hay không. Ví dụ, nếu X_1 và X_2 đều có $\text{do-main} \{1, 2, 3\}$, thì ràng buộc nói rằng X_1 phải lớn hơn X_2 có thể được viết là $\langle (X_1, X_2), \{(3, 1), (3, 2), (2, 1)\} \rangle$ hoặc là $\langle (X_1, X_2), X_1 > X_2 \rangle$.

Vấn đề thỏa mãn ràng buộc

Mối quan hệ

CSP xử lý việc gán giá trị cho các biến, $\{X_i = v_i, X_j = v_j, \dots\}$. Một bài tập không vi phạm bất kỳ ràng buộc nào được gọi là nhiệm vụ nhất quán hoặc hợp pháp. Một hoàn chỉnh nhiệm vụ là một trong đó mỗi biến được gán một giá trị và một giải pháp cho CSP là một nhiệm vụ nhất quán, hoàn chỉnh. Một bài tập một phần là một tập trong những đề lại một số biến số không được gán và giải pháp một phần là một sự gán một phần nhất quán. Giải quyết CSP nói chung là một bài toán NP-complete, mặc dù có các lớp con quan trọng của CSP có thể được giải quyết rất hiệu quả.	Tập Phù hợp Giải pháp hoàn thành nhiệm vụ Phân công một phần Giải pháp một phần
--	---

5.1.1 Ví dụ về vấn đề: Tô màu bản đồ

Giả sử rằng, sau khi một mối với Romania, chúng ta đang xem bản đồ của Úc hiển thị từng tiểu bang và vùng lãnh thổ của nó (Hình 5.1(a)). Chúng ta được giao nhiệm vụ tô màu cho mỗi vùng màu đỏ, xanh lá cây hoặc xanh lam sao cho không có hai vùng lân cận nào có cùng màu. Để xây dựng điều này như một CSP, chúng tôi xác định các biến là các vùng:

$X = \{WA, NT, Q, NSW, V, SA, T\}$. Miền của mỗi biến là tập $D_i = \{\text{đỏ, xanh lá cây, xanh lam}\}$. Các ràng buộc đòi hỏi các vùng lân cận phải có màu sắc riêng biệt. Vì có chín nơi giáp ranh với các khu vực, nên có chín ràng buộc:

$$C = \{SA \neq WA, SA \neq NT, SA \neq Q, SA \neq NSW, SA \neq V, \\ WA \neq NT, NT \neq Q, Q \neq NSW, NSW \neq V\}.$$

Ở đây chúng tôi đang sử dụng các từ viết tắt; $SA \neq WA$ là một phím tắt cho $\langle (SA, WA), (SA \neq WA) \rangle$, trong đó $SA \neq WA$ có thể được liệt kê đầy đủ lần lượt là

$\{(\text{đỏ, xanh lá cây}), (\text{đỏ, xanh dương}), (\text{xanh lá cây, đỏ}), (\text{xanh lá cây, xanh dương}), (\text{xanh lam, đỏ}), (\text{xanh dương, xanh lá cây})\}$. Có nhiều giải pháp khả thi cho vấn đề này, chẳng hạn như

$\{WA = \text{đỏ}, NT = \text{xanh lá cây}, Q = \text{đỏ}, NSW = \text{xanh lá cây}, V = \text{đỏ}, SA = \text{xanh lam}, T = \text{đỏ}\}$. Có

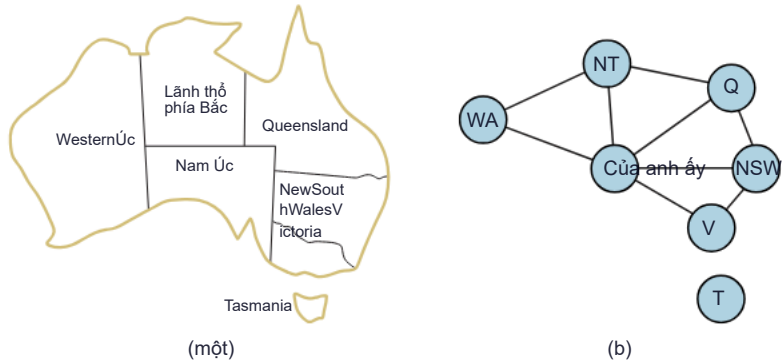
thể hữu ích khi hình dung CSP dưới dạng biểu đồ ràng buộc, như trong Hình 5.1 (b). Các

Biểu đồ ràng buộc

Các nút của đồ thị tương ứng với các biến của vấn đề và một cạnh kết nối bất kỳ hai biến nào tham gia vào một ràng buộc.

Tại sao phải xây dựng một vấn đề với tư cách là CSP? Một lý do là các CSP mang lại sự phản đối tự nhiên cho nhiều vấn đề; thường dễ dàng để xây dựng một vấn đề với tư cách là CSP. Một điều khác là nhiều năm công việc phát triển đã được đưa vào việc làm cho các trình giải CSP nhanh chóng và hiệu quả. Thứ ba là một trình giải CSP có thể nhanh chóng cắt tía các vùng rộng lớn của không gian tìm kiếm mà một người tìm kiếm không gian trạng thái nguyên tử không thể. Ví dụ, một khi chúng ta đã chọn $\{SA = \text{blue}\}$ trong bài toán Úc, chúng ta có thể kết luận rằng không có năm nào trong số năm biến lân cận có thể nhận giá trị màu xanh lam. Một quy trình tìm kiếm không sử dụng các ràng buộc sẽ phải chấp nhận $35 = 243$ gán cho năm biến lân cận; với các ràng buộc, chúng tôi chỉ có $25 = 32$ nhiệm vụ để xem xét, giảm 87%.

Trong tìm kiếm trạng thái-không gian nguyên tử, chúng ta chỉ có thể hỏi: trạng thái cụ thể này có phải là một mục tiêu không? Không? Còn cái này thì sao? Với CSP, một khi chúng ta phát hiện ra rằng một phần gán vi phạm một ràng buộc, chúng ta có thể ngay lập tức loại bỏ các tình hình tiếp theo của việc gán một phần. Hơn nữa, chúng ta có thể thấy tại sao việc gán không phải là một giải pháp - chúng ta thấy biến nào vi phạm ràng buộc - vì vậy chúng ta có thể tập trung sự chú ý vào các biến quan trọng. Kết quả là, nhiều vấn đề khó giải quyết đối với tìm kiếm không gian trạng thái nguyên tử có thể được giải quyết nhanh chóng khi được xây dựng như một CSP.



Hình 5.1 (a) Các tiểu bang và vùng lãnh thổ chính của Úc. Tô màu bản đồ này có thể được coi là một vấn đề thỏa mãn ràng buộc (CSP). Mục tiêu là gán màu cho mỗi re-gion để không có vùng lân cận nào có cùng màu. (b) Vấn đề tô màu bản đồ được biểu diễn dưới dạng biểu đồ ràng buộc.

5.1.2 Vấn đề ví dụ: Lập lịch trình công việc

Các nhà máy có vấn đề về việc sắp xếp các công việc trong một ngày, tùy thuộc vào nhiều ràng buộc khác nhau. Trong thực tế, nhiều vấn đề trong số này được giải quyết bằng các kỹ thuật CSP. Hãy xem xét vấn đề lên lịch lắp ráp một chiếc ô tô. Toàn bộ công việc bao gồm các nhiệm vụ và chúng ta có thể mô hình hóa từng nhiệm vụ như một biến, trong đó giá trị của mỗi biến là thời gian mà nhiệm vụ bắt đầu, được biểu thị bằng số nguyên phút. Các ràng buộc có thể khẳng định rằng một nhiệm vụ phải xảy ra trước một nhiệm vụ khác - ví dụ, một bánh xe phải được lắp đặt trước khi nắp bánh xe được lắp vào - và chỉ có rất nhiều nhiệm vụ có thể diễn ra cùng một lúc. Các ràng buộc cũng có thể chỉ định rằng một nhiệm vụ mất một khoảng thời gian nhất định để hoàn thành.

Chúng tôi xem xét một phần nhỏ của cụm xe, bao gồm 15 nhiệm vụ: lắp trục (trước và sau), gắn cả bốn bánh xe (phải và trái, trước và sau), siết chặt đai ốc cho mỗi bánh xe, gắn nắp bánh xe và kiểm tra cụm cuối cùng. Chúng ta có thể biểu diễn các nhiệm vụ với 15 biến:

$$X = \{\text{AxleF, AxleB, WheelRF, WheelLF, WheelRB, WheelLB, NutsRF, NutsLF, NutsRB, NutsLB, CapRF, CapLF, CapRB, CapLB, Kiểm tra}\}.$$

Precedence constraint Tiếp theo, chúng tôi thể hiện các ràng buộc ưu tiên giữa các nhiệm vụ riêng lẻ. Bất cứ khi nào một nhiệm vụ T1 phải xảy ra trước nhiệm vụ T2 và nhiệm vụ T1 mất thời gian d1 để hoàn thành, chúng ta thêm một ràng buộc số học ở dạng

$$T1 + d1 \leq T2.$$

Trong ví dụ của chúng tôi, các trục phải được đặt đúng vị trí trước khi đặt bánh xe và mất 10 phút để lắp trục, vì vậy chúng tôi viết

$$\begin{aligned} \text{AxleF} + 10 \text{ bánh xe} &\leq \text{RF}; \text{AxleF} + 10 \leq \text{WheelLF}; \\ \text{AxleB} + 10 &\leq \text{WheelRB}; \text{AxleB} + 10 \leq \text{WheelLB}. \end{aligned}$$

Tiếp theo, chúng tôi nói rằng đối với mỗi bánh xe, chúng ta phải gắn bánh xe (mất 1 phút), sau đó siết chặt các đai ốc (2 phút) và cuối cùng gắn nắp bánh xe (1 phút, nhưng chưa được hiển thị):

Bánh xeRF + 1 đai ốc ≤ RF; NutsRF + 2 ≤ CapRF; Bánh xeLF + 1 đai ốc ≤ LF;
NutsLF + 2 ≤ CapLF; Bánh xeRB + 1 đai ốc ≤ RB; Các loại hạtRB + 2 ≤ CapRB;
Bánh xeLB + 1 đai ốc ≤ LB; Các loại hạtLB + 2 ≤ CapLB. Giả sử chúng ta có bốn công nhân để lắp đặt bánh xe, nhưng họ phải chia sẻ một công cụ giúp đặt trực vào đúng vị trí. Chúng ta cần một ràng buộc tách rời để nói rằng AxleF và AxleB không được

Ràng buộc tách rời

chồng chéo về thời gian; một trong hai đến trước hoặc cái kia đến:

(AxleF + 10 ≤ AxleB) hoặc (AxleB + 10 ≤ AxleF). Điều này trông giống như một ràng buộc phức tạp hơn, kết hợp số học và logic. Nhưng nó vẫn giảm xuống một tập hợp các cặp giá trị mà AxleF và AxleB có thể đảm nhận.

Chúng ta cũng cần khẳng định rằng việc kiểm tra đến cuối cùng và mất 3 phút. Đối với mọi biến ngoại trừ Inspect, chúng ta thêm một ràng buộc ở dạng $X + dX \leq \text{Inspect}$. Cuối cùng, giả sử có một yêu cầu phải hoàn thành toàn bộ việc lắp ráp trong 30 phút. Chúng ta có thể đạt được điều đó bằng cách giới hạn miền của tất cả các biến:

$Di = \{0, 1, 2, 3, \dots, 30\}$. Vấn đề cụ thể này rất tầm thường để giải quyết, nhưng CSP đã được áp dụng thành công cho các vấn đề lập lịch trình như thế này với hàng ngàn biến.

5.1.3 Các biến thể của chủ nghĩa hình thức CSP

Loại CSP đơn giản nhất liên quan đến các biến có các miền rời rạc, hữu hạn. Tô màu bản đồ Miền rời rạc Miền hữu hạn
Các vấn đề và lịch trình với giới hạn thời gian đều thuộc loại này. Bài toán 8 ong chúa (Hình 4.3) cũng có thể được xem như một CSP miền hữu hạn, trong đó các biến $Q_1, \dots, Q_8$ tương ứng với các nữ hoàng trong cột 1 đến 8 và miền của mỗi biến chỉ định số hàng có thể có cho nữ hoàng trong cột đó, $Di = \{1, 2, 3, 4, 5, 6, 7, 8\}$. Các ràng buộc nói rằng không có hai nữ hoàng nào có thể ở cùng một hàng hoặc đường chéo.

Một miền rời rạc có thể là vô hạn, chẳng hạn như tập hợp các số nguyên hoặc chuỗi. (Nếu chúng tôi không đặt

Vô tận

thời hạn cho vấn đề lập lịch công việc, sẽ có vô số thời gian bắt đầu cho mỗi biến.) Với các miền vô hạn, chúng ta phải sử dụng các ràng buộc ngầm như $T_1 + d_1 \leq T_2$ thay vì các bộ giá trị rõ ràng. Các thuật toán giải pháp đặc biệt (mà chúng ta không thảo luận ở đây) tồn tại đối với các ràng buộc tuyến tính trên các biến số nguyên — nghĩa là, các ràng buộc, chẳng hạn như ràng buộc vừa đưa ra, trong mà mỗi biến chỉ xuất hiện ở dạng tuyến tính. Có thể chỉ ra rằng không có thuật toán nào tồn tại để giải quyết các ràng buộc phi tuyến chung trên các biến số nguyên - vấn đề là không thể quyết định.

Ràng buộc tuyến tính

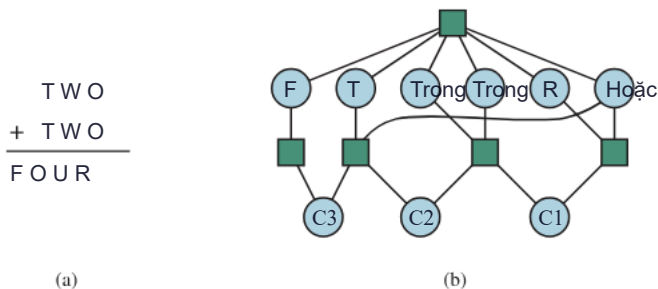
Các ràng buộc phi tuyến tính

Các vấn đề thỏa mãn ràng buộc với các miền liên tục là phổ biến trong thế giới thực

Miền liên tục

và được nghiên cứu rộng rãi trong lĩnh vực nghiên cứu hoạt động. Ví dụ, việc lên lịch cho các thử nghiệm trên Kính viễn vọng Không gian Hubble đòi hỏi thời gian quan sát rất chính xác; Bắt đầu và kết thúc của mỗi quan sát và thao tác là các biến có giá trị liên tục phải tuân theo nhiều ràng buộc về thiên văn, ưu tiên và sức mạnh. Loại CSP miền liên tục nổi tiếng nhất là các bài toán lập trình tuyến tính, trong đó các ràng buộc phải là bằng hoặc bất đẳng thức tuyến tính. Các bài toán lập trình tuyến tính có thể được giải quyết bằng đa danh nghĩa thời gian về số biến. Các vấn đề với các loại ràng buộc và hàm mục tiêu khác nhau cũng đã được nghiên cứu - lập trình bậc hai, lập trình hình nón bậc hai, v.v. Những vấn đề này tạo thành một lĩnh vực quan trọng của toán học ứng dụng.

Ràng buộc đơn nguyên	Ngoài việc kiểm tra các loại biến có thể xuất hiện trong CSP, sẽ rất hữu ích khi xem xét các loại ràng buộc. Loại đơn giản nhất là ràng buộc đơn nguyên, hạn chế giá trị của một biến duy nhất. Ví dụ, trong vấn đề tô màu bản đồ, có thể là trường hợp người Nam Úc sẽ không chấp nhận màu xanh lá cây; chúng ta có thể biểu thị điều đó với ràng buộc đơn sơ 項 $\langle (SA), SA \neq \text{màu xanh lá cây} \rangle$. (Đặc tả ban đầu của miền của một biến cũng có thể được coi là một ràng buộc đơn hạn.)
Ràng buộc nhị phân CSP nhị phân	Một ràng buộc nhị phân liên quan đến hai biến. Ví dụ: $SA \neq NSW$ là một ràng buộc nhị phân. CSP nhị phân là một CSP chỉ có các ràng buộc đơn nguyên và nhị phân; nó có thể được biểu diễn dưới dạng một biểu đồ ràng buộc, như trong Hình 5.1 (b). Chúng ta cũng có thể xác định các ràng buộc bậc cao hơn. Ví dụ, ràng buộc bậc ba Between(X, Y, Z), có thể được định nghĩa là $\langle (X, Y, Z), X < Y < Z \text{ hoặc } X > Y > Z \rangle$.
Ràng buộc toàn cục	Một ràng buộc liên quan đến một số biến tùy ý được gọi là ràng buộc toàn cục. (Tên truyền thống nhưng khó hiểu vì một ràng buộc toàn cục không cần phải liên quan đến tất cả các biến trong một vấn đề). Một trong những ràng buộc toàn cục phổ biến nhất là AllDiff, nói rằng tất cả các biến liên quan đến ràng buộc phải có các giá trị khác nhau. Trong Sudokuproblems (xem Phần 5.2.6), tất cả các biến trong một hàng, cột hoặc hộp 3×3 phải đáp ứng ràng buộc AllDiff.
Cryptarithmic	Một ví dụ khác được cung cấp bởi các câu đố cryptarithmic (Hình 5.2 (a)). Mỗi chữ cái trong Một câu đố cryptarithmic đại diện cho một chữ số khác. Đối với trường hợp trong Hình 5.2 (a), điều này sẽ được biểu diễn dưới dạng ràng buộc toàn cục AllDiff (F, T, U, W, R, O). Các ràng buộc cộng trên bốn cột của câu đố có thể được viết dưới dạng các ràng buộc n sau đây: $\begin{aligned} O + O &= R + 10 \cdot C1C1 \\ + W + W &= U + 10 \cdot \\ C2C2 + T + T &= O + 10 \\ \cdot C3C3 &= F, \end{aligned}$
Constrainthypergraph	trong đó C1, C2 và C3 là các biến phụ đại diện cho chữ số được chuyển vào cột hàng chục, hàng trăm hoặc nghìn. Các ràng buộc này có thể được biểu diễn trong một biểu đồ hypergraph ràng buộc, chẳng hạn như trong Hình 5.2 (b). Một hypergraph bao gồm các nút thông thường (các vòng tròn trong hình) và các siêu nút (các hình vuông), đại diện cho các ràng buộc n-ary — các ràng buộc liên quan đến n biến.
Biểu đồ kép	Ngoài ra, như Bài tập 5.NARY yêu cầu bạn chứng minh, mọi ràng buộc miền hữu hạn có thể được rút gọn thành một tập hợp các ràng buộc nhị phân nếu có đủ các biến phụ được giới thiệu. Điều này có nghĩa là chúng ta có thể chuyển đổi bất kỳ CSP nào thành một chỉ với các ràng buộc nhị phân - điều này làm cho cuộc sống của nhà thiết kế thuật toán trở nên đơn giản hơn. Một cách khác để chuyển đổi một CSP thứ n thành một CSP nhị phân là chuyển đổi đồ thị kép: tạo một đồ thị mới trong đó sẽ có Một biến cho mỗi ràng buộc trong biểu đồ gốc và một ràng buộc nhị phân cho mỗi cặp ràng buộc trong biểu đồ gốc chia sẻ biến. Ví dụ: xem xét một CSP với các biến $X = \{X, Y, Z\}$, mỗi biến có miền $\{1, 2, 3, 4, 5\}$ và với hai ràng buộc C1: $\langle (X, Y, Z), X + Y = Z \rangle$ và C2: $\langle (X, Y), X + 1 = Y \rangle$. Sau đó, đồ thị kép sẽ có các biến $X = \{C1, C2\}$, trong đó miền của biến C1 trong đồ thị kép là tập hợp các bộ $\{(xi, y, zk)\}$ từ ràng buộc C1 trong bài toán ban đầu, và tương tự như vậy, miền của C2 là tập hợp các bộ $\{(xi, y, jj)\}$. Đồ thị kép có ràng buộc nhị phân $\langle \{C1, C2\}, R1 \rangle$, trong đó R1 là một quan hệ mới xác định ràng buộc giữa C1 và C2; trong trường hợp này, nó sẽ là $R1 = \{((1, 2, 3), (1, 2)), ((2, 3, 5), (2, 3))\}$.



Hình 5.2 (a) Một vấn đề cryptarithmic. Mỗi chữ cái là viết tắt của một chữ số riêng biệt; Mục đích là tìm sự thay thế các chữ số cho các chữ cái sao cho tổng kết quả là chính xác về mặt số học, với hạn chế bổ sung là không cho phép số không đứng đầu. (b) Siêu đồ thị ràng buộc cho bài toán cryptarithmic, hiển thị ràng buộc AllDIFF (hộp vuông ở trên cùng) cũng như các ràng buộc bổ sung cột (bốn hộp vuông ở giữa). Các biến C1, C2 và C3 đại diện cho các chữ số mang cho ba cột từ phải sang trái.

Tuy nhiên, có hai lý do tại sao chúng ta có thể thích một ràng buộc toàn cục như AllDIFF hơn là một tập hợp các ràng buộc nhị phân. Đầu tiên, sẽ dễ dàng hơn và ít bị lỗi hơn để viết mô tả vấn đề bằng cách sử dụng AllDIFF. Thứ hai, có thể thiết kế các thuật toán suy luận có mục đích đặc biệt cho các ràng buộc toàn cục hiệu quả hơn so với hoạt động với các kết hợp nguyên thủy. Chúng tôi mô tả các thuật toán suy luận này trong Phần 5.2.5.

Các ràng buộc mà chúng tôi đã mô tả cho đến nay đều là những ràng buộc tuyệt đối, vi phạm loại trừ một giải pháp tiềm năng. Nhiều CSP trong thế giới thực bao gồm các hạn chế ưu tiên Tùy chọn ràng buộc cho biết giải pháp nào được ưu tiên. Ví dụ, trong một vấn đề lập lịch học của trường đại học, có những ràng buộc tuyệt đối là không giáo sư nào có thể dạy hai lớp cùng một lúc. Nhưng chúng tôi cũng có thể cho phép các hạn chế về sở thích: Giáo sư R có thể thích giảng dạy vào buổi sáng, trong khi Giáo sư N thích giảng dạy vào buổi chiều. Một lịch trình có Giáo sư R giảng dạy lúc 2 giờ chiều vẫn sẽ là một giải pháp được phép (trừ khi Giáo sư R tình cờ là chủ nhiệm khoa) nhưng sẽ không phải là một giải pháp tối ưu.

Các ràng buộc ưu tiên thường có thể được mã hóa dưới dạng chi phí cho các nhiệm vụ biên riêng lẻ — ví dụ: chỉ định một khoảng thời gian buổi chiều cho Giáo sư R có giá 2 điểm so với hàm mục tiêu tổng thể, trong khi một khoảng thời gian buổi sáng có giá 1. Với công thức này, các CSP có sở thích có thể được giải quyết bằng các phương pháp tìm kiếm tối ưu hóa, dựa trên đường dẫn hoặc cục bộ. Chúng tôi gọi một vấn đề như vậy là bài toán tối ưu hóa bị hạn chế, hoặc COP. Các chương trình tuyến tính là một lớp COP. Vấn đề tối ưu hóa bị hạn chế

5.2 Lan truyền ràng buộc: Suy luận trong CSP

Một thuật toán tìm kiếm không gian trạng thái nguyên tử tạo ra tiến bộ chỉ theo một cách: bằng cách mở rộng cục bộ để truy cập các người kế nhiệm. Thuật toán CSP có các lựa chọn. Nó có thể tạo ra các kế thừa bằng cách chọn một giá trị mới hoặc nó có thể thực hiện một loại suy luận cụ thể được gọi là constraint propagation: sử dụng các ràng buộc để giảm số lượng giá trị hợp pháp cho một biến, mà

Constraint propagation

lần lượt có thể giảm các giá trị hợp pháp cho một biến khác, v.v. Ý tưởng là điều này sẽ để lại ít lựa chọn hơn để xem xét khi chúng ta đưa ra lựa chọn tiếp theo của một nhiệm vụ biến. Sự lan truyền ràng buộc có thể được đan xen với tìm kiếm, hoặc nó có thể được thực hiện như một bước tiền xử lý, trước khi tìm kiếm bắt đầu. Đôi khi tiền xử lý này có thể giải quyết toàn bộ vấn đề, vì vậy không cần tìm kiếm.

Tính nhất quán cục bộ

Ý tưởng chính là tính nhất quán của địa phương. Nếu chúng ta coi mỗi biến như một nút trong đồ thị (xem

Hình 5.1 (b)) và mỗi ràng buộc nhị phân như một cạnh, sau đó quá trình thực thi tính nhất quán cục bộ trong mỗi phần của đồ thị làm cho các giá trị không nhất quán bị loại bỏ trong suốt biểu đồ. Có nhiều loại tính nhất quán cục bộ khác nhau, mà bây giờ chúng ta lần lượt đề cập.

5.2.1 Tính nhất quán của nút

Tính nhất quán của nút

Một biến duy nhất (tương ứng với một nút trong biểu đồ CSP) là nhất quán với nút nếu tất cả các giá trị trong miền của biến đáp ứng các ràng buộc đơn nguyên của biến. Ví dụ, trong biến thể của bài toán tô màu bản đồ Úc (Hình 5.1) nơi người Nam Úc không thích màu xanh lá cây, biến SA bắt đầu bằng miền {đỏ, xanh lá cây, xanh lam} và chúng ta có thể làm cho nó phù hợp với nút bằng cách loại bỏ màu xanh lá cây, để lại SA với miền giảm {đỏ, xanh lam}. Chúng ta nói rằng agraph là nhất quán với nút nếu mọi biến trong đồ thị đều nhất quán với nút.

Thật dễ dàng để loại bỏ tất cả các ràng buộc đơn nguyên trong CSP bằng cách giảm miền của các biến thể với các ràng buộc đơn nguyên khi bắt đầu quá trình giải quyết. Như đã đề cập trước đó, cũng có thể chuyển đổi tất cả các ràng buộc n -ary thành các ràng buộc nhị phân. Do đó, một số trình giải CSP chỉ hoạt động với các ràng buộc nhị phân, mong đợi người dùng loại bỏ các ràng buộc khác trước thời hạn. Chúng tôi đưa ra giả định đó cho phần còn lại của chương này, trừ khi được ghi chú.

5.2.2 Tính nhất quán của hồ quang

Tính nhất quán của hồ quang

Một biến trong CSP nhất quán với vòng cung¹ nếu mọi giá trị trong miền của nó thỏa mãn biến ràng buộc nhị phân. Về mặt chính thức hơn, Xi phù hợp với cung đối với một biến khác Xj nếu đối với mỗi giá trị trong miền hiện tại Di có một số giá trị trong miền D j thỏa mãn ràng buộc nhị phân trên cung (Xi, Xj). Một biểu đồ là nhất quán cung nếu mọi biến đều phù hợp với mọi biến khác. Ví dụ, hãy xem xét ràng buộc $Y = X^2$ trong đó miền của cả X và Y là tập hợp các chữ số thập phân. Chúng ta có thể viết ràng buộc này một cách rõ ràng như

$\{(X, Y), \{(0, 0), (1, 1), (2, 4), (3, 9)\}\}$. Để làm cho cung X nhất quán đối với Y, chúng ta giảm miền của X thành $\{0, 1, 2, 3\}$. Nếu chúng ta cũng làm cho cung Y nhất quán đối với X, thì miền của Y trở thành $\{0, 1, 4, 9\}$, và toàn bộ CSP là nhất quán với vòng cung. Mặt khác, tính nhất quán của vòng cung không thể làm gì cho vấn đề màu bản đồ Úc. Hãy xem xét ràng buộc bất bình đẳng sau đây đối với (SA, WA):

$\{(\text{đỏ}, \text{xanh lá cây}), (\text{đỏ}, \text{xanh dương}), (\text{xanh lá cây}, \text{đỏ}), (\text{xanh lá cây}, \text{xanh dương}), (\text{xanh lam}, \text{đỏ}), (\text{xanh dương}, \text{xanh lá cây})\}$. Bất kể bạn chọn giá trị nào cho SA (hoặc cho WA), vẫn có một giá trị hợp lệ cho biến khác. Vì vậy, việc áp dụng tính nhất quán cung không ảnh hưởng đến các miền của một trong hai biến.

Thuật toán phổ biến nhất để thực thi tính nhất quán của hồ quang được gọi là AC-3 (xem Hình 5.3). Để làm cho mọi biến nhất quán với vòng cung, thuật toán AC-3 duy trì một hàng đợi các cung để con-sider. Ban đầu, hàng đợi chứa tất cả các vòng cung trong CSP. (Mỗi ràng buộc nhị phân trở thành hai vòng cung, một vòng cung theo mỗi hướng.) AC-3 sau đó bật ra một cung tùy ý (Xi, Xj) từ hàng đợi

¹ Chúng tôi đã sử dụng thuật ngữ "cạnh" thay vì "vòng cung", vì vậy sẽ hợp lý hơn nếu gọi nó là "cạnh nhất quán", nhưng cái tên "vòng cung" là lịch sử.

hàm AC-3(csp) trả về false nếu tìm thấy sự không nhất quán và true nếu không
 Queue $\leftarrow$ một hàng đợi các vòng cung, ban đầu tất cả các vòng cung trong CSP

trong khi hàng đợi không trống, hãy
 (Xi, Xj) $\leftarrow$ POP(hàng đợi) nếu REVISE(csp, Xi, Xj) thì

 nếu kích thước của Di = 0 thì trả về false cho mỗi Xk trong Xi.NEIGHBORS - {Xj} do

 thêm (Xk, Xi) vào queue trả về true

hàm REVISE(csp, Xi, Xj) trả về true nếu chúng ta sửa đổi miền của Xi
 sửa đổi $\leftarrow$ false cho mỗi x trong Di do

 nếu không có giá trị y trong Dj cho phép (x,y) thỏa mãn ràng buộc giữa Xi và Xj thì
 xóa x khỏi Direvised
 $\leftarrow$ true return revised

Hình 5.3 Thuật toán độ nhất quán hồ quang AC-3. Sau khi áp dụng AC-3, mọi cung nhất quán hoặc một số biến có miền trống, cho thấy rằng CSP không thể được giải quyết. Tên "AC-3" được sử dụng bởi nhà phát minh của thuật toán (Mackworth, 1977) vì nó là phiên bản thứ ba được phát triển trong bài báo.

và làm cho Xi arc-nhất quán đối với Xj. Nếu điều này không thay đổi, thuật toán chỉ chuyển sang vòng cung tiếp theo. Nhưng nếu điều này sửa đổi Di (làm cho miền nhỏ hơn), thì chúng ta thêm vào hàng đợi tất cả các cung (Xk, Xi) trong đó Xk là hàng xóm của Xi. Chúng ta cần làm điều đó bởi vì sự thay đổi trong Di có thể cho phép giảm thêm Dk, ngay cả khi trước đây chúng ta đã xem xét Xk. Nếu Di được sửa đổi xuống không, thì chúng ta biết toàn bộ CSP không có giải pháp nhất quán và AC-3 có thể ngay lập tức trả về lỗi. Nếu không, chúng tôi tiếp tục kiểm tra, cố gắng loại bỏ các giá trị khỏi miền của các biến cho đến khi không còn cung nào trong hàng đợi. Tại thời điểm đó, chúng ta chỉ còn lại một CSP tương đương với CSP ban đầu - cả hai đều có cùng giải pháp - nhưng CSP nhất quán vòng cung sẽ tìm kiếm nhanh hơn vì các biến của nó có các miền nhỏ hơn. Trong một số trường hợp, nó giải quyết vấn đề hoàn toàn (bằng cách giảm mọi miền xuống kích thước 1) và trong những trường hợp khác, nó chứng minh rằng không có giải pháp nào tồn tại (bằng cách giảm một số miền xuống kích thước 0).

Độ phức tạp của AC-3 có thể được phân tích như sau. Giả sử một CSP có n biến, mỗi biến có kích thước miền tối đa d và với c ràng buộc nhị phân (vòng cung). Mỗi cung (Xk, Xi) chỉ có thể được chen vào hàng đợi d lần vì Xi có nhiều nhất d giá trị để xóa. Kiểm tra tính nhất quán của một hồ quang có thể được thực hiện trong thời gian O(d²), vì vậy chúng tôi nhận được tổng thời gian trường hợp xấu nhất của O(cd³).

5.2.3 Tính nhất quán của đường dẫn

Giả sử chúng ta tô màu bản đồ Úc chỉ với hai màu, đỏ và xanh lam. Sự đồng bộ của vòng cung không làm gì vì mọi ràng buộc có thể được thỏa mãn riêng lẻ với màu đỏ ở một đầu và màu xanh lam ở đầu kia. Nhưng rõ ràng không có giải pháp cho vấn đề: bởi vì Western Australia, Northern Territory và South Australia đều chạm vào nhau, chúng ta cần ít nhất ba màu cho riêng chúng.

Tính nhất quán của vòng cung thất chặt các miền (ràng buộc đơn nguyên) bằng cách sử dụng các vòng cung (ràng buộc nhị phân). Để đạt được tiến bộ trong các vấn đề như tô màu bản đồ, chúng ta cần một khái niệm mạnh mẽ hơn về tính nhất quán. Tính nhất quán của đường dẫn thất chặt các ràng buộc nhị phân bằng cách sử dụng các ràng buộc ngầm

Tính nhất quán của
đường dẫn

được suy ra bằng cách nhìn vào bộ ba biến.

Một tập hợp hai biến $\{X_i, X_j\}$ là nhất quán đường dẫn đối với biến thứ ba X_m nếu, đối với mỗi gán $\{X_i = a, X_j = b\}$ phù hợp với các ràng buộc (nếu có) trên $\{X_i, X_j\}$, có một gán cho X_m thỏa mãn các ràng buộc trên $\{X_i, X_m\}$ và $\{X_m, X_j\}$. Cái tên đề cập đến sự nhất quán tổng thể của con đường từ X_i đến X_j với X_m ở giữa.

Hãy xem tính nhất quán của con đường như thế nào khi tô màu bản đồ Úc với hai màu. Chúng tôi sẽ làm cho tập hợp $\{WA, SA\}$ phù hợp với đường dẫn đối với NT. Chúng tôi bắt đầu bằng cách liệt kê các nhiệm vụ nhất quán cho tập hợp. Trong trường hợp này, chỉ có hai: $\{WA = \text{đỏ}, SA = \text{xanh lam}\}$ và $\{WA = \text{xanh lam}, SA = \text{đỏ}\}$. Chúng ta có thể thấy rằng với cả hai nhiệm vụ này, NT không thể có màu đỏ cũng không có màu xanh lam (vì nó sẽ xung đột với WA hoặc SA). Bởi vì không có lựa chọn hợp lệ cho NT, chúng tôi loại bỏ cả hai nhiệm vụ và cuối cùng chúng tôi không có nhiệm vụ hợp lệ nào cho $\{WA, SA\}$. Do đó, chúng tôi biết rằng không thể có giải pháp cho vấn đề này.

5.2.4 K-nhất quán

K-nhất quán

Các hình thức nhân giống mạnh hơn có thể được định nghĩa với khái niệm k-consistency. Một CSP là k-nhất quán nếu, đối với bất kỳ tập hợp nào các biến k-1 và đối với bất kỳ gán nhất quán nào cho các biến đó, một giá trị nhất quán luôn có thể được gán cho bất kỳ biến thứ k nào. 1-consistency nói rằng, với tập trống, chúng ta có thể làm cho bất kỳ tập hợp nào của một biến nhất quán: đây là những gì chúng ta gọi là nodeconsistency. Tính nhất quán 2 cũng giống như độ nhất quán của vòng cung. Đối với đồ thị ràng buộc nhị phân, tính nhất quán 3 giống như tính nhất quán đường dẫn.

Nhất quán mạnh mẽ

Một CSP là k-nhất quán mạnh nếu nó là k-nhất quán và cũng là $(k-1)$ -nhất quán, $(k-2)$ -nhất quán... tất cả các cách chọn 1 nhất quán. Bây giờ giả sử chúng ta có một CSP với n nút và làm cho nó có tính nhất quán mạnh mẽ (tức là nhất quán k mạnh cho $k = n$). Sau đó, chúng ta có thể giải quyết vấn đề như sau: Đầu tiên, chúng ta chọn một giá trị nhất quán cho X_1 . Sau đó, chúng ta được đảm bảo có thể chọn một giá trị cho X_2 vì biểu đồ là 2 nhất quán, đối với X_3 vì nó nhất quán 3, v.v. Đối với mỗi biến X_i , chúng ta chỉ cần tìm kiếm qua các giá trị d trong miền để tìm một giá trị phù hợp với $X_1, \dots, X_{i-1}$. Tổng thời gian chạy chỉ là $O(n^2d)$.

Tất nhiên, không có bữa trưa miễn phí: sự thỏa mãn ràng buộc nói chung là NP-complete, và bất kỳ thuật toán nào để thiết lập tính nhất quán n phải mất thời gian theo cấp số nhân trong n trong trường hợp xấu nhất. Tệ hơn, tính nhất quán n cũng yêu cầu không gian theo cấp số nhân trong n. Trong thực tế, xác định mức độ kiểm tra tính nhất quán thích hợp chủ yếu là một khoa học thực nghiệm. Tính toán tính nhất quán 2 là phổ biến và tính nhất quán 3 ít phổ biến hơn.

5.2.5 Các ràng buộc toàn cầu

Hãy nhớ rằng một ràng buộc toàn cục là một ràng buộc liên quan đến một số biến tùy ý (nhưng không nhất thiết phải tất cả các biến). Các ràng buộc toàn cầu xảy ra thường xuyên trong các vấn đề thực tế và có thể được xử lý bởi các thuật toán có mục đích đặc biệt hiệu quả hơn các phương pháp có mục đích chung được mô tả cho đến nay. Ví dụ, ràng buộc Alldiff nói rằng tất cả các biến liên quan phải có các giá trị riêng biệt (như trong bài toán cryptarithmic ở trên và các câu đố Sudoku thấp). Một hình thức phát hiện không nhất quán đơn giản cho các ràng buộc Alldiff hoạt động như sau: nếu m biến có liên quan đến ràng buộc và nếu chúng có n giá trị riêng biệt có thể alto-gether và $m > n$, thì không thể thỏa mãn ràng buộc.

Điều này dẫn đến thuật toán đơn giản sau: Đầu tiên, loại bỏ bất kỳ biến nào trong ràng buộc có miền đơn và xóa giá trị của biến đó khỏi các miền của các biến còn lại. Lập lại miền là có các biến singleton. Nếu tại bất kỳ thời điểm nào một miền trống được tạo ra hoặc có nhiều biến hơn giá trị miền còn lại, thì sự không nhất quán đã được phát hiện.

Phương pháp này có thể phát hiện sự không nhất quán trong nhiệm vụ $\{WA = \text{đỏ}, NSW = \text{đỏ}\}$ cho Hình 5.1. Lưu ý rằng các biến SA, NT và Q được kết nối hiệu quả bởi một AllDiffConstraint vì mỗi cặp phải có hai màu khác nhau. Sau khi áp dụng AC-3 với gán một phần, các miền của SA, NT và Q đều được giảm xuống $\{\text{xanh lá cây}, \text{xanh lam}\}$. Nghĩa là, chúng ta có ba biến và chỉ có hai màu, vì vậy ràng buộc AllDiff bị vi phạm. Do đó, một thủ tục nhất quán đơn giản cho một ràng buộc bậc cao hơn đôi khi hiệu quả hơn so với áp dụng tính nhất quán cung cho một tập hợp các ràng buộc nhị phân tương đương.

Một ràng buộc bậc cao quan trọng khác là ràng buộc tài nguyên, đôi khi được gọi là Hạn chế tài nguyên ràng buộc Atmos. Ví dụ, trong một bài toán lập kế hoạch, hãy để $P_1, \dots, P_4$ biểu thị số lượng nhân sự được giao cho mỗi nhiệm vụ trong số bốn nhiệm vụ. Ràng buộc rằng tổng cộng không quá 10 nhân sự được chỉ định được viết là $\text{Atmost}(10, P_1, P_2, P_3, P_4)$. Chúng ta có thể phát hiện sự không nhất quán đơn giản bằng cách kiểm tra tổng các giá trị tối thiểu của các miền hiện tại; ví dụ: nếu mỗi biến có miền $\{3, 4, 5, 6\}$, thì không thể thỏa mãn ràng buộc Atmos. Chúng tôi cũng có thể thực thi tính nhất quán bằng cách xóa giá trị lớn nhất của bất kỳ miền nào nếu nó không nhất quán với giá trị tối thiểu của các miền khác. Do đó, nếu mỗi biến trong ví dụ của chúng ta có miền $\{2, 3, 4, 5, 6\}$, các giá trị 5 và 6 có thể bị xóa khỏi mỗi miền.

Đối với các vấn đề lớn bị giới hạn tài nguyên với các giá trị số nguyên - chẳng hạn như các vấn đề hậu cần liên quan đến việc di chuyển hàng nghìn người trong hàng trăm phương tiện - thường không thể biểu diễn miền của mỗi biến dưới dạng một tập hợp lớn các số nguyên và giảm dần bộ đó bằng các phương pháp kiểm tra tính nhất quán. Thay vào đó, các miền được biểu thị bằng giới hạn trên và dưới và được quản lý bằng sự lan truyền giới hạn. Ví dụ: trong một lịch trình của hãng hàng không

Lan truyền giới hạn

vấn đề, giả sử có hai chuyến bay, F1 và F2, trong đó các máy bay có sức chứa 165 và 385, tương ứng. Các miền ban đầu cho số lượng hành khách trên các chuyến bay F1 và F2 sau đó là

$$D1 = [0, 165] \text{ và } D2 = [0, 385].$$

Bây giờ giả sử chúng ta có ràng buộc bổ sung rằng hai chuyến bay cùng nhau phải chở 420 người: $F1 + F2 = 420$. Truyền bá các ràng buộc giới hạn, chúng tôi giảm các miền thành

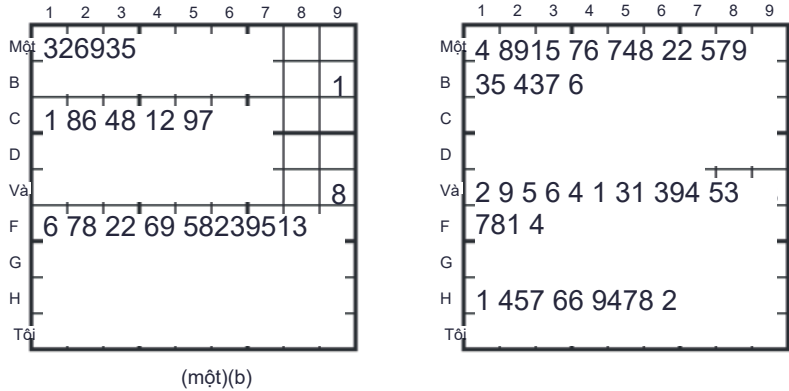
$$D1 = [35, 165] \text{ và } D2 = [255, 385].$$

Chúng tôi nói rằng một CSP là giới hạn nhất quán nếu cho mọi biến X và cho cả giới hạn dưới và các giá trị giới hạn trên của X, tồn tại một số giá trị của Y thỏa mãn ràng buộc giữa X và Y cho mọi biến Y. Loại lan truyền giới hạn này được sử dụng rộng rãi trong các vấn đề ràng buộc thực tế.

Bounds-nhất quán

5.2.6 Sudoku

Câu đố Sudoku phổ biến đã giới thiệu cho hàng triệu người về sự hài lòng hạn Sudoku ~~khó~~; mặc dù họ có thể không nhận ra điều đó. Bảng Sudoku bao gồm 81 ô vuông, một số trong số đó ban đầu được lấp đầy bằng các chữ số từ 1 đến 9. Câu đố là điền vào tất cả các ô vuông còn lại sao cho không có chữ số nào xuất hiện hai lần trong bất kỳ hàng, cột hoặc ô 3×3 nào (xem Hình 5.4). Một hàng, cột hoặc hộp được gọi là đơn vị.



Hình 5.4 (a) Một câu đố Sudoku và (b) lời giải của nó.

Các câu đố Sudoku xuất hiện trên báo và sách giải đố có đặc tính là chính xác là một giải pháp. Mặc dù một số câu đố có thể khó giải bằng tay, mất hàng chục phút, nhưng trình giải CSP có thể xử lý hàng nghìn câu đố mỗi giây.

Một câu đố Sudoku có thể được coi là CSP với 81 biến, một cho mỗi ô vuông. Chúng tôi sử dụng tên biến A1 đến A9 cho hàng trên cùng (từ trái sang phải), xuống I1 đến I9 cho hàng dưới cùng. Các ô vuông trống có miền {1, 2, 3, 4, 5, 6, 7, 8, 9} và các ô vuông được điền trước có một miền bao gồm một giá trị duy nhất. Ngoài ra, có 27 AllDiff constraints khác nhau, một cho mỗi đơn vị (hàng, cột và hộp gồm 9 ô vuông):

AllDiff (A1, A2, A3, A4, A5, A6, A7, A8, A9) AllDiff (B1, B2, B3, B4, B5, B6, B7, B8, B9) · · · AllDiff (A1, B1, C1, D1, E1, F1, G1, H1, I1) AllDiff (A2, B2, C2, D2, E2, F2, G2, H2, I2) · · · AllDiff (A1, A2, A3, B1, B2, B3, C1, C2, C3) AllDiff (A4, A5, A6, B4, B5, B6, C4, C5, C6) · · · Hãy xem tính nhất quán của vòng cung có thể đưa chúng ta đi bao xa. Giả sử rằng các ràng buộc AllDiff đã được mở rộng thành các ràng buộc nhị phân (chẳng hạn như $A1 = A2$) để chúng ta có thể áp dụng thuật toán AC-3 trực tiếp. Hãy xem xét biến E6 từ Hình 5.4 (a) — hình vuông trống giữa 2 và 8 trong ô giữa. Từ các ràng buộc trong hộp, chúng ta có thể loại bỏ 1, 2, 7 và 8 khỏi tên miền của E6. Từ các ràng buộc trong cột của nó, chúng ta có thể loại bỏ 5, 6, 2, 8, 9 và 3 (mặc dù 2 và 8 đã bị loại bỏ). Điều đó khiến E6 có miền {4}; nói cách khác, chúng tôi biết câu trả lời cho E6. Bây giờ hãy xem xét biến I6 — hình vuông trong hộp giữa dưới cùng được bao quanh bởi 1, 3 và 3. Áp dụng tính nhất quán của vòng cung trong cột của nó, chúng tôi loại bỏ 5, 6, 2, 4 (vì bây giờ chúng tôi biết E6 phải là 4), 8, 9 và 3. Chúng tôi loại bỏ 1 bằng độ nhất quán của vòng cung với I5,

và chúng ta chỉ còn lại giá trị 7 trong miền I6. Bây giờ có 8 giá trị đã biết trong cột 6, vì vậy tính nhất quán của vòng cung có thể suy ra rằng A6 phải là 1. Suy luận tiếp tục theo những dòng này, và cuối cùng, AC-3 có thể giải toàn bộ câu đố - tất cả các biến đều có các miền của chúng được giảm xuống một giá trị duy nhất, như thể hiện trong Hình 5.4 (b).

Tất nhiên, Sudoku sẽ sớm mất đi sức hấp dẫn của nó nếu mọi câu đố có thể được giải bằng một ứng dụng cơ khí của AC-3, và thực sự AC-3 chỉ hoạt động cho các câu đố Sudoku dễ dàng nhất. Những cái khó hơn một chút có thể được giải quyết bằng PC-2, nhưng với chi phí tính toán lớn hơn: có 255.960 ràng buộc đường dẫn khác nhau để xem xét trong một câu đố Sudoku. Để giải quyết những vấn đề khó khăn nhất và đạt được tiến bộ hiệu quả, chúng ta sẽ phải thông minh hơn.

Thật vậy, sức hấp dẫn của câu đố Sudoku đối với người giải là như câu đố vật trong việc áp dụng các chiến lược suy luận phức tạp hơn. Những người hâm mộ đặt cho chúng những cái tên đầy màu sắc, chẳng hạn như "bộ ba khóa thân". Chiến lược đó hoạt động như sau: trong bất kỳ đơn vị nào (hàng, cột hoặc hộp), tìm ba ô vuông mà mỗi ô vuông có một miền chứa ba số giống nhau hoặc một tập hợp con của các số đó. Ví dụ: ba miền có thể là $\{1, 8\}$, $\{3, 8\}$ và $\{1, 3, 8\}$. Từ đó chúng ta không biết hình vuông nào chứa 1, 3 hoặc 8, nhưng chúng ta biết rằng ba số phải được phân phối giữa ba ô vuông. Do đó, chúng ta có thể loại bỏ 1, 3 và 8 khỏi miền của mọi ô vuông khác trong đơn vị.

Thật thú vị khi lưu ý rằng chúng ta có thể đi bao xa mà không cần nói nhiều về Sudoku. Tất nhiên chúng ta phải nói rằng có 81 biến, rằng các miền của chúng là các chữ số từ 1 đến 9 và có 27 ràng buộc AllDiff. Nhưng ngoài ra, tất cả các chiến lược — tính nhất quán của vòng cung, tính nhất quán của đường dẫn, v.v. — thường áp dụng cho tất cả các CSP, không chỉ cho các vấn đề Sudoku. Evennaked triples thực sự là một chiến lược để thực thi tính nhất quán của các ràng buộc AllDiff và không dành riêng cho bản thân Sudoku. Đây là sức mạnh của chủ nghĩa hình thức CSP: đối với mỗi lĩnh vực vấn đề mới, chúng ta chỉ cần xác định vấn đề theo các ràng buộc; Sau đó, các cơ chế giải quyết ràng buộc chung có thể tiếp quản.

5.3 Tìm kiếm theo dõi ngược cho CSP

Đôi khi chúng ta có thể hoàn thành quá trình lan truyền ràng buộc và vẫn có các biến với nhiều giá trị có thể. Trong trường hợp đó, chúng ta phải tìm kiếm một giải pháp. Trong phần này, chúng tôi đề cập đến các thuật toán tìm kiếm quay ngược hoạt động trên các nhiệm vụ một phần; Trong phần tiếp theo, chúng ta xem xét các thuật toán tìm kiếm cục bộ trên các bài tập hoàn chỉnh.

Hãy xem xét cách tìm kiếm giới hạn độ sâu tiêu chuẩn (từ Chương 3) có thể giải quyết các CSP. Astate sẽ là một nhiệm vụ một phần và một hành động sẽ mở rộng nhiệm vụ, thêm vào, chẳng hạn, NSW = đỏ hoặc SA = xanh lam cho bài toán tô màu bản đồ Úc. Đối với một CSP có n biến có kích thước miền d , chúng ta sẽ kết thúc với một cây tìm kiếm trong đó tất cả các nhiệm vụ hoàn chỉnh (và do đó tất cả các giải pháp) là các nút lá ở độ sâu n . Nhưng lưu ý rằng hệ số phân nhánh ở cấp trên cùng sẽ là nd vì bất kỳ giá trị d nào cũng có thể được gán cho bất kỳ biến nào trong số n biến. Ở cấp độ tiếp theo, hệ số phân nhánh là $(n-1)d$, và cứ như vậy đối với n mức. Vì vậy, cây có $n! \cdot DN$ rời đi, mặc dù chỉ có DN có thể hoàn thành nhiệm vụ!

Chúng ta có thể lấy lại hệ số n đó bằng cách nhận ra một đặc tính quan trọng của CSP: khả năng giao thông. Một vấn đề là giao hoán nếu thứ tự áp dụng của bất kỳ tập hợp hành động nhất định nào

Khả năng giao hoán

không quan trọng. Trong CSP, không có gì khác biệt nếu trước tiên chúng ta gán NSW = đỏ và sau đó SA = xanh lam, hoặc ngược lại. Do đó, chúng ta chỉ cần xem xét một biến duy nhất ở mỗi nút trong cây tìm kiếm. Ở gốc, chúng ta có thể lựa chọn giữa SA = đỏ, SA = xanh lá cây và

hàm BACKTRACKING-SEARCH(csp) trả về một giải pháp hoặc lỗi
trả về BACKTRACK(csp, { })

hàm BACKTRACK(csp, assignment) trả về một giải pháp hoặc lỗi
nếu nhiệm vụ hoàn tất thì trả về assignmentvar $\leftarrow$ SELECT-UNASSIGNED-VARIABLE(csp,
assignment) cho mỗi giá trị trong ORDER-DOMAIN-VALUES(csp, var, assignment) do

```

    nếu giá trị phù hợp với gán thì
        thêm {var = value} vào
        assignmentinferences  $\leftarrow$  INFERENCE(csp,
        var, assignment)if inferences 6= thất bại thì
        thêm suy luận vào cspresult  $\leftarrow$ 
        BACKTRACK (csp, assignment) nếu kết
        quả 6 = thất bại thì trả về resultremove
        inferences from cspremove {var = value}
        from assignmentreturn failure

```

Hình 5.5 Một thuật toán quay ngược đơn giản cho các vấn đề thỏa mãn ràng buộc. Thuật toán được mô hình hóa dựa trên tìm kiếm độ sâu đệ quy đầu tiên của Chương 3. Các chức năng SELECT-UNASSIGNED-VARIABLE và ORDER-DOMAIN-VALUES thực hiện các phương pháp phỏng đoán mục đích chung được thảo luận trong Phần 5.3.1. Hàm INFERENCE có thể tùy chọn áp đặt tính nhất quán arc, path- hoặc k, như mong muốn. Nếu một lựa chọn giá trị dẫn đến thất bại (được nhận thấy bởi INFERENCE hoặc bởi BACKTRACK), thì việc gán giá trị (bao gồm cả những giá trị được thực hiện bởi INFERENCE) sẽ bị rút lại và một giá trị mới được thử.

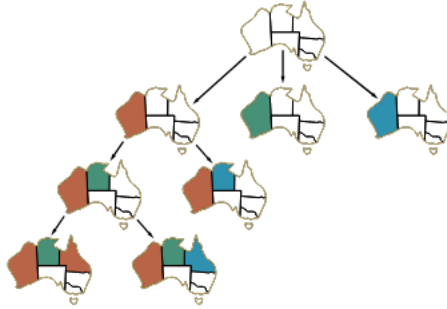
SA = xanh lam, nhưng chúng tôi sẽ không bao giờ chọn giữa NSW = đỏ và SA = xanh lam. Với sự hạn chế này, số lượng lá là 4^n , như chúng ta hy vọng. Ở mỗi cấp độ của cây, chúng ta phải chọn biến nào chúng ta sẽ xử lý, nhưng chúng ta không bao giờ phải quay lại lựa chọn đó.

Hình 5.5 cho thấy quy trình tìm kiếm quay trở lại cho CSP. Nó liên tục chọn một biến chưa gán và sau đó lần lượt thử tất cả các giá trị trong miền của biến đó, cố gắng mở rộng từng giá trị thành một giải pháp thông qua một lệnh gọi đệ quy. Nếu cuộc gọi thành công, giải pháp sẽ được trả về và nếu nó không thành công, việc gán sẽ được khôi phục về trạng thái trước đó và chúng ta thử giá trị tiếp theo. Nếu không có giá trị nào hoạt động thì chúng tôi trả về thất bại. Một phần của cây tìm kiếm cho bài toán Úc được thể hiện trong Hình 5.6, trong đó chúng ta đã gán các biến theo thứ tự WA, NT, Q, . . .

Lưu ý rằng BACKTRACKING-SEARCH chỉ giữ một biểu diễn duy nhất của một trạng thái (assignment) và thay đổi biểu diễn đó thay vì tạo ra những biểu diễn mới (xem trang 98).

Trong khi các thuật toán tìm kiếm thiếu thông tin của Chương 3 chỉ có thể được cải thiện bằng cách hỗ trợ chúng với các phương pháp phỏng đoán theo tên miền cụ thể, hóa ra rằng tìm kiếm theo dõi ngược có thể được chứng minh bằng cách sử dụng các phương pháp phỏng đoán độc lập với tên miền tận dụng lợi thế của đại diện nhân tố của CSP. Trong bốn phần sau, chúng tôi chỉ ra cách thực hiện điều này:

- (5.3.1) Biến nào nên được gán tiếp theo (SELECT-UNASSIGNED-VARIABLE) và các giá trị của nó nên được thử theo thứ tự nào (ORDER-DOMAIN-VALUES)?
- (5.3.2) Những suy luận nào nên được thực hiện ở mỗi bước trong tìm kiếm (INFERENCE)?
- (5.3.3) Chúng ta có thể QUAY LẠI nhiều hơn một bước khi thích hợp không?
- (5.3.4) Chúng tôi có thể lưu và sử dụng lại một phần kết quả từ tìm kiếm không?



Hình 5.6 Một phần của cây tìm kiếm cho bài toán tô màu bản đồ trong Hình 5.1.

5.3.1 Thứ tự biến và giá trị

Thuật toán quay trở lại chứa dòng

```
var ← SELECT-UNASSIGNED-VARIABLE(csp, assignment) .
```

Chiến lược đơn giản nhất cho SELECT-UNASSIGNED-VARIABLE là thứ tự tĩnh: chọn các biến theo thứ tự, $\{X_1, X_2, \dots\}$. Đơn giản tiếp theo là chọn ngẫu nhiên. Cả hai chiến lược đều không tối ưu. Ví dụ, sau khi gán cho WA = đỏ và NT = xanh lá cây trong Hình 5.6, chỉ có một giá trị có thể cho SA, vì vậy bạn nên gán SA = xanh lam tiếp theo thay vì gán Q. Trên thực tế, sau khi SA được gán, các lựa chọn cho Q, NSW và V đều bị bắt buộc.

Ý tưởng trực quan này - chọn biến có ít giá trị "hợp pháp" nhất - được gọi là suy nghiệm giá trị còn lại tối thiểu (MRV). Nó cũng được gọi là "hạn chế nhất

Giá trị còn lại tối thiểu

variable" hoặc "fail-first", sau này vì nó chọn một biến có nhiều khả năng gây ra lỗi sớm, do đó cắt tỉa cây tìm kiếm. Nếu một số biến X không còn giá trị hợp pháp, phương pháp phỏng đoán MRV sẽ chọn X và thất bại sẽ được phát hiện ngay lập tức — tránh tìm kiếm vô nghĩa thông qua các biến khác. Phỏng đoán MRV thường hoạt động tốt hơn so với thứ tự ngẫu nhiên hoặc tĩnh, đôi khi theo thứ tự độ lớn, mặc dù kết quả khác nhau tùy thuộc vào vấn đề.

Mô phỏng MRV hoàn toàn không giúp ích gì trong việc chọn khu vực đầu tiên để tô màu ở Úc, bởi vì ban đầu mọi khu vực đều có ba màu hợp pháp. Trong trường hợp này, mức độ heuristic đến

Phỏng đoán bằng cấp

trong tiện ích. Nó cố gắng giảm yếu tố phân nhánh trên các lựa chọn trong tương lai bằng cách chọn biến có liên quan đến số lượng ràng buộc lớn nhất trên các biến không được gán khác. Trong Hình 5.1, SA là biến có bậc cao nhất, 5; các biến khác có bậc 2 hoặc 3, ngoại trừ T, có bậc 0. Nếu chúng ta gán SA trước, chúng ta có thể đi vòng quanh năm vùng đất liền theo thứ tự theo chiều kim đồng hồ hoặc ngược chiều kim đồng hồ và gán cho mỗi vùng một màu khác với SA và khác với vùng trước đó. Heuristic giá trị còn lại tối thiểu thường là một hướng dẫn mạnh mẽ hơn, nhưng heuristic mức độ có thể hữu ích như một công cụ hòa.

Khi một biến đã được chọn, thuật toán phải quyết định thứ tự kiểm tra các giá trị của nó. Phương pháp phỏng đoán giá trị hạn chế nhất có hiệu quả cho việc này. Nó thích

Giá trị hạn chế tối thiểu

giá trị loại trừ ít lựa chọn nhất cho các biến lân cận trong ràng buộc

đồ thị. Ví dụ, giả sử rằng trong Hình 5.1, chúng ta đã tạo ra một phần gán với $WA = \text{đỏ}$ và $NT = \text{xanh lá cây}$ và lựa chọn tiếp theo của chúng ta là cho Q . Màu xanh lam sẽ là một lựa chọn tồi vì nó loại bỏ giá trị hợp pháp cuối cùng còn lại cho hàng xóm của Q , SA . Do đó, heuristic có giá trị hạn chế ít nhất thích màu đỏ hơn màu xanh lam. Nói chung, heuristic đang cố gắng để lại sự linh hoạt tối đa cho các nhiệm vụ biến tiếp theo. Tại sao lựa chọn biến phải là không thành công trước, nhưng lựa chọn giá trị lại là không thành công? Mọi biến số cuối cùng phải được gán, vì vậy bằng cách chọn những biến có khả năng thất bại trước, trung bình chúng ta sẽ có ít nhiệm vụ thành công hơn để quay lại. Đối với thứ tự giá trị, bí quyết là chúng ta chỉ cần một giải pháp; do đó, bạn nên tìm kiếm các giá trị có khả năng xảy ra nhất trước. Nếu chúng ta muốn liệt kê tất cả các giải pháp thay vì chỉ tìm một giải, thì thứ tự giá trị sẽ không liên quan.

5.3.2 Tìm kiếm xen kẽ và suy luận

Chúng ta đã thấy AC-3 có thể giảm các miền của các biến trước khi chúng ta bắt đầu tìm kiếm. Nhưng suy luận thậm chí có thể mạnh mẽ hơn trong quá trình tìm kiếm: mỗi khi chúng ta lựa chọn một giá trị cho một biến, chúng ta có một cơ hội hoàn toàn mới để suy ra các miền mới trên các biến lân cận.

Kiểm tra chuyển tiếp

Một trong những hình thức suy luận đơn giản nhất được gọi là kiểm tra chuyển tiếp. Bắt cứ khi nào một biến

X được gán, quá trình kiểm tra chuyển tiếp thiết lập tính nhất quán của vòng cung cho nó: đối với mỗi biến không có dấu Y được kết nối với X bằng một ràng buộc, xóa khỏi miền của Y bất kỳ giá trị nào không phù hợp với giá trị được chọn cho X .

Hình 5.7 cho thấy tiến trình tìm kiếm ngược trên CSP của Úc với kiểm tra chuyển tiếp. Có hai điểm quan trọng cần lưu ý về ví dụ này. Đầu tiên, hãy lưu ý rằng sau khi $WA = \text{đỏ}$ và $Q = \text{xanh lá cây}$ được gán, các miền của NT và SA được giảm xuống một giá trị duy nhất; chúng tôi đã loại bỏ hoàn toàn sự phân nhánh trên các biến này bằng cách truyền thông tin từ WA và Q . Điểm thứ hai cần lưu ý là sau $V = \text{màu xanh lam}$, miền của SA trống rỗng. Do đó, kiểm tra chuyển tiếp đã phát hiện ra rằng gán một phần $\{WA = \text{đỏ}, Q = \text{xanh lá cây}, V = \text{xanh lam}\}$ không phù hợp với các ràng buộc của vấn đề và thuật toán quay ngược ngay lập tức.

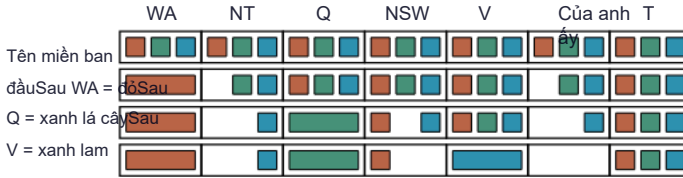
Đối với nhiều vấn đề, tìm kiếm sẽ hiệu quả hơn nếu chúng ta kết hợp phỏng đoán MRV với kiểm tra chuyển tiếp. Hãy xem xét Hình 5.7 sau khi gán $\{WA = \text{đỏ}\}$. Theo trực giác, có vẻ như việc gán đó hạn chế các hàng xóm của nó, NT và SA , vì vậy chúng ta nên xử lý các biến đó tiếp theo, và sau đó tất cả các biến khác sẽ rơi vào đúng vị trí. Đó chính xác là những gì xảy ra với MRV: NT và SA mỗi người có hai giá trị, vì vậy một trong số chúng được chọn trước, sau đó là giá trị kia, sau đó là Q , NSW và V theo thứ tự. Cuối cùng T vẫn có ba giá trị, và bất kỳ một trong số chúng đều hoạt động. Chúng ta có thể xem kiểm tra chuyển tiếp như một cách hiệu quả để tính toán dần dần thông tin mà phỏng đoán MRV cần để thực hiện công việc của nó.

Mặc dù kiểm tra chuyển tiếp phát hiện nhiều điểm mâu thuẫn, nhưng nó không phát hiện được tất cả chúng. Vấn đề là nó không nhìn về phía trước đủ xa. Ví dụ, hãy xem xét hàng $Q = \text{màu xanh lá cây}$ của Hình 5.7. Chúng tôi đã làm cho vòng cung WA và Q nhất quán, nhưng chúng tôi đã để cả NT và SA với màu xanh lam là giá trị duy nhất có thể có của chúng, đó là một sự không nhất quán, vì chúng là hàng xóm.

Duy trì ArcConsistency

Thuật toán được gọi là MAC (để duy trì tính nhất quán của hồ quang) phát hiện sự không nhất quán

Thích cái này. Sau khi một biến X_i được gán một giá trị, thủ tục INFERENCE gọi AC-3, nhưng thay vì một hàng đợi của tất cả các cung trong CSP, chúng ta chỉ bắt đầu với các cung (X_j, X_i) cho tất cả các X_j là các biến chưa được gán là hàng xóm của X_i . Từ đó, AC-3 thực hiện lan truyền ràng buộc theo cách thông thường và nếu bất kỳ biến nào có miền của nó được giảm xuống tập trống, thì lệnh gọi đến AC-3



Hình 5.7 Tiến trình tìm kiếm tô màu bản đồ với kiểm tra chuyển tiếp. WA = màu đỏ được ký trước; sau đó kiểm tra chuyển tiếp xóa màu đỏ khỏi các miền của các biển lân cận NT và SA. Sau khi Q = green được gán, màu xanh lá cây sẽ bị xóa khỏi tên miền của NT, SA và NSW. Sau khi V = blue được gán, màu xanh lam sẽ bị xóa khỏi tên miền của NSW và SA, để lại SA không có giá trị pháp lý.

thất bại và chúng tôi biết phải quay lại ngay lập tức. Chúng ta có thể thấy rằng MAC hoàn toàn mạnh hơn kiểm tra chuyển tiếp vì kiểm tra chuyển tiếp thực hiện điều tương tự như MAC trên các vòng cung ban đầu trong hàng đợi của MAC; nhưng không giống như MAC, kiểm tra chuyển tiếp không lan truyền đệ quy các ràng buộc khi các thay đổi được thực hiện đối với các miền của các biển.

5.3.3 Quay ngược thông minh: Nhìn lại

Thuật toán BACKTRACKING-SEARCH trong Hình 5.5 có một chính sách rất đơn giản về những gì cần làm khi một nhánh của tìm kiếm thất bại: sao lưu đến biển trước đó và thử một giá trị khác cho nó. Điều này được gọi là quay ngược theo thời gian vì quyết định gần đây nhất

Quay lại theo thứ tự thời gian

điểm được xem xét lại. Trong tiểu mục này, chúng tôi xem xét các khả năng tốt hơn. Hãy xem xét điều gì xảy ra khi chúng ta áp dụng backtrack đơn giản trong Hình 5.1 với một thứ tự biến cố định Q, NSW, V, T, SA, WA, NT. Giả sử chúng ta đã tạo ra một phần gán {Q = đỏ, NSW = xanh lá cây, V = xanh lam, T = đỏ}. Khi chúng ta thử biển tiếp theo, SA, chúng ta thấy rằng mọi giá trị đều vi phạm một ràng buộc. Chúng tôi quay lại T và thử một màu mới cho Tasmania! Rõ ràng điều này thật ngớ ngẩn - việc đổi màu Tasmania không thể giúp giải quyết vấn đề với Nam Úc.

Một cách tiếp cận thông minh hơn là quay trở lại một biển có thể khắc phục vấn đề - một biển chịu trách nhiệm làm cho một trong những giá trị có thể có của SA trở nên bất khả thi. Để làm điều này, chúng tôi sẽ theo dõi một tập hợp các nhiệm vụ xung đột với một số giá trị cho SA. Tập hợp (trong trường hợp này {Q = đỏ, NSW = xanh lá cây, V = xanh lam}), được gọi là tập xung đột cho SA. Các

Bộ xung đột
Nhảy lùi

phương pháp nhảy lùi quay trở lại nhiệm vụ gần đây nhất trong tập xung đột; trong điều này trường hợp, nhảy lùi sẽ nhảy qua Tasmania và thử một giá trị mới cho V. Phương pháp này dễ dàng thực hiện bằng cách sửa đổi BACKTRACK sao cho nó tích lũy bộ xung đột trong khi kiểm tra giá trị pháp lý để gán. Nếu không tìm thấy giá trị pháp lý, thuật toán sẽ trả về phần tử gần đây nhất của bộ xung đột cùng với chỉ báo lỗi.

Người đọc có đôi mắt nhạy bén có thể nhận thấy rằng kiểm tra chuyển tiếp có thể cung cấp bộ xung đột mà không cần làm thêm: bất cứ khi nào kiểm tra chuyển tiếp dựa trên một nhiệm vụ $X = x$ xóa một giá trị khỏi miền của Y , nó sẽ thêm $X = x$ vào tập xung đột của Y . Nếu giá trị cuối cùng bị xóa khỏi tên miền của Y , thì các nhiệm vụ trong tập xung đột Y sẽ được thêm vào tập xung đột X . Đó là, bây giờ chúng ta biết rằng $X = x$ dẫn đến một mâu thuẫn (trong Y), và do đó nên thử một nhiệm vụ khác cho X .

Người đọc có đôi mắt đại bàng có thể nhận thấy một điều kỳ lạ: nhảy lùi xảy ra khi giá trị ev-ery trong một lĩnh vực xung đột với nhiệm vụ hiện tại; Nhưng kiểm tra chuyển tiếp phát hiện sự kiện này và ngăn tìm kiếm đến một nút như vậy! Trên thực tế, có thể chỉ ra rằng mọi cảnh được cắt tia bằng cách nhảy ngược cũng được cắt tia bằng cách kiểm tra về phía trước. Do đó, nhảy lùi đơn giản là dư thừa trong tìm kiếm kiểm tra chuyển tiếp hoặc thực sự, trong một tìm kiếm sử dụng kiểm tra tính nhất quán mạnh hơn, chẳng hạn như MAC — bạn chỉ cần thực hiện một trong hai cách khác.

Bất chấp những quan sát của đoạn trước, ý tưởng đằng sau việc nhảy lùi vẫn còn một ý tưởng tốt: lùi lại dựa trên lý do thất bại. Backjumping nhận thấy thất bại khi miền của một biến trở nên trống, nhưng trong nhiều trường hợp, một nhánh đã bị diệt vong rất lâu trước khi điều này xảy ra. Hãy xem xét lại việc gán một phần $\{WA = \text{đỏ}, NSW = \text{đỏ}\}$ (mà, từ cuộc thảo luận trước đó của chúng tôi, không nhất quán). Giả sử chúng ta thử $T = \text{đỏ}$ tiếp theo và sau đó gán NT, Q, V, SA . Chúng tôi biết rằng không có bài tập nào có thể hoạt động cho bốn biến cuối cùng này, vì vậy cuối cùng chúng tôi đã hết giá trị để thử ở NT . Bây giờ, câu hỏi đặt ra là, lùi lại ở đâu? Nhảy lùi không thể hoạt động, bởi vì NT có các giá trị phù hợp với các biến được gán trước đó — NT không có một tập hợp xung đột hoàn chỉnh của các biến trước đó khiến nó thất bại. Tuy nhiên, chúng ta biết rằng bốn biến NT, Q, V và SA , kết hợp với nhau, đã thất bại vì một tập hợp các biến trước đó, đó phải là những biến xung đột trực tiếp với bốn biến đó.

Điều này dẫn đến một khái niệm khác – và sâu sắc hơn – về tập hợp xung đột cho một biến như NT : đó là tập hợp các biến trước đó đã khiến NT , cùng với bất kỳ biến tiếp theo nào, không có lời giải nhất quán. Trong trường hợp này, tập hợp là WA và NSW , vì vậy thuật toán nên quay trở lại NSW và bỏ qua Tasmania. Một thuật toán nhảy lùi sử dụng các tập xung đột được xác định theo cách này được gọi là nhảy lùi theo hướng xung đột. Bây giờ chúng ta phải giải thích cách tính toán các tập hợp xung đột mới này. Trên thực tế, phương pháp này khá đơn giản. Lỗi "đầu cuối" của một nhánh của tìm kiếm luôn xảy ra vì miền của một biến trở nên trống; biến đó có một bộ xung đột tiêu chuẩn. Trong ví dụ của chúng ta, SA thất bại và tập xung đột của nó là (giả sử) $\{WA, NT, Q\}$. Chúng ta quay trở lại Q , và Q hấp thụ tập xung đột từ SA (tất nhiên là trừ Q) vào tập xung đột trực tiếp của riêng nó, đó là $\{NT, NSW\}$; bộ xung đột mới là $\{WA, NT, NSW\}$. Nghĩa là, không có lời giải nào từ Q trở đi, với sự gán trước đó cho $\{WA, NT, NSW\}$. Do đó, chúng tôi quay trở lại NT , gần đây nhất trong số này. NT hấp thụ $\{WA, NT, NSW\} - \{NT\}$ vào tập xung đột trực tiếp $\{WA\}$ của riêng nó, cho $\{WA, NSW\}$ (như đã nêu trong đoạn trước). Bây giờ thuật toán quay trở lại NSW , như chúng tôi hy vọng. Tóm lại: hãy để X_j là biến hiện tại và hãy để $\text{conf}(X_j)$ là tập xung đột của nó. Nếu mọi giá trị có thể có cho X_j không thành công, hãy nhảy ngược đến biến gần đây nhất X_i trong $\text{conf}(X_j)$ và tính toán lại xung đột được đặt cho X_i như sau:

$$\text{conf}(X_i) \leftarrow \text{conf}(X_i) \cup \text{conf}(X_j) - \{X_j\}.$$

5.3.4 Học ràng buộc

Khi chúng ta đạt đến một mâu thuẫn, nhảy lùi có thể cho chúng ta biết phải lùi bao xa, vì vậy chúng ta không lãng phí thời gian thay đổi các biến số sẽ không khắc phục được vấn đề. Nhưng chúng tôi cũng muốn tránh gặp phải vấn đề tương tự một lần nữa. Khi tìm kiếm đi đến một mâu thuẫn, chúng ta biết rằng một số tập hợp con của tập xung đột chịu trách nhiệm cho vấn đề. Học ràng buộc là

ý tưởng tìm một tập hợp các biến tối thiểu từ tập hợp xung đột gây ra vấn đề. Tập hợp các biến này, cùng với các giá trị tương ứng của chúng, được gọi là không tốt. Sau đó, chúng tôi ghi lại sự không tốt, bằng cách thêm một ràng buộc mới vào CSP để cấm sự kết hợp các nhiệm vụ này hoặc bằng cách giữ một bộ đệm riêng biệt của các hàng hóa không tốt.

Nhảy lùi theo hướng
xung đột

Học ràng buộc

Không tốt

Ví dụ, hãy xem xét trạng thái $\{WA = \text{đỏ}, NT = \text{xanh lá cây}, Q = \text{xanh lam}\}$ ở hàng dưới cùng của Hình 5.6. Kiểm tra chuyển tiếp có thể cho chúng ta biết trạng thái này là không tốt vì không có sự chỉ định hợp lệ cho SA. Trong trường hợp cụ thể này, việc ghi lại điều không tốt sẽ không giúp ích gì, bởi vì một khi chúng ta cắt tỉa nhánh này khỏi cây tìm kiếm, chúng ta sẽ không bao giờ gặp phải sự kết hợp này nữa. Nhưng giả sử rằng cây tìm kiếm trong Hình 5.6 thực sự là một phần của cây tìm kiếm lớn hơn bắt đầu bằng cách gán các giá trị cho V và T. Sau đó, sẽ rất đáng để ghi lại $\{WA = \text{đỏ}, NT = \text{xanh lá cây}, Q = \text{xanh lam}\}$ là không tốt vì chúng ta sẽ gặp phải vấn đề tương tự một lần nữa cho mỗi tập hợp các bài tập có thể có cho V và T.

Không có hàng hóa có thể được sử dụng hiệu quả bằng cách kiểm tra về phía trước hoặc bằng cách nhảy lùi. Constraint learning là một trong những kỹ thuật quan trọng nhất được sử dụng bởi các giải CSP hiện đại để đạt được hiệu quả đối với các vấn đề phức tạp.

5.4 Tìm kiếm CSP cục bộ

Các thuật toán tìm kiếm cục bộ (xem Phần 4.1) hóa ra rất hiệu quả trong việc giải quyết nhiều CSP. Chúng sử dụng một công thức trạng thái hoàn chỉnh (như được giới thiệu trong Phần 4.1.1) trong đó mỗi trạng thái ký một giá trị cho mọi biến và tìm kiếm thay đổi giá trị của một biến tại một thời điểm. Ví dụ, chúng ta sẽ sử dụng bài toán 8-queens, như được định nghĩa là CSP ở trang 167. Trong Hình 5.8, chúng ta bắt đầu từ bên trái với một bài tập hoàn chỉnh cho 8 biến; Thông thường điều này sẽ vi phạm một số ràng buộc. Sau đó, chúng ta chọn ngẫu nhiên một biến xung đột, hóa ra là Q8, cột ngoài cùng bên phải. Chúng tôi muốn thay đổi giá trị thành một cái gì đó đưa chúng tôi đến gần hơn với một giải pháp; Cách tiếp cận rõ ràng nhất là chọn giá trị dẫn đến số lượng xung đột tối thiểu với các biến khác — suy nghiệm xung đột tối thiểu.

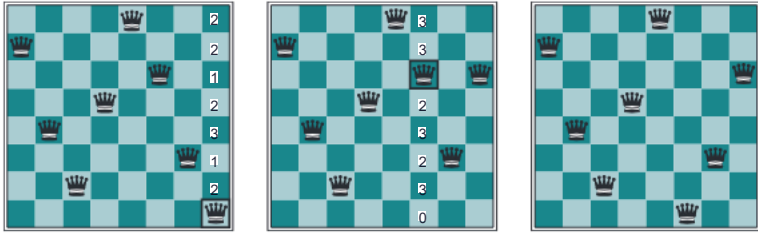
Xung đột tối thiểu

Trong hình chúng ta thấy có hai hàng chỉ vi phạm một ràng buộc; chúng tôi chọn Q8 = 3 (nghĩa là chúng tôi di chuyển nữ hoàng đến cột thứ 8, hàng thứ 3). Trong lần lặp tiếp theo, ở bảng giữa của hình, chúng ta chọn Q6 làm biến để thay đổi và lưu ý rằng việc di chuyển nữ hoàng đến hàng thứ 8 dẫn đến không có xung đột. Tại thời điểm này không còn các biến mâu thuẫn nữa, vì vậy chúng tôi có một giải pháp. Thuật toán được thể hiện trong Hình 5.9.2

Xung đột tối thiểu có hiệu quả đáng ngạc nhiên đối với nhiều CSP. Thật ngạc nhiên, về vấn đề n-queens, nếu bạn không tính vị trí ban đầu của các nữ hoàng, thời gian chạy của xung đột tối thiểu gần như không phụ thuộc vào quy mô vấn đề. Nó giải quyết ngay cả vấn đề triệu nữ hoàng trong trung bình 50 bước (sau bài tập ban đầu). Quan sát đáng chú ý này là sự kích thích dẫn đến rất nhiều nghiên cứu trong những năm 1990 về tìm kiếm địa phương và sự khác biệt giữa các vấn đề dễ và khó, mà chúng tôi đưa ra trong Phần 7.6.3. Nói một cách đại khái, n-queen rất dễ tìm kiếm cục bộ vì các giải pháp được phân bố dày đặc trong không gian trạng thái. Min-conflict cũng hoạt động tốt cho các vấn đề khó. Ví dụ, nó đã được sử dụng để lên lịch quan sát cho Kính viễn vọng Không gian Hubble, giảm thời gian lên lịch quan sát một tuần từ ba tuần xuống còn khoảng 10 phút.

Tất cả các kỹ thuật tìm kiếm cục bộ từ Phần 4.1 đều là ứng cử viên để áp dụng cho CSP và một số trong số đó đã được chứng minh là đặc biệt hiệu quả. Bối cảnh của một CSP theo phương pháp phỏng đoán xung đột nhỏ thường có một loạt các cao nguyên. Có thể có hàng triệu dấu hiệu thay đổi chỉ cách một xung đột để giải quyết. Tìm kiếm cao nguyên — cho phép di chuyển sang một tiểu bang khác với cùng điểm số — có thể giúp tìm kiếm địa phương tìm đường thoát khỏi trạng thái này

2 Tìm kiếm cục bộ có thể dễ dàng được mở rộng đến các bài toán tối ưu hóa bị hạn chế (COP). Trong trường hợp đó, tất cả các công nghệ để leo đồi và ủ mô phỏng có thể được áp dụng để tối ưu hóa chức năng mục tiêu.



Hình 5.8 Giải pháp hai bước sử dụng xung đột tối thiểu cho bài toán 8 chúa. Ở mỗi giai đoạn, một nữ hoàng được chọn để gắn lại trong cột của nó. Số lượng xung đột (trong trường hợp này là số lượng nữ hoàng tấn công) được hiển thị trong mỗi ô vuông. Thuật toán di chuyển nữ hoàng đến hình vuông xung đột của họ, phá vỡ mối quan hệ một cách ngẫu nhiên.

hàm MIN-CONFLICTS(csp, max steps) trả về một giải pháp hoặc thất bại

Đầu vào: CSP, một vấn đề thỏa mãn ràng buộc

max-steps, số bước được phép trước khi bỏ cuộc

hiện tại $\leftarrow$ một nhiệm vụ hoàn thành ban đầu cho CSP cho tới = 1 đến các bước tối đa thực hiện

Nếu current là một giải pháp cho CSP thì trả về currentvar $\leftarrow$ một biến xung đột được chọn ngẫu nhiên từ CSP. VARIABLES giá trị $\leftarrow$ giá trị v cho var giảm thiểu CONFLICTS (csp, var, v, current) đặt var = giá trị trong lỗi currentreturn

Hình 5.9 Thuật toán tìm kiếm cục bộ MIN-CONFLICTS cho CSP. Trạng thái ban đầu có thể được chọn ngẫu nhiên hoặc bởi một quá trình gắn tham lam lần lượt chọn một giá trị xung đột tối thiểu cho mỗi biến. Hàm CONFLICTS đếm số lượng ràng buộc bị vi phạm bởi một giá trị cụ thể, với phần còn lại của nhiệm vụ hiện tại.

cao nguyên. Việc lang thang trên cao nguyên này có thể được định hướng bằng một kỹ thuật gọi là tìm kiếm tabu: giữ một danh sách nhỏ các tiểu bang đã truy cập gần đây và cấm thuật toán quay trở lại các tiểu bang đó. Ủ mô phỏng cũng có thể được sử dụng để thoát khỏi cao nguyên.

Trọng số ràng buộc

Một kỹ thuật khác được gọi là trọng số ràng buộc nhằm mục đích tập trung tìm kiếm vào những ràng buộc quan trọng. Mỗi ràng buộc được cung cấp một trọng số số, ban đầu tất cả đều là 1. Ở mỗi bước của tìm kiếm, thuật toán chọn một cặp biến / giá trị để thay đổi sẽ dẫn đến tổng trọng số thấp nhất của tất cả các ràng buộc bị vi phạm. Sau đó, trọng số được điều chỉnh bằng cách tăng trọng số của từng ràng buộc bị vi phạm bởi nhiệm vụ hiện tại. Điều này có hai lợi ích: nó thêm địa hình vào cao nguyên, đảm bảo rằng có thể cải thiện từ trạng thái hiện tại và nó cũng bổ sung học tập: theo thời gian, các ràng buộc khó khăn được gán trọng lượng cao hơn.

Một ưu điểm khác của tìm kiếm cục bộ là nó có thể được sử dụng trong cài đặt trực tuyến (xem Phần 4.5) khi sự cố thay đổi. Hãy xem xét một vấn đề về lịch trình cho các chuyến bay hàng tuần của một hãng hàng không. Lịch trình có thể liên quan đến hàng nghìn chuyến bay và hàng chục nghìn nhân viên

nhưng thời tiết xấu tại một sân bay có thể khiến lịch trình không khả thi. Chúng tôi muốn sửa chữa lịch trình với số lượng thay đổi tối thiểu. Điều này có thể dễ dàng thực hiện với thuật toán tìm kiếm cục bộ bắt đầu từ lịch trình hiện tại. Tìm kiếm ngược dòng với tập hợp áp dụng cho các vấn đề khác ngoài CSP, chẳng hạn như suy luận xác suất. có thể tìm thấy giải pháp với nhiều thay đổi so với lịch trình hiện tại.

5.5 Cấu trúc của các vấn đề

Trong phần này, chúng tôi xem xét các cách mà cấu trúc của vấn đề, như được biểu thị bởi biểu đồ ràng buộc, có thể được sử dụng để tìm ra giải pháp một cách nhanh chóng. Hầu hết các cách tiếp cận ở đây cũng áp dụng cho các vấn đề khác ngoài CSP, chẳng hạn như suy luận xác suất.

Cách duy nhất chúng ta có thể hy vọng đối phó với thế giới thực rộng lớn là phân tách nó thành các vấn đề phụ. Nhìn lại biểu đồ ràng buộc cho Úc (Hình 5.1 (b)), lặp lại như Hình 5.12 (a)), một thực thể nổi bật: Tasmania không được kết nối với đất liền.³ Về mặt trực giác, rõ ràng là tô màu Tasmania và tô màu đất liền là các vấn đề phụ độc lập — bất kỳ giải pháp nào cho đất liền kết hợp với bất kỳ giải pháp nào cho Tasmania đều mang lại một giải pháp

Các vấn đề con độc lập

tion cho toàn bộ bản đồ.

Tính độc lập có thể được xác định đơn giản bằng cách tìm các thành phần được kết nối của cấu trúc

Thành phần được kết nối

biểu đồ constraint. Mỗi thành phần tương ứng với một vấn đề con CSPi. Nếu gán Si là dung dịch của CSPi, thì Ui Si là lời giải của Ui CSPi. Tại sao điều này lại quan trọng? Giả sử mỗi CSPi có các biến c từ tổng số n biến, trong đó c là một hằng số. Sau đó, có các vấn đề n / csuib, mỗi bài toán mất nhiều nhất công việc dc để giải quyết, trong đó d là kích thước của miền. Do đó, tổng công là O (dcn / c), tuyến tính trong n; nếu không có sự phân hủy, tổng công là O(dn), là hàm mũ tính bằng n. Hãy làm cho điều này cụ thể hơn: chia một BooleanCSP với 100 biến thành bốn bài toán con làm giảm thời gian giải pháp trong trường hợp xấu nhất từ tuổi thọ của vũ trụ xuống dưới một giây.

Các vấn đề phụ hoàn toàn độc lập là ngon, nhưng hiếm gặp. May mắn thay, một số cấu trúc đồ thị khác cũng dễ giải. Ví dụ: đồ thị ràng buộc là một cây khi hai biến bất kỳ được kết nối chỉ bằng một đường dẫn. Chúng tôi sẽ chỉ ra rằng bất kỳ JCSP có cấu trúc cây nào cũng có thể được giải quyết theo thời gian tuyến tính về số lượng biến.⁴ Chia khóa là một khái niệm mới về

tính nhất quán, được gọi là tính nhất quán hồ quang định hướng hoặc DAC. CSP được định nghĩa là định hướng

Tính nhất quán của vòng cung định hướng

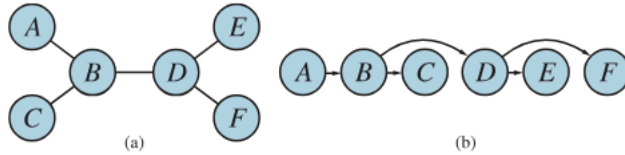
quang nhất quán theo thứ tự của các biến X1, X2, . . . , Xn nếu và chỉ khi mọi Xi phù hợp với mọi Xj cho j > i.

Để giải CSP có cấu trúc cây, trước tiên hãy chọn bất kỳ biến nào làm gốc của cây và chọn thứ tự của các biến sao cho mỗi biến xuất hiện sau biến gốc của nó trong cây. Thứ tự như vậy được gọi là một sắp xếp tô pô. Hình 5.10 (a) cho thấy một cây mẫu và (b) cho thấy

Sắp xếp tô pô

một đơn đặt hàng có thể. Bất kỳ cây nào có n nút đều có n - 1 cạnh, vì vậy chúng ta có thể làm cho vòng cung biểu đồ này nhất quán trong các bước O (n), mỗi bước phải so sánh tối đa d giá trị miền có thể cho hai biến, trong tổng thời gian là O (nd²). Khi chúng ta có một biểu đồ nhất quán cung có hướng, chúng ta có thể chỉ cần đi xuống danh sách các biến và chọn bất kỳ giá trị nào còn lại. Vì mỗi cạnh từ cha đến con của nó là nhất quán theo vòng cung, chúng ta biết rằng đối với bất kỳ giá trị nào chúng ta chọn cho cha, sẽ có một giá trị hợp lệ còn lại để chọn cho con. Điều đó có nghĩa là chúng tôi sẽ không

³ Một nhà vẽ bản đồ cẩn thận hoặc người Tasmania yêu nước có thể phản đối rằng Tasmania không nên được tô màu giống như người láng giềng gần nhất trên đất liền, để tránh ấn tượng rằng nó có thể là một phần của tiểu bang đó.
⁴ Đáng buồn thay, rất ít khu vực trên thế giới có bản đồ có cấu trúc cây, mặc dù Sulawesi đã đến gần.



Hình 5.10 (a) Biểu đồ ràng buộc của CSP cấu trúc cây. (b) Thứ tự tuyến tính của các biến phù hợp với cây có A làm gốc. Đây được gọi là một loại tô pô của các biến.

hàm TREE-CSP-SOLVER(csp) trả về một giải pháp hoặc thất bại
 đầu vào: csp, CSP với các thành phần X, D, C

```

n ← số lượng biến trong
Xassignment ← một assignment
root rỗng ← bất kỳ biến nào trong
XX ← TOPOLOGICALSORT(X,
root)for j = n xuống 2 do
MAKE-ARC-CONSISTENT (CHA ME (Xj), Xj) nếu nó không
thể được làm nhất quán thì trả về thất bại cho i = 1 đến n do

```

```

assignment [Xi] ← bất kỳ giá trị nhất quán nào
từ Diif không có giá trị nhất quán thì trả về
failurereturn assignment

```

Hình 5.11 Thuật toán TREE-CSP-SOLVER để giải các CSP có cấu trúc cây. Nếu CSP có một giải pháp, chúng ta sẽ tìm thấy nó trong thời gian tuyến tính; nếu không, chúng ta sẽ phát hiện ra một mâu thuẫn.

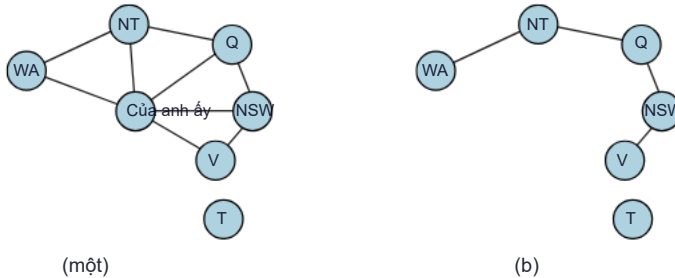
phải lùi lại; chúng ta có thể di chuyển tuyến tính qua các biến. Thuật toán hoàn chỉnh được thể hiện trong Hình 5.11.

Bây giờ chúng ta đã có một thuật toán hiệu quả cho cây, chúng ta có thể xem xét liệu nhiều đồ thị ràng buộc thể hệ hơn có thể được rút gọn thành cây bằng cách nào đó hay không. Có hai cách để làm điều này: bằng cách loại bỏ các nút (Phần 5.5.1) hoặc bằng cách thu gọn các nút lại với nhau (Phần 5.5.2).

5.5.1 Điều hòa cắt

Cách đầu tiên để giảm biểu đồ ràng buộc thành cây liên quan đến việc gán giá trị cho một số biến để các biến còn lại tạo thành một cây. Hãy xem xét biểu đồ ràng buộc cho Úc, được hiển thị một lần nữa trong Hình 5.12 (a). Nếu không có Nam Úc, đồ thị sẽ trở thành một cái cây, như trong (b). May mắn thay, chúng ta có thể xóa Nam Úc (trong biểu đồ, không phải quốc gia) bằng cách cố định một giá trị cho SA và xóa khỏi miền của các biến khác bất kỳ giá trị nào không phù hợp với giá trị được chọn cho SA.

Bây giờ, bất kỳ giải pháp nào cho CSP sau khi SA và các ràng buộc của nó bị loại bỏ sẽ phù hợp với giá trị được chọn cho SA. (Điều này hoạt động cho CSP nhị phân; tình hình phức tạp hơn với các ràng buộc bậc cao hơn.) Do đó, chúng ta có thể giải quyết cây còn lại bằng thuật toán



Hình 5.12 (a) Biểu đồ ràng buộc ban đầu từ Hình 5.1. (b) Sau khi loại bỏ SA, biểu đồ ràng buộc trở thành một khu rừng gồm hai cây.

đã đưa ra ở trên và do đó giải quyết toàn bộ vấn đề. Tất nhiên, trong trường hợp chung (trái ngược với tô màu bản đồ), giá trị được chọn cho SA có thể sai, vì vậy chúng ta cần thử từng giá trị có thể. Thuật toán chung như sau:

1. Chọn một tập hợp con S của các biến của CSP sao cho biểu đồ ràng buộc trở thành một cây sau khi loại bỏ S. S được gọi là bộ cắt chu kỳ.
2. Đối với mỗi lần gán có thể cho các biến trong S đáp ứng tất cả các ràng buộc trên S, (a) loại bỏ khỏi các miền của các biến còn lại bất kỳ giá trị nào không phù hợp với việc gán cho S, và (b) nếu CSP còn lại có giải pháp, trả về nó cùng với gán cho S. Nếu bộ cắt chu kỳ có kích thước c, thì tổng thời gian chạy là $O(d^c \cdot (n - c)d^2)$: chúng ta phải thử từng tổ hợp dc của các giá trị cho các biến trong S, và đối với mỗi tổ hợp, chúng ta phải giải một bài toán cây có kích thước $n - c$. Nếu biểu đồ là "gần như một cái cây", thì c sẽ nhỏ và khoản tiết kiệm khi quay ngược thẳng sẽ rất lớn — đối với ví dụ biến 100 Boolean của chúng ta, nếu chúng ta có thể tìm thấy một bộ cắt có kích thước $c = 20$, điều này sẽ giúp chúng ta giảm từ tuổi thọ của Vũ trụ xuống còn vài phút. Tuy nhiên, trong trường hợp xấu nhất, c có thể lớn bằng $(n - 2)$. Tìm bộ cắt chu kỳ nhỏ nhất là NP-hard, nhưng một số thuật toán xấp xỉ hiệu quả đã được biết đến. Cách tiếp cận thuật toán tổng thể được gọi là điều kiện cutset; nó xuất hiện một lần nữa trong Chương 13, nơi

Bộ cắt chu kỳ

Điều hòa cắt

nó được sử dụng để suy luận về xác suất.

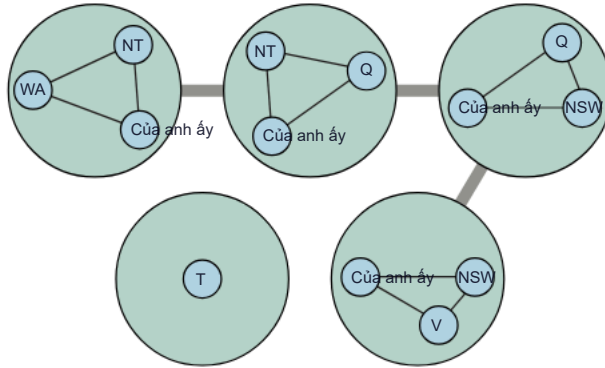
5.5.2 Phân hủy cây

Cách thứ hai để giảm một biểu đồ ràng buộc thành một cây dựa trên việc xây dựng một vị trí phân hủy cây của đồ thị ràng buộc: một sự chuyển đổi của đồ thị ban đầu thành một cây trong đó mỗi

Sự phân hủy của cây

nút trong cây bao gồm một tập hợp các biến, như trong Hình 5.13. Sự phân hủy của cây phải đáp ứng ba yêu cầu sau:

- Mọi biến trong bài toán ban đầu xuất hiện trong ít nhất một trong các nút cây. Nếu hai biến được kết nối bằng một ràng buộc trong bài toán ban đầu, chúng phải xuất hiện cùng nhau (cùng với ràng buộc) trong ít nhất một trong các nút cây.
- Nếu một biến xuất hiện trong hai nút trong cây, nó phải xuất hiện trong mọi nút dọc theo đường dẫn kết nối các nút đó.



Hình 5.13 Phân rã cây của biểu đồ ràng buộc trong Hình 5.12 (a).

Hai điều kiện đầu tiên đảm bảo rằng tất cả các biến và ràng buộc được biểu diễn trong quá trình phân rã cây. Điều kiện thứ ba có vẻ khá kỹ thuật, nhưng cho phép chúng ta nói rằng bất kỳ biến nào từ bài toán ban đầu phải có cùng giá trị ở bất cứ nơi nào nó xuất hiện: các ràng buộc trong cây nói rằng một biến trong một nút của cây phải có cùng giá trị với biến tương ứng trong nút liền kề trong cây. Ví dụ, SA xuất hiện trong tất cả bốn nút được kết nối trong Hình 5.13, vì vậy mỗi cạnh trong quá trình phân hủy cây do đó bao gồm ràng buộc rằng giá trị của SA trong một nút phải giống với giá trị của SA trong nút tiếp theo. Bạn có thể xác minh từ Hình 5.12 rằng sự phân hủy này có ý nghĩa.

Khi chúng ta có một biểu đồ có cấu trúc cây, chúng ta có thể áp dụng TREE-CSP-SOLVER để có được một giải pháp trong thời gian $O(nd^2)$, trong đó n là số nút cây và d là kích thước của miền lớn nhất. Nhưng lưu ý rằng trong cây, miền là một tập hợp các bộ giá trị, không chỉ các giá trị riêng lẻ.

Ví dụ, nút trên cùng bên trái trong Hình 5.13 đại diện, ở cấp độ của prob-lem ban đầu, một bài toán con với các biến $\{WA, NT, SA\}$, miền $\{\text{đỏ, xanh lá cây, xanh lam}\}$ và ràng buộc $WA = NT, SA = NT, WA = SA$. Ở cấp độ của cây, nút đại diện cho một biến thể duy nhất, mà chúng ta có thể gọi là SANTWA, có giá trị phải là ba bộ màu, chẳng hạn như (đỏ, xanh lá cây, xanh dương), nhưng không phải (đỏ, đỏ, xanh dương), vì điều đó sẽ vi phạm ràng buộc $SA = NT$ từ bài toán ban đầu. Sau đó, chúng ta có thể di chuyển từ nút đó sang nút liền kề, với biến chúng ta có thể gọi SANTQ và thấy rằng chỉ có một bộ dữ liệu, (đỏ, xanh lá cây, xanh dương), phù hợp với sự lựa chọn cho SANTWA. Quá trình tương tự được lặp lại cho hai nút tiếp theo và một cách độc lập chúng ta có thể đưa ra bất kỳ lựa chọn nào cho T.

Chúng ta có thể giải quyết bất kỳ vấn đề phân hủy cây nào trong thời gian $O(nd^2)$ với TREE-CSP-SOLVER, điều này sẽ hiệu quả miễn là d vẫn nhỏ. Quay trở lại ví dụ của chúng ta với các biến 100 Boolean, nếu mỗi nút có 10 biến, thì $d = 2^{10}$ và chúng ta sẽ có thể giải quyết vấn đề trong vài giây. Nhưng nếu có một nút có 30 biến, nó sẽ mất nhiều thế kỷ.

Một biểu đồ nhất định thừa nhận nhiều sự phân hủy của cây; Khi chọn một sự phân hủy, mục đích là làm cho các vấn đề phụ càng nhỏ càng tốt. (Đặt tất cả các biến vào một nút về mặt kỹ thuật là một cây, nhưng không hữu ích.) Chiều rộng cây của sự phân hủy cây của đồ thị là

một nhỏ hơn kích thước của nút lớn nhất; Chiều rộng cây của bản thân đồ thị được định nghĩa là chiều rộng tối thiểu trong số tất cả các phân tách cây của nó. Nếu một đồ thị có chiều rộng cây w thì vấn đề có thể được giải quyết trong thời gian $O((ndw + 1))$ với sự phân hủy cây tương ứng. Do đó, CSP với đồ thị ràng buộc của chiều rộng cây có giới hạn có thể giải được trong thời gian đa thức.

Thật không may, việc tìm ra sự phân hủy với chiều rộng cây tối thiểu là NP-khó, nhưng có những phương pháp phỏng đoán hoạt động tốt trong thực tế. Cái nào tốt hơn: phân hủy bộ cắt với thời gian $O((dc \cdot (n - c)d^2))$, hay sự phân hủy của cây với thời gian $O((ndw + 1))$? Bất cứ khi nào bạn có một bộ cắt chu kỳ có kích thước c , cũng có chiều rộng cây có kích thước $w < c + 1$ và nó có thể nhỏ hơn nhiều trong một số trường hợp. Vì vậy, việc xem xét thời gian có lợi cho sự phân hủy cây, nhưng ưu điểm của phương pháp cắt chu kỳ là nó có thể được thực thi trong bộ nhớ tuyến tính, trong khi giải nén cây yêu cầu bộ nhớ theo cấp số nhân tính bằng w .

5.5.3 Đối xứng giá trị

Cho đến nay, chúng ta đã xem xét cấu trúc của biểu đồ ràng buộc. Cũng có thể có cấu trúc quan trọng trong các giá trị của biến, hoặc trong cấu trúc của chính các mối quan hệ ràng buộc. Hãy xem xét vấn đề tô màu bản đồ với màu d . Đối với mỗi giải pháp nhất quán, thực sự có một tập hợp $d!$ các giải pháp được hình thành bằng cách hoán vị tên màu. Ví dụ, trên bản đồ Úc, chúng ta biết rằng WA, NT và SA đều phải có màu sắc khác nhau, nhưng có $3! = 6$ cách để gán ba màu cho ba vùng. Đây được gọi là đối xứng giá trị. Chúng tôi sẽ

Thích giảm không gian tìm kiếm theo hệ số $d!$ bằng cách phá vỡ sự đối xứng trong các bài tập. Chúng ta làm điều này bằng cách giới thiệu một ràng buộc phá vỡ đối xứng. Đối với ví dụ của chúng ta, chúng ta có thể áp đặt một ràng buộc thứ tự tùy ý, $NT < SA < WA$, yêu cầu ba giá trị phải theo thứ tự bảng chữ cái. Ràng buộc này đảm bảo rằng chỉ có một trong các $d!$ có thể giải pháp: $\{NT = \text{xanh lam}, SA = \text{xanh lá cây}, WA = \text{đỏ}\}$.

Đối với tô màu bản đồ, thật dễ dàng để tìm thấy một ràng buộc loại bỏ sự đối xứng. Trong thế giới, rất khó để loại bỏ tất cả các đối xứng, nhưng sự đối xứng giá trị phá vỡ đã được chứng minh là quan trọng và hiệu quả đối với một loạt các vấn đề.

Đối xứng giá trị

Phá vỡ đối xứng ràng buộc

Tóm tắt

- Các bài toán thỏa mãn ràng buộc (CSP) đại diện cho một trạng thái với một tập hợp các biến / cặp giá trị và đại diện cho các điều kiện cho một giải pháp bằng một tập hợp các ràng buộc trên các biến. Nhiều vấn đề quan trọng trong thế giới thực có thể được mô tả là CSP.
- Một số kỹ thuật suy luận sử dụng các ràng buộc để loại trừ một số dấu hiệu biến đổi nhất định. Chúng bao gồm nút, vòng cung, đường dẫn và k -consistency.
- Tìm kiếm theo dõi ngược, một hình thức tìm kiếm độ sâu đầu tiên, thường được sử dụng để giải CSP.
- Các giá trị còn lại tối thiểu và phỏng đoán mức độ là các phương pháp độc lập với miền để quyết định biến nào sẽ chọn tiếp theo trong tìm kiếm ngược dòng. Phép phỏng đoán giá trị hạn chế ít nhất giúp quyết định giá trị nào để thử trước cho một biến nhất định. Quay ngược xảy ra khi không thể tìm thấy gán hợp pháp cho một biến. Nhảy lùi theo hướng xung đột quay ngược trực tiếp đến nguồn gốc của vấn đề. Con-straint learning ghi lại các xung đột khi chúng gặp phải trong quá trình tìm kiếm để tránh xung đột tương tự sau này trong quá trình tìm kiếm.

- Tìm kiếm cục bộ sử dụng phương pháp phỏng đoán xung đột nhỏ cũng đã được áp dụng cho các vấn đề phe thỏa mãn với thành công lớn.
- Độ phức tạp của việc giải CSP có liên quan chặt chẽ đến cấu trúc của biểu đồ ràng buộc của nó. Các vấn đề có cấu trúc cây có thể được giải quyết trong thời gian tuyến tính. Điều hòa cutset có thể giảm CSP chung thành CSP có cấu trúc cây và khá hiệu quả (chỉ yêu cầu bộ nhớ lin-ear) nếu có thể tìm thấy một bộ cắt nhỏ. Các kỹ thuật phân rã cây biến đổi CSP thành một cây các vấn đề con và hiệu quả nếu chiều rộng cây của biểu đồ ràng buộc nhỏ; tuy nhiên chúng cần bộ nhớ theo cấp số nhân trong chiều rộng cây của biểu đồ con-strain. Kết hợp điều hòa cutset với sự phân hủy cây có thể cho phép đánh đổi trí nhớ tốt hơn so với thời gian.

Ghi chú thư mục và lịch sử

Phương trình
Diophantine

Nhà toán học Hy Lạp Diophantus (khoảng 200–284) đã trình bày và giải quyết các vấn đề liên quan đến các ràng buộc về phương trình, mặc dù ông không phát triển một phương pháp luận tổng quát. Bây giờ chúng ta gọi phương trình trên miền số nguyên là phương trình Diophantine. Toán học Ấn Độ-

nhà toán học Brahmagupta (khoảng 650) là người đầu tiên đưa ra một giải pháp tổng quát trên miền số nguyên cho phương trình $ax + by = c$. Các phương pháp có hệ thống để giải các phương trình tuyến tính bằng cách loại bỏ có thể thay đổi đã được nghiên cứu bởi Gauss (1829); giải pháp của các ràng buộc bất đẳng thức tuyến tính quay trở lại Fourier (1827).

Các vấn đề thỏa mãn ràng buộc miền hữu hạn cũng có lịch sử lâu đời. Ví dụ, tô màu đồ thị (trong đó tô màu bản đồ là một trường hợp đặc biệt) là một vấn đề cũ trong toán học. Phỏng đoán bốn màu (rằng mọi đồ thị phẳng có thể được tô màu với bốn màu hoặc ít hơn) lần đầu tiên được thực hiện bởi Francis Guthrie, một học trò của De Morgan, vào năm 1852. Nó chống lại giải pháp - mặc dù một số tuyên bố đã được công bố ngược lại - cho đến khi một bằng chứng được đưa ra bởi Appel và Haken (1977) (xem cuốn sách Four Colors Suffice (Wilson, 2004)). Những người theo chủ nghĩa thuần túy đã thất vọng rằng một phần của chứng minh dựa vào máy tính, vì vậy Georges Gonthier (2008), sử dụng người chứng minh định lý COQ, đã rút ra một chứng minh chính thức rằng chương trình chứng minh của Appel và Haken là đúng.

Các lớp cụ thể của các vấn đề thỏa mãn ràng buộc xảy ra trong suốt lịch sử của khoa học tính toán. Một trong những ví dụ ban đầu có ảnh hưởng nhất là SKETCHPAD (Sutherland, 1963), giải quyết các ràng buộc hình học trong sơ đồ và là tiền thân của các chương trình vẽ hiện đại và các công cụ CAD. Việc xác định CSP là một lớp chung là do Ugo Montanari (1974). Việc giảm các CSP bậc cao xuống các CSP nhị phân thuần túy với các biến phụ (xem Bài tập 5.NARY) ban đầu là do nhà logic học thế kỷ 19 Charles Sanders Peirce. Nó được đưa vào tài liệu CSP bởi Dechter (1990b) và được Bacchus và van Beek (1998) đánh giá cao. Các CSP có ưu tiên trong số các giải pháp được nghiên cứu rộng rãi trong tài liệu tối ưu hóa; xem Bistarelli et al. (1997) để khái quát hóa khung CSP để cho phép các tùy chọn.

Các phương pháp lan truyền ràng buộc đã được phổ biến bởi thành công của Waltz (1975) về các vấn đề dán nhãn đường đa diện cho thị giác máy tính. Waltz cho thấy rằng trong nhiều vấn đề, sự lan truyền hoàn toàn loại bỏ nhu cầu quay lại. Montanari (1974) đã giới thiệu khái niệm về đồ thị ràng buộc và lan truyền theo tính nhất quán của đường dẫn. Alan Mackworth (1977) đã đề xuất thuật toán AC-3 để thực thi tính nhất quán của vòng cung cũng như ý tưởng chung về việc kết hợp việc quay ngược với một số mức độ thực thi tính nhất quán. AC-4, hiệu quả hơn

thuật toán arc-consistency được phát triển bởi Mohr và Henderson (1986), chạy trong thời gian xấu nhất $O(cd^2)$ nhưng có thể chậm hơn AC-3 trong các trường hợp trung bình. Thuật toán PC-2 (Mackworth, 1977) đạt được tính nhất quán đường dẫn giống như cách AC-3 đạt được tính nhất quán hồ quang.

Ngay sau khi bài báo của Mackworth xuất hiện, các nhà nghiên cứu bắt đầu thử nghiệm với sự đánh đổi giữa chi phí thực thi tính nhất quán và lợi ích về mặt giảm tìm kiếm. Haralick và Elliott (1980) ủng hộ thuật toán kiểm tra chuyển tiếp tối thiểu được mô tả bởi McGregor (1979), trong khi Gaschnig (1979) đề xuất kiểm tra tính nhất quán cung đầy đủ sau mỗi lần gán biến - một thuật toán sau này được Sabin và Freuder (1994) gọi là MAC. Bài báo thứ hai cung cấp bằng chứng thuyết phục rằng trên các CSP khó hơn, kiểm tra tính nhất quán hồ quang đầy đủ sẽ được đền đáp. Freuder (1978, 1982) đã điều tra khái niệm về tính nhất quán k và mối quan hệ của nó với sự phức tạp của việc giải CSP. Dechter và Dechter (1987) đã giới thiệu tính nhất quán cung định hướng. Apt (1999) mô tả một khung thuật toán chung trong đó các thuật toán lan truyền nhất quán có thể được phân tích và các cuộc khảo sát được đưa ra bởi Bessi'ere (2006) và Bart'ak et al. (2010).

Các phương pháp đặc biệt để xử lý các ràng buộc bậc cao hơn hoặc toàn cục được phát triển đầu tiên trong bối cảnh lập trình logic ràng buộc. Marriott và Stuckey (1998) pro-

Lập trình logic ràng buộc

vide bao quát tuyệt vời về nghiên cứu trong lĩnh vực này. Hạn chế Alldiff được nghiên cứu bởi Regin (1994), Stergiou và Walsh (1999), và van Hoeve (2001). Có nhiều thuật toán suy luận phức tạp hơn cho Alldiff (xem van Hoeve và Katriel, 2006) truyền bá nhiều ràng buộc hơn nhưng tốn kém hơn về mặt tính toán để chạy. Các ràng buộc giới hạn đã được kết hợp vào lập trình logic con-straint bởi Van Hentenryck et al. (1998). Một cuộc khảo sát về các hạn chế toàn cầu được cung cấp bởi van Hoeve và Katriel (2006).

Sudoku đã trở thành CSP được biết đến rộng rãi nhất và được mô tả như vậy bởi Simonis (2005). Agerbeck và Hansen (2008) mô tả một số chiến lược và chỉ ra rằng Sudokuon, một bảng $n^2 \times n^2$ nằm trong lớp các vấn đề khó NP.

Năm 1850, C. F. Gauss đã mô tả một thuật toán quay ngược đệ quy để giải bài toán 8 chúa, đã được xuất bản trên tạp chí cờ vua Đức Schachzeitung trong năm

1848. Gauss gọi phương pháp của mình là Tatonniren, bắt nguồn từ từ tiếng Pháp t'attonner — mò mẫm xung quanh, như thể trong bóng tối. Theo Donald Knuth (giao tiếp cá nhân), RJ Walker đã giới thiệu thuật ngữ backtrack vào những năm 1950. Walker (1960) đã mô tả thuật toán quay ngược cơ bản và sử dụng nó để tìm ra tất cả các giải pháp cho bài toán 13 chúa. Golomb và Baumert (1965) đã xây dựng, với các ví dụ, lớp chung của các bài toán tổ hợp mà có thể áp dụng backtrack, và giới thiệu cái mà chúng ta gọi là phỏng đoán MRV. Bitner và Reingold (1975) đã cung cấp một cuộc khảo sát có ảnh hưởng về các kỹ thuật quay trở lại. Brelaz (1979) đã sử dụng phương pháp phỏng đoán cấp độ làm atiebreaker sau khi áp dụng phương pháp phỏng đoán MRV. Thuật toán kết quả, mặc dù đơn giản nhưng vẫn là phương pháp tốt nhất cho các đồ thị tùy ý màu k . Haralick và Elliott (1980) đề xuất phương pháp phỏng đoán giá trị ít hạn chế nhất. Phương pháp nhảy lùi cơ bản là do John Gaschnig (1977, 1979). Kondrak and van Beek (1997) đã chỉ ra rằng thuật toán này về cơ bản được gộp vào kiểm tra chuyển tiếp. Nhảy lùi theo hướng xung đột được phát minh bởi Prosser (1993). Dechter (1990a) đã giới thiệu nhảy lùi dựa trên đồ thị, giới hạn độ phức tạp của các thuật toán dựa trên nhảy lùi như một hàm của đồ thị ràng buộc (Dechter và Frost, 2002). Một hình thức quay ngược thông minh rất chung đã được phát triển sớm bởi Stallman và Sussman (1977). Kỹ thuật quay trở lại theo hướng phụ thuộc của họ kết hợp quay trở lại-

Dependency
directedbacktracking

nhảy với việc học không tốt (McAllester, 1990) và dẫn đến sự phát triển của các hệ thống duy trì sự thật (Doyle, 1979), mà chúng ta thảo luận trong Phần 10.6.2. Mối liên hệ giữa hai lĩnh vực được phân tích bởi de Kleer (1989).

Học ràng buộc

Công trình của Stallman và Sussman cũng giới thiệu ý tưởng về học tập ràng buộc, trong kết quả mà phần thu được bằng tìm kiếm có thể được lưu và sử dụng lại sau này trong tìm kiếm. Ý tưởng này đã được chính thức hóa bởi Dechter (1990a). Đánh dấu ngược (Gaschnig, 1979) là một phương pháp đặc biệt đơn giản trong đó các nhiệm vụ theo cặp nhất quán và không nhất quán được lưu và sử dụng để tránh kiểm tra lại các ràng buộc. Đánh dấu lùi có thể được kết hợp với nhảy lùi theo hướng xung đột; Kondrak và van Beek (1997) trình bày một thuật toán lại có thể chứng minh được bao gồm một trong hai phương pháp được thực hiện riêng biệt.

Phương pháp quay ngược động (Ginsberg, 1993) giữ lại các dấu hiệu một phần thành công từ các tập hợp con sau của các biến khi quay ngược qua một lựa chọn trước đó không làm mất hiệu lực của thành công sau đó. Moskewicz et al. (2001) cho thấy cách các kỹ thuật này và các kỹ thuật khác được sử dụng để tạo ra một trình giải SAT hiệu quả. Các nghiên cứu thực nghiệm về một số phương pháp theo dõi ngẫu nhiên đã được thực hiện bởi Gomes et al. (2000) và Gomes and Selman (2001). Van Beek (2006) khảo sát quay trở lại.

Tìm kiếm cục bộ trong các vấn đề thỏa mãn ràng buộc đã được phổ biến bởi công trình của Kirkpatrick et al. (1983) về ủ mô phỏng (xem Chương 4), được sử dụng rộng rãi cho các vấn đề VLSI layout và lập lịch. Beck et al. (2011) đưa ra một cái nhìn tổng quan về công việc gần đây về lập lịch trình công việc. Phương pháp phỏng đoán xung đột nhỏ lần đầu tiên được đề xuất bởi Gu (1989) và được Minton et al. (1992) phát triển độc lập. Sosic và Gu (1994) đã chỉ ra cách nó có thể được áp dụng để giải quyết bài toán 3.000.000 nữ hoàng trong vòng chưa đầy một phút. Thành công đáng kinh ngạc của tìm kiếm địa phương sử dụng xung đột nhỏ về vấn đề n-queens đã dẫn đến việc đánh giá lại bản chất và mức độ phổ biến của các vấn đề "dễ" và "khó". Peter Cheeseman et al. (1991) đã khám phá độ khó của các CSP được tạo ngẫu nhiên và phát hiện ra rằng hầu hết tất cả các vấn đề như vậy đều dễ dàng hoặc không có giải pháp. Chỉ khi các tham số của trình tạo vấn đề được đặt lại trong một phạm vi hẹp nhất định, trong đó khoảng một nửa số vấn đề có thể giải quyết được, chúng ta mới tìm thấy các trường hợp vấn đề "khó". Chúng ta thảo luận thêm về hiện tượng này trong Chương 7.

Konolige (1994) chỉ ra rằng tìm kiếm cục bộ kém hơn so với tìm kiếm ngược lại các vấn đề có mức độ cấu trúc cục bộ nhất định; điều này dẫn đến công việc kết hợp tìm kiếm và suy luận cục bộ, chẳng hạn như của Pinkas và Dechter (1995). Hoos và Tsang (2006) cung cấp một cuộc khảo sát về các kỹ thuật tìm kiếm địa phương, và sách giáo khoa được cung cấp bởi Hoos và Stützle (2004) và Aarts và Lenstra (2003).

Công trình liên quan đến cấu trúc và độ phức tạp của CSP bắt nguồn từ Freuder (1985) và Mackworth và Freuder (1985), những người đã chỉ ra rằng tìm kiếm trên các cây phù hợp với vòng cung hoạt động mà không có bất kỳ sự quay ngược nào. Một kết quả tương tự, với các phần mở rộng cho siêu đồ thị không tuần hoàn, đã được phát triển trong cộng đồng cơ sở dữ liệu (Beeri et al., 1983). Bayardo và Miranker (1994) trình bày một al-gorithm cho các CSP có cấu trúc cây chạy trong thời gian tuyến tính mà không cần bất kỳ quá trình tiền xử lý nào. Dechter (1990a) mô tả cách tiếp cận cắt chu kỳ.

Kể từ khi những bài báo đó được xuất bản, đã có rất nhiều tiến bộ trong việc phát triển các kết quả tổng quát hơn liên quan đến độ phức tạp của việc giải CSP với cấu trúc của biểu đồ ràng buộc của nó. Khái niệm về chiều rộng cây được giới thiệu bởi các nhà lý thuyết đồ thị Robertson và Seymour (1986). Dechter và Pearl (1987, 1989), dựa trên công trình của Freuder, đã áp dụng khái niệm liên quan (mà họ gọi là chiều rộng cảm ứng nhưng giống với chiều rộng cây) cho các vấn đề thỏa mãn ràng buộc và phát triển phương pháp phân hủy cây được phác thảo trong Phần 5.5.

Dựa trên công trình này và kết quả từ lý thuyết cơ sở dữ liệu, Gottlob et al. (1999a, 1999b) đã phát triển một khái niệm, chiều rộng siêu cây, dựa trên đặc điểm của CSP là ahypergraph. Ngoài việc chỉ ra rằng bất kỳ CSP nào có chiều rộng siêu cây w đều có thể được giải quyết trong thời gian $O(nw + 1 \log n)$, họ cũng chỉ ra rằng chiều rộng siêu cây bao gồm tất cả các thước đo được xác định trước đó của "chiều rộng" theo nghĩa là có những trường hợp chiều rộng siêu cây bị giới hạn và các thước đo khác không giới hạn.

Thuật toán RELSAT của Bayardo và Schrag (1997) kết hợp học ràng buộc và nhảy lùi và được chứng minh là vượt trội hơn nhiều thuật toán khác vào thời điểm đó.

Điều này dẫn đến các thuật toán tìm kiếm AND-OR áp dụng cho cả CSP và suy luận xác suất (Dechter and Mateescu, 2007). Brown et al. (1988) giới thiệu ý tưởng về sự phá vỡ đối xứng trong CSP, và Gent et al. (2006) đưa ra một cuộc khảo sát.

Lĩnh vực thỏa mãn ràng buộc phân tán xem xét việc giải quyết các CSP khi có một bộ sưu tập các tác nhân, mỗi tác nhân kiểm soát một tập hợp con của các biến ràng buộc. Đã có các hội thảo hàng năm về vấn đề này kể từ năm 2000 và phôi sống tốt ở những nơi khác (Collinet al., 1999; Pearce và cộng sự, 2008).

So sánh các thuật toán CSP chủ yếu là một khoa học thực nghiệm: một số kết quả lý thuyết cho thấy rằng một thuật toán thống trị thuật toán khác trên tất cả các vấn đề; thay vào đó, chúng ta cần chạy thử nghiệm để xem thuật toán nào hoạt động tốt hơn trên các trường hợp điển hình của vấn đề. Như Hooker (1995) chỉ ra, chúng ta cần phải cẩn thận để phân biệt giữa thử nghiệm cạnh tranh - như xảy ra giữa các thuật toán dựa trên thời gian chạy - và thử nghiệm khoa học, có mục tiêu xác định các thuộc tính của một thuật toán xác định hiệu quả của nó đối với một lớp vấn đề.

Sách giáo khoa của Apt (2003), Dechter (2003), Tsang (1993) và Lecoutre (2009), và bộ sưu tập của Rossi et al. (2006), là những nguồn tài nguyên tuyệt vời về xử lý ràng buộc. Có một số bài khảo sát hay, bao gồm cả những bài của Dechter và Frost (2002), và Bartak et al. (2010). Carbonnel và Cooper (2016) khảo sát các lớp CSP có thể xử lý được. Kondrak và van Beek (1997) đưa ra một cuộc khảo sát phân tích về các thuật toán tìm kiếm quay trở lại và Bacchus and van Run (1995) đưa ra một cuộc khảo sát thực nghiệm hơn. Lập trình ràng buộc được đề cập trong các cuốn sách của Apt (2003) và Fruhwirth và Abdennadher (2003). Các bài báo về thỏa mãn ràng buộc xuất hiện thường xuyên trên Trí tuệ nhân tạo và trên tạp chí chuyên ngành Constraints; những người giải SAT mới nhất được mô tả trong Cuộc thi SAT Quốc tế hàng năm. Địa điểm hội nghị chính là Hội nghị Quốc tế về Nguyên tắc và Thực hành Lập trình Ràng buộc, thường được gọi là CP.

CHƯƠNG 6

TÌM KIẾM VÀ TRÒ CHƠI ĐỐI NGỊCH

Trong đó chúng ta khám phá môi trường nơi các tác nhân khác đang âm mưu chống lại chúng ta.

Tìm kiếm đối nghịch

Trong chương này, chúng tôi đề cập đến môi trường cạnh tranh, trong đó hai hoặc nhiều tác nhân có mục tiêu phù hợp, làm phát sinh các vấn đề tìm kiếm đối nghịch. Thay vì đối phó với sự hỗn loạn

trong các cuộc giao tranh trong thế giới thực, chúng tôi sẽ tập trung vào các trò chơi, chẳng hạn như cờ vua, cờ vây và poker. Đối với các nhà nghiên cứu AI, bản chất đơn giản hóa của các trò chơi này là một điểm cộng: trạng thái của trò chơi để thể hiện và các tác nhân thường bị giới hạn ở một số lượng nhỏ các hành động có tác dụng được xác định bởi các quy tắc chính xác. Các trò chơi thể chất, chẳng hạn như croquet và khúc côn cầu trên băng, có nhiều mô tả phức tạp hơn, một loạt các hành động có thể xảy ra và các quy tắc khá không chính xác xác định tính hợp pháp của các hành động. Ngoại trừ bóng đá robot, những trò chơi vật lý này không thu hút nhiều sự quan tâm của cộng đồng AI.

6.1 Lý thuyết trò chơi

Sự tiết kiệm

Có ít nhất ba lập trường mà chúng ta có thể thực hiện đối với môi trường đa tác nhân. Lập trường đầu tiên, thích hợp khi có một số lượng rất lớn các tác nhân, là xem xét chúng trong tổng thể như một nền kinh tế, cho phép chúng ta làm những việc như dự đoán rằng nhu cầu ngày càng tăng sẽ khiến giá tăng, mà không cần phải dự đoán hành động của bất kỳ tác nhân riêng lẻ nào. Thứ hai, chúng ta có thể coi các tác nhân đối nghịch chỉ là một phần của môi trường - một phần làm cho môi trường không xác định. Nhưng nếu chúng ta mô phỏng các kẻ thù giống như vậy, chẳng hạn, mưa đôi khi rơi và đôi khi không, chúng ta bỏ lỡ ý tưởng rằng những kẻ thù của chúng ta đang tích cực cố gắng đánh bại chúng ta, trong khi mưa được cho là không có ý định như vậy.

Tia

Lập trường thứ ba là mô hình hóa rõ ràng các tác nhân đối nghịch với các kỹ thuật tìm kiếm cây trò chơi đối nghịch. Đó là những gì chương này đề cập. Chúng ta bắt đầu với một lớp trò chơi hạn chế và xác định nước đi tối ưu và một thuật toán để tìm nó: tìm kiếm minimax, một gen-eralization của tìm kiếm AND-OR (từ Hình 4.11). Chúng tôi chỉ ra rằng việc cắt tia tạo ra tìm kiếm

hiệu quả hơn bằng cách bỏ qua các phần của cây tìm kiếm không tạo ra sự khác biệt đối với Optimalmove. Đối với các trò chơi không tầm thường, chúng ta thường sẽ không có đủ thời gian để chắc chắn tìm ra nước đi tối ưu (ngay cả khi cắt tia); Chúng ta sẽ phải cắt bỏ cuộc tìm kiếm vào một thời điểm nào đó.

Đối với mỗi tiểu bang mà chúng tôi chọn ngừng tìm kiếm, chúng tôi hỏi ai là người chiến thắng. Để trả lời câu hỏi này, chúng ta có một lựa chọn: chúng ta có thể áp dụng một hàm đánh giá heuristic để ước tính ai đang chiến thắng dựa trên các đặc điểm của trạng thái (Phần 6.3), hoặc chúng ta có thể tính trung bình kết quả của nhiều mô phỏng nhanh của trò chơi từ trạng thái đó cho đến cuối (Phần 6.4).

Thông tin không hoàn hảo

Phần 6.5 thảo luận về các trò chơi bao gồm yếu tố may rủi (thông qua việc tung xúc xắc hoặc xáo trộn) và Phần 6.6 bao gồm các trò chơi có thông tin không hoàn hảo (chẳng hạn như poker và cầu, nơi không phải tất cả các thẻ đều hiển thị cho tất cả người chơi).

6.1.1 Trò chơi có tổng bằng không hai người chơi

Các trò chơi được nghiên cứu phổ biến nhất trong AI (chẳng hạn như cờ vua và cờ vây) là những gì các nhà lý thuyết trò chơi gọi là trò chơi xác định, hai người chơi, thay lượt, thông tin hoàn hảo, tổng bằng không. "Hoàn hảo

Thông tin hoàn hảo
Trò chơi có tổng bằng không

thông tin" là từ đồng nghĩa với "hoàn toàn có thể quan sát được"¹ và "tổng bằng không" có nghĩa là những gì tốt cho người chơi này cũng tệ cho người kia: không có kết quả "đôi bên cùng có lợi". Đối với các trò chơi, chúng tôi thường sử dụng thuật ngữ di chuyển như một từ đồng nghĩa với "hành động" và vị trí như một từ đồng nghĩa với "trạng thái".

Di chuyển
Vị trí

Chúng tôi sẽ gọi hai người chơi của mình là MAX và MIN, vì những lý do sẽ sớm trở nên rõ ràng. MAX di chuyển trước, sau đó người chơi thay phiên nhau di chuyển cho đến khi trò chơi kết thúc. Khi kết thúc trò chơi, điểm được trao cho người chơi chiến thắng và hình phạt được trao cho người thua cuộc. Agame có thể được định nghĩa chính thức với các yếu tố sau:

- S0: Trạng thái ban đầu, chỉ định cách trò chơi được thiết lập khi bắt đầu.
 - TO-MOVE(s): Người chơi có lượt di chuyển ở trạng thái s.
 - HÀNH ĐỘNG: Tập hợp các động thái pháp lý trong tiểu bang.
 - KẾT QUẢ (s, a): Mô hình chuyển tiếp, xác định trạng thái kết quả của việc lấy action a ở tiểu bang s.
 - IS-TERMINAL(s): Một bài kiểm tra đầu cuối, đúng khi trò chơi kết thúc và sai khác. Các trạng thái mà trò chơi đã kết thúc được gọi là trạng thái cuối.
 - UTILITY(s, p): Một hàm tiện ích (còn được gọi là hàm mục tiêu hoặc hàm thanh toán), xác định giá trị số cuối cùng cho người chơi p khi trò chơi kết thúc ở trạng thái cuối cùng s. In cờ vua, kết quả là thắng, thua hoặc hòa, với các giá trị 1, 0 hoặc 1/2.
- Một số trò chơi có nhiều kết quả có thể xảy ra — ví dụ: phần thưởng trong cờ thỏ cáo nằm trong khoảng từ 0 đến 192. Giống như trong Chương 3, trạng thái ban đầu, hàm ACTIONS và hàm RESULT xác định biểu đồ không gian trạng thái — một đồ thị trong đó các đỉnh là trạng thái, các cạnh là di chuyển và một trạng thái có thể tiếp cận bằng nhiều đường dẫn. Như trong Chương 3, chúng ta có thể chồng lên một cây tìm kiếm một phần của biểu đồ đó để xác định động thái nào cần thực hiện. Chúng tôi định nghĩa cây trò chơi hoàn chỉnh là một cây tìm kiếm theo dõi mọi chuỗi di chuyển đến trạng thái cuối. Gametree có thể là vô hạn nếu bản thân không gian trạng thái không giới hạn hoặc nếu các quy tắc của trò chơi cho phép các vị trí lặp lại vô hạn.

Mô hình chuyển tiếp

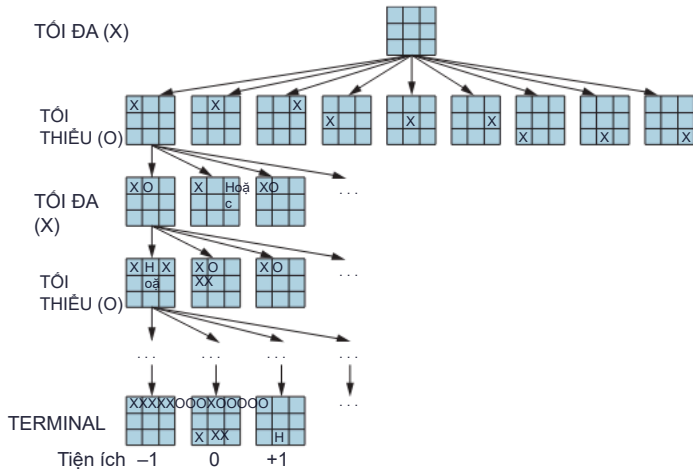
Kiểm tra đầu cuối
Trạng thái đầu cuối

Biểu đồ không gian trạng
thái
Cây tìm kiếm
Cây trò chơi

Hình 6.1 cho thấy một phần của cây trò chơi cho tic-tac-toe (noughts và crosses). Từ trạng thái ban đầu, MAX có chín nước đi có thể có. Chơi xen kẽ giữa MAX đặt X và MIN đặt O cho đến khi chúng ta đến các nút lá tương ứng với trạng thái đầu cuối sao cho một người chơi có ba ô vuông liên tiếp hoặc tất cả các ô vuông được lấp đầy. Số trên mỗi nút lá cho biết giá trị tiện ích của trạng thái đầu cuối từ quan điểm của MAX; giá trị cao là tốt cho MAX và xấu cho MIN (đó là cách người chơi lấy tên của họ).

Đối với tic-tac-toe, cây trò chơi tương đối nhỏ — ít hơn $9! = 362,880$ nút đầu cuối (chỉ với 5.478 trạng thái riêng biệt). Nhưng đối với cờ vua có hơn 1040 nút, vì vậy cây trò chơi được coi là một cấu trúc lý thuyết mà chúng ta không thể thực hiện trong thế giới vật lý.

¹ Một số tác giả tạo ra sự khác biệt, sử dụng "trò chơi thông tin không hoàn hảo" cho một trò chơi như poker trong đó người chơi có được thông tin riêng tư về ván bài của chính họ mà những người chơi khác không có, và "trò chơi có thể quan sát được một phần" có nghĩa là một trò chơi như StarCraft II nơi mỗi người chơi có thể nhìn thấy môi trường gần đó, nhưng không phải môi trường ở xa.² Cờ vua được coi là một trò chơi "tổng bằng không". Mặc dù tổng kết quả của hai người chơi là +1 cho mỗi trò chơi, không phải bằng không. "Tổng không đổi" sẽ là một thuật ngữ chính xác hơn, nhưng tổng bằng không là truyền thống và có ý nghĩa nếu bạn tưởng tượng mỗi người chơi bị tính phí vào cửa là 1/2.



Hình 6.1 Cây trò chơi (một phần) cho trò chơi tic-tac-toe. Nút trên cùng là trạng thái ban đầu và MAX di chuyển đầu tiên, đặt một X vào một ô vuông trống. Chúng tôi hiển thị một phần của cây, đưa ra các bước di chuyển xen kẽ theo MIN (O) và MAX (X), cho đến khi cuối cùng chúng tôi đạt đến trạng thái đầu cuối, có thể được gán các tiện ích theo luật chơi.

6.2 Quyết định tối ưu trong trò chơi

MAX muốn tìm một chuỗi hành động dẫn đến chiến thắng, nhưng MIN có điều gì đó để nói về điều đó. Điều này có nghĩa là chiến lược của MAX phải là một kế hoạch có điều kiện — một chiến lược ngẫu nhiên xác định phản ứng đối với từng động thái có thể có của MIN. Trong các trò chơi có kết quả nhị phân (thắng hoặc thua), chúng ta có thể sử dụng tìm kiếm AND-OR (trang 143) để tạo kế hoạch có điều kiện. Trên thực tế, đối với những trò chơi như vậy, định nghĩa về chiến lược chiến thắng cho trò chơi giống hệt với định nghĩa giải pháp cho một vấn đề lập kế hoạch không xác định: trong cả hai trường hợp, kết quả mong muốn phải được đảm bảo bất kể "bên kia" làm gì. Đối với các trò chơi có điểm số nhiều kết quả, chúng ta cần một thuật toán tổng quát hơn một chút được gọi là tìm kiếm minimax.

Tìm kiếm Minimax

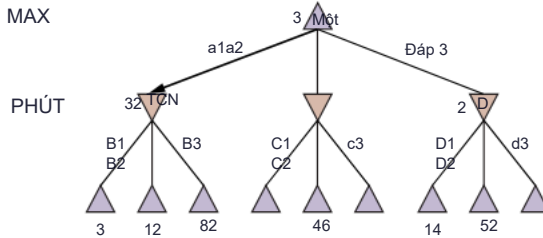
Hãy xem xét trò chơi tầm thường trong Hình 6.2. Các nước đi có thể có cho MAX tại nút gốc được dán nhãn a_1 , a_2 và a_3 . Các câu trả lời có thể có cho a_1 cho MIN là b_1 , b_2 , b_3 , v.v. Trò chơi đặc biệt này kết thúc sau một nước đi mỗi người theo MAX và MIN. (Lưu ý: Trong một số trò chơi, từ "di chuyển" có nghĩa là cả hai người chơi đã thực hiện một hành động; do đó từ ply được sử dụng để

Lớp

rõ ràng có nghĩa là một nước đi của một người chơi, đưa chúng ta sâu hơn một cấp độ trong cây trò chơi.) Các tiện ích của các trạng thái đầu cuối trong trò chơi này dao động từ 2 đến 14.

Giá trị tối thiểu

Cho một cây trò chơi, chiến lược tối ưu có thể được xác định bằng cách tính ra giá trị tối thiểu của mỗi trạng thái trong cây, mà chúng ta viết là MINIMAX(s). Giá trị minimax là tiện ích (đối với MAX) ở trạng thái đó, giả sử rằng cả hai người chơi đều chơi tối ưu từ đó cho đến khi kết thúc trò chơi. Giá trị tối thiểu của trạng thái cuối cùng chỉ là tiện ích của nó. Ở trạng thái không đầu cuối, MAX thích chuyển sang trạng thái có giá trị lớn nhất khi đến lượt MAX



Hình 6.2 Cây trò chơi hai lớp. 4 nút là "nút MAX", trong đó đến lượt MAX di chuyển và 5 nút là "nút MIN". Các nút đầu cuối hiển thị các giá trị tiện ích cho MAX; Các nút khác được gắn nhãn với các giá trị minimax của chúng. Di chuyển tốt nhất của MAX ở gốc là a1, vì nó dẫn đến trạng thái có giá trị tối đa nhỏ nhất và câu trả lời tốt nhất của MIN là b1, bởi vì nó dẫn đến trạng thái có giá trị tối đa nhỏ nhất.

di chuyển và MIN thích trạng thái có giá trị nhỏ nhất (nghĩa là giá trị nhỏ nhất cho MAX và do đó giá trị lớn nhất cho MIN). Vì vậy, chúng tôi có:

MINIMAX =

- ☐ TIỆN ÍCH, TỐI ĐA) nếu IS-TERMINAL (các)
- ☐ $\max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a))$ nếu TO-MOVE(s) = MAX
- ☐ $\min_{a \in (\text{các}) \text{ hành động}} \text{MINIMAX}(\text{KẾT QUẢ}, A))$ nếu TO-MOVE(s) = MIN

Chúng ta hãy áp dụng các định nghĩa này cho cây trò chơi trong Hình 6.2. Các nút đầu cuối ở cấp dưới lấy giá trị tiện ích của chúng từ chức năng UTILITY của trò chơi. Nút MIN đầu tiên, được dán nhãn B, có ba trạng thái kế thừa với giá trị 3, 12 và 8, vì vậy giá trị minimax của nó là 3. Tương tự, hai nút MIN còn lại có giá trị tối đa nhỏ 2. Nút gốc là nút MAX; các quốc gia kế nhiệm của nó có giá trị minimax 3, 2 và 2; Vì vậy, nó có giá trị minimax là 3. Chúng ta cũng có thể xác định quyết định tối đa ở gốc: hành động a1 là lựa chọn tối ưu cho MAX vì nó dẫn đến trạng thái có giá trị minimax cao nhất.

Định nghĩa về lối chơi tối ưu cho MAX giả định rằng MIN cũng chơi tối ưu. Điều gì sẽ xảy ra nếu MIN không chơi tối ưu? Sau đó, MAX sẽ làm ít nhất cũng như chống lại một người chơi tối ưu, có thể tốt hơn. Tuy nhiên, điều đó không có nghĩa là tốt nhất bạn nên chơi chiêu bài tối ưu tối đa khi đối mặt với đối thủ dưới mức tối ưu. Hãy xem xét một tình huống mà lối chơi tối ưu của cả hai bên sẽ dẫn đến hòa, nhưng có một nước đi mạo hiểm cho MAX dẫn đến trạng thái trong đó có 10 nước đi phản ứng có thể có của MIN mà tất cả đều có vẻ hợp lý, nhưng 9 trong số đó là thua cho MIN và một là thua cho MAX. Nếu MAX tin rằng MIN không có đủ sức mạnh tính toán để khám phá nước đi tối ưu, MAX có thể muốn thử nước đi mạo hiểm, với lý do cơ hội thắng 9/10 tốt hơn một trận hòa nhất định.

6.2.1 Thuật toán tìm kiếm minimax

Bây giờ chúng ta có thể tính toán (các) MINIMAX, chúng ta có thể biến nó thành một thuật toán tìm kiếm nước đi tốt nhất cho MAX bằng cách thử tất cả các hành động và chọn hành động có trạng thái kết quả có giá trị MINIMAX cao nhất. Hình 6.3 cho thấy thuật toán. Nó là một thuật toán đệ quy tiến hành xuống tất cả các lá của cây và sau đó sao lưu các giá trị tối đa nhỏ qua cây khi đệ quy kéo dài. Ví dụ, trong Hình 6.2, thuật toán

hàm MINIMAX-SEARCH(game, state) trả về một hành động
 người chơi $\leftarrow$ trò chơi. TO-MOVE(trạng
 thái) giá trị, di chuyển $\leftarrow$ MAX-VALUE(trò
 chơi, trạng thái) trả lại di chuyển

hàm MAX-VALUE(game, state) trả về một cặp (tiện ích, di chuyển)
 nếu trò chơi. IS-TERMINAL(state) sau đó trả lại trò chơi. UTILITY(trạng thái,
 người chơi), nullv, di chuyển $\leftarrow -\infty$ cho mỗi a trong trò chơi. ACTIONS(state) làm

$v2, a2 \leftarrow$ GIÁ TRỊ TỐI THIỂU (TRÒ CHƠI, TRÒ CHƠI. RESULT(state, a)) nếu $v2 > v$ thì

v , di chuyển $\leftarrow v2$, trả v , di chuyển

function MIN-VALUE(game, state) trả về một cặp (utility, move)
 nếu trò chơi. IS-TERMINAL(state) sau đó trả lại trò chơi. UTILITY(trạng thái,
 người chơi), nullv, di chuyển $\leftarrow +\infty$ cho mỗi a trong trò chơi. ACTIONS(state) làm

$v2, a2 \leftarrow$ GIÁ TRỊ TỐI ĐA (TRÒ CHƠI, TRÒ CHƠI. RESULT(state, a)) nếu $v2 < v$ thì

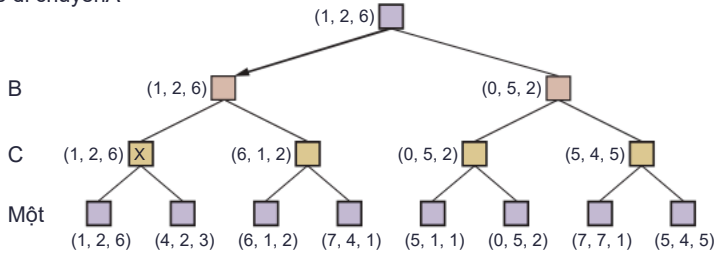
v , di chuyển $\leftarrow v2$, trả v , di chuyển

Hình 6.3 Một thuật toán để tính toán nước đi tối ưu bằng cách sử dụng minimax - nước đi dẫn đến trạng thái cuối cùng với tiện ích tối đa, với giả định rằng đối thủ chơi để giảm thiểu tiện ích. Các hàm MAX-VALUE và MIN-VALUE đi qua cây toàn bộ trò chơi, cho đến tận lá, để xác định giá trị dự phòng của một trạng thái và di chuyển để đến đó.

đầu tiên đệ quy xuống ba nút dưới cùng bên trái và sử dụng hàm UTILITY trên chúng để khám phá rằng giá trị của chúng lần lượt là 3, 12 và 8. Sau đó, nó lấy giá trị tối thiểu của các giá trị này, 3 và trả về nó dưới dạng giá trị được sao lưu của nút B. Một quá trình tương tự cho các giá trị được sao lưu là 2 cho C và 2 cho D. Cuối cùng, chúng ta lấy giá trị tối đa là 3, 2 và 2 để có được giá trị sao lưu là 3 cho nút gốc.

Thuật toán minimax thực hiện khám phá hoàn chỉnh theo chiều sâu đầu tiên của cây trò chơi. Nếu độ sâu tối đa của cây là m và có b di chuyển hợp pháp tại mỗi điểm, thì độ phức tạp thời gian của thuật toán minimax là $O(b^m)$. Độ phức tạp không gian là $O(bm)$ đối với một thuật toán tạo ra tất cả các hành động cùng một lúc, hoặc $O(m)$ đối với một thuật toán tạo ra từng hành động một (xem trang 98). Độ phức tạp theo cấp số nhân làm cho MINIMAX trở nên không thực tế đối với các trò chơi phức tạp; Ví dụ, cờ vua có hệ số phân nhánh khoảng 35 và trò chơi trung bình có độ sâu khoảng 80 lớp, và không khả thi để tìm kiếm $3580 \approx 10123$ trạng thái. Tuy nhiên, MINIMAX đóng vai trò là cơ sở cho việc phân tích toán học của trò chơi. Bằng cách xấp xỉ phân tích minimax theo nhiều cách khác nhau, chúng ta có thể rút ra các thuật toán thực tế hơn.

để di chuyển A



Hình 6.4 Ba lớp đầu tiên của cây trò chơi với ba người chơi (A, B, C). Mỗi nút được dán nhãn với các giá trị từ quan điểm của mỗi người chơi. Nước đi tốt nhất được đánh dấu ở gốc.

6.2.2 Quyết định tối ưu trong trò chơi nhiều người chơi

Nhiều trò chơi phổ biến cho phép nhiều hơn hai người chơi. Hãy để chúng tôi xem xét cách mở rộng minimaxidea sang các trò chơi nhiều người chơi. Điều này rất đơn giản từ quan điểm kỹ thuật, nhưng đặt ra một số vấn đề khái niệm mới thú vị.

Đầu tiên, chúng ta cần thay thế giá trị duy nhất cho mỗi nút bằng một vector các giá trị. Ví dụ: trong một trò chơi ba người chơi với người chơi A, B và C, một vector $\langle v_A, v_B, v_C \rangle$ được liên kết với mỗi nút. Đối với các trạng thái đầu cuối, vector này cung cấp tiện ích của trạng thái từ quan điểm của mỗi người chơi. (Trong các trò chơi hai người chơi, tổng bằng không, vector hai phần tử có thể được rút gọn thành một giá trị duy nhất vì các giá trị luôn ngược lại.) Cách đơn giản nhất để thực hiện điều này là có hàm UTILITY trả về một vector các tiện ích.

Bây giờ chúng ta phải xem xét các trạng thái không kết thúc. Hãy xem xét nút được đánh dấu X trong gametree được hiển thị trong Hình 6.4. Ở trạng thái đó, người chơi C chọn những gì để làm. Hai lựa chọn dẫn đến trạng thái đầu cuối với các vector tiện ích $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$ và $\langle v_A = 4, v_B = 2, v_C = 3 \rangle$. Vì 6 lớn hơn 3, C nên chọn nước đi đầu tiên. Điều này có nghĩa là nếu đạt đến trạng thái X, lần chơi tiếp theo sẽ dẫn đến trạng thái cuối cùng với các tiện ích $\langle v_A = 1, v_B = 2, v_C = 6 \rangle$. Do đó, giá trị được sao lưu của X là vector này. Nói chung, giá trị dự phòng của một nút n là vector tiện ích của trạng thái kế thừa với giá trị cao nhất cho người chơi chọn ở n.

Bất cứ ai chơi trò chơi nhiều người chơi, chẳng hạn như Diplomacy hoặc Settlers of Catan, nhanh chóng nhận ra rằng có nhiều điều đang diễn ra so với trò chơi hai người chơi. Trò chơi nhiều người chơi thường liên quan đến các liên minh, dù chính thức hay không chính thức, giữa những người chơi. Liên minh được thực hiện

Alliance

và bị hỏng khi trò chơi tiếp tục. Làm thế nào để chúng ta hiểu hành vi như vậy?

Các liên minh có phải là hậu quả tự nhiên của các chiến lược tối ưu cho mỗi người chơi trong một trò chơi nhiều người chơi? Hóa ra họ có thể như vậy.

Ví dụ: giả sử A và B ở vị trí yếu và C ở vị trí mạnh hơn. Sau đó, nó thường là tối ưu cho cả A và B để tấn công C thay vì nhau, kéo C tiêu diệt từng người trong số họ. Bằng cách này, sự hợp tác xuất hiện từ hành vi ích kỷ thuần túy. Tất nhiên, ngay khi C suy yếu dưới sự tấn công dữ dội chung, liên minh sẽ mất giá trị của nó, và A hoặc B có thể vi phạm thỏa thuận.

Trong một số trường hợp, các liên minh rõ ràng chỉ đơn thuần làm cụ thể những gì sẽ xảy ra theo bất kỳ cách nào. Trong các trường hợp khác, sự kỳ thị xã hội gắn liền với việc phá vỡ một liên minh, vì vậy người chơi phải cân bằng

lợi thế trước mắt của việc phá vỡ một liên minh chống lại bất lợi lâu dài của việc bị coi là không đáng tin cậy. Xem Phần 17.2 để biết thêm về những biến chứng này.

Nếu trò chơi không có tổng bằng không, thì sự hợp tác cũng có thể xảy ra chỉ với hai người chơi. Ví dụ, giả sử rằng có một trạng thái cuối cùng với các tiện ích $\langle v_A = 1000, v_B = 1000 \rangle$ và 1000 là tiện ích cao nhất có thể cho mỗi người chơi. Sau đó, chiến lược tối ưu là cả hai người chơi làm mọi thứ có thể để đạt được trạng thái này — nghĩa là người chơi sẽ tự động hợp tác để đạt được mục tiêu mong muốn chung.

6.2.3 Cắt tỉa Alpha – Beta

Cắt tỉa alpha-beta

Số lượng trạng thái trò chơi theo cấp số nhân ở độ sâu của cây. Không có thuật toán nào có thể loại bỏ hoàn toàn số mũ, nhưng đôi khi chúng ta có thể cắt nó làm đôi, tính toán quyết định minimax chính xác mà không kiểm tra mọi trạng thái bằng cách cắt tỉa (xem trang 108) các phần lớn của cây không tạo ra sự khác biệt đối với kết quả. Kỹ thuật cụ thể mà chúng tôi kiểm tra được gọi là cắt tỉa alpha-beta.

Hãy xem xét lại cây trò chơi hai lớp từ Hình 6.2. Chúng ta hãy xem qua tính toán quyết định tối ưu một lần nữa, lần này chú ý cẩn thận đến những gì chúng ta biết tại từng điểm trong quá trình. Các bước được giải thích trong Hình 6.5. Kết quả là chúng ta có thể xác định quyết định tối thiểu mà không cần đánh giá hai trong số các nút lá.

Một cách khác để xem xét điều này là đơn giản hóa công thức cho MINIMAX. Giả sử hai phần kế thừa chưa được đánh giá của nút C trong Hình 6.5 có giá trị x và y . Sau đó, giá trị của nút gốc được cho bởi

$$\begin{aligned} \text{MINIMAX (gốc)} &= \text{tối đa (tối thiểu (3, 12, 8), tối thiểu (2, x, y), tối thiểu (14, 5, 2))} \\ &= \max(3, \min(2, x, y), 2) = \max(3, z, \\ &2) \text{ trong đó } z = \min(2, x, y) \leq 2 = 3. \end{aligned}$$

Nói cách khác, giá trị của gốc và do đó quyết định tối thiểu không phụ thuộc vào giá trị của lá x và y , và do đó chúng ta có thể được cắt tỉa.

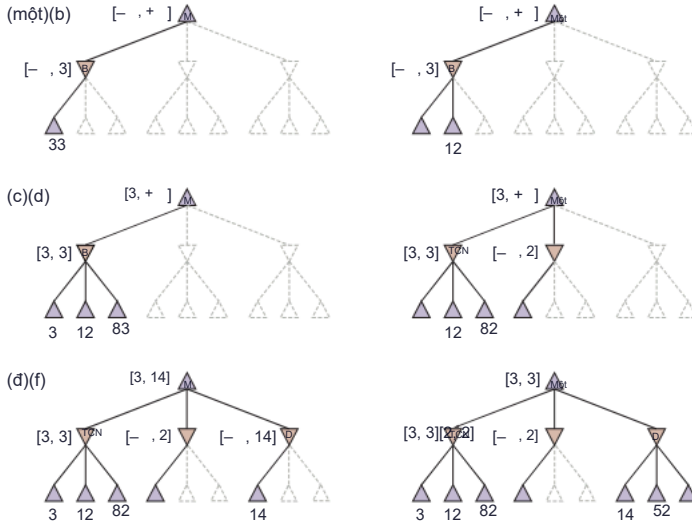
Cắt tỉa alpha-beta có thể được áp dụng cho cây ở bất kỳ độ sâu nào và thường có thể cắt tỉa toàn bộ cây phụ thay vì chỉ lá. Nguyên tắc chung là: xem xét một nút n ở đầu đó trong cây (xem Hình 6.6), sao cho Người chơi có lựa chọn di chuyển đến n . Nếu Người chơi có một lựa chọn tốt hơn ở cùng cấp độ (ví dụ: m' trong Hình 6.6) hoặc ở bất kỳ điểm nào cao hơn trên cây (ví dụ: m trong Hình 6.6), thì Người chơi sẽ không bao giờ di chuyển đến n . Vì vậy, một khi chúng ta đã tìm ra đủ về n (bằng cách kiểm tra một số hậu duệ của nó) để đi đến kết luận này, chúng ta có thể cắt tỉa nó.

Hãy nhớ rằng tìm kiếm minimax là độ sâu đầu tiên, vì vậy tại bất kỳ thời điểm nào chúng ta chỉ cần xem xét các nút dọc theo một đường dẫn duy nhất trong cây. Cắt tỉa alpha-beta lấy tên từ hai tham số bổ sung trong MAX-VALUE(state, α , β) (xem Hình 6.7) mô tả các giới hạn trên các giá trị được sao lưu xuất hiện ở bất kỳ đâu dọc theo đường dẫn:

α = giá trị của lựa chọn tốt nhất (tức là giá trị cao nhất) mà chúng ta đã tìm thấy cho đến nay tại bất kỳ điểm lựa chọn nào

dọc theo con đường cho MAX. Hãy nghĩ: α = "ít nhất". β = giá trị của lựa chọn tốt nhất (tức là giá trị thấp nhất) mà chúng ta đã tìm thấy cho đến nay tại bất kỳ điểm lựa chọn nào

dọc theo con đường cho MIN. Hãy nghĩ: β = "nhiều nhất".

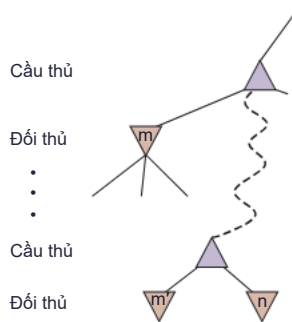


Hình 6.5 Các giai đoạn trong tính toán quyết định tối ưu cho cây trò chơi trong Hình 6.2. At mỗi điểm, chúng tôi hiển thị phạm vi các giá trị có thể có cho mỗi nút. (a) Lá đầu tiên bên dưới B có giá trị 3. Do đó, B, là một nút MIN, có giá trị tối đa là 3. (b) Lá thứ hai bên dưới B có giá trị là 12; MIN sẽ tránh được động thái này, vì vậy giá trị của B vẫn nhiều nhất là 3. (c) Lá thứ ba bên dưới B có giá trị là 8; chúng ta đã thấy tất cả các trạng thái kế thừa của B, vì vậy giá trị của B chính xác là 3. Bây giờ chúng ta có thể suy ra rằng giá trị của gốc ít nhất là 3, bởi vì MAX có một lựa chọn có giá trị 3 ở gốc. (d) Lá đầu tiên dưới C có giá trị 2. Do đó, C, là nút MIN, có giá trị tối đa là 2. Nhưng chúng ta biết rằng B có giá trị 3, vì vậy MAX sẽ không bao giờ chọn C. Do đó, không có ích gì khi nhìn vào các trạng thái kế thừa khác của C. Đây là một ví dụ về cắt tỉa alpha-beta. (e) Lá đầu tiên bên dưới D có giá trị 14, vì vậy D có giá trị tối đa là 14. Con số này vẫn cao hơn so với lựa chọn thay thế tốt nhất của MAX (tức là 3), vì vậy chúng ta cần tiếp tục khám phá các trạng thái kế thừa của D. Cũng lưu ý rằng bây giờ chúng ta có giới hạn trên tất cả các kế thừa của gốc, vì vậy giá trị của gốc cũng nhiều nhất là 14. (f) Người kế thừa thứ hai của D có giá trị 5, vì vậy một lần nữa chúng ta cần tiếp tục khám phá. Người kế thừa thứ ba có giá trị 2, vì vậy bây giờ D có giá trị chính xác là 2. Quyết định gốc của MAX là chuyển sang B, cho giá trị là 3.

Tìm kiếm alpha-beta cập nhật các giá trị của α và β khi nó diễn ra và cắt tỉa các nhánh còn lại tại một nút (tức là chấm dứt lệnh gọi đệ quy) ngay khi giá trị của nút hiện tại được biết là kém hơn giá trị α hoặc β hiện tại cho MAX hoặc MIN, tương ứng. Thuật toán hoàn chỉnh được đưa ra trong Hình 6.7. Hình 6.5 theo dõi tiến trình của thuật toán trên cây trò chơi.

6.2.4 Di chuyển thứ tự

Hiệu quả của việc cắt tỉa alpha-beta phụ thuộc nhiều vào thứ tự kiểm tra các trạng thái. Ví dụ, trong Hình 6.5 (e) và (f), chúng ta không thể cắt tỉa bất kỳ người kế thừa nào của D vì những người kế thừa tối tệ nhất (theo quan điểm của MIN) được tạo ra trước. Nếu



Hình 6.6 Trường hợp chung cho việc cắt tỉa alpha-beta. Nếu m hoặc m' tốt hơn n cho Người chơi, chúng ta sẽ không bao giờ đạt được n trong cuộc chơi.

function ALPHA-BETA-SEARCH(game, state) trả về một hành động
 người chơi $\leftarrow$ trò chơi. TO-MOVE(trạng
 thái) giá trị, di chuyển $\leftarrow$ MAX-VALUE(trò chơi,
 trạng thái, $-\infty$, $+\infty$) trả lại di chuyển

hàm MAX-VALUE(game, state, α , β) trả về một cặp (tiện ích, di chuyển)
 nếu trò chơi. IS-TERMINAL(state) sau đó trả lại trò chơi. UTILITY(state,
 player), null $\leftarrow -\infty$ cho mỗi a trong trò chơi. ACTIONS(state) làm

$v_2, a_2 \leftarrow$ GIÁ TRỊ TỐI THIỂU (TRÒ CHƠI, $\text{TRÒ CHƠI.RESULT}(\text{state}, a)$, α , β) if $v_2 > v$ thì

v , di chuyển $\leftarrow v_2, a \leftarrow$
 MAX(α , v) nếu $v \geq \beta$ thì trả
 lại v , di chuyển trả lại v , di
 chuyển

function MIN-VALUE(game, state, α , β) trả về một cặp (utility, move)
 nếu trò chơi. IS-TERMINAL(state) sau đó trả lại trò chơi. UTILITY(trạng
 thái, người chơi), null $\leftarrow +\infty$ cho mỗi a trong trò chơi. ACTIONS(state) làm

$v_2, a_2 \leftarrow$ GIÁ TRỊ TỐI ĐA (TRÒ CHƠI, $\text{TRÒ CHƠI.RESULT}(\text{state}, a)$, α , β) if $v_2 < v$ thì

v , di chuyển $\leftarrow v_2, a \leftarrow$
 MIN(β , v) nếu $v \leq \alpha$ thì trả v ,
 di chuyển trả lại v , di chuyển

Hình 6.7 Thuật toán tìm kiếm alpha-beta. Lưu ý rằng các hàm này giống như các hàm MINIMAX-SEARCH trong Hình 6.3, ngoại trừ việc chúng ta duy trì các giới hạn trong các biến α và β , và sử dụng chúng để cắt tìm kiếm khi một giá trị nằm ngoài giới hạn.

người kế vị thứ ba của D đã được tạo ra trước, với giá trị 2, chúng ta sẽ có thể cắt tỉa hai người kế vị còn lại. Điều này cho thấy rằng có thể đáng để thử trước tiên kiểm tra những người có nhiệm vụ khả năng là tốt nhất.

Nếu điều này có thể được thực hiện một cách hoàn hảo, alpha-beta sẽ chỉ cần kiểm tra các nút O ($b \cdot m / 2$) để chọn nước đi tốt nhất, thay vì O ($b \cdot m$) cho minimax. Điều này có nghĩa là hệ số phân nhánh hiệu quả trở thành $\sqrt{b}$ thay vì b — đối với cờ vua, khoảng 6 thay vì 35. Nói cách khác, alpha-beta với thứ tự di chuyển hoàn hảo có thể giải một cây sáu gấp đôi minimax trong cùng một khoảng thời gian. Với thứ tự di chuyển ngẫu nhiên, tổng số nút được kiểm tra sẽ xấp xỉ O ($b \cdot m / 4$) đối với b vừa phải. Bây giờ, rõ ràng là chúng ta không thể đạt được moveordering hoàn hảo — trong trường hợp đó, chức năng đặt hàng có thể được sử dụng để chơi một trò chơi hoàn hảo! Nhưng chúng ta thường có thể đến khá gần. Đối với cờ vua, một chức năng sắp xếp thứ tự khá đơn giản (chẳng hạn như thử bắt trước, sau đó là đe dọa, sau đó di chuyển tiến, và sau đó là di chuyển lùi) đưa bạn đến khoảng 2 của kết quả O ($b \cdot m / 2$) trong trường hợp tốt nhất.

Thêm các sơ đồ sắp xếp thứ tự di chuyển động, chẳng hạn như thử trước các nước đi được tìm thấy là tốt nhất trong quá khứ, đưa chúng ta khá gần đến giới hạn lý thuyết. Quá khứ có thể là động thái trước đó - thường là những mối đe dọa tương tự vẫn còn - hoặc nó có thể đến từ việc khám phá trước đó về động thái hiện tại thông qua một quá trình đào sâu lặp đi lặp lại (xem trang 98). Đầu tiên, tìm kiếm sâu và ghi lại thứ hạng của các nước đi dựa trên đánh giá của họ. Sau đó tìm kiếm một lớp sâu hơn, sử dụng xếp hạng trước đó để thông báo thứ tự di chuyển; và như vậy. Thời gian tìm kiếm tăng lên từ việc đào sâu lặp đi lặp lại có thể được bù đắp nhiều hơn từ việc sắp xếp thứ tự di chuyển tốt hơn. Các chiều thức tốt nhất được gọi là chiều thức sát thủ, và thử chúng trước được gọi là chiều thức giết người.

Di chuyển sát thủ

Trong Phần 3.3.3, chúng tôi lưu ý rằng các đường dẫn dư thừa đến các trạng thái lặp lại có thể gây ra sự gia tăng theo cấp số nhân trong chi phí tìm kiếm và việc giữ một bảng các trạng thái đã đạt được trước đó có thể giải quyết vấn đề này. Trong tìm kiếm cây trò chơi, các trạng thái lặp lại có thể xảy ra do chuyển vị — khác nhau

Transposition

hoán vị của chuỗi di chuyển kết thúc ở cùng một vị trí và vấn đề có thể được giải quyết bằng một bảng chuyển vị lưu trữ giá trị heuristic của các trạng thái.

Bảng chuyển vị

Ví dụ, giả sử Trắng có một nước đi w_1 có thể được trả lời bởi Đen với b_1 và nước đi không liên quan w_2 ở phía bên kia của bàn cờ có thể được trả lời bởi b_2 và chúng ta tìm kiếm trình tự nước đi $[w_1, b_1, w_2, b_2]$; Hãy gọi trạng thái kết quả là S. Sau khi khám phá một cây con lớn bên dưới s, chúng ta tìm thấy giá trị dự phòng của nó, chúng ta lưu trữ trong bảng chuyển vị. Sau đó, khi chúng ta tìm kiếm chuỗi các nước đi $[w_2, b_2, w_1, b_1]$, chúng ta lại kết thúc trong s và chúng ta có thể tra cứu giá trị thay vì lặp lại tìm kiếm. Trong cờ vua, việc sử dụng các bảng chuyển vị rất hiệu quả, cho phép chúng ta tăng gấp đôi độ sâu tìm kiếm có thể truy cập trong cùng một khoảng thời gian.

Ngay cả với việc cắt tỉa alpha-beta và sắp xếp thứ tự di chuyển thông minh, minimax sẽ không hoạt động cho các trò chơi như cờ vua và cờ vây, bởi vì vẫn còn quá nhiều trạng thái để khám phá trong thời gian có sẵn. Trong bài báo đầu tiên về chơi trò chơi trên máy tính, Lập trình máy tính để chơi cờ vua (Shannon, 1950), Claude Shannon đã nhận ra vấn đề này và đề xuất hai chiến lược: chiến lược loại A xem xét tất cả các nước đi có thể có đến một độ sâu nhất định trong cây tìm kiếm, và sau đó

Chiến lược loại A

sử dụng hàm đánh giá heuristic để ước tính tiện ích của các trạng thái ở độ sâu đó. Nó khám phá một phần rộng nhưng nông của cây. Chiến lược Loại B bỏ qua các nước đi có vẻ xấu và

Chiến lược loại B

theo những dòng hứa hẹn "càng xa càng tốt". Nó khám phá một phần sâu nhưng hẹp của cây. Các lịch sử, hầu hết các chương trình cờ vua đều là Loại A (mà chúng tôi sẽ đề cập trong phần tiếp theo), trong khi các chương trình cờ vây thường là Loại B (được đề cập trong Phần 6.4), bởi vì hệ số phân nhánh cao hơn nhiều trong cờ vây. Gần đây hơn, các chương trình Loại B đã cho thấy lỗi chơi cấp độ vô địch thế giới trên nhiều trò chơi khác nhau, bao gồm cả cờ vua (Silver et al., 2018).

6.3 Tìm kiếm cây Alpha – Beta heuristic

Kiểm tra giới hạn

Để tận dụng thời gian tính toán hạn chế của chúng ta, chúng ta có thể cắt bỏ tìm kiếm sớm và áp dụng chức năng đánh giá heuristic cho các trạng thái, xử lý hiệu quả các nút không phải là đầu cuối. Nói cách khác, chúng ta thay thế hàm UTILITY bằng EVAL, ước tính tiện ích của astate.

Chúng tôi cũng thay thế kiểm tra đầu cuối bằng một thử nghiệm cắt, phải trả về true cho

trạng thái cuối cùng, nhưng nếu không thì được tự do quyết định khi nào nên cắt tìm kiếm, dựa trên độ sâu tìm kiếm và bất kỳ thuộc tính nào của trạng thái mà nó chọn xem xét. Điều đó cho chúng ta công thức H-MINIMAX(s, d) cho giá trị minimax heuristic của trạng thái s ở độ sâu tìm kiếm d:

H-MINIMAX (s, d) =

- ☐ EVAL(s, MAX) nếu IS-CUTOFF(s, d)
- ☐ $\max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1)$ if TO-MOVE(s) =
- ☐ $\max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1)$ if TO-MOVE(s) = MIN.

6.3.1 Chức năng đánh giá

Một hàm đánh giá heuristic EVAL(s, p) trả về ước tính về tiện ích dự kiến của các trạng thái đối với người chơi p, giống như các hàm heuristic của Chương 3 trả về ước tính khoảng cách đến mục tiêu. Đối với các trạng thái đầu cuối, nó phải là EVAL(s, p) = UTILITY(s, p) và đối với các trạng thái không phải là giai đoạn cuối, đánh giá phải ở đâu đó giữa thua lỗ và thắng: UTILITY(thua, p) ≤ EVAL(s, p) ≤ UTILITY(thắng, p).

Ngoài những yêu cầu đó, điều gì tạo nên một chức năng đánh giá tốt? Đầu tiên, việc tính toán không được mất quá nhiều thời gian! (Toàn bộ vấn đề là tìm kiếm nhanh hơn.) Thứ hai, chức năng đánh giá phải có mối tương quan chặt chẽ với cơ hội chiến thắng thực tế. Người ta có thể thất mắc về cụm từ "cơ hội chiến thắng". Rốt cuộc, cờ vua không phải là một trò chơi may rủi: chúng ta biết chắc chắn tình trạng hiện tại và không có xúc xắc nào liên quan; Nếu không có người chơi nào thực hiện sai lầm, kết quả sẽ được xác định trước. Nhưng nếu tìm kiếm phải bị cắt ở các trạng thái không kết thúc, thì thuật toán sẽ nhất thiết không chắc chắn về kết quả cuối cùng của các trạng thái đó (mặc dù sự không chắc chắn đó có thể được giải quyết bằng tài nguyên tính toán vô hạn).

Tính năng

Chúng ta hãy làm cho ý tưởng này cụ thể hơn. Hầu hết các hàm đánh giá hoạt động bằng cách tính toán các tính năng khác nhau của trạng thái — ví dụ, trong cờ vua, chúng ta sẽ có các tính năng cho số

Tốt trắng, Tốt đen, Hậu trắng, Nữ hoàng đen, v.v. Các đặc điểm, kết hợp với nhau, xác định các loại hoặc lớp tương đương khác nhau của trạng thái: các trạng thái trong một danh mục có cùng giá trị cho tất cả các đặc điểm. Ví dụ: một thể loại có thể chứa tất cả các trò chơi kết thúc hai tốt so với một tốt. Bất kỳ danh mục nhất định nào cũng sẽ chứa một số trạng thái dẫn đến chiến thắng (với lỗi chơi hoàn hảo), một số dẫn đến hòa và một số dẫn đến thua.

Giá trị kỳ vọng

Chức năng đánh giá không biết trạng thái nào là trạng thái nào, nhưng nó có thể trả về một giá trị duy nhất ước tính tỷ lệ các trạng thái với mỗi kết quả. Ví dụ, giả sử kinh nghiệm của chúng tôi cho thấy rằng 82% các tiểu bang gặp phải trong hai con tốt so với một con tốt dẫn đến chiến thắng (tiện ích +1); 2% để thua (0) và 16% cho một trận hòa (1/2). Sau đó, một đánh giá hợp lý cho các trạng thái trong danh mục là giá trị kỳ vọng: $(0,82 \times +1) + (0,02 \times 0) +$

$(0,16 \times 1/2) = 0,90$. Về nguyên tắc, giá trị kỳ vọng có thể được xác định cho từng loại trạng thái, dẫn đến một chức năng đánh giá hoạt động cho bất kỳ trạng thái nào.

Trong thực tế, loại phân tích này đòi hỏi quá nhiều phạm trù và do đó quá nhiều kinh nghiệm để ước tính tất cả các xác suất. Thay vào đó, hầu hết các hàm đánh giá tính toán các đóng góp số riêng biệt từ mỗi tính năng và sau đó kết hợp chúng để tìm tổng giá trị. Cho

loại kinh nghiệm không có sẵn, trọng số của hàm đánh giá có thể được ước tính bằng các kỹ thuật học máy của Chương 23. Áp dụng các kỹ thuật này vào cờ vua đã xác nhận rằng một giám mục thực sự có giá trị khoảng ba con tốt, và có vẻ như kinh nghiệm hàng thế kỷ của con người có thể được sao chép chỉ trong vài giờ học máy.

6.3.2 Cắt tìm kiếm

Bước tiếp theo là sửa đổi ALPHA-BETA-SEARCH để nó sẽ gọi hàm EVAL heuristic khi thích hợp để cắt tìm kiếm. Chúng tôi thay thế hai dòng trong Hình 6.7 để cập đến IS-TERMINAL bằng dòng sau:

nếu trò chơi. IS-CUTOFF(trạng thái, độ sâu) sau đó trở lại trò chơi. EVAL(trạng thái, người chơi), null

Chúng ta cũng phải sắp xếp một số sổ sách kế toán để độ sâu hiện tại được tăng lên trên mỗi lần gọi đệ quy. Cách tiếp cận đơn giản nhất để kiểm soát số lượng tìm kiếm là đặt giới hạn độ sâu cố định để IS-CUTOFF(state, depth) trả về true cho tất cả độ sâu lớn hơn độ sâu cố định d (cũng như cho tất cả các trạng thái đầu cuối). Độ sâu d được chọn để một nước đi được chọn trong thời gian được phân bổ. Một cách tiếp cận mạnh mẽ hơn là áp dụng đào sâu lặp đi lặp lại. (Xem Chương 3.) Khi hết thời gian, chương trình trả về nước đi được chọn bởi tìm kiếm hoàn thành sâu nhất. Như một phần thưởng, nếu trong mỗi vòng đào sâu lặp đi lặp lại, chúng tôi giữ các mục trong bảng chuyển vị, các vòng tiếp theo sẽ nhanh hơn và chúng tôi có thể sử dụng các đánh giá để cải thiện thứ tự di chuyển.

Những cách tiếp cận đơn giản này có thể dẫn đến lỗi do bản chất gần đúng của hàm evaluation. Hãy xem xét lại chức năng đánh giá đơn giản cho cờ vua dựa trên lợi thế vật chất. Giả sử chương trình tìm kiếm đến giới hạn độ sâu, đạt đến vị trí trong Hình 6.8 (b), trong đó Đen dẫn trước một hiệp sĩ và hai con tốt. Nó sẽ báo cáo điều này như giá trị heuristic của tiểu bang, do đó tuyên bố rằng tiểu bang có thể là một chiến thắng có thể xảy ra của Black. Nước đi tiếp theo của ButWhite bắt được nữ hoàng của Đen mà không có sự đền bù. Do đó, vị trí này thực sự thuận lợi cho White, nhưng điều này chỉ có thể được nhìn thấy bằng cách nhìn về phía trước.

Sự im lặng

Chức năng đánh giá chỉ nên được áp dụng cho các vị trí tĩnh lặng — nghĩa là, các vị trí mà không có động thái nào đang chờ xử lý (chẳng hạn như bắt Nữ hoàng) sẽ làm thay đổi đánh giá một cách điên cuồng. Đối với các vị trí không tĩnh, IS-CUTOFF trả về false và tìm kiếm tiếp tục cho đến khi đạt đến các vị trí tĩnh. Tìm kiếm tĩnh lặng bổ sung này đôi khi là

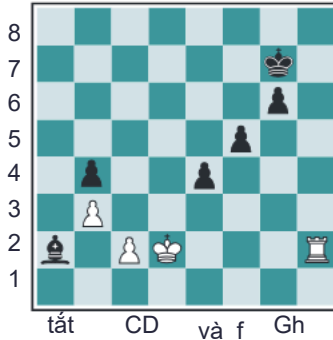
Tìm kiếm tĩnh lặng

bị hạn chế chỉ xem xét một số loại nước đi nhất định, chẳng hạn như nước đi bắt bóng, sẽ nhanh chóng giải quyết những điều không chắc chắn trong vị trí.

Hiệu ứng chân trời

Hiệu ứng chân trời khó loại bỏ hơn. Nó phát sinh khi chương trình đang đối mặt với một nước đi của đối thủ gây sát thương nghiêm trọng và cuối cùng là không thể tránh khỏi, nhưng có thể tạm thời tránh được bằng cách sử dụng các chiến thuật trì hoãn. Hãy xem xét vị trí cờ vua trong Hình 6.9. It rõ ràng là không có cách nào để giám mục đen trốn thoát. Ví dụ, xe trắng có thể bắt nó bằng cách di chuyển đến h1, sau đó là a1, sau đó là a2; chụp ở độ sâu 6 lớp.

Nhưng Black có một chuỗi các động thái đẩy việc bắt giữ giám mục "qua đường chân trời". Giả sử Đen tìm kiếm độ sâu 8 lớp. Hầu hết các nước đi của Đen sẽ dẫn đến việc cuối cùng bắt giữ giám mục, và do đó sẽ được đánh dấu là nước đi "xấu". Nhưng Đen cũng sẽ xem xét chuỗi các nước đi bắt đầu bằng cách kiểm tra vua bằng một con tốt và lôi kéo vua bắt được con tốt. Đen sau đó có thể làm điều tương tự với một con tốt thứ hai. Điều đó có đủ động thái để việc bắt giữ giám mục sẽ không bị phát hiện trong phần còn lại của cuộc tìm kiếm của Black. Black nghĩ rằng lỗi chơi đã cứu giám mục với cái giá của hai



Hình 6.9 Hiệu ứng đường chân trời. Với việc Black di chuyển, giám mục da đen chắc chắn sẽ bị diệt vong. Nhưng Đen có thể ngăn chặn sự kiện đó bằng cách kiểm tra vua trắng bằng những con tốt của nó, khuyến khích vua bắt được những con tốt. Điều này đẩy sự mất mát không thể tránh khỏi của giám mục qua đường chân trời, và do đó thuật toán tìm kiếm hy sinh tốt được coi là nước đi tốt hơn là nước đi xấu.

tốt, trong khi thực tế tất cả những gì nó làm là lãng phí những con tốt và đẩy việc bắt giữ giám mục không thể tránh khỏi ra ngoài đường chân trời mà Đen có thể nhìn thấy.

Một chiến lược để giảm thiểu hiệu ứng chân trời là cho phép các phần mở rộng đơn lẻ, di chuyển Phần mở rộng số ít "rõ ràng tốt hơn" so với tất cả các nước đi khác ở một vị trí nhất định, ngay cả khi việc tìm kiếm thường bị cắt tại thời điểm đó. Trong ví dụ của chúng ta, một cuộc tìm kiếm sẽ tiết lộ rằng ba nước đi của xe trắng — h2 đến h1, sau đó là h1 đến a1 và sau đó là a1 bắt giữ tượng trên a2 — mỗi nước đi rõ ràng là nước đi tốt hơn, vì vậy ngay cả khi một chuỗi các nước đi tốt đẩy chúng ta đến chân trời, những nước đi rõ ràng tốt hơn này sẽ có cơ hội để mở rộng tìm kiếm. Điều này làm cho cây sâu hơn, nhưng vì thường có một vài phần mở rộng đơn lẻ, chiến lược này không thêm nhiều nút tổng thể vào cây và đã được chứng minh là có hiệu quả trong thực tế.

6.3.3 Cắt tia về phía trước

Cắt tia alpha-beta cắt tia cành cây không thể ảnh hưởng đến đánh giá cuối cùng, nhưng cắt tia về phía trước cắt tia các động tác có vẻ là động tác kém, nhưng có thể tốt

Cắt tia về phía trước

một. Do đó, chiến lược tiết kiệm thời gian tính toán với nguy cơ mắc lỗi. Theo thuật ngữ của Shannon, đây là một chiến lược Loại B. Rõ ràng, hầu hết những người chơi cờ vua con người đều làm điều này, chỉ xem xét một vài nước đi từ mỗi vị trí (ít nhất là có ý thức).

Một cách tiếp cận để cắt tia về phía trước là tìm kiếm chùm tia (xem trang 133): trên mỗi lớp, chỉ xem xét một "chùm tia" của n nước đi tốt nhất (theo chức năng đánh giá) thay vì xem xét tất cả các nước đi có thể. Thật không may, cách tiếp cận này khá nguy hiểm vì không có gì đảm bảo rằng nước đi tốt nhất sẽ không bị cắt tia.

Thuật toán PROBCUT, hoặc cắt xác suất, (Buro, 1995) là một phiên bản cắt tia chuyển tiếp của tìm kiếm alpha-beta sử dụng số liệu thống kê thu được từ kinh nghiệm trước đó để giảm khả năng di chuyển tốt nhất sẽ được cắt tia. Tìm kiếm alpha-beta cắt tia bất kỳ nút nào có thể chứng minh được bên ngoài cửa sổ hiện tại (α , β). PROBCUT cũng cắt tia các nút có thể nằm ngoài win-dow. Nó tính toán xác suất này bằng cách thực hiện một tìm kiếm nông để tính toán giá trị được sao lưu

v của một nút và sau đó sử dụng kinh nghiệm trong quá khứ để ước tính khả năng điểm v ở độ sâu d trong cây sẽ ở bên ngoài (α , β). Buro đã áp dụng kỹ thuật này cho chương trình Othello của mình, LOGISTELLO, và nhận thấy rằng một phiên bản chương trình của anh ấy với PROBCUT đánh bại phiên bản thông thường 64% thời gian, ngay cả khi phiên bản thông thường được cho gấp đôi thời gian.

Giảm di chuyển muộn

Một kỹ thuật khác, giảm di chuyển muộn, hoạt động theo giả định rằng thứ tự di chuyển đã được thực hiện tốt, và do đó các nước đi xuất hiện sau đó trong danh sách các nước đi có thể xảy ra ít có khả năng là nước đi tốt. Nhưng thay vì cắt tỉa chúng hoàn toàn, chúng ta chỉ giảm độ sâu mà chúng ta tìm kiếm các động tác này, do đó tiết kiệm thời gian. Nếu tìm kiếm giảm trở lại với giá trị cao hơn giá trị α hiện tại, chúng ta có thể chạy lại tìm kiếm với độ sâu đầy đủ.

Kết hợp tất cả các kỹ thuật được mô tả ở đây dẫn đến một chương trình có thể chơi cờ vua đáng tin dụng (hoặc các trò chơi khác). Giả sử chúng ta đã thực hiện một hàm đánh giá forcess, một thử nghiệm cắt hợp lý với tìm kiếm tĩnh lặng. Chúng ta cũng hãy giả sử rằng, sau nhiều tháng bit-bashing tê liệt, chúng ta có thể tạo và đánh giá khoảng một triệu nút mỗi giây trên PC mới nhất. Hệ số phân nhánh cho cờ vua trung bình là khoảng 35 và 355 là khoảng 50 triệu, vì vậy nếu chúng ta sử dụng tìm kiếm minimax, chúng ta có thể nhìn về phía trước chỉ năm lớp trong khoảng một phút tính toán; Các quy tắc cạnh tranh sẽ không cho chúng tôi đủ thời gian để tìm kiếm sáu lớp. Mặc dù không phải là không đủ năng lực, một chương trình như vậy có thể bị đánh bại bởi một người chơi cờ vua trung bình, người đôi khi có thể lên kế hoạch sáu hoặc tám lớp trước.

Với tìm kiếm alpha-beta và một bảng chuyển vị lớn, chúng tôi có được khoảng 14 lớp, dẫn đến trình độ chơi chuyên nghiệp. Chúng tôi có thể đổi PC của mình để lấy một máy trạm với 8 GPU, giúp chúng tôi có hơn một tỷ nút mỗi giây, nhưng để có được trạng thái kiện tướng, chúng tôi vẫn cần một chức năng đánh giá được điều chỉnh rộng rãi và một cơ sở dữ liệu lớn về các bước di chuyển cuối trò chơi. Các chương trình topchess như STOCKFISH có tất cả những điều này, thường đạt độ sâu từ 30 trở lên trong cây tìm kiếm và vượt xa khả năng của bất kỳ người chơi nào.

6.3.4 Tìm kiếm so với tra cứu

Bằng cách nào đó, có vẻ như quá mức cần thiết đối với một chương trình cờ vua để bắt đầu một trò chơi bằng cách xem xét một cái cây của một tỷ trạng thái trò chơi, chỉ để kết luận rằng nó sẽ chơi tốt cho e4 (nước đi đầu tiên phổ biến nhất). Những cuốn sách mô tả lối chơi tốt trong trò chơi mở đầu và kết thúc trong cờ vua đã có sẵn trong hơn một thế kỷ (Tattersall, 1911). Do đó, không có gì ngạc nhiên khi nhiều chương trình chơi trò chơi sử dụng tra cứu bảng thay vì tìm kiếm phân mở đầu và kết thúc của trò chơi.

Đối với các lỗ hổng, máy tính chủ yếu dựa vào chuyên môn của con người. Lời khuyên tốt nhất của các chuyên gia con người về cách chơi mỗi phần mở có thể được sao chép từ sách và nhập vào bảng để máy tính sử dụng. Ngoài ra, máy tính có thể thu thập số liệu thống kê từ cơ sở dữ liệu của các trò chơi đã chơi trước đó để xem chuỗi mở đầu nào thường dẫn đến chiến thắng nhất. Đối với một vài nước đi đầu tiên, có rất ít khả năng và hầu hết các vị trí sẽ nằm trong bảng. Thông thường sau khoảng 10 hoặc 15 nước đi, chúng ta kết thúc ở một vị trí hiếm khi thấy và chương trình phải chuyển từ tra cứu bảng sang tìm kiếm.

Gần cuối trò chơi, một lần nữa có ít vị trí khả thi hơn, và do đó việc tra cứu sẽ dễ dàng hơn. Nhưng đây là máy tính có chuyên môn: phân tích máy tính về trò chơi kết thúc vượt xa khả năng của con người. Người mới làm quen có thể giành chiến thắng trong trò chơi kết thúc vua và xe đầu với vua (KRK) bằng cách tuân theo một vài quy tắc đơn giản. Các kết thúc khác, chẳng hạn như vua, giám mục và hiệp sĩ vua (KBNK), rất khó thành thạo và không có mô tả chiến lược ngắn gọn.

Mặt khác, máy tính hoàn toàn có thể giải quyết trò chơi cuối cùng bằng cách tạo ra một chính sách, đó là ánh xạ từ mọi trạng thái có thể đến nước đi tốt nhất trong trạng thái đó. Sau đó là máy tính

có thể chơi hoàn hảo bằng cách tra cứu nước đi đúng trong bảng này. Bảng được xây dựng bằng tìm kiếm minimax ngược dòng: bắt đầu bằng cách xem xét tất cả các cách để đặt các mảnh KBNK trên Retrograde

ván. Một số vị trí là chiến thắng cho người da trắng; đánh dấu chúng như vậy. Sau đó đảo ngược các quy tắc của cờ vua để thực hiện các nước đi đảo ngược thay vì nước đi. Bắt kỳ nước đi nào của Trắng, bất kể Black phản ứng bằng nước đi nào, kết thúc ở vị trí được đánh dấu là thắng, cũng phải là một chiến thắng. Tiếp tục tìm kiếm này cho đến khi tất cả các vị trí có thể được giải quyết là thắng, thua hoặc hòa, và bạn có bảng tra cứu không thể sai lầm cho tất cả các trò chơi kết thúc với những quân cờ đó. Điều này đã được thực hiện không chỉ đối với kết thúc KBNK mà còn cho tất cả các kết thúc có bảy mảnh hoặc ít hơn. Các bảng chứa 400 nghìn tỷ vị trí. Một chiếc bàn tám mảnh sẽ yêu cầu 40 triệu tỷ vị trí.

6.4 Tìm kiếm cây Monte Carlo

Trò chơi cờ vây minh họa hai điểm yếu chính của tìm kiếm cây alpha-beta heuristic: Thứ nhất, cờ vây có hệ số phân nhánh bắt đầu từ 361, có nghĩa là tìm kiếm alpha-beta sẽ chỉ giới hạn ở 4 hoặc 5 lớp. Thứ hai, rất khó để xác định một chức năng đánh giá tốt cho cờ vây vì giá trị vật chất không phải là một chỉ số mạnh mẽ và hầu hết các vị trí đều thay đổi cho đến khi kết thúc. Để đáp ứng hai thách thức này, các chương trình cờ vây hiện đại đã từ bỏ tìm kiếm alpha-beta và thay vào đó sử dụng một chiến lược gọi là tìm kiếm cây Monte Carlo (MCTS).3

Chiến lược MCTS cơ bản không sử dụng chức năng đánh giá heuristic. Thay vào đó, giá trị của một trạng thái được ước tính là tổng ích trung bình trên một số mô phỏng của các trò chơi hoàn chỉnh bắt đầu từ tiểu bang. Mô phỏng (còn được gọi là rollout hoặc rollout) chọn các nước đi trước cho một người chơi, sau đó cho người kia, lặp lại cho đến khi đạt đến vị trí đầu cuối. Tại thời điểm đó, các quy tắc của trò chơi (không phải phỏng đoán sai lầm) xác định ai đã thắng hoặc thua, và bằng số điểm nào. Đối với các trò chơi mà kết quả duy nhất là thắng hoặc thua, "tiền ích trung bình" giống như "tỷ lệ thắng".

Làm thế nào để chúng ta chọn những động thái để thực hiện trong quá trình rollout? Nếu chúng ta chỉ chọn ngẫu nhiên, thì sau nhiều lần mô phỏng, chúng ta sẽ nhận được câu trả lời cho câu hỏi "nước đi tốt nhất là gì nếu cả hai người chơi chơi ngẫu nhiên?" Đối với một số trò chơi đơn giản, đó là câu trả lời giống như "nước đi tốt nhất nếu cả hai người chơi đều chơi tốt?", nhưng đối với hầu hết các trò chơi thì không. Để có được thông tin hữu ích từ rollout, chúng ta cần một chính sách rollout thiên vị các động thái hướng tới

những cái tốt. Đối với cờ vây và các trò chơi khác, các chính sách chơi đã được học thành công từ tự chơi bằng cách sử dụng mạng nơ-ron. Đôi khi các phương pháp phỏng đoán dành riêng cho trò chơi được sử dụng, chẳng hạn như "xem xét các nước đi bất bóng" trong cờ vua hoặc "chiếm góc vuông" trong Othello.

Với một chính sách rollout, tiếp theo chúng ta cần quyết định hai điều: chúng ta bắt đầu rollouts từ vị trí nào và chúng ta phân bổ bao nhiêu rollout cho mỗi vị trí? Câu trả lời đơn giản nhất, được gọi là tìm kiếm Monte Carlo thuần túy, là thực hiện N mô phỏng bắt đầu từ hiện tại

trạng thái của trò chơi và theo dõi nước đi nào có thể xảy ra từ vị trí hiện tại có tỷ lệ thắng cao nhất.

Đối với một số trò chơi ngẫu nhiên, điều này hội tụ để chơi tối ưu khi N tăng lên, nhưng đối với hầu hết các trò chơi thì điều đó là không đủ — chúng ta cần một chính sách lựa chọn tập trung một cách có chọn lọc tính toán-tài nguyên tional trên các phần quan trọng của cây trò chơi. Nó cân bằng hai yếu tố: khám phá của các quốc gia đã có ít playouts, và khai thác các quốc gia đã làm tốt trong quá khứ

3 thuật toán "Monte Carlo" là các thuật toán ngẫu nhiên được đặt theo tên của Casino de Monte-Carlo ở Monaco.

Tìm kiếm cây Monte Carlo (MCTS)

Mô phỏng

Phát ra

Triển khai

Chính sách phát

Monte Carlosearch tinh khiết

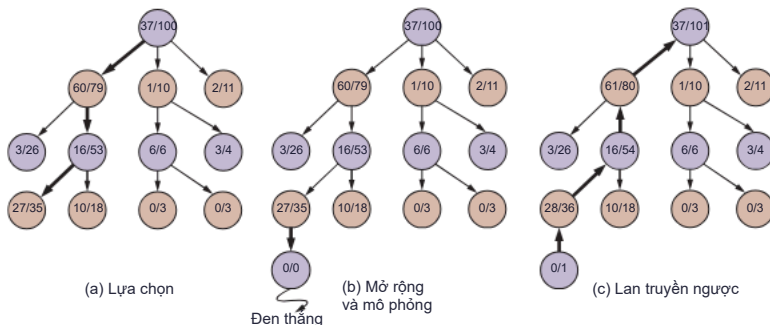
Chính sách lựa chọn

Thăm dò

Khai thác

đánh đổi thăm dò/khai thác.) Tìm kiếm cây Monte Carlo thực hiện điều đó bằng cách duy trì một cây tìm kiếm và phát triển nó trên mỗi lần lặp lại của bốn bước sau, như thể hiện trong Hình 6.10:

- **Lựa chọn:** Bắt đầu từ gốc của cây tìm kiếm, chúng tôi chọn một di chuyển (được hướng dẫn bởi chính sách lựa chọn), dẫn đến một nút kế thừa và lặp lại quá trình đó, di chuyển xuống cây đến một lá. Hình 6.10 (a) cho thấy một cây tìm kiếm với gốc đại diện cho một tiểu bang nơi trắng vừa di chuyển và trắng đã thắng 37 trong số 100 lượt chơi được thực hiện cho đến nay. Mũi tên dày cho thấy việc lựa chọn một nước đi theo màu đen dẫn đến một nút mà màu đen đã thắng 60/79 payout. Đây là tỷ lệ thắng tốt nhất trong số ba nước đi, vì vậy việc chọn nó là một ví dụ về khai thác. Nhưng cũng sẽ hợp lý nếu chọn nút 2/11 để thăm dò — chỉ với 11 playouts, nút vẫn có độ không chắc chắn cao trong định giá của nó và có thể kết thúc tốt nhất nếu chúng ta có thêm thông tin về nó. Lựa chọn tiếp tục đến nút lá được đánh dấu 27/35.
- **Mở rộng:** Chúng tôi phát triển cây tìm kiếm bằng cách tạo một con mới của nút đã chọn; Hình 6.10 (b) cho thấy nút mới được đánh dấu bằng 0/0. (Một số phiên bản tạo nhiều hơn một con trong bước này.)
- **Mô phỏng:** Chúng tôi thực hiện một bước phát từ nút con mới được tạo, lựa chọn các bước di chuyển cho cả hai người chơi theo chính sách phát ra. Những bước di chuyển này không được ghi lại trong cây tìm kiếm. Trong hình, mô phỏng dẫn đến chiến thắng cho màu đen.
- **Lan truyền ngược:** Bây giờ chúng ta sử dụng kết quả của mô phỏng để cập nhật tất cả các nút cây tìm kiếm đi lên gốc. Vì đen thắng payout, các nút đen được tăng lên trong cả số lần thắng và số lần payout, vì vậy 27/35 trở thành 28/36 và 60/79 trở thành 61/80. Vì trắng thua, các nút trắng chỉ được tăng lên trong số lần chơi, vì vậy 16/53 trở thành 16/54 và gốc 37/100 trở thành 37/101.



Hình 6.10 Một lần lặp lại của quá trình chọn di chuyển với tìm kiếm cây Monte Carlo (MCTS) bằng cách sử dụng giới hạn tin cậy trên được áp dụng cho chỉ số lựa chọn cây (UCT), được hiển thị sau khi 100 lần lặp đã được thực hiện. Trong (a) chúng tôi chọn các nước đi, xuống cây, kết thúc tại nút lá được đánh dấu 27/35 (cho 27 chiến thắng cho màu đen trong số 35 payout). Trong (b) chúng tôi mở rộng nút đã chọn và thực hiện mô phỏng (playout), kết thúc bằng chiến thắng cho màu đen. Trong (c), kết quả của mô phỏng được lan truyền ngược lên cây.

hàm MONTE-CARLO-TREE-SEARCH(state) trả về một hành động
 cây ← NODE(state) trong khi IS-TIME-REMAINING () làm

leaf ← SELECT(tree)child ← EXPAND(leaf) kết quả ←
 SIMULATE(child)BACK-PROPAGATE(result, child) trả về nước đi trong
 ACTIONS(state) có nút có số lần phát cao nhất

Hình 6.11 Thuật toán tìm kiếm cây Monte Carlo. Một cây trò chơi, cây, được khởi tạo, và sau đó chúng ta lặp lại một chu kỳ SELECT / EXPAND / SIMULATE / BACK-PROPAGATE cho đến khi hết thời gian và trả về nước đi dẫn đến nút có số lượng payout cao nhất.

Chúng ta lặp lại bốn bước này cho một số lần lặp lại nhất định hoặc cho đến khi hết thời gian được phân bổ, và sau đó trả về nước đi có số lần lặp lại cao nhất.

Một chính sách lựa chọn rất hiệu quả được gọi là "giới hạn tin cậy trên áp dụng cho cây" hoặc UCT. Chính sách xếp hạng từng động thái có thể dựa trên công thức giới hạn tin cậy trên

UCT

được gọi là UCB1. (Xem Phần 16.3.3 để biết thêm chi tiết.) Đối với nút n, công thức là:

UCB1

$$UCB1(n) = \frac{U(n) + C \times \sqrt{\log N(PARENT(n))}}{N(n)}$$

trong đó $U(n)$ là tổng tiện ích của tất cả các playout đã đi qua nút n , $N(n)$ là số lần phát qua nút n và $PARENT(n)$ là nút mẹ của n trong cây. Do đó, $U(n)N(n)$ là thuật ngữ khai thác: tiện ích trung bình của n . Thuật ngữ có căn bậc hai là thuật ngữ thăm dò: nó có số $N(n)$ trong mẫu số, có nghĩa là số hạng sẽ cao đối với các nút chỉ được khám phá một vài lần. Trong tử số, nó có nhật ký về số lần chúng ta đã khám phá cha mẹ của n . Điều này có nghĩa là nếu chúng ta chọn n một số tỷ lệ phần trăm không phải bằng không, thuật ngữ khám phá sẽ về 0 khi số lượng tăng lên và cuối cùng các lần phát được đưa cho nút có tiện ích trung bình cao nhất.

C là một hằng số cân bằng giữa khai thác và thăm dò. Có một lập luận lý thuyết rằng C nên là $\sqrt{2}$, nhưng trên thực tế, các lập trình viên trò chơi thử nhiều giá trị cho C và chọn giá trị hoạt động tốt nhất. (Một số chương trình sử dụng các công thức hơi khác nhau; ví dụ, ALPHAZERO thêm vào một thuật ngữ cho xác suất di chuyển, được tính toán bởi một mạng nơ-ron được đào tạo từ quá khứ tự chơi.) Với $C = 1.4$, nút 60/79 trong Hình 6.10 có điểm UCB1 cao nhất, nhưng với $C = 1.5$, nó sẽ là nút 2/11.

Hình 6.11 cho thấy thuật toán UCT MCTS hoàn chỉnh. Khi các lần lặp lại kết thúc, nước đi có số lần chơi cao nhất sẽ được trả về. Bạn có thể nghĩ rằng sẽ tốt hơn nếu trả về nút có tiện ích trung bình cao nhất, nhưng ý tưởng là một nút có 65/100 thắng sẽ tốt hơn một nút có 2/3 chiến thắng, bởi vì nút sau có rất nhiều sự không chắc chắn. Trong mọi trường hợp, công thức UCB1 đảm bảo rằng nút có nhiều lần chơi nhất hầu như luôn là nút có tỷ lệ thắng cao nhất, bởi vì quá trình lựa chọn ủng hộ tỷ lệ chiến thắng ngày càng nhiều khi số lần chơi tăng lên.

Thời gian để tính toán một lần chơi là tuyến tính, không theo cấp số nhân, trong độ sâu của cây trò chơi, bởi vì chỉ có một nước đi được thực hiện tại mỗi điểm lựa chọn. Điều đó cho chúng tôi nhiều thời gian cho nhiều

playouts. Ví dụ: hãy xem xét một trò chơi có hệ số phân nhánh là 32, trong đó trò chơi trung bình kéo dài 100 lớp. Nếu chúng ta có đủ sức mạnh tính toán để xem xét một tỷ trạng thái trò chơi trước khi chúng ta phải thực hiện một động thái, thì minimax có thể tìm kiếm sâu 6 lớp, alpha-beta với thứ tự di chuyển hoàn hảo có thể tìm kiếm 12 lớp và tìm kiếm Monte Carlo có thể thực hiện 10 triệu playout. Cách tiếp cận nào sẽ tốt hơn? Điều đó phụ thuộc vào độ chính xác của hàm heuristic so với các chính sách lựa chọn và phát ra.

Sự khôn ngoan thông thường là tìm kiếm Monte Carlo có lợi thế hơn alpha-beta đối với các trò chơi như Go nơi hệ số phân nhánh rất cao (và do đó alpha-beta không thể tìm kiếm đủ sâu), hoặc khi khó xác định một chức năng đánh giá tốt. Những gì alpha-beta làm là chọn đường dẫn đến một nút có điểm chức năng đánh giá cao nhất có thể đạt được, vì đối thủ sẽ cố gắng giảm thiểu điểm số. Do đó, nếu hàm đánh giá không chính xác, alpha-beta sẽ không chính xác. Một tính toán sai lầm trên một nút duy nhất có thể khiến alpha-beta chọn nhầm (hoặc tránh) đường dẫn đến nút đó. Nhưng tìm kiếm Monte Carlo dựa trên tổng hợp của nhiều playout, và do đó không dễ bị tổn thương bởi một lỗi duy nhất. Có thể kết hợp MCTS và các chức năng đánh giá bằng cách thực hiện một lần chơi cho một số lần di chuyển nhất định, nhưng sau đó cắt bớt phần chơi và áp dụng chức năng đánh giá.

Chấm dứt sớm

Cũng có thể kết hợp các khía cạnh của tìm kiếm alpha-beta và Monte Carlo. Ví dụ, trong các trò chơi có thể kéo dài nhiều nước đi, chúng ta có thể muốn sử dụng kết thúc playout sớm, trong mà chúng tôi dừng một playout đang thực hiện quá nhiều nước đi và đánh giá nó bằng hàm heuristic evaluation hoặc chỉ tuyên bố nó là hòa.

Tìm kiếm Monte Carlo có thể được áp dụng cho các trò chơi hoàn toàn mới, trong đó không có cơ quan kinh nghiệm nào để rút ra để xác định chức năng đánh giá. Miễn là chúng ta biết luật chơi, tìm kiếm Monte Carlo không cần thêm bất kỳ thông tin nào. Các chính sách lựa chọn và phát có thể sử dụng tốt kiến thức chuyên môn thủ công khi có sẵn, nhưng các chính sách tốt có thể được học bằng cách sử dụng mạng nơ-ron được đào tạo chỉ bằng cách tự chơi.

Tìm kiếm Monte Carlo có một bất lợi khi có khả năng một nước đi duy nhất có thể thay đổi diễn biến của trò chơi, bởi vì bản chất ngẫu nhiên của tìm kiếm Monte Carlo có nghĩa là nó có thể không xem xét động thái đó. Nói cách khác, việc cắt tỉa Loại B trong tìm kiếm Monte Carlo có nghĩa là một lối chơi quan trọng có thể không được khám phá. Tìm kiếm Monte Carlo cũng có một nhược điểm khi có những trạng thái trò chơi "rõ ràng" là chiến thắng cho bên này hay bên kia (theo kiến thức của con người và chức năng đánh giá), nhưng vẫn sẽ mất nhiều nước đi trong một playout để xác minh người chiến thắng. Từ lâu, người ta cho rằng tìm kiếm alpha-beta phù hợp hơn với các trò chơi như cờ vua với hệ số phân nhánh thấp và chức năng đánh giá tốt, nhưng gần đây các phương pháp tiếp cận Monte Carlo đã chứng minh thành công trong cờ vua và các trò chơi khác.

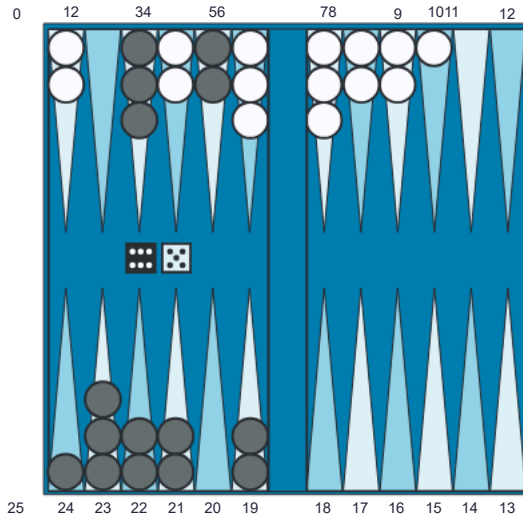
Ý tưởng chung về mô phỏng các bước di chuyển trong tương lai, quan sát kết quả và sử dụng kết quả để xác định nước đi nào là tốt là một loại học tăng cường, được đề cập trong Chương 23.

6.5 Trò chơi Stochastic

Trò chơi ngẫu nhiên

Các trò chơi ngẫu nhiên đưa chúng ta đến gần hơn một chút với sự không thể đoán trước của cuộc sống thực bằng cách bao gồm

ngẫu nhiên, chẳng hạn như ném xúc xắc. Backgammon là một trò chơi ngẫu nhiên điển hình kết hợp may mắn và kỹ năng. Ở vị trí cờ thỏ cáo của Hình 6.12, Đen đã tung 6-5 và có bốn nước đi có thể (mỗi nước đi chuyển một quân về phía trước (theo chiều kim đồng hồ) 5 vị trí và một quân về phía trước 6 vị trí).



Hình 6.12 Một vị trí backgammon điển hình. Mục tiêu của trò chơi là di chuyển tất cả các mảnh ghép của một người ra khỏi bàn cờ. Đen di chuyển theo chiều kim đồng hồ về phía 25 và Trắng di chuyển ngược chiều kim đồng hồ về phía 0. Một quân cờ có thể di chuyển đến bất kỳ vị trí nào trừ khi có nhiều quân cờ đối thủ ở đó; nếu có một đối thủ, nó sẽ bị bắt và phải bắt đầu lại. Ở vị trí được hiển thị, Đen đã tung 6–5 và phải chọn trong số bốn nước đi hợp pháp: (5–11, 5–10), (5–11, 19–24), (5–10, 10–16) và (5–11, 11–16), trong đó ký hiệu (5–11, 11–16) có nghĩa là di chuyển một quân cờ từ vị trí 5 đến 11, và sau đó di chuyển một quân cờ từ 11 đến 16.

Tại thời điểm này, Đen biết những động thái nào có thể được thực hiện, nhưng không biết Trắng sẽ tung ra những gì và do đó không biết những động thái hợp pháp của Trắng sẽ là gì. Điều đó có nghĩa là Blackcannot xây dựng một cây trò chơi tiêu chuẩn như chúng ta đã thấy trong cờ vua và tic-tac-toe. Một gametree trong backgammon phải bao gồm các nút cơ hội ngoài các nút MAX và MIN. Cơ hội

Các nút cơ hội

các nút được hiển thị dưới dạng vòng tròn trong Hình 6.13. Các nhánh dẫn từ mỗi nút cơ hội biểu thị các lần tung xúc xắc có thể có; Mỗi nhánh được dán nhãn với cuộn và xác suất của nó. Có 36 cách để tung hai viên xúc xắc, mỗi cách có khả năng như nhau; nhưng vì 6–5 giống với 5–6, chỉ có 21 cuộn riêng biệt. Sáu kép (1–1 đến 6–6) mỗi người có xác suất là $1/36$, vì vậy chúng ta nói $P(1-1) = 1/36$. 15 cuộn riêng biệt khác nhau mỗi cuộn có xác suất $1/18$.

Bước tiếp theo là hiểu cách đưa ra quyết định đúng đắn. Rõ ràng, chúng tôi vẫn muốn chọn nước đi dẫn đến vị trí tốt nhất. Tuy nhiên, các vị trí không có giá trị tối đa tối thiểu xác định.

Thay vào đó, chúng ta chỉ có thể tính toán giá trị kỳ vọng của một vị thế: trung bình trên tất cả

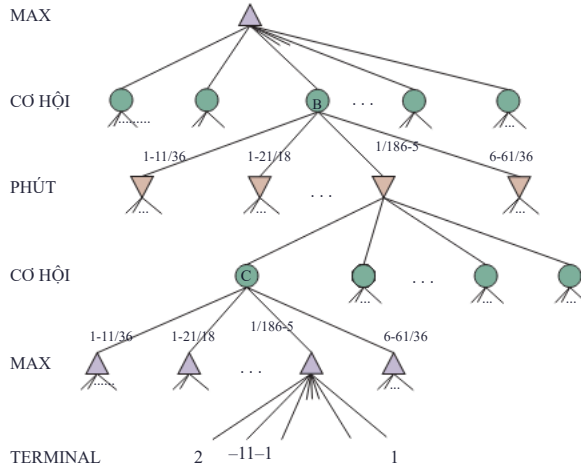
Giá trị kỳ vọng

kết quả có thể xảy ra của các nút cơ

hội. Điều này dẫn chúng ta đến giá trị expectiminimax cho các trò chơi có nút cơ hội, một khái quát hóa

Expectiminimaxvalue

của giá trị minimax cho các trò chơi xác định. Các nút đầu cuối và các nút MAX và MIN hoạt động giống hệt như trước đây (với lưu ý rằng các nước đi hợp pháp cho MAX và MIN sẽ phụ thuộc vào kết quả của việc tung xúc xắc trong nút cơ hội trước đó). Đối với các nút cơ hội, chúng tôi



Hình 6.13 Cây trò chơi sơ đồ cho vị trí cờ thỏ cáo.

Tính giá trị kỳ vọng, là tổng giá trị trên tất cả các kết quả, được trọng số bởi xác suất của mỗi hành động cơ hội:

EXPECTIMINIMAX (CÁC) =

- TIỆN ÍCH, TỐI ĐA) nếu IS-TERMINAL (các)
- $\max_a \text{EXPECTIMINIMAX}(\text{KẾT QUẢ}(S, A))$ nếu TO-MOVE(s) = MAX
- $\min_a \text{EXPECTIMINIMAX}(\text{KẾT QUẢ}, a)$ nếu TO-MOVE(S) = MIN
- $\sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r))$ nếu TO-MOVE(s) = CHANCE

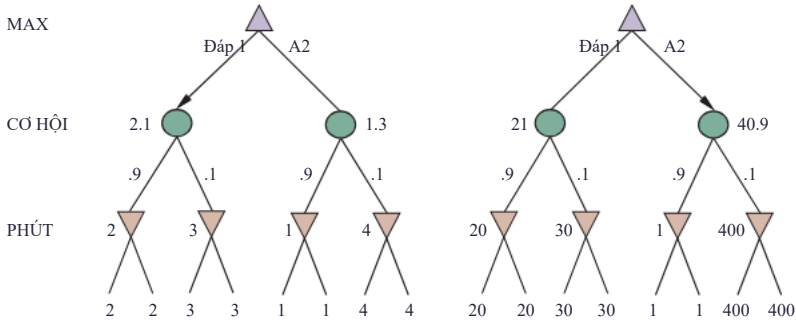
trong đó are đại diện cho một lần tung xúc xắc có thể xảy ra (hoặc sự kiện ngẫu nhiên khác) và $\text{RESULT}(s, r)$ là cùng trạng thái với s, với thực tế bổ sung là kết quả của lần tung xúc xắc là r.

6.5.1 Chức năng đánh giá cho các trò chơi may rủi

Như với minimax, xấp xỉ rõ ràng để thực hiện với expectiminimax là cắt bỏ tìm kiếm tại một số điểm và áp dụng một hàm đánh giá cho mỗi lá. Người ta có thể nghĩ rằng các hàm đánh giá cho các trò chơi như cờ thỏ cáo nên giống như các hàm đánh giá cho cờ vua — chúng chỉ cần đưa ra các giá trị cao hơn cho các vị trí tốt hơn. Nhưng trên thực tế, sự hiện diện của các nút cơ hội có nghĩa là người ta phải cẩn thận hơn về ý nghĩa của các giá trị.

Hình 6.14 cho thấy điều gì xảy ra: với một hàm đánh giá gán các giá trị [1, 2, 3, 4] cho các lá, di chuyển a1 là tốt nhất; Với các giá trị [1, 20, 30, 400], nước đi A2 là tốt nhất. Do đó, chương trình hoạt động hoàn toàn khác nếu chúng ta thực hiện thay đổi một số giá trị đánh giá, ngay cả khi thứ tự ưu tiên vẫn giữ nguyên.

Hóa ra để tránh vấn đề này, hàm đánh giá phải trả về các giá trị có diện tích biến đổi tuyến tính dương của xác suất chiến thắng (hoặc của tiện ích mong đợi, đối với các trò chơi có kết quả khác ngoài thắng/thua). Mỗi quan hệ này với xác suất là một điều quan trọng



Hình 6.14 Biến đổi giữ nguyên thứ tự trên giá trị lá thay đổi động thái tốt nhất.

và thuộc tính chung của các tình huống có liên quan đến sự không chắc chắn, và chúng ta thảo luận thêm về nó trong Chương 15.

Nếu chương trình biết trước tất cả các lần tung xúc xắc sẽ xảy ra trong phần còn lại của trò chơi, giải một trò chơi bằng xúc xắc sẽ giống như giải một trò chơi mà không có xúc xắc, điều mà minimax thực hiện trong thời gian $O(bm)$, trong đó b là hệ số phân nhánh và m là độ sâu tối đa của cây trò chơi. Bởi vì expectiminimax cũng đang xem xét tất cả các chuỗi tung xúc xắc có thể có, nó sẽ lấy $O(bnm)$, trong đó n là số lần tung riêng biệt.

Ngay cả khi việc tìm kiếm bị giới hạn ở một số độ sâu nhỏ d , chi phí bổ sung so với chi phí của minimax khiến việc xem xét nhìn xa trong hầu hết các trò chơi mà máy phải trở nên không thực tế. Inbackgammon n là 21 và b thường là khoảng 20, nhưng trong một số tình huống có thể cao tới 4000 đối với việc tung xúc xắc là đối. Chúng tôi có lẽ chỉ có thể quản lý ba lớp tìm kiếm.

Một cách khác để nghĩ về vấn đề là: lợi thế của alpha-beta là nó bỏ qua những phát triển trong tương lai sẽ không xảy ra, nếu chơi tốt nhất. Do đó, nó tập trung vào các sự kiện có thể xảy ra. Nhưng trong một trò chơi mà một lần ném hạt xúc xắc trước mỗi nước đi, không có chuỗi nước đi nào có thể xảy ra; Ngay cả nước đi có khả năng xảy ra nhất chỉ xảy ra 2/36 thời gian, bởi vì để nước đi diễn ra, xúc xắc trước tiên phải đi đúng cách để làm cho nó hợp pháp. Đây là một vấn đề chung bất cứ khi nào sự không chắc chắn bước vào bức tranh: các khả năng được nhân lên rất nhiều, và hình thành các kế hoạch hành động chi tiết trở nên vô nghĩa vì thế giới có thể sẽ không chơi cùng.

Bạn có thể nghĩ rằng một cái gì đó như cắt tỉa alpha-beta có thể được áp dụng cho cây trò chơi với các nút cơ hội. Hóa ra nó có thể. Phân tích cho các nút MIN và MAX không thay đổi, nhưng chúng ta cũng có thể cắt tỉa các nút cơ hội, sử dụng một chút khéo léo. Hãy xem xét nút cơ hội C trong Hình 6.13 và điều gì xảy ra với giá trị của nó khi chúng ta đánh giá các con của nó. Có thể tìm ra giới hạn trên giá trị của C trước khi chúng ta xem xét tất cả các con của nó không? (Hãy nhớ lại rằng đây là những gì alpha-beta cần để cắt tỉa một nút và cây con của nó.)

Thoạt nhìn, nó có vẻ không thể vì giá trị của C là giá trị trung bình của giá trị chil-dren của nó, và để tính trung bình của một tập hợp các số, chúng ta phải xem xét tất cả các số. Nhưng nếu chúng ta đặt giới hạn cho các giá trị có thể có của hàm tiện ích, thì chúng ta có thể

đến giới hạn cho mức trung bình mà không cần nhìn vào mọi con số. Ví dụ, giả sử rằng tất cả các giá trị tiện ích nằm trong khoảng từ -2 đến $+2$; Sau đó, giá trị của các nút lá được giới hạn, và lần lượt chúng ta có thể đặt giới hạn trên cho giá trị của một nút cơ hội mà không cần nhìn vào tất cả các con của nó.

Trong các trò chơi mà hệ số phân nhánh cho các nút cơ hội cao — hãy xem xét một trò chơi như Yahtzee nơi bạn tung 5 viên xúc xắc mỗi lượt — bạn có thể muốn xem xét việc cắt tia về phía trước để lấy mẫu một số lượng nhỏ hơn các nhánh cơ hội có thể có. Hoặc bạn có thể muốn tránh sử dụng hoàn toàn chức năng đánh giá và thay vào đó chọn tìm kiếm cây Monte Carlo, trong đó mỗi lần chơi bao gồm các lần tung xúc xắc ngẫu nhiên.

6.6 Trò chơi có thể quan sát được một phần

Bobby Fischer tuyên bố rằng "cờ vua là chiến tranh", nhưng cờ vua thiếu ít nhất một đặc điểm chính của các cuộc chiến tranh thực sự, đó là khả năng quan sát một phần. Trong "sương mù chiến tranh", nơi ở của các đơn vị địch thường không được biết cho đến khi được tiết lộ bằng cách tiếp xúc trực tiếp. Kết quả là, chiến tranh bao gồm việc sử dụng trinh sát và gián điệp để thu thập thông tin và sử dụng che giấu và vô tội vạ để gây nhầm lẫn cho kẻ thù.

Các trò chơi có thể quan sát được một phần có chung những đặc điểm này và do đó khác về chất so với các trò chơi trong các phần trước. Các trò chơi điện tử như StarCraft đặc biệt khó khăn, có thể quan sát được một phần, đa tác nhân, không xác định, năng động và không biết.

Trong các trò chơi có thể quan sát được một phần xác định, sự không chắc chắn về trạng thái của bàn cờ phát sinh hoàn toàn do thiếu quyền truy cập vào các lựa chọn của đối thủ. Lớp này bao gồm các trò chơi của chil-dren như Battleship (trong đó tàu của mỗi người chơi được đặt ở các vị trí ẩn khỏi đối thủ) và Stratego (nơi các vị trí quân cờ được biết nhưng các loại quân cờ được giấu kín). Chúng ta sẽ xem xét trò chơi Kriegspiel, một biến thể có thể quan sát được một phần của cờ vua trong

những quân cờ nào hoàn toàn vô hình đối với đối thủ. Các trò chơi khác cũng có các phiên bản có thể quan sát được một phần: Phantom Go, Phantom tic-tac-toe và Screen Shogi.

6.6.1 Kriegspiel: Cờ vua có thể quan sát được một phần

Các quy tắc của Kriegspiel như sau: Trắng và Đen mỗi người nhìn thấy một bảng chỉ chứa các quân cờ của riêng họ. Một trọng tài, người có thể nhìn thấy tất cả các quân cờ, phân quyết trận đấu và định kỳ đưa ra các thông báo mà cả hai cầu thủ đều nghe thấy. Đầu tiên, White đề xuất với trọng tài một động thái sẽ hợp pháp nếu không có quân đen. Nếu các quân đen ngăn chặn động thái, trọng tài sẽ thông báo "bất hợp pháp", và Trắng tiếp tục đề xuất các nước đi cho đến khi tìm thấy một quân cờ hợp pháp — tìm hiểu thêm về vị trí của các quân cờ của Đen trong quá trình này.

Khi một nước đi hợp pháp được đề xuất, trọng tài sẽ thông báo một hoặc nhiều điều sau: "Bắt trên ô X" nếu có một vụ bắt và "Kiểm tra bởi D" nếu vua đen đang kiểm tra, trong đó D là hướng của kiểm tra và có thể là một trong "Hiệp sĩ", "Xe hạng", "Tạ tin", "Đường chéo dài" hoặc "Đường chéo ngắn". Nếu Đen bị kiểm tra hoặc bắt, trọng tài sẽ nói như vậy; nếu không, đến lượt Đen di chuyển.

Kriegspiel có vẻ không thể một cách đáng sợ, nhưng con người quản lý nó khá tốt và các chương trình thu âm đang bắt đầu bắt kịp. Nó giúp nhớ lại khái niệm về một trạng thái niềm tin được định nghĩa trong Phần 4.4 và được minh họa trong Hình 4.14 - tập hợp tất cả các trạng thái hội đồng có thể có một cách hợp lý với lịch sử đầy đủ của các nhận thức cho đến nay. Ban đầu, trạng thái niềm tin của White là một tập lỏi vì các quân cờ của Black vẫn chưa di chuyển. Sau khi Trắng thực hiện một động thái và Đen phản hồi, trạng thái niềm tin của Trắng chứa 20 vị trí, bởi vì Đen có 20 câu trả lời cho bất kỳ

nước đi mở đầu. Theo dõi trạng thái niềm tin khi trò chơi diễn ra chính xác là vấn đề ước tính trạng thái, mà bước cập nhật được đưa ra trong Phương trình (4.6) trên trang 150. Chúng ta có thể ánh xạ ước tính trạng thái Kriegspiel trực tiếp vào khuôn khổ không xác định, có thể quan sát được một phần của Phần 4.4 nếu chúng ta coi đối thủ là nguồn gốc của thuyết không xác định; nghĩa là, KẾT QUẢ của nước đi của Trắng được tạo thành từ kết quả (có thể dự đoán) của nước đi của Trắng và kết quả không thể đoán trước được đưa ra bởi câu trả lời của Đen.4

Với trạng thái niềm tin hiện tại, White có thể hỏi, "Tôi có thể thắng trò chơi không?" Đối với một trò chơi có thể quan sát một phần, khái niệm về chiến lược bị thay đổi; Thay vì chỉ định một nước đi để thực hiện cho mỗi nước đi có thể thực hiện mà đối thủ có thể thực hiện, chúng ta cần một nước đi cho mọi trình tự nhận thức có thể nhận được.

Đối với Kriegspiel, một chiến lược chiến thắng, hoặc chiếu hết được đảm bảo, là một chiến lược, đối với mỗi khả năng. Đảm bảo chiếu hết

trình tự nhận thức ble, dẫn đến một chiếu hết thực tế cho mọi trạng thái bàn cờ có thể có trong trạng thái niềm tin hiện tại, bất kể đối thủ di chuyển như thế nào. Với định nghĩa này, trạng thái niềm tin của đối thủ là không liên quan - chiến lược phải hoạt động ngay cả khi đối thủ có thể nhìn thấy tất cả các quân cờ. Điều này đơn giản hóa rất nhiều việc tính toán. Hình 6.15 cho thấy một phần của một chiếu hết được đảm bảo cho trò chơi kết thúc KRK (vua và xe so với vua). Trong trường hợp này, Đen chỉ có một quân (vua), vì vậy trạng thái niềm tin cho Trắng có thể được hiển thị trong một bảng duy nhất bằng cách đánh dấu từng vị trí có thể có của vua Đen.

Thuật toán tìm kiếm AND-OR chung có thể được áp dụng cho không gian trạng thái niềm tin để tìm các chiếu hết được đảm bảo, giống như trong Phần 4.4. Thuật toán trạng thái niềm tin gia tăng được đề cập trong Phần 4.4.2 thường tìm thấy những kẻ chiếu hết giữa trò chơi lên đến độ sâu 9 — vượt xa khả năng của hầu hết người

chơi. Việc chiếu hết được đảm bảo, Kriegspiel thừa nhận một khái niệm hoàn toàn mới không có ý nghĩa trong các trò chơi hoàn toàn có thể quan sát được: chiếu hết xác suất. Những kẻ chiếu hết như vậy là

Xác suất checkmate

vẫn được yêu cầu làm việc ở mọi tiểu bang hội đồng quản trị trong trạng thái tín ngưỡng; Chúng ta xác suất đối với sự ngẫu nhiên hóa các nước đi của người chơi chiến thắng. Để có được ý tưởng cơ bản, hãy xem xét vấn đề tìm một vị vua đen đơn độc chỉ bằng cách sử dụng vị vua trắng. Đơn giản bằng cách di chuyển ngẫu nhiên, vua trắng cuối cùng sẽ đụng phải vua đen ngay cả khi vua sau cố gắng tránh số phận này, vì Đen không thể tiếp tục đoán những động tác né tránh đúng với thời hạn. Trong thuật ngữ lý thuyết xác suất, phát hiện xảy ra với xác suất 1.

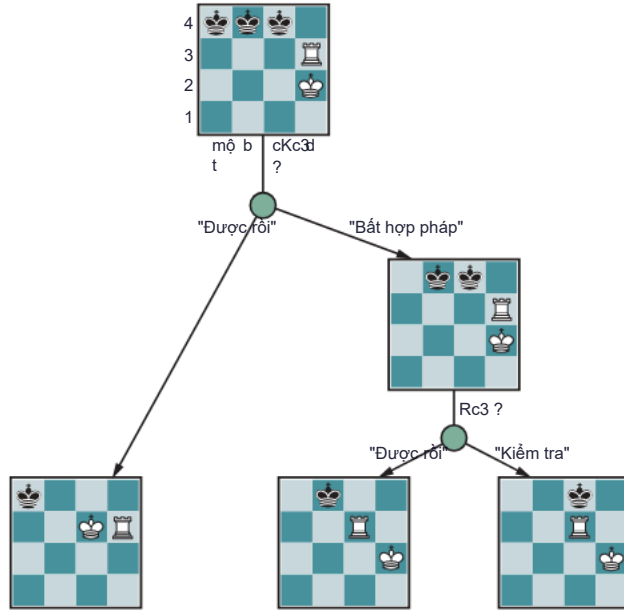
Trò chơi kết thúc của KBNK — vua, giám mục và hiệp sĩ đấu với vua — được thắng theo nghĩa này; White trình bày cho Black một chuỗi lựa chọn ngẫu nhiên vô hạn, một trong số đó Black sẽ đoán sai và tiết lộ vị trí của mình, dẫn đến chiếu tướng. Mặt khác, KBBKendgame giành chiến thắng với xác suất $1 - \epsilon$. Trắng chỉ có thể buộc phải giành chiến thắng bằng cách để một trong những giám mục của mình không được bảo vệ trong một nước đi. Nếu Đen tình cờ ở đúng nơi và bắt được giám mục (một động thái sẽ là bất hợp pháp nếu các giám mục được bảo vệ), trò chơi sẽ hòa. White có thể chọn thực hiện nước đi mạo hiểm tại một số điểm được chọn ngẫu nhiên ở giữa một chuỗi rất dài, do đó giảm xuống một hằng số nhỏ tùy ý, nhưng không thể giảm xuống không.

Đôi khi, chiến lược chiếu hết hiệu quả đối với một số tiểu bang hội đồng quản trị trong trạng thái niềm tin hiện tại nhưng không phải những bang khác. Thử một chiến lược như vậy có thể thành công, dẫn đến một cuộc chiếu hết vô tình - tình cờ theo nghĩa là White không thể biết rằng đó sẽ là chiếu hết - nếu

Vô tình kiểm tra

Các mảnh ghép của Black tình cờ ở đúng chỗ. (Hầu hết các chiếu hết trong trò chơi giữa con người

4 Đôi khi, trạng thái niềm tin sẽ trở nên quá lớn để đại diện cho một danh sách các tiểu bang của hội đồng quản trị, nhưng chúng ta sẽ bỏ qua vấn đề này ngay bây giờ; Chương 7 và 8 đề xuất các phương pháp để thể hiện một cách chặt chẽ các trạng thái niềm tin rất lớn.



Hình 6.15 Một phần của một chiếu hết được đảm bảo trong trò chơi kết thúc KRK, được hiển thị trên một bảng thu nhỏ. Trong trạng thái niềm tin ban đầu, vua của Đen đang ở một trong ba địa điểm có thể xảy ra. Bằng cách kết hợp các động thái thăm dò, chiến lược thu hẹp điều này xuống còn một. Hoàn thành của checkmate được để lại như một bài tập.

có tính chất ngẫu nhiên này.) Ý tưởng này tự nhiên dẫn đến câu hỏi về khả năng chiến lược của một chiến lược sẽ giành chiến thắng, điều này dẫn đến câu hỏi về khả năng mỗi hội đồng quản trị trong trạng thái niềm tin hiện tại là trạng thái hội đồng quản trị thực sự là như thế nào.

Xu hướng đầu tiên của người ta có thể là đề xuất rằng tất cả các tiểu bang hội đồng quản trị trong trạng thái niềm tin hiện tại đều có khả năng như nhau — nhưng điều này không thể đúng. Ví dụ, hãy xem xét trạng thái niềm tin của White sau nước đi đầu tiên của Black trong trò chơi. Theo định nghĩa (giả sử rằng Đen chơi tối ưu), Đen phải chơi một nước đi tối ưu, vì vậy tất cả các trạng thái bản cờ do nước đi dưới mức tối ưu phải được chỉ định xác suất bằng không.

Lập luận này cũng không hoàn toàn đúng, bởi vì mục tiêu của mỗi người chơi không chỉ là di chuyển các quân cờ đến đúng ô vuông mà còn để giảm thiểu thông tin mà đối thủ có về vị trí của họ. Chơi bất kỳ chiến lược "tối ưu" nào có thể dự đoán được sẽ cung cấp cho đối thủ đội hình. Do đó, cách chơi tối ưu trong các trò chơi có thể quan sát được một phần đòi hỏi sự sẵn sàng chơi một cách ngẫu nhiên. (Đây là lý do tại sao thanh tra vệ sinh nhà hàng thực hiện các chuyển kiểm tra ngẫu nhiên.) Điều này có nghĩa là thỉnh thoảng chọn các nước đi có vẻ "về bản chất" yếu — nhưng chúng đạt được sức mạnh từ sự không thể đoán trước của chúng, bởi vì đối thủ không có khả năng chuẩn bị bất kỳ khả năng phòng thủ nào chống lại chúng.

Từ những cân nhắc này, có vẻ như xác suất liên quan đến các trạng thái hội đồng quản trị trong trạng thái niềm tin hiện tại chỉ có thể được tính toán với một chiến lược ngẫu nhiên tối ưu; Đổi lại, tính toán chiến lược đó dường như yêu cầu biết xác suất của các tiểu bang khác nhau mà hội đồng quản trị có thể ở. Câu hỏi hóc búa này có thể được giải quyết bằng cách áp dụng khái niệm lý thuyết trò chơi về một giải pháp cân bằng, mà chúng ta theo đuổi thêm trong Chương 16. Một cân bằng chỉ định một chiến lược ngẫu nhiên tối ưu cho mỗi người chơi. Cân bằng máy tính là quá đắt đối với Kriegspiel. Hiện nay, việc thiết kế các thuật toán hiệu quả cho trò chơi Kriegspiel nói chung là một chủ đề nghiên cứu mở. Hầu hết các hệ thống thực hiện nhìn về phía trước có giới hạn trong không gian trạng thái niềm tin của riêng họ, bỏ qua trạng thái niềm tin của đối thủ. Các chức năng đánh giá tương tự như các chức năng cho trò chơi có thể thuyết phục được nhưng bao gồm một thành phần cho kích thước của trạng thái niềm tin — nhỏ hơn là tốt hơn! Chúng ta sẽ quay trở lại các trò chơi có thể quan sát được một phần theo chủ đề Lý thuyết trò chơi trong Phần 17.2.

6.6.2 Trò chơi bài

Các trò chơi bài như cầu, huýt sáo, trái tim và poker có khả năng quan sát một phần ngẫu nhiên, trong đó thông tin còn thiếu được tạo ra bởi việc chia bài ngẫu nhiên.

Thoạt nhìn, có vẻ như những trò chơi bài này giống như trò chơi xúc xắc: các thẻ được chia ngẫu nhiên và xác định các nước đi có sẵn cho mỗi người chơi, nhưng tất cả các "xúc xắc" đều được tung ngay từ đầu! Mặc dù phép so sánh này hóa ra là không chính xác, nhưng nó gợi ý một thuật toán: coi việc bắt đầu trò chơi như một nút cơ hội với mọi giao dịch có thể là kết quả, và sau đó sử dụng công thức EXPECTIMINIMAX để chọn nước đi tốt nhất. Lưu ý rằng trong cách tiếp cận này, nút duy nhất là nút gốc; Sau đó, trò chơi trở nên hoàn toàn có thể quan sát được. Cách tiếp cận này đôi khi được gọi là tính trung bình về khả năng thấu thị vì nó giả định rằng một khi thỏa thuận thực tế đã xảy ra, trò chơi trở nên hoàn toàn có thể quan sát được đối với cả hai người chơi. Mặc dù có sức hấp dẫn trực quan, chiến lược này có thể khiến người ta lạc lối. Hãy xem xét câu chuyện sau đây:

Ngày 1: Đường A dẫn đến một hũ vàng; Đường B dẫn đến một ngã ba. Bạn có thể thấy rằng ngã ba bên trái dẫn đến hai hũ vàng, và ngã ba bên phải dẫn đến việc bạn bị xe buýt cán qua.

Ngày 2: Đường A dẫn đến một hũ vàng; Đường B dẫn đến một ngã ba. Bạn có thể thấy rằng ngã ba bên phải dẫn đến hai hũ vàng, và ngã ba bên trái dẫn đến việc bạn bị xe buýt cán qua.

Ngày 3: Đường A dẫn đến một hũ vàng; Đường B dẫn đến một ngã ba. Bạn được nói rằng một nửa dẫn đến hai hũ vàng, và một nửa dẫn đến bạn bị xe buýt cán qua. Thật không may, bạn không biết ngã ba nào là ngã ba nào.

Tính trung bình về khả năng thấu thị dẫn đến lý do sau: vào Ngày 1, B là lựa chọn đúng đắn; vào Ngày 2, B là lựa chọn đúng đắn; vào Ngày thứ 3, tình hình giống như Ngày 1 hoặc Ngày 2, vì vậy B vẫn phải là lựa chọn đúng đắn.

Bây giờ chúng ta có thể thấy cách trung bình về khả năng thấu thị thất bại: nó không xem xét trạng thái niềm tin rằng tác nhân sẽ ở trong sau khi hành động. Một trạng thái niềm tin của sự ngu dốt hoàn toàn là không mong muốn, đặc biệt là khi một khả năng là cái chết chắc chắn. Bởi vì nó giả định rằng mọi trạng thái trong tương lai sẽ tự động là một trong những kiến thức hoàn hảo, cách tiếp cận thấu thị không bao giờ chọn các hành động thu thập thông tin (như động thái đầu tiên trong Hình 6.15); Nó cũng sẽ không chọn các hành động che giấu thông tin từ đối thủ hoặc cung cấp thông tin cho đối tác, bởi vì nó giả định rằng họ đã biết thông tin; và nó sẽ không bao giờ vô tội vạ trong poker, 5 bởi vì nó giả định

Loài

đối thủ có thể nhìn thấy quân bài của mình. Trong Chương 16, chúng ta chỉ ra cách xây dựng các thuật toán thực hiện

5 Lừa đảo — cá cược như thể ván bài của một người tốt, ngay cả khi nó không phải — là một phần cốt lõi của chiến lược poker.

tất cả những điều này nhờ giải quyết vấn đề quyết định có thể quan sát được một phần thực sự, dẫn đến một chiến lược cân bằng tối ưu (xem Phần 17.2).

Mặc dù có những nhược điểm, tính trung bình về khả năng thấu thị có thể là một chiến lược hiệu quả, với một số thủ thuật để làm cho nó hoạt động tốt hơn. Trong hầu hết các trò chơi bài, số lượng giao dịch có thể có là khá lớn. Ví dụ, trong chơi cầu, mỗi người chơi chỉ nhìn thấy hai trong số bốn ván bài; Có hai ván bài vô hình, mỗi ván bài 13 lá bài, vì vậy số lượng giao dịch là $(2613) = 10.400.600$. Giải quyết ngay cả một thỏa thuận là khá khó, vì vậy việc giải quyết mười triệu là điều không thể bàn cãi. Một cách để đối phó với con số khổng lồ này là trừu tượng: tức là bằng cách coi các bàn tay tương tự như giống hệt nhau. Ví dụ, điều rất quan trọng là quân át và vua nào trong một ván bài, nhưng việc ván bài có a4 hay 5 không quan trọng bằng, và có thể được trừu tượng hóa.

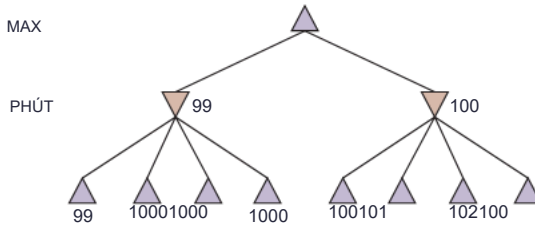
Một cách khác để đối phó với số lượng lớn là cắt tỉa chuyển tiếp: chỉ xem xét một mẫu ngẫu nhiên nhỏ của N giao dịch và một lần nữa tính điểm EXPECTIMINIMAX. Ngay cả đối với N khá nhỏ - ví dụ, 100 đến 1.000 - phương pháp này cho một xấp xỉ tốt. Nó cũng có thể được áp dụng cho các trò chơi xác định như Kriegspiel, nơi chúng ta lấy mẫu các trạng thái có thể xảy ra của trò chơi hơn là các giao dịch có thể xảy ra, miễn là chúng ta có một số cách để ước tính khả năng của mỗi trạng thái. Cũng có thể hữu ích khi thực hiện tìm kiếm phỏng đoán với ngưỡng độ sâu hơn là tìm kiếm toàn bộ cây trò chơi.

Cho đến nay chúng tôi đã giả định rằng mỗi thỏa thuận có khả năng như nhau. Điều đó có ý nghĩa đối với các trò chơi như huýt sáo và trái tim. Nhưng đối với cầu nổi, lối chơi được đặt trước bởi một giai đoạn đấu thầu trong đó mỗi đội cho biết họ mong đợi có bao nhiêu thủ thuật để giành chiến thắng. Vì người chơi đặt giá thầu dựa trên các lá bài mà họ nắm giữ, những người chơi khác sẽ học được điều gì đó về xác suất P của mỗi giao dịch. Tính đến điều này khi quyết định cách chơi ván bài là rất khó khăn, vì những lý do được đề cập trong mô tả của chúng tôi về Kriegspiel: người chơi có thể đặt giá thầu theo cách để giảm thiểu thông tin được truyền tải đến đối thủ của họ.

Máy tính đã đạt đến mức hiệu suất siêu phàm trong poker. Pro-gram poker Libratus đã đối đầu với bốn trong số những người chơi poker hàng đầu thế giới trong một trận đấu kéo dài 20 ngày của Texas hold 'em không giới hạn và dứt khoát đánh bại tất cả. Vì có rất nhiều trạng thái có thể có trong poker, Libratus sử dụng trừu tượng hóa để giảm không gian trạng thái: nó có thể coi hai ván bài AAA72 và AAA64 là tương đương nhau (cả hai đều là "ba quân át và một số lá bài thấp"), và nó có thể coi đại cuộc 200 đô la giống với 201 đô la. Nhưng Libratus cũng giám sát những người chơi khác và nếu phát hiện ra họ đang khai thác một sự trừu tượng, nó sẽ thực hiện một số tính toán bổ sung qua đêm để bịt lỗ hổng đó. Nhìn chung, nó đã sử dụng 25 triệu giờ CPU trên một siêu máy tính để giành chiến thắng.

Chi phí tính toán mà Libratus phải chịu (và các chi phí tương tự của ALPHAZERO và các hệ thống khác) cho thấy rằng việc chơi trò chơi vô địch thế giới có thể không đạt được đối với các nhà nghiên cứu có ngân sách hạn chế. Ở một mức độ nào đó, điều đó đúng: cũng giống như bạn không nên mong đợi có thể lắp ráp một chiếc xe đua Công thức 1 vô địch từ phụ tùng thay thế trong nhà để xe của mình, có một lợi thế khi có quyền truy cập vào siêu máy tính hoặc phần cứng đặc biệt như Tensor Pro-cessing Units. Điều đó đặc biệt đúng khi đào tạo một hệ thống, nhưng việc đào tạo cũng có thể thông qua nguồn lực cộng đồng. Ví dụ, hệ thống LEEAZERO mã nguồn mở là một bản tóm tắt của ALPHAZERO đào tạo thông qua tự chơi trên máy tính của những người tham gia tình nguyện. Sau khi được đào tạo, các yêu cầu tính toán cho việc chơi giải đấu thực tế là khiêm tốn.

ALPHASTAR đã giành chiến thắng trong các trò chơi StarCraft II chạy trên máy tính để bàn hàng hóa với một GPU duy nhất và ALPHAZERO có thể đã được chạy ở chế độ đó.



Hình 6.16 Một cây trò chơi hai lớp mà minimax heuristic có thể mắc lỗi.

6.7 Hạn chế của thuật toán tìm kiếm trò chơi

Bởi vì việc tính toán các quyết định tối ưu trong các trò chơi phức tạp là khó giải quyết, tất cả các thuật toán phải đưa ra một số giả định và xấp xỉ. Tìm kiếm alpha-beta sử dụng hàm đánh giá heuristic làm xấp xỉ và tìm kiếm Monte Carlo tính toán trung bình gần đúng trên một lựa chọn ngẫu nhiên của các replay. Việc lựa chọn thuật toán nào để sử dụng phụ thuộc vào các tính năng của từng trò chơi: khi hệ số phân nhánh cao hoặc khó xác định chức năng đánh giá, tìm kiếm Monte Carlo được ưu tiên hơn. Nhưng cả hai thuật toán đều có những hạn chế cơ bản.

Một hạn chế của tìm kiếm alpha-beta là tính dễ bị tổn thương đối với các lỗi trong chức năng heuristic. Hình 6.16 cho thấy một cây trò chơi hai lớp mà minimax đề xuất lấy nhánh bên phải vì $100 > 99$. Đó là động thái đúng đắn nếu tất cả các đánh giá đều chính xác. Nhưng giả sử rằng việc đánh giá mỗi nút có một sai số độc lập với các nút khác và được phân bố ngẫu nhiên với độ lệch chuẩn là σ . Sau đó, nhánh bên trái thực sự tốt hơn 71% thời gian khi $\sigma = 5$ và 58% thời gian khi $\sigma = 2$ (bởi vì một trong bốn lá bên phải có khả năng trượt xuống dưới 99 trong những trường hợp này). Nếu lỗi trong chức năng đánh giá không độc lập, thì khả năng xảy ra lỗi sẽ tăng lên. Rất khó để bù đắp cho điều này vì chúng ta không có một mô hình tốt về sự phụ thuộc giữa các giá trị của các nút anh chị em.

Hạn chế thứ hai của cả alpha-beta và Monte Carlo là chúng được thiết kế để tính toán (giới hạn trên) các giá trị của các động thái hợp pháp. Nhưng đôi khi có một nước đi rõ ràng là tốt nhất (ví dụ khi chỉ có một nước đi hợp pháp), và trong trường hợp đó, không có ích gì để lãng phí thời gian tính toán để tìm ra giá trị của nước đi — tốt hơn là chỉ thực hiện động thái. Một thuật toán tìm kiếm tốt hơn sẽ sử dụng ý tưởng về tiện ích của một mở rộng nút, chọn các mở rộng nút có tiện ích cao - nghĩa là, những cái có khả năng dẫn đến việc khám phá ra một động thái tốt hơn đáng kể. Nếu không có phần mở rộng nút nào có tiện ích cao hơn chi phí của chúng (về mặt thời gian), thì thuật toán nên ngừng tìm kiếm và thực hiện một động thái. Điều này không chỉ hiệu quả đối với các tình huống yêu thích rõ ràng mà còn đối với trường hợp nước đi đối xứng, mà không có tìm kiếm nào cho thấy rằng nước này tốt hơn nước đi khác.

Loại lý luận này về những gì tính toán cần làm được gọi là siêu lý luận (reasoning về lý luận). Nó không chỉ áp dụng cho việc chơi trò chơi mà còn cho bất kỳ loại lý luận nào. Tất cả các tính toán được thực hiện để cố gắng đạt được các quyết định tốt hơn, tất cả đều có chi phí và tất cả đều có khả năng dẫn đến sự cải thiện nhất định về chất lượng quyết định. Monte

Carlo search cố gắng thực hiện siêu lý luận để phân bổ tài nguyên cho các phần quan trọng nhất của cây, nhưng không làm như vậy một cách tối ưu.

Hạn chế thứ ba là cả alpha-beta và Monte Carlo đều thực hiện tất cả các lý luận của họ ở cấp độ di chuyển riêng lẻ. Rõ ràng, con người chơi trò chơi khác nhau: họ có thể suy luận ở mức độ trừu tượng hơn, xem xét mục tiêu cấp cao hơn - ví dụ, bẫy nữ hoàng của đối thủ - và sử dụng mục tiêu để tạo ra các kế hoạch hợp lý một cách có chọn lọc. Trong Chương 11, chúng ta sẽ nghiên cứu loại quy hoạch này, và trong Phần 11.4, chúng ta sẽ chỉ ra cách lập kế hoạch với một hệ thống phân cấp của các biểu diễn trừu tượng đến cụ thể.

Vấn đề thứ tư là khả năng kết hợp máy học vào quá trình tìm kiếm trò chơi. Các chương trình trò chơi ban đầu dựa vào chuyên môn của con người để chế tạo thủ công các chức năng đánh giá, mở sách, chiến lược tìm kiếm và thủ thuật hiệu quả. Chúng ta chỉ mới bắt đầu thấy các chương trình như ALPHAZERO (Silver et al., 2018), dựa vào máy học từ tự chơi hơn là chuyên môn do con người tạo ra dành riêng cho trò chơi. Chúng tôi đề cập sâu về học máy bắt đầu từ Chương 19.

Tóm tắt

Chúng tôi đã xem xét nhiều trò chơi khác nhau để hiểu ý nghĩa của lối chơi tối ưu, để hiểu cách chơi tốt trong thực tế và để cảm nhận cách một đặc vụ nên hành động trong bất kỳ loại môi trường đối nghịch nào. Các ý tưởng quan trọng nhất như sau:

- Một trò chơi có thể được xác định bởi trạng thái ban đầu (cách bàn cờ được thiết lập), các hành động pháp lý trong mỗi trạng thái, kết quả của mỗi hành động, kiểm tra đầu cuối (cho biết khi trò chơi kết thúc) và một chức năng tiện ích áp dụng cho các trạng thái đầu cuối để cho biết ai đã thắng và điểm số cuối cùng là bao nhiêu.
- Trong các trò chơi hai người chơi, rời rạc, xác định, có tổng bằng không theo luật với thông tin hoàn hảo, thuật toán minimax có thể chọn các bước di chuyển tối ưu bằng cách liệt kê độ sâu đầu tiên của cây trò chơi.
- Thuật toán tìm kiếm alpha-beta tính toán động thái tối ưu tương tự như minimax, nhưng đạt được hiệu quả cao hơn nhiều bằng cách loại bỏ các cây con được chứng minh là không liên quan.
- Thông thường, không khả thi để xem xét toàn bộ cây trò chơi (ngay cả với alpha-beta), vì vậy chúng ta cần cắt bỏ tìm kiếm tại một thời điểm nào đó và áp dụng một hàm đánh giá heuristic ước tính tiện ích của một trạng thái.
- Một giải pháp thay thế được gọi là tìm kiếm cây Monte Carlo (MCTS) đánh giá các trạng thái không phải bằng cách áp dụng một hàm phỏng đoán, mà bằng cách chơi trò chơi đến cuối cùng và sử dụng các quy tắc của trò chơi để xem ai thắng. Vì các nước đi được chọn trong quá trình chơi có thể không phải là nước đi tối ưu, quá trình này được lặp lại nhiều lần và đánh giá là mức trung bình của kết quả.
- Nhiều chương trình trò chơi tính toán trước các bảng nước đi tốt nhất trong trò chơi mở đầu và kết thúc để họ có thể tra cứu nước đi thay vì tìm kiếm.
- Trò chơi may rủi có thể được xử lý bởi expectiminimax, một phần mở rộng của thuật toán minimax đánh giá một nút cơ hội bằng cách lấy tiện ích trung bình của tất cả các con của nó, được tính trọng số theo xác suất của mỗi đứa trẻ.
- Trong các trò chơi thông tin không hoàn hảo, chẳng hạn như Kriegspiel và poker, cách chơi tối ưu đòi hỏi lý luận về trạng thái niềm tin hiện tại và tương lai của mỗi người chơi. Một đơn giản

Xấp xỉ có thể thu được bằng cách tính trung bình giá trị của một hành động trên mỗi cấu hình có thể có của thông tin bị thiếu.

- Các chương trình đã đánh bại những người chơi vô địch ở cờ vua, cờ đam, Othello, Go, poker và nhiều trò chơi khác. Con người vẫn giữ được lợi thế trong một số trò chơi thông tin hoàn hảo, chẳng hạn như bridge và Kriegspiel. Trong các trò chơi điện tử như StarCraft và Dota 2, các chương trình cạnh tranh với các chuyên gia con người, nhưng một phần thành công của chúng có thể là do khả năng thực hiện nhiều hành động rất nhanh.

Ghi chú thư mục và lịch sử

Năm 1846, Charles Babbage thảo luận về tính khả thi của cờ vua và cờ vua máy tính (Morrisson và Morrison, 1961). Ông không hiểu sự phức tạp theo cấp số nhân của cây tìm kiếm, tuyên bố "các kết hợp liên quan đến Công cụ phân tích vượt trội hơn rất nhiều so với bất kỳ yêu cầu nào, ngay cả trong trò chơi cờ vua." Babbage cũng thiết kế, nhưng không chế tạo, một cỗ máy chuyên dụng để chơi tic-tac-toe. Cỗ máy chơi game đầu tiên được chế tạo vào khoảng năm 1890 bởi kỹ sư người Tây Ban Nha Leonardo Torres y Quevedo. Nó chuyên về trò chơi kết thúc cờ vua "KRK" (vua và xe đấu với vua), đảm bảo chiến thắng khi bên có xe có nước đi. Thuật toán minimax được bắt nguồn từ một bài báo năm 1912 của Ernst Zermelo, nhà phát triển lý thuyết tập hợp hiện đại.

Chơi trò chơi là một trong những nhiệm vụ đầu tiên được thực hiện trong AI, với những nỗ lực ban đầu của những người như Konrad Zuse (1945), Norbert Wiener trong cuốn sách *Cybernetics* (1948) và Alan Turing (1953). Nhưng chính bài viết của Claude Shannon *Lập trình máy tính cho Playing Chess* (1950) đã đưa ra tất cả các ý tưởng chính: một đại diện cho các vị trí bàn cờ, một chức năng đánh giá, tìm kiếm tĩnh lặng và một số ý tưởng cho tìm kiếm cây trò chơi có chọn lọc. Slater (1950) đã có ý tưởng về một hàm đánh giá như một sự kết hợp tuyến tính của các đặc điểm và nhấn mạnh tính năng di động trong cờ vua.

John McCarthy đã hình thành ý tưởng tìm kiếm alpha-beta vào năm 1956, mặc dù ý tưởng này không xuất hiện trên bản in cho đến sau này (Hart và Edwards, 1961). Knuth và Moore (1975) đã chứng minh tính đúng đắn của alpha-beta và phân tích độ phức tạp về thời gian của nó, trong khi Pearl (1982b) cho thấy alpha-beta là tối ưu tiệm cận trong số tất cả các thuật toán tìm kiếm cây trò chơi có độ sâu cố định. Berliner (1979) đã giới thiệu B^* , một thuật toán tìm kiếm phỏng đoán duy trì giới hạn khoảng thời gian giá trị có thể có của một nút trong cây trò chơi thay vì đưa ra cho nó một ước tính có giá trị điểm duy nhất. Tìm kiếm số âm mưu của David McAllester (1988) mở rộng các nút lá, bằng cách thay đổi giá trị của chúng, có thể khiến chương trình thích một động thái mới ở gốc cây. MGSS* (Russell và Wefald, 1989) sử dụng các kỹ thuật lý thuyết quyết định của Chương 15 để ước tính giá trị của việc mở rộng mỗi chiếc lá về sự cải thiện dự kiến về chất lượng quyết định ở gốc. Thuật toán SSS* (Stockman, 1979) có thể được xem như một A hai người chơi* không bao giờ có nhiều nút hơn alpha-beta. Yêu cầu bộ nhớ làm cho nó không thực tế, nhưng phiên bản không gian tuyến tính đã được phát triển từ thuật toán RBFS (Korf và Chickering, 1996). Baum và Smith (1997) đề xuất một sự thay thế dựa trên xác suất cho minimax, cho thấy rằng nó dẫn đến những lựa chọn tốt hơn trong một số trò chơi nhất định. Thuật toán expectiminimax được đề xuất bởi Donald Michie (1966). Bruce Ballard (1983) mở rộng cấu trúc alpha-beta để che phủ cây có các nút cơ hội.

Cuốn sách *Heuristics* (1984) của Pearl phân tích kỹ lưỡng nhiều thuật toán chơi trò chơi. Mô phỏng Monte Carlo được tiên phong bởi Metropolis và Ulam (1949) cho các tính toán liên quan đến sự phát triển của bom nguyên tử. Tìm kiếm cây Monte Carlo (MCTS) được giới thiệu bởi Abramson (1987). Tesauro và Galperin (1997) đã chỉ ra cách tìm kiếm Monte Carlo có thể được kết hợp với chức năng đánh giá cho trò chơi cờ thò cáo. Chấm dứt chơi sớm được nghiên cứu bởi Lorentz (2015). ALPHAGO đã chấm dứt các lần chơi và áp dụng chức năng đánh giá (Silver et al., 2016). Kocsis và Szepesvari (2006) đã tinh chỉnh cách tiếp cận với cơ chế lựa chọn "Giới hạn tin cậy trên áp dụng cho cây". Chaslot et al. (2008) cho thấy cách MCTS có thể được áp dụng cho nhiều trò chơi khác nhau và Browne et al. (2012) đưa ra một cuộc khảo sát.

Koller và Pfeffer (1997) mô tả một hệ thống để giải quyết hoàn toàn các trò chơi có thể quan sát được một phần. Nó xử lý các trò chơi lớn hơn các hệ thống trước đó, nhưng không phải là phiên bản đầy đủ của các trò chơi phức tạp như poker và bridge. Frank et al. (1998) mô tả một số biến thể của Monte Carlo tìm kiếm cho các trò chơi có thể quan sát được một phần, bao gồm một trò chơi mà MIN có thông tin đầy đủ nhưng MAX thì không. Schofield và Thielscher (2005) điều chỉnh một hệ thống chơi trò chơi chung cho các trò chơi có thể quan sát được một phần.

Ferguson đã đưa ra các chiến lược ngẫu nhiên để giành chiến thắng Kriegspiel với một giám mục và hiệp sĩ (1992) hoặc hai giám mục (1995) chống lại một vị vua. Các chương trình Kriegspiel đầu tiên tập trung vào việc tìm kiếm những người chiếu hết cuối trò chơi và thực hiện tìm kiếm VÀ-HOẶC trong không gian trạng thái niềm tin (Sakuta và Iida, 2002; Bolognesi và Ciancarini, 2003). Trạng thái niềm tin gia tăng al-gorithms cho phép tìm thấy các chiếu hết giữa trò chơi phức tạp hơn nhiều (Russell và Wolfe, 2005; Wolfe và Russell, 2007), nhưng ước tính trạng thái hiệu quả vẫn là trở ngại chính đối với trò chơi chung hiệu quả (Parker và cộng sự, 2005). Ciancarini và Favini (2010) áp dụng MCTS cho Kriegspiel, và Wang et al. (2018b) mô tả một phiên bản trạng thái niềm tin của MCTS cho Phantom Go.

Các cột mốc cờ vua đã được đánh dấu bằng những người chiến thắng liên tiếp của Giải thưởng Fredkin: BELLE (Condon và Thompson, 1982), chương trình đầu tiên đạt được trạng thái bậc thầy; SUY NGHĨ SÂU SẮC (HSU và cộng sự, 1990), người đầu tiên đạt được trạng thái bậc thầy quốc tế; và Deep Blue (Campbell et al., 2002; Hsu, 2004), đã đánh bại nhà vô địch thế giới Garry Kasparov trong một trận đấu triển lãm năm 1997. Deep Blue chạy tìm kiếm alpha-beta ở hơn 100 triệu vị trí mỗi giây và có thể tạo ra các phần mở rộng đơn lẻ để đôi khi đạt đến độ sâu 40 lớp.

Các chương trình cờ vua hàng đầu ngày nay (ví dụ: STOCKFISH, KOMODO, HOUDINI) vượt xa bất kỳ người chơi nào. Các chương trình này đã giảm hệ số phân nhánh hiệu quả xuống dưới 3 (so với hệ số phân nhánh thực tế khoảng 35), tìm kiếm xuống còn khoảng 20 lớp với tốc độ khoảng một triệu nút mỗi giây trên máy tính 1 lõi tiêu chuẩn. Họ sử dụng các công nghệ cắt tỉa như suy nghiệm di chuyển rỗng, tạo ra giới hạn dưới tốt về giá trị của

Di chuyển rỗng

một vị trí, sử dụng một tìm kiếm nông trong đó đối thủ có thể di chuyển hai lần ngay từ đầu. Một điều quan trọng nữa là cắt tỉa vô ích, giúp quyết định trước những động thái nào sẽ gây ra một đoạn cắt beta trong các nút kế tiếp. SUNFISH là một chương trình cờ vua đơn giản hóa cho mục đích giảng dạy; lõi là ít hơn 200 dòng Python.

Cắt tỉa vô ích

Ý tưởng về phân tích nghịch hành để tính toán các bảng kết thúc trò chơi là do Bellman (1965). Sử dụng ý tưởng này, Ken Thompson (1986, 1996) và Lewis Stiller (1992, 1996) đã giải quyết tất cả các ván cờ với tối đa năm quân cờ. Stiller phát hiện ra một trường hợp có một người bạn đòi cưỡng bức nhưng cần 262 lần di chuyển; Điều này gây ra một số kinh ngạc vì các quy tắc của cờ vua yêu cầu bắt hoặc di chuyển tốt phải xảy ra trong vòng 50 nước đi, nếu không thì hòa được tuyên bố. Năm 2012 Vladimir Makhnychev và Victor Zakharov biên soạn Lomonosov Endgame Tablebase,

giải quyết tất cả các vị trí kết thúc trò chơi với tối đa bảy quân cờ — một số yêu cầu hơn 500 lần di chuyển mà không bị bắt. Bảng 7 phần tiêu thụ 140 terabyte; một chiếc bàn 8 mảnh sẽ lớn hơn 100 lần.

Năm 2017, ALPHAZERO (Silver et al., 2018) đã đánh bại STOCKFISH (nhà vô địch cờ vua TCEC 2017) trong một thử nghiệm 1000 trận, với 155 trận thắng và 6 trận thua. Các trận đấu bổ sung cũng mang lại chiến thắng quyết định cho ALPHAZERO, ngay cả khi nó chỉ được trao 1/10 thời gian được phân bổ cho STOCKFISH.

Đại kiện tướng Larry Kaufman đã rất ngạc nhiên về sự thành công của chương trình Monte Carlo này và lưu ý, "Có thể sự thống trị hiện tại của động cơ cờ vua minimax có thể kết thúc, nhưng còn quá sớm để nói như vậy." Garry Kasparov nhận xét: "Đó là một thành tựu đáng chú ý, ngay cả khi chúng ta nên mong đợi nó sau ALPHAGO. Nó tiếp cận cách tiếp cận giống con người Loại B đối với cờ vua máy mà Claude Shannon và Alan Turing mơ ước thay vì vũ phu. Ông tiếp tục dự đoán "Cờ vua đã bị ALPHAZERO rung chuyển đến gốc rễ của nó, nhưng đây chỉ là một ví dụ nhỏ về những gì sắp xảy ra. Các ngành học ẩn nấp như giáo dục và y học cũng sẽ bị lung lay" (Sadler và Regan, 2019).

Cờ đam là trò chơi cổ điển đầu tiên được chơi bằng máy tính (Strachey, 1952). Arthur Samuel (1959, 1967) đã phát triển một chương trình kiểm tra học chức năng đánh giá của riêng nó thông qua tự chơi bằng cách sử dụng một hình thức học tăng cường. Đó là một thành tựu khá lớn khi Samuel có thể tạo ra một chương trình chơi tốt hơn anh ấy, trên một máy tính IBM 704 chỉ với bộ nhớ 10.000 từ và bộ xử lý 0,000001 GHz. MENACE — Máy Educable Noughts And Crosses Engine (Michie, 1963) — cũng sử dụng học tăng cường để trở nên thành thạo tic-tac-toe. Bộ xử lý của nó thậm chí còn chậm hơn: một bộ sưu tập 304 hươu diêm chứa các hạt màu để đại diện cho động tác học tốt nhất ở mỗi vị trí.

Năm 1992, chương trình cờ đam CHINOOK của Jonathan Schaeffer đã thách thức huyền thoại Marion Tinsley, người đã là nhà vô địch thế giới trong hơn 20 năm. Tinsley đã thắng trận đầu, nhưng thua hai trận — trận thua thứ tư và thứ năm trong toàn bộ sự nghiệp của anh ấy. Sau khi Tinsley giải nghệ vì lý do sức khỏe, CHINOOK đã lên ngôi. Câu chuyện được ghi lại bởi Schaeffer (2008).

Năm 2007, Schaeffer và nhóm của ông đã "giải quyết" các trận cờ đam (Schaeffer et al., 2007): trò chơi là một trận hòa với lối chơi hoàn hảo. Richard Bellman (1965) đã dự đoán điều này: "Trong cờ đam, số lượng nước đi có thể xảy ra trong bất kỳ tình huống nhất định nào là rất nhỏ đến mức chúng ta có thể tự tin mong đợi một giải pháp máy tính kỹ thuật số hoàn chỉnh cho vấn đề chơi tối ưu trong trò chơi này." Bellman đã không lường trước được quy mô của nỗ lực: bàn kết thúc trò chơi cho 10 quân cờ có 39 nghìn tỷ mục nhập. Với bảng này, phải mất 18 năm CPU-năm tìm kiếm alpha-beta để giải quyết trò chơi.

I. J. Good, người được Alan Turing dạy Trò chơi cờ vây, đã viết (1965a) "Tôi nghĩ sẽ còn khó khăn hơn để lập trình một máy tính để chơi một trò chơi cờ vây hợp lý hơn là cờ vua." Anh ấy đã đúng: cho đến năm 2015, các chương trình cờ vây chỉ chơi ở cấp độ nghiệp dư. Các tài liệu ban đầu được tóm tắt bởi Bouzy và Cazenave (2001) và Müller (2002).

Nhận dạng mẫu trực quan được đề xuất như một kỹ thuật đầy hứa hẹn cho Go bởi Zobrist (1970), trong khi Schraudolph et al. (1994) phân tích việc sử dụng học tăng cường. Lubbertsand Miikkulainen (2001) đề xuất mạng nơ-ron, và Brügmann (1993) giới thiệu tìm kiếm cây Monte Carlo cho Go. ALPHAGO (Silver et al., 2016) kết hợp bốn ý tưởng đó lại với nhau để đánh bại các chuyên gia hàng đầu Lee Sedol (với tỷ số 4–1 vào năm 2015) và Ke Jie (với tỷ số 3–0 vào năm 2016).

Ke Jie nhận xét: "Sau khi nhân loại dành hàng ngàn năm để cải thiện chiến thuật của chúng ta, những người tính toán nói với chúng ta rằng con người hoàn toàn sai lầm. Tôi sẽ đi xa đến mức nói rằng không một con người nào

đã chạm đến ranh giới của sự thật của cờ vây." Lee Sedol đã nghỉ hưu từ Go, than thở, "Ngay cả khi tôi trở thành số một, vẫn có một thực thể không thể bị đánh bại."

Vào năm 2018, ALPHAZERO đã vượt qua ALPHAGO tại cờ vây, và cũng đánh bại các chương trình hàng đầu inch và shogi, học thông qua tự chơi mà không có bất kỳ kiến thức chuyên môn nào của con người và không có quyền truy cập vào bất kỳ trò chơi nào trong quá khứ. (Tất nhiên, nó dựa vào con người để xác định kiến trúc cơ bản như tìm kiếm cây Monte Carlo với mạng nơ-ron sâu và học tăng cường, đồng thời mã hóa các quy tắc của trò chơi.) Thành công của ALPHAZERO đã dẫn đến sự quan tâm ngày càng tăng đối với học tăng cường như một thành phần quan trọng của AI nói chung (xem Chương 23). Tiến thêm một bước, hệ thống MUZERO hoạt động mà không cần được cho biết các quy tắc của trò chơi mà nó đang chơi — nó phải tìm ra các quy tắc bằng cách thực hiện các vở kịch. MUZERO đã đạt được kết quả hiện đại trong các trò chơi Pacman, cờ vua, cờ vây và 75 trò chơi Atari (Schrittwieser và cộng sự, 2019). Nó học cách khái quát hóa; ví dụ, nó học được rằng trong Pacman, hành động "lên" di chuyển người chơi lên một ô vuông (trừ khi có một bức tường ở đó), mặc dù nó chỉ quan sát kết quả của hành động "lên" trong một tỷ lệ nhỏ các vị trí trên bàn cờ.

Othello, còn được gọi là Reversi, có không gian tìm kiếm nhỏ hơn cờ vua, nhưng việc xác định một hàm đánh giá rất khó, bởi vì lợi thế vật chất không quan trọng bằng tính di động. Pro-gram đã ở cấp độ siêu phàm từ năm 1997 (Buro, 2002).

Backgammon, một trò chơi may rủi, được phân tích toán học bởi Gerolamo Cardano (1663), và được sử dụng để chơi trên máy tính với chương trình BKG (Berliner, 1980b), sử dụng chức năng đánh giá được xây dựng thủ công và chỉ tìm kiếm đến độ sâu 1. Đây là chương trình đầu tiên đánh bại một nhà vô địch thế giới của con người tại một trận đấu lớn (Berliner, 1980a), mặc dù Berliner dễ dàng thừa nhận rằng BKG rất may mắn với xúc xắc. TD-GAMMON của Gerry Tesauro (1995) đã học chức năng đánh giá của nó bằng cách sử dụng mạng nơ-ron được đào tạo bằng cách tự chơi. Nó liên tục chơi ở cấp độ vô địch thế giới và khiến các nhà phân tích con người thay đổi quan điểm của họ về nước đi mờ đầu tốt nhất cho một số lần tung xúc xắc.

Poker, giống như cờ vây, đã chứng kiến những tiến bộ đáng ngạc nhiên trong những năm gần đây. Bowling et al. (2015) đã sử dụng lý thuyết trò chơi (xem Phần 17.2) để xác định chiến lược tối ưu chính xác cho một phiên bản poker chỉ có hai người chơi và số lần tăng cổ định với quy mô đặt cược cố định. Vào năm 2017, lần đầu tiên, những người chơi poker vô địch bị đánh bại ở hai trận đấu riêng biệt với các chương trình Libratus (Brown và Sandholm, 2017) và DeepStack (Moravčík et al., 2017). Vào năm 2019, Pluribus (Brown và Sandholm, 2019) đã đánh bại những người chơi chuyên nghiệp được xếp hạng hàng đầu trong các trò chơi giữ bóng ở Texas với sáu người chơi. Trò chơi nhiều người chơi giới thiệu một số mối quan tâm chiến lược mà chúng ta sẽ đề cập trong Chương 17. Petosa và Balch (2019) triển khai phiên bản nhiều người chơi của ALPHAZERO.

Bridge: Smith et al. (1998) báo cáo về cách BRIDGE BARON giành chức vô địch cầu máy tính năm 1998, sử dụng các kế hoạch phân cấp (xem Chương 11) và các hành động cấp cao, chẳng hạn như tính tế và bốp, quen thuộc với người chơi cầu. Ginsberg (2001) mô tả chương trình GIB của mình, dựa trên mô phỏng Monte Carlo (lần đầu tiên được đề xuất cho cầu bởi Levy (1989)), đã giành chức vô địch máy tính sau đó và làm tốt một cách đáng ngạc nhiên trước những người chơi chuyên nghiệp. Trong thế kỷ 21, giải vô địch cầu máy tính đã bị thống trị bởi hai chương trình thương mại, JACK và WBRIDGE5. Cả hai đều không được mô tả trong các bài báo đã xuất bản, nhưng cả hai đều được cho là sử dụng kỹ thuật Monte Carlo. Nhìn chung, các chương trình cầu ở cấp độ vô địch của con người khi thực sự chơi ván bài, nhưng tụt lại phía sau trong giai đoạn đầu thầu, vì chúng không hoàn toàn hiểu các quy ước mà con người sử dụng để giao tiếp với đối tác của họ. Các lập trình viên Bridge đã tập trung nhiều hơn vào sản xuất

các chương trình hữu ích và giáo dục khuyến khích mọi người tham gia trò chơi, thay vì đánh bại các nhà vô địch của con người.

Scrabble là một trò chơi mà những người chơi nghiệp dư gặp khó khăn trong việc nghĩ ra những từ có điểm cao, nhưng đối với máy tính, thật dễ dàng để tìm thấy điểm cao nhất có thể cho một ván bài nhất định (Gordon, 1994); Phần khó khăn là lập kế hoạch trước trong một trò chơi ngẫu nhiên, có thể quan sát được một phần. Tuy nhiên, vào năm 2006, chương trình QUACKLE đã đánh bại cựu vô địch thế giới, David Boys, 3–2. Các chàng trai đã chấp nhận điều đó, nói rằng, "Vấn đề tốt hơn là làm người hơn là trở thành một máy tính." Một mô tả tốt về một chương trình hàng đầu, MAVEN, được đưa ra bởi Sheppard (2002).

Các trò chơi điện tử như StarCraft II liên quan đến hàng trăm đơn vị có thể quan sát được một phần di chuyển trong thời gian thực với không gian quan sát và hành động gần liên tục⁶ chiều cao với các quy tắc phức tạp. Oriol Vinyals, nhà vô địch StarCraft của Tây Ban Nha ở tuổi 15, đã mô tả cách trò chơi có thể đóng vai trò như một thử nghiệm và thử thách lớn cho việc học tăng cường (Vinyals và cộng sự, 2017a). Vào năm 2019, Vinyals và nhóm tại DeepMind đã tiết lộ chương trình ALPHASTAR, dựa trên học sâu và học tăng cường, đánh bại những người chơi chuyên nghiệp với tỷ lệ 10 trò chơi và xếp hạng trong 0,02% người chơi được xếp hạng chính thức (Vinyals và cộng sự, 2019).

ALPHASTAR đã thực hiện các bước để hạn chế số lượng hành động mỗi phút mà nó có thể thực hiện các vụ nổ không quan trọng, để đáp lại những người chỉ trích cảm thấy nó có lợi thế không công bằng.

Máy tính đã đánh bại những con người hàng đầu trong các trò chơi điện tử nổi tiếng khác như Super SmashBros. (Firoiu và cộng sự, 2017), Quake III (Jaderberg và cộng sự, 2019) và Dota 2 (Fernandez và Mahlmann, 2018), tất cả đều sử dụng kỹ thuật học sâu.

Các trò chơi thể chất như bóng đá robot (Visser et al., 2008; Barrett và Stone, 2015), bida (Lam và Greenspan, 2008; Archibald et al., 2009) và ping-pong (Silva et al., 2015) đã thu hút một số sự chú ý trong AI. Chúng kết hợp tất cả các phức tạp của trò chơi điện tử với sự lộn xộn của thế giới thực.

Các cuộc thi trò chơi máy tính diễn ra hàng năm, bao gồm cả Olympic Máy tính kể từ đó 1989. Cuộc thi trò chơi chung (Love et al., 2006) kiểm tra các chương trình phải học cách chơi một trò chơi chưa biết chỉ được mô tả hợp lý về các quy tắc của trò chơi. Hiệp hội Trò chơi Máy tính Quốc tế (ICGA) xuất bản Tạp chí ICGA và tổ chức hai hội nghị hai năm một lần, Hội nghị Quốc tế về Máy tính và Trò chơi (ICCGor CG) và Hội nghị Quốc tế về Tiến bộ trong Trò chơi Máy tính (ACG). IEEE xuất bản Giao dịch IEEE về Trò chơi và tổ chức Hội nghị hàng năm về Trí tuệ tính toán và Trò chơi.

⁶ Đối với một người chơi con người, có vẻ như các vật thể di chuyển liên tục, nhưng chúng thực sự rời rạc ở mức apixel trên màn hình.

CHƯƠNG 7

TÁC NHÂN LOGIC

Trong đó chúng ta thiết kế các tác nhân có thể hình thành các đại diện của một thế giới phức tạp, sử dụng một quá trình suy luận để rút ra các đại diện mới về thế giới và sử dụng những đại diện mới này để suy ra những gì cần làm.

Tác nhân dựa
trên tri thức
luậnĐại diện

Con người, dường như, biết mọi thứ; và những gì họ biết giúp họ làm mọi việc. Trong AI, các tác nhân dựa trên tri thức sử dụng một quá trình suy luận trên một biểu diễn nội bộ của kiến thức để quyết định những hành động cần thực hiện.

Các tác nhân giải quyết vấn đề của Chương 3 và 4 biết mọi thứ, nhưng chỉ theo nghĩa rất hạn chế, không linh hoạt. Họ biết những hành động nào có sẵn và kết quả của việc thực hiện một hành động cụ thể từ một trạng thái cụ thể sẽ như thế nào, nhưng họ không biết sự thật chung. Một đại lý tìm đường không biết rằng một con đường không thể có số km âm. Một đặc vụ 8 câu đố không biết rằng hai ô không thể chiếm cùng một không gian. Kiến thức họ có rất hữu ích để tìm ra con đường từ đầu đến mục tiêu, nhưng không phải cho bất cứ điều gì khác.

Các biểu diễn nguyên tử được sử dụng bởi các tác nhân giải quyết vấn đề cũng rất hạn chế. Ví dụ, trong môi trường có thể quan sát được một phần, lựa chọn duy nhất của tác nhân giải quyết vấn đề để thể hiện những gì nó biết về trạng thái hiện tại là liệt kê tất cả các trạng thái cụ thể có thể có. Tôi có thể đặt mục tiêu cho một con người là lái xe đến một thị trấn của Hoa Kỳ với dân số dưới 10.000 người, nhưng để nói rằng với một đại lý giải quyết vấn đề, tôi có thể chính thức mô tả mục tiêu chỉ như một tập hợp rõ ràng của 16.000 thị trấn hoặc lâu hơn thỏa mãn mô tả.

Chương 5 giới thiệu biểu diễn thừa số đầu tiên của chúng ta, theo đó các trạng thái được biểu diễn dưới dạng gán giá trị cho các biến; Đây là một bước đi đúng hướng, cho phép một số phần của tác nhân hoạt động theo cách độc lập với miền và cho phép các thuật toán hiệu quả hơn. Trong chương này, chúng ta đưa bước này đến kết luận logic của nó, có thể nói như vậy - chúng ta phát triển logic như một lớp biểu diễn chung để hỗ trợ các tác nhân dựa trên tri thức. Các tác nhân này có thể kết hợp và kết hợp lại thông tin để phù hợp với vô số mục đích. Điều này có thể khác xa với nhu cầu của thời điểm này - như khi một nhà toán học chứng minh một định lý hoặc một nhà thiên văn học tính toán tuổi thọ của Trái đất. Các tác nhân dựa trên tri thức có thể chấp nhận các nhiệm vụ mới dưới dạng các mục tiêu được mô tả rõ ràng; họ có thể đạt được năng lực nhanh chóng bằng cách được nói hoặc học kiến thức mới về môi trường; và họ có thể thích ứng với những thay đổi của môi trường bằng cách cập nhật kiến thức liên quan.

Chúng ta bắt đầu trong Phần 7.1 với thiết kế tổng thể của tác nhân. Phần 7.2 giới thiệu một môi trường đơn giản mới, thế giới wumpus, và minh họa hoạt động của một tác nhân dựa trên tri thức mà không đi sâu vào bất kỳ chi tiết kỹ thuật nào. Sau đó, chúng tôi giải thích các nguyên tắc chung của logic trong Phần 7.3 và các chi tiết cụ thể của logic mệnh đề trong Phần 7.4. Logic mệnh đề là một đại diện cơ yếu tố; Trong khi ít biểu cảm hơn logic bậc nhất (Chương 8), là biểu diễn có cấu trúc kinh điển, logic mệnh đề minh họa tất cả các khái niệm cơ bản

hàm KB-AGENT(percept) trả về một hành động
persistent: KB, một cơ sở kiến thức
t, bộ đếm, ban đầu là 0, biểu thị thời gian

```
TELL(KB, MAKE-PERCEPT-
SENTENCE(percept, t))action ← ASK(KB,
MAKE-ACTION-QUERY(t))TELL(KB, MAKE-
ACTION-SENTENCE(action, t))t ← t + 1return
action
```

Hình 7.1 Một tác nhân dựa trên tri thức chung. Với một nhận thức, tác nhân thêm nhận thức vào cơ sở kiến thức của nó, yêu cầu cơ sở tri thức cho hành động tốt nhất và nói với cơ sở tri thức rằng nó đã thực hiện hành động đó.

của logic. Nó cũng đi kèm với các công nghệ suy luận được phát triển tốt, mà chúng tôi mô tả trong phần 7.5 và 7.6. Cuối cùng, Phần 7.7 kết hợp khái niệm tác nhân dựa trên tri thức với công nghệ logic mệnh đề để xây dựng một số tác nhân đơn giản cho thế giới wumpus.

7.1 Đại lý dựa trên tri thức

Thành phần trung tâm của một tác nhân dựa trên tri thức là cơ sở tri thức của nó, hoặc KB. Một kiến thức-Edge Base là một tập hợp các câu. (Ở đây "câu" được sử dụng như một thuật ngữ kỹ thuật. Nó có liên quan

Cơ sở kiến thức

Câu

nhưng không giống với các câu của tiếng Anh và các ngôn ngữ tự nhiên khác.) Mỗi câu được thể hiện bằng một ngôn ngữ được gọi là ngôn ngữ biểu diễn tri thức và đại diện cho một số

Ngôn ngữ đại diện kiến thức

khẳng định về thế giới. Khi câu được coi là được đưa ra mà không bắt nguồn từ các câu khác, chúng ta gọi nó là tiên đề.

Chân lý

Phải có một cách để thêm các câu mới vào cơ sở tri thức và một cách để truy vấn những gì đã biết. Tên tiêu chuẩn cho các hoạt động này lần lượt là TELL và ASK. Cả hai hoạt động đều có thể liên quan đến suy luận — nghĩa là rút ra các câu mới từ câu cũ. Suy luận

Suy luận

phải tuân theo yêu cầu rằng khi một người trả lời một câu hỏi về cơ sở tri thức, câu trả lời phải tuân theo từ những gì đã được nói (hoặc TELLED) đến cơ sở tri thức trước đó. Sau chương này, chúng ta sẽ chính xác hơn về từ quan trọng "đi theo". Bây giờ, hãy coi nó có nghĩa là quá trình suy luận không nên tạo ra mọi thứ khi nó diễn ra.

Hình 7.1 cho thấy phác thảo của một chương trình tác nhân dựa trên tri thức. Giống như tất cả các tác nhân của chúng tôi, nó nhận một nhận thức làm đầu vào và trả về một hành động. Đại lý duy trì một cơ sở kiến thức, KB, ban đầu có thể chứa một số kiến thức cơ bản.

Kiến thức nền tảng

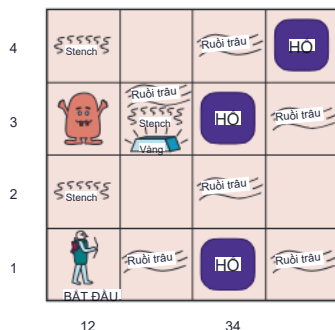
Mỗi khi chương trình tác nhân được gọi, nó thực hiện ba việc. Đầu tiên, nó TELLS cơ sở tri thức những gì nó nhận thức. Thứ hai, nó hỏi cơ sở kiến thức về hành động mà nó nên thực hiện. Trong quá trình trả lời câu hỏi này, lý luận mở rộng có thể được thực hiện về tình trạng hiện tại của thế giới, về kết quả của các chuỗi hành động có thể xảy ra, v.v. Thứ ba, chương trình tác nhân TELLS cơ sở tri thức hành động đã được chọn và trả về hành động để nó có thể được thực thi.

Các chi tiết của ngôn ngữ biểu diễn được ẩn bên trong ba chức năng thực hiện giao diện giữa các cảm biến và bộ truyền động ở một bên và hệ thống biểu diễn và suy luận cốt lõi ở bên kia. MAKE-PERCEPT-SENTENCE xây dựng một câu như -

	Serting rằng tác nhân nhận thức được tri giác đã cho tại thời điểm nhất định. MAKE-ACTION-QUERYxây dựng một câu hỏi hành động nào nên được thực hiện tại thời điểm hiện tại. Cuối cùng, MAKE-ACTION-SENTENCE xây dựng một câu khẳng định rằng hành động đã chọn đã được loại bỏ. Các chi tiết của cơ chế suy luận được ẩn bên trong TELL và ASK. Các phần sau sẽ tiết lộ những chi tiết này. Tác nhân trong Hình 7.1 xuất hiện khá giống với các tác nhân có trạng thái bên trong được mô tả trong Chương 2. Tuy nhiên, do các định nghĩa của TELL và ASK, tác nhân dựa trên tri thức không phải là một chương trình tùy ý để tính toán các hành động. Nó phù hợp với một mô tả ở cấp độ tri thức, nơi chúng ta chỉ cần xác định những gì tác nhân biết và mục tiêu của nó là gì, để xác định hành vi của nó. Ví dụ, một chiếc taxi tự động có thể có mục tiêu đưa một hành khách từ SanFrancisco đến Hạt Marin và có thể biết rằng Cầu Cổng Vàng là liên kết duy nhất giữa hai địa điểm. Sau đó, chúng ta có thể mong đợi nó băng qua Cầu Cổng Vàng vì nó biết rằng điều đó sẽ đạt được mục tiêu của nó. Lưu ý rằng phân tích này độc lập với cách taxi hoạt động ở cấp độ thực hiện. Không quan trọng kiến thức địa lý của nó là được thực hiện dưới dạng danh sách liên kết hoặc bản đồ pixel, hoặc liệu nó có lý do bằng cách thao tác các chuỗi ký hiệu được lưu trữ trong thanh ghi hoặc bằng cách truyền tín hiệu nhiều trong mạng lưới tế bào thần kinh. Một tác nhân dựa trên tri thức có thể được xây dựng đơn giản bằng cách cung cấp cho nó những gì nó cần biết. Bắt đầu với một cơ sở tri thức trống rỗng, nhà thiết kế tác nhân có thể KẾ từng câu một cho đến khi tác nhân biết cách hoạt động trong môi trường của nó. Đây được gọi là cách tiếp cận khai báo để xây dựng hệ thống. Ngược lại, cách tiếp cận thủ tục mã hóa trực tiếp các hành vi mong muốn làm mã chương trình. Trong những năm 1970 và 1980, những người ủng hộ hai cách tiếp cận đã tham gia vào các cuộc tranh luận sôi nổi. Bây giờ chúng ta hiểu rằng một tác nhân thành công thường kết hợp cả hai yếu tố khai báo và thủ tục trong thiết kế của nó, và kiến thức khai báo thường có thể được biên dịch thành mã thủ tục hiệu quả hơn. Chúng ta cũng có thể cung cấp một tác nhân dựa trên tri thức với các cơ chế cho phép nó tự học. Những cơ chế này, được thảo luận trong Chương 19, tạo ra kiến thức chung về môi trường từ một loạt các nhận thức. Một tác nhân học tập có thể hoàn toàn tự chủ.
Trình độ kiến thức	
Mức độ thực hiện	
Khai báo	
Thủ tục	

7.2 Thế giới Wumpus

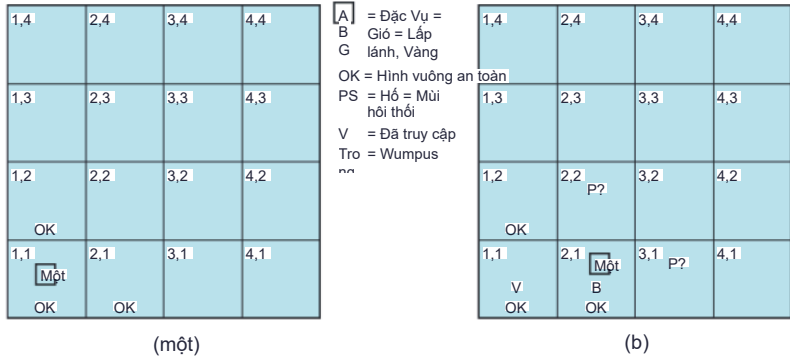
Thế giới Wumpus	Trong phần này, chúng tôi mô tả một môi trường trong đó các tác nhân dựa trên tri thức có thể thể hiện giá trị của họ. Thế giới wumpus là một hang động bao gồm các phòng được nối với nhau bằng lối đi. Bí mật Đầu đó trong hang động là Wumpus khủng khiếp, một con thú ăn thịt bất cứ ai bước vào phòng của nó. Wumpus có thể bị bắn bởi một đặc vụ, nhưng đặc vụ chỉ có một mũi tên. Một số phòng chứa những hồ không đáy sẽ bẫy bất kỳ ai đi lang thang vào những căn phòng này (ngoại trừ wumpus, quá lớn để rơi vào). Tính năng chuộc lỗi duy nhất của môi trường âm đạm này là khả năng tìm thấy một đồng vàng. Mặc dù thế giới wumpus khá thuần hóa theo tiêu chuẩn trò chơi máy tính hiện đại, nhưng nó minh họa một số điểm quan trọng về trí thông minh. Một mẫu thế giới wumpus được thể hiện trong Hình 7.2. Định nghĩa chính xác về môi trường nhiệm vụ được đưa ra, như được đề xuất trong Phần 2.3, bởi mô tả PEAS: • Thước đo hiệu suất: +1000 khi trèo ra khỏi hang với vàng, −1000 rơi xuống hồ hoặc bị wumpus ăn thịt, −1 cho mỗi hành động được thực hiện và −10 cho mũi tên lên. Trò chơi kết thúc khi đặc vụ chết hoặc khi đặc vụ trèo ra khỏi hang.
-----------------	--



Hình 7.2 Một thế giới wumpus điển hình. Đặc vụ ở góc dưới cùng bên trái, hướng về phía đông (bên phải).

- **Môi trường:** Một lưới phòng 4×4 , với các bức tường bao quanh lưới. Tác nhân luôn bắt đầu từ hình vuông có nhãn $[1,1]$, quay mặt về phía đông. Vị trí của vàng và wumpus được chọn ngẫu nhiên, với sự phân bố đồng đều, từ các ô vuông khác với ô vuông bắt đầu. Ngoài ra, mỗi ô vuông khác với điểm bắt đầu có thể là một hồ, với xác suất 0,2.
- **Bộ truyền động:** Tác nhân có thể di chuyển về phía trước, Rẽ trái bằng 90° hoặc Rẽ phải bằng 90° . Đặc vụ chết cách thức thăm hại nếu nó đi vào một ô vuông chứa một cái hồ hoặc một wumpus sống. (Thật an toàn, mặc dù có mùi, khi bước vào một quảng trường với một con wumpus đã chết.) Nếu một tác nhân cố gắng di chuyển về phía trước và va vào tường, thì tác nhân đó sẽ không di chuyển. Hành động Grab có thể được sử dụng để nhặt vàng nếu nó nằm trong cùng một ô vuông với đại lý. Hành động Bắn có thể được sử dụng để bắn một mũi tên theo một đường thẳng theo hướng mà đặc vụ đang đối mặt. Mũi tên tiếp tục cho đến khi nó bắn trúng (và do đó giết) wumpus hoặc đâm vào tường. Đặc vụ chỉ có một mũi tên, vì vậy chỉ hành động Bắn đầu tiên mới có bất kỳ tác dụng nào. Cuối cùng, hành động Leo có thể được sử dụng để trèo ra khỏi hang, nhưng chỉ từ ô vuông $[1,1]$.
- **Cảm biến:** Tác nhân có năm cảm biến, mỗi cảm biến cung cấp một chút thông tin duy nhất: – Trong các ô vuông trực tiếp (không theo đường chéo) liền kề với wumpus, tác nhân sẽ cảm nhận được mùi hôi thối. 1– Trong các ô vuông liền kề với một cái hồ, tác nhân sẽ cảm nhận được một làn gió. – Trong hình vuông nơi có vàng, tác nhân sẽ cảm nhận được một ánh sáng lấp lánh. – Khi một tác nhân bước vào tường, nó sẽ cảm nhận được một Bump. – Khi wumpus bị giết, nó phát ra một tiếng hét đáng buồn có thể cảm nhận được ở bất cứ đâu trong hang. Các nhận thức sẽ được đưa ra cho chương trình tác nhân dưới dạng một danh sách năm biểu tượng; ví dụ: nếu có mùi hôi thối và gió, nhưng không có lấp lánh, và chạm hoặc la hét, chương trình đại lý sẽ nhận được [Hôi thối, Gió, Không có, Không có, Không có].

1 Có lẽ hình vuông chứa wumpus cũng có mùi hôi thối, nhưng bất kỳ tác nhân nào đi vào ô vuông đó đều bị ăn thịt trước khi có thể nhận thức được bất cứ thứ gì.



Hình 7.3 Bước đầu tiên được thực hiện bởi tác nhân trong thế giới wumpus. (a) Trạng thái ban đầu, sau khi nhận thức [Không có, Không có, Không có, Không có, Không có]. (b) Sau khi di chuyển đến [2,1] và nhận thức [Không có, Gió, Không có, Không có, Không có].

Chúng ta có thể mô tả đặc điểm của môi trường wumpus dọc theo các chiều khác nhau được đưa ra trong Chương 2. Rõ ràng, nó là xác định, rời rạc, tĩnh và đơn tác nhân. (May mắn thay, wumpus không di chuyển.) Nó là tuần tự, bởi vì phần thưởng chỉ có thể đến sau khi thực hiện nhiều hành động. Nó có thể quan sát được một phần, bởi vì một số khía cạnh của trạng thái không thể cảm nhận trực tiếp: vị trí của tác nhân, tình trạng sức khỏe của wumpus và sự sẵn có của một mùi tên. Đối với vị trí của các hố và wumpus: chúng ta có thể coi chúng là những phần không được quan sát của trạng thái - trong trường hợp đó, mô hình chuyển tiếp cho môi trường là hoàn toàn được biết đến, và việc tìm ra vị trí của các hố hoàn thiện kiến thức của tác nhân về trạng thái. Ngoài ra, chúng ta có thể nói rằng bản thân mô hình chuyển tiếp là không biết vì tác nhân không biết hành động chuyển tiếp nào gây tử vong — trong trường hợp đó, việc khám phá vị trí của các hố và wumpus hoàn thành kiến thức của tác nhân về mô hình chuyển tiếp.

Đối với một tác nhân trong môi trường, thách thức chính là sự thiếu hiểu biết ban đầu của nó về cấu hình môi trường; Vượt qua sự thiếu hiểu biết này dường như đòi hỏi lý luận logic. Trong hầu hết các trường hợp của thế giới wumpus, đặc vụ có thể lấy lại vàng một cách an toàn. Đôi khi, đặc vụ phải lựa chọn giữa việc về nhà tay không và liều chết để tìm vàng. Khoảng 21% môi trường hoàn toàn không công bằng, bởi vì vàng nằm trong một piton được bao quanh bởi các hố.

Chúng ta hãy xem một tác nhân wumpus dựa trên kiến thức khám phá môi trường được hiển thị trong Hình 7.2. Chúng tôi sử dụng một ngôn ngữ biểu diễn tri thức không chính thức bao gồm các ký hiệu viết trong lưới (như trong Hình 7.3 và 7.4).

Cơ sở kiến thức ban đầu của tác nhân chứa các quy tắc của môi trường, như đã mô tả trước đây; đặc biệt, nó biết rằng nó nằm trong [1,1] và [1,1] là một ô vuông an toàn; chúng ta biểu thị rằng với "A" và "OK", tương ứng, trong hình vuông [1,1].

Nhận thức đầu tiên là [None, None, None, None, None, None], từ đó tác nhân có thể kết luận rằng các ô vuông lân cận của nó, [1,2] và [2,1], không có nguy hiểm — chúng ổn. Hình 7.3 (a) cho thấy trạng thái kiến thức của tác nhân tại thời điểm này.

1,42,4		3,44,4	
1,32,3		3,34,3	
1,2	2,2	3,2	4,2
1,1	2,1	3,1	4,1

[A] = Đặc Vụ =
 B = Gió = Lấp
 G = lảnh, Vàng
 OK = Hình vuông an toàn
 PS = Hồ = Mùi
 hôi thối
 V = Đã truy cập
 Tro = Wumpus
 ~ =

1,42,4	P?	3,44,4	
1,32,3	Một	3,34,3	
1,2	SG		
1,1	B		
1,2	S	2,2	3,2
V	OK	V	OK
1,1		2,1	3,1
V	OK	B	PI
		V	

(b)(một)

Hình 7.4 Hai giai đoạn sau trong quá trình tiến trình của tác nhân. (a) Sau khi chuyển đến [1,1] và sau đó [1,2], và nhận thức được [Hôi thối, Không có, Không có, Không có, Không có]. (b) Sau khi di chuyển đến [2,2] và sau đó [2,3], và cảm nhận được [Hôi thối, Gió, Lấp lảnh, Không có, Không có].

Một tác nhân thận trọng sẽ chỉ di chuyển vào một ô vuông mà nó biết là ổn. Giả sử tác nhân quyết định chuyển sang [2,1]. Tác nhân cảm nhận được một làn gió (được ký hiệu bằng "B") trong [2,1], vì vậy phải có một cái hồ ở một ô vuông lân cận. Hồ không thể ở trong [1,1], theo luật chơi, vì vậy phải có hồ trong [2,2] hoặc [3,1] hoặc cả hai. Ký hiệu "P?" trong Hình 7.3 (b) chỉ ra một hồ có thể xảy ra trong các ô vuông đó. Tại thời điểm này, chỉ có một quảng trường được biết đến là OK và chưa được ghé thăm. Vì vậy, tác nhân thận trọng sẽ quay lại, quay trở lại [1,1], và sau đó tiến hành [1,2].

Tác nhân cảm nhận được mùi hôi thối trong [1,2], dẫn đến trạng thái kiến thức được thể hiện trong Hình 7.4 (a). Mùi hôi thối trong [1,2] có nghĩa là phải có một wumpus gần đó. Nhưng wumpus không thể ở trong [1,1], theo luật chơi, và nó không thể ở trong [2,2] (hoặc tác nhân sẽ phát hiện ra mùi hôi thối khi nó ở trong [2,1]). Do đó, tác nhân có thể suy ra rằng wumpus trong [1,3]. Ký hiệu W! chỉ ra suy luận này. Hơn nữa, việc thiếu gió trong [1,2] ngụ ý rằng không có hồ trong [2,2]. Tuy nhiên, tác nhân đã suy luận rằng phải có một pitin hoặc [2,2] hoặc [3,1], vì vậy điều này có nghĩa là nó phải ở trong [3,1]. Đây là một suy luận khá khó khăn, bởi vì nó kết hợp kiến thức thu được vào những thời điểm khác nhau ở những nơi khác nhau và dựa vào việc thiếu nhận thức để thực hiện một bước quan trọng.

Tác nhân hiện đã chứng minh với chính mình rằng không có hồ hay wumpus trong [2,2], vì vậy có thể di chuyển đến đó. Chúng tôi không hiển thị trạng thái kiến thức của tác nhân tại [2,2]; chúng ta chỉ giả định rằng tác nhân quay và di chuyển đến [2,3], cho chúng ta Hình 7.4(b). Trong [2,3], đặc vụ phát hiện ra aglitter, vì vậy nó nên lấy vàng và sau đó trở về nhà.

Lưu ý rằng trong mỗi trường hợp mà đại lý rút ra kết luận từ thông tin có sẵn, kết luận đó được đảm bảo là chính xác nếu thông tin có sẵn là chính xác. Đây là một thuộc tính cơ bản của lý luận logic. Trong phần còn lại của chương này, chúng tôi mô tả cách xây dựng các tác nhân logic có thể đại diện cho thông tin và rút ra kết luận như những gì được mô tả trong các đoạn trước.

7.3 Logic

Phần này tóm tắt các khái niệm cơ bản về biểu diễn logic và suy luận. Những ý tưởng đẹp đẽ này độc lập với bất kỳ hình thức cụ thể nào của logic. Do đó, chúng tôi hoãn các chi tiết kỹ thuật của các hình thức đó cho đến phần tiếp theo, thay vào đó sử dụng ví dụ quen thuộc về số học thông thường.

Cú pháp

Trong Phần 7.1, chúng tôi đã nói rằng cơ sở kiến thức bao gồm các câu. Các câu này được biểu thị theo cú pháp của ngôn ngữ biểu diễn, chỉ định tất cả các các câu được hình thành tốt. Khái niệm cú pháp đủ rõ ràng trong số học thông thường: " $x + y = 4$ " là một câu được định dạng tốt, trong khi " $x4y + =$ " thì không.

Ngữ nghĩa học

Sự thật thế giới có thể

Một logic cũng phải xác định ngữ nghĩa, hoặc ý nghĩa, của câu. Các ngữ nghĩa xác định sự thật của mỗi câu liên quan đến mỗi thế giới có thể xảy ra. Ví dụ, ngữ nghĩa Đối với số học chỉ định rằng câu " $x + y = 4$ " là đúng trong một thế giới mà x là 2 và y là 2, nhưng sai trong một thế giới mà x là 1 và y là 1. Trong logic tiêu chuẩn, mỗi câu phải đúng hoặc sai trong mỗi thế giới có thể - không có "ở giữa". 2

Mẫu

Khi chúng ta cần chính xác, chúng ta sử dụng thuật ngữ mô hình thay cho "thế giới có thể". Trong khi các thế giới có thể được coi là môi trường thực tế (tiềm năng) mà tác nhân có thể có hoặc không có thể ở trong đó, các mô hình là trừu tượng toán học, mỗi mô hình có giá trị cố định (đúng hoặc sai) cho mỗi câu có liên quan. Một cách thân mật, chúng ta có thể nghĩ về một thế giới khả dĩ, ví dụ, có x đàn ông và y phụ nữ ngồi ở bàn chơi cầu, và câu $x + y = 4$ là đúng khi có tổng cộng bốn người. Về mặt chính thức, các mô hình có thể chỉ là tất cả các gán có thể có của các số nguyên không âm cho các biến x và y . Mỗi gán như vậy xác định tính đúng của bất kỳ câu số học nào có biến là x và y . Nếu câu α đúng trong mô hình m , chúng ta nói rằng m thỏa mãn α hoặc đôi khi m là mô hình của α .

Hài lòng

Chúng tôi sử dụng ký hiệu $M(\alpha)$ để chỉ tập hợp tất cả các mô hình α . Bây giờ chúng ta đã có một khái niệm về sự thật, chúng ta đã sẵn sàng để nói về lý luận logic. Điều này liên quan đến mối quan hệ logic kéo theo giữa các câu – ý tưởng rằng một câu theo sau logic từ một câu khác. Trong ký hiệu toán học, chúng ta viết

Bao gồm

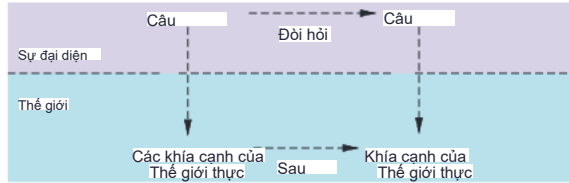
$$\alpha \models \beta$$

có nghĩa là câu α đòi hỏi câu β . Định nghĩa chính thức của bao gồm là: $\alpha \models \beta$ nếu và chỉ khi, trong mọi mô hình mà α là đúng, β cũng đúng. Sử dụng ký hiệu vừa được giới thiệu, chúng ta có thể viết

$\alpha \models \beta$ nếu và chỉ khi $M(\alpha) \subseteq M(\beta)$. (Lưu ý hướng của $\subseteq$ ở đây: nếu $\alpha \models \beta$, thì α là một khẳng định mạnh mẽ hơn β : nó loại trừ nhiều thế giới có thể xảy ra.) Mối quan hệ của sự bao gồm quen thuộc từ số học; Chúng tôi hài lòng với ý tưởng rằng câu $x = 0$ đòi hỏi câu $xy = 0$. Rõ ràng, trong bất kỳ mô hình nào mà x bằng không, đó là trường hợp xy bằng không (bất kể giá trị của y).

Chúng ta có thể áp dụng loại phân tích tương tự cho ví dụ lý luận thế giới wumpus được đưa ra trong phần trước. Hãy xem xét tình huống trong Hình 7.3 (b): tác nhân không phát hiện ra gì trong [1,1] và một làn gió trong [2,1]. Những nhận thức này, kết hợp với kiến thức của tác nhân về các quy tắc của thế giới wumpus, tạo thành KB. Tác nhân quan tâm đến việc liệu các ô vuông liền kề [1,2], [2,2] và [3,1] có chứa các hồ hay không. Mỗi ô trong số ba ô vuông có thể hoặc có thể

2 Logic mờ, được thảo luận trong Chương 13, cho phép các mức độ chân lý.



Hình 7.6 Câu là các cấu hình vật lý của tác nhân, và lý luận là một quá trình xây dựng các cấu hình mới đại diện cho các khía cạnh của thế giới thực sự theo sau các khía cạnh mà các cấu hình cũ đại diện.

Khi hiểu được sự bao gồm và suy luận, có thể hữu ích khi nghĩ về tập hợp tất cả các hậu quả của KB như một đồng cỏ khô và của α như một cây kim. Kéo theo giống như cây kim đang ở trong đồng cỏ khô; suy luận giống như tìm thấy nó. Sự khác biệt này được thể hiện trong một số ký hiệu chính thức: nếu thuật toán suy luận i có thể lấy α từ KB, chúng tôi viết

KB "i α ", được phát âm là " α có nguồn gốc từ KB bởi i" hoặc "i bắt nguồn từ KB α từ KB."

Âm thanh

Giữ gìn sự thật

Một thuật toán suy luận chỉ dẫn đến các câu được gọi là âm thanh hoặc sự thật-Bảo tồn. Độ lành mạnh là một tài sản rất mong muốn. Một thủ tục suy luận không hợp lý es-tạo ra mọi thứ một cách hữu ích khi nó diễn ra — nó thông báo việc phát hiện ra những chiếc kim không tồn tại. Có thể dễ dàng nhận thấy rằng việc kiểm tra mô hình, khi nó có thể áp dụng, là một quy trình hợp lý.

Đầy đủ

Thuộc tính của tính hoàn chỉnh cũng được mong muốn: một thuật toán suy luận là hoàn chỉnh nếu nó có thể rút ra bất kỳ câu nào kèm theo. Đối với những đồng cỏ khô thực sự, có phạm vi hữu hạn, dường như rõ ràng là một cuộc kiểm tra có hệ thống luôn có thể quyết định xem kim có nằm trong đồng cỏ hay không. Tuy nhiên, đối với nhiều cơ sở tri thức, đồng cỏ khô của các hậu quả là vô hạn, và tính đầy đủ trở thành một vấn đề quan trọng.⁶ May mắn thay, có các quy trình suy luận hoàn chỉnh cho các logic đủ biểu hiện để xử lý nhiều cơ sở tri thức.

Chúng tôi đã mô tả một quá trình lý luận mà các kết luận được đảm bảo là đúng trong bất kỳ thế giới nào mà các tiền đề là đúng; đặc biệt, nếu KB đúng trong thế giới thực, thì bất kỳ câu nào α xuất phát từ KB bằng một quy trình suy luận hợp lý cũng đúng trong thế giới thực. Như vậy

Trong khi một quá trình suy luận hoạt động trên "cú pháp" - các cấu hình vật lý bên trong như các bit trong thanh ghi hoặc các mẫu nhấp nháy điện trong não - quá trình tương ứng với mỗi quan hệ trong thế giới thực, theo đó một số khía cạnh của thế giới thực là trường hợp nhờ các khía cạnh khác của thế giới thực là trường hợp.⁷ Sự tương ứng giữa thế giới và biểu diễn này được minh họa trong Hình 7.6.

Nền tảng

Vấn đề cuối cùng cần xem xét là nền tảng - mối liên hệ giữa lý luận logic processes và môi trường thực tế mà tác nhân tồn tại. Đặc biệt, làm thế nào để chúng ta biết điều đó

Tôi

5 Kiểm tra mô hình hoạt động nếu không gian của mô hình là hữu hạn - ví dụ, trong thế giới wumpus có kích thước cố định. Mặt khác, đối với số học, không gian của các mô hình là vô hạn: ngay cả khi chúng ta giới hạn bản thân trong các số nguyên, vẫn có vô số cặp giá trị cho x và y trong câu $x + y = 4.6$ So sánh với trường hợp của không gian tìm kiếm vô hạn trong Chương 3, trong đó tìm kiếm độ sâu đầu tiên là không hoàn chỉnh. Như Wittgenstein (1922) đã nói trong cuốn Tractatus nổi tiếng của ông: "Thế giới là tất cả mọi thứ."

KB có đúng trong thế giới thực? (Rốt cuộc, KB chỉ là "cú pháp" bên trong đầu tác nhân.) Đây là một câu hỏi triết học mà rất nhiều, rất nhiều cuốn sách đã được viết. (Xem Chương 28.) Một câu trả lời đơn giản là các cảm biến của tác nhân tạo ra kết nối. Ví dụ, tác nhân thế giới wumpus của chúng tôi có một cảm biến mùi. Chương trình tác nhân tạo ra một câu phủ hợp bất cứ khi nào có mùi. Sau đó, bất cứ khi nào câu đó nằm trong cơ sở tri thức, nó là đúng trong thế giới thực. Do đó, ý nghĩa và sự thật của các câu tri giác được xác định bởi các quá trình cảm nhận và xây dựng câu tạo ra chúng. Còn phần còn lại của kiến thức của tác nhân, chẳng hạn như niềm tin của nó rằng wumpuses gây ra mùi ở các ô vuông liền kề thì sao? Đây không phải là một sự phản đối trực tiếp của một nhận thức duy nhất, mà là một quy tắc chung - có lẽ bắt nguồn từ kinh nghiệm tri giác nhưng không giống với một tuyên bố về kinh nghiệm đó. Các quy tắc chung như thế này được tạo ra bởi một quá trình xây dựng câu được gọi là học tập, là chủ đề của Phần V. Học tập là sai lầm. Có thể là trường hợp wumpuses gây ra mùi ngoại trừ vào ngày 29 tháng 2 trong những năm nhuận, đó là khi họ tắm. Do đó, KB có thể không đúng trong thế giới thực, nhưng với quy trình học tập tốt, có lý do để lạc quan.

7.4 Logic mệnh đề: Một logic rất đơn giản

Bây giờ chúng ta trình bày logic mệnh đề. Chúng tôi mô tả cú pháp của nó (cấu trúc của câu) và ngữ nghĩa của nó (cách xác định tính đúng đắn của câu). Từ những điều này, chúng tôi rút ra một thuật toán cú pháp đơn giản để suy luận logic thực hiện khái niệm ngữ nghĩa của. Tất nhiên, mọi thứ đều diễn ra trong thế giới wumpus.

Logic mệnh đề

7.4.1 Cú pháp

Cú pháp của logic mệnh đề xác định các câu được phép. Các câu nguyên tử bao gồm một biểu tượng mệnh đề duy nhất. Mỗi biểu tượng như vậy đại diện cho một mệnh đề có thể đúng hay sai. Chúng tôi sử dụng các ký hiệu bắt đầu bằng chữ hoa và có thể chứa các chữ cái hoặc chỉ số dưới khác, ví dụ: P, Q, R, W1,3 và FacingEast. Các tên gọi là tùy ý nhưng thường được chọn để có một số giá trị ghi nhớ — chúng tôi sử dụng W1,3 để đại diện cho mệnh đề rằng wumpus nằm trong [1,3]. (Hãy nhớ rằng các ký hiệu như W1,3 là nguyên tử, tức là W, 1 và 3 không phải là phần có ý nghĩa của ký hiệu.) Có hai biểu tượng mệnh đề với ý nghĩa cố định: Đúng là mệnh đề luôn đúng và Sai là mệnh đề luôn sai. Các câu phức tạp được xây dựng từ các câu đơn giản hơn, sử dụng dấu ngoặc đơn và toán tử

Câu nguyên tử
Biểu tượng đề xuất

được gọi là liên kết logic. Có năm liên kết được sử dụng phổ biến:

Câu phức tạp
Liên kết logic

- ¬ (không). Một câu như $\neg W1,3$ được gọi là phủ định của W1,3. Một nghĩa đen là một câu nguyên tử (nghĩa đen dương) hoặc câu nguyên tử phủ định (nghĩa đen phủ định). ∧ (và). Một câu có liên kết chính là ∧, chẳng hạn như $W1,3 \wedge P3,1$, được gọi là liên kết-

Phủ định
Chữ

Tion; Các bộ phận của nó là các liên kết. (Chữ ∧ trông giống như chữ "A" cho "And.")

Liên từ

- ∨ (hoặc). Một câu có liên kết chính là ∨, chẳng hạn như $(W1,3 \wedge P3,1) \vee W2,2$, là một tách rời Tion; các bộ phận của nó là tách rời—trong ví dụ này, $(W1,3 \wedge P3,1)$ và $W2,2$.

Tách rời

- ⇒ (ngụ ý). Một câu như $(W1,3 \wedge P3,1) \Rightarrow \neg W2,2$ được gọi là hàm ý (hoặc conditional). Tiền đề hoặc tiền đề của nó là $(W1,3 \wedge P3,1)$, và kết luận hoặc hậu quả của nó là $\neg W2,2$. Ý nghĩa còn được gọi là quy tắc hoặc câu lệnh nếu-thì. Hàm ý

Ngụ ý
Tiền đề kết luận

- biểu tượng đôi khi được viết trong các sách khác là $\supset$ hoặc $\rightarrow$. ⇔ (nếu và chỉ nếu). Câu $W1,3 \Leftrightarrow \neg W2,2$ là một câu hai điều kiện.

Quy tắc
Hai điều kiện

Câu $\rightarrow$ AtomicSentence |
 ComplexSentence AtomicSentence $\rightarrow$ đúng | Sai | P |
 Hỏi | R | . . . ComplexSentence $\rightarrow$ (Câu)
 $\neg$ Câu | Câu $\wedge$ câu | Câu $\vee$
 câu | Câu $\Rightarrow$ câu | Câu $\Leftrightarrow$
 câu

U'U TIỀN TOÁN TỬ : $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Hình 7.7 Ngữ pháp BNF (Backus-Naur Form) của các câu trong logic mệnh đề, cùng với các ưu tiên toán tử, từ cao nhất đến thấp nhất.

Hình 7.7 đưa ra một ngữ pháp chính thức của logic mệnh đề. (Ký hiệu BNF được giải thích ở trang 1081.) Ngữ pháp BNF được tăng cường với danh sách ưu tiên toán tử để loại bỏ am-biguity khi nhiều toán tử được sử dụng. Toán tử "không" ($\neg$) có mức độ ưu tiên cao nhất, có nghĩa là trong câu $\neg A \wedge B$, $\neg$ liên kết chặt chẽ nhất, cho chúng ta tương đương với $(\neg A) \wedge B$ chứ không phải $\neg(A \wedge B)$. (Ký hiệu cho số học thông thường là giống nhau: $-2 + 4$ là 2, không phải -6 .) Khi thích hợp, chúng tôi cũng sử dụng dấu ngoặc đơn và dấu ngoặc vuông để làm rõ cấu trúc câu dự định và cải thiện khả năng đọc.

7.4.2 Ngữ nghĩa

Sau khi xác định cú pháp của logic mệnh đề, bây giờ chúng ta xác định ngữ nghĩa của nó. Semantics xác định các quy tắc để xác định sự thật của một câu đối với một mô hình cụ thể. Trong logic mệnh đề, một mô hình chỉ đơn giản đặt giá trị chân lý - đúng hoặc sai - cho mọi

biểu tượng đề xuất. Ví dụ, nếu các câu trong cơ sở tri thức sử dụng các ký hiệu mệnh đề P1,2, P2,2 và P3,1, thì một mô hình có thể là

$m1 = \{P1,2 = \text{sai}, P2,2 = \text{sai}, P3,1 = \text{đúng}\}$. Với ba ký hiệu mệnh đề, có $2^3 = 8$ mô hình khả thi - chính xác những mô hình được mô tả trong Hình 7.5. Tuy nhiên, hãy lưu ý rằng các mô hình là các đối tượng toán học thuần túy không có mối liên hệ cần thiết với thế giới wumpus. P1,2 chỉ là một biểu tượng; nó có thể có nghĩa là "có một pitin [1,2]" hoặc "Tôi đang ở Paris hôm nay và ngày mai."

Các ngữ nghĩa cho logic mệnh đề phải xác định cách tính giá trị chân lý của bất kỳ câu nào, đưa ra một mô hình. Điều này được thực hiện đệ quy. Tất cả các câu được xây dựng từ các câu nguyên tử và năm từ liên kết; Do đó, chúng ta cần xác định cách tính toán sự thật của các câu nguyên tử và cách tính toán sự thật của các câu được hình thành với mỗi trong số năm liên kết. Câu nguyên tử rất dễ:

- Đúng là đúng trong mọi mô hình và Sai là sai trong mọi mô hình.
- Giá trị chân lý của mọi biểu tượng mệnh đề khác phải được xác định trực tiếp trong mô hình. Ví dụ, trong mô hình m1 đưa ra trước đó, P1,2 là sai.

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
saisai		thật	saisai		đúngđúng	
sai	đúngđúng		sai	đúngđúng		sai
thật	saisai		sai	thật	saisai	
thật	thật	sai	thật	thật	thật	thật

Hình 7.8 Bảng sự thật cho năm liên kết logic. Ví dụ, để sử dụng bảng để tính toán giá trị của $P \vee Q$ khi P là đúng và Q là sai, trước tiên hãy nhìn ở bên trái cho hàng trong đó P là đúng và Q là sai (hàng thứ ba). Sau đó, nhìn vào hàng đó dưới cột $P \vee Q$ để xem kết quả: true.

Đối với các câu phức tạp, chúng ta có năm quy tắc, áp dụng cho bất kỳ câu con nào P và Q (phức nguyên tử) trong bất kỳ mô hình m nào (ở đây "iff" có nghĩa là "nếu và chỉ nếu"):

- $\neg P$ là đúng nếu P là sai tính bằng m .
- $P \wedge Q$ là đúng nếu cả P và Q đều đúng trong m .
- $P \vee Q$ là đúng nếu P hoặc Q là đúng tính bằng m .
- $P \Rightarrow Q$ là đúng trừ khi P là đúng và Q là sai trong m .
- $P \Leftrightarrow Q$ là đúng nếu P và Q đều đúng hoặc cả hai đều sai tính bằng m . Các quy tắc cũng có thể được biểu thị bằng các bảng sự thật chỉ định giá trị chân lý của một phức

Bảng sự thật

câu cho mỗi lần gán giá trị chân lý có thể cho các thành phần của nó. Bảng sự thật cho năm liên kết được đưa ra trong Hình 7.8. Từ các bảng này, giá trị chân lý của bất kỳ câu nào có thể được tính toán đối với bất kỳ mô hình m nào bằng một đánh giá đệ quy đơn giản. Ví dụ, câu $\neg P_1, 2 \wedge (P_2, 2 \vee P_3, 1)$, được đánh giá bằng m_1 , cho true $\wedge$ (false $\vee$ true) = true $\wedge$ true = true. Bài tập 7.TRUV yêu cầu bạn viết thuật toán PL-TRUE? (s, m), tính giá trị chân lý của một câu logic mệnh đề s trong một mô hình m .

Các bảng sự thật cho "và", "hoặc" và "không" gần giống với trực giác của chúng ta về các từ tiếng Anh. Điểm chính của sự nhầm lẫn có thể xảy ra là $P \vee Q$ là đúng khi P là đúng hoặc Q là đúng hoặc cả hai. Một liên kết khác, được gọi là "độc quyền hoặc" (viết tắt là xor⁸), tạo ra sai khi cả hai từ tách biệt đều đúng. Không có sự đồng thuận về biểu tượng cho độc quyền hoặc; Một số lựa chọn là $\vee$ hoặc $\oplus$ hoặc $\oplus$.

Bảng sự thật cho $\Rightarrow$ có thể không hoàn toàn phù hợp với sự hiểu biết trực giác của một người về "P ngụ ý Q" hoặc "nếu P thì Q". Đối với một điều, logic mệnh đề không yêu cầu bất kỳ mối quan hệ nhân quả hoặc liên quan nào giữa P và Q . Câu "5 is odd implies Tokyo is the capital of Japan" là một câu đúng của logic mệnh đề (theo cách giải thích thông thường), mặc dù nó là một câu rõ ràng kỳ quặc của tiếng Anh. Một điểm nhầm lẫn khác là bất kỳ hàm ý nào cũng đúng bất cứ khi nào tiền đề của nó là sai. Ví dụ, "5 thậm chí còn ngụ ý Sam thông minh" là đúng, bất kể Sam có thông minh hay không. Điều này có vẻ kỳ lạ, nhưng nó có ý nghĩa nếu bạn nghĩ về "P $\Rightarrow$ Q" như nói, "Nếu P là đúng, thì tôi đang khẳng định rằng Q là đúng; nếu không thì tôi sẽ không đưa ra yêu cầu nào." Cách duy nhất để câu này là sai là nếu P là đúng nhưng Q là sai.

Hai điều kiện, $P \Leftrightarrow Q$, là đúng bất cứ khi nào cả $P \Rightarrow Q$ và $Q \Rightarrow P$ đều đúng. Trong tiếng Anh, điều này thường được viết là "P if and only if Q." Nhiều quy tắc của thế giới wumpus là tốt

⁸ Tiếng Latinh sử dụng hai từ riêng biệt: "vel" là bao gồm hoặc và "aut" là loại trừ hoặc.

được viết bằng $\Leftrightarrow$. Ví dụ, một hình vuông có gió nếu một hình vuông lân cận có một hồ và hình vuông chỉ có gió nếu một hình vuông lân cận có một hồ. Vì vậy, chúng ta cần một hai điều kiện,

$B1,1 \Leftrightarrow (P1,2 \vee P2,1)$, trong đó $B1,1$ có

nghĩa là có một làn gió trong $[1,1]$.

7.4.3 Cơ sở kiến thức đơn giản

Bây giờ chúng ta đã xác định ngữ nghĩa cho logic mệnh đề, chúng ta có thể xây dựng một cơ sở tri thức cho thế giới wumpus. Trước tiên, chúng tôi tập trung vào các khía cạnh bất biến của thế giới wumpus, để lại các khía cạnh có thể thay đổi cho phần sau. Bây giờ, chúng ta cần các ký hiệu sau cho mỗi vị trí $[x, y]$:

Px,y là đúng nếu có một hồ trong $[x, y]$. Wx, y đúng nếu có một wumpus trong $[x, y]$, chết hay sống.

Bx,y đúng nếu có gió trong $[x, y]$. Sx,y đúng nếu có mùi hôi thối trong $[x, y]$. Lx,y

đúng nếu tác nhân ở vị trí $[x, y]$. Các câu chúng ta viết sẽ đủ để suy ra $\neg P1,2$

(không có hồ trong $[1,2]$), như đã được thực hiện một cách không chính thức trong Phần 7.3. Chúng ta dán nhãn cho mỗi câu R_i để chúng ta có thể tham khảo chúng:

- Không có hồ trong $[1,1]: R1: \neg P1,1$.
- Một hình vuông có gió mát nếu và chỉ khi có một hồ trong một quảng trường lân cận. Điều này phải được nêu cho mỗi ô vuông; bây giờ, chúng tôi chỉ bao gồm các ô vuông có liên quan: $R2: B1,1 \Leftrightarrow (P1,2 \vee P2,1)$. $R3: B2,1 \Leftrightarrow (P1,1 \vee P2,2 \vee P3,1)$.
- Những câu trước là đúng trong tất cả các thế giới wumpus. Bây giờ chúng ta bao gồm các breezepercepts cho hai ô vuông đầu tiên được truy cập trong thế giới cụ thể mà tác nhân đang ở, dẫn đến tình huống trong Hình 7.3 (b). $R4: \neg B1,1$. $R5: B2,1$.

7.4.4 Quy trình suy luận đơn giản

Mục tiêu của chúng ta bây giờ là quyết định xem $KB \models \alpha$ cho một câu nào đó α hay không. Ví dụ, $\neg P1,2$ có được bao gồm bởi KB của chúng ta không? Thuật toán đầu tiên của chúng tôi để suy luận là một cách tiếp cận kiểm tra mô hình thực hiện trực tiếp định nghĩa về sự bao gồm: liệt kê các mô hình và kiểm tra xem α có đúng trong mọi mô hình trong đó KB là đúng hay không. Các mô hình là gần đúng hoặc sai cho mọi biểu tượng mệnh đề. Quay trở lại ví dụ thế giới wumpus của chúng ta, các ký hiệu mệnh đề có liên quan là $B1,1, B2,1, P1,1, P1,2, P2,1, P2,2$ và $P3,1$. Với bảy ký hiệu, có $2^7 = 128$ mô hình có thể; trong ba trong số này, KB là đúng (Hình 7.9). Trong ba mô hình đó, $\neg P1,2$ là đúng, do đó không có hồ trong $[1,2]$. Mặt khác, $P2,2$ là đúng trong hai trong số ba mô hình và sai trong một, vì vậy chúng ta chưa thể biết liệu có hồ trong $[2,2]$ hay không.

Hình 7.9 tái tạo ở dạng chính xác hơn lý luận được minh họa trong Hình 7.5. Thuật toán chung để quyết định sự bao gồm trong logic mệnh đề được thể hiện trong Hình 7.10. Giống như thuật toán BACKTRACKING-SEARCH ở trang 176, TT-ENTAILS? thực hiện đệ quy

B1,1	B2,1	P1,1	P1,2	P2,1	P2,2	P3,1	R1	R2	R3	R4	R5	KB
saisai		saisai		saisai		sai	đúng	đúng	đúng	đúng	saisai	
saisai		saisai		saisai		đúng	đúng	thật	sai	thật	saisai	
...	...	...	...	...	...	...	...	...	...	...	...	...
sai	thật	saisai		saisai		sai	đúng	đúng	sai	đúng	đúng	sai
sai	thật	saisai		saisai		đúng	đúng	đúng	đúng	đúng	đúng	thật
sai	thật	saisai		sai	thật	sai	đúng	đúng	đúng	đúng	đúng	đúng
sai	thật	saisai		sai	đúng	đúng	đúng	đúng	đúng	đúng	đúng	đúng
sai	thật	saisai		thật	saisai		thật	saisai		đúng	đúng	sai
...	...	...	...	...	...	...	...	...	...	...	...	...
đúng	đúng	đúng	đúng	đúng	đúng	thật	sai	đúng	đúng	sai	thật	sai

Hình 7.9 Một bảng sự thật được xây dựng cho cơ sở tri thức được đưa ra trong văn bản. KB là đúng nếu R1 đến R5 là đúng, chỉ xảy ra ở 3 trong số 128 hàng (những hàng được gạch chân ở cột bên phải). Trong cả 3 hàng, P1,2 là sai, vì vậy không có hồ trong [1,2]. Mặt khác, có thể có (hoặc có thể không) có một cái hồ trong [2,2].

chức năng TT-ENTAILS? (KB, α) trả về true hoặc false

đầu vào: KB, cơ sở tri thức, một câu trong logic mệnh đề

α , truy vấn, một câu trong logic mệnh đề

biểu tượng $\leftarrow$ danh sách các ký hiệu mệnh đề bằng KB và areturn TT-CHECK-ALL(KB, α , ký hiệu, { })

hàm TT-CHECK-ALL(KB, α , ký hiệu, mô hình) trả về true hoặc false

nếu RÖNG? (ký hiệu) thì

nếu PL-TRUE? (KB, model) sau đó trả về PL-TRUE? (α , model)else return true// khi KB là false, luôn trả về trueelse

$P \leftarrow \text{FIRST}(\text{ký hiệu})$ nghe $\leftarrow \text{REST}(\text{ký hiệu})$ trả về (TT-CHECK-ALL(KB, α , REST, model $\cup \{P = \text{true}\}$)

vàTT-CHECK-ALL(KB, α , phần còn lại, mô hình $\cup \{P = \text{sai}\}$)

Hình 7.10 Một thuật toán liệt kê bảng sự thật để quyết định liên quan đến mệnh đề.

(TT là viết tắt của bảng sự thật.) PL-ĐÚNG? trả về true nếu một câu giữ trong một mô hình. Mô hình biến đại diện cho một mô hình một phần — một gán cho một số ký hiệu. Từ khóa và ở đây là ký hiệu hàm tiền tố trong ngôn ngữ lập trình giả mã, không phải toán tử trong logic mệnh đề; nó nhận hai đối số và trả về true hoặc false.

liệt kê một không gian hữu hạn của các gán cho các ký hiệu. Thuật toán hợp lý vì nó thực hiện trực tiếp định nghĩa của entailment, và hoàn chỉnh vì nó hoạt động cho bất kỳ KBand nào α và luôn chấm dứt — chỉ có rất nhiều mô hình để kiểm tra.

Tất nhiên, "hữu hạn nhiều" không phải lúc nào cũng giống với "ít". Nếu KB và α chứa n ký hiệu trong tất cả, thì có $2n$ mô hình. Do đó, độ phức tạp thời gian của thuật toán là $O(2n)$. (Độ phức tạp không gian chỉ là $O(n)$ vì bảng liệt kê là độ sâu đầu tiên.) Sau đó trong chương này, chúng tôi chỉ ra các thuật toán hiệu quả hơn nhiều trong nhiều trường hợp. Thật không may, propositional entailment là co-NP-complete (tức là có lẽ không dễ hơn NP-complete — xem Phụ lục A), vì vậy mọi thuật toán suy luận đã biết cho logic mệnh đề đều có độ phức tạp trong trường hợp xấu nhất mà là theo cấp số nhân về kích thước của đầu vào.

7.5 Chứng minh định lý mệnh đề

Cho đến nay, chúng tôi đã chỉ ra cách xác định sự bao gồm bằng cách kiểm tra mô hình: liệt kê các mô hình và chỉ ra rằng câu phải giữ trong tất cả các mô hình. Trong phần này, chúng tôi chỉ ra cách entailment có thể được thực hiện bằng cách chứng minh định lý - áp dụng các quy tắc suy luận trực tiếp vào các câu

trong cơ sở kiến thức của chúng tôi để xây dựng một chứng minh của câu mong muốn mà không cần tham khảo các mô hình. Nếu số lượng mô hình lớn nhưng độ dài của chứng minh ngắn, thì chứng minh định lý có thể hiệu quả hơn kiểm tra mô hình.

Đôi khi chúng ta đi sâu vào chi tiết của các thuật toán chứng minh định lý, chúng ta sẽ cần một số khái niệm bổ sung liên quan đến sự bao gồm. Khái niệm đầu tiên là tương đương logic: hai sent-các α và β tương đương về mặt logic nếu chúng đúng trong cùng một tập hợp các mô hình. Chúng tôi viết điều này như $\alpha \equiv \beta$. (Lưu ý rằng $\equiv$ được sử dụng để đưa ra tuyên bố về câu, trong khi $\Leftrightarrow$ được sử dụng như một phần của câu.) Ví dụ, chúng ta có thể dễ dàng chỉ ra (sử dụng bảng sự thật) rằng $P \wedge Q$ và $Q \wedge P$ tương đương về mặt logic; các tương đương khác được thể hiện trong Hình 7.11. Những tương đương này đóng vai trò tương tự trong logic như các bản sắc số học trong toán học thông thường. Một định nghĩa xen kẽ về sự tương đương như sau: bất kỳ hai câu nào α và β đều tương đương nếu và chỉ khi mỗi câu trong số chúng đòi hỏi câu kia:

$\text{một} \equiv \text{b}$ nếu và chỉ khi $\alpha \models \beta$ và $\beta \models \alpha$.

Khái niệm thứ hai chúng ta sẽ cần là tính hợp lệ. Một câu có giá trị nếu nó đúng trong tất cả các mô hình. Cho

ví dụ, câu $P \vee \neg P$ là hợp lệ. Các câu hợp lệ còn được gọi là tautologies—chúng nhất thiết phải đúng. Bởi vì câu True là đúng trong tất cả các mô hình, mọi câu hợp lệ về mặt logic tương đương với True. Câu hợp lệ có ích gì? Từ định nghĩa của chúng ta về entailment, chúng ta có thể rút ra định lý suy luận, được người Hy Lạp cổ đại biết đến:

Đối với bất kỳ câu nào α và β , $\alpha \models \beta$ nếu và chỉ khi câu $(\alpha \Rightarrow \beta)$ hợp lệ. (Bài tập 7.DEDU yêu cầu một bằng chứng.) Do đó, chúng ta có thể quyết định xem $\alpha \models \beta$ có hay không bằng cách kiểm tra rằng $(\alpha \Rightarrow \beta)$ là đúng trong mọi mô hình - về cơ bản là những gì thuật toán suy luận trong Hình 7.10 làm - hoặc bằng cách chứng minh rằng $(\alpha \Rightarrow \beta)$ tương đương với True. Ngược lại, định lý suy luận rằng mọi câu ngụ ý hợp lệ đều mô tả một suy luận hợp pháp.

Khái niệm cuối cùng chúng ta sẽ cần là sự thỏa mãn. Một câu là thỏa mãn nếu nó đúng trong, hoặc hài lòng bởi, một số mô hình. Ví dụ, cơ sở kiến thức được đưa ra trước đó, $(R1 \wedge R2 \wedge R3 \wedge R4 \wedge R5)$, có thể thỏa mãn vì có ba mô hình trong đó nó đúng, như thể hiện trong Hình 7.9. Khả năng thỏa mãn có thể được kiểm tra bằng cách liệt kê các mô hình có thể cho đến khi tìm thấy một mô hình thỏa mãn câu. Vấn đề xác định mức độ thỏa mãn của câu

Chứng minh định lý

Tương đương logic

Hiệu lực

Tautology

Định lý
suy diễn
I

Khả năng thỏa mãn

$(a \wedge b) \equiv (b \wedge a)$ tính giao hoán của $\wedge$ $(a \vee b)$
 $\equiv (b \vee a)$ tính giao hoán của $\vee$ $((a \wedge b) \wedge c) \equiv$
 $(a \wedge (b \wedge c))$ liên kết của $\wedge$ $((a \vee b) \vee c) \equiv (a \vee$
 $(b \vee c))$ liên kết của $\vee$

$\neg(\neg a) \equiv$ một loại bỏ phủ định kép
 $(a \Rightarrow b) \equiv (\neg b \Rightarrow \neg a)$ đối lập $(a \Rightarrow b) \equiv (\neg a \vee b)$ loại bỏ hàm
 y $(a \Leftrightarrow b) \equiv (a \Rightarrow b) \wedge (b \Rightarrow a)$ loại bỏ hai điều kiện

$\neg(a \wedge b) \equiv (\neg a \vee \neg b)$ De Morgan $\neg(a \vee b) \equiv (\neg a \wedge \neg b)$ De Morgan $(a \wedge (b \vee$
 $c)) \equiv ((a \wedge b) \vee (a \wedge c))$ phân phối của $\wedge$ trên $\vee$ $(a \vee (b \wedge c)) \equiv ((a \vee b) \wedge (a \vee$
 $y))$ phân phối của $\vee$ trên $\wedge$ Hình 7.11 Tương đương logic tiêu chuẩn. Các ký
 hiệu α , β và γ là viết tắt của các câu tùy tiện của logic mệnh đề.

trong logic mệnh đề - bài toán SAT - là vấn đề đầu tiên được chứng minh là NP-complete. Nhiều vấn đề trong khoa học máy tính thực sự là vấn đề thỏa mãn. Ví dụ, tất cả các vấn đề thỏa mãn ràng buộc trong Chương 5 hỏi liệu các ràng buộc có thỏa mãn được bằng một số nhiệm vụ hay không.

Tính hợp lệ và khả năng thỏa mãn tất nhiên có mối liên hệ với nhau: α là hợp lệ nếu $\neg\alpha$ không thỏa mãn; Về mặt tích cực, α thỏa mãn nếu $\neg\alpha$ không hợp lệ. Chúng tôi cũng có kết quả hữu ích sau:

$\alpha \models \beta$ nếu và chỉ khi câu $(\alpha \wedge \neg\beta)$ không thỏa mãn. Chứng minh β từ α bằng cách kiểm tra tính không thỏa mãn của $(\alpha \wedge \neg\beta)$ tương ứng chính xác với kỹ thuật chứng minh toán học tiêu chuẩn của reductio ad absurdum (nghĩa đen, "rút gọn thành một điều vô lý"). Nó còn được gọi là chứng minh bằng cách bác bỏ hoặc chứng minh bằng mâu thuẫn. Người ta giả định một câu β là sai và cho thấy rằng điều này dẫn đến mâu thuẫn với các tiên đề đã biết α . Mâu thuẫn này chính xác là những gì có nghĩa khi nói rằng câu $(\alpha \wedge \neg\beta)$ là không thỏa mãn.

NGÔI

Reductio ad absurdum

Bác bỏ Mâu thuẫn

7.5.1 Suy luận và chứng minh

Phần này bao gồm các quy tắc suy luận có thể được áp dụng để rút ra một bằng chứng - một chuỗi kết luận-sions dẫn đến mục tiêu mong muốn. Quy tắc nổi tiếng nhất được gọi là Modus Ponens (tiếng Latinh có nghĩa là chế độ khẳng định) và được viết

$\alpha \Rightarrow \beta, \alpha \vdash \beta$ Ký hiệu có nghĩa là, bất cứ khi nào bất kỳ câu nào có dạng $\alpha \Rightarrow \beta$ và α được đưa ra, thì câu β có thể được suy ra. Ví dụ: nếu $(WumpusAhead \wedge WumpusAlive) \Rightarrow Shoot$ and $(WumpusAhead \wedge WumpusAlive)$ được đưa ra, thì có thể suy ra Shoot.

Quy tắc suy luận Bằng chứng

Phương thức Ponens

Một quy tắc suy luận hữu ích khác là And-Elimination, nói rằng, từ một liên từ, Bất kỳ liên kết nào cũng có thể được suy ra:

Và loại bỏ

$\frac{\text{một } \wedge b}{\text{Một}}$

Ví dụ: từ $(WumpusAhead \wedge WumpusAlive)$, $WumpusAlive$ có thể được suy ra.

Bằng cách xem xét các giá trị chân lý có thể có của α và β , người ta có thể dễ dàng chứng minh một lần và mãi mãi rằng Modus Ponens và And-Elimination là hợp lý. Các quy tắc này sau đó có thể được sử dụng trong bất kỳ trường hợp cụ thể nào mà chúng được áp dụng, tạo ra các suy luận hợp lý mà không cần liệt kê các mô hình.

Tất cả các tương đương logic trong Hình 7.11 có thể được sử dụng làm quy tắc suy luận. Ví dụ, tương đương với loại bỏ hai điều kiện mang lại hai quy tắc suy luận

$$\frac{a \Leftrightarrow b \quad (a \Rightarrow b) \wedge (b \Rightarrow a)}{\Rightarrow a} \quad \text{và} \quad \frac{(a \Rightarrow b) \wedge (b \Rightarrow a)}{\text{Một} \Leftrightarrow B}.$$

Không phải tất cả các quy tắc suy luận đều hoạt động theo cả hai hướng như thế này. Ví dụ: chúng ta không thể chạy Modus Ponens theo hướng ngược lại để lấy $\alpha \Rightarrow \beta$ và α từ β .

Chúng ta hãy xem các quy tắc suy luận và tương đương này có thể được sử dụng như thế nào trong thế giới wumpus. Chúng tôi bắt đầu với cơ sở kiến thức chứa R1 đến R5 và chỉ ra cách chứng minh $\neg P1,2$, nghĩa là không có hồ trong [1,2]:

1. Áp dụng loại bỏ hai điều kiện cho R2 để đạt R6 : $(B1,1) \Rightarrow (P1,2 \vee P2,1) \wedge ((P1,2 \vee P2,1) \Rightarrow B1,1)$.
2. Áp dụng và loại bỏ R6 để đạt được R7 : $((P1,2 \vee P2,1) \Rightarrow B1,1)$.
3. Tương đương logic cho các đối đương cho R8 : $(\neg B1,1 \Rightarrow \neg (P1,2 \vee P2,1))$.
4. Áp dụng Modus Ponens với R8 và tri giác R4 (tức là $\neg B1,1$), để đạt được R9 : $\neg (P1,2 \vee P2,1)$.
5. Áp dụng quy tắc De Morgan, đưa ra kết luận R10 : $\neg P1,2 \wedge \neg P2,1$. Nghĩa là, cả [1,2] và [2,1] đều không chứa một cái hồ. Bất kỳ thuật toán tìm kiếm nào trong Chương 3 đều có thể được sử dụng để tìm một chuỗi các bước tạo thành một chứng minh như thế này. Chúng ta chỉ cần định nghĩa một bài toán chứng minh như sau:

- **TRẠNG THÁI BAN ĐẦU:** cơ sở kiến thức ban đầu.
- **HÀNH ĐỘNG:** tập hợp các hành động bao gồm tất cả các quy tắc suy luận được áp dụng cho tất cả các cảm giác phù hợp với nửa trên của quy tắc suy luận.
- **KẾT QUẢ:** kết quả của một hành động là thêm câu vào nửa dưới của quy tắc suy luận.

• **MỤC TIÊU:** mục tiêu là một trạng thái chứa câu mà chúng ta đang cố gắng chứng minh. Do đó, tìm kiếm bằng chứng là một giải pháp thay thế cho việc liệt kê các mô hình. Trong nhiều trường hợp thực tế, việc tìm kiếm một chứng minh có thể hiệu quả hơn vì bằng chứng có thể bỏ qua các mệnh đề không liên quan, không quan trọng có bao nhiêu trong số chúng. Ví dụ, bằng chứng vừa đưa ra dẫn đến $\neg P1,2 \wedge \neg P2,1$

không đề cập đến các mệnh đề B2,1, P1,1, P2,2 hoặc P3,1. Chúng có thể bị bỏ qua vì mệnh đề mục tiêu, P1,2, chỉ xuất hiện trong câu R2; các mệnh đề khác trong R2 chỉ xuất hiện trong R4 và R2; vì vậy R1, R3 và R5 không liên quan đến chứng minh. Điều tương tự cũng sẽ xảy ra ngay cả khi chúng ta thêm một triệu câu nửa vào cơ sở tri thức; Mặt khác, thuật toán bằng sự thật đơn giản sẽ bị choáng ngợp bởi sự bùng nổ theo cấp số nhân của các mô hình.

Một thuộc tính cuối cùng của các hệ thống logic là tính đơn điệu, nói rằng tập hợp của en-
Câu đuôi chỉ có thể tăng lên khi thông tin được thêm vào cơ sở kiến thức.⁹ Đối với
bất kỳ câu nào α và β ,

Tính đơn điệu

nếu $KB \models \alpha$ thì $KB \wedge \beta \models \alpha$.

Ví dụ, giả sử cơ sở tri thức chứa khẳng định bổ sung β nói rằng có chính xác tám hổ trên thế giới. Kiến thức này có thể giúp tác nhân rút ra các kết luận bổ sung, nhưng nó không thể vô hiệu hóa bất kỳ kết luận nào α đã suy ra - chẳng hạn như kết luận rằng không có hổ trong [1,2]. Tính đơn điệu có nghĩa là các quy tắc suy luận có thể được áp dụng bất cứ khi nào các tiền đề phù hợp được tìm thấy trong cơ sở tri thức - kết luận của quy tắc phải tuân theo bất kể những gì khác trong cơ sở tri thức.

7.5.2 Chứng minh bằng độ phân giải

Chúng tôi đã lập luận rằng các quy tắc suy luận được đề cập cho đến nay là hợp lý, nhưng chúng tôi đã không thảo luận về câu hỏi về tính đầy đủ cho các thuật toán suy luận sử dụng chúng. Các thuật toán tìm kiếm chẳng hạn như tìm kiếm sâu lặp đi lặp lại (trang 99) là hoàn chỉnh theo nghĩa là chúng sẽ tìm thấy bất kỳ mục tiêu nào có thể đạt được, nhưng nếu các quy tắc suy luận có sẵn là không đủ, thì mục tiêu là không thể đạt được — không có bằng chứng nào tồn tại chỉ sử dụng các quy tắc suy luận đó. Ví dụ, nếu chúng ta loại bỏ quy tắc loại bỏ hai điều kiện, bằng chứng trong phần trước sẽ không được thông qua. Phần hiện tại giới thiệu một quy tắc suy luận duy nhất, độ phân giải, mang lại một thuật toán suy luận hoàn chỉnh khi kết hợp với bất kỳ thuật toán tìm kiếm hoàn chỉnh nào.

Chúng tôi bắt đầu bằng cách sử dụng một phiên bản đơn giản của quy tắc giải quyết trong thế giới wumpus. Chúng ta hãy xem xét các bước dẫn đến Hình 7.4 (a): tác nhân quay trở lại từ [2,1] đến [1,1] và sau đó đi đến [1,2], nơi nó cảm nhận được mùi hôi thối, nhưng không có gió. Chúng tôi thêm các thông tin sau vào cơ sở tri thức:

$R11 : \neg B1,2$. $R12 : B1,2 \Leftrightarrow (P1,1 \vee P2,2 \vee P1,3)$. Bảng cùng một quá trình dẫn đến $R10$ trước đó, bây giờ chúng ta có thể rút ra sự vắng mặt của các hổ trong [2,2] và [1,3] (hãy nhớ rằng [1,1] đã được biết là không hổ):

$R13 : \neg P2,2$. $R14 : \neg P1,3$. Chúng ta cũng có thể áp dụng loại bỏ hai điều kiện cho $R3$, tiếp theo là Modus Ponens với $R5$, để có được thực tế là có một hổ trong [1,1], [2,2], hoặc [3,1]:

$R15 : P1,1 \vee P2,2 \vee P3,1$.

Bây giờ là ứng dụng đầu tiên của quy tắc độ phân giải: nghĩa đen $\neg P2,2$ trong $R13$ phân giải với nghĩa đen $P2,2$ trong $R15$ để đưa ra khả năng hòa giải

Hòa giải

$R16 : P1,1 \vee P3,1$.

bằng tiếng Anh; Nếu có một hổ trong một trong [1,1], [2,2] và [3,1] và nó không nằm trong [2,2], thì nó nằm trong [1,1] hoặc [3,1]. Tương tự, nghĩa đen $\neg P1,1$ trong $R1$ giải quyết với nghĩa đen $P1,1$ trong $R16$ để đưa ra

$R17 : P3,1$.

Trong tiếng Anh: nếu có một hổ trong [1,1] hoặc [3,1] và nó không có trong [1,1], thì nó nằm trong [3,1]. Hai bước suy luận cuối cùng này là ví dụ về quy tắc suy luận độ phân giải đơn vị

Độ phân giải đơn vị

⁹ Các logic không đơn điệu, vi phạm tính đơn điệu, nắm bắt được một thuộc tính chung của tính đơn điệu: thay đổi suy nghĩ của một người. Chúng được thảo luận trong Phần 10.6.

Bổ sung
$$\frac{1 \vee \dots \vee m}{1 \vee \dots \vee i-1 \vee i+1 \vee \dots \vee k \text{ trong đó mỗi } i \text{ là một nghĩa đen và } i \text{ và } m \text{ là các nghĩa đen bổ sung (tức là một là phủ định của người kia). Do đó, quy tắc phân giải đơn vị lấy một mệnh đề - một sự tách rời của các nghĩa đen - và một theo nghĩa đen và tạo ra một mệnh đề mới. Lưu ý rằng một ký tự có thể được xem như một sự tách rời của một ký tự, còn được gọi là mệnh đề đơn vị.}}$$

Điều

Mệnh đề đơn vị

Nghị quyết Quy tắc độ phân giải đơn vị có thể được khái quát hóa thành quy tắc độ phân giải đầy đủ

$$\frac{1 \vee \dots \vee M1 \vee \dots \vee}{1 \vee \dots \vee i-1 \vee i+1 \vee k \vee m1 \vee \dots \vee m j-1 \vee m j+1 \vee \dots \vee mn \text{ trong đó } i \text{ và } m j \text{ là những nghĩa đen bổ sung. Điều này nói rằng resolution lấy hai mệnh đề và tạo ra một mệnh đề mới chứa tất cả các nghĩa đen của hai mệnh đề ban đầu ngoại trừ hai nghĩa đen bổ sung. Ví dụ, chúng ta có}}$$

$$\frac{P1.1 \vee P3.1. \neg P1.1 \vee}{\neg P2.2 P3.1 \vee \neg P2.2}$$

Bạn chỉ có thể giải quyết một cặp ký tự bổ sung tại một thời điểm. Ví dụ, chúng ta có thể phân giải P và $\neg P$ để suy ra

$$\frac{P \vee \neg Q \vee R, \neg P \vee}{\neg Q \vee Q \vee R},$$

Bao thanh toán

nhưng bạn không thể giải quyết cả P và Q cùng một lúc để suy ra R. Có một khía cạnh kỹ thuật nữa của quy tắc giải quyết: mệnh đề kết quả chỉ nên chứa một bản sao của mỗi chữ đen.10 Việc loại bỏ nhiều bản sao của các ký tự được gọi là bao thanh toán. Ví dụ, nếu chúng ta giải quyết $(A \vee B)$

với $(A \vee \neg B)$, chúng ta thu được $(A \vee A)$, được giảm xuống chỉ A bằng cách phân tích. Tính hợp lý của quy tắc giải quyết có thể được nhìn thấy dễ dàng bằng cách xem xét nghĩa đen i bổ sung cho nghĩa đen j trong mệnh đề khác. Nếu i là đúng, thì m j là sai, và từ đó $1 \vee \dots \vee m j-1 \vee m j+1 \vee \dots \vee mn$ phải đúng, vì m $1 \vee \dots \vee$ triệu được đưa ra. Nếu i là sai, thì $1 \vee \dots \vee i-1 \vee i+1 \vee \dots \vee k$ phải đúng vì $1 \vee \dots \vee k$ được đưa ra. Bây giờ 'tôi là đúng hoặc sai, vì vậy một trong những kết luận này được giữ nguyên — chính xác như quy tắc nghị quyết đã nêu.

Điều đáng ngạc nhiên hơn về quy tắc giải quyết là nó tạo thành cơ sở cho một gia đình các thủ tục suy luận hoàn chỉnh. Một người chứng minh định lý dựa trên độ phân giải có thể, đối với bất kỳ câu α và β nào trong logic mệnh đề, quyết định xem $\alpha \models \beta$ hay không. Hai tiểu mục tiếp theo giải thích cách

giải pháp thực hiện điều này.

Dạng bình thường liên kết

Quy tắc giải quyết chỉ áp dụng cho các mệnh đề (nghĩa là sự tách rời của các nghĩa đen), vì vậy nó có vẻ chỉ liên quan đến các cơ sở tri thức và truy vấn bao gồm các mệnh đề. Vậy thì làm thế nào nó có thể dẫn đến một thủ tục suy luận hoàn chỉnh cho tất cả logic mệnh đề? Câu trả lời là mọi câu của logic mệnh đề đều tương đương về mặt logic với sự kết hợp của các mệnh đề.

Biểu mẫu bình thường liên kết CNF

Một câu được biểu thị dưới dạng liên kết của các mệnh đề được cho là ở dạng chuẩn liên kết hoặc CNF (xem Hình 7.12). Bây giờ chúng tôi mô tả một quy trình để chuyển đổi sang CNF. Chúng ta minh họa thủ tục bằng cách chuyển đổi câu $B1,1 \Leftrightarrow (P1,2 \vee P2,1)$ thành CNF. Các bước như sau:

10 Nếu một mệnh đề được xem như một tập hợp các ký tự, thì hạn chế này sẽ tự động được tôn trọng. Sử dụng ký hiệu tập cho các mệnh đề làm cho quy tắc giải quyết gọn gàng hơn nhiều, với chỉ phí là giới thiệu ký hiệu bổ sung.

CNFSentence $\rightarrow$ khoản 1 $\wedge \dots \wedge$ Clausen

Điều khoản $\rightarrow$ Nghĩa đen1 $\vee \dots \vee$ Nghĩa đen

Sự thật $\rightarrow$ Biểu tượngBiểu tượng $\rightarrow$

theo nghĩa đen | $\neg$ Biểu tượngBiểu

tượng $\rightarrow$ P | Hỏi | R | ...

HornClauseForm $\rightarrow$ DefiniteClauseForm | GoalClauseFormDefiniteClauseForm

$\rightarrow$ Thực tế | (Ký hiệu1 $\wedge \dots \wedge$ Biểu tượng) Biểu tượng $\rightarrow$

GoalClauseForm $\rightarrow$ (Ký hiệu1 $\wedge \dots \wedge$ Ký hiệu) $\Rightarrow$ Sai

Hình 7.12 Ngữ pháp cho dạng chuẩn liên kết, mệnh đề Sừng và mệnh đề xác định. Mệnh đề ACNF như $\neg A \vee \neg B \vee C$ có thể được viết dưới dạng mệnh đề xác định là $A \wedge B \Rightarrow C$.

1. Loại bỏ $\Leftrightarrow$, thay thế $\alpha \Leftrightarrow \beta$ bằng $(\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha)$.

$(B1,1 \Rightarrow (P1,2 \vee P2,1)) \wedge ((P1,2 \vee P2,1) \Rightarrow B1,1)$.

2. Loại bỏ $\Rightarrow$, thay thế $\alpha \Rightarrow \beta$ bằng $\neg \alpha \vee \beta$: $(\neg B1,1$

$\vee P1,2 \vee P2,1) \wedge (\neg (P1,2 \vee P2,1) \vee B1,1)$.

3. CNF yêu cầu $\neg$ chỉ xuất hiện trong các nghĩa đen, vì vậy chúng ta "di chuyển $\neg$ vào trong" bằng cách áp dụng lặp đi lặp lại các tương đương sau đây từ Hình 7.11: $\neg (\neg \alpha) \equiv \alpha$ (loại bỏ phủ định kép) $\neg (\alpha \wedge \beta) \equiv (\neg \alpha \vee \neg \beta)$ (De Morgan) $\neg (\alpha \vee \beta) \equiv (\neg \alpha \wedge \neg \beta)$ (De Morgan) Trong ví dụ, chúng ta chỉ yêu cầu một ứng dụng của quy tắc cuối cùng: $(\neg B1,1 \vee P1,2 \vee P2,1) \wedge ((\neg P1,2 \wedge \neg P2,1) \vee B1,1)$.

4. Bây giờ chúng ta có một câu chứa các toán tử $\wedge$ và $\vee$ lồng nhau được áp dụng cho các ký tự. Chúng tôi áp dụng định luật phân phối từ Hình 7.11, phân phối $\wedge$ trên $\vee$ bất cứ khi nào có thể. $(\neg B1,1 \vee P1,2 \vee P2,1) \wedge (\neg P1,2 \vee B1,1) \wedge (\neg P2,1 \vee B1,1)$. Câu ban đầu bây giờ là CNF, như một sự kết hợp của ba mệnh đề. Nó khó đọc hơn nhiều, nhưng nó có thể được sử dụng làm đầu vào cho quy trình giải quyết.

Thuật toán độ phân giải

Các thủ tục suy luận dựa trên công việc giải quyết bằng cách sử dụng nguyên tắc chứng minh bằng cách chống lại được giới thiệu ở trang 241. Nghĩa là, để chỉ ra rằng $KB \models \alpha$, chúng ta chỉ ra rằng $(KB \wedge \neg \alpha)$ là không thỏa mãn. Chúng ta làm điều này bằng cách chứng minh một mâu thuẫn. Thuật toán độ phân giải được thể hiện trong Hình 7.13. Đầu tiên, $(KB \wedge \neg \alpha)$ được chuyển đổi thành CNF. Sau đó, quy tắc giải quyết được áp dụng cho các điều khoản kết quả. Mỗi cặp chứa các ký tự hoàn chỉnh được phân giải để tạo ra một mệnh đề mới, được thêm vào tập hợp nếu nó chưa có. Quá trình này tiếp tục cho đến khi một trong hai điều xảy ra:

- không có điều khoản mới nào có thể được thêm vào, trong trường hợp đó KB không đòi hỏi α ; hoặc
- hai điều khoản giải quyết để tạo ra mệnh đề trống, trong trường hợp đó KB đòi hỏi α .

hàm PL-RESOLUTION(KB, α) trả về true hoặc false

đầu vào: KB, cơ sở tri thức, một câu trong logic mệnh đề
 α , truy vấn, một câu trong logic mệnh đề

các mệnh đề $\leftarrow$ tập hợp các mệnh đề trong biểu diễn CNF
 của KB $\wedge$ α new $\leftarrow \{ \}$ while true do

đối với mỗi cặp mệnh đề Ci, Cj trong các mệnh đề do
 resolves $\leftarrow$ PL-RESOLVE(Ci,Cj) nếu resolves chứa
 mệnh đề rỗng thì trả về true new $\leftarrow$ new $\cup$ resolves if
 mới $\subseteq$ điều khoản thì trả về false clauses $\leftarrow$ mệnh đề
 $\cup$ mới

Hình 7.13 Một thuật toán độ phân giải đơn giản cho logic mệnh đề. PL-RESOLVE trả về tập hợp tất cả các mệnh đề có thể thu được bằng cách giải quyết hai đầu vào của nó.

Mệnh đề trống – một sự tách rời không có sự tách rời – tương đương với False vì một sự tách rời chỉ đúng nếu ít nhất một trong các sự tách rời của nó là đúng. Hơn nữa, mệnh đề trống chỉ phát sinh từ việc giải quyết hai mệnh đề đơn vị mâu thuẫn như P và $\neg P$.

Chúng ta có thể áp dụng quy trình giải quyết cho một suy luận rất đơn giản trong thế giới wumpus. Khi tác nhân ở trong [1,1], không có gió, vì vậy không thể có hồ ở các ô vuông lân cận. Cơ sở kiến thức có liên quan là

KB = $R2 \wedge R4 = (B1,1 \Leftrightarrow (P1,2 \vee P2,1)) \wedge \neg B1,1$ và chúng tôi muốn chứng minh α , đó là, giả sử, $\neg P1,2$. Khi chúng ta chuyển đổi (KB $\wedge \neg \alpha$) thành CNF, chúng ta thu được các mệnh đề được hiển thị ở đầu Hình 7.14. Hàng thứ hai của hình hiển thị các mệnh đề thu được bằng cách giải quyết các cặp trong hàng đầu tiên. Sau đó, khi P1,2 được giải quyết bằng $\neg P1,2$, chúng ta thu được mệnh đề trống, được hiển thị dưới dạng một hình vuông nhỏ. Kiểm tra Hình 7.14 cho thấy nhiều bước giải quyết là vô nghĩa. Ví dụ, mệnh đề $B1,1 \vee \neg B1,1 \vee P1,2$ tương đương với True $\vee P1,2$ tương đương với True. Suy luận rằng Đúng là đúng không hữu ích lắm. Do đó, bất kỳ mệnh đề nào trong đó hai ký tự bổ sung xuất hiện đều có thể bị loại bỏ.

Tính đầy đủ của giải pháp

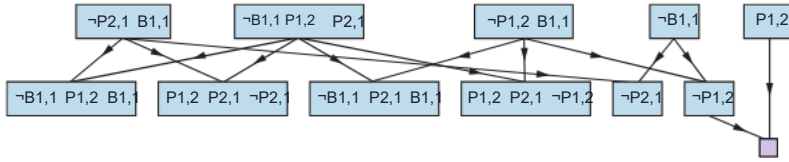
Đóng giải pháp

Để kết thúc cuộc thảo luận của chúng tôi về nghị quyết, bây giờ chúng tôi chỉ ra lý do tại sao PL-RESOLUTION là hoàn tất. Để làm điều này, chúng tôi giới thiệu RC (S) đóng giải quyết của một tập hợp các mệnh đề S, là tập hợp của tất cả các điều khoản có thể được phát sinh bằng cách áp dụng lặp đi lặp lại quy tắc giải quyết cho các điều khoản trong S hoặc các điều khoản phái sinh của chúng. Đóng độ phân giải là những gì PL-RESOLUTION tính toán làm giá trị cuối cùng của các mệnh đề biến. Thật dễ dàng để thấy rằng RC (S) phải là hữu hạn: nhờ bước bao thầy, chỉ có nhiều mệnh đề riêng biệt có thể được xây dựng từ các ký hiệu P1, . . . , Pk xuất hiện trong S. Do đó, PL-RESOLUTION luôn chấm dứt.

Định lý hoàn chỉnh để giải quyết trong logic mệnh đề được gọi là định lý giải quyết cơ bản:

Định lý độ phân giải
 mật đất

Nếu một tập hợp các mệnh đề không thỏa mãn, thì việc đóng giải quyết của các điều khoản đó chứa mệnh đề trống.



Hình 7.14 Áp dụng một phần PL-RESOLUTION cho một suy luận đơn giản trong thế giới wumpusworld để chứng minh truy vấn $\neg P1,2$. Mỗi mệnh đề trong số bốn mệnh đề ngoài cùng bên trái ở hàng trên cùng được ghép nối với mỗi mệnh đề trong số ba mệnh đề còn lại và quy tắc giải quyết được áp dụng để tạo ra các mệnh đề ở hàng dưới cùng. Chúng ta thấy rằng các mệnh đề thứ ba và thứ tư ở hàng trên cùng kết hợp để tạo ra mệnh đề $\neg P1,2$, sau đó được giải quyết với $P1,2$ để tạo ra mệnh đề trống, có nghĩa là truy vấn đã được chứng minh.

Định lý này được chứng minh bằng cách chứng minh nghịch đảo của nó: nếu RC (S) đóng không chứa mệnh đề rỗng, thì S là thỏa mãn. Trên thực tế, chúng ta có thể xây dựng một mô hình cho S với các giá trị chân lý phù hợp cho $P1, \dots, Pk$. Quy trình thi công như sau:

Đối với i từ 1 đến k ,

- Nếu một mệnh đề trong RC (S) chứa nghĩa đen $\neg Pi$ và tất cả các ký tự khác của nó là sai trong gán được chọn cho $P1, \dots, Pi-1$, sau đó gán false cho Pi .- Nếu không, gán true cho Pi .Gán này cho $P1, \dots, Pk$ là một mô hình của S. Để thấy điều này, giả sử ngược lại - rằng, ở một số giai đoạn i trong dãy, gán ký hiệu Pi khiến một số mệnh đề C trở nên sai. Để điều này xảy ra, phải là trường hợp tất cả các nghĩa đen khác trong C phải đã bị giả mạo bằng cách gán cho $P1, \dots, Pi-1$. Do đó, C bây giờ phải trông giống như $(false \vee false \vee \dots \vee false \vee Pi)$ hoặc giống như $(false \vee false \vee \dots \vee false \vee \neg Pi)$. Nếu chỉ một trong hai mệnh đề này nằm trong RC (S), thì thuật toán sẽ gán giá trị chân lý thích hợp cho Pi để làm cho C đúng, vì vậy C chỉ có thể bị sai lệch nếu cả hai mệnh đề này đều nằm trong RC (S).

Bây giờ, vì RC (S) đã được đóng lại theo nghị quyết, nó sẽ chứa sự giải quyết của hai điều khoản này, và việc giải quyết đó sẽ có tất cả các nghĩa đen của nó đã bị sai lệch bởi các gán cho $P1, \dots, Pi-1$. Điều này mâu thuẫn với giả định của chúng tôi rằng mệnh đề giả mạo đầu tiên xuất hiện ở stage i . Do đó, chúng tôi đã chứng minh rằng cấu trúc không bao giờ làm sai lệch một điều khoản trong RC (S); nghĩa là, nó tạo ra một mô hình RC (S). Cuối cùng, vì S được chứa trong RC(S), bất kỳ mô hình nào của RC(S) đều là mô hình của chính S.

7.5.3 Mệnh đề sừng và mệnh đề xác định

Tính đầy đủ của độ phân giải làm cho nó trở thành một phương pháp suy luận rất quan trọng. Tuy nhiên, trong nhiều tình huống thực tế, không cần toàn bộ khả năng giải quyết. Một số cơ sở tri thức trong thế giới thực đáp ứng những hạn chế nhất định về hình thức câu mà chúng chứa, cho phép chúng sử dụng thuật toán suy luận hạn chế và hiệu quả hơn.

Một dạng hạn chế như vậy là mệnh đề xác định, là sự tách rời của các nghĩa đen mà chính xác một là tích cực. Ví dụ, mệnh đề $(\neg L1,1 \vee \neg Breeze \vee B1,1)$ là một mệnh đề xác định, trong khi $(\neg B1,1 \vee P1,2 \vee P2,1)$ thì không, vì nó có hai mệnh đề dương.

Tổng quát hơn một chút là mệnh đề Horn, là một sự tách rời của các nghĩa đen mà tại

Mệnh đề xác định

Mệnh đề còi

Điều khoản mục tiêu

hầu hết là tích cực. Vì vậy, tất cả các mệnh đề xác định đều là mệnh đề Sừng, cũng như các mệnh đề không có nghĩa đen; Chúng được gọi là mệnh đề mục tiêu. Các điều khoản còi được đóng theo nghị quyết: nếu bạn giải quyết

hai mệnh đề Sừng, bạn nhận lại một mệnh đề Sừng. Một lớp nữa là câu k-CNF, là một câu CNF trong đó mỗi mệnh đề có nhiều nhất k ký từ.

Cơ sở kiến thức chỉ chứa mệnh đề xác định là thú vị vì ba lý do:

1. Mỗi mệnh đề xác định có thể được viết như một ngụ ý có tiền đề là sự kết hợp của các nghĩa đen tích cực và kết luận của nó là một nghĩa đen tích cực duy nhất. (Xem Bài tập 7.DISJ.) Ví dụ, mệnh đề xác định $(\neg L1,1 \vee \neg Breeze \vee B1,1)$ có thể được viết dưới dạng ngụ ý $(L1,1 \wedge Breeze) \Rightarrow B1,1$. Ở dạng ngụ ý, câu dễ hiểu hơn: nó nói rằng nếu tác nhân ở trong [1,1] và có một cảm giác gió, thì [1,1] là gió mát. Trong dạng Sừng, tiền đề được gọi là cơ thể và kết luận được gọi là đầu. Một câu bao gồm một nghĩa đen dương duy nhất, chẳng hạn như $L1,1$, được gọi là sự thật. Nó cũng có thể được viết dưới dạng ngụ ý là $True \Rightarrow L1,1$, nhưng đơn giản hơn nếu chỉ viết $L1,1$.
2. Suy luận với các mệnh đề Sừng có thể được thực hiện thông qua chuỗi tiến và lùi-thuật toán chuỗi, mà chúng tôi giải thích tiếp theo. Cả hai thuật toán này đều tự nhiên, trong đó các bước suy luận rõ ràng và dễ dàng cho con người làm theo. Loại suy luận này là cơ sở cho lập trình logic, được thảo luận trong Chương 9.
3. Quyết định liên quan đến các mệnh đề Kèn có thể được thực hiện trong thời gian tuyến tính về kích thước của cơ sở kiến thức — một bất ngờ thú vị.

Thân thể

Đầu

Điều

Chuỗi chuyển tiếp

Chuỗi ngược

7.5.4 Chuỗi tiến và lùi

Thuật toán chuỗi chuyển tiếp PL-FC-ENQUIRES? (KB, q) xác định xem một ký hiệu mệnh đề duy nhất q - truy vấn - có được đòi hỏi bởi một cơ sở kiến thức của các mệnh đề xác định hay không. Nó bắt đầu từ các sự kiện đã biết (nghĩa đen tích cực) trong cơ sở tri thức. Nếu tất cả các tiền đề của một hàm ý được biết, thì kết luận của nó được thêm vào tập hợp các sự kiện đã biết. Ví dụ: nếu $L1,1$ và $Breeze$ được biết và $(L1,1 \wedge Breeze) \Rightarrow B1,1$ nằm trong cơ sở tri thức, thì $B1,1$ có thể được thêm vào. Quá trình này tiếp tục cho đến khi truy vấn q được thêm vào hoặc cho đến khi không thể đưa ra thêm suy luận. Thuật toán được thể hiện trong Hình 7.15; Điểm chính cần nhớ là nó chạy trong thời gian tuyến tính.

Cách tốt nhất để hiểu thuật toán là thông qua một ví dụ và một hình ảnh. Hình 7.16 (a) cho thấy một cơ sở kiến thức đơn giản về mệnh đề Sừng với A và B là các sự kiện đã biết. Hình 7.16 (b) cho thấy cùng một cơ sở kiến thức được vẽ dưới dạng đồ thị AND-OR (xem Chương 4). Trong đồ thị AND-OR, nhiều cạnh được nối bằng một vòng cung cho thấy một liên kết - mọi cạnh phải được chứng minh - trong khi nhiều cạnh không có vòng cung cho thấy sự tách rời - bất kỳ cạnh nào cũng có thể được chứng minh. Thật dễ dàng để thấy cách chuỗi chuyển tiếp hoạt động trong biểu đồ. Các lá đã biết (ở đây, A và B) được thiết lập, và suy luận lan truyền lên đồ thị càng xa càng tốt. Bất cứ nơi nào một liên kết xuất hiện, sự lan truyền sẽ đợi cho đến khi tất cả các liên kết được biết trước khi tiếp tục. Người đọc được khuyến khích làm việc chi tiết qua ví dụ.

Thật dễ dàng để thấy rằng chuỗi chuyển tiếp là hợp lý: mọi suy luận về cơ bản là một ứng dụng của Modus Ponens. Chuỗi chuyển tiếp cũng hoàn chỉnh: mọi câu nguyên tử kéo theo sẽ được rút ra. Cách dễ nhất để thấy điều này là xem xét trạng thái cuối cùng của bảng suy luận (sau khi thuật toán đạt đến điểm cố định mà không thể suy luận mới). Bảng chứa true cho mỗi ký hiệu được suy ra trong quá trình và false cho tất cả các ký hiệu khác. Chúng ta có thể xem bảng như một mô hình logic; hơn nữa, mọi mệnh đề xác định trong KB gốc đều đúng trong mô hình này.

chức năng PL-FC-ENQUIRES? (KB, q) trả về true hoặc false
 đầu vào: KB, cơ sở tri thức, một tập hợp các mệnh đề xác định mệnh đề
 q, truy vấn, một biểu tượng mệnh đề $\leftarrow$ một bảng, trong đó count[c] ban đầu là
 số lượng ký hiệu trong tiền đề của mệnh đề C được suy ra $\leftarrow$ một bảng, trong
 đó inferred[s] ban đầu là false cho tất cả các symbolqueue $\leftarrow$ một hàng đợi
 các ký hiệu, ban đầu các ký hiệu được biết là đúng trong KB

trong khi hàng đợi không trống, hãy
 $p \leftarrow \text{POP}(\text{hàng đợi})$ nếu $p = q$ thì trả
 về true nếu suy luận[p] = false thì

suy luận[p] $\leftarrow$ true cho mỗi mệnh đề c trong KB trong đó p nằm trong c. PREMISE do

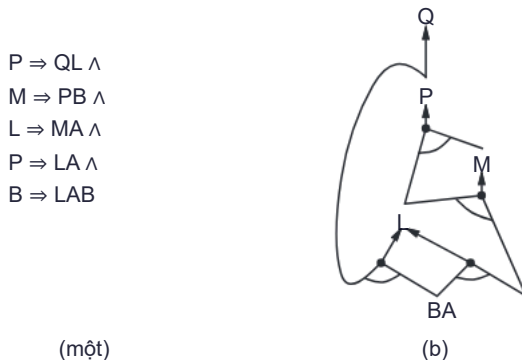
số lượng giảm[c] nếu count[c] = 0 thì thêm c.CONCLUSION vào hàng đợi

Trả về false

Hình 7.15 Thuật toán chuỗi chuyển tiếp cho logic mệnh đề. Hàng đợi theo dõi các ký hiệu được biết là đúng nhưng chưa được "xử lý". Bảng đếm theo dõi có bao nhiêu tiền đề của mỗi hàm ý chưa được chứng minh. Bất cứ khi nào một ký hiệu p mới từ chương trình nghị sự được xử lý, số lượng sẽ giảm xuống một cho mỗi hàm ý mà tiền đề xuất hiện (để đảm bảo xác định trong thời gian không đổi với lập chỉ mục thích hợp.) Nếu một số lượng đạt đến không, tất cả các tiền đề của hàm ý đều được biết, vì vậy kết luận của nó có thể được thêm vào chương trình nghị sự. Cuối cùng, chúng ta cần theo dõi những biểu tượng nào đã được xử lý; Một biểu tượng đã có trong tập hợp các ký hiệu suy luận không cần phải được thêm vào chương trình nghị sự một lần nữa. Điều này tránh được công việc dư thừa và ngăn chặn các vòng lặp do các tác động như $P \Rightarrow Q$ và $Q \Rightarrow P$.

Để thấy điều này, hãy giả định điều ngược lại, cụ thể là một số điều khoản $a_1 \wedge \dots \wedge a_k \Rightarrow b$ là sai trong mô hình. Sau đó, $a_1 \wedge \dots \wedge a_k$ phải đúng trong mô hình và b phải là sai trong mô hình. Nhưng điều này mâu thuẫn với giả định của chúng tôi rằng thuật toán đã đạt đến một điểm cố định, bởi vì bây giờ chúng tôi sẽ được cấp phép để thêm b vào KB. Do đó, chúng ta có thể kết luận rằng tập hợp các câu nguyên tử được suy ra tại điểm cố định xác định một mô hình của KB gốc. Hơn nữa, bất kỳ câu nguyên tử nào q được bao gồm bởi KB phải đúng trong tất cả các mô hình của nó và trong mô hình này nói riêng. Do đó, mọi câu nguyên tử q phải được suy ra bởi thuật toán.

Forward chaining là một ví dụ về khái niệm chung về suy luận dựa trên dữ liệu — rằng Dựa trên dữ liệu là, lý luận trong đó trọng tâm của sự chú ý bắt đầu với dữ liệu đã biết. Nó có thể được sử dụng trong một tác nhân để rút ra kết luận từ các nhận thức đến, thường không có một truy vấn cụ thể. Ví dụ, tác nhân wumpus có thể NÓI VỚI nhận thức của nó với cơ sở tri thức bằng cách sử dụng thuật toán chuỗi chuyển tiếp gia tăng, trong đó các sự kiện mới có thể được thêm vào chương trình nghị sự để bắt đầu các suy luận mới. Ở con người, một lượng nhất định của lý luận dựa trên dữ liệu xảy ra khi thông tin mới đến. Ví dụ, nếu tôi đang ở trong nhà và nghe thấy tiếng mưa bắt đầu rơi, tôi có thể nghĩ rằng buổi dã ngoại sẽ bị hủy bỏ. Tuy nhiên, có lẽ tôi sẽ không nghĩ rằng cánh hoa thứ mười bảy trên bông hồng lớn nhất trong vườn hàng xóm của tôi sẽ bị ướt; Con người tiếp tục tiến về phía trước dưới sự kiểm soát cẩn thận, kéo họ bị ngập trong những hậu quả không liên quan. Thuật toán chuỗi ngược, như tên gọi của nó, hoạt động ngược từ truy vấn. Nếu truy vấn q được biết là đúng, thì không cần công việc. Nếu không, thuật toán tìm thấy



Hình 7.16 (a) Một tập hợp các mệnh đề Horn. (b) Biểu đồ AND–OR tương ứng.

Những hàm ý đó trong cơ sở tri thức có kết luận là Q . Nếu tất cả các tiền đề của một trong những hàm ý đó có thể được chứng minh là đúng (bằng cách chuỗi ngược), thì q là đúng. Khi áp dụng cho truy vấn Q trong Hình 7.16, nó hoạt động trở lại đồ thị cho đến khi nó đạt được một tập hợp các sự kiện đã biết, A và B , tạo thành cơ sở cho một chứng minh. Thuật toán về cơ bản giống hệt với thuật toán AND-OR-GRAPH-SEARCH trong Hình 4.11. Cũng như chuỗi chuyển tiếp, một triển khai hiệu quả chạy trong thời gian tuyến tính.

Lý luận định hướng
mục tiêu

Chuỗi ngược là một hình thức suy luận hướng đến mục tiêu. Nó rất hữu ích để trả lời những câu hỏi cụ thể như "Tôi phải làm gì bây giờ?" và "Chìa khóa của tôi ở đâu?" Thông thường, chi phí của chuỗi ngược thấp hơn nhiều so với tuyến tính về kích thước của cơ sở tri thức, bởi vì quá trình này chỉ chạm đến các sự kiện có liên quan.

7.6 Kiểm tra mô hình mệnh đề hiệu quả

Trong phần này, chúng tôi mô tả hai họ thuật toán hiệu quả cho suy luận mệnh đề chung dựa trên kiểm tra mô hình: một cách tiếp cận dựa trên tìm kiếm theo dõi lùi và một cách tìm kiếm leo đồi cục bộ. Các thuật toán này là một phần của "công nghệ" của thuyết minh luận. Phần này có thể được đọc lướt qua khi đọc lần đầu tiên của chương.

Các thuật toán chúng tôi mô tả là để kiểm tra khả năng thỏa mãn: bài toán SAT. (Như đã lưu ý trong Phần 7.5, thử nghiệm bao gồm, $\alpha \models \beta$, có thể được thực hiện bằng cách kiểm tra tính không thỏa mãn của $\alpha \wedge \neg\beta$.) Chúng tôi đã đề cập ở trang 241 mối liên hệ giữa việc tìm kiếm một mô hình thỏa mãn cho một câu logic và tìm ra giải pháp cho một vấn đề thỏa mãn ràng buộc, vì vậy có lẽ không có gì ngạc nhiên khi hai họ thuật toán thỏa mãn mệnh đề gần giống với các thuật toán quay trở lại của Phần 5.3 và các thuật toán tìm kiếm cục bộ của Phần 5.4. Tuy nhiên, chúng cực kỳ quan trọng theo đúng nghĩa của chúng vì rất nhiều vấn đề tổ hợp trong khoa học máy tính có thể được rút gọn để kiểm tra khả năng thỏa mãn của một câu mệnh đề. Bất kỳ cải tiến nào trong các thuật toán khả năng thỏa mãn đều có hậu quả rất lớn đối với khả năng xử lý sự phức tạp nói chung của chúng ta.

7.6.1 Thuật toán quay ngược hoàn chỉnh

Thuật toán đầu tiên mà chúng tôi xem xét thường được gọi là thuật toán Davis-Putnam, sau thuật toán sem-

Davis–Putnam thuật toán

bài báo của Martin Davis và Hilary Putnam (1960). Thuật toán trên thực tế là phiên bản được mô tả bởi Davis, Logemann và Loveland (1962), vì vậy chúng tôi sẽ gọi nó là DPLL theo ý kiến của cả bốn tác giả. DPLL lấy một câu ở dạng thông tin liên kết làm đầu vào — một tập hợp mệnh đề. Giống như BACKTRACKING-SEARCH và TT-ENTAILS?, về cơ bản nó là một liệt kê đệ quy, độ sâu đầu tiên của các mô hình có thể. Nó thể hiện ba cải tiến so với sơ đồ đơn giản của TT-ENTAILS?:

- **Chấm dứt sớm:** Thuật toán phát hiện câu phải đúng hay sai, ngay cả với mô hình đã hoàn thành một phần. Một mệnh đề là đúng nếu bất kỳ nghĩa đen nào là đúng, ngay cả khi các nghĩa đen khác chưa có giá trị chân lý; do đó, toàn bộ câu có thể được đánh giá đúng ngay cả trước khi mô hình hoàn thành. Ví dụ, câu $(A \vee B) \wedge (A \vee C)$ là đúng nếu A là đúng, bất kể giá trị của B và C. Tương tự, một câu là sai nếu bất kỳ mệnh đề nào là sai, điều này xảy ra khi mỗi nghĩa đen của nó là sai. Một lần nữa, điều này có thể xảy ra rất lâu trước khi mô hình hoàn thành. Chấm dứt sớm tránh kiểm tra toàn bộ cây con trong không gian tìm kiếm.
- **Phòng đoán biểu tượng thuần túy:** Biểu tượng thuần túy là biểu tượng luôn xuất hiện cùng một "ký" trong tất cả các mệnh đề. Ví dụ, trong ba mệnh đề $(A \vee \neg B)$, $(\neg B \vee \neg C)$ và $(C \vee A)$, ký hiệu A là thuần túy vì chỉ có nghĩa đen dương xuất hiện, B là thuần túy vì chỉ có nghĩa đen phủ xuất hiện và C là không trong sạch. Thật dễ dàng để thấy rằng nếu một câu có một mô hình, thì nó có một mô hình với các ký hiệu thuần túy được gán để làm cho nghĩa đen của chúng đúng, bởi vì làm như vậy không bao giờ có thể làm cho một mệnh đề sai. Lưu ý rằng, khi xác định độ tinh khiết của một ký hiệu, thuật toán có thể bỏ qua các mệnh đề đã được biết là đúng trong mô hình được xây dựng cho đến nay. Ví dụ, nếu mô hình chứa $B = \text{false}$, thì mệnh đề $(\neg B \vee \neg C)$ đã đúng, và trong các mệnh đề còn lại, C chỉ xuất hiện dưới dạng dương tính; do đó C trở nên thuần khiết.
- **Phòng đoán mệnh đề đơn vị:** Mệnh đề đơn vị được định nghĩa trước đó là mệnh đề chỉ có một chữ đen. Trong ngữ cảnh của DPLL, nó cũng có nghĩa là các mệnh đề trong đó tất cả các ký tự ngoại trừ một ký tự đã được mô hình gán sai. Ví dụ: nếu mô hình chứa $B = \text{true}$, thì $(\neg B \vee \neg C)$ đơn giản hóa thành $\neg C$, là mệnh đề đơn vị. Rõ ràng, để mệnh đề này đúng, C phải được đặt thành false. Phòng đoán mệnh đề đơn vị gán tất cả các ký hiệu như vậy trước khi phân nhánh trên phần còn lại. Một hệ quả quan trọng của heuristic là bất kỳ nỗ lực nào để chứng minh (bằng cách bác bỏ) một nghĩa đen đã có trong cơ sở tri thức sẽ thành công ngay lập tức (Bài tập 7.KNOW). Cũng lưu ý rằng gán một mệnh đề đơn vị có thể tạo ra một mệnh đề đơn vị khác—ví dụ, khi C được đặt thành false, $(C \vee A)$ trở thành mệnh đề đơn vị, khiến true được gán cho A. "Tàng" của các nhiệm vụ cưỡng bức này được gọi là lan truyền đơn vị.

Nhân giống đơn vị

Nó giống với quá trình chuỗi chuyển tiếp với các mệnh đề xác định, và thực sự, nếu biểu thức CNF chỉ chứa các mệnh đề xác định thì DPLL về cơ bản sao chép chuỗi chuyển tiếp. (Xem Bài tập 7.DPLL.) Thuật toán DPLL được hiển thị trong Hình 7.17, cung cấp khung thiết yếu của quá trình tìm kiếm mà không có chi tiết triển khai.

Những gì Hình 7.17 không cho thấy là các thủ thuật cho phép các giải SAT mở rộng quy mô lên các vấn đề lớn. Điều thú vị là hầu hết các thủ thuật này trên thực tế khá chung chung, và chúng ta đã thấy chúng trước đây dưới các vỏ bọc khác:

chức năng DPLL-SATISFIABLE? (s) trả về true hoặc false

đầu vào: S, một câu trong logic mệnh đề

các mệnh đề $\leftarrow$ tập hợp các mệnh đề trong biểu diễn CNF của các ký hiệu
 $\leftarrow$ danh sách các ký hiệu mệnh đề trong sreturn DPLL(clauses, symbols, { })

hàm DPLL(mệnh đề, ký hiệu, mô hình) trả về đúng hoặc sai

nếu mọi mệnh đề trong mệnh đề là đúng trong mô hình thì trả về true
 nếu một số mệnh đề trong mệnh đề là sai trong mô hình thì trả về false
 P, giá trị $\leftarrow$ FIND-PURE-SYMBOL(ký hiệu, mệnh đề, mô hình)
 nếu P không phải là rỗng thì trả về DPLL(mệnh đề, ký hiệu - P, mô hình $\cup$ {P=giá trị})
 P, giá trị $\leftarrow$ FIND-UNIT-CLAUSE(mệnh đề, mô hình)
 nếu P không phải là rỗng thì trả về DPLL(mệnh đề, ký hiệu - P, mô hình $\cup$ {P= giá trị})
 P $\leftarrow$ FIRST (ký hiệu); rest $\leftarrow$ REST(ký hiệu)
 trả về DPLL(mệnh đề, phần nghỉ, mô hình $\cup$ {P=true}) hoặc DPLL(mệnh đề, phần còn lại, mô hình $\cup$ {P=sai})

Hình 7.17 Thuật toán DPLL để kiểm tra khả năng thỏa mãn của một câu trong thuyết minh luận. Các ý tưởng đằng sau FIND-PURE-SYMBOL và FIND-UNIT-CLAUSE được mô tả trong văn bản; mỗi cái trả về một ký hiệu (hoặc null) và giá trị chân lý để gán cho ký hiệu đó. Giống như TT-ENTAILS?, DPLL hoạt động trên các mô hình từng phần.

1. Phân tích thành phần (như đã thấy với Tasmania trong CSP): Khi DPLL gán giá trị chân thật cho các biến, tập hợp các mệnh đề có thể được tách ra thành các tập con rời rạc, được gọi là thành phần, không chia sẻ các biến không được gán. Với một cách hiệu quả để phát hiện khi điều này xảy ra, một bộ giải có thể đạt được tốc độ đáng kể bằng cách làm việc trên từng thành phần riêng biệt.
2. Thứ tự biến và giá trị (như đã thấy trong Phần 5.3.1 cho CSP): Việc triển khai DPLL đơn giản của chúng tôi sử dụng một thứ tự biến tùy ý và luôn thử giá trị true trước false. Mức độ heuristic (xem trang 177) gợi ý chọn biến xuất hiện thường xuyên nhất trên tất cả các mệnh đề còn lại.
3. Quay ngược thông minh (như đã thấy trong Phần 5.3.3 cho CSP): Nhiều vấn đề không thể giải quyết trong vài giờ chạy với tính năng quay ngược theo thứ tự thời gian có thể được giải quyết trong vài giây với tính năng quay ngược thông minh sao lưu đến điểm xung đột có liên quan. Tất cả các giải SAT thực hiện quay ngược thông minh đều sử dụng một số hình thức học mệnh đề xung đột để ghi lại các xung đột để chúng không bị lặp lại sau này trong quá trình tìm kiếm. Thông thường một tập hợp các xung đột có kích thước giới hạn được giữ lại, và những xung đột hiếm khi được sử dụng sẽ bị loại bỏ.
4. Khởi động lại ngẫu nhiên (như đã thấy ở trang 131 để leo đồi): Đôi khi một cuộc chạy đường như không đạt được tiến bộ. Trong trường hợp này, chúng ta có thể bắt đầu lại từ đầu cây tìm kiếm, thay vì cố gắng tiếp tục. Sau khi khởi động lại, các lựa chọn ngẫu nhiên khác nhau (trong lựa chọn biến và giá trị) được thực hiện. Các mệnh đề được học trong lần chạy đầu tiên được giữ lại sau khi khởi động lại và có thể giúp cắt tía không gian tìm kiếm. Khởi động lại không đảm bảo rằng giải pháp sẽ được tìm thấy nhanh hơn, nhưng nó làm giảm phương sai về thời gian giải pháp.
5. Lập chỉ mục thông minh (như đã thấy trong nhiều thuật toán): Các phương pháp tăng tốc được sử dụng trong DPLL, cũng như các thủ thuật được sử dụng trong các trình giải hiện đại, yêu cầu lập chỉ mục nhanh chóng những thứ như vậy

hàm WALKSAT(clauses, p, max flips) trả về một mô hình thỏa mãn hoặc thất bại

đầu vào: mệnh đề, một tập hợp các mệnh đề trong logic mệnh đề

p, xác suất chọn thực hiện động tác "đi bộ ngẫu nhiên", thường là khoảng 0,5 lần lật tối đa, số lần lật giá trị được phép trước khi bỏ cuộc

mô hình $\leftarrow$ gán ngẫu nhiên đúng / sai cho các ký hiệu trong mệnh đề cho mỗi $i = 1$ đến lật tối đa làm

nếu model thỏa mãn các mệnh đề thì trả về modelclause $\leftarrow$ một mệnh đề được chọn ngẫu nhiên từ các mệnh đề là false trong modelif RANDOM(0, 1) $\leq$ p thì

Lật giá trị trong mô hình của một ký hiệu được chọn ngẫu nhiên từ mệnh đề khác lật bất kỳ ký hiệu nào trong mệnh đề tối đa hóa số lượng mệnh đề thỏa mãntrả lại thất bại

Hình 7.18 Thuật toán WALKSAT để kiểm tra khả năng thỏa mãn bằng cách lật ngẫu nhiên các giá trị của các biến. Nhiều phiên bản của thuật toán tồn tại.

là "tập hợp các mệnh đề trong đó biến X_i xuất hiện dưới dạng nghĩa đen dương."

Nhiệm vụ này phức tạp bởi thực tế là các thuật toán chỉ quan tâm đến các mệnh đề chưa được thỏa mãn bởi các gán trước đó cho các biến, vì vậy các cấu trúc lập chỉ mục phải được cập nhật động khi quá trình tính toán diễn ra. Với những cải tiến này, các giải pháp hiện đại có thể xử lý các vấn đề với hàng chục triệu biến thể. Họ đã cách mạng hóa các lĩnh vực như xác minh phần cứng và xác minh giao thức bảo mật, trước đây đòi hỏi các bằng chứng thủ công, được hướng dẫn bằng tay.

7.6.2 Thuật toán tìm kiếm cục bộ

Chúng ta đã thấy một số thuật toán tìm kiếm cục bộ cho đến nay trong cuốn sách này, bao gồm HILL-CLIMBING (trang 129) và SIMULATED-ANNEALING (trang 133). Các thuật toán này có thể được áp dụng trực tiếp cho các vấn đề về khả năng thỏa mãn, miễn là chúng ta chọn đúng chức năng đánh giá. Bởi vì mục tiêu là tìm ra một nhiệm vụ thỏa mãn mọi mệnh đề, một hàm đánh giá đếm số lượng mệnh đề không thỏa mãn sẽ thực hiện công việc. Trên thực tế, đây chính xác là thước đo được sử dụng bởi thuật toán MIN-CONFLICTS cho CSP (trang 182). Tất cả các thuật toán này thực hiện các bước trong không gian của các nhiệm vụ hoàn chỉnh, lật giá trị chân lý của một biểu tượng tại một thời điểm. Không gian thường chứa nhiều cực nhỏ cục bộ, để thoát khỏi đó cần có nhiều hình thức ngẫu nhiên khác nhau. Trong những năm gần đây, đã có rất nhiều thử nghiệm để tìm ra sự cân bằng tốt giữa tham lam và ngẫu nhiên.

Một trong những thuật toán đơn giản và hiệu quả nhất xuất hiện từ tất cả các công việc này được gọi là WALKSAT (Hình 7.18). Trên mỗi lần lặp, thuật toán chọn một mệnh đề không thỏa mãn và chọn một ký hiệu trong mệnh đề để lật. Nó chọn ngẫu nhiên giữa hai cách để chọn biểu tượng nào để lật: (1) bước "xung đột tối thiểu" giảm thiểu số lượng mệnh đề không thỏa mãn trong trạng thái mới và (2) bước "đi bộ ngẫu nhiên" chọn biểu tượng một cách ngẫu nhiên.

Khi WALKSAT trả về một mô hình, câu đầu vào thực sự có thể thỏa mãn, nhưng khi nó trả về thất bại, có hai nguyên nhân có thể xảy ra: câu không thỏa mãn hoặc chúng ta cần cho thuật toán thêm thời gian. Nếu chúng ta đặt số lần lật tối đa $= \infty$ và $p > 0$, WALKSAT cuối cùng sẽ trả về một mô hình (nếu có), bởi vì các bước đi bộ ngẫu nhiên cuối cùng sẽ chạm vào

giải pháp. Than ôi, nếu lật tối đa là vô hạn và câu không thỏa mãn, thì thuật toán không bao giờ kết thúc!

Vì lý do này, WALKSAT hữu ích nhất khi chúng ta mong đợi một giải pháp tồn tại - ví dụ, các vấn đề được thảo luận trong Chương 3 và 5 thường có giải pháp. Mặt khác, WALKSAT không phải lúc nào cũng phát hiện ra sự không thỏa mãn, điều này là cần thiết để quyết định liên quan. Ví dụ, một tác nhân không thể sử dụng WALKSAT một cách đáng tin cậy để chứng minh rằng một hình vuông là an toàn trong thế giới wumpus. Thay vào đó, nó có thể nói, "Tôi đã suy nghĩ về nó trong một giờ và không thể nghĩ ra một thế giới có thể xảy ra trong đó quảng trường không an toàn." Đây có thể là một chỉ số thực nghiệm tốt cho thấy hình vuông là an toàn, nhưng nó chắc chắn không phải là bằng chứng.

7.6.3 Bối cảnh của các bài SAT ngẫu nhiên

Một số bài SAT khó hơn những bài khác. Các vấn đề dễ dàng có thể được giải quyết bằng bất kỳ thuật toán cũ nào, nhưng vì chúng ta biết rằng SAT là NP-complete, ít nhất một số trường hợp vẫn đề phải yêu cầu thời gian chạy theo cấp số nhân. Trong Chương 5, chúng ta đã thấy một số khám phá đáng ngạc nhiên về một số loại vấn đề. Ví dụ, bài toán n-queens - được cho là khá phức tạp đối với các thuật toán tìm kiếm theo đối ngẫu - hóa ra lại dễ dàng đối với các phương pháp tìm kiếm cục bộ, chẳng hạn như xung đột min. Điều này là do các giải pháp được phân bố rất dày đặc trong không gian phân công và bất kỳ nhiệm vụ ban đầu nào cũng được đảm bảo có giải pháp gần đó. Do đó, n-queens là dễ dàng vì nó bị hạn chế thấp.

Khi chúng ta xem xét các vấn đề về khả năng thỏa mãn ở dạng chuẩn liên kết, một vấn đề thiếu ràng buộc là một vấn đề có tương đối ít mệnh đề ràng buộc các biến. Ví dụ: đây là câu 3-CNF được tạo ngẫu nhiên với năm ký hiệu và năm mệnh đề:

$$(\neg D \vee \neg B \vee C) \wedge (B \vee \neg A \vee \neg C) \wedge (\neg C \vee \neg B \vee \neg D \vee B) \wedge (B \vee E \vee \neg C) \wedge (E \vee D \vee B) \wedge (B \vee E \vee \neg C)$$

Mười sáu trong số 32 bài tập có thể là mô hình của câu này, vì vậy, trung bình, chỉ cần hai lần đoán ngẫu nhiên để tìm ra một mô hình. Đây là một vấn đề dễ dàng thỏa mãn, cũng như hầu hết các vấn đề thiếu hạn chế như vậy. Mặt khác, một bài toán quá hạn chế có nhiều mệnh đề liên quan đến số biến và có khả năng không có lời giải. Các vấn đề bị hạn chế quá mức thường dễ giải quyết, bởi vì các ràng buộc nhanh chóng dẫn đến một giải pháp hoặc đến ngõ cụt mà không có lối thoát.

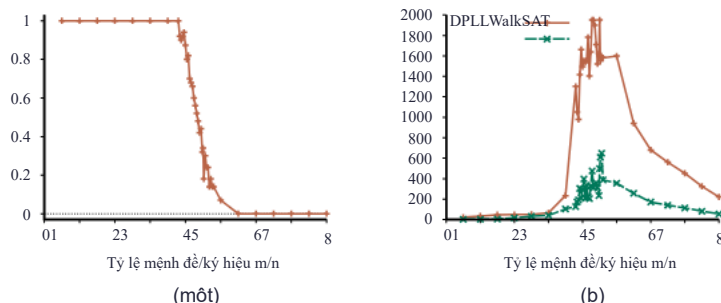
Để vượt ra ngoài những trực giác cơ bản này, chúng ta phải xác định chính xác cách các câu ngẫu nhiên được tạo ra. Ký hiệu CNFk(m, n) biểu thị một câu k-CNF với mệnh đề m và n ký hiệu, trong đó các mệnh đề được chọn đồng nhất, độc lập và không thay thế từ tất cả các mệnh đề có k ký tự khác nhau, dương hoặc âm ngẫu nhiên. (Một ký hiệu không được xuất hiện hai lần trong một mệnh đề, cũng như một mệnh đề không được xuất hiện hai lần trong một câu.)

Với một nguồn các câu ngẫu nhiên, chúng ta có thể đo lường xác suất thỏa mãn. Hình 7.19 (a) vẽ xác suất cho CNF3(m, 50), nghĩa là các câu có 50 biến và 3 ký tự cho mỗi mệnh đề, như một hàm của tỷ lệ mệnh đề / ký hiệu, m / n. Như chúng ta mong đợi, đối với small m/n, xác suất thỏa mãn gần bằng 1 và ở m/n lớn, xác suất gần bằng

0. Xác suất giảm khá mạnh xung quanh m / n = 4,3. Theo kinh nghiệm, chúng ta thấy rằng "vách đá" ở cùng một vị trí (đối với k = 3) và ngày càng sắc nét hơn khi n tăng lên. Về mặt lý thuyết, phỏng đoán ngưỡng thỏa mãn nói rằng cứ k ≥ 3, có một tỷ lệ ngưỡng rk sao cho khi n đi đến vô cực, xác suất CNFk(rn, n) có thể thỏa mãn trở thành 1 cho tất cả các giá trị của r dưới ngưỡng và 0 cho tất cả các giá trị ở trên. Phỏng đoán vẫn được chứng minh, ngay cả đối với các trường hợp đặc biệt như k = 3. Cho dù đó có phải là một định lý hay không, loại này

Thiếu hạn chế

Phỏng đoán ngưỡng thỏa mãn



Hình 7.19 (a) Biểu đồ cho thấy xác suất mà một câu 3-CNF ngẫu nhiên với $n = 50$ ký hiệu là thỏa mãn, như một hàm của tỷ lệ mệnh đề/ký hiệu m/n . (b) Biểu đồ thời gian chạy trung bình (được đo bằng số lần lặp) cho cả DPLL và WALKSAT trên các câu 3-CNF ngẫu nhiên. Các bài toán khó nhất có tỷ lệ mệnh đề/ký hiệu khoảng 4,3.

của hiệu ứng ngưỡng chắc chắn là phổ biến, đối với các vấn đề thỏa mãn cũng như các loại vấn đề NP-cứng khác.

Bây giờ chúng ta đã có một ý tưởng tốt về những vấn đề có thể thỏa mãn và không thỏa mãn ở đâu, câu hỏi tiếp theo là, những vấn đề khó ở đâu? Nó chỉ ra rằng chúng cũng thường ở giá trị ngưỡng. Hình 7.19 (b) cho thấy các bài toán 50 ký hiệu ở giá trị ngưỡng 4,3 khó giải hơn khoảng 20 lần so với các bài toán ở tỷ lệ 3,3. Các vấn đề thiếu hạn chế là dễ giải quyết nhất (vì nó rất dễ đoán một giải pháp); Những vấn đề bị hạn chế quá mức không dễ dàng như những vấn đề bị hạn chế thấp, nhưng vẫn dễ dàng hơn nhiều so với những vấn đề ngay ở ngưỡng cửa.

7.7 Tác nhân dựa trên logic mệnh đề

Trong phần này, chúng tôi tập hợp những gì chúng tôi đã học được cho đến nay để xây dựng các tác nhân wumpusworld sử dụng logic mệnh đề. Bước đầu tiên là cho phép tác nhân suy luận, trong phạm vi có thể, trạng thái của thế giới dựa trên lịch sử nhận thức của nó. Điều này đòi hỏi phải viết ra mô hình logic hoàn chỉnh về tác động của hành động. Sau đó, chúng tôi chỉ ra cách suy luận logic có thể được sử dụng bởi một tác nhân trong thế giới wumpus. Chúng tôi cũng chỉ ra cách tác nhân có thể theo dõi thế giới một cách hiệu quả mà không cần quay trở lại lịch sử nhận thức cho mỗi suy luận. Cuối cùng, chúng tôi chỉ ra cách tác nhân có thể sử dụng suy luận logic để xây dựng các kế hoạch được đảm bảo đạt được mục tiêu của nó, miễn là cơ sở kiến thức của nó đúng trong thế giới thực.

7.7.1 Tình trạng hiện tại của thế giới

Như đã nêu ở đầu chương, một tác nhân logic hoạt động bằng cách suy ra những gì cần làm từ cơ sở kiến thức của các câu về thế giới. Cơ sở tri thức bao gồm các tiên đề - kiến thức chung về cách thế giới hoạt động - và các câu nhận thức thu được từ kinh nghiệm của tác nhân trong một thế giới cụ thể. Trong phần này, chúng ta tập trung vào vấn đề suy ra trạng thái hiện tại của thế giới wumpus - tôi đang ở đâu, hình vuông đó có an toàn không, v.v.

Chúng tôi bắt đầu thu thập các tiên đề trong Phần 7.4.3. Tác nhân biết rằng ô vuông bắt đầu không chứa hổ ($\neg P1,1$) và không có wumpus ($\neg W1,1$). Hơn nữa, đối với mỗi ô vuông, nó biết rằng hình vuông là gió nếu và chỉ khi một hình vuông lân cận có một hổ; và một hình vuông có mùi nếu và chỉ khi một quảng trường lân cận có wumpus. Do đó, chúng tôi bao gồm một bộ sưu tập lớn các câu của hình thức sau:

$B1,1 \Leftrightarrow (P1,2 \vee P2,1) S1,1 \Leftrightarrow (W1,2 \vee W2,1) \dots$ Đặc vụ cũng biết rằng có chính xác một wumpus. Điều này được thể hiện trong hai phần. Đầu tiên, chúng ta phải nói rằng có ít nhất một wumpus:

$W1,1 \vee W1,2 \vee \dots \vee W4,3 \vee W4,4$. Vậy thì chúng ta phải nói rằng nhiều nhất là một wumpus. Đối với mỗi cặp địa điểm, chúng tôi thêm một câu nói rằng ít nhất một trong số chúng phải không có wumpus:

$\neg W1,1 \vee \neg W1,2 \vee \neg W1,3 \vee \dots \vee \neg W4,3 \vee \neg W4,4$. Càng xa càng tốt. Bây giờ chúng ta hãy xem xét nhận thức của tác nhân. Chúng tôi đang sử dụng $S1,1$ để có nghĩa là có mùi hôi thối trong $[1,1]$; chúng ta có thể sử dụng một mệnh đề duy nhất, Mùi hôi thối để có nghĩa là tác nhân nhận thức được mùi hôi thối không? Thật không may, chúng ta không thể: nếu không có mùi hôi thối ở bước thời gian trước đó, thì hôi thối sẽ được khẳng định, và khẳng định mới sẽ chỉ đơn giản dẫn đến một từ ngữ trái ngược. Vấn đề được giải quyết khi chúng ta nhận ra rằng một tri giác khẳng định điều gì đó chỉ về thời gian hiện tại. Do đó, nếu bước thời gian (như được cung cấp cho MAKE-PERCEPT-SENTENCE trong Hình 7.1) là 4, thì chúng ta thêm $Stench4$ vào cơ sở tri thức, thay vì $Stench$ — tránh được một cách gọn gàng bất kỳ mâu thuẫn nào với $\neg Stench3$. Điều tương tự cũng xảy ra với làn gió, va chạm, lấp lánh và tiếng hét.

Ý tưởng liên kết các mệnh đề với các bước thời gian mở rộng đến bất kỳ khía cạnh nào của thế giới thay đổi theo thời gian. Ví dụ: cơ sở kiến thức ban đầu bao gồm $L01,1$ — tác nhân là insquare $[1, 1]$ tại thời điểm 0 — cũng như $FacingEast0$, $HaveArrow0$ và $WumpusAlive0$. Chúng tôi sử dụng thenoun thông thạo (từ tiếng Latin fluens, flowing) để chỉ một khía cạnh của thế giới thay đổi.

"Thông thạo" là một từ đồng nghĩa với "biến trạng thái", theo nghĩa được mô tả trong cuộc thảo luận về các biểu diễn thừa số trong Phần 2.4.7 trên trang 76. Các biểu tượng liên quan đến các khía cạnh vĩnh viễn của thế giới không cần chỉ số trên thời gian và đôi khi được gọi là biến tạm thời.

Chúng ta có thể kết nối trực tiếp các nhận thức hôi thối và gió với các thuộc tính của các hình vuông mà chúng được trải nghiệm như sau.¹¹ Đối với bất kỳ bước thời gian nào t và bất kỳ hình vuông nào $[x, y]$, chúng ta khẳng định

$Ltx, y \Rightarrow (Breezet \Leftrightarrow Bx, y) Ltx, y \Rightarrow (Stencht \Leftrightarrow Sx, y)$. Bây giờ, tất nhiên, chúng ta cần các tiên đề cho phép tác nhân theo dõi các thông thạo như Ltx,y . Những thông thạo này thay đổi do các hành động được thực hiện bởi tác nhân, vì vậy, trong thuật ngữ của Chương 3, chúng ta cần viết ra mô hình chuyển tiếp của thế giới wumpus dưới dạng một tập hợp các câu logic.

¹¹ Mục 7.4.3 đã che giấu yêu cầu này một cách thuận tiện.

Đầu tiên chúng ta cần các biểu tượng mệnh đề cho sự xuất hiện của các hành động. Cũng như tri giác, những biểu tượng này được lập chỉ mục theo thời gian; do đó, Forward0 có nghĩa là tác nhân thực hiện Forwardaction tại thời điểm 0. Theo quy ước, nhận thức cho một bước thời gian nhất định xảy ra đầu tiên, tiếp theo là hành động cho bước thời gian đó, tiếp theo là chuyển tiếp sang bước thời gian tiếp theo.

Để mô tả thế giới thay đổi như thế nào, chúng ta có thể thử viết các tiên đề hiệu ứng chỉ định kết quả của một hành động ở bước thời gian tiếp theo. Ví dụ: nếu tác nhân đang ở vị trí [1, 1] quay mặt về phía đông tại thời điểm 0 và đi về phía trước, kết quả là tác nhân nằm trong hình vuông [2, 1] và không còn ở trong [1, 1]:

$$L01,1 \wedge \text{FacingEast0} \wedge \text{Forward0} \Rightarrow (L12,1 \wedge \neg L11,1) \quad (7.1)$$

Chúng ta sẽ cần một câu như vậy cho mỗi bước thời gian có thể, cho mỗi trong số 16 ô vuông, và mỗi trong bốn hướng. Chúng ta cũng sẽ cần các câu tương tự cho các hành động khác: Grab, Shoot, Climb, TurnLeft và TurnRight.

Giả sử rằng tác nhân quyết định di chuyển về phía trước tại thời điểm 0 và khẳng định thực tế này vào cơ sở kiến thức của nó. Với tiên đề hiệu ứng trong Phương trình (7.1), kết hợp với các khẳng định ban đầu về trạng thái tại thời điểm 0, tác nhân bây giờ có thể suy ra rằng nó nằm trong [2, 1]. Đó là, $\text{ASK}(\text{KB}, L12,1) = \text{đúng}$. Càng xa càng tốt. Thật không may, nếu chúng ta $\text{ASK}(\text{KB}, \text{HaveArrow1})$, câu trả lời là sai, nghĩa là, tác nhân không thể chứng minh rằng nó vẫn còn mũi tên; nó cũng không thể chứng minh nó không có nó! Thông tin đã bị mất vì tiên đề hiệu ứng không nêu rõ những gì còn lại đã thay đổi như là kết quả của một hành động. Sự cần thiết phải làm điều này làm nảy sinh vấn đề khung.

Một giải pháp khả thi cho vấn đề khung là thêm các tiên đề khung khẳng định rõ ràng tất cả các mệnh đề vẫn giữ nguyên. Ví dụ, đối với mỗi lần t chúng ta sẽ có $\text{Forward} \Rightarrow (\text{HaveArrow}t \Leftrightarrow \text{HaveArrow}t+1) \text{Forward}t \Rightarrow (\text{WumpusAlive}t \Leftrightarrow \text{WumpusAlive}t+1)$. Trong đó chúng tôi đề cập rõ ràng mọi mệnh đề không thay đổi theo thời gian t đến thời gian $t + 1$ trong hành động Chuyển tiếp. Mặc dù tác nhân bây giờ biết rằng nó vẫn còn mũi tên sau khi di chuyển về phía trước và wumpus chưa chết hoặc sống lại, sự gia tăng của các tiên đề khung dường như không hiệu quả đáng kể. Trong một thế giới có m hành động khác nhau và n trôi chảy, tập hợp các tiên đề khung sẽ có kích thước $O(mn)$. Biểu hiện cụ thể của vấn đề khung đôi khi được gọi là vấn đề khung đại diện. Vấn đề đã chơi một

vai trò quan trọng trong lịch sử AI; Chúng tôi khám phá thêm nó trong các ghi chú ở cuối chương.

Vấn đề khung đại diện rất quan trọng vì thế giới thực có rất nhiều thông thái, nói một cách nhẹ nhàng. May mắn cho con người chúng ta, mỗi hành động thường thay đổi không quá một số ít k trong số những người trôi chảy đó - thế giới thể hiện tính cục bộ. Giải quyết vấn đề

bài toán khung hình yêu cầu xác định mô hình chuyển tiếp với một tập hợp các tiên đề có kích thước $O(mk)$ chứ không phải kích thước $O(mn)$. Ngoài ra còn có một vấn đề khung suy luận: vấn đề

dự đoán về phía trước kết quả của kế hoạch hành động bước t trong thời gian O (kt) thay vì O (kt). Đại pháp cho vấn đề liên quan đến việc thay đổi trọng tâm của một người từ viết tiên đề về hành động sang viết tiên đề về trôi chảy. Do đó, đối với mỗi F trôi chảy, chúng ta sẽ có một tiên đề xác định giá trị chân thật của F_{t+1} theo thông thạo (bao gồm cả bản thân F) tại thời điểm t và các hành động có thể đã xảy ra tại thời điểm t . Bây giờ, giá trị chân lý của F_{t+1} có thể được đặt theo một trong hai cách:

12 Cái tên "bài toán khung" xuất phát từ "khung tham chiếu" trong vật lý - nền đứng yên giả định đối với chuyển động được đo. Nó cũng có một sự tương tự với các khung hình của một bộ phim, trong đó thông thường hầu hết hậu cảnh không đổi trong khi những thay đổi xảy ra ở tiền cảnh.

hành động tại thời điểm t làm cho F đúng tại $t + 1$, hoặc F đã đúng tại thời điểm t và hành động tại thời điểm t không làm cho nó sai. Một tiên đề ở dạng này được gọi là tiên đề nhà nước kế thừa và có dạng này:

$Ft + 1 \Leftrightarrow \text{ActionCauses} Ft \vee (ft \wedge \neg \text{ActionCausesNot} Ft)$. Một trong những tiên đề trạng thái kế thừa đơn giản nhất là tiên đề dành cho HaveArrow. Bởi vì không có hành động nào để tải lại, phần ActionCausesFt sẽ biến mất và chúng ta còn lại

$$\text{HaveArrow}t + 1 \Leftrightarrow (\text{HaveArrow}t \wedge \neg \text{Shoot}t) \quad (7.2)$$

Đối với vị trí của đặc vụ, các tiên đề nhà nước kế thừa phức tạp hơn. Ví dụ: Lt + 11, 1 là đúng nếu (a) tác nhân di chuyển về phía trước từ [1, 2] khi quay mặt về phía nam hoặc từ [2, 1] khi quay mặt về phía tây; hoặc (b) Lt1, 1 đã đúng và hành động không gây ra chuyển động (vì hành động không phải là Phía trước hoặc vì hành động va vào tường). Được viết ra trong logic mệnh đề, điều này trở thành

$$\begin{aligned} \text{LT}_{11.1}^+ &\Leftrightarrow (\text{Lt}1, 1 \wedge (\neg \text{Forward}t \vee \text{Bump}t + \\ &\quad 1)) \vee (\text{Lt}1, 2 \wedge (\text{FacingSouth}t \wedge \text{Forward}t \\ &\quad \vee (\text{Lt}2, 1 \wedge (\text{FacingWest}t \wedge \text{Forward}t))) \end{aligned} \quad (7.3)$$

Bài tập 7.SSAX yêu cầu bạn viết ra các tiên đề cho những người thông thạo thế giới wumpus còn lại.

Với một tập hợp đầy đủ các tiên đề trạng thái kế thừa và các tiên đề khác được liệt kê ở phần đầu của phần này, tác nhân sẽ có thể HỎI và trả lời bất kỳ câu hỏi có thể trả lời được về tình trạng hiện tại của thế giới. Ví dụ, trong Phần 7.2, trình tự ban đầu của các nhận thức và hành động là

¬Hỏi thối0 ∧ ¬Gió0 ∧ ¬Lấp lánh0 ∧ ¬Bump0 ∧ ¬Scream0 ;
Forward0¬Hỏi thối1 ∧ Breeze1 ∧ ¬Glitter1 ∧ ¬Bump1 ∧ ¬Scream1 ; Rẽ
phải1
Hỏi thối2 ∧ Breeze2 ∧ ¬Glitter2 ∧ ¬Bump2 ∧ ¬Scream2 ;
TurnRight2¬Hỏi thối3 ∧ Breeze3 ∧ ¬Glitter3 ∧ ¬Bump3 ∧ ¬Scream3 ;
Forward3¬Hỏi thối4 ∧ ¬Breeze4 ∧ ¬Glitter4 ∧ ¬Bump4 ∧ ¬Scream4 ;
TurnRight4¬Hỏi thối5 ∧ ¬Breeze5 ∧ ¬Glitter5 ∧ ¬Bump5 ∧ ¬Scream5 ;

Chuyển tiếp 5
Mùi hôi thối6 ∧ Breeze6 ∧ ¬Glitter6 ∧ ¬Bump6 ∧ ¬Scream6 Tại thời điểm này, chúng ta có $\text{ASK}(\text{KB}, \text{L61}, 2) = \text{true}$, vì vậy tác nhân biết nó ở đâu. Hơn nữa, $\text{ASK}(\text{KB}, \text{W1}, 3) = \text{true}$ và $\text{ASK}(\text{KB}, \text{P3}, 1) = \text{true}$, vì vậy tác nhân đã tìm thấy wumpus và một trong các hố. Câu hỏi quan trọng nhất đối với tác nhân là liệu một hình vuông có ổn để di chuyển vào hay không - nghĩa là, liệu hình vuông có không có hố hay wumpus sống hay không. Thật thuận tiện để thêm các tiên đề cho việc này, có biểu

thức
 $\text{OK}x, y \Leftrightarrow \neg Px, y \wedge \neg (Wx, y \wedge \text{WumpusAlivet})$. Cuối cùng, $\text{ASK}(\text{KB}, \text{OK62}, 2) = \text{true}$, vì vậy ở vuông [2, 2] có thể di chuyển vào. Trên thực tế, với thuật toán suy luận hợp lý và hoàn chỉnh như DPLL, tác nhân có thể trả lời bất kỳ câu hỏi nào có thể trả lời về hình vuông nào là OK — và có thể làm như vậy chỉ trong vài mili giây đối với thế giới wumpus vừa và nhỏ.

Giải quyết các vấn đề khung đại diện và suy luận là một bước tiến lớn, nhưng vấn đề nguy hiểm vẫn còn: chúng ta cần xác nhận rằng tất cả các điều kiện tiên quyết cần thiết của anaction giữ cho nó có tác dụng dự định của nó. Chúng tôi đã nói rằng hành động Forward di chuyển tác nhân

phía trước trừ khi có một bức tường cản đường, nhưng có nhiều trường hợp ngoại lệ bất thường khác có thể khiến hành động thất bại: đặc vụ có thể vấp ngã và bị đau tim, bị rơi không lồ mang đi, v.v. Chỉ định tất cả các ngoại lệ này được gọi là bài toán đủ điều kiện.

Vấn đề trình độ

Không có giải pháp hoàn chỉnh trong logic; Các nhà thiết kế hệ thống phải sử dụng phán đoán tốt để quyết định mức độ chi tiết mà họ muốn xác định mô hình của họ và những chi tiết họ muốn bỏ qua. Chúng ta sẽ thấy trong Chương 12 rằng lý thuyết xác suất cho phép chúng ta tóm tắt tất cả các ngoại lệ mà không cần nêu tên chúng một cách rõ ràng.

7.7.2 Một tác nhân lai

Khả năng suy ra các khía cạnh khác nhau của trạng thái thế giới có thể được kết hợp khá thẳng thắn với các quy tắc điều kiện-hành động (xem Phần 2.4.2) và với các thuật toán giải quyết vấn đề từ Chương 3 và 4 để tạo ra một tác nhân lai cho thế giới wumpus. Hình 7.20

Tác nhân lai

cho thấy một cách khả thi để làm điều này. Chương trình đại lý duy trì và cập nhật cơ sở kiến thức cũng như kế hoạch hiện tại. Cơ sở kiến thức ban đầu chứa các tiên đề phi thời gian — những tiên đề không phụ thuộc vào t , chẳng hạn như tiên đề liên quan đến sự gió mát của hình vuông với sự hiện diện của các hố. Tại mỗi bước thời gian, cầu nhận thức mới được thêm vào cùng với tất cả các tiên đề phụ thuộc vào t , chẳng hạn như các tiên đề trạng thái kế thừa. (Phần tiếp theo giải thích lý do tại sao tác nhân không cần tiên đề cho các bước thời gian trong tương lai.) Sau đó, tác nhân sử dụng suy luận logic, bằng cách hỏi các câu hỏi của cơ sở tri thức, để tìm ra ô vuông nào an toàn và ô vuông nào chưa được truy cập.

Nội dung chính của chương trình đại lý xây dựng một kế hoạch dựa trên mức độ ưu tiên giảm dần của mục tiêu. Đầu tiên, nếu có ánh sáng lấp lánh, chương trình xây dựng một kế hoạch để lấy vàng, đi theo một tuyến đường trở lại vị trí ban đầu và treo ra khỏi hang. Nếu không, nếu không có kế hoạch hiện tại, chương trình sẽ lập kế hoạch cho một tuyến đường đến quảng trường an toàn gần nhất mà nó chưa ghé thăm, đảm bảo tuyến đường chỉ đi qua các ô vuông an toàn.

Lập kế hoạch tuyến đường được thực hiện bằng tìm kiếm A^* chứ không phải với ASK. Nếu không có ô vuông an toàn để khám phá, bước tiếp theo — nếu đặc vụ vẫn còn mũi tên — là cố gắng tạo ra một ô vuông an toàn bằng cách bắn tại một trong những vị trí wumpus có thể có. Chúng được xác định bằng cách hỏi $\text{whereASK}(KB, \neg Wx, y)$ là sai - nghĩa là, nơi không biết rằng không có wumpus. Chức năng PLAN-SHOT (không hiển thị) sử dụng PLAN-ROUTE để lập kế hoạch một chuỗi các hành động sẽ sắp xếp ảnh này. Nếu điều này không thành công, chương trình sẽ tìm kiếm một hình vuông để khám phá mà không có thể chứng minh được là không an toàn — nghĩa là một hình vuông mà $\text{ASK}(KB, \neg \text{OKtx}, y)$ trả về false. Nếu không có hình vuông như vậy, thì nhiệm vụ là không thể và đặc vụ rút lui về $[1, 1]$ và treo ra khỏi hang.

7.7.3 Ước tính trạng thái logic

Chương trình tác nhân trong Hình 7.20 hoạt động khá tốt, nhưng nó có một điểm yếu lớn: khi thời gian trôi qua, chi phí tính toán liên quan đến các cuộc gọi đến ASK tăng lên. Điều này xảy ra chủ yếu bởi vì các suy luận cần thiết phải quay trở lại ngày càng xa hơn theo thời gian và liên quan đến ngày càng nhiều ký hiệu mệnh đề. Rõ ràng, điều này là không bền vững - chúng ta không thể có chất độc nhân mà thời gian để xử lý mỗi tri giác tăng lên tương ứng với độ dài tuổi thọ của nó! Những gì chúng ta thực sự cần là thời gian cập nhật liên tục - nghĩa là, độc lập với t . Câu trả lời rõ ràng là lưu, hoặc lưu vào bộ nhớ đệm, kết quả suy luận, để quá trình suy luận ở bước thời gian tiếp theo có thể

Caching

Xây dựng dựa trên kết quả của các bước trước đó thay vì phải bắt đầu lại từ đầu.

Như chúng ta đã thấy trong Phần 4.4, lịch sử của tri giác và tất cả các phân nhánh của chúng có thể được đặt lại bởi trạng thái niềm tin — nghĩa là, một số đại diện của tập hợp tất cả các trạng thái hiện tại có thể xảy ra

hàm HYBRID-WUMPUS-AGENT(percept) trả về một hành động

đầu vào: nhận thức, một danh sách, [hồi thối, gió, lấp lánh, va chạm, la hét] dai dẳng: KB, một cơ sở kiến thức, ban đầu là "vật lý wumpus" tạm thời

t, một bộ đếm, ban đầu là 0, biểu thị kế hoạch thời gian, một chuỗi hành động, ban đầu trống

TELL(KB, MAKE-PERCEPT-SENTENCE(percept, t))
TELL KB các câu "vật lý" thời gian cho thời gian
tsafe $\leftarrow \{[x, y] : \text{ASK}(\text{KB}, \text{OKtx}, y) = \text{true}\}$ if $\text{ASK}(\text{KB}, \text{Glittert}) = \text{true}$ thì

kế hoạch $\leftarrow [\text{Grab}] + \text{PLAN-ROUTE}(\text{current}, \{[1, 1]\}, \text{safe}) +$
[Climb] nếu kế hoạch trống thì không được truy cập $\leftarrow \{[x, y] : \text{ASK}(\text{KB}, \text{Lt}'x, y) = \text{false} \text{ cho tất cả } t' \leq t\}$

kế hoạch $\leftarrow \text{PLAN-ROUTE}(\text{hiện tại}, \text{chưa được truy cập} \cap \text{an toàn}, \text{an toàn})$ nếu kế hoạch trống và $\text{ASK}(\text{KB}, \text{HaveArrowt}) = \text{true}$ thì

wumpus có thể $\leftarrow \{[x, y] : \text{ASK}(\text{KB}, \neg \text{Wx}, y) = \text{false}\}$ plan $\leftarrow$
PLAN-SHOT(current, possible wumpus, safe) if plan is empty
thì// không còn lựa chọn nào khác ngoài việc chấp nhận rủi ro
not unsafe $\leftarrow \{[x, y] : \text{ASK}(\text{KB}, \neg \text{OKtx}, y) = \text{false}\}$ plan

$\leftarrow \text{PLAN-ROUTE}(\text{hiện tại}, \text{chưa được truy cập} \cap$
không an toàn, an toàn) nếu kế hoạch trống thì
plan $\leftarrow \text{PLAN-ROUTE}(\text{current}, \{[1, 1]\}, \text{safe}) +$
[Climb] action $\leftarrow \text{POP}(\text{plan})$ TELL(KB, MAKE-
ACTION-SENTENCE(action, t)) t $\leftarrow t + 1$ return
action

hàm PLAN-ROUTE(current, goals, allowed) trả về một chuỗi hành động

Đầu vào: hiện tại, vị trí hiện tại của tác nhân

mục tiêu, một tập hợp các ô vuông; Cố gắng lên kế hoạch cho một tuyến đường đến một trong số
chúng được phép, một tập hợp các ô vuông có thể tạo thành một phần của tuyến đường

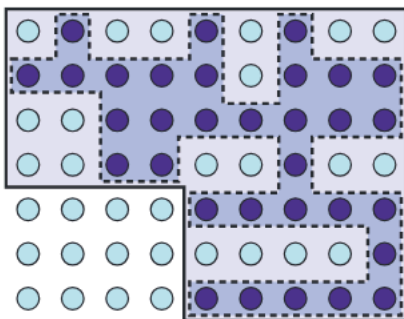
problem $\leftarrow \text{ROUTE-PROBLEM}(\text{current}, \text{goals}, \text{allowed})$ return SEARCH(problem)// Bất kỳ
thuật toán tìm kiếm nào từ Chương 3

Hình 7.20 Một chương trình tác nhân lai cho thế giới wumpus. Nó sử dụng cơ sở kiến thức mệnh đề để suy ra trạng thái của thế giới và kết hợp tìm kiếm giải quyết vấn đề và mã miền cụ thể để chọn hành động. Mỗi khi HYBRID-WUMPUS-AGENT được gọi, nó thêm nhận thức vào cơ sở tri thức, và sau đó dựa vào một kế hoạch đã xác định trước đó hoặc tạo ra một kế hoạch mới, và bật ra bước đầu tiên của kế hoạch như hành động cần làm tiếp theo.

của thế giới.¹³ Quá trình cập nhật trạng thái niềm tin khi nhận thức mới xuất hiện được gọi là ước tính trạng thái (xem trang 150). Trong khi trong Phần 4.4, trạng thái niềm tin là một danh sách rõ ràng các trạng thái, ở đây chúng ta có thể sử dụng một câu logic liên quan đến các ký hiệu mệnh đề liên quan đến bước thời gian hiện tại, cũng như các biểu tượng không thời gian. Ví dụ, câu logic

$$\text{WumpusAlive1} \wedge \text{L12,1} \wedge \text{B2,1} \wedge (\text{P3,1} \vee \text{P2,2}) \quad (7.4)$$

¹³ Chúng ta có thể nghĩ về bản thân lịch sử tri giác như một đại diện của trạng thái niềm tin, nhưng một điều khiến suy luận ngày càng tốn kém khi lịch sử trở nên dài hơn.



Hình 7.21 Mô tả trạng thái niềm tin 1-CNF (đường viền đậm) như một sự xấp xỉ bảo thủ, có thể đại diện đơn giản cho trạng thái niềm tin chính xác (lắc lư) (vùng bóng mờ với đường viền đứt nét). Mỗi thể giới có thể được hiển thị dưới dạng một vòng tròn; những cái bóng mờ phù hợp với tất cả các nhận thức.

đại diện cho tập hợp của tất cả các trạng thái tại thời điểm 1 trong đó wumpus còn sống, tác nhân ở [2, 1], hình vuông đó là gió và có một hố trong [3, 1] hoặc [2, 2] hoặc cả hai.

Duy trì một trạng thái niềm tin chính xác như một công thức logic hóa ra không phải là điều dễ dàng. Nếu có n ký hiệu trôi chảy cho thời gian t , thì có $2n$ trạng thái có thể có — nghĩa là gán các giá trị chân lý cho các ký hiệu đó. Bây giờ, tập hợp các trạng thái niềm tin là tập lũy thừa (tập hợp của tất cả các tập hợp con) của tập hợp các trạng thái vật lý. Có $2n$ trạng thái vật lý, do đó có 2^{2n} trạng thái niềm tin. Ngay cả khi chúng ta sử dụng mã hóa các công thức logic nhỏ gọn nhất có thể, với mỗi trạng thái niềm tin được biểu thị bởi một số nhị phân duy nhất, chúng ta sẽ cần các số có $\log_2(2^{2n}) = 2n$ bit để dán nhãn trạng thái niềm tin hiện tại. Nghĩa là, ước tính trạng thái chính xác có thể yêu cầu các công thức logic có kích thước theo cấp số nhân về số lượng ký hiệu.

Một sơ đồ rất phổ biến và tự nhiên để ước tính trạng thái gần đúng là biểu diễn các trạng thái niềm tin dưới dạng liên từ của nghĩa đen, tức là công thức 1-CNF. Để làm điều này, chương trình tác nhân chỉ đơn giản cố gắng chứng minh X_t và $\neg X_t$ cho mỗi ký hiệu X_t (cũng như mỗi ký hiệu không thời gian mà giá trị chân lý chưa được biết đến), với trạng thái niềm tin tại $t - 1$. Sự kết hợp của các nghĩa đen chứng minh trở thành trạng thái niềm tin mới, và trạng thái niềm tin trước đó bị loại bỏ.

Điều quan trọng là phải hiểu rằng kế hoạch này có thể mất một số thông tin theo thời gian. Ví dụ, nếu câu trong Phương trình (7.4) là trạng thái niềm tin thực sự, thì cả $P_{3,1}$ và $P_{2,2}$ sẽ không thể chứng minh được riêng lẻ và cả hai sẽ không xuất hiện trong trạng thái niềm tin 1-CNF. (Bài tập 7.HYBR khám phá một giải pháp khả thi cho vấn đề này.) Mặt khác, bởi vì mọi nghĩa đen trong trạng thái niềm tin 1-CNF đều được chứng minh từ trạng thái niềm tin trước đó, và trạng thái niềm tin ban đầu là một khẳng định đúng, chúng ta biết rằng toàn bộ trạng thái niềm tin 1-CNF phải đúng. Do đó, tập hợp các trạng thái có thể được đại diện bởi trạng thái niềm tin 1-CNF bao gồm tất cả các trạng thái J_m mà trên thực tế là có thể với lịch sử nhận thức đầy đủ. Như minh họa trong Hình 7.21, 1-

Trạng thái niềm tin CNF hoạt động như một phong bì bên ngoài đơn giản, hoặc xấp xỉ bảo thủ, xung quanh

Xấp xỉ bảo thủ

trạng thái niềm tin chính xác. Chúng tôi thấy ý tưởng về sự xấp xỉ thận trọng cho các tập hợp phức tạp này là chủ đề lặp đi lặp lại trong nhiều lĩnh vực của AI.

hàm SATPLAN(init, transition, goal, T max) trả về giải pháp hoặc thất bại
 đầu vào: init, transition, goal, tạo thành mô tả vấn đề
 T max, giới hạn trên cho độ dài kế hoạch

đối với $t = 0$ đến T max do
 cnf $\leftarrow$ TRANSLATE-TO-SAT(init, transition, goal,
 t) model $\leftarrow$ SAT-SOLVER(cnf) nếu model không rỗng thì

trả về EXTRACT-SOLUTION (model) trả lại thất bại

Hình 7.22 Thuật toán SATPLAN. Vấn đề lập kế hoạch được chuyển thành cảm giác CNF trong đó mục tiêu được khẳng định là giữ ở một bước thời gian cố định t và các tiên đề được bao gồm cho mỗi bước thời gian lên t . Nếu thuật toán khả năng thỏa mãn tìm thấy một mô hình, thì một kế hoạch được trích xuất bằng cách xem xét các ký hiệu mệnh đề đề cập đến các hành động và được gán đúng trong mô hình. Nếu không có mô hình nào tồn tại, thì quá trình này sẽ được lặp lại với mục tiêu được di chuyển một bước sau đó.

7.7.4 Lập kế hoạch bằng suy luận mệnh đề

Tác nhân trong Hình 7.20 sử dụng suy luận logic để xác định hình vuông nào an toàn, nhưng sử dụng tìm kiếm * A để lập kế hoạch. Trong phần này, chúng tôi chỉ ra cách lập kế hoạch bằng suy luận logic. Ý tưởng cơ bản rất đơn giản:

1. Xây dựng một câu bao gồm (a) Init0, một tập hợp các khẳng định về trạng thái ban đầu; (b) Chuyển tiếp₁, . . . , Chuyển tiếp_t, các tiên đề trạng thái kế thừa cho tất cả các hành động có thể xảy ra tại mỗi thời điểm cho đến một thời gian tối đa t ; (c) khẳng định rằng mục tiêu đã đạt được tại thời điểm t : HaveGoldt $\wedge$ ClimbedOutt .
2. Trình bày toàn bộ câu cho người giải SAT. Nếu người giải tìm thấy một mô hình thỏa mãn, thì mục tiêu có thể đạt được; Nếu câu không thỏa mãn, thì vấn đề không thể giải quyết được.
3. Giả sử một mô hình được tìm thấy, trích xuất từ mô hình những biến đại diện cho hoạt động và được gán đúng. Cùng nhau, chúng đại diện cho một kế hoạch để đạt được các mục tiêu. Một quy trình lập kế hoạch mệnh đề, SATPLAN, được thể hiện trong Hình 7.22. Nó thực hiện ý tưởng cơ bản vừa được đưa ra, với một bước ngoặt. Bởi vì tác nhân không biết sẽ mất bao nhiêu bước để đạt được mục tiêu, thuật toán sẽ thử từng bước có thể có t , lên đến một số chiều dài kế hoạch tối đa có thể tưởng tượng được Tmax. Bằng cách này, nó được đảm bảo để tìm thấy kế hoạch ngắn nhất nếu một người tồn tại. Do cách SATPLAN tìm kiếm giải pháp, cách tiếp cận này không thể được sử dụng trong một môi trường có thể quan sát được một phần; SATPLAN sẽ chỉ đặt các biến không thể quan sát được thành các giá trị cần thiết để tạo ra một giải pháp. Bước quan trọng trong việc sử dụng SATPLAN là xây dựng cơ sở tri thức. Có vẻ như, khi kiểm tra tình cờ, các tiên đề thế giới wumpus trong Mục 7.7.1 đủ cho các bước 1 (a) và 1 (b) ở trên. Tuy nhiên, có một sự khác biệt đáng kể giữa các yêu cầu về yêu cầu (như được kiểm tra bởi ASK) và các yêu cầu về khả năng đáp ứng. Ví dụ, hãy xem xét vị trí của tác nhân, ban đầu là [1, 1], và giả sử mục tiêu đơn giản của tác nhân là ở trong [2, 1] tại thời điểm 1. Cơ sở kiến thức ban đầu chứa L01,1 và mục tiêu là L12,1. Sử dụng ASK, chúng ta có thể chứng minh L12,1 nếu Forward0 được khẳng định, và yên tâm thay, chúng ta không thể

chứng minh L12,1 nếu, giả sử, Shoot0 được khẳng định thay thế. Bây giờ, SATPLAN sẽ tìm thấy kế hoạch [Forward0]; Càng xa càng tốt.

Thật không may, SATPLAN cũng tìm thấy kế hoạch [Shoot0]. Làm thế nào điều này có thể xảy ra? Để tìm hiểu, chúng tôi kiểm tra mô hình mà SATPLAN xây dựng: nó bao gồm nhiệm vụ L02,1, nghĩa là tác nhân có thể ở [2, 1] tại thời điểm 1 bằng cách ở đó tại thời điểm 0 và bắn. Người ta có thể hỏi, "Không phải chúng ta đã nói rằng tác nhân đang ở trong [1, 1] tại thời điểm 0 sao?" Vâng, chúng tôi đã làm, nhưng chúng tôi đã không nói với nhân viên rằng nó không thể ở hai nơi cùng một lúc! Đối với entailment, L02,1 là không rõ và do đó, không thể được sử dụng làm bằng chứng; mặt khác, L02,1 là không xác định và do đó, có thể được đặt thành bất kỳ giá trị nào giúp làm cho mục tiêu trở thành sự thật.

SATPLAN là một công cụ gỡ rối tốt cho cơ sở tri thức vì nó tiết lộ những nơi thiếu kiến thức. Trong trường hợp cụ thể này, chúng ta có thể sửa chữa cơ sở kiến thức bằng cách khẳng định rằng, tại mỗi bước thời gian, tác nhân ở chính xác một vị trí, sử dụng một tập hợp các câu tương tự như những câu được sử dụng để khẳng định sự tồn tại của chính xác một wumpus. Ngoài ra, chúng ta có thể khẳng định $\neg L0x,y$ cho tất cả các vị trí khác ngoài [1, 1]; tiên đề trạng thái kế thừa cho vị trí quan tâm đến các bước thời gian tiếp theo. Các bản sửa lỗi tương tự cũng hoạt động để đảm bảo tổng đài viên có một và chỉ một định hướng tại một thời điểm.

Tuy nhiên, SATPLAN có nhiều điều bất ngờ hơn trong cửa hàng. Đầu tiên là nó tìm thấy các mô hình có hành động bất khả thi, chẳng hạn như bắn mà không có mũi tên. Để hiểu tại sao, chúng ta cần xem xét cẩn thận hơn những gì các tiên đề trạng thái kế thừa (chẳng hạn như Phương trình (7.3)) nói về các hành động mà các điều kiện tiên quyết không được thỏa mãn. Các tiên đề dự đoán chính xác rằng sẽ không có gì xảy ra khi một hành động như vậy được thực hiện (xem Bài tập 7.SATP), nhưng chúng không nói rằng hành động đó không thể được thực hiện! Để tránh tạo ra các kế hoạch với các hành động bất hợp pháp, chúng ta phải thêm các tiên đề điều kiện tiên quyết

Tiên đề điều kiện tiên quyết

nói rằng một hành động xảy ra đòi hỏi các điều kiện tiên quyết phải được thỏa mãn.¹⁴ Ví dụ, chúng ta cần nói, đối với mỗi lần t , rằng

Shoot t $\Rightarrow$ HaveArrow t .

Điều này đảm bảo rằng nếu một kế hoạch chọn hành động Bắn vào bất kỳ lúc nào, đó phải là trường hợp đặc vụ có một mũi tên tại thời điểm đó.

Điều ngạc nhiên thứ hai của SATPLAN là việc tạo ra các kế hoạch với nhiều hành động đồng thời. Ví dụ: nó có thể đưa ra một mô hình trong đó cả Forward0 và Shoot0 đều đúng, điều này không được phép. Để loại bỏ vấn đề này, chúng tôi giới thiệu các tiên đề loại trừ hành động: cho

Loại trừ hành động tiên đề

mỗi cặp hành động A_i và A_j chúng ta thêm tiên

đề $\neg A_i \vee \neg A_j$. Có thể chỉ ra rằng đi về phía trước và bắn cùng một lúc không quá khó để thực hiện, trong khi đó, chẳng hạn, bắn và nắm bắt cùng một lúc là khá không thực tế. Bằng cách áp đặt các tiên đề loại trừ hành động chỉ trên các cặp hành động thực sự can thiệp vào nhau, chúng ta có thể cho phép các kế hoạch bao gồm nhiều hành động đồng thời — và bởi vì SATPLAN tìm thấy kế hoạch pháp lý ngắn nhất, chúng ta có thể chắc chắn rằng nó sẽ tận dụng khả năng này.

Tóm lại, SATPLAN tìm các mô hình cho một câu chứa trạng thái ban đầu, mục tiêu, tiên đề trạng thái kế nhiệm, tiên đề tiên điều kiện và tiên đề loại trừ hành động. Có thể chứng minh rằng bộ sưu tập các tiên đề này là đủ, theo nghĩa là không còn bất kỳ "giải pháp" giả mạo nào nữa. Bất kỳ mô hình nào thỏa mãn câu mệnh đề sẽ là một

¹⁴ Lưu ý rằng việc bổ sung các tiên đề điều kiện có nghĩa là chúng ta không cần phải bao gồm các điều kiện tiên quyết cho các hành động trong các tiên đề trạng thái kế thừa.

kế hoạch hợp lệ cho vấn đề ban đầu. Công nghệ giải SAT hiện đại làm cho cách tiếp cận này trở nên khá thiết thực. Ví dụ, một bộ giải kiểu DPLL không gặp khó khăn trong việc tạo ra giải pháp cho thực thể thế giới wumpus được hiển thị trong Hình 7.2.

Phần này đã mô tả một cách tiếp cận khai báo để xây dựng tác nhân: tác nhân hoạt động bằng sự kết hợp của các câu khẳng định trong cơ sở tri thức và thực hiện suy luận logic. Cách tiếp cận này có một số điểm yếu ẩn trong các cụm từ như "cho mỗi lần t" và "cho mỗi ô vuông $[x, y]$ ". Đối với bất kỳ tác nhân thực tế nào, các cụm từ này phải được thực hiện bằng mã tạo ra các trường hợp của lược đồ câu chung tự động để chèn vào cơ sở tri thức. Đối với một thế giới wumpus có kích thước hợp lý - một thế giới có thể so sánh với một trò chơi máy tính nhỏ - chúng ta có thể cần một 100×100 bảng và 1000 bước thời gian, dẫn đến các cơ sở kiến thức với hàng chục hoặc hàng trăm triệu câu.

Điều này không chỉ trở nên khá không thực tế, mà nó còn minh họa một vấn đề sâu sắc hơn: chúng ta biết một điều gì đó về thế giới wumpus – cụ thể là, "vật lý" hoạt động giống nhau trên tất cả các ô vuông và tất cả các bước thời gian – mà chúng ta không thể diễn tả trực tiếp trong ngôn ngữ logic mệnh đề. Để giải quyết vấn đề này, chúng ta cần một ngôn ngữ biểu cảm hơn, một ngôn ngữ trong đó các cụm từ như "cho mỗi thời gian t" và "cho mỗi hình vuông $[x, y]$ " có thể được viết một cách tự nhiên. Logic bậc nhất, được mô tả trong Chương 8, là một ngôn ngữ như vậy; Trong logic bậc nhất, một thế giới wumpus ở bất kỳ kích thước và thời gian nào cũng có thể được mô tả trong khoảng mười câu logic thay vì mười triệu hoặc mười nghìn tỷ.

Tóm tắt

Chúng tôi đã giới thiệu các tác nhân dựa trên tri thức và đã chỉ ra cách xác định một logic mà các tác nhân đó có thể suy luận về thế giới. Những điểm chính như sau:

- Các tác nhân thông minh cần kiến thức về thế giới để đưa ra quyết định đúng đắn.
- Kiến thức được chứa trong các tác nhân dưới dạng câu trong ngôn ngữ đại diện tri thức được lưu trữ trong cơ sở tri thức.
- Một tác nhân dựa trên tri thức bao gồm một cơ sở tri thức và một cơ chế suy luận. Nó hoạt động bằng cách lưu trữ các câu về thế giới trong cơ sở kiến thức của nó, sử dụng cơ chế suy luận để suy ra các câu mới và sử dụng những câu này để quyết định hành động cần thực hiện.
- Một ngôn ngữ biểu diễn được xác định bởi cú pháp của nó, xác định cấu trúc của câu và ngữ nghĩa của nó, xác định sự thật của mỗi câu trong mỗi thế giới hoặc mô hình có thể.
- Mỗi quan hệ của sự bao gồm giữa các câu là rất quan trọng đối với sự hiểu biết của chúng ta về lý luận. Một câu α đòi hỏi một câu khác β nếu β là đúng trong tất cả các thế giới mà α là đúng. Các định nghĩa tương đương bao gồm tính hợp lệ của câu $\alpha \Rightarrow \beta$ và tính không thỏa mãn của câu $\alpha \wedge \neg \beta$.
- Suy luận là quá trình rút ra các câu mới từ những câu cũ. Thuật toán suy luận âm thanh chỉ rút ra các câu kèm theo; Các thuật toán hoàn chỉnh rút ra tất cả các câu.
- Logic mệnh đề là một ngôn ngữ đơn giản bao gồm các ký hiệu mệnh đề và các liên kết logic. Nó có thể xử lý các mệnh đề được biết là đúng, được biết là sai, hoặc hoàn toàn không biết.

Tập hợp các mô hình khả thi, được cho một từ vựng mệnh đề cố định, là hữu hạn, vì vậy entailment có thể được kiểm tra bằng cách liệt kê các mô hình. Kiểm tra mô hình hiệu quả Các thuật toán suy luận cho logic mệnh đề bao gồm các phương pháp tìm kiếm cục bộ và thường có thể giải quyết các vấn đề lớn một cách nhanh chóng.

- Quy tắc suy luận là các mẫu suy luận hợp lý có thể được sử dụng để tìm bằng chứng. Đó quy tắc giải pháp mang lại một thuật toán suy luận hoàn chỉnh cho các cơ sở kiến thức được biểu thị ở dạng chuẩn liên kết. Forward chaining và backward chaining là các thuật toán suy luận rất tự nhiên cho các cơ sở tri thức ở dạng Horn.
 - Các phương pháp tìm kiếm cục bộ như WALKSAT có thể được sử dụng để tìm giải pháp. Những thuật toán như vậy là hợp lý nhưng không hoàn chỉnh.
 - Ước tính trạng thái logic liên quan đến việc duy trì một câu logic mô tả tập hợp các trạng thái có thể phù hợp với lịch sử quan sát. Mỗi bước cập nhật yêu cầu suy luận bằng cách sử dụng mô hình chuyển tiếp của môi trường, được xây dựng từ các tiên đề trạng thái kế thừa chỉ định cách mỗi thay đổi thông loát.
 - Các quyết định trong một tác nhân logic có thể được thực hiện bằng cách giải quyết SAT: tìm các mô hình khả thi xác định các chuỗi hành động trong tương lai đạt được mục tiêu. Cách tiếp cận này chỉ hoạt động trong môi trường có thể quan sát được hoặc không có cảm biến.
- Logic mệnh đề không mở rộng cho các môi trường có kích thước không giới hạn vì nó thiếu sức mạnh biểu đạt để xử lý ngắn gọn với thời gian, không gian và các mô hình phổ quát của mối quan hệ giữa các đối tượng.

Ghi chú thư mục và lịch sử

Bài báo của John McCarthy "Các chương trình với ý thức chung" (McCarthy, 1958, 1968) đã đưa ra khái niệm về các tác nhân sử dụng lý luận logic để làm trung gian giữa nhận thức và hành động. Nó cũng giương cao lá cờ của chủ nghĩa tuyên bố, chỉ ra rằng nói với một tác nhân những gì nó cần biết là một cách thanh lịch để xây dựng phần mềm. Bài báo "Mức độ tri thức" của Allen Newell (1982) đưa ra trường hợp rằng các tác nhân hợp lý có thể được mô tả và phân tích ở mức độ trừu tượng được xác định bởi kiến thức mà họ sở hữu hơn là các chương trình mà họ chạy.

Bản thân logic có nguồn gốc từ triết học và toán học Hy Lạp cổ đại. Plato đã thảo luận về cấu trúc có pháp của câu, sự thật và sai lầm của chúng, ý nghĩa của chúng và tính hợp lệ của các lập luận logic. Nghiên cứu có hệ thống đầu tiên được biết đến về logic là Organon của Aristotle. Thuyết mô học là những gì bây giờ chúng ta gọi là quy tắc suy luận, mặc dù chúng thiếu tính thành phần

Tam đoạn luận

của các quy tắc hiện tại của chúng tôi.

Các trường phái Megarian và Khắc kỷ bắt đầu nghiên cứu có hệ thống về các kết nối logic cơ bản vào thế kỷ thứ năm trước Công nguyên. Bảng sự thật là do Philo của Megara. Những người theo chủ nghĩa khắc kỷ đã lấy năm quy tắc suy luận cơ bản là hợp lệ mà không cần bằng chứng, bao gồm cả quy tắc mà ngày nay chúng ta gọi là Modus Ponens. Họ đã rút ra một số quy tắc khác từ năm quy tắc này, sử dụng, trong số các nguyên tắc khác, định lý suy diễn (trang 240) và rõ ràng hơn về chứng minh so với Aristotle (Mates, 1953).

Ý tưởng giảm suy luận logic thành một quá trình cơ học thuần túy là do Wilhelm Leibniz (1646–1716). George Boole (1847) đã giới thiệu hệ thống logic hình thức toàn diện và có thể thực hiện được đầu tiên trong cuốn sách của mình Phân tích toán học của logic. Logic của Boole được mô hình hóa chặt chẽ trên đại số thông thường của các số thực và sử dụng sự thay thế các biểu thức tương đương logi-cally làm phương pháp suy luận chính của nó. Mặc dù nó không xử lý tất cả

logic mệnh đề, các nhà toán học khác nhanh chóng lấp đầy những mảnh còn thiếu. Schröder (1877) mô tả dạng bình thường liên kết, trong khi dạng Sừng được giới thiệu muộn hơn nhiều bởi Alfred Horn (1951). Sự giải thích toàn diện đầu tiên về logic mệnh đề hiện đại (và logic bậc nhất) được tìm thấy trong Begriffsschrift ("Viết khái niệm" hoặc "Ký hiệu nhận thức" của Gottlob Frege (1879).

Thiết bị cơ khí đầu tiên thực hiện các suy luận logic là Stanhope Demonstrator, được chế tạo bởi Bá tước Stanhope thứ ba (1753–1816). William Stanley Jevons, một trong những nhà toán học đã mở rộng công trình của Boole, đã xây dựng "piano logic" của mình vào năm 1869 để làm các suy luận trong logic Boolean. Một lịch sử thú vị của các thiết bị suy luận cơ học ban đầu này được đưa ra bởi Martin Gardner (1968). Các chương trình máy tính đầu tiên để suy luận logic là chương trình chứng minh trong số học Presburger năm 1954 của Martin Davis (Davis, 1957), và nhà lý thuyết logic của Newell, Shaw và Simon (1957).

Emil Post (1921) và Ludwig Wittgenstein (1922) đã sử dụng độc lập các bảng sự thật như một phương pháp kiểm tra tính hợp lệ của các câu logic mệnh đề. Thuật toán Davis-Putnam (Davis và Putnam, 1960) là thuật toán đầu tiên để giải quyết mệnh đề và thuật toán ngược DPLL chưa được chứng minh (Davis et al., 1962) đã được chứng minh là hiệu quả hơn. Quy tắc giải pháp và bằng chứng về tính đầy đủ của nó đã được phát triển một cách tổng quát đầy đủ cho logic bậc nhất bởi J. A. Robinson (1965).

Stephen Cook (1971) đã chỉ ra rằng việc quyết định khả năng thỏa mãn của một câu trong mệnh đề luận (bài toán SAT) là NP-complete. Nhiều tập hợp con của logic mệnh đề được biết đến mà vấn đề thỏa mãn có thể giải quyết được về mặt đa tử; Mệnh đề kèn là một trong những tập hợp con như vậy.

Các nghiên cứu ban đầu cho thấy DPLL có độ phức tạp trường hợp trung bình đa thức đối với phân bố tự nhiên của các bài toán. Thậm chí tốt hơn, Franco và Paull (1983) đã chỉ ra rằng các vấn đề tương tự có thể được giải quyết trong thời gian liên tục chỉ bằng cách đoán các nhiệm vụ ngẫu nhiên. Được thúc đẩy bởi sự thành công thực nghiệm của tìm kiếm cục bộ, Koutsoupias và Papadimitriou (1992) đã chỉ ra rằng một thuật toán leo đồi đơn giản có thể giải quyết hầu hết các vấn đề thỏa mãn rất nhanh, cho thấy rằng các vấn đề khó là rất hiếm. Schöningh (1999) đã thể hiện thuật toán leo đồi ngẫu nhiên có thời gian chạy dự kiến trong trường hợp xấu nhất trên các bài toán 3-SAT là $O(1.333n)$ — vẫn theo cấp số nhân, nhưng nhanh hơn đáng kể so với giới hạn trường hợp xấu nhất trước đó. Kỷ lục hiện tại là $O(1.32216n)$ (Rolf, 2006).

Tăng hiệu quả trong các giải mệnh đề đã diễn ra nhanh chóng. Với mười phút tính toán, thuật toán DPLL ban đầu trên phần cứng năm 1962 chỉ có thể giải quyết các vấn đề với 10 hoặc 15 biến (trên máy tính xách tay năm 2019 sẽ là khoảng 30 biến). Đến năm 1995, bộ giải SATZ (Lland Anbulagan, 1997) có thể xử lý 1.000 biến, nhờ cấu trúc dữ liệu được tối ưu hóa cho các biến lập chỉ mục. Hai đóng góp quan trọng là kỹ thuật lập chỉ mục theo nghĩa đen theo dõi

Xem theo nghĩa đen

của Zhang và Stickel (1996), làm cho việc truyền bá đơn vị rất hiệu quả và giới thiệu các kỹ thuật học mệnh đề (tức là ràng buộc) từ cộng đồng CSP của Bayardo và Schrag (1997). Sử dụng những ý tưởng này và được thúc đẩy bởi triển vọng giải quyết các vấn đề xác minh mạch quy mô công nghiệp, Moskewicz et al. (2001) đã phát triển trình giải CHAFF, có thể xử lý các vấn đề với hàng triệu biến. Bắt đầu từ năm 2002, các cuộc thi SAT hàng năm đã được tổ chức; hầu hết các bài dự thi chiến thắng đều là các biến thể của CHAFF. Cảnh quan của các bộ giải được khảo sát bởi Gomes et al. (2008).

Các thuật toán tìm kiếm cục bộ để thỏa mãn đã được thử nghiệm bởi nhiều tác giả khác nhau suốt những năm 1980, dựa trên ý tưởng giảm thiểu số lượng mệnh đề không thỏa mãn (Hansen và Jau-mard, 1990). Một thuật toán đặc biệt hiệu quả đã được phát triển bởi Gu (1989) và indepen-

bởi Selman et al. (1992), người đã gọi nó là GSAT và chỉ ra rằng nó có khả năng giải quyết một loạt các vấn đề rất khó rất nhanh chóng. Thuật toán WALKSAT được mô tả trong chương này là do Selman et al. (1996).

"Chuyển pha" trong khả năng thỏa mãn các vấn đề k-SAT ngẫu nhiên lần đầu tiên được quan sát bởi Simon và Dubois (1989) và đã làm phát sinh rất nhiều nghiên cứu lý thuyết và thực nghiệm - một phần do mối liên hệ với các hiện tượng chuyển pha trong vật lý thống kê. Crawford và Auton (1993) đã xác định vị trí chuyển đổi 3-SAT ở tỷ lệ mệnh đề / biến số khoảng

4.26, lưu ý rằng điều này trùng với một đỉnh cao trong thời gian chạy của bộ giải SAT của họ. Cookand Mitchell (1997) cung cấp một bản tóm tắt tuyệt vời về các tài liệu ban đầu về vấn đề này. Các thuật toán như lan truyền khảo sát (Parisi và Zecchina, 2002; Maneva và cộng sự, 2007)

Tuyên truyền khảo sát

tận dụng các thuộc tính đặc biệt của các phiên bản SAT ngẫu nhiên gần ngưỡng khả năng thỏa mãn và vượt trội hơn rất nhiều so với các trình giải SAT chung trên các trường hợp như vậy. Tình trạng hiện tại của sự hiểu biết lý thuyết được tóm tắt bởi Achlioptas (2009).

Các nguồn thông tin tốt về khả năng thỏa mãn, cả lý thuyết và thực tiễn, bao gồm Sổ tay về khả năng thỏa mãn (Biere và cộng sự, 2009), cuốn sách của Donald Knuth (2015) về sự hài lòng, và các Hội nghị Quốc tế thường xuyên về Lý thuyết và Ứng dụng của Kiểm tra khả năng thỏa mãn, được gọi là SAT.

Ý tưởng xây dựng các tác nhân với logic mệnh đề có thể bắt nguồn từ bản thân của McCulloch và Pitts (1943), nổi tiếng với việc khởi xướng lĩnh vực hoạt động mạng thần kinh, nhưng thực sự quan tâm đến việc thực hiện một thiết kế tác nhân dựa trên mạch Boolean trong não. Stan Rosenschein (Rosenstein, 1985; Kaelbling và Rosenstein, 1990) đã phát triển các cách để biên dịch các tác nhân dựa trên mạch từ các mô tả khai báo về môi trường nhiệm vụ. Rod Brooks (1986, 1989) chứng minh hiệu quả của các thiết kế dựa trên mạch để điều khiển robot (xem Chương 26). Brooks (1991) lập luận rằng các thiết kế dựa trên mạch là tất cả những gì cần thiết cho AI - rằng sự thể hiện và lý luận là công kênh, tốn kém và không cần thiết. Theo quan điểm của chúng tôi, cả lý luận và mạch đều cần thiết. Williams et al. (2003) mô tả một tác nhân lai - không quá khác biệt so với tác nhân wumpus của chúng ta - điều khiển tàu vũ trụ của NASA, lập kế hoạch các chuỗi hành động và chẩn đoán và phục hồi sau các lỗi.

Vấn đề chung của việc theo dõi một môi trường có thể quan sát được một phần đã được giới thiệu cho các biểu diễn dựa trên nhà nước trong Chương 4. Sự khởi tạo của nó cho các representation mệnh đề đã được nghiên cứu bởi Amir và Russell (2003), những người đã xác định một số lớp môi trường chấp nhận các thuật toán ước tính trạng thái hiệu quả và chỉ ra rằng đối với một số lớp khác, vấn đề là khó giải quyết. Vấn đề phóng chiếu thời gian, liên quan đến việc răn đe

Chiều thời gian

khai thác những mệnh đề nào đúng sau khi một chuỗi hành động được thực hiện, có thể được coi là một trường hợp đặc biệt của ước tính trạng thái với các nhận thức trống rỗng. Nhiều tác giả đã nghiên cứu vấn đề này vì tầm quan trọng của nó trong việc lập kế hoạch; một số kết quả độ cứng quan trọng đã được thiết lập bởi Liberatore (1997). Ý tưởng đại diện cho một trạng thái niềm tin với các mệnh đề có thể bắt nguồn từ Wittgenstein (1922).

Cách tiếp cận ước tính trạng thái logic bằng cách sử dụng các chỉ số thời gian trên các biến thể mệnh đề đã được đề xuất bởi Kautz và Selman (1992). Các thể hệ sau của SATPLAN đã có thể tận dụng những tiến bộ trong các giải SAT và vẫn là một trong những cách hiệu quả nhất để giải quyết các vấn đề lập kế hoạch khó khăn (Kautz, 2006).

Vấn đề khung lần đầu tiên được McCarthy và Hayes (1969) nhận ra. Nhiều người tìm kiếm lại cho rằng vấn đề này không thể giải quyết được trong logic bậc nhất, và nó đã thúc đẩy rất nhiều nghiên cứu về logic không đơn điệu. Các nhà triết học từ Dreyfus (1972) đến Crockett

(1994) đã trích dẫn vấn đề khung như một triệu chứng của sự thất bại không thể tránh khỏi của toàn bộ doanh nghiệp AI. Giải pháp cho vấn đề khung với các tiên đề trạng thái kế thừa là do RayReiter (1991). Thielscher (1999) xác định vấn đề khung suy luận như một ý tưởng riêng biệt và cung cấp một giải pháp. Nhìn lại, người ta có thể thấy rằng các tác nhân của Rosenschein (1985) đã sử dụng các mạch thực hiện các tiên đề trạng thái kế nhiệm, nhưng Rosenschein không nhận thấy rằng vấn đề khung do đó đã được giải quyết phần lớn.

Các giải mệnh đề hiện đại đã được áp dụng cho nhiều ứng dụng công nghiệp, chẳng hạn như tổng hợp phần cứng máy tính (Nowick et al., 1993). SATMC: satisfiabilitychecker được sử dụng để phát hiện một lỗi hổng chưa được biết đến trước đây trong giao thức đăng nhập trình duyệt Web (Armando et al., 2008).

Thế giới wumpus được phát minh như một trò chơi bởi Gregory Yob (1975). Trớ trêu thay, Yob phát triển nó vì anh ấy cảm thấy nhàm chán với các trò chơi được chơi trên một lưới hình chữ nhật: anh ấy đặt mũ của mình lên một hình tứ diện, và chúng tôi đặt nó trở lại lưới cũ nhàm chán. Michael Genesereth đề nghị rằng thế giới wumpus được sử dụng như một thử nghiệm đặc vụ.

CHƯƠNG 8

LOGIC BẠC NHẤT

Trong đó chúng ta nhận thấy rằng thế giới được ban phước với nhiều đối tượng, một số trong số đó có liên quan đến các đối tượng khác, và trong đó chúng ta cố gắng lý luận về chúng.

Logic mệnh đề đủ để minh họa các khái niệm cơ bản về logic, suy luận và các tác nhân dựa trên tri thức. Thật không may, logic mệnh đề bị giới hạn trong những gì nó có thể nói. Trong chương này, chúng ta xem xét logic bậc nhất, 1 có thể thể hiện một cách ngắn gọn nhiều hơn thế. Chúng tôi bắt đầu trong

Logic bậc nhất

Mục 8.1 với thảo luận về các ngôn ngữ đại diện nói chung; Phần 8.2 bao gồm cú pháp và ngữ nghĩa của logic bậc nhất; Phần 8.3 và 8.4 minh họa việc sử dụng logic bậc nhất cho các biểu diễn đơn giản.

8.1 Đại diện được xem xét lại

Trong phần này, chúng ta thảo luận về bản chất của ngôn ngữ biểu diễn. Ngôn ngữ lập trình (chẳng hạn như C++ hoặc Java hoặc Python) là lớp ngôn ngữ chính thức lớn nhất được sử dụng phổ biến. Cấu trúc dữ liệu trong chương trình có thể được sử dụng để thể hiện sự thật; Ví dụ, một chương trình có thể sử dụng mảng 4×4 để đại diện cho nội dung của thế giới Wumpus. Do đó, câu lệnh ngôn ngữ lập trình `World[2,2] ← Pit` là một cách khá tự nhiên để khẳng định rằng có một hố hình vuông [2,2]. Tập hợp một chuỗi các tuyên bố như vậy là đủ để chạy một mô phỏng thế giới wumpus.

Những gì các ngôn ngữ lập trình thiếu là một cơ chế chung để rút ra sự thật từ các sự kiện khác; Mỗi cập nhật cấu trúc dữ liệu được thực hiện bởi một quy trình dành riêng cho miền mà các chi tiết được lập trình viên lấy từ kiến thức của chính họ về miền. Cách tiếp cận thủ tục-màng cứng này có thể tương phản với bản chất khai báo của logic mệnh đề, trong đó kiến thức và suy luận là riêng biệt, và suy luận hoàn toàn độc lập với lĩnh vực. SQL cơ sở dữ liệu có sự kết hợp giữa kiến thức khai báo và thủ tục.

Một nhược điểm thứ hai của cấu trúc dữ liệu trong các chương trình (và của cơ sở dữ liệu) là thiếu bất kỳ cách nào để dàng để nói, ví dụ, "Có một cái hồ trong [2,2] hoặc [3,1]" hoặc "Nếu wumpus ở trong [1,1] thì anh ta không ở trong [2,2]." Các chương trình có thể lưu trữ một giá trị duy nhất cho mỗi biến và một số hệ thống cho phép giá trị là "không xác định", nhưng chúng thiếu tính biểu đạt cần thiết để xử lý trực tiếp thông tin một phần.

Logic mệnh đề là một ngôn ngữ khai báo vì ngữ nghĩa của nó dựa trên mối quan hệ chân lý giữa các câu và các thế giới có thể. Nó cũng có đủ khả năng biểu đạt để xử lý thông tin từng phần, sử dụng sự tách rời và phủ định. Logic mệnh đề có một thuộc tính thứ ba được mong muốn trong các ngôn ngữ biểu diễn, cụ thể là tính thành phần. Trong một com-

Tính thành phần

ngôn ngữ vị trí, ý nghĩa của một câu là một chức năng của ý nghĩa của các bộ phận của nó.

Cho
† Logic bậc nhất còn được gọi là giải tích vị ngữ bậc nhất; nó có thể được viết tắt là FOL hoặc FOPL.

ví dụ, ý nghĩa của " $S_{1,4} \wedge S_{1,2}$ " có liên quan đến ý nghĩa của " $S_{1,4}$ " và " $S_{1,2}$ ". Sẽ rất kỳ lạ nếu " $S_{1,4}$ " có nghĩa là có mùi hôi thối ở ô vuông [1,4] và " $S_{1,2}$ " có nghĩa là có mùi hôi thối ở ô vuông [1,2], nhưng " $S_{1,4} \wedge S_{1,2}$ " có nghĩa là Pháp và Ba Lan hòa 1-1 trong trận đấu vòng loại khúc côn cầu trên băng tuần trước.

Tuy nhiên, logic mệnh đề, như một biểu diễn nhân số, thiếu sức mạnh biểu đạt để mô tả một cách ngắn gọn một môi trường với nhiều đối tượng. Ví dụ, chúng tôi buộc phải viết một quy tắc riêng về gió và hồ cho mỗi ô vuông, chẳng hạn như

$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$. Mặt khác, trong tiếng Anh, có vẻ đủ dễ dàng để nói, một lần và mãi mãi, "Các ô vuông liền kề với các hồ là gió mát." Cú pháp và ngữ nghĩa của tiếng Anh giúp bạn có thể mô tả môi trường một cách ngắn gọn: Tiếng Anh, giống như logic bậc nhất, là một biểu diễn có cấu trúc.

8.1.1 Ngôn ngữ của tư tưởng

Ngôn ngữ tự nhiên (chẳng hạn như tiếng Anh hoặc tiếng Tây Ban Nha) thực sự rất biểu cảm. Chúng tôi đã cố gắng viết gần như toàn bộ cuốn sách này bằng ngôn ngữ tự nhiên, chỉ thỉnh thoảng chuyển sang các ngôn ngữ khác (chủ yếu là toán học và sơ đồ). Có một truyền thống lâu đời trong ngôn ngữ học và triết học ngôn ngữ xem ngôn ngữ tự nhiên như một ngôn ngữ đại diện kiến thức khai báo. Nếu chúng ta có thể khám phá ra các quy tắc cho ngôn ngữ tự nhiên, chúng ta có thể sử dụng chúng các hệ thống đại diện và lý luận và đạt được lợi ích của hàng tỷ trang đã được viết bằng ngôn ngữ tự nhiên.

Quan điểm hiện đại về ngôn ngữ tự nhiên là nó đóng vai trò như một phương tiện để giao tiếp hơn là đại diện thuần túy. Khi một người nói chỉ và nói, "Nhìn kia!" người nghe biết rằng, chẳng hạn, Siêu nhân cuối cùng đã xuất hiện trên các mái nhà. Tuy nhiên, chúng ta sẽ không muốn nói rằng câu "Hãy nhìn!" đại diện cho sự thật đó. Thay vào đó, ý nghĩa của câu phụ thuộc vào cả bản thân câu và ngữ cảnh mà câu được nói. Rõ ràng, người ta không thể lưu trữ một câu như "Look!" trong một cơ sở kiến thức và mong đợi khôi phục ý nghĩa của nó mà không lưu trữ một biểu diễn của ngữ cảnh – điều này đặt ra câu hỏi làm thế nào bản thân ngữ cảnh có thể được đại diện.

Ngôn ngữ tự nhiên cũng bị mơ hồ, một vấn đề đối với ngôn ngữ đại diện. Như Pinker (1995) đã nói: "Khi mọi người nghĩ về mùa xuân, chắc chắn họ không nhằm lẫn liệu họ đang nghĩ về một mùa hay điều gì đó đang diễn ra - và nếu một từ có thể tương ứng với hai suy nghĩ, suy nghĩ không thể là từ ngữ."

Giả thuyết Sapir-Whorf nổi tiếng (Whorf, 1956) tuyên bố rằng sự hiểu biết của chúng ta về thế giới bị ảnh hưởng mạnh mẽ bởi ngôn ngữ chúng ta nói. Chắc chắn đúng là các cộng đồng ngôn ngữ khác nhau chia rẽ thế giới theo cách khác nhau. Người Pháp có hai từ "chaise" và "fauteuil", cho một khái niệm mà người nói tiếng Anh bao gồm một: "ghế". Nhưng những người nói tiếng Anh có thể dễ dàng nhận ra thể loại fauteuil và đặt cho nó một cái tên - đại khái là "ghế hồ tay" - vậy ngôn ngữ có thực sự tạo ra sự khác biệt không? Whorf chủ yếu dựa vào trực giác và suy đoán, và ý tưởng của ông phần lớn đã bị bác bỏ, nhưng trong những năm xen kẽ, chúng ta thực sự có dữ liệu thực tế từ các nghiên cứu nhân chủng học, tâm lý học và thần kinh.

Ví dụ, bạn có nhớ cụm từ nào trong hai cụm từ sau đây tạo thành phần mở đầu của Phần 8.1 không?

"Trong phần này, chúng tôi thảo luận về bản chất của ngôn ngữ đại diện. . . ""Phần này bao gồm chủ đề ngôn ngữ đại diện tri thức..."

Wanner (1974) đã thực hiện một thí nghiệm tương tự và phát hiện ra rằng các đối tượng đã đưa ra lựa chọn đúng ở mức độ ngẫu nhiên - khoảng 50% thời gian - nhưng nhớ nội dung của những gì họ đọc với độ chính xác hơn 90%. Điều này cho thấy rằng mọi người giải thích các từ mà họ đọc và hình thành một biểu diễn phi ngôn ngữ bên trong, và các từ chính xác không phải là hệ quả.

Thú vị hơn là trường hợp một khái niệm hoàn toàn vắng mặt trong một ngôn ngữ. Những người nói ngôn ngữ thổ dân Úc Guugu Yimithirr không có từ nào cho các hướng tương đối (hoặc lấy bản ngã làm trung tâm), chẳng hạn như trước, sau, phải hoặc trái. Thay vào đó, họ sử dụng các hướng tuyệt đối, ví dụ, tương đương với "Tôi bị đau ở cánh tay phía bắc của tôi." Sự khác biệt này trong ngôn ngữ tạo ra sự khác biệt trong hành vi: Người nói Guugu Yimithirr giỏi hơn trong việc điều hướng địa hình mở, trong khi người nói tiếng Anh giỏi hơn trong việc đặt nĩa ở bên phải đĩa.

Ngôn ngữ dường như cũng ảnh hưởng đến tư tưởng thông qua các tính năng ngữ pháp dường như tùy tiện như giới tính của danh từ. Ví dụ, "bridge" là nam tính trong tiếng Tây Ban Nha và fem-inine trong tiếng Đức. Boroditsky (2003) yêu cầu các đối tượng chọn tính từ tiếng Anh để mô tả một bức ảnh của một cây cầu cụ thể. Những người nói tiếng Tây Ban Nha chọn to lớn, nguy hiểm, mạnh mẽ và cao chót vót, trong khi người nói tiếng Đức chọn đẹp, thanh lịch, mỏng manh và mảnh mai.

Lời nói có thể đóng vai trò là điểm neo ảnh hưởng đến cách chúng ta nhận thức về thế giới. Loftus và Palmer (1974) cho các đối tượng thử nghiệm xem một bộ phim về một vụ tai nạn ô tô. Các đối tượng được hỏi "Những chiếc xe chạy nhanh như thế nào khi chúng liên lạc với nhau?" báo cáo tốc độ trung bình là 32 dặm/giờ, trong khi các đối tượng được hỏi câu hỏi với từ "đập phá" thay vì "liên lạc" báo cáo 41 dặm/giờ cho cùng một chiếc xe trong cùng một bộ phim. Nhìn chung, có những khác biệt có thể đo lường được nhưng nhỏ trong quá trình xử lý nhận thức của những người nói các ngôn ngữ khác nhau, nhưng không có bằng chứng thuyết phục nào cho thấy điều này dẫn đến sự khác biệt lớn trong thế giới quan.

Trong một hệ thống suy luận logic sử dụng dạng chuẩn liên kết (CNF), chúng ta có thể thấy rằng các dạng ngôn ngữ " $\neg(A \vee B)$ " và " $\neg A \wedge \neg B$ " giống nhau vì chúng ta có thể nhìn vào bên trong hệ thống và thấy rằng hai câu được lưu trữ dưới cùng một dạng CNF kinh điển. Nó bắt đầu có thể làm điều gì đó tương tự với bộ não con người. Mitchell et al. (2008) đưa các đối tượng vào một máy chụp cộng hưởng từ chức năng (fMRI), cho họ thấy các từ như "cần tây" và chụp ảnh não của họ. Một chương trình học máy được đào tạo trên các cặp (từ, hình ảnh) có thể dự đoán chính xác 77% thời gian về các nhiệm vụ lựa chọn nhị phân (ví dụ: "cần tây" hoặc "máy bay"). Hệ thống thậm chí có thể dự đoán ở mức độ cơ hội cao hơn cho các từ, nó chưa bao giờ thấy hình ảnh fMRI trước đây (bằng cách xem xét hình ảnh của các từ liên quan) và đối với những người mà nó chưa từng thấy trước đây (chứng minh rằng fMRI tiết lộ một số mức độ đại diện chung giữa mọi người). Loại công việc này vẫn còn sơ khai, nhưng fMRI (và các công nghệ hình ảnh khác như điện sinh lý nội sọ (Sahin et al., 2009)) hứa hẹn sẽ cung cấp cho chúng ta những ý tưởng cụ thể hơn nhiều về các đại diện tri thức của con người như thế nào.

Từ quan điểm của logic hình thức, biểu diễn cùng một kiến thức theo hai cách khác nhau hoàn toàn không tạo ra sự khác biệt; Các sự kiện tương tự sẽ có thể rút ra từ một cách hay sự phản đối. Tuy nhiên, trong thực tế, một đại diện có thể yêu cầu ít bước hơn để rút ra kết luận, có nghĩa là một nhà lý luận với nguồn lực hạn chế có thể đi đến kết luận bằng cách sử dụng một biểu diễn nhưng không phải biểu diễn kia. Đối với các nhiệm vụ không suy diễn như học hỏi từ kinh nghiệm, kết quả nhất thiết phụ thuộc vào hình thức của các biểu diễn được sử dụng. Chúng tôi chỉ ra trong Chương 19 rằng khi một chương trình học tập xem xét hai lý thuyết khả dĩ của thế giới, cả hai đều phù hợp với tất cả các dữ liệu, cách phổ biến nhất để phá vỡ mối quan hệ là chọn lý thuyết ngắn gọn nhất - và điều đó phụ thuộc vào ngôn ngữ được sử dụng để đại diện cho các lý thuyết. Do đó, ảnh hưởng của ngôn ngữ đối với suy nghĩ là không thể tránh khỏi đối với bất kỳ tác nhân nào học tập.

8.1.2 Kết hợp những gì tốt nhất của ngôn ngữ chính thức và ngôn ngữ tự nhiên

Chúng ta có thể áp dụng nền tảng của logic mệnh đề - một ngữ nghĩa tuyên bố, thành phần, độc lập với ngữ cảnh và rõ ràng - và xây dựng một logic biểu cảm hơn trên nền tảng đó, vay mượn các ý tưởng đại diện từ ngôn ngữ tự nhiên trong khi tránh những nhược điểm của nó. Khi chúng ta nhìn vào cú pháp của ngôn ngữ tự nhiên, các yếu tố rõ ràng nhất là danh từ và cụm danh từ đề cập đến các đối tượng (hình vuông, hổ, wumpuses) và động từ và cụm động từ

Đối tượng

Mối quan hệ

Chức năng

cùng với các tính từ và trạng từ đề cập đến mối quan hệ giữa các đối tượng (is breezy, is adjacent đến, chụp). Một số mối quan hệ này là các chức năng – các mối quan hệ trong đó chỉ có một "giá trị" cho một "đầu vào" nhất định. Thật dễ dàng để bắt đầu liệt kê các ví dụ về đối tượng, quan hệ và hàm:

Tài sản

- Đối tượng: con người, nhà cửa, con số, lý thuyết, Ronald McDonald, màu sắc, trò chơi bóng chày, chiến tranh, thế kỷ. . .
- Quan hệ: đây có thể là các quan hệ đơn nguyên hoặc các thuộc tính như đỏ, tròn, già, nguyên tố, nhiều tầng. . . , hoặc các mối quan hệ tổng quát hơn như anh em của, lớn hơn, bên trong, một phần của, có màu sắc, xảy ra sau, sở hữu, đến giữa, . . .
- Chức năng: cha của, bạn thân, hiệp thứ ba của, một nhiều hơn, bắt đầu của. . .

Thật vậy, hầu hết mọi khẳng định đều có thể được coi là đề cập đến các đối tượng và thuộc tính hoặc mối quan hệ. Một số ví dụ sau đây:

- "Một cộng hai bằng ba." Đối tượng: một, hai, ba, một cộng hai; Quan hệ: bằng; Chức năng: cộng. ("Một cộng hai" là tên của đối tượng thu được bằng cách áp dụng hàm "cộng" cho các đối tượng "một" và "hai". "Ba" là tên gọi khác của đối tượng này.)
- "Các ô vuông lân cận wumpus có mùi." Đồ vật: wumpus, hình vuông; Tài sản: có mùi; Mối quan hệ: láng giềng.
- "Vua độc ác John cai trị nước Anh vào năm 1200." Đồ vật: John, Anh, 1200; Quan hệ: được cai trị trong; Tính chất: ác quỷ, vua.

Ngôn ngữ của logic bậc nhất, có cú pháp và ngữ nghĩa mà chúng ta định nghĩa trong phần tiếp theo, được xây dựng xung quanh các đối tượng và mối quan hệ. Nó rất quan trọng đối với toán học, triết học và trí tuệ nhân tạo chính xác bởi vì những lĩnh vực đó - và thực sự, phần lớn sự tồn tại hàng ngày của con người - có thể được coi là hữu ích để xử lý các đối tượng và các mối quan hệ giữa chúng. Logic bậc nhất cũng có thể diễn tả sự thật về một số hoặc tất cả các vật thể trong vũ trụ. Điều này cho phép người ta đại diện cho các luật hoặc quy tắc chung, chẳng hạn như tuyên bố "Các ô vuông lân cận wumpus có mùi."

Cam kết bản thể học

Sự khác biệt chính giữa logic mệnh đề và logic bậc nhất nằm ở cam kết bản thể luận được thực hiện bởi mỗi ngôn ngữ - nghĩa là, những gì nó giả định về bản chất của thực tại.

Về mặt toán học, cam kết này được thể hiện thông qua bản chất của các mô hình hình thức liên quan đến sự thật của các câu được định nghĩa. Ví dụ, logic mệnh đề giả định rằng có những sự kiện có hoặc không giữ được trong thế giới. Mỗi sự kiện có thể ở một trong hai trạng thái – đúng hoặc sai – và mỗi mô hình gán đúng hoặc sai cho mỗi biểu tượng mệnh đề (xem Phần 7.4.2). Logic bậc nhất giả định nhiều hơn; cụ thể là, thế giới bao gồm các đối tượng với các mối quan hệ nhất định giữa chúng có hoặc không giữ vững. (Xem Hình 8.1.) Các mô hình hình thức tương ứng phức tạp hơn so với các mô hình cho logic mệnh đề.

Ngôn ngữ	Cam kết bản thể học (Những gì tồn tại trên thế giới)	Cam kết nhận thức luận (Những gì một đại lý tin tưởng về sự thật)
Logic mệnh đề	Sự kiện	đúng/sai/không xác định
Logic bậc nhất	sự kiện, đối tượng, quan hệ	đúng/sai/không xác định
Logic thời gian	Sự kiện, Đối tượng, Mối quan hệ, Thời Gian	đúng/sai/không xác định
Lý thuyết xác suất	Sự kiện	Mức độ niềm tin $\in [0, 1]$
Logic mờ	Sự kiện với mức độ chân lý $\in [0, 1]$	Giá trị khoảng thời gian đã biết

Hình 8.1 Các ngôn ngữ chính thức và các cam kết bản thể học và nhận thức luận của chúng.

Cam kết bản thể luận này là một sức mạnh lớn của logic (cả mệnh đề và bậc nhất), bởi vì nó cho phép chúng ta bắt đầu với các tuyên bố đúng và suy ra các tuyên bố đúng khác. Nó đặc biệt mạnh mẽ trong các lĩnh vực mà mọi mệnh đề đều có ranh giới rõ ràng, chẳng hạn như toán học hoặc thế giới wumpus, nơi một hình vuông có hoặc không có hồ; không có khả năng có một hình vuông có vết lõm giống như hồ mơ hồ. Nhưng trong thế giới thực, nhiều đề xuất có ranh giới mơ hồ: Vienna có phải là một thành phố lớn không? Nhà hàng này có phục vụ đồ ăn ngon không? Người đó có cao không? Nó phụ thuộc vào bạn hỏi ai và câu trả lời của họ có thể là "đại

lại". Một câu trả lời là tinh chỉnh biểu diễn: nếu một đường thô phân chia các thành phố thành "lớn" và "không lớn" để lại quá nhiều thông tin cho ứng dụng được đề cập, thì người ta có thể tăng số lượng các loại kích thước hoặc sử dụng biểu tượng hàm Dân số. Một giải pháp khác được đề xuất đến từ logic mờ, làm cho cam kết bản thể luận rằng

Logic mờ

mệnh đề có một mức độ chân lý từ 0 đến 1. Ví dụ, câu "Vienna là một

Mức độ chân lý

thành phố lớn" có thể đúng đến mức độ 0,8 trong logic mờ, trong khi "Paris là một thành phố lớn" có thể đúng đến mức 0,9. Điều này tương ứng tốt hơn với quan niệm trực giác của chúng ta về thế giới, nhưng nó làm cho việc suy luận trở nên khó khăn hơn: thay vì một quy tắc để xác định sự thật của $A \wedge B$, logic mờ cần các quy tắc khác nhau tùy thuộc vào lĩnh vực. Một khả năng khác, được đề cập trong Phần 25.1, là gán mỗi khái niệm cho một điểm trong một không gian đa chiều, và sau đó đo khoảng cách giữa khái niệm "thành phố lớn" và khái niệm "Vienna" hoặc

Đại lý mục đích đặc biệt khác nhau thực hiện các cam kết bản thể học hơn nữa; Ví dụ, logic thời gian giả định rằng các sự kiện có hiệu lực tại những thời điểm cụ thể và những thời điểm đó (có thể

Logic thời gian

là điểm hoặc khoảng thời gian) được sắp xếp. Do đó, logic có mục đích đặc biệt cung cấp cho một số loại đối tượng nhất định (và các tiên đề về chúng) trạng thái "hạng nhất" trong logic, thay vì chỉ đơn giản là xác định chúng trong cơ sở tri thức. Logic bậc cao xem các mối quan hệ và chức năng

Logic bậc cao hơn

được logic bậc nhất gọi là các đối tượng trong chính chúng. Điều này cho phép người ta đưa ra những khẳng định về tất cả các mối quan hệ - ví dụ, người ta có thể muốn xác định ý nghĩa của nó đối với một mối quan hệ là chuyển tiếp. Không giống như hầu hết các logic có mục đích đặc biệt, logic bậc cao hoàn toàn có tính kinh nghiệm hơn logic bậc nhất, theo nghĩa là một số câu của logic bậc cao hơn không thể được biểu thị bằng bất kỳ số lượng hữu hạn nào của câu logic bậc nhất.

Một logic cũng có thể được đặc trưng bởi các cam kết nhận thức luận của nó – trạng thái kiến thức mà nó cho phép đối với từng sự kiện. Trong cả logic mệnh đề và bậc nhất, một câu đại diện cho một sự kiện và tác nhân hoặc tin rằng câu đó là đúng, tin rằng nó là sai hoặc không có ý kiến. Do đó, những logic này có ba trạng thái kiến thức có thể xảy ra liên quan đến bất kỳ câu nào.

Cam kết nhận thức luận

Mặt khác, các hệ thống sử dụng lý thuyết xác suất có thể có bất kỳ mức độ tin tưởng, hoặc khả năng chủ quan nào, từ 0 (hoàn toàn không tin) đến 1 (hoàn toàn tin tưởng). Điều quan trọng là không nhầm lẫn mức độ tin tưởng vào lý thuyết xác suất với mức độ chân lý trong logic mờ. Thật vậy, một số hệ thống mờ cho phép sự không chắc chắn (mức độ tin tưởng) về mức độ chân lý. Ví dụ, một tác nhân thế giới wumpus xác suất có thể tin rằng wumpus nằm trong [1,3] với xác suất 0,75 và trong [2, 3] với xác suất 0,25 (mặc dù wumpus chắc chắn nằm trong một hình vuông cụ thể).

8.2 Cú pháp và ngữ nghĩa của logic bậc nhất

Chúng ta bắt đầu phần này bằng cách xác định chính xác hơn cách thức mà trong đó các thế giới có thể có của logic bậc nhất phản ánh cam kết bản thể luận đối với các đối tượng và các mối quan hệ. Sau đó, chúng tôi giới thiệu các yếu tố khác nhau của ngôn ngữ, giải thích ngữ nghĩa của chúng khi chúng tôi tiếp tục. Những điểm chính là cách ngôn ngữ tạo điều kiện cho các biểu diễn ngắn gọn và cách ngữ nghĩa của nó dẫn đến các quy trình lý luận hợp lý.

8.2.1 Mô hình cho logic bậc nhất

Chương 7 nói rằng các mô hình của một ngôn ngữ logic là các cấu trúc hình thức cấu thành các thế giới có thể được xem xét. Mỗi mô hình liên kết từ vựng của các cảm giác logic với các yếu tố của thế giới có thể xảy ra, để có thể xác định được sự thật của bất kỳ câu nào. Do đó, các mô hình cho logic mệnh đề liên kết các ký hiệu mệnh đề với các giá trị chân lý được xác định trước.

Các mô hình cho logic bậc nhất thú vị hơn nhiều. Đầu tiên, chúng có các đồ vật trong đó! Miền của mô hình là tập hợp các đối tượng hoặc phần tử miền mà nó chứa. Tên miền là bắt buộc phải không trống—mọi thế giới có thể có phải chứa ít nhất một đối tượng. (Xem Exer-cise 8.EMPT để thảo luận về thế giới trống rỗng.) Về mặt toán học, không quan trọng những đối tượng này là gì - tất cả những gì quan trọng là có bao nhiêu trong mỗi mô hình cụ thể - nhưng đối với mục đích sư phạm, chúng ta sẽ sử dụng một ví dụ cụ thể. Hình 8.2 cho thấy một mô hình với năm vật thể: Richard the Lionheart, Vua của Anh từ năm 1189 đến năm 1199; em trai của ông, Vua John độc ác, người cai trị từ năm 1199 đến năm 1215; chân trái của Richard và John; và một chiếc vương miện.

Các đối tượng trong mô hình có thể liên quan theo nhiều cách khác nhau. Trong hình, anh em nhà Richard và Johnare. Nói một cách chính thức, một quan hệ chỉ là tập hợp các bộ của các đối tượng có liên quan.

(Bộ là một tập hợp các đối tượng được sắp xếp theo một thứ tự cố định và được viết bằng dấu ngoặc nhọn bao quanh các đối tượng.) Vì vậy, mỗi quan hệ anh em trong mô hình này là tập hợp

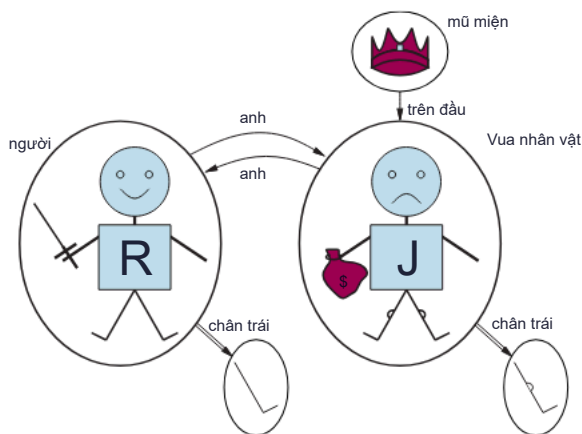
$$\{ \langle \text{Richard the Lionheart, King John} \rangle, \langle \text{King John, Richard the Lionheart} \rangle \}. \quad (8.1)$$

(Ở đây chúng tôi đã đặt tên cho các đồ vật bằng tiếng Anh, nhưng nếu muốn, bạn có thể thay thế hình ảnh cho tên.) Vương miện nằm trên đầu của Vua John, vì vậy mỗi quan hệ "trên đầu" chỉ chứa một tuple, Quan hệ "anh em" và "trên đầu" là quan hệ nhị phân - nghĩa là, chúng liên quan đến các cặp đối tượng. Mô hình này cũng chứa các quan hệ đơn nguyên, hoặc thuộc tính: thuộc tính "người" đúng với cả Richard và John; tài sản "vua" chỉ đúng với John (có lẽ vì Richard đã chết vào thời điểm này); và thuộc tính "vương miện" chỉ đúng với vương miện.

Một số loại mối quan hệ nhất định được coi là hàm, trong đó một đối tượng nhất định phải liên quan đến chính xác một đối tượng theo cách này. Ví dụ, mỗi người có một chân trái, vì vậy mô hình có một hàm "chân trái" đơn nguyên — ánh xạ từ bộ một phần tử đến một đối tượng —

Các phần tử
DomainDomain

Bộ



Hình 8.2 Một mô hình chứa năm đối tượng, hai quan hệ nhị phân (anh em và trên đầu), quan hệ ba đơn (người, vua và vương miện) và một hàm đơn nguyên (chân trái).

Bao gồm các ánh xạ sau:

$$\begin{aligned} \text{Richard the Lionheart} &\rightarrow \text{chân trái của} \\ \text{Richard (Vua John)} &\rightarrow \text{chân trái của John.} \end{aligned} \quad (8.2)$$

Nói một cách chính xác, các mô hình trong logic bậc nhất yêu cầu tổng các hàm, tức là phải có một giá trị cho mọi bộ dữ liệu đầu vào. Do đó, vương miện phải có một chân trái và mỗi chân trái cũng vậy. Có một giải pháp kỹ thuật cho vấn đề khó xử này liên quan đến một vật thể "vô hình" bổ sung là chân trái của mọi thứ không có chân trái, bao gồm cả chính nó. May mắn thay, chừng nào người ta không khẳng định về chân trái của những thứ không có chân trái, những kỹ thuật này không có ý nghĩa.

Cho đến nay, chúng tôi đã mô tả các yếu tố điền vào các mô hình cho logic bậc nhất. Phần thiết yếu khác của một mô hình là mối liên hệ giữa các yếu tố đó và từ vựng của các câu thần học, mà chúng tôi giải thích tiếp theo.

8.2.2 Biểu tượng và giải thích

Bây giờ chúng ta chuyển sang cú pháp của logic bậc nhất. Người đọc thiếu kiên nhẫn có thể nhận được mô tả đầy đủ từ ngữ pháp chính thức trong Hình 8.3.

Các yếu tố cơ bản của logic bậc nhất là các ký hiệu đại diện cho các đối tượng, quan hệ và hàm. Do đó, các biểu tượng có ba loại: biểu tượng không đổi, viết tắt của các đối tượng; ký hiệu vị ngữ, viết tắt của các mối quan hệ; và chức năng symbols, viết tắt của các chức năng. Chúng tôi áp dụng quy ước mà các biểu tượng này sẽ bắt đầu

chữ hoa. Ví dụ, chúng ta có thể sử dụng các ký hiệu hằng số Richard và John; các biểu tượng vị ngữ Brother, OnHead, Person, King, và Crown; và hàm symbolLeftLeg. Cũng như các ký hiệu mệnh đề, việc lựa chọn tên hoàn toàn phụ thuộc vào người dùng. Mỗi ký hiệu vị ngữ và hàm đi kèm với một arity cố định số lượng đối số.

Tổng số chức năng

Biểu tượng hằng số

Ký hiệu vị ngữ

Biểu tượng chức năng

Arity

Câu $\rightarrow$ AtomicSentence | ComplexSentence
 AtomicSentence $\rightarrow$
 vị ngữ | Vị ngữ (Thuật ngữ, . . .) | Term = Term
 ComplexSentence
 $\rightarrow$ (Câu)

| $\neg$ Câu|Câu $\wedge$ câu|Câu $\vee$
 câu|Câu $\Rightarrow$ câu |Câu $\Leftrightarrow$ Câu
 |Biến định lượng, . . . Câu

Thuật ngữ $\rightarrow$ chức năng (Thuật ngữ, . . .)

|Hằng số
 |Biến số

Định lượng $\rightarrow \forall | \exists$

Hằng số $\rightarrow A | X1 | \text{Giăng} | \dots$

Biến $\rightarrow a | x | s | \dots$ Vị ngữ $\rightarrow$ đúng | Sai | Sau |

Tình yêu | Mưa | $\dots$

Chức năng $\rightarrow$ mẹ | Chân trái | $\dots$

ƯU TIÊN TOÁN TỬ : $\neg, =, \wedge, \vee, \Rightarrow, \Leftrightarrow$

Hình 8.3 Cú pháp của logic bậc nhất với bình đẳng, được chỉ định ở dạng Backus–Naur (xem trang 1081 nếu bạn không quen thuộc với ký hiệu này). Mức độ ưu tiên của toán tử được chỉ định, từ cao nhất đến thấp nhất. Mức độ ưu tiên của các định lượng là sao cho một định lượng giữ mọi thứ ở bên phải của nó.

Mỗi mô hình phải cung cấp thông tin cần thiết để xác định xem bất kỳ câu nào là đúng hay sai. Do đó, ngoài các đối tượng, quan hệ và chức năng của nó, mỗi mô hình bao gồm một cách giải thích xác định chính xác đối tượng, quan hệ và chức năng nào được đề cập bởi các ký hiệu hằng số, vị ngữ và hàm. Một cách giải thích khá dễ cho ví dụ của chúng ta - mà một nhà logic học sẽ gọi là cách giải thích dự định - như sau:

Giải thích

Dự định giải thích

- Richard đề cập đến Richard the Lionheart và John đề cập đến Vua John độc ác.
- Brother đề cập đến mối quan hệ anh em - nghĩa là tập hợp các bộ của các đối tượng được đưa ra trong Phương trình (8.1); OnHead là mối quan hệ giữa vương miện và Vua John; Người, Vua và Vương miện là những mối quan hệ thống nhất xác định người, vua và vương miện.
- LeftLeg đề cập đến hàm "chân trái" như được định nghĩa trong Phương trình (8.2). Tất nhiên, có nhiều cách giải thích khác có thể xảy ra. Ví dụ, một cách giải thích lập bản đồ Richard với vương miện và John với chân trái của Vua John. Có năm đối tượng trong mô hình, vì vậy có 25 cách giải thích có thể xảy ra chỉ cho các ký hiệu hằng số Richard và John.



Hình 8.4 Một số thành viên của tập hợp tất cả các mô hình cho một ngôn ngữ có hai ký hiệu hằng số, R và J, và một ký hiệu quan hệ nhị phân. Cách giải thích của từng ký hiệu hằng số được hiển thị bằng một tên màu xám. Trong mỗi mô hình, các đối tượng liên quan được kết nối bằng mũi tên.

Lưu ý rằng không phải tất cả các đồ vật đều cần phải có tên — ví dụ, cách giải thích dự định không đặt tên cho vương miện hoặc chân. Một đối tượng cũng có thể có một số tên; có một cách giải thích mà theo đó cả Richard và John đều đề cập đến vương miện.² Nếu bạn thấy khả năng này gây nhầm lẫn, hãy nhớ rằng, trong logic mệnh đề, hoàn toàn có thể có một mô hình trong đó Cloudy và Sunny đều đúng; Công việc của cơ sở tri thức là loại trừ các mô hình không phù hợp với kiến thức của chúng ta.

Tóm lại, một mô hình trong logic bậc nhất bao gồm một tập hợp các đối tượng và một cách giải thích ánh xạ các ký hiệu hằng số với các đối tượng, ký hiệu hàm với các hàm trên các đối tượng đó và ký hiệu vị ngữ cho các mối quan hệ. Cũng giống như với logic mệnh đề, bao gồm, hợp lệ và sớm được định nghĩa theo tất cả các mô hình có thể. Để có được ý tưởng về tập hợp của tất cả các mô hình có thể trông như thế nào, hãy xem Hình 8.4. Nó cho thấy rằng các mô hình khác nhau về số lượng đối tượng mà chúng chứa - từ một đến vô cực - và cách các ký hiệu hằng số ánh xạ đến các đối tượng.

Bởi vì số lượng các mô hình bậc nhất là không giới hạn, chúng ta không thể kiểm tra sự bao gồm bằng cách liệt kê tất cả chúng (như chúng ta đã làm đối với logic mệnh đề). Ngay cả khi số lượng đối tượng bị hạn chế, số lượng kết hợp có thể rất lớn. (Xem Bài tập 8.MCNT.) Đối với ví dụ trong Hình 8.4, có 137.506.194.466 mô hình có sáu đối tượng trở xuống.

8.2.3 Điều khoản

Thuật ngữ là một biểu thức logic đề cập đến một đối tượng. Ký hiệu hằng số là các thuật ngữ, nhưng nó là Thời hạn

không phải lúc nào cũng thuận tiện để có một biểu tượng riêng biệt để đặt tên cho mọi đối tượng. Trong tiếng Anh, chúng ta có thể sử dụng cụm từ "Chân trái của Vua John" thay vì đặt tên cho chân của ông. Đây là mục đích của các ký hiệu chức năng: thay vì sử dụng một ký hiệu hằng số, chúng ta sử dụng LeftLeg (John).³

Trong trường hợp chung, một số hạng phức được hình thành bởi một ký hiệu hàm theo sau là một danh sách các số hạng có kích thước theo dấu nhon làm đối số cho ký hiệu hàm. Điều quan trọng cần nhớ là một thuật ngữ phức tạp chỉ là một loại tên phức tạp. Nó không phải là một "cuộc gọi tiểu tục" "trả về giá trị". Không có chương trình con LeftLeg nào lấy một người làm đầu vào và trả về một chân. Chúng ta có thể lý luận về chân trái (ví dụ: nếu quy tắc chung rằng mọi người đều có một chân và sau đó suy ra

² Sau đó, trong Phần 8.2.8, chúng ta xem xét một ngữ nghĩa trong đó mỗi đối tượng phải có chính xác một tên. Biểu thức λ (biểu thức lambda) cung cấp một ký hiệu hữu ích trong đó các ký hiệu hàm mới được xây dựng "một cách nhanh chóng". Ví dụ, hàm bình phương đối số của nó có thể được viết là $(\lambda x: x \times x)$ và có thể được áp dụng cho các đối số giống như bất kỳ ký hiệu hàm nào khác. Biểu thức λ cũng có thể được định nghĩa và sử dụng như một ký hiệu vị ngữ. Toán tử lambda trong Lisp và Python đóng vai trò chính xác giống nhau. Lưu ý rằng việc sử dụng λ theo cách này không làm tăng sức mạnh biểu đạt hình thức của logic bậc nhất, bởi vì bất kỳ câu nào bao gồm biểu thức λ đều có thể được viết lại bằng cách "cắm" các đối số của nó để tạo ra một câu tương đương.

rằng John phải có một) mà không bao giờ cung cấp định nghĩa về LeftLeg. Đây là điều không thể thực hiện được với các chương trình con trong ngôn ngữ lập trình.

Ngữ nghĩa chính thức của các thuật ngữ rất đơn giản. Hãy xem xét một thuật ngữ $f(t_1, \dots, t_n)$. Ký hiệu hàm f đề cập đến một số hàm trong mô hình (gọi là F); các thuật ngữ đối số đề cập đến các đối tượng trong lĩnh vực (gọi chúng là $d_1, \dots, d_n$); và thuật ngữ nói chung đề cập đến đối tượng đó là giá trị của hàm F được áp dụng cho $d_1, \dots, d_n$. Ví dụ: giả sử biểu tượng hàm LeftLeg đề cập đến hàm được hiển thị trong Phương trình (8.2) và John đề cập đến Vua John, sau đó LeftLeg (John) đề cập đến chân trái của Vua John. Bằng cách này, cách giải thích cố định tham chiếu của mọi thuật ngữ.

8.2.4 Câu nguyên tử

Bây giờ chúng ta có các thuật ngữ để chỉ các đối tượng và các ký hiệu vị ngữ để đề cập đến các mối quan hệ, chúng ta có thể kết hợp chúng để tạo ra các câu nguyên tử nêu các sự kiện. Một câu nguyên tử (hoặc viết tắt là nguyên tử) được hình thành từ một ký hiệu vị ngữ tùy chọn theo sau bởi một dấu ngoặc đơn danh sách các thuật ngữ, chẳng hạn

Anh trai (Richard, John). Điều này nói, theo cách giải thích dự định được đưa ra trước đó, rằng Richard the Lionheart là anh trai của Vua John.⁴ Các câu nguyên tử có thể có các thuật ngữ phức tạp làm lập luận. Vậy

Đã kết hôn (Cha (Richard), Mẹ (John)) nói rằng cha của Richard the Lionheart đã kết hôn với mẹ của Vua John (một lần nữa, theo cách giải thích phù hợp).⁵

Một câu nguyên tử là đúng trong một mô hình nhất định nếu mối quan hệ được đề cập bởi ký hiệu vị ngữ giữ giữa các đối tượng được tham chiếu bởi các đối số.

8.2.5 Câu phức tạp

Chúng ta có thể sử dụng các liên kết logic để xây dựng các câu phức tạp hơn, với cùng cú pháp và ngữ nghĩa như trong giải tích mệnh đề. Dưới đây là bốn câu đúng trong mô hình của Hình 8.2 theo cách giải thích dự định của chúng tôi:

$\neg$ Anh trai (Chân trái (Richard), John) Anh
trai (Richard, John) $\wedge$ Anh trai (John,
Richard) Vua (Richard) $\vee$ Vua (John) $\neg$ Vua
(Richard) $\Rightarrow$ Vua (John) .

8.2.6 Định lượng

Một khi chúng ta có một logic cho phép các đối tượng, việc muốn biểu thị các thuộc tính của toàn bộ các tập hợp các đối tượng là điều tự nhiên, thay vì liệt kê các đối tượng theo tên. Định lượng cho chúng ta làm điều này. Logic bậc nhất chứa hai định lượng tiêu chuẩn, được gọi là phổ quát và hiện sinh.

Định lượng phổ quát ($\forall$)

Hãy nhớ lại khó khăn mà chúng ta đã gặp phải trong Chương 7 với việc diễn đạt các quy tắc chung trong logic tuyên truyền. Các quy tắc như "Các ô vuông lân cận wumpus có mùi" và "Tất cả các vị vua

4 Chúng ta thường tuân theo quy ước thứ tự đối số rằng $P(x,y)$ được đọc là "x là P của y". 5 Bản thể luận này chỉ công nhận một người cha và một người mẹ cho mỗi người. Một bản thể phức tạp hơn có thể nhận ra mẹ ruột, mẹ đẻ, mẹ nuôi, v.v.

Câu nguyên tử
Nguyên tử

Định lượng

là con người" là bánh mì và bơ của logic bậc nhất. Chúng tôi giải quyết vấn đề đầu tiên trong số này trong Phần 8.3. Quy tắc thứ hai, "Tất cả các vị vua đều là con người", được viết theo logic bậc nhất như

$\forall x \text{ Vua}(x) \Rightarrow \text{Người}(x)$. Định lượng phổ quát $\forall$ thường được phát âm là "Cho tất cả . . .". (Hãy nhớ rằng mặt trái-

Định lượng phổ quát

down A là viết tắt của "tất cả.") Do đó, câu nói, "Đối với tất cả x, nếu x là vua, thì x là một người." Ký hiệu x được gọi là biến. Theo quy ước, các biến là chữ thường. Một biến

Biến số

là một thuật ngữ tự nó, và như vậy cũng có thể đóng vai trò là đối số của một hàm — ví dụ, LeftLeg (x). Một thuật ngữ không có biến được gọi là thuật ngữ cơ bản.

Thuật ngữ cơ bản

Theo trực giác, câu $\forall x P$, trong đó P là bất kỳ câu logic nào, nói rằng P đúng cho mọi đối tượng x. Chính xác hơn, $\forall x P$ đúng trong một mô hình nhất định nếu P đúng trong tất cả các cách giải thích mở rộng có thể được xây dựng từ cách giải thích được đưa ra trong mô hình, trong đó mỗi giải thích mở rộng

Phiên dịch mở rộng

Interpretation chỉ định một phần tử miền mà x đề cập

Điều này nghe có vẻ phức tạp, nhưng nó thực sự chỉ là một cách cẩn thận để nêu ý nghĩa trực quan của định lượng phổ quát. Hãy xem xét mô hình được hiển thị trong Hình 8.2 và cách giải thích dự định đi kèm với nó. Chúng ta có thể mở rộng cách giải thích theo năm cách:

$x \rightarrow$ Richard the Lionheart, $x \rightarrow$ King John, $x \rightarrow$ chân trái của Richard, $x \rightarrow$ chân trái của John, $x \rightarrow$ vương miện. Câu định lượng phổ quát $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$ là đúng trong mô hình ban đầu nếu câu $\text{King}(x) \Rightarrow \text{Person}(x)$ đúng theo mỗi trong số năm cách giải thích mở rộng. Nghĩa là, câu định lượng phổ quát tương đương với việc khẳng định năm câu sau:

Richard the Lionheart là một vị vua $\Rightarrow$ Richard the Lionheart là một con người. Vua John là một vị vua $\Rightarrow$ Vua John là một con người. Chân trái của Richard là một vị vua $\Rightarrow$ chân trái của Richard là một con người. Chân trái của John là một vị vua $\Rightarrow$ chân trái của John là một con người. Vương miện là một vị vua $\Rightarrow$ vương miện là một con người. Chúng ta hãy xem xét cẩn thận tập hợp các khẳng định này. Bởi vì, trong mô hình của chúng tôi, Vua John là vị vua duy nhất, câu thứ hai khẳng định rằng ông là một con người, như chúng ta hy vọng. Nhưng còn bốn câu còn lại, dường như đưa ra tuyên bố về chân và vương miện thì sao? Đó có phải là một phần của ý nghĩa của "Tất cả các vị vua đều là con người"? Trên thực tế, bốn khẳng định khác là đúng trong mô hình, nhưng không đưa ra bất kỳ tuyên bố nào về trình độ nhân cách của chân, vương miện, hoặc thực sự là Richard. Điều này là do không có đồ vật nào trong số này là vua. Nhìn vào bảng sự thật cho $\Rightarrow$ (Hình 7.8 trên trang 237), chúng ta thấy rằng hàm ý là đúng bất cứ khi nào tiền đề của nó là sai - bất kể sự thật của kết luận. Do đó, bằng cách khẳng định câu được định lượng phổ quát, tương đương với việc khẳng định toàn bộ danh sách các hàm ý riêng lẻ, chúng ta kết thúc việc khẳng định kết luận của quy tắc chỉ dành cho những đối tượng mà tiền đề là đúng và không nói gì về những đối tượng mà tiền đề là sai. Do đó, định nghĩa bảng sự thật của $\Rightarrow$ hóa ra là hoàn hảo để viết các quy tắc chung với các định lượng phổ quát.

Một sai lầm phổ biến, thường xuyên mắc phải ngay cả bởi những độc giả siêng năng đã đọc paragraph này nhiều lần, là sử dụng liên từ thay vì ngụ ý. Câu

$\forall x \text{ Vua}(x) \wedge \text{người}$

sẽ tương đương với việc khẳng

định Richard the Lionheart là một vị vua $\wedge$ Richard the Lionheart là một con người, Vua John là một vị vua $\wedge$ Vua John là một người, chân trái của Richard là một vị vua $\wedge$ chân trái của Richard là một người, v.v. Rõ ràng, điều này không nắm bắt được những gì chúng ta muốn.

Định lượng hiện sinh ($\exists$)

Định lượng phổ quát đưa ra các tuyên bố về mọi đối tượng. Tương tự như vậy, chúng ta có thể đưa ra một tuyên bố về một đối tượng nào đó mà không cần đặt tên cho nó, bằng cách sử dụng một định lượng hiện sinh. Để nói,

ví dụ, rằng Vua John có một vương miện trên đầu, chúng tôi viết

$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$. $\exists x$ được phát âm là "Có một x như vậy . . ." hoặc "Đối với một số x . . .".

Theo trực giác, câu $\exists x \text{ P}$ nói rằng P đúng đối với ít nhất một đối tượng x. Chính xác hơn $\exists x \text{ P}$ đúng trong một mô hình nhất định nếu P đúng trong ít nhất một cách giải thích mở rộng gán x cho một phần tử miền. Nghĩa là, ít nhất một trong những điều sau đây là đúng:

Richard the Lionheart là một chiếc vương miện $\wedge$ Richard the Lionheart nằm trên đầu John; Vua John là một chiếc vương miện $\wedge$ Vua John nằm trên đầu John; Chân trái của Richard là vương miện $\wedge$ chân trái của Richard nằm trên đầu John; Chân trái của John là vương miện $\wedge$ chân trái của John nằm trên đầu John; Vương miện là một chiếc vương miện $\wedge$ vương miện nằm trên đầu của John. Khẳng định thứ năm là đúng trong mô hình, vì vậy câu định lượng hiện sinh ban đầu là đúng trong mô hình. Lưu ý rằng, theo định nghĩa của chúng tôi, câu này cũng sẽ đúng trong một mô hình trong đó Vua John đang đội hai vương miện. Điều này hoàn toàn phù hợp với câu gốc "Vua John có vương miện trên đầu." 6

Cũng giống như $\Rightarrow$ dường như là kết nối tự nhiên để sử dụng với $\forall$, $\wedge$ là kết nối tự nhiên để sử dụng với $\exists$. Sử dụng $\wedge$ làm liên kết chính với $\forall$ dẫn đến một tuyên bố quá mạnh trong ví dụ trong phần trước; Sử dụng $\Rightarrow$ với $\exists$ thường dẫn đến một tuyên bố rất yếu, thực sự. Hãy xem câu sau:

$$\exists x \text{ Vương miện}(x) \Rightarrow \text{OnHead}(x, \text{John}).$$

Nhìn bên ngoài, điều này có thể giống như một cách diễn đạt hợp lý của câu của chúng ta. Áp dụng những luận thuyết này, chúng ta thấy rằng câu nói rằng ít nhất một trong những khẳng định sau đây là đúng:

Richard the Lionheart là một chiếc vương miện $\Rightarrow$ Richard the Lionheart nằm trên đầu John; Vua John là một chiếc vương miện $\Rightarrow$ Vua John nằm trên đầu John; Chân trái của Richard là vương miện $\Rightarrow$ chân trái của Richard nằm trên đầu John; và như vậy. Một hàm ý là đúng nếu cả tiền đề và kết luận đều đúng, hoặc nếu tiền đề của nó sai; vì vậy nếu Richard the Lionheart không phải là một vương miện, thì khẳng định đầu tiên là đúng và sự tồn tại được thỏa mãn. Vì vậy, một câu ngụ ý định lượng tồn tại là đúng khi bất kỳ đối tượng nào không thỏa mãn tiền đề; do đó những câu như vậy thực sự không nói lên nhiều điều gì cả.

6 Có một biến thể của định lượng hiện sinh, thường được viết $\exists!$ hoặc $\exists!$, có nghĩa là "Có chính xác một." Ý nghĩa tương tự có thể được thể hiện bằng cách sử dụng các tuyên bố bình đẳng.

Định lượng lồng nhau

Chúng ta thường sẽ muốn thể hiện các câu phức tạp hơn bằng cách sử dụng nhiều định lượng. Trường hợp simplest là nơi các định lượng cùng loại. Ví dụ: "Anh em là anh chị em" có thể được viết là

$\forall x \forall y$ Anh trai $(x, y) \Rightarrow$ Anh chị em (x, y) . Các định lượng liên tiếp cùng loại có thể được viết thành một định lượng với một số biến thể. Ví dụ, để nói rằng anh chị em là một mối quan hệ đối xứng, chúng ta có thể viết

$\forall x, y$ Anh chị em $(x, y) \Leftrightarrow$ Anh chị em (y, x) . Trong các trường hợp khác, chúng tôi sẽ có hỗn hợp. "Mọi người đều yêu ai đó" có nghĩa là đối với mỗi người, có một người mà người đó yêu thương:

$\forall x \exists y$ Tình yêu (x, y) . Mặt khác, để nói "Có một người được mọi người yêu thương", chúng ta viết

$\exists y \forall x$ Tình yêu (x, y) . Do đó, thứ tự định lượng là rất quan trọng. Nó trở nên rõ ràng hơn nếu chúng ta chèn các luận điểm dấu vòng. $\forall x (\exists y \text{ Loves}(x, y))$ nói rằng mọi người đều có một thuộc tính cụ thể, cụ thể là, thuộc tính mà họ yêu ai đó. Mặt khác, $\exists y (\forall x \text{ Loves}(x, y))$ nói rằng ai đó trong thế giới có một thuộc tính đặc biệt, đó là thuộc tính được mọi người yêu thương.

Một số nhầm lẫn có thể phát sinh khi hai định lượng được sử dụng với cùng một tên biến. Xem xét câu

$\forall x (\text{Vương miện}(x) \vee (\exists x \text{ Anh trai}(\text{Richard}, x)))$. Ở đây x trong Brother $(\text{Richard}, x)$ được định lượng hiện sinh. Quy tắc là biến thuộc về định lượng trong cùng đề cập đến nó; thì nó sẽ không phải chịu bất kỳ định lượng nào khác. Một cách khác để nghĩ về nó là: $\exists x$ Anh trai $(\text{Richard}, x)$ là một câu về Richard (rằng anh ấy có một anh trai), không phải về x ; Vì vậy, đặt một $\forall x$ bên ngoài nó không có tác dụng. Nó cũng có thể được viết $\exists z \text{ Brother}(\text{Richard}, z)$. Bởi vì đây có thể là một nguồn gây nhầm lẫn, chúng ta sẽ luôn sử dụng các tên biến khác nhau với các định lượng lồng nhau.

Kết nối giữa $\forall$ và $\exists$

Hai định lượng thực sự được kết nối mật thiết với nhau, thông qua phủ định. Nói rằng mọi người không thích củ cải cũng giống như khẳng định không có ai thích chúng và ngược lại:

$\forall x \neg \text{Likes}(x, \text{Parsnips})$ tương đương với $\neg \exists x \text{ Likes}(x, \text{Parsnips})$. Chúng ta có thể tiến thêm một bước nữa: "Mọi người đều thích kem" có nghĩa là không có ai không thích kem:

$\forall x \text{ Likes}(x, \text{IceCream})$ tương đương với $\neg \exists x \neg \text{Likes}(x, \text{IceCream})$. Bởi vì $\forall$ thực sự là một sự kết hợp trên vũ trụ của các vật thể và $\exists$ là một sự tách rời, không có gì ngạc nhiên khi chúng tuân theo các quy tắc của De Morgan. Các quy tắc của De Morgan cho các câu định lượng và không định lượng như sau:

$$\begin{array}{ll} \neg \exists x P \equiv \forall x \neg P & \neg (P \vee Q) \equiv \neg P \wedge \neg Q \\ \neg \forall x P \equiv \exists x \neg P & \neg (P \wedge Q) \equiv \neg P \vee \neg Q \\ \forall x P \equiv \neg \exists x \neg P & P \wedge Q \equiv \neg (\neg P \vee \neg Q) \\ \exists x P \equiv \neg \forall x \neg P & P \vee Q \equiv \neg (\neg P \wedge \neg Q) \end{array}$$

Vì vậy, chúng ta không thật sự cần cả $\forall$ và $\exists$, cũng như chúng ta không thật sự cần cả $\wedge$ và $\vee$. Tuy nhiên, khả năng đọc quan trọng hơn sự tiết kiệm, vì vậy chúng tôi sẽ giữ cả hai định lượng.

8.2.7 Bình đẳng

Biểu tượng bình đẳng

Logic bậc nhất bao gồm một cách nữa để tạo ra các câu nguyên tử, ngoài việc sử dụng một predi-cate và các thuật ngữ như đã mô tả trước đó. Chúng ta có thể sử dụng ký hiệu bình đẳng để biểu thị rằng hai thuật ngữ

tham chiếu đến cùng một đối tượng. Chẳng hạn

Cha (John) = Henry nói rằng đối tượng được đề cập bởi Cha (John) và đối tượng được Henry đề cập là giống nhau. Bởi vì một cách giải thích cố định tham chiếu của bất kỳ thuật ngữ nào, việc xác định tính đúng đắn của câu không bình đẳng chỉ đơn giản là vấn đề thấy rằng các tham chiếu của hai thuật ngữ là cùng một đối tượng.

Biểu tượng bình đẳng có thể được sử dụng để nêu các sự kiện về một hàm nhất định, như chúng ta vừa làm với biểu tượng Cha. Nó cũng có thể được sử dụng với phù định để nhấn mạnh rằng hai số hạng không phải là cùng một đối tượng. Để nói rằng Richard có ít nhất hai anh trai, chúng tôi sẽ viết

$\exists x, y$ Anh trai (x, Richard) $\wedge$ Anh trai (y, Richard) $\wedge \neg (x = y)$
. Câu

$\exists x, y$ Brother(x, Richard) $\wedge$ Brother(y, Richard) không có ý nghĩa dự kiến. Đặc biệt, nó đúng trong mô hình của Hình 8.2, trong đó Richard chỉ có một người anh trai. Để thấy điều này, hãy xem xét cách giải thích mở rộng trong đó cả x và y được gán cho Vua John. Việc bổ sung $\neg(x = y)$ loại trừ các mô hình như vậy. Ký hiệu $x \neq y$ đôi khi được sử dụng như một chữ viết tắt của $\neg(x = y)$.

8.2.8 Ngữ nghĩa cơ sở dữ liệu

Tiếp tục ví dụ từ phần trước, giả sử rằng chúng ta tin rằng Richard có hai anh em, John và Geoffrey.⁷ Chúng ta có thể viết

Anh trai (John, Richard) $\wedge$ Anh trai (Geoffrey, Richard), (8.3)

Nhưng điều đó sẽ không hoàn toàn nắm bắt được tình hình. Đầu tiên, khẳng định này đúng trong một mô hình mà Richard chỉ có một anh em - chúng ta cần thêm $\text{Giảng 6} = \text{Geoffrey}$. Thứ hai, câu này không loại trừ các mô hình trong đó Richard có nhiều anh em hơn ngoài John và Geoffrey. Do đó, bản dịch chính xác của "Anh trai của Richard là John và Geoffrey" như sau:

Anh trai (John, Richard) $\wedge$ Anh trai (Geoffrey, Richard) $\wedge \text{John} \neq \text{Geoffrey}$
 $\wedge \forall x$ Anh trai (x, Richard) $\Rightarrow (x = \text{John} \vee x = \text{Geoffrey})$. Câu logic này có vẻ rườm rà hơn nhiều so với cảm giác tiếng Anh tương ứng. Nhưng nếu chúng ta không dịch tiếng Anh đúng cách, hệ thống lý luận logic của chúng ta sẽ mắc sai lầm. Chúng ta có thể nghĩ ra một ngữ nghĩa cho phép một câu logic đơn giản hơn không?

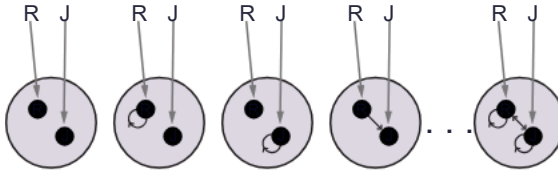
Tên duy nhấtgiả định

Một đề xuất rất phổ biến trong hệ thống cơ sở dữ liệu hoạt động như sau. Đầu tiên, chúng ta xác định rằng mọi ký hiệu hằng số đề cập đến một đối tượng riêng biệt - giả định tên duy nhất.

Closed-world assumption
Domain closure

Thứ hai, chúng ta giả định rằng các câu nguyên tử không được biết là đúng trên thực tế là sai - giả định thế giới khép kín. Cuối cùng, chúng ta gọi domain closure, có nghĩa là mỗi mô hình chứa không có nhiều phần tử miền hơn những phần tử được đặt tên bởi các ký hiệu hằng số.

⁷ Thực ra ông có bốn người, những người khác là William và Henry.



Hình 8.5 Một số thành viên của tập hợp tất cả các mô hình cho một ngôn ngữ có hai ký hiệu hằng số, R và J, và một ký hiệu quan hệ nhị phân, theo ngữ nghĩa cơ sở dữ liệu. Việc giải thích các ký hiệu hằng số là cố định và có một đối tượng riêng biệt cho mỗi ký hiệu hằng số.

Theo ngữ nghĩa kết quả, Phương trình (8.3) thực sự nói rằng Richard có chính xác hai anh em, John và Geoffrey. Chúng tôi gọi cơ sở dữ liệu này là ngữ nghĩa để phân biệt nó với ngữ nghĩa tiêu chuẩn của logic bậc nhất. Ngữ nghĩa cơ sở dữ liệu cũng được sử dụng trong các hệ thống lập trình logic, như được giải thích trong Phần 9.4.4. Ngữ nghĩa cơ sở dữ liệu

Sẽ rất hữu ích khi xem xét tập hợp tất cả các mô hình có thể có trong ngữ nghĩa cơ sở dữ liệu cho cùng một trường hợp như được hiển thị trong Hình 8.4 (trang 277). Hình 8.5 cho thấy một số mô hình, nằm trong phạm vi từ mô hình mà không có bộ nào thỏa mãn mỗi quan hệ với mô hình với tất cả các bộ thỏa mãn mỗi quan hệ. Với hai đối tượng, có bốn bộ hai phần tử có thể có, vì vậy có $24 = 16$ tập con khác nhau của các bộ có thể thỏa mãn mỗi quan hệ. Do đó, có 16 mô hình có thể có trong tất cả — ít hơn rất nhiều so với số lượng vô hạn mô hình cho ngữ nghĩa bậc nhất tiêu chuẩn. Mặt khác, ngữ nghĩa cơ sở dữ liệu đòi hỏi kiến thức xác định về những gì thế giới chứa đựng.

Ví dụ này đưa ra một điểm quan trọng: không có ngữ nghĩa "đúng" nào cho logic. Tính hữu ích của bất kỳ ngữ nghĩa nào được đề xuất phụ thuộc vào mức độ ngắn gọn và trực quan mà nó thể hiện các loại kiến thức mà chúng ta muốn viết ra, và mức độ dễ dàng và tự nhiên để phát triển các quy tắc suy luận tương ứng. Ngữ nghĩa cơ sở dữ liệu hữu ích nhất khi chúng ta chắc chắn về danh tính của tất cả các đối tượng được mô tả trong cơ sở tri thức và khi chúng ta có tất cả các sự kiện trong tay; trong các trường hợp khác, nó khá khó xử. Đối với phần còn lại của chương này, chúng ta giả định ngữ nghĩa tiêu chuẩn trong khi lưu ý các trường hợp trong đó sự lựa chọn này dẫn đến các biểu thức rườm rà.

8.3 Sử dụng logic bậc nhất

Bây giờ chúng ta đã định nghĩa một ngôn ngữ logic biểu cảm, chúng ta hãy học cách sử dụng nó. Trong phần này, chúng tôi cung cấp các câu ví dụ trong một số lĩnh vực đơn giản. Trong biểu diễn tri thức, Miền một lĩnh vực chỉ là một phần nào đó của thế giới mà chúng ta muốn thể hiện một số kiến thức. Chúng ta bắt đầu với một mô tả ngắn gọn về giao diện TELL/ASK cho cơ sở tri thức bậc nhất. Sau đó, chúng ta nhìn vào các lĩnh vực của các mối quan hệ gia đình, con số, bộ và danh sách, và thế giới wumpus. Phần 8.4.2 chứa một ví dụ quan trọng hơn (mạch điện tử) và Chương 10 bao gồm mọi thứ trong vũ trụ.

8.3.1 Xác nhận và truy vấn trong logic bậc nhất

Các câu được thêm vào cơ sở tri thức bằng cách sử dụng TELL, chính xác như trong logic mệnh đề. Những câu như vậy được gọi là khẳng định. Ví dụ, chúng ta có thể khẳng định rằng John là một vị vua, Richard là một Khẳng định

Người, và tất cả các vị vua đều là người:

TELL(KB, Vua (John)) . TELL(KB, Người (Richard)) . TELL(KB,
 $\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$) . Chúng ta có thể đặt câu hỏi về cơ
 sở tri thức bằng cách sử dụng ASK. Chẳng hạn

ASK(KB, King(John)) trả về true. Các câu hỏi được đặt ra với ASK được gọi là truy
 vấn hoặc mục tiêu. Nói chung, bất kỳ truy vấn nào được kéo theo một cách hợp lý
 bởi cơ sở kiến thức nên được trả lời một cách khẳng định. Ví dụ, với ba xác nhận
 ở trên, truy vấn

ASK(KB, Person(John)) cũng sẽ trả về true. Chúng ta có
 thể hỏi các truy vấn định lượng, chẳng hạn như

ASK (KB, $\exists x \text{ Người}(x)$) . Câu trả lời là đúng, nhưng điều này có lẽ không hữu ích
 như chúng ta mong muốn. Nó giống như trả lời "Bạn có thể cho tôi biết thời gian
 không?" bằng "Có". Nếu chúng ta muốn biết giá trị của x makes câu đúng, chúng ta
 sẽ cần một hàm khác, mà chúng ta gọi là ASKVARs,

ASKVARs(KB, Person(x)) và mang lại một luồng câu trả lời. Trong trường hợp này
 sẽ có hai câu trả lời: { x /John} và { x /Richard}. Câu trả lời như vậy được gọi là danh
 sách thay thế hoặc ràng buộc. ASKVARs là

thường được dành riêng cho các cơ sở tri thức chỉ bao gồm các mệnh đề Horn, bởi
 vì trong các cơ sở tri thức như vậy, mọi cách để làm cho truy vấn đúng sẽ liên kết
 các biến với các giá trị cụ thể. Đó không phải là trường hợp của logic bậc nhất; trong
 một KB chỉ được thông báo rằng King(John) $\vee$ King(Richard) không có ràng buộc duy
 nhất với x làm cho truy vấn $\exists x \text{ King}(x)$ đúng, mặc dù truy vấn trên thực tế là đúng.

8.3.2 Miền họ hàng

Ví dụ đầu tiên chúng tôi xem xét là lĩnh vực của các mối quan hệ gia đình, hoặc quan hệ họ hàng.
 Miền này bao gồm các sự kiện như "Elizabeth là mẹ của Charles" và "Charles là cha của William"
 và các quy tắc như "Bà của một người là mẹ của cha mẹ của một người".

Rõ ràng, các đối tượng trong lĩnh vực của chúng tôi là con người. Vị ngữ đơn nguyên bao gồm
 Nam và Nữ, trong số những vị ngữ khác. Mối quan hệ họ hàng — tình cha mẹ, tình anh em, hôn
 nhân, v.v. — được phản ánh bởi các vị ngữ nhị phân: Cha mẹ, Anh chị em, Anh trai, Chị gái, Con
 gái, Con trai, Vợ, Chồng, Ông bà, Cháu cháu, Anh em họ, Dì và Bác. Chúng ta sử dụng các chức
 năng cho Mẹ và Cha, bởi vì mỗi người đều có chính xác một trong số này, về mặt sinh học (mặc
 dù chúng ta có thể giới thiệu các chức năng bổ sung cho mẹ nuôi, mẹ mang thai hộ, v.v.).

Chúng ta có thể xem qua từng chức năng và vị ngữ, viết ra những gì chúng ta biết về các
 ký hiệu khác. Ví dụ, mẹ của một người là cha mẹ của một người là nữ:

$\forall m, c \text{ Mẹ}(c) = m \Leftrightarrow \text{Nữ}(m) \wedge \text{Cha mẹ}(m, c)$.

Chồng của một người là vợ/chồng nam của một

người:

$\forall w, h \text{ Chồng}(h, w) \Leftrightarrow \text{Nam}(h) \wedge \text{Vợ} / \text{chồng}(h, w)$

. Cha mẹ và con là quan hệ nghịch đảo:

$\forall p, c \text{ Cha mẹ}(p, c) \Leftrightarrow \text{Con}(c, p)$.

Ông bà là cha mẹ của cha mẹ của một người:

$\forall g, c \text{ Ông bà } (g, c) \Leftrightarrow \exists p \text{ Cha mẹ } (g, p) \wedge \text{Cha mẹ } (p, c) .$

Anh chị em là một người con khác của cha mẹ của một người:

$\forall x, y \text{ Anh chị em } (x, y) \Leftrightarrow x \neq y \wedge \exists p \text{ Cha mẹ } (p, x) \wedge \text{Cha mẹ } (p, y) .$ Chúng ta có thể tiếp tục thêm vài trang như thế này và Bài tập 8.KINS yêu cầu bạn làm điều

Đôi câu này có thể được xem như một tiên đề của lĩnh vực họ hàng, như được giải thích trong Phần 7.1. Các tiên đề thường được liên kết với các lĩnh vực toán học thuần túy - chúng ta sẽ sớm thấy một số tiên đề cho các con số - nhưng chúng cần thiết trong tất cả các lĩnh vực. Chúng cung cấp thông tin thực tế cơ bản mà từ đó có thể rút ra kết luận hữu ích. Tiên đề họ hàng của chúng ta cũng là định nghĩa; chúng có dạng $\forall x, y P(x, y) \Leftrightarrow \dots$. Các tiên đề xác định Mẹ

Định nghĩa

chức năng và các vị ngữ Chồng, Nam, Cha mẹ, Ông bà và Anh chị em theo các vị ngữ khác. Các định nghĩa của chúng tôi "chạm đáy" ở một tập hợp các vị ngữ cơ bản (Child, Female, v.v.) theo thuật ngữ mà các vị ngữ khác được định nghĩa cuối cùng.

Đây là một cách tự nhiên để xây dựng biểu diễn của một miền, và nó giống với cách mà các gói phần mềm được xây dựng bởi các định nghĩa liên tiếp của các quy trình con từ các hàm thư viện nguyên thủy. Lưu ý rằng không nhất thiết phải có một tập hợp vị ngữ nguyên thủy duy nhất; chúng ta cũng có thể sử dụng Parent thay vì Child. Trong một số miền, như chúng tôi chỉ ra, không có tập hợp cơ bản nào có thể xác định rõ ràng.

Không phải tất cả các câu logic về một miền đều là tiên đề. Một số là các định lý - nghĩa là, chúng được bao gồm bởi các tiên đề. Ví dụ, hãy xem xét khẳng định rằng anh chị em là đối xứng:

$\forall x, y \text{ Anh chị em } (x, y) \Leftrightarrow \text{Anh chị em } (y, x) .$ Đây là một tiên đề hay một định lý?

Trên thực tế, đó là một định lý theo logic từ tiên đề xác định tình anh chị em. Nếu chúng ta HỎI cơ sở kiến thức câu này, nó sẽ trả về true.

Từ quan điểm logic thuần túy, một cơ sở tri thức chỉ cần chứa các tiên đề và không có định lý, bởi vì các định lý không làm tăng tập hợp các kết luận theo sau từ cơ sở tri thức. Từ quan điểm thực tế, các định lý là điều cần thiết để giảm chi phí tính toán để rút ra các câu mới. Không có chúng, một hệ thống lý luận phải bắt đầu từ các nguyên tắc đầu tiên mọi lúc, giống như một nhà vật lý phải suy ra các quy tắc giải tích cho mọi vấn đề mới.

Không phải tất cả các tiên đề đều là định nghĩa. Một số cung cấp thông tin tổng quát hơn về một số vị ngữ nhất định mà không cấu thành một định nghĩa. Thật vậy, một số vị ngữ không có định nghĩa hoàn toàn vì chúng ta không biết đủ để mô tả chúng một cách đầy đủ. Ví dụ, không có cách dứt khoát nào rõ ràng để hoàn thành câu

$\forall x \text{ Người } (x) \Leftrightarrow \dots$. May mắn thay, logic bậc nhất cho phép chúng ta sử dụng vị ngữ Person mà không cần xác định hoàn toàn nó. Thay vào đó, chúng ta có thể viết một phần đặc tả của các thuộc tính mà mọi người đều có và các thuộc tính tạo nên một cái gì đó thành một người:

$\forall x \text{ Người } (x) \Rightarrow \dots \forall x \dots \Rightarrow \text{người}(x) .$ Tiên đề cũng có thể là "sự thật đơn giản", chẳng hạn như Nam (Jim) và Vợ / chồng (Jim, Laura). Những sự kiện như vậy tạo thành các mô tả của các trường hợp vấn đề cụ thể, cho phép các câu hỏi cụ thể được

Trả lời. Nếu mọi thứ suôn sẻ, câu trả lời cho những câu hỏi này sẽ là các định lý theo sau các tiên đề.

Thông thường, người ta thấy rằng các câu trả lời mong đợi không được đưa ra — ví dụ, từ Spouse (Jim, Laura) người ta mong đợi (theo luật của nhiều quốc gia) có thể suy ra rằng $\neg$ Spouse (George, Laura); nhưng điều này không tuân theo các tiên đề được đưa ra trước đó — ngay cả sau khi chúng ta thêm Jim 6= George như được đề xuất trong Phần 8.2.8. Đây là một dấu hiệu cho thấy một tiên đề bị thiếu. Bài tập 8.HILL yêu cầu người đọc cung cấp nó.

8.3.3 Số, bộ và danh sách

Số tự nhiên

Những con số có lẽ là ví dụ sống động nhất về cách một lý thuyết lớn có thể được xây dựng từ một hạt nhân nhỏ của các tiên đề. Ở đây chúng tôi mô tả lý thuyết về số tự nhiên hoặc không âm

Tiên đề Peano

số nguyên. Chúng ta cần một vị ngữ NatNum sẽ đúng với các số tự nhiên; chúng ta cần một ký hiệu hằng số, 0; và chúng ta cần một ký hiệu hàm, S (người kế nhiệm). Tiên đề Peano

Xác định các số tự nhiên và phép cộng.⁸ Các số tự nhiên được định nghĩa đệ

quy: $\text{NatNum}(0) \wedge \forall n \text{ NatNum}(n) \Rightarrow \text{NatNum}(S(n))$. Nghĩa là, 0 là một số tự nhiên, và đối với mỗi đối tượng n, nếu n là một số tự nhiên, thì S(n) là một số tự nhiên. Vì vậy, các số tự nhiên là 0, S(0), S(S(0)), v.v. Chúng ta cũng cần tiên đề để hạn chế hàm kế tiếp:

$\forall n \ 0 \neq S(n) \wedge \forall m, n \ m \neq n \Rightarrow S(m) \neq S(n)$. Bây giờ chúng ta có thể định nghĩa phép cộng theo hàm kế nhiệm:

$\forall m \text{ NatNum}(m) \Rightarrow + (0, m) = m \wedge \forall m, n \text{ NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow + (S(m), n) = S(+ (m, n))$. Tiên đề đầu tiên nói rằng cộng 0 vào bất kỳ số tự nhiên nào m cho chính m cho m. Lưu ý việc sử dụng ký hiệu hàm nhị phân "+" trong thuật ngữ $+ (m, 0)$; Trong toán học thông thường, thuật ngữ sẽ được viết $m + 0$ bằng cách sử dụng ký hiệu tiền tố. (Ký hiệu chúng tôi đã sử dụng cho bậc nhất

Trung tố

Tiền tố

logic được gọi là tiền tố.) Để làm cho các câu của chúng tôi về các con số dễ đọc hơn, chúng tôi cho phép sử dụng

của ký hiệu tiền tố. Chúng ta cũng có thể viết S(n) là $n + 1$, vì vậy tiên đề thứ hai trở thành

$\forall m, n \text{ NatNum}(m) \wedge \text{NatNum}(n) \Rightarrow (m + 1) + n = (m + n) + 1$. Tiên đề này làm giảm việc bổ sung vào việc áp dụng lặp đi lặp lại của hàm kế nhiệm.

Đường cú pháp

Việc sử dụng ký hiệu tiền tố là một ví dụ về đường cú pháp, tức là một phần mở rộng của hoặc viết tắt của cú pháp tiêu chuẩn không thay đổi ngữ nghĩa. Bất kỳ câu nào sử dụng đường đều có thể được "khử đường" để tạo ra một câu tương đương trong logic bậc nhất thông thường. Một ví dụ khác là sử dụng dấu ngoặc vuông thay vì dấu ngoặc đơn để dễ dàng xem dấu ngoặc bên trái nào khớp với dấu ngoặc bên phải. Tuy nhiên, một ví dụ khác là thu gọn định lượng: thay thế $\forall x \forall y P(x, y)$ bằng $\forall x, y P(x, y)$. Khi chúng ta có phép cộng, thật đơn giản để định nghĩa phép nhân là phép cộng lặp lại, lũy thừa là phép nhân lặp lại, phép chia số nguyên và số dư, số nguyên tố, v.v. Do đó, toàn bộ lý thuyết số (bao gồm cả mật mã) có thể được xây dựng từ một hằng số, một hàm, một vị ngữ và bốn tiên đề.

⁸ Các tiên đề Peano cũng bao gồm nguyên tắc quy nạp, là một câu của logic bậc hai chứ không phải của logic bậc một. Tầm quan trọng của sự khác biệt này được giải thích trong Chương 9.

Lĩnh vực của tập hợp cũng là nền tảng cho toán học cũng như đối với lẽ thường. Cài Soning. (Trên thực tế, có thể định nghĩa lý thuyết số theo lý thuyết tập hợp.) Chúng ta muốn có thể đại diện cho các tập hợp riêng lẻ, bao gồm cả tập trống. Chúng ta cần một cách để xây dựng các tập từ các phần tử hoặc từ các hoạt động trên các tập khác. Chúng ta sẽ muốn biết liệu một phần tử có phải là thành viên của một tập hợp hay không và chúng ta sẽ muốn phân biệt các tập hợp với các đối tượng không phải là tập hợp.

Chúng ta sẽ sử dụng từ vựng thông thường của lý thuyết tập hợp làm đường cú pháp. Tập hợp trống là hằng số được viết là $\{\}$. Có một vị ngữ đơn một, Set, đúng với các tập hợp. Các vị ngữ nhị phân là $x \in s$ (x là thành viên của tập s) và $s_1 \subseteq s_2$ (tập s_1 là tập con của s_2 , có thể bằng s_2). Các hàm nhị phân là $s_1 \cap s_2$ (giao nhau), $s_1 \cup s_2$ (liên hợp) và $\text{Add}(x, s)$ (tập kết quả từ việc thêm phần tử x vào tập s). Một tập hợp các tiên đề có thể có như sau:

1. Các tập hợp duy nhất là tập hợp trống và những bộ được tạo ra bằng cách thêm thứ gì đó vào một tập hợp: $\forall s \text{ Set } (s \Leftrightarrow (s = \{\}) \vee (\exists x, s_2 \text{ Set } (s_2) \wedge s = \text{Add}(x, s_2)))$

2. Tập hợp trống không có yếu tố nào được thêm vào đó. Nói cách khác, không có cách nào để phân phối $\{\}$ thành một tập hợp nhỏ hơn và một phần tử: $\neg \exists x, s \text{ Add}(x, s) = \{\}$.

3. Thêm một phần tử đã có trong tập hợp không có tác dụng: $\forall x, s \ x \in s \Leftrightarrow s = \text{Add}(x, s)$.

4. Các thành viên duy nhất của một tập hợp là các yếu tố đã được thêm vào đó. Chúng tôi thể hiện điều này một cách đệ quy, nói rằng x là một thành viên của s nếu và chỉ khi s bằng một số phần tử được thêm vào một tập hợp nào đó s_2 , trong đó y giống với x hoặc x là một thành viên của s_2 : $\forall x, s \ x \in s \Leftrightarrow \exists y, s_2 (s = \text{Add}(y, s_2) \wedge (x = y \vee x \in s_2))$.

5. Một tập hợp con là một tập hợp con của một tập hợp khác nếu và chỉ khi tất cả các thành viên của tập thứ nhất là thành viên của tập thứ hai: $\forall s_1, s_2 \ s_1 \subseteq s_2 \Leftrightarrow (\forall x \ x \in s_1 \Rightarrow x \in s_2)$.

6. Hai tập hợp bằng nhau nếu và chỉ khi mỗi tập là một tập con của tập hợp kia: $\forall s_1, s_2 (s_1 = s_2) \Leftrightarrow (s_1 \subseteq s_2 \wedge s_2 \subseteq s_1)$.

7. Một đối tượng nằm trong giao điểm của hai tập nếu và chỉ khi nó là thành viên của cả hai tập hợp: $\forall x, s_1, s_2 \ x \in (s_1 \cap s_2) \Leftrightarrow (x \in s_1 \wedge x \in s_2)$.

8. Một đối tượng nằm trong sự kết hợp của hai tập hợp nếu và chỉ khi nó là thành viên của một trong hai tập hợp: $\forall x, s_1, s_2 \ x \in (s_1 \cup s_2) \Leftrightarrow (x \in s_1 \vee x \in s_2)$. Danh sách tương tự như

bộ. Sự khác biệt là danh sách được sắp xếp và cùng một phần tử có thể

Danh sách

xuất hiện nhiều lần trong danh sách. Chúng ta có thể sử dụng từ vựng của Lisp cho danh sách: Không có là danh sách hằng số không có phần tử; Nhược điểm, Bổ sung, Đầu tiên và Phần còn lại là các chức năng; và Find là vị ngữ làm cho danh sách những gì Member làm cho các tập hợp. List là một vị ngữ chỉ đúng với danh sách. Cũng như các tập hợp, người ta thường sử dụng đường cú pháp trong các câu logic liên quan đến danh sách. Danh sách trống là $[\]$. Thuật ngữ $\text{Cons}(x, \text{Nil})$ (tức là danh sách chứa phần tử x theo sau là không có gì) được viết là $[x]$. Danh sách một số phần tử, chẳng hạn như $[A, B, C]$, tương ứng với $\text{termCons}(A, \text{Cons}(B, \text{Cons}(C, \text{Nil})))$. Bài tập 8.LIST yêu cầu bạn viết ra các tiên đề cho danh sách.

8.3.4 Thế giới wumpus

Một số tiên đề logic mệnh đề cho thế giới wumpus đã được đưa ra trong Chương 7. Các tiên đề bậc nhất trong phần này ngắn gọn hơn nhiều, nắm bắt một cách tự nhiên chính xác những gì chúng ta muốn nói.

Hãy nhớ lại rằng tác nhân wumpus nhận được một vector nhận thức với năm yếu tố. Câu bậc nhất tương ứng được lưu trữ trong cơ sở kiến thức phải bao gồm cả nhận thức và thời gian nó xảy ra; Nếu không, đại lý sẽ bối rối không biết khi nào nó nhìn thấy cái gì. Chúng tôi sử dụng số nguyên cho các bước thời gian. Một câu nhận thức điển hình sẽ là

Nhận thức ([Mùi hôi thối, Gió, Lấp lánh, Không có, Không có], 5) .

Ở đây, Percept là một vị ngữ nhị phân, và Stench và vãn vãn là các hằng số được đặt trong một danh sách. Các hành động trong thế giới wumpus có thể được biểu diễn bằng các thuật ngữ logic:

Rẽ (phải), Rẽ (trái), Tiến lên, Bắn, Ngoáy, Leo .

Để xác định cái nào là tốt nhất, chương trình tác nhân thực hiện truy vấn

ASKVARS(KB, Hành động tốt nhất (a, 5)) ,

trả về một danh sách ràng buộc như {a/Grab}. Sau đó, chương trình đại lý có thể trả lại Grab như hành động cần thực hiện. Dữ liệu tri giác thô ngụ ý một số sự kiện nhất định về trạng thái hiện tại. Ví dụ:

$\forall t, s, g, w, c \text{ Percept}([s, \text{Breeze}, g, w, c], t) \Rightarrow \text{Breeze}(t) \forall t, s, g, w, c \text{ Percept}([s, \text{None}, g, w, c], t) \Rightarrow \neg \text{Breeze}(t) \forall t, s, b, w, c \text{ Percept}([s, b, \text{Glitter}, w, c], t) \Rightarrow \text{Glitter}(t) \forall t, s, b, w, c \text{ Percept}([s, b, \text{None}, w, c], t) \Rightarrow \neg \text{Glitter}(t)$, v.v. Những quy tắc này thể hiện một hình thức tầm thường của quá trình lý luận được gọi là nhận thức, mà chúng ta nghiên cứu sâu trong Chương 27. Lưu ý định lượng theo thời gian t . Trong mệnh đề luận, chúng ta sẽ cần các bản sao của mỗi câu cho mỗi bước thời gian.

Hành vi "phản xạ" đơn giản cũng có thể được thực hiện bằng các câu ngữ ý định lượng. Ví dụ, chúng ta có

$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$.

Với nhận thức và quy tắc từ các đoạn trước, điều này sẽ mang lại kết luận mong muốn $\text{BestAction}(\text{Grab}, 5)$ — nghĩa là, Grab là điều đúng đắn để làm.

Chúng tôi đã đại diện cho đầu vào và đầu ra của tác nhân; Bây giờ là lúc để đại diện cho chính môi trường. Chúng ta hãy bắt đầu với các đối tượng. Các ứng cử viên rõ ràng là hình vuông, hồ và thewumpus. Chúng ta có thể đặt tên cho mỗi hình vuông - Square1,2 và vãn vãn - nhưng thực tế là Square1,2 và Square1,3 liên kết sẽ phải là một sự kiện "bổ sung" và chúng ta sẽ cần một sự kiện như vậy cho mỗi cặp hình vuông. Tốt hơn là sử dụng một thuật ngữ phức tạp trong đó hàng và cột xuất hiện dưới dạng số nguyên; Ví dụ, chúng ta có thể chỉ cần sử dụng thuật ngữ danh sách [1, 2]. Sự tiếp giáp của bất kỳ hai ô vuông nào có thể được định nghĩa là

$\forall x, y, a, b \text{ Liên kết}([x, y], [a, b]) \Leftrightarrow (x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1))$

Chúng ta có thể đặt tên cho từng hồ, nhưng điều này sẽ không phù hợp vì một lý do khác: không có

lý do để phân biệt giữa các hố.9 Sẽ đơn giản hơn để sử dụng một vị ngữ đơn hợp Pit đúng của các ô vuông chứa hố. Cuối cùng, vì có chính xác một wumpus, một Wumpus hằng số cũng tốt như một vị ngữ đơn nguyên (và có lẽ trang nghiêm hơn từ quan điểm của wumpus).

Vị trí của tác nhân thay đổi theo thời gian, vì vậy chúng tôi viết $At(Agent, s, t)$ có nghĩa là tác nhân ở ô vuông s tại thời điểm t . Chúng ta có thể cố định wumpus vào một vị trí cụ thể mãi mãi với $At(Wumpus, [1, 3], t)$. Sau đó, chúng ta có thể nói rằng các đối tượng chỉ có thể ở một vị trí tại một thời điểm:

$\forall x, s1, s2, t \quad At(x, s1, t) \wedge At(x, s2, t) \Rightarrow s1 = s2$. Với vị trí hiện tại của nó, tác nhân có thể suy ra các thuộc tính của hình vuông từ các thuộc tính của nhận thức hiện tại của nó. Ví dụ, nếu tác nhân đang ở một hình vuông và cảm nhận được một làn gió, thì hình vuông đó là gió mát:

$\forall s, t \quad At(Agent, s, t) \wedge Breeze(t) \Rightarrow Breezy(s)$. Sẽ rất hữu ích khi biết rằng một hình vuông có gió mát vì chúng ta biết rằng các hố không thể di chuyển. Lưu ý rằng Breezy không có lập luận về thời gian.

Sau khi phát hiện ra những nơi nào có gió (hoặc có mùi) và rất quan trọng là không có gió (hoặc không có mùi), tác nhân có thể suy ra vị trí của các hố (và vị trí của wumpus). Trong khi logic mệnh đề đòi hỏi một tiên đề riêng biệt cho mỗi hình vuông (xem R2 và R3 ở trang 238) và sẽ cần một tập hợp các tiên đề khác nhau cho mỗi bố cục địa lý của thế giới, logic bậc nhất chỉ cần một tiên đề:

$$\forall s \quad Breezy(s) \Leftrightarrow \exists r \quad Adjacent(r, s) \wedge Pit(r) \quad (8.4)$$

Tương tự, trong logic bậc nhất, chúng ta có thể định lượng theo thời gian, vì vậy chúng ta chỉ cần một trạng thái kế thừa cho mỗi vị ngữ, thay vì một bản sao khác nhau cho mỗi bước thời gian. Ví dụ, tiên đề cho mũi tên (Phương trình (7.2) trên trang 258) trở thành

$\forall t \quad HaveArrow(t+1) \Leftrightarrow (HaveArrow(t) \wedge \neg Action(Shoot, t))$. Từ hai câu ví dụ này, chúng ta có thể thấy rằng công thức logic bậc nhất không kém phần ngắn gọn so với mô tả tiếng Anh gốc được đưa ra trong Chương 7. Người đọc được mời xây dựng các tiên đề tương tự cho vị trí và định hướng của tác nhân; Trong những trường hợp này, các tiên đề định lượng qua cả không gian và thời gian. Như trong trường hợp ước tính trạng thái mệnh đề, một tác nhân có thể sử dụng suy luận logic với các tiên đề loại này để theo dõi các khía cạnh của thế giới không được quan sát trực tiếp. Chương 11 đi sâu hơn về chủ đề các tiên đề nhà nước kế thừa bậc nhất và việc sử dụng chúng để xây dựng các kế hoạch.

8.4 Kỹ thuật tri thức trong logic bậc nhất

Phần trước đã minh họa việc sử dụng logic bậc nhất để biểu diễn kiến thức trong ba lĩnh vực đơn giản. Phần này mô tả quá trình chung của xây dựng cơ sở tri thức - một quá trình được gọi là kỹ thuật tri thức. Kỹ sư tri thức là người điều tra

Kỹ thuật tri thức

một lĩnh vực cụ thể, tìm hiểu những khái niệm nào là quan trọng trong lĩnh vực đó và tạo ra một đại diện cho các đối tượng và mối quan hệ trong lĩnh vực đó. Chúng tôi minh họa quy trình kỹ thuật tri thức trong một lĩnh vực mạch điện tử. Cách tiếp cận chúng tôi thực hiện phù hợp để phát triển các cơ sở tri thức có mục đích đặc biệt mà lĩnh vực được giới hạn cẩn thận và

9 Tương tự như vậy, hầu hết chúng ta không đặt tên cho từng con chim bay trên đầu khi nó di cư đến những vùng ấm hơn vào mùa đông. Nhà dinh dưỡng học muốn nghiên cứu các mô hình di cư, tỷ lệ sống sót, v.v. đã đặt tên cho từng con chim, bằng cách cày chân của nó, bởi vì các loài chim riêng lẻ phải được theo dõi.

có phạm vi truy vấn được biết trước. Cơ sở tri thức mục đích chung, bao gồm nhiều kiến thức của con người và nhằm hỗ trợ các nhiệm vụ như hiểu ngôn ngữ tự nhiên, được thảo luận trong Chương 10.

8.4.1 Quy trình kỹ thuật tri thức

Các dự án kỹ thuật tri thức rất khác nhau về nội dung, phạm vi và độ khó, nhưng tất cả các dự án như vậy bao gồm các bước sau:

1. Xác định các câu hỏi. Kỹ sư tri thức phải mô tả phạm vi câu hỏi mà cơ sở tri thức sẽ hỗ trợ và các loại dữ kiện sẽ có sẵn cho từng trường hợp vấn đề cụ thể. Ví dụ, cơ sở tri thức wumpus cần có khả năng lựa chọn hành động, hay chỉ cần trả lời các câu hỏi về nội dung của môi trường? Các dữ kiện cảm biến có bao gồm vị trí hiện tại không? Nhiệm vụ sẽ xác định kiến thức nào phải được thể hiện để kết nối các trường hợp vấn đề với câu trả lời. Bước này tương tự như quy trình PEAS để thiết kế các tác nhân trong Chương 2.

Tiếp thu kiến thức

2. Tập hợp các kiến thức liên quan. Kỹ sư tri thức có thể đã là một chuyên gia trong lĩnh vực này, hoặc có thể cần làm việc với các chuyên gia thực sự để trích xuất những gì họ biết - một quá trình được gọi là thu thập kiến thức. Ở giai đoạn này, kiến thức không được thể hiện

Chính thức. Ý tưởng là để hiểu phạm vi của cơ sở tri thức, như được xác định bởi nhiệm vụ và để hiểu lĩnh vực thực sự hoạt động như thế nào.

Đối với thế giới wumpus, được xác định bởi một bộ quy tắc nhân tạo, kiến thức có liên quan rất dễ xác định. (Tuy nhiên, lưu ý rằng định nghĩa về sự liên kết không được cung cấp rõ ràng trong các quy tắc thế giới wumpus.) Đối với các lĩnh vực thực, vấn đề liên quan có thể khá khó khăn — ví dụ, một hệ thống mô phỏng thiết kế VLSI có thể cần hoặc không cần tính đến điện dung đi lạc và hiệu ứng da.

3. Quyết định từ vựng của vị ngữ, hàm và hằng số. Đó là, dịch các khái niệm cấp miền quan trọng thành tên cấp logic. Điều này liên quan đến nhiều câu hỏi về phong cách kỹ thuật tri thức. Giống như phong cách lập trình, điều này có thể có tác động đáng kể đến sự thành công cuối cùng của dự án. Ví dụ, các hồ nên được biểu diễn bởi các đối tượng hay bằng một vị ngữ đơn nguyên trên các ô vuông? Định hướng của tác nhân nên là chức năng hay vị ngữ? Vị trí của wumpus có nên phụ thuộc vào thời gian không? Một khi các lựa chọn đã được thực hiện, kết quả là một từ vựng được gọi là bản thể luận của

Ontology

miền. Từ bản thể luận có nghĩa là một lý thuyết cụ thể về bản chất của hiện hữu hoặc tồn tại. Bản thể luận xác định những loại sự vật nào tồn tại, nhưng không xác định các thuộc tính cụ thể và mối quan hệ lẫn nhau của chúng.

4. Mã hóa kiến thức chung về miền. Kỹ sư tri thức viết ra các tiên đề cho tất cả các thuật ngữ từ vựng. Điều này xác định (trong phạm vi có thể) ý nghĩa của các thuật ngữ, cho phép chuyên gia kiểm tra nội dung. Thông thường, bước này tiết lộ những quan niệm sai lầm hoặc lỗ hổng trong từ vựng phải được khắc phục bằng cách quay lại bước 3 và lập lại quy trình.

5. Mã hóa mô tả của trường hợp sự cố. Nếu bản thể học được suy nghĩ kỹ lưỡng, bước này rất dễ dàng. Nó liên quan đến việc viết các câu nguyên tử đơn giản về các trường hợp của các khái niệm đã là một phần của bản thể luận. Đối với một tác nhân logic, các trường hợp vấn đề được cung cấp bởi các cảm biến, trong khi cơ sở tri thức "không có cơ thể" được đưa ra các câu giống như cách các chương trình truyền thống được cung cấp dữ liệu đầu vào.

6. Đặt câu hỏi cho quy trình suy luận và nhận câu trả lời. Đây là nơi phần thưởng: chúng ta có thể để quy trình suy luận hoạt động dựa trên các tiên đề và các sự kiện cụ thể theo vấn đề để rút ra các sự kiện mà chúng ta muốn biết. Do đó, chúng tôi tránh phải viết một thuật toán giải pháp dành riêng cho ứng dụng.

7. Gỡ lỗi và đánh giá cơ sở kiến thức. Than ôi, câu trả lời cho các truy vấn sẽ hiếm khi chính xác trong lần thử đầu tiên. Chính xác hơn, câu trả lời sẽ chính xác cho cơ sở tri thức như đã viết, giả sử rằng quy trình suy luận là hợp lý, nhưng chúng sẽ không phải là những câu trả lời mà người dùng mong đợi. Ví dụ: nếu thiếu một tiên đề, một số truy vấn sẽ không thể trả lời được từ cơ sở tri thức. Một quá trình gỡ lỗi đáng kể có thể xảy ra. Các tiên đề bị thiếu hoặc các tiên đề quá yếu có thể dễ dàng được xác định bằng cách nhận thấy những nơi mà chuỗi lý luận dừng lại bất ngờ. Ví dụ, nếu cơ sở kiến thức bao gồm một quy tắc chẩn đoán (xem Bài tập 8.WUMD) để tìm wumpus, $\forall s \text{ Smelly}(s) \Rightarrow \text{Adjacent}(\text{Home}(\text{Wumpus}), s)$, thay vì hai điều kiện, thì tác nhân sẽ không bao giờ có thể chứng minh sự vắng mặt của wumpus. Các tiên đề không chính xác có thể được xác định vì chúng là những tuyên bố sai lệch về thế giới. Ví dụ: câu $\forall x \text{ NumOfLegs}(x, 4) \Rightarrow \text{Mammal}(x)$

là sai đối với bò sát, động vật lưỡng cư và bần. Sự sai lầm của câu này có thể được xác định độc lập với phần còn lại của cơ sở kiến thức. Ngược lại, một lỗi điển hình

trong một chương trình trông như thế này:

offset = vị trí + 1. Không thể biết độ lệch nên là vị trí hay vị trí + 1 mà không hiểu ngữ cảnh xung quanh. Khi bạn đạt đến điểm mà không có lỗi rõ ràng trong cơ sở kiến thức của mình, thật hấp dẫn để tuyên bố thành công. Nhưng trừ khi rõ ràng là không có lỗi, tốt hơn là bạn nên đánh giá chính thức hệ thống của bạn bằng cách chạy nó trên một bộ truy vấn thử nghiệm và đo lường số lượng bạn nhận được. Nếu không đo lường khách quan, quá dễ dàng để thuyết phục bản thân rằng công việc đã hoàn thành. Để hiểu rõ hơn về quy trình bảy bước này, bây giờ chúng ta áp dụng nó cho một ví dụ mở rộng - lĩnh vực của mạch điện tử.

8.4.2 Miền mạch điện tử

Chúng ta sẽ phát triển một bản thể và cơ sở kiến thức cho phép chúng ta suy luận về các mạch kỹ thuật số thuộc loại được hiển thị trong Hình 8.6. Chúng tôi tuân theo quy trình bảy bước cho kỹ thuật tri thức.

Xác định các câu hỏi

Có rất nhiều nhiệm vụ lý luận liên quan đến mạch kỹ thuật số. Ở cấp độ cao nhất, người ta phân tích chức năng của mạch. Ví dụ, mạch trong Hình 8.6 có thực sự thêm đúng cách không? Nếu tất cả các đầu vào đều cao, đầu ra của cổng A2 là bao nhiêu? Các câu hỏi về cấu trúc của mạch cũng rất thú vị. Ví dụ, tất cả các cổng được kết nối với thiết bị đầu cuối đầu vào đầu tiên là gì? Mạch có chứa các vòng phản hồi không? Đây sẽ là nhiệm vụ của chúng tôi trong phần này. Có các cấp độ phân tích chi tiết hơn, bao gồm cả những mức độ liên quan đến độ trễ thời gian, diện tích mạch, mức tiêu thụ điện năng, chi phí sản xuất, v.v. Mỗi cấp độ này sẽ yêu cầu kiến thức bổ sung.

Tập hợp các kiến thức liên quan

Chúng ta biết gì về mạch kỹ thuật số? Đối với mục đích của chúng tôi, chúng bao gồm dây và cổng. Tín hiệu chạy dọc theo dây đến các đầu vào của cổng và mỗi cổng tạo ra một tín hiệu trên đầu cuối đầu ra chạy dọc theo một dây khác. Để xác định những tín hiệu này sẽ là gì, chúng ta cần biết cách các cổng biến đổi tín hiệu đầu vào của chúng. Có bốn loại cổng: cổng AND, OR và XOR có hai đầu cuối đầu vào và cổng NOT có một. Allgates có một thiết bị đầu cuối đầu ra. Các mạch, giống như cổng, có các thiết bị đầu cuối đầu vào và đầu ra.

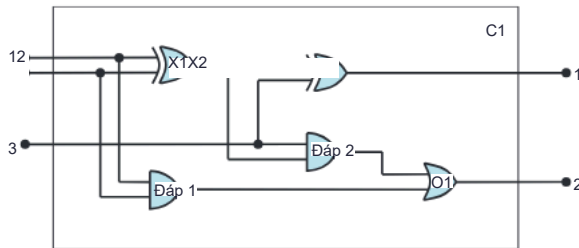
Để suy luận về chức năng và kết nối, chúng ta không cần phải nói về bản thân dây, đường đi của chúng hoặc các mối nối nơi chúng kết hợp với nhau. Tất cả những gì quan trọng là kết nối giữa các thiết bị đầu cuối — chúng ta có thể nói rằng một thiết bị đầu cuối đầu ra được kết nối với một thiết bị đầu cuối đầu vào khác mà không cần phải nói điều gì thực sự kết nối chúng. Các yếu tố khác như kích thước, hình dạng, màu sắc hoặc chi phí của các thành phần khác nhau không liên quan đến phân tích của chúng tôi.

Nếu mục đích của chúng tôi là một cái gì đó khác hơn là xác minh thiết kế ở cấp độ cổng, thì bản thể sẽ khác. Ví dụ, nếu chúng ta quan tâm đến việc gỡ lỗi các mạch bị lỗi, thì có lẽ sẽ là một ý tưởng hay để đưa các dây vào bản thể, bởi vì một dây bị lỗi có thể làm hỏng tín hiệu chạy dọc theo nó. Để giải quyết lỗi thời gian, chúng ta sẽ cần bao gồm độ trễ cổng. Nếu chúng ta quan tâm đến việc thiết kế một sản phẩm có lợi nhuận, thì chi phí của mạch và tốc độ của nó so với các sản phẩm khác trên thị trường sẽ rất quan trọng.

Quyết định từ vựng

Bây giờ chúng ta biết rằng chúng ta muốn nói về mạch, thiết bị đầu cuối, tín hiệu và cổng. Bước tiếp theo là chọn hàm, vị ngữ và hằng số để biểu diễn chúng. Đầu tiên, chúng ta cần có khả năng phân biệt các cổng với nhau và với các đối tượng khác. Mỗi cổng được biểu diễn cho đối tượng asan được đặt tên bởi một hằng số, mà chúng ta khẳng định rằng nó là một cổng với, chẳng hạn như Gate(X1). Hành vi của mỗi cổng được xác định bởi loại của nó: một trong các hằng số AND, OR, XOR, hoặc NOT. Bởi vì một cổng có chính xác một loại, một hàm là thích hợp: $Type(X1) = XOR$. Các mạch, giống như cổng, được xác định bằng một vị ngữ: Mạch (C1).

Tiếp theo, chúng ta xem xét các thiết bị đầu cuối, được xác định bởi vị ngữ Terminal(x). Một mạch có thể có một hoặc nhiều đầu cuối đầu vào và một hoặc nhiều đầu cuối đầu ra. Chúng tôi sử dụng hàm



Hình 8.6 Một mạch kỹ thuật số C1, có mục đích là một bộ cộng đầy đủ một bit. Hai đầu vào đầu tiên là hai bit được thêm vào và đầu vào thứ ba là bit carry. Đầu ra đầu tiên là tổng và đầu ra thứ hai là bit mang cho bộ cộng tiếp theo. Mạch chứa hai cổng XOR, hai cổng AND và một cổng OR.

In (1, X1) để biểu thị đầu cuối đầu vào đầu tiên cho mạch X1. Một chức năng tương tự Out (n, c) được sử dụng cho các thiết bị đầu cuối đầu ra. Vị ngữ Arity (c, i, j) nói rằng mạch c có đầu vào i và đầu ra j. Kết nối giữa các cổng có thể được biểu diễn bằng một vị ngữ, Đã kết nối, lấy hai thiết bị đầu cuối làm đối số, như trong Connected(Out(1, X1), In(1, X2)).
 Cùng, chúng ta cần biết tín hiệu đang bật hay tắt. Một khả năng là sử dụng vị ngữ đơn nghĩa, On(t), đúng khi tín hiệu tại thiết bị đầu cuối được bật. Tuy nhiên, điều này làm cho việc đặt ra các câu hỏi như "Tất cả các giá trị có thể có của tín hiệu ở các cực đầu ra của mạch C1 là gì?" Do đó, chúng tôi giới thiệu các đối tượng hai giá trị tín hiệu, 1 và 0, tương ứng đại diện cho "bật" và "tắt" và một hàm Tín hiệu (t) biểu thị giá trị tín hiệu cho thiết bị đầu cuối t.

Mã hóa kiến thức chung về miền

Một dấu hiệu cho thấy chúng ta có một bản thể luận tốt là chúng ta chỉ yêu cầu một vài quy tắc chung, có thể được nêu rõ ràng và ngắn gọn. Đây là tất cả các tiên đề mà chúng ta sẽ cần:

1. Nếu hai thiết bị đầu cuối được kết nối, thì chúng có cùng tín hiệu: $\forall t_1, t_2 \text{ Thiết bị đầu cuối } (t_1) \wedge \text{Thiết bị đầu cuối } (t_2) \wedge \text{Kết nối } (t_1, t_2) \Rightarrow \text{Tín hiệu } (t_1) = \text{Tín hiệu } (t_2)$.
2. Tín hiệu ở mọi thiết bị đầu cuối là 1 hoặc 0: $\forall t \text{ Thiết bị đầu cuối } (t) \Rightarrow \text{Tín hiệu } (t) = 1 \vee \text{Tín hiệu } (t) = 0$.
3. Được kết nối là giao hoán: $\forall t_1, t_2 \text{ Đã kết nối } (t_1, t_2) \Leftrightarrow \text{Đã kết nối } (t_2, t_1)$.
4. Có bốn loại cổng: $\forall g \text{ Cổng } (g) \wedge k = \text{Loại } (g) \Rightarrow k = \text{VÀ} \vee k = \text{HOẶC} \vee k = \text{XOR} \vee k = \text{KHÔNG}$.
5. Đầu ra của cổng AND là 0 nếu và chỉ khi bất kỳ đầu vào nào của nó là 0: $\forall g \text{ Cổng } (g) \wedge \text{Loại } (g) = \text{VÀ} \Rightarrow \text{Tín hiệu } (Ra(1, g)) = 0 \Leftrightarrow \exists n \text{ Tín hiệu } (Trong(n, g)) = 0$.
6. Đầu ra của cổng OR là 1 nếu và chỉ khi bất kỳ đầu vào nào của nó là 1: $\forall g \text{ Cổng } (g) \wedge \text{Loại } (g) = \text{HOẶC} \Rightarrow \text{Tín hiệu } (Ra(1, g)) = 1 \Leftrightarrow \exists n \text{ Tín hiệu } (Trong(n, g)) = 1$.
7. Đầu ra của cổng XOR là 1 nếu và chỉ khi đầu vào của nó khác nhau: $\forall g \text{ Cổng } (g) \wedge \text{Loại } (g) = \text{XOR} \Rightarrow \text{Tín hiệu } (Ra(1, g)) = 1 \Leftrightarrow \text{Tín hiệu } (Trong(1, g)) \neq \text{Tín hiệu } (Trong(2, g))$.
8. Đầu ra của cổng NOT khác với đầu vào của nó: $\forall g \text{ Cổng } (g) \wedge \text{Loại } (g) = \text{KHÔNG} \Rightarrow \text{Tín hiệu } (Ra(1, g)) \neq \text{Tín hiệu } (Trong(1, g))$.
9. Các cổng (ngoại trừ NOT) có hai đầu vào và một đầu ra. $\forall g \text{ Cổng } (g) \wedge \text{Loại } (g) = \text{KHÔNG} \Rightarrow \text{Arity } (g, 1, 1)$. $\forall g \text{ Cổng } (g) \wedge k = \text{Loại } (g) \wedge (k = \text{VÀ} \vee k = \text{HOẶC} \vee k = \text{XOR}) \Rightarrow \text{Arity } (g, 2, 1)$.
10. Một mạch có các thiết bị đầu cuối, lên đến độ phân cực đầu vào và đầu ra của nó, và không có gì ngoài độ phân cực của nó: $\forall c, i, j \text{ Mạch } (c) \wedge \text{Arity } (c, i, j) \Rightarrow$
 $\forall n (n \leq i \Rightarrow \text{Thiết bị đầu cuối } (In(n, c))) \wedge (n > i \Rightarrow In(n, c) = \text{Không có gì})$
 $\wedge \forall n (n \leq j \Rightarrow \text{Thiết bị đầu cuối } (Ra(n, c))) \wedge (n > j \Rightarrow Ra(n, c) = \text{Không có gì})$

11. Các cổng, thiết bị đầu cuối và tín hiệu đều khác nhau. $\forall g, t, s$ Cổng (g) $\wedge$ Thiết bị đầu cuối (t) $\wedge$ (các) tín hiệu $\Rightarrow$

$$g \ 6 = t \wedge g \ 6 = s \wedge t \ 6 = s .$$

12. Cổng là mạch. $\forall g$ Cổng (g) $\Rightarrow$ Mạch (g)

Mã hóa phiên bản sự cố cụ thể

Mạch hiển thị trong Hình 8.6 được mã hóa dưới dạng mạch C1 với mô tả sau. Đầu tiên, chúng tôi phân loại mạch và các cổng thành phần của nó:

Mạch (C1) $\wedge$ Arity (C1, 3, 2) Cổng (X1) $\wedge$
 Loại (X1) = XORGate (X2) $\wedge$ Loại (X2) =
 XORG (A1) $\wedge$ Loại (A1) = ANDGate (A2)
 $\wedge$ Loại (A2) = ANDGate (O1) $\wedge$ Loại (O1)
 = HOẶC . Sau đó, chúng tôi chỉ ra các
 mối liên hệ giữa chúng:

Đã kết nối (Ra (1, X1), Vào (1, X2)) Đã kết nối (In (1, C1), In (1, X1))
 Đã kết nối (Ra (1, X1), Vào (2, A2)) Đã kết nối (In (1, C1), In (1, A1))
 Đã kết nối (Out (1, A2), In (1, O1)) Đã kết nối (In (2, C1), In (2, X1))
 Đã kết nối (Ra (1, A1), Vào (2, O1)) Đã kết nối (In (2, C1), In (2, A1))
 Đã kết nối (Out (1, X2), Out (1, C1)) Đã kết nối (In (3, C1), In (2, X2))
 Đã kết nối (Out (1, O1), Out (2, C1)) Đã kết nối (In (3, C1), In (1, A2)) .

Đặt truy vấn cho quy trình suy luận

Những tổ hợp đầu vào nào sẽ làm cho đầu ra đầu tiên của C1 (bit tổng) là 0 và đầu ra thứ hai của C1 (bit carry) là 1?

$$\exists i1, i2, i3 \text{ Tín hiệu (In (1, C1))} = i1 \wedge \text{Tín hiệu (In (2, C1))} = i2 \wedge \text{Tín hiệu (In (3, C1))} = i3 \\ \wedge \text{Tín hiệu (Out (1, C1))} = 0 \wedge \text{Tín hiệu (Out (2, C1))} = 1 .$$

Câu trả lời là thay thế cho các biến $i1$, $i2$ và $i3$ sao cho câu kết quả được bao gồm bởi cơ sở tri thức. ASKVARs sẽ cung cấp cho chúng tôi ba sự thay thế như vậy:

$$\{i1/1, i2/1, i3/0\} \quad \{i1/1, i2/0, i3/1\} \quad \{i1/0, i2/1, i3/1\} .$$

Các tập hợp giá trị có thể có của tất cả các thiết bị đầu cuối cho mạch cộng là gì?

$$\exists i1, i2, i3, o1, o2 \text{ Tín hiệu (In (1, C1))} = i1 \wedge \text{Tín hiệu (In (2, C1))} = i2 \\ \wedge \text{Tín hiệu (In (3, C1))} = i3 \wedge \text{Tín hiệu (Out (1, C1))} = o1 \wedge \text{Tín hiệu (Out (2, C1))} = o2 .$$

Truy vấn cuối cùng này sẽ trả về một bảng đầu vào-đầu ra hoàn chỉnh cho thiết bị, có thể được sử dụng để kiểm tra xem nó có thực sự thêm đầu vào của nó một cách chính xác hay không. Đây là một ví dụ đơn giản về xác minh mạch. Chúng ta cũng có thể sử dụng định nghĩa của mạch để xây dựng các hệ thống kỹ thuật số lớn hơn, cho

mà cùng một loại thủ tục xác minh có thể được thực hiện. (Xem Bài tập 8.ADDR.) Nhiều lĩnh vực phù hợp với cùng một loại phát triển cơ sở tri thức có cấu trúc, trong đó các khái niệm phức tạp hơn được xác định trên các khái niệm đơn giản hơn.

Gỡ lỗi cơ sở kiến thức

Chúng ta có thể làm xáo trộn cơ sở kiến thức theo nhiều cách khác nhau để xem những loại hành vi sai lầm nào xuất hiện. Ví dụ, giả sử chúng ta không đọc được Phần 8.2.8 và do đó quên khẳng định rằng $1 \neq 0$. Giả sử chúng ta thấy rằng hệ thống không thể chứng minh bất kỳ đầu ra nào cho mạch, ngoại trừ các trường hợp đầu vào 000 và 110. Chúng ta có thể xác định chính xác vấn đề bằng cách yêu cầu đầu ra của mỗi cổng. Ví dụ, chúng ta có thể hỏi

$\exists i1, i2, o$ Tín hiệu $(In(1, C1)) = i1 \wedge$ Tín hiệu $(In(2, C1)) = i2 \wedge$ Tín hiệu $(Out(1, X1)) = o$, tiết lộ rằng không có đầu ra nào được biết ở X1 cho các trường hợp đầu vào 10 và 01. Sau đó, chúng ta xem xét tiên đề cho các cổng XOR, như được áp dụng cho

X1: Tín hiệu $(Out(1, X1)) = 1 \Leftrightarrow$ Tín hiệu $(In(1, X1)) \neq$ Tín hiệu $(In(2, X1))$.

Nếu các đầu vào được biết là, ví dụ, 1 và 0, thì điều này giảm xuống còn

Tín hiệu $(Ra(1, X1)) = 1 \Leftrightarrow 1 \neq 0$. Bây giờ vấn đề là rõ ràng: hệ thống không thể suy ra rằng $Signal(Out(1, X1)) = 1$, vì vậy chúng ta cần cho nó biết rằng $1 \neq 0$.

Tóm tắt

Chương này đã giới thiệu logic bậc nhất, một ngôn ngữ biểu diễn mạnh mẽ hơn nhiều so với logic mệnh đề. Những điểm quan trọng như sau:

- Ngôn ngữ biểu diễn tri thức phải khai báo, bố cục, biểu cảm, độc lập ngữ cảnh và rõ ràng.
- Logic khác nhau trong các cam kết bản thể luận và cam kết nhận thức luận của chúng. Trong khi logic mệnh đề chỉ cam kết với sự tồn tại của các sự kiện, logic bậc nhất kết hợp với sự tồn tại của các đối tượng và các mối quan hệ và do đó đạt được sức mạnh biểu hiện, chiếm đoạt cho các lĩnh vực như thế giới wumpus và các mạch điện tử.
- Cả logic mệnh đề và logic bậc nhất đều có chung một khó khăn trong việc biểu diễn các mệnh đề mở hồ. Khó khăn này hạn chế khả năng ứng dụng của chúng trong các lĩnh vực đòi hỏi phán đoán cá nhân, như chính trị hoặc ẩm thực.
- Cú pháp của logic bậc nhất được xây dựng dựa trên cú pháp của logic mệnh đề. Nó thêm các thuật ngữ để biểu diễn các đối tượng và có các định lượng phổ quát và hiện sinh để xây dựng các khẳng định về tất cả hoặc một số giá trị có thể có của các biến định lượng.
- Một thế giới khả thi, hoặc mô hình, cho logic bậc nhất bao gồm một tập hợp các đối tượng và một sự liên kết ánh xạ các ký hiệu hằng số với các đối tượng, ký hiệu vị ngữ với các mối quan hệ giữa các đối tượng và ký hiệu hàm với các hàm trên đối tượng.
- Một câu nguyên tử chỉ đúng khi mối quan hệ được đặt tên bởi vị ngữ giữ giữa các đối tượng được đặt tên bởi các thuật ngữ. Giải thích mở rộng, ánh xạ các biến thể định lượng với các đối tượng trong mô hình, xác định sự thật của các câu định lượng.
- Phát triển cơ sở kiến thức về logic bậc nhất đòi hỏi một quá trình phân tích lĩnh vực cần thận, chọn từ vựng và mã hóa các tiên đề cần thiết để hỗ trợ các suy luận mong muốn.

Ghi chú thư mục và lịch sử

Mặc dù logic của Aristotle đề cập đến sự khái quát hóa đối với các đối tượng, nhưng nó đã không đạt được sức mạnh áp lực của logic bậc nhất. Một rào cản lớn đối với sự phát triển hơn nữa của nó là sự tập trung của nó vào các vị ngữ một vị trí để loại trừ các vị ngữ quan hệ nhiều nơi. Cách xử lý có hệ thống đầu tiên về các mối quan hệ được đưa ra bởi Augustus De Morgan (1864), người đã trích dẫn ví dụ sau đây để chỉ ra các loại suy luận mà logic của Aristotle không thể xử lý: "Tất cả ngựa là động vật; do đó, đầu của một con ngựa là đầu của một con vật." Aristotle không thể tiếp cận suy luận này vì bất kỳ quy tắc hợp lệ nào có thể hỗ trợ suy luận này trước tiên phải phân tích câu bằng cách sử dụng vị ngữ hai vị trí "x là đầu của y". Logic của các mối quan hệ được nghiên cứu sâu bởi Charles Sanders Peirce (Peirce, 1870; Misak, 2004).

Logic bậc nhất thực sự bắt nguồn từ việc giới thiệu các định lượng trong Begriffsschrift của Gottlob Frege (1879) ("Viết khái niệm" hoặc "Ký hiệu khái niệm"). Peirce (1883) cũng phát triển logic bậc nhất độc lập với Frege, mặc dù muộn hơn một chút. Khả năng lồng các quan-tifier của Frege là một bước tiến lớn, nhưng anh ấy đã sử dụng một ký hiệu khó xử. Ký hiệu hiện tại cho logic bậc nhất về cơ bản là do Giuseppe Peano (1889), nhưng ngữ nghĩa hầu như giống hệt với Frege. Thật kỳ lạ, các tiền đề của Peano phần lớn là do Grassmann (1861) và Dedekind (1888).

Leopold Löwenheim (1915) đã đưa ra một cách xử lý có hệ thống lý thuyết mô hình cho logic bậc nhất, bao gồm cả cách xử lý thích hợp đầu tiên đối với ký hiệu bình đẳng. Kết quả của Löwenheim được mở rộng thêm bởi Thoralf Skolem (1920). Alfred Tarski (1935, 1956) đã đưa ra một định nghĩa rõ ràng về sự thật và sự thỏa mãn lý thuyết mô hình trong logic bậc nhất, sử dụng lý thuyết tập hợp.

John McCarthy (1958) chịu trách nhiệm chính về việc giới thiệu logic bậc nhất như một công cụ để xây dựng hệ thống AI. Triển vọng của AI dựa trên logic đã được nâng cao một cách đáng kể bởi sự phát triển của Robinson (1965) về độ phân giải, một quy trình hoàn chỉnh cho suy luận bậc nhất. Cách tiếp cận logic học bắt nguồn từ Đại học Stanford. Cordell Green (1969a, 1969b) đã phát triển một hệ thống suy luận bậc nhất, QA3, dẫn đến những nỗ lực đầu tiên để chế tạo một robot logic tại SRI (Fikes và Nilsson, 1971). Logic bậc nhất được áp dụng bởi Zohar Manna và Richard Waldinger (1971) để suy luận về các chương trình và sau đó bởi Michael Genesereth (1984) để suy luận về mạch. Ở châu Âu, lập trình logic (một dạng hạn chế của suy luận bậc nhất) đã được phát triển để phân tích ngôn ngữ (Colmerauer et al., 1973) và cho các hệ thống khai báo chung (Kowalski, 1974). Logic tính toán cũng được phát triển tốt tại Edinburgh thông qua dự án LCF (Logic cho các hàm có thể tính toán) (Gordon et al., 1979). Những phát triển này được ghi lại thêm trong Chương 9 và 10.

Các ứng dụng thực tế được xây dựng với logic bậc nhất bao gồm một hệ thống để đánh giá các yêu cầu sản xuất con người cho các sản phẩm điện tử (Mannion, 2002), một hệ thống lý luận về các chính sách truy cập tệp và quản lý quyền kỹ thuật số (Halpern và Weissman, 2008), và một hệ thống cho thành phần tự động của các dịch vụ Web (McIlraith và Zeng, 2001).

Phản ứng đối với giả thuyết Whorf (Whorf, 1956) và vấn đề ngôn ngữ và tư tưởng nói chung, xuất hiện trong nhiều cuốn sách (Pullum, 1991; Pinker, 2003) bao gồm các tiêu đề dường như đối lập Tại sao thế giới trông khác nhau trong các ngôn ngữ khác (Deutscher, 2010) và Tại sao thế giới trông giống nhau trong bất kỳ ngôn ngữ nào (McWhorter, 2014) (mặc dù cả hai tác giả đều đồng ý rằng có sự khác biệt và sự khác biệt là nhỏ). Lý thuyết "lý thuyết" (Gopnik và Glymour, 2002; Tenenbaum et al., 2007) xem việc học của trẻ em về thế giới tương tự như việc xây dựng các lý thuyết khoa học. Cũng giống như những dự đoán của một ma-

Thuật toán học tiếng Trung phụ thuộc rất nhiều vào từ vựng được cung cấp cho nó, vì vậy việc xây dựng lý thuyết của trẻ sẽ phụ thuộc vào môi trường ngôn ngữ mà việc học diễn ra.

Có một số văn bản giới thiệu tốt về logic bậc nhất, bao gồm một số nhân vật hàng đầu trong lịch sử logic: Alfred Tarski (1941), Alonzo Church (1956), và W.V. Quine (1982) (là một trong những cuốn dễ đọc nhất). Enderton (1972) đưa ra một quan điểm định hướng toán học hơn. Một cách xử lý chính thức cao về logic bậc nhất, cùng với nhiều chủ đề nâng cao hơn trong logic, được cung cấp bởi Bell và Machover (1977). Manna và Waldinger (1985) đưa ra một giới thiệu dễ đọc về logic từ một chuyên gia khoa học máy tính, cũng như Huth và Ryan (2004), những người tập trung vào xác minh chương trình. Barwise và Etchemendy (2002) có một cách tiếp cận tương tự như cách tiếp cận được sử dụng ở đây. Smullyan (1995) trình bày kết quả một cách ngắn gọn, sử dụng định dạng hoạt cảnh. Gallier (1986) cung cấp một giải thích toán học cực kỳ nghiêm ngặt về logic bậc nhất, cùng với rất nhiều tài liệu về việc sử dụng nó trong lý luận không tự động. Nền tảng logic của trí tuệ nhân tạo (Genesereth và Nilsson, 1987) vừa là một phần giới thiệu vững chắc về logic vừa là cách xử lý có hệ thống đầu tiên về các tác nhân logic với nhận thức và hành động, và có hai cuốn sổ tay hay: van Bentham và ter Meulen (1997) và Robinson và Voronkov (2001). Tạp chí ghi nhận cho lĩnh vực logic toán học-cảm hóa thuần túy là Tạp chí Logic biểu tượng, trong khi Tạp chí Logic Ứng dụng giải quyết các mối quan tâm gần gũi hơn với trí tuệ nhân tạo.

CHƯƠNG 9

SUY LUẬN TRONG LOGIC BẬC NHẤT

Trong đó chúng tôi xác định các thủ tục hiệu quả để trả lời các câu hỏi được đặt ra trong logic bậc nhất.

Trong chương này, chúng tôi mô tả các thuật toán có thể trả lời bất kỳ câu hỏi logic bậc nhất nào có thể trả lời. Phần 9.1 giới thiệu các quy tắc suy luận cho các định lượng và chỉ ra cách giảm suy luận bậc nhất thành suy luận mệnh đề, mặc dù có khả năng tốn kém lớn. Phần 9.2 mô tả show thống nhất có thể được sử dụng để xây dựng các quy tắc suy luận hoạt động trực tiếp với các câu bậc nhất. Sau đó, chúng ta thảo luận về ba họ chính của các thuật toán suy luận bậc nhất: chuỗi chuyển tiếp (Phần 9.3), chuỗi ngược (Phần 9.4) và chứng minh định lý dựa trên độ phân giải (Phần 9.5).

9.1 Suy luận mệnh đề so với bậc nhất

Một cách để thực hiện suy luận bậc nhất là chuyển đổi cơ sở kiến thức bậc nhất thành logic tuyến bố và sử dụng suy luận mệnh đề, điều mà chúng ta đã biết cách thực hiện. Bước đầu tiên là loại bỏ các định lượng phổ quát. Ví dụ, giả sử cơ sở kiến thức của chúng ta chứa tiên đề dân gian tiêu chuẩn rằng tất cả các vị vua tham lam đều xấu xa:

$\forall x \text{ Vua}(x) \wedge \text{Tham lam}(x) \Rightarrow \text{Ác ma}(x)$. Từ đó

chúng ta có thể suy ra bất kỳ câu nào sau đây:

$\text{Vua}(\text{John}) \wedge \text{Tham lam}(\text{John}) \Rightarrow \text{Ác ma}(\text{John})$
 $\text{Vua}(\text{Richard}) \wedge \text{Tham lam}(\text{Richard}) \Rightarrow \text{Ác ma}(\text{Richard})$
 $\text{Vua}(\text{Cha}(\text{John})) \wedge \text{Tham lam}(\text{Cha}(\text{John})) \Rightarrow \text{Ác}(\text{Cha}(\text{John}))$.

... Nói chung, quy tắc của Universal Instantiation (viết tắt là UI) nói rằng chúng ta có thể suy ra bất kỳ

UniversalInstantiation

câu thu được bằng cách thay thế một thuật ngữ cơ bản (một thuật ngữ không có biến) cho một biến được định lượng phổ quát.¹

Để viết ra quy tắc suy luận một cách chính thức, chúng tôi sử dụng khái niệm thay thế được giới thiệu trong Phần 8.3. Giả sử $\text{SUBST}(\theta, \alpha)$ biểu thị kết quả của việc áp dụng sự thay thế θ cho câu α . Sau đó, quy tắc được viết

$\forall v \alpha \text{SUBST}(\{v/g\}, \alpha)$ cho bất kỳ biến v nào và số hạng nền g . Ví dụ, ba câu được đưa ra trước đó được thu được với các thay thế $\{x/\text{John}\}$, $\{x/\text{Richard}\}$ và $\{x/\text{Father}(\text{John})\}$.

¹ Đừng nhầm lẫn các thay thế này với các cách giải thích mở rộng được sử dụng để xác định ngữ nghĩa của định lượng trong Phần 8.2.6. Sự thay thế thay thế một biến bằng một thuật ngữ (một đoạn cú pháp) để tạo ra một câu mới, trong khi một cách giải thích ánh xạ một biến với một đối tượng trong miền.

Tương tự, quy tắc Instantiation hiện sinh thay thế một biến thể được định lượng hiện sinh- Instantiation hiện sinh có thể với một ký hiệu hằng số mới duy nhất. Tuyên bố chính thức như sau: đối với bất kỳ câu nào, biến v và ký hiệu hằng số k không xuất hiện ở nơi khác trong cơ sở tri thức,

$\exists v \alpha \text{SUBST}(\{v/k\}, \alpha)$. Ví dụ, từ câu

$\exists x \text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ chúng ta có thể suy ra câu

$\text{Crown}(C1) \wedge \text{OnHead}(C1, \text{John})$ miễn là $C1$ không xuất hiện ở nơi khác trong cơ sở tri thức. Về cơ bản, câu hiện sinh nói rằng có một đối tượng nào đó thỏa mãn một điều kiện, và áp dụng quy tắc hiện sinh chỉ đặt tên cho đối tượng đó. Tất nhiên, tên đó không phải thuộc về một đối tượng khác. Toán học cung cấp một ví dụ hay: giả sử chúng ta phát hiện ra rằng có một số lớn hơn một chút so với 2.71828 và thỏa mãn phương trình $d(xy)/dy = xy$ cho x . Chúng ta có thể đặt tên cho số này là e , nhưng sẽ là một sai lầm nếu đặt tên cho nó một đối tượng hiện có, chẳng hạn như π . Về logic, tên mới được gọi là hằng số Skolem.

Hằng số Skolem

Trong khi Universal Instantiation có thể được áp dụng nhiều lần cho cùng một tiền đề để tạo ra nhiều hậu quả khác nhau, Existential Instantiation chỉ cần được áp dụng một lần, và sau đó câu định lượng hiện sinh có thể bị loại bỏ. Ví dụ: chúng ta không còn cần $\exists x \text{Kill}(x, \text{Victim})$ sau khi chúng ta đã thêm câu $\text{Kill}(\text{Murderer}, \text{Victim})$.

9.1.1 Rút gọn thành suy luận mệnh đề

Bây giờ chúng ta chỉ ra cách chuyển đổi bất kỳ cơ sở kiến thức bậc nhất nào thành cơ sở kiến thức mệnh đề. Ý tưởng đầu tiên là, giống như một câu được định lượng về mặt hiện sinh có thể được thay thế bằng một bản mẫu, một câu được định lượng phổ quát có thể được thay thế bằng tập hợp của tất cả các lần thực hiện có thể. Ví dụ, giả sử cơ sở kiến thức của chúng ta chỉ chứa các câu

$$\begin{aligned} \forall x \text{Vua}(x) \wedge \text{Tham lam}(x) &\Rightarrow \\ \text{Ác ma}(x) \text{Vua}(\text{John}) \text{Tham lam} & \\ (\text{John}) \text{Anh trai}(\text{Richard}, \text{John}) & \end{aligned} \quad (9.1)$$

và đồ vật duy nhất là John và Richard. Chúng tôi áp dụng giao diện người dùng cho câu đầu tiên bằng cách sử dụng tất cả các thay thế có thể, $\{x/\text{John}\}$ và $\{x/\text{Richard}\}$. Chúng tôi có được

$\text{Vua}(\text{John}) \wedge \text{Tham lam}(\text{John}) \Rightarrow \text{Ác ma}(\text{John}) \text{Vua}(\text{Richard}) \wedge \text{Tham lam}(\text{Richard}) \Rightarrow \text{Ác ma}(\text{Richard})$. Tiếp theo, thay thế các câu nguyên tử cơ bản, chẳng hạn như $\text{King}(\text{John})$, bằng các ký hiệu mệnh đề, chẳng hạn như JohnIsKing . Cuối cùng, áp dụng bất kỳ thuật toán mệnh đề hoàn chỉnh nào trong Chương 7 để có được kết luận như JohnIsEvil , tương đương với $\text{Ác}(\text{John})$.

Kỹ thuật mệnh đề hóa này có thể được thực hiện hoàn toàn tổng quát, như chúng ta chỉ ra trong Mục 9.5. Tuy nhiên, có một vấn đề: khi cơ sở tri thức bao gồm một ký hiệu chức năng, tập hợp các thay thế thuật ngữ cơ sở có thể là vô hạn! Ví dụ, nếu cơ sở tri thức đề cập đến biểu tượng Cha, thì có thể xây dựng vô số thuật ngữ lồng nhau như $\text{Cha}(\text{Cha}(\text{Cha}(\text{John})))$

Đề xuất hóa

